

Teamtrack



2026 FINAL SUBMISSION

Group
Report

Prepared By :

Muhammad Hashir

Zernish Shahid Nazir

Omar Waheed Khan

Maria Waheed

Mohammed Siraj Hussain

Table of Contents

1. User Manual.....	2
2. Executive Summary	8
3. Part 2: Attribution	9
4. Part 3: Agile Development Evidence.....	18
5. Project Timeline and Dependencies	27
6. SRS Report	35
7. TIS Report	139
8. SAS Report.....	276
9. Handover Report.....	434

Note Bookmarks are present in this document and named in a different way due to collision with the Milestone 1, Milestone 2, Milestone 3 and Milestone 4 bookmarks

1. User Manual

CCP1 Final Submission: TeamTrack

This manual explains the structure of the CCP1 Final Submission document and provides quick access links to every major section, milestone document, and supporting artefact. It is intended for markers, supervisors, and team members who need to navigate the document efficiently or locate specific evidence without reading the document linearly.

1. Document Structure

The CCP1 Final Submission is organized into the following top-level parts, each accessible via the Table of Contents at the start of the document. All headings in the Table of Contents are hyperlinked directly to their respective sections.

- Part 1: Executive Summary, a concise overview of the project, the team, and the full semester of work
- Part 2: Attribution, a summary of team contributions across all four milestones, supported by detailed per-section breakdowns below each milestone document
- Part 3: Agile Development Evidence, sprint structure, meeting logs, client communication, Trello task board evidence, and Toggl time tracking
- Part 4: Project Timeline and Dependencies, phase timeline, milestone schedule, deliverable dependency map, and the Figma low-fidelity prototype
- Milestone 1: Software Requirements Specification (SRS), full resubmission of the SRS at its final version
- Milestone 2: Technology Investigation Specification (TIS), full resubmission of the TIS at its final version
- Milestone 3: Software Architecture Specification (SAS), full resubmission of the SAS including the Component Interaction Document
- Milestone 4: Handover Report, full resubmission of the Handover document
- Appendices, glossary, references, Toggl reports, Trello evidence, Figma links, Bitbucket repository, and presentation slides

2. Quick Access Navigation

Use the table below to jump directly to any major section of the document. All links in the final compiled document are clickable.

Section	Quick Link
Part 1: Executive Summary	Link
Part 2: Attribution	Link
Part 3: Agile Development Evidence	Link
Part 4: Project Timeline and Dependencies	Link
Milestone 1: Software Requirements Specification (SRS)	Link
Milestone 2: Technology Investigation Specification (TIS)	Link
Milestone 3: Software Architecture Specification (SAS)	Link
Milestone 4: Handover Report	Link

3. Attribution, How to Read It

Part 2 contains the full attribution section for the CCP1 Final Submission. It is structured in two levels.

3.1 Summary Level

The opening of Part 2 presents the overall team contribution summary table, the document version history, and the cross-milestone shared responsibilities table. These give a high-level view of how work was distributed across the full semester and provide the estimated contribution percentages derived from actual section counts across all four milestone documents.

3.2 Detailed Level

Below each milestone document in the submission, a detailed section-by-section attribution table is included. These tables show exactly which team member held primary ownership of each individual section within that milestone. The summary percentages in Part 2 are calculated from and directly traceable to these detailed tables.

Table 1: Attribution Supporting Links

Item	Description	Link
Milestone 1 Attribution Detail	Full section-by-section breakdown of SRS contributions	Link
Milestone 2 Attribution Detail	Full section-by-section breakdown of TIS contributions	Link
Milestone 3 Attribution Detail	Full section-by-section breakdown of SAS contributions	Link
Milestone 4 Attribution Detail	Full section-by-section breakdown of Handover contributions	Link
Toggl Report: Muhammad Hashir	Time tracking log across all four sprints	Link
Toggl Report: Zernish Shahid Nazir	Time tracking log across all four sprints	Link
Toggl Report: Omar Waheed Khan	Time tracking log across all four sprints	Link
Toggl Report: Maria Waheed	Time tracking log across all four sprints	Link
Toggl Report: Mohammed Siraj Hussain	Time tracking log across all four sprints	Link

4. Agile Development Evidence, Supporting Links

Part 3 covers the full Agile evidence for the semester including the sprint structure, all meeting logs, Trello task board screenshots, Toggl time tracking records, and the continuous improvement retrospective notes. The links below provide direct access to all supporting artefacts referenced within Part 3.

Table 2: Agile Evidence Supporting Links

Item	Description	Link
Trello Board	Full project task board across all four sprints	Link
Meeting Minutes: Sprint 1 (SRS)	Internal and client meeting records for Milestone 1	Link
Meeting Minutes: Sprint 2 (TIS)	Internal and client meeting records for Milestone 2	Link
Meeting Minutes: Sprint 3 (SAS)	Internal and client meeting records for Milestone 3	Link
Meeting Minutes: Sprint 4 (Handover)	Internal and client meeting records for Milestone 4	Link
Toggl Reports: All Members	Individual time tracking exports for all five team members	Link
Figma Prototype	Interactive low-fidelity prototype covering all five screens	Link
Version History	Version control done in word for each Milestone	Link

5. Milestone Documents and Supporting Artefacts

Each milestone document included in this submission has its own set of supporting artefacts, detailed attribution tables, and version history. Use the tables below to access these directly for each milestone.

5.1 Milestone 1: Software Requirements Specification (SRS)

Sprint 1: 12/01/26 to 25/01/26. The SRS was produced across four versions following iterative client feedback. The final version (v4) is included in full in the submission.

Table 3: SRS Supporting Artefacts and Links

Artefact	Description	Link
SRS Document (v1)	Initial Software Requirements Specification	Link
SRS Document (v2)	Updated following first client consultation, 20/01/26	Link
SRS Document (v3)	Quick-link navigation anchors added by Muhammad Hashir	Link
SRS Document (v4)	Further client-requested requirements changes by Zernish Shahid Nazir	Link

5.2 Milestone 2: Technology Investigation Specification (TIS)

Sprint 2: 26/01/26 to 15/02/26. The TIS was produced across two versions, with v2 adding section-level summaries following client review. The final version (v2) is included in full in the submission.

Table 4: TIS Supporting Artefacts and Links

Artefact	Description	Link
TIS Document (v1)	Initial Technology Investigation Specification	Link
TIS Document (v2)	Section-level summaries added for client clarity	Link

5.3 Milestone 3: Software Architecture Specification (SAS)

Sprint 3: 15/02/26 to 08/03/26. The SAS includes the Component Interaction Document and was accompanied by a low-fidelity Figma prototype, wireframes, user flow diagram, and ERD. All artefacts are linked below.

Table 5: SAS Supporting Artefacts and Links

Artefact	Description	Link
SAS Document	Software Architecture Specification including CID	Link

5.4 Milestone 4: Handover Report

Sprint 4: 09/03/26 to 29/03/26. The Handover Report consolidates all prior milestone artefacts and includes the full development task backlog, deployment plan, and client acceptance evidence. The milestone presentation was delivered on 24/03/26.

Table 6: Handover Supporting Artefacts and Links

Artefact	Description	Link
Handover Report	Milestone 4 Handover Document	Link

6. Document Navigation Features

The following navigation features are embedded throughout the final compiled document to support efficient reading and review.

- Table of Contents, all entries are hyperlinked directly to their sections
- List of Tables, all tables are numbered and listed for quick reference
- List of Figures, all figures and diagrams are numbered and listed for quick reference
- Back to Top links, available at the end of each major section to return to the Table of Contents instantly
- Section cross-references, where content in one section relates to another, an inline reference link is provided

7. Intended Audience

This user manual and the CCP1 Final Submission are intended for the following readers.

- Academic markers and assessors reviewing the full submission for the CCP1 unit
- The project supervisor and client reviewing the design readiness and process evidence

- Team members navigating back to specific sections or artefacts during the second project unit

2. Executive Summary

TeamTrack is an AI-supported, web-based collaborative project management application that seeks to solve the fundamental problems of academic group work, namely, disjointed tooling, uneven workforce distribution, and lack of clear contribution tracking. The platform is a unification of task management, time management, contribution analytics, team communication, and AI-assisted insights into one system and provides team members and instructors with a transparent and responsible understanding of project progress.

This CCP1 Final Submission is the compilation of the whole body of work done in the first project unit by a team of five members namely, Muhammad Hashir (Project Manager and Coordinator), Zernish Shahid Nazir (UI/UX Designer), Omar Waheed Khan (Architecture and Backend), Maria Waheed (Requirements and Testing) and Mohammed Siraj Hussain (Research and Testing). The project was provided in four successive three-week Agile sprints. Sprint 1 generated the Software Requirements Specification, which defined all system requirements, both functional and non-functional, user roles, and system scope. Sprint 2 also created the Technology Investigation Specification, which assessed and justified the chosen stack of Nuxt.js, FastAPI, PostgreSQL, Firebase authentication, and LLaMA 3.1 via Ollama to the AI module, deployed on Curtin University's physical infrastructure. Sprint 3 created the Software Architecture Specification and Component Interaction Document, as well as a five-screen low-fidelity prototype developed in Figma that included the Landing Page, Login, Sign Up, Dashboard, and AI Chatbot. The Sprint 4 generated the Handover Report, which summarizes all the preceding artefacts and provides a complete set of tasks, deployment plan, and evidence of client acceptance, and ends with a presentation by the team on 24 March 2026.

This last submittal is a culmination of all four milestone reports along with the artefacts of the UI design, the evidence of prototyping, Agile processes records, reports of individual contribution, and supplementary materials. The team had an active client contact throughout all the sprints, added feedback at all milestone gates, and has generated a complete tracing, planning, and design package that shows full preparation to be implemented in the second project unit.

3. Part 2: Attribution

This CCP1 Final Submission is a group work of a team of five individuals who are working under four milestones. The Software Requirements Specification (SRS), Technology Investigation Specification (TIS), Software Architecture Specification (SAS), and Handover Report. Under a hybrid Agile and Waterfall methodology. The division of work was accomplished through joint efforts in team meetings at the beginning of every milestone, and every member had their part that corresponded to their job and specialization. Muhammad Hashir went on to draft all four initial Tables of Contents, served as the main channel of communication between the team and the client, wrote meeting minutes at all milestones, and kept document formatting, List of Tables, and List of Figures in place. Muhammad Hashir and Omar Waheed Khan created and prepared the milestone presentation slides. Division of tasks was not enforced individually but achieved through the mutual consent of the entire team during the planning sessions.

Table 1: Overall Team Contribution Summary

Team Member	Role	Estimated Contribution (%)
Muhammad Hashir	Project Manager and Coordinator	27%
Mohammed Siraj Hussain	Research and Testing	20%
Omar Waheed Khan	Architecture and Backend	19%
Maria Waheed	Requirements and Testing	17%
Zernish Shahid Nazir	UI/UX Designer	17%

Detailed Sections for all the milestones are below the document of the Milestones Attached in Part 2. **The tables below are just a rough estimate of the contribution in all the milestones.**

Table 2: Milestone 1: Software Requirements Specification (SRS)

Section	Hashir	Zernish	Omar	Maria	Siraj
Introduction: Purpose, Report Structure, Intended Audience, Scope	●				
Introduction: Definitions, Acronyms and Abbreviations			●		
Introduction: Competitor Analysis, Proposed System Overview			●		
Overall Description: Design Constraints, Assumptions, System Workflow	●				
Overall Description: Product Perspective, Functions, User Classes, Operating Environment					●
Functional Requirements: User Registration, RBAC, Project Management, Task Management, Time Tracking				●	
Functional Requirements: Team Chat, Contribution Analytics Dashboard		●			
Functional Requirements: AI Chatbot, Notifications, Activity Log			●		
External Interface Requirements (complete)					●
Non-Functional Requirements: Security, Performance, Usability		●			
Non-Functional Requirements: Availability, Data Integrity, AI Ethics	●				
Data Requirements: Data Entities, Data Relationships		●			

Section	Hashir	Zernish	Omar	Maria	Siraj
Data Requirements: Data Storage and Retention				●	
Appendices: Use Case Diagrams, ERD, Glossary		●			
Appendices: References			●		
Appendices: Agile Development, Meeting Minutes	●				
Table of Contents / List of Tables / Formatting	●				

Table 3: Milestone 2: Technology Investigation Specification (TIS)

Section	Hashir	Zernish	Omar	Maria	Siraj
Executive Summary	●				
Introduction: Background, Problem Statement, Aim and Objectives		●		●	
Introduction: Scope, Assumptions and Constraints		●		●	
Project Overview: Stakeholders, Roles and Responsibilities, Deliverables	●				
Project Overview: Tools and Technologies			●		
Technology Stack Overview (complete)	●				
Updated Requirements: Functional and Non-Functional Requirements			●	●	●

Section	Hashir	Zernish	Omar	Maria	Siraj
Updated Requirements: User Roles and Permissions			●		
Updated Requirements: Use Cases			●	●	
System Design: System Architecture				●	
System Design: Database Design (ERD)		●			
System Design: User Flow Diagram, System Flow Diagram					●
Implementation (complete)					●
Testing and Validation: Proposed Approach, Test Scenarios	●				
Testing and Validation: Issue Resolution, UAT			●		
Project Management: Communication Plan, Agile Development, Meeting Minutes	●				
Project Management: Kanban Integration				●	
TIS: Frontend Technology Evaluation		●			
TIS: Backend Technology Evaluation			●		
TIS: Database Technology Evaluation		●			
TIS: External Services and APIs	●				
TIS: Evaluation Against Objectives				●	

Section	Hashir	Zernish	Omar	Maria	Siraj
TIS: Limitations and Future Enhancements			●		
Appendices: Glossary, UI Design Artefacts, User Manual	●				
Appendices: References, WBS			●	●	
Table of Contents / List of Tables / Formatting	●				

Table 4: Milestone 3: Software Architecture Specification (SAS)

Section	Hashir	Zernish	Omar	Maria	Siraj
Executive Summary	●				
Overview: Background, Purpose, Scope, Summary of Changes			●		
Project Description: Problem Statement, Solution Overview		●			
Project Description: Target Users and Use Cases, Assumptions and Constraints				●	
TIS Updates		●			
SAS Architectural Overview and Style					●
SAS System Context and Boundaries					●
SAS Major Components and Modules	●				
SAS Data Architecture		●			

Section	Hashir	Zernish	Omar	Maria	Siraj
SAS Integration Architecture	●				
SAS Security Architecture		●			
SAS Architecture Decisions and Alternatives		●			
System Design High-Level Architecture Diagram	●				
System Design Frontend Design (Pages, Navigation, Components)				●	
System Design Backend Design and Endpoint List	●				
System Design Database and ERD Design		●			
System Design Authentication and RBAC				●	
System Design Design Constraints and Scalability			●		
Component Interaction Diagram CID (complete)		●			
UI Design Approach and Standards			●		
UI Design Wireframes and Low-Fidelity Prototype	●				
UI Design Improvements and Conclusion			●		
Validation and Testing Plan (complete)					●
Agile Development Approach, Module Breakdown, Tools and Collaboration				●	

Section	Hashir	Zernish	Omar	Maria	Siraj
Agile Development Timeline and Meeting Minutes	●				
Risks and Mitigation (complete)				●	
Appendices Glossary, References, WBS			●		
Appendices Trello / Task Board Evidence				●	
Table of Contents / List of Tables / Formatting	●				

Table 5: Milestone 4: Handover Report

Section	Hashir	Zernish	Omar	Maria	Siraj
Executive Summary	●				
Introduction Purpose, Contents, Out-of-Scope (complete)					●
Project Overview Problem Statement				●	
Project Overview Solution Overview, Key Features, Architecture Summary, Technology Stack				●	
Project Overview Objectives and Success Criteria			●		
SRS Functional Requirements, NFR, Constraints, User Roles, Updates		●			
TIS Technology Stack Selection, Architecture Design, Security, Justification					●

Section	Hashir	Zernish	Omar	Maria	Siraj
Deployment Frontend Hosting and Deployment				●	
Deployment Backend Hosting, Database Hosting, AI Module Deployment			●		
Deployment CI/CD Workflow					●
Deployment Product Delivery Plan					●
Technical Readiness Environment Setup, Spike Testing, Codebase Skeleton		●			
Task Backlog Snapshot, Effort and Priority, Dependencies and Definition of Done	●				
Client Communication UI Feedback and Sign-Off				●	
Client Communication Technology and Development Readiness		●			
Client Communication Summary of Client Alignment		●			
Meeting Minutes	●				
Appendix Glossary		●			
Appendix Links	●				
Appendix Presentation Slides	●		●		
Appendix References			●		
Appendix Contribution Matrix			●		

Section	Hashir	Zernish	Omar	Maria	Siraj
Appendix Toggl Reports	●				
Table of Contents / List of Tables / Formatting	●				

Table 6: Document Version History

Document	Version	Change Description	Owner
SRS	v1	Initial release	Full Team
SRS	v2	First round of client-requested content updates	Omar Waheed Khan, Mohammed Siraj Hussain
SRS	v3	Addition of quick-link navigation anchors per requirement	Muhammad Hashir
SRS	v4	Further requirements changes from additional client feedback	Zernish Shahid Nazir
TIS	v1	Initial release	Full Team
TIS	v2	Section-level summaries added throughout for client clarity	Full Team (section owners)

Table 7: Cross-Milestone Shared Responsibilities

Responsibility	Owner	Scope
Initial Table of Contents drafting	Muhammad Hashir	All four milestones
Document formatting, List of Tables, List of Figures	Muhammad Hashir	All four milestones

Responsibility	Owner	Scope
Meeting minutes authoring	Muhammad Hashir	All four milestones
Primary client communication and feedback coordination	Muhammad Hashir	All four milestones
Milestone presentation design and production	Muhammad Hashir, Omar Waheed Khan	Milestone 4
Work breakdown and section allocation facilitation	Muhammad Hashir	All four milestones
Toggl time tracking	All Team Members	All four milestones
Bitbucket version control	Omar Waheed Khan	All four milestones
Trello task board maintenance	Maria Waheed	All four milestones

The contribution percentages presented above are based on assigned section counts across all four milestone documents and are intended as an honest approximation of workload distribution. They are inherently subjective and do not fully capture variation in section complexity, research depth, revision effort, or the coordination work that occurred outside of formal document sections. Every team member contributed meaningfully to the success of this project across all four sprints, and the figures above should be read as a general guide rather than a precise measurement of individual effort.

4. Part 3: Agile Development Evidence

4.1 Overview

The TeamTrack project was managed using an Agile approach across the full semester, structured into four three-week sprints, one per milestone. Each sprint followed a consistent cycle: sprint planning at the start, collaborative execution through the sprint, a client-facing review or presentation at the end, and a retrospective to inform the next sprint. This approach allowed the team to adapt requirements iteratively based on client feedback, maintain accountability through task ownership, and deliver meaningful documented progress at each milestone gate.

The team communicated through three main channels throughout the semester: Google Meet for structured sprint planning and review sessions, a WhatsApp group for fast daily coordination between meetings, and Trello for task tracking and ownership visibility across all sprints.

4.2 Sprint Structure

The project ran from 12 January 2026 to 29 March 2026 across four consecutive sprints, with the Milestone 4 presentation delivered on 24 March 2026. Each sprint produced a primary milestone deliverable and was governed by defined sprint goals, task allocation, and a Definition of Done.

Table 8: Sprint Overview Across the Full Semester

Sprint	Milestone Deliverable	Duration	Sprint Goal
Sprint 1	Software Requirements Specification (SRS)	12/01/26 to 25/01/26	Define system scope, elicit all requirements, confirm user roles and use cases with client
Sprint 2	Technology Investigation Specification (TIS)	26/01/26 to 15/02/26	Evaluate and justify the full technology stack, confirm architecture direction with client
Sprint 3	Software Architecture Specification (SAS)	15/02/26 to 08/03/26	Produce system architecture, component design, UI prototypes, and validation plan
Sprint 4	Handover Report	09/03/26 to 29/03/26	Consolidate all artefacts, confirm client acceptance, demonstrate implementation readiness. Presentation: 24/03/26

4.3 Client Meetings and Feedback

The team maintained consistent client engagement throughout all four sprints. At least one formal client consultation was held during each sprint, with feedback directly incorporated into the milestone deliverable before submission. All client meetings were documented in formal meeting minutes, referenced and linked in each milestone report.

Table 9: Client Meeting Log, Full Semester

Date	Sprint	Meeting Type	Key Focus	Outcome
13/01/26	Sprint 1	Client: Project Confirmation	Confirming capstone idea and system scope	Project direction agreed and confirmed
20/01/26	Sprint 1	Client: Requirements Review	SRS requirement updates and scope clarification	Requirements refined and updated in SRS v2
27/01/26	Sprint 2	Client: Technology Consultation	Feasibility of proposed technology stack	Initial infrastructure direction confirmed
10/02/26	Sprint 2	Client: Architecture Validation	Validation of system architecture and deployment plan	Infrastructure planning confirmed by client
12/03/26	Sprint 4	Client: Handover Consultation	SRS and TIS updates, additional handover requirements	Client feedback recorded, SRS v4 and TIS v2 changes applied

A total of five formal client meetings were conducted across the semester. These covered requirement elicitation and confirmation, technology feasibility review, architecture validation, and handover acceptance. All meetings were recorded in structured meeting minutes and linked in the relevant milestone appendices. Additionally, client feedback was communicated through formal email correspondence between meetings, providing written confirmation of agreed scope changes and deliverable expectations.

4.4 Internal Team Communication

The team operated a three-channel internal communication model across all four sprints: Google Meet for structured planning and review sessions, WhatsApp for daily quick coordination, and Trello for asynchronous task visibility and accountability.

The team operated a three-channel communication model across all four sprints: Google Meet for structured planning and review sessions, WhatsApp for daily quick coordination, and Trello for asynchronous task visibility and accountability.

4.4.1 Sprint Planning and Review Meetings

At the start of each sprint, the team held a planning session to agree on the Table of Contents, allocate sections to team members by consensus, identify cross-dependencies, and set individual completion targets. At the end of each sprint, a review meeting was held to consolidate sections, align formatting, and confirm the document was ready for submission.

Table 10: Internal Meeting Log, Full Semester

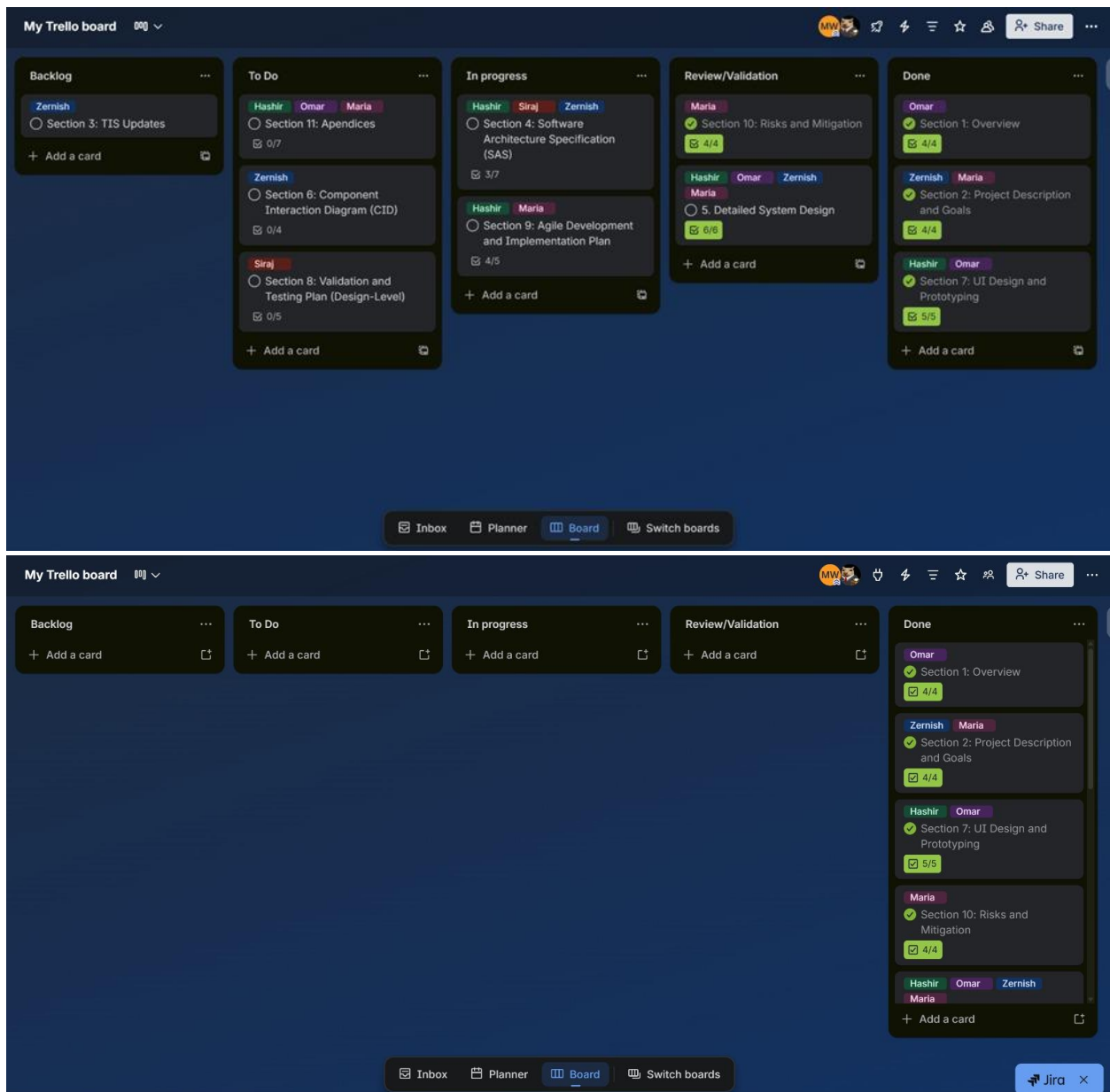
Date	Sprint	Meeting Type	Key Focus	Outcome
09/01/26	Sprint 1	Internal: Kickoff	Deciding on project idea and team roles	TeamTrack confirmed as project
14/01/26	Sprint 1	Internal: Planning	Work breakdown and Agile method agreement	Section allocation and sprint plan finalised
27/01/26	Sprint 2	Internal: Planning	Technology evaluation and section allocation	Technologies shortlisted, responsibilities assigned
29/01/26	Sprint 2	Internal: Alignment	Technology stack finalisation and documentation planning	Documentation responsibilities allocated
03/02/26	Sprint 2	Internal: Progress	Monitoring workload and resolving blockers	Task distribution adjusted
10/03/26	Sprint 4	Internal: Planning	Initial ToC, section distribution, presentation allocation	ToC and section ownership finalised
17/03/26	Sprint 4	Internal: Progress	Work progress check, presentation design review, document revisions	Team aligned on remaining changes
23/03/26	Sprint 4	Internal: Rehearsal	Milestone presentation rehearsal and script review	Presentation scripts finalised

Date	Sprint	Meeting Type	Key Focus	Outcome
27/03/26	Sprint 4	Internal: Final Review	Finalising remaining document changes	Document compilation completed

A total of nine internal team meetings were recorded across the semester, in addition to the five client meetings, bringing the total documented meeting count to fourteen across all four sprints. Stand-up style coordination was also conducted between formal meetings using WhatsApp, where blockers were flagged, quick decisions confirmed, and document editing turns coordinated to prevent formatting conflicts.

4.4.2 Trello Task Board

The team integrated Trello as the central task management tool across all four sprints. Tasks were created, assigned, and tracked through the board, giving every team member visibility over what was in progress, what was pending, and what had been completed. The board served as a shared reference point during planning meetings and WhatsApp coordination, ensuring accountability and reducing the risk of duplicated or missed work.



4.4.3 Communication Tools Summary

Table 11: Communication and Collaboration Tools

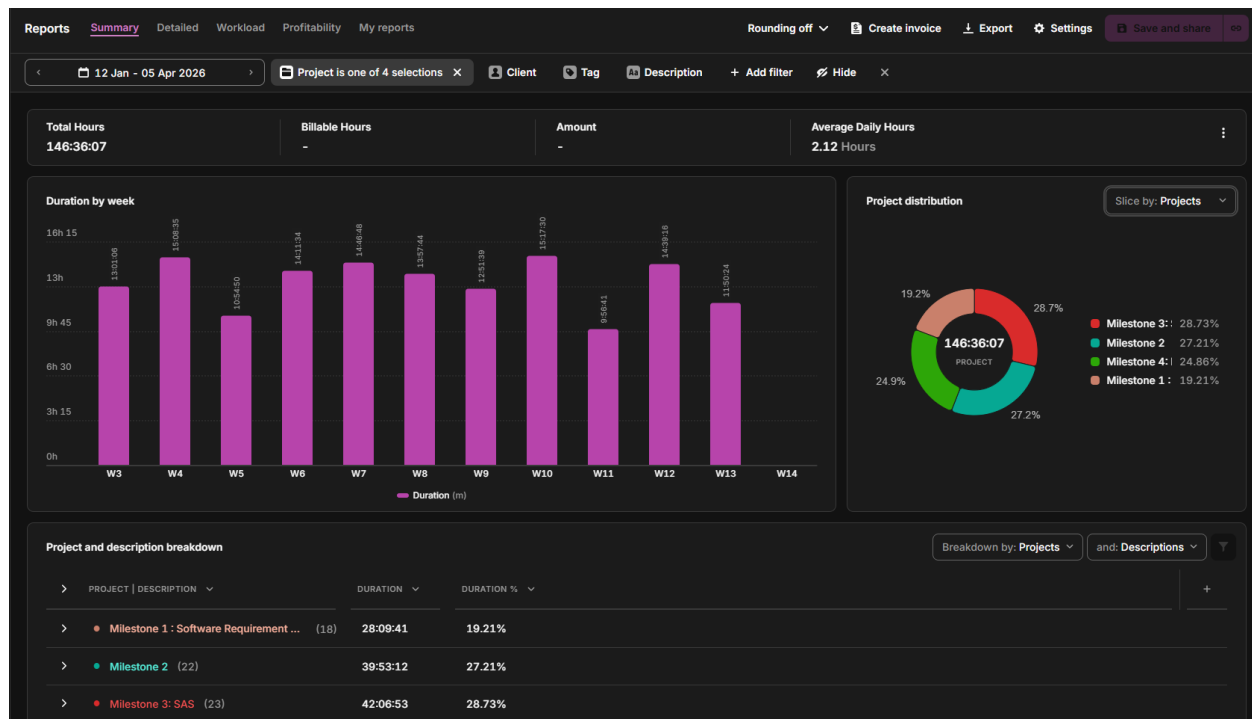
Tool	Purpose	Frequency
Google Meet	Sprint planning, progress reviews, retrospectives, presentation rehearsals	Once or twice per sprint
WhatsApp	Daily stand-up updates, blocker flagging, quick decisions, editing coordination	Daily during active sprint phases
Trello	Task ownership, sprint backlog, progress tracking, Definition of Done	Continuous throughout all sprints
Toggl	Individual time tracking per task and session	Continuous throughout all sprints
Bitbucket	Version control and code repository management	Continuous from Sprint 3 onward

4.5 Progress Tracking and Metrics

The team used Toggl Track as the primary time-logging tool, with each member recording time against specific tasks throughout every sprint. Toggl reports were exported at the end of each milestone and are linked in each milestone appendix. The full Toggl report links for all members are included in the appendices of this document.

4.5.1 Time Tracking: Toggl

All five team members maintained individual Toggl logs across all four sprints. The charts below show total hours logged per team member and the distribution of effort across sprints.



4.5.2 Task Completion: Trello

Task counts were tracked per sprint using the Trello board. The table below summarises task volume and completion rates across all four sprints based on Trello card records. Populate the figures below from your Trello export before final submission.

4.6 Sprint Retrospectives and Continuous Improvement

At the conclusion of each sprint the team conducted informal retrospectives to assess what worked, what caused delays, and what should be adjusted in the following sprint. Key improvements applied across the semester included the following.

After Sprint 1, the team identified that formatting inconsistencies arose from multiple members editing the same document simultaneously. From Sprint 2 onward, the team adopted a coordinated editing approach using Trello ownership tags and WhatsApp messaging to signal who held the document at any given time, preventing conflicts.

After Sprint 2, the team recognised that some sections were dependent on others being completed first, causing short delays mid-sprint. From Sprint 3 onward, section dependencies were explicitly identified during the sprint planning meeting and high-dependency sections were prioritised at the start of the sprint.

After Sprint 3, the team increased the frequency of internal check-ins during Sprint 4 to accommodate the complexity of consolidating all prior milestone content into a single handover document, adding a mid-sprint progress review that had not been formally scheduled in earlier sprints.

These adjustments demonstrate a genuine iterative improvement process across the semester, consistent with the continuous improvement principles of an Agile approach.

4.7 Summary of Agile Evidence

Table 12: Agile Evidence Summary

Category	Evidence	Reference
Client Meetings	5 formal client consultations across all 4 sprints	Meeting minutes in M1, M2, M4 appendices
Internal Meetings	9 formal internal team meetings across all 4 sprints	Meeting minutes in M1, M2, M3, M4 appendices
Stand-Up Coordination	Daily WhatsApp updates throughout all sprints	Team communication records
Task Management	Trello Kanban board with full task lifecycle tracking	Trello board link in appendices
Time Tracking	Individual Toggl logs for all 5 team members across all 4 sprints	Toggl reports in appendices
Version Control	Bitbucket repository with commit history	Bitbucket link in appendices
Sprint Planning	Formal planning session at the start of each sprint	Meeting minutes

Category	Evidence	Reference
Sprint Retrospectives	Informal retrospectives at end of each sprint with documented process improvements	Reflected in process changes from Sprint 2 onward
Definition of Done	Formal DoD applied from Sprint 4 onward	Section 8.3 of Handover Report

5. Project Timeline and Dependencies

5.1 Overview

This section provides the timeline of development, major milestones, dependencies between tasks, and deliverables within the TeamTrack project. The project was designed in four successive three-week sprints, each of which was aligned to a particular milestone stage. Each stage was constructed on the deliverables of the last stage in a logical sequence of dependencies between requirements and final handover.

The project was designed in a sequential way so that the design and technical decisions were always made on the basis of proven, documented foundations. Prerequisites were made prior to technology choice, the technology stack was mandated prior to architecture design, and then the architecture was finalised prior to prototyping and planning to implement. This dependence chain controlled the time of working as well as the sequence in which the client consultations would take place.

5.2 Phase Timeline

The project progressed through four defined phases across the semester, each corresponding to a three-week sprint and a primary milestone deliverable. The table below outlines each phase, its duration, and its key output.

Table 13: Project Phase Timeline

Phase	Description	Sprint	Dates	Key Deliverable
Phase 1: Requirements	Problem definition, stakeholder analysis, scope clarification, and functional requirements gathering	Sprint 1	12/01/26 to 25/01/26	Software Requirements Specification (SRS)
Phase 2: Planning and Investigation	Technology evaluation, feasibility review, stack selection, and architecture direction	Sprint 2	26/01/26 to 15/02/26	Technology Investigation Specification (TIS)
Phase 3: Architecture and Prototyping	System architecture, database design, UI/UX design, low-fidelity prototype, component interaction diagrams	Sprint 3	15/02/26 to 08/03/26	Software Architecture Specification (SAS)
Phase 4: Handover and Readiness	Consolidation of all artefacts, task backlog, deployment readiness, client acceptance, and final presentation	Sprint 4	09/03/26 to 29/03/26	Handover Report and Presentation (24/03/26)

5.3 Milestone Schedule

The following table records every significant milestone achieved across the full semester, including submission dates, client review sessions, and prototype completion.

Table 14: Milestone Schedule

Milestone	Date	Description
Project Idea Confirmed	09/01/26	TeamTrack selected as the capstone project concept after team discussion and client confirmation

Milestone	Date	Description
SRS Submitted	25/01/26	Full Software Requirements Specification delivered covering functional requirements, non-functional requirements, user roles, use cases, and system scope
Client Requirements Review	20/01/26	Client consultation session resulting in SRS v2 updates to requirements content
TIS Submitted	15/02/26	Full Technology Investigation Specification delivered covering technology stack evaluation and justification
Client Architecture Validation	10/02/26	Client confirmed infrastructure and architecture direction for the system
SAS Submitted	08/03/26	Full Software Architecture Specification delivered including system architecture, database design, component interaction diagrams, and low-fidelity Figma prototype
Low-Fidelity Prototype Completed	08/03/26	Five-screen Figma prototype delivered covering Landing Page, Login, Sign Up, Dashboard, and AI Chatbot
Handover Report Submitted	29/03/26	Final handover document consolidating all prior milestone artefacts with deployment readiness and client sign-off
Final Presentation	24/03/26	Team presentation delivered to client and assessors covering the full project scope, prototype walkthrough, and technical readiness

5.4 Task and Deliverable Dependencies

A key characteristic of the TeamTrack project was the tight dependency chain between milestone deliverables. No document or artefact was produced in isolation; each relied on confirmed outputs

from previous phases. The table below maps the most significant dependencies across the project lifecycle.

Table 15: Deliverable Dependency Map

Successor Task	Depends On	Reason
TIS (Technology Stack)	SRS (Requirements)	Technology choices must be grounded in the confirmed functional and non-functional requirements from the SRS
SAS (Architecture)	TIS (Technology Stack)	System architecture design is built on the confirmed technology stack and infrastructure decisions from the TIS
Low-Fidelity Prototype	SRS (User Roles and Use Cases)	Prototype screens must reflect the confirmed user journeys and functional requirements established in the SRS
Database Design (ERD)	SRS (Functional Requirements) and SAS (Architecture)	Entity relationships and schema are derived from system requirements and the confirmed architecture style
Component Interaction Diagram (CID)	SAS (Architecture)	Component interactions can only be mapped once the major modules and their boundaries are defined in the SAS
Task Backlog and Sprint Estimates	SAS and Prototype	Development tasks are scoped from the confirmed architecture and validated interface designs
Deployment and Hosting Plan	TIS (Technology Stack)	Deployment decisions follow directly from the confirmed stack selections made in the TIS
Client Acceptance and Handover	All Prior Milestones	Handover can only proceed once all prior artefacts are complete, reviewed, and client-confirmed

These dependencies were identified explicitly at the start of each sprint during planning sessions, and cross-dependent tasks were prioritised to avoid blocking other team members mid-sprint. Where a dependency could not be resolved immediately, the dependent task remained in the Trello backlog until the predecessor was confirmed complete.

5.5 Low-Fidelity Prototype

A central output of Sprint 3 was the low-fidelity prototype of the TeamTrack web application, developed using Figma. The prototype was not intended to be pixel-perfect or visually polished. Its purpose was to validate the information hierarchy, navigation flow, and interaction logic of the platform before any high-fidelity or development work began.

The prototype covered five core screens, each mapped directly to the functional requirements and user roles established in the SRS. The screens were fully interactively linked within Figma, allowing the team and client to walk through the complete user journey from landing on the platform through to using the AI Chatbot.

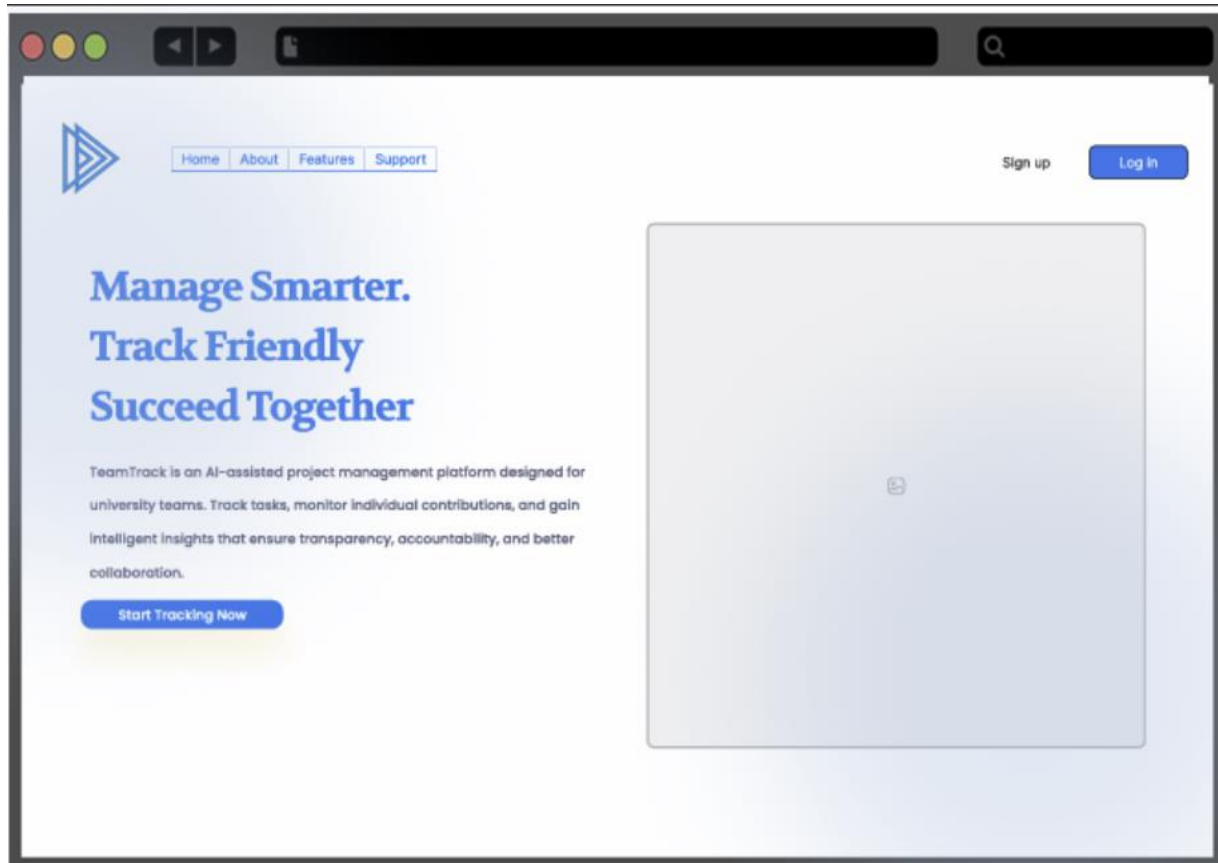
3.5.1 Prototype Screens

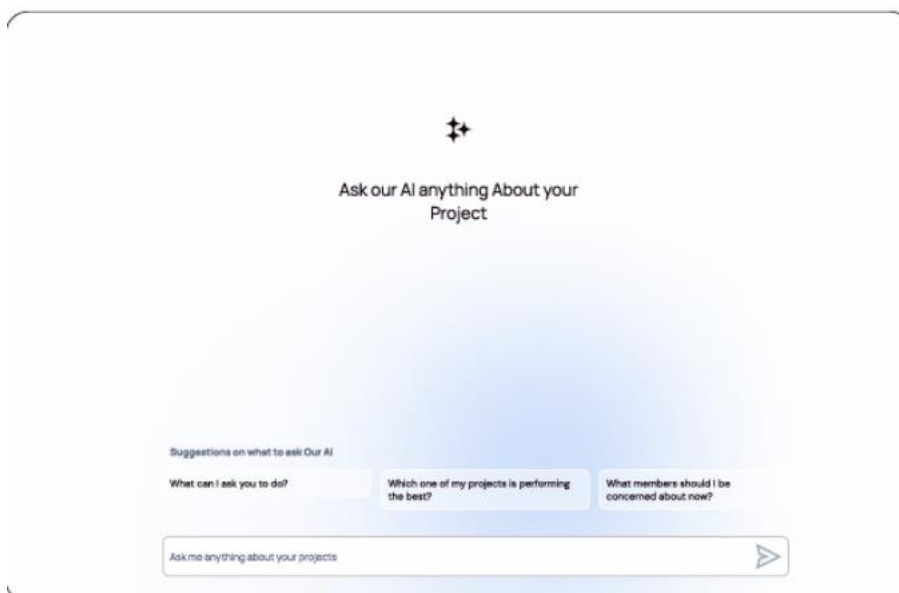
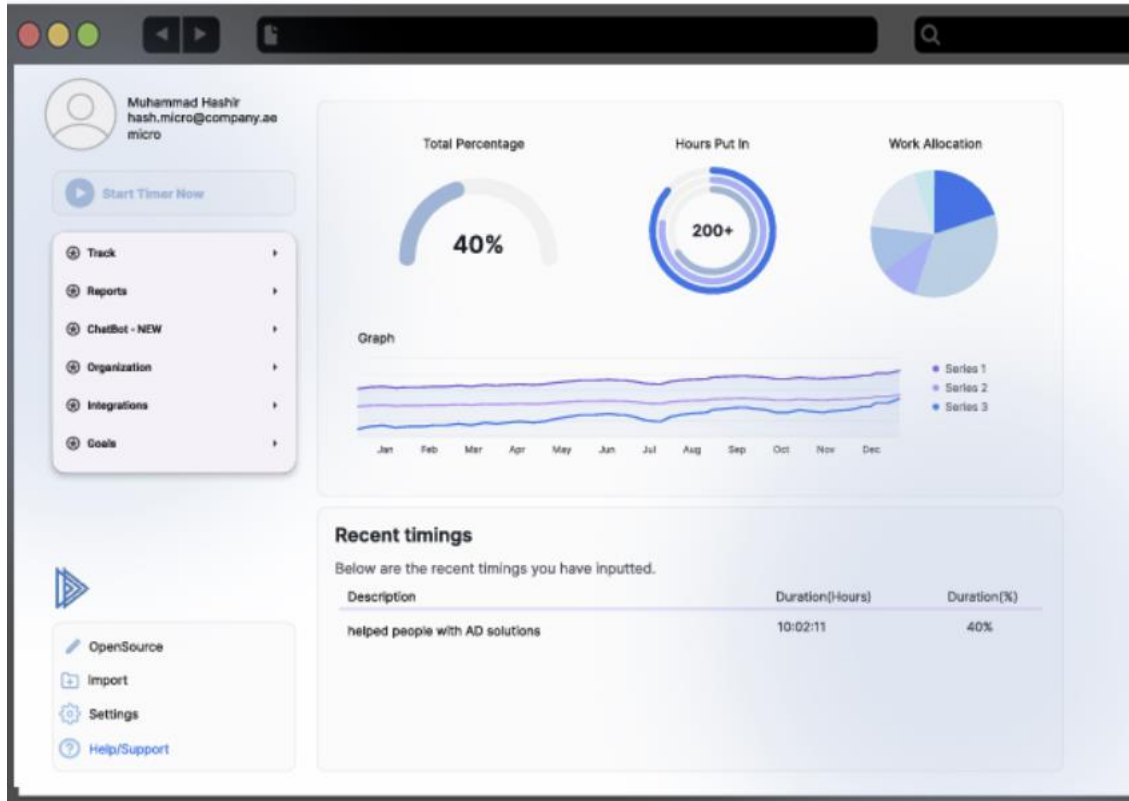
Table 16: Low-Fidelity Prototype Screen Summary

Screen	Navigation Path	Key UI Elements
Landing Page	Start of application	Hero headline, platform tagline, call-to-action button, navigation bar, login shortcut
Login Page	Landing to Login	Email field, password field, Remember Me toggle, login button, link to Sign Up
Sign Up Page	Landing to Sign Up	Email, password, retype-password fields, terms and conditions checkbox, Sign Up button, link back to Login
Student Dashboard	Login to Dashboard	User profile panel, sidebar navigation, KPI widgets (percentage, hours, work allocation), time-series graph, recent timings table, start timer button
AI Chatbot	Dashboard to Chatbot	Prompt input field, AI response area, suggested prompts panel, project-aware question examples

3.5.2 Prototype Screenshots

The following screenshots show the five screens produced in Figma as part of the low-fidelity prototype delivered in Sprint 3.

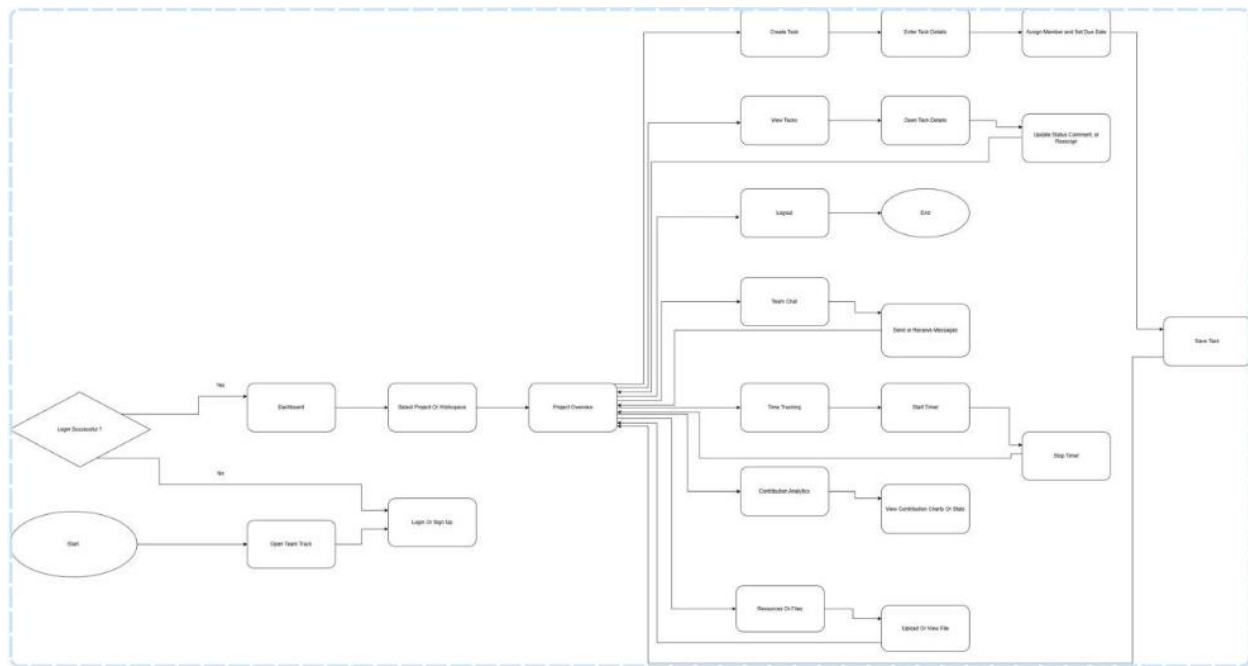




3.5.3 User Flow

The prototype followed a linear, authentication-gated user flow. A user opens TeamTrack and arrives at the Landing Page. From there they proceed to Login or Sign Up. On successful authentication, they are directed to the Student Dashboard, from which they can access the AI

Chatbot via the sidebar navigation. No authenticated screens are reachable without passing through the Login or Sign Up gate, consistent with the role-based access control requirements defined in the SRS.



3.5.4 Figma Prototype Link

The full interactive Figma prototype, including all five linked screens and annotated components, is accessible via the link in the appendices of this document.

[Figma Link to Access Prototype](#)

5.6 Summary

The project schedule was designed in a way that all of the significant deliverables would be created in the right logical order, and dependencies would be clearly followed and managed throughout all four sprints. The major result of such an organized process was the low-fidelity Figma prototype, which transformed the requirements and architecture that were confirmed into the real, navigable interface that could be reviewed by the client prior to the implementation work being undertaken. The above complete milestone schedule, dependency map, and prototype evidence indicate that the team was well organized and on schedule with the delivery during the semester.

6. SRS Report



List of Tables

Table 1: Document Formatting Standards	39
Table 2: Requirement Identification Format	41
Table 3: Definitions of Terms	44
Table 4: Acronyms and Abbreviations	46
Table 5: Comparative Analysis of Existing Tools vs. TeamTrack	48
Table 6: System Context and Scope (Section 2.1 – Product Perspective)	52
Table 7: Core Product Functions	55
Table 8: User Classes and Roles (Section 2.3 – User Classes & Characteristics)	56
Table 9: Work Flow Table	63
Table 10: Primary Navigation Structure	87
Table 11: Core Screens and Primary User Actions	89
Table 12: UI Feedback and Error Message Examples	90
Table 13: Example user navigation flow	90
Table 14: Key Software Interfaces	92
Table 15: Software Interface Summary table	93
Table 16: Key Attributes User Table	104
Table 17: Key Attributes Project Table	105
Table 18: Key Attributes Task Table	105
Table 19: Key Attributes Time Log Table	106
Table 20 : Key Attributes Activity Log Table	106
Table 21: Key Attributes Message Table	106
Table 22: Key Attributes Notification table	107
Table 23: Key Attributes Project Member table	107
Table 24: Proposed Technology Stack Summary	110
Table 25: Software Quality Attributes Table	114
Table 26: WBS/Attribution Table	120
Table 27: Meeting Minutes Summary	125
Table 28: Toggl Report Table	125

List of Figures

Figure 1: Entity Relationship Diagram for the TeamTrack system	112
Figure 2: Use Case Diagram for the TeamTrack System	113

Version	Date	Description of Changes	Link
---------	------	------------------------	------

1.0	25/01/26	Initial SRS created with core system requirements and basic structure of TeamTrack	Link
2.0	15/02/26	Updated SRS aligned with Software Requirements (UPDATED) section, refining functional areas and implementation direction based on TIS	Link
3.0	08/03/26	Further updates for Milestone 2 and 3, improved requirement organization and clarity, added structured navigation for requirements	Link
4.0	29/03/26	Added new functional requirements to improve accountability, workflow logic, usability, reliability, and AI transparency	This is version 4.

Quick Link to Requirements

Function Requirements:

[User Registration and Authentication](#)

[Role based Access Control \(RBAC\)](#)

[Project and Team Management](#)

[Task Management System](#)

[Time Tracking and Effort Logging](#)

[Team Chat and Communication Module](#)

[Contribution Analytics Dashboard](#)

Non-Functional Requirements:

[Security Requirements](#)

[Performance Requirements](#)

[Usability Requirements](#)

[Availability and Reliability Requirements](#)

[Data Integrity and Privacy](#)

Ethical and Responsible Use Requirements

External Interface Requirements

User Interface Requirements

Software Interface Requirements

Communication Interface Requirements

1. Introduction

1.1 Purpose

The initial objective of this Software Requirements Specification (SRS) document is to be in a position to postulate and model the requirements of the proposed system in a clear, structured and unambiguous way. It outlines what the system is going to be attempting to achieve and the scope and expected functioning of the system and provides out the shared vision of all stakeholders to include the system developers, the academic supervisors and the end users.

This SRS is referred to as the best source in the entire software development life cycle that assists in system design, software development, testing, validating, and evaluating the end system. The document will assist in making sure that the system is designed as per the needs of the user, the constraints of the technical elements, and the demands and orientations of the academic project.

Additionally, the document will not take risks that can be associated with the misconception of the requirements, uncontrolled scope alterations, and uneven implementation. It compares the measuring pole with which the system's progress and completion can be seen to ensure that the stakeholders determine that the completed system is playing the role wanted.

The SRS also aids in maintenance of the future system and potential improvements by way of a well-documented and detailed description of system requirements. Therefore, the paper will assist in providing a long-term usability, scalability, and sustainability of the system even after the initial development period.

In order to facilitate such aims, the general framework of the SRS and the emphasis of every subsection in the Introduction are also outlined in this section. In particular, this section contains:

- The nature and purpose of the SRS document in the definition of system requirements (**Section 1.1**).
- The guidelines of documentation, notation, and formatting to be employed during the report (**Section 1.2**)
- The SRS target audience: academic supervisors, developers and other stakeholders (**Section 1.3**)
- The limits of the proposed system, the definition of system boundaries and constraints (**Section 1.4**)
- They include definitions, acronyms, and abbreviations to ensure clarity and consistency (**Section 1.5**)
- Existing or competitor system analysis that are pertinent to the proposed solution (**Section 1.6**)

Together, these subsections provide the necessary background and context required to understand the system requirements detailed in the remainder of this document.

1.2 Report Structure and Formatting

This section will determine the structure, formatting rules, and documentation conventions to be used in this Software Requirements Specification (SRS). Standardized requirement notation and consistent formatting are required in order to make the system requirements clear, readable, and traceable. Those guidelines that are stated in this section are applied throughout the document in order to ensure effective communication between stakeholders and comply with the requirements of academic and professional documentation.

The report is organized in some order with distinct headings, even typography, and standard requirement representation. This methodology ensures that the system requirements become easily located, read, and verified throughout the software development life cycle.

1.2.1 Document Formatting Standards

The formatting rules applied in this document are summarized in Table 1 to ensure consistency across all sections.

Table 17: Document Formatting Standards

Element Type	Font Family	Font Size	Color (hex)	Style	Usage Description
Normal Text	Aptos	11 pt	#000000	Regular	Paragraphs and bullet points
Heading 1	Aptos Display	20 pt	#2c3f66	Bold & Italic	Main sections and major headings
Heading 2	Aptos Display	16 pt	#4b6285	Bold	Subsections
Heading 3	Aptos Display	14 pt	#4b6285	Bold	Sub-subsections
Heading 4	Aptos Display	12 pt	#4b6285	Bold	Minor subsections
Table Heading	Aptos	11 pt	#2c3f66 (Fill)	Bold	Heading of tables
Table Text	Aptos	11 pt	#000000	Regular	Content of the tables
Emphasis	Aptos	11 pt	#000000	<i>Italic</i>	Examples or explanatory notes
Critical Terms	Aptos	11 pt	#000000	Bold	Important rules or constraints

1.2.2 Section Numbering and Organization

The document is organized using a hierarchical numbering system to improve readability and navigation. Each major section and subsection is uniquely numbered to support cross-referencing and traceability.

- **Level 1 headings** represent major document sections
- **Level 2 and Level 3 headings** represent subsections and detailed topics
- **Bulleted lists** are used where appropriate to improve clarity without replacing explanatory text

This structured approach ensures logical progression from high-level concepts to detailed system requirements.

1.2.3 Requirement Representation and Identification

System requirements in this document are defined using a consistent and structured format to ensure clarity, traceability, and testability.

Each requirement is written as a **clear, atomic statement** and is uniquely identified using a standardized identifier format, as shown below:

Example:

FR 1.1.1: The system shall allow users to create and manage group tasks.

Table 18: Requirement Identification Format

Requirement Type	Prefix	Description
Functional Requirement	FR	Defines system behaviors and functions
Non-Functional Requirement	NFR	Defines performance, security, and quality attributes
System Constraint	SC	Defines technical or operational limitations

1.2.4 Requirement Writing Guidelines

To maintain quality and consistency, all requirements in this SRS adhere to the following rules:

- **Each requirement shall be uniquely identifiable**
- **Each requirement shall be testable and measurable**
- **Requirements shall be stated using “shall” to indicate mandatory behavior**
- **Compound or ambiguous statements are avoided**
- **Requirements are written from a system perspective, not an implementation perspective**

Italicized text is used only for examples or explanatory notes and does not represent enforceable requirements.

1.2.5 Use of Emphasis and Visual Aids

To improve readability and comprehension:

- **Bold text** is used for critical rules, key terms, and constraints
- *Italic text* is used exclusively for examples or clarifications

- Tables are used to summarize structured information such as formatting rules and requirement identifiers

This consistent use of emphasis ensures that important information is easily distinguishable without reducing academic formality.

1.3 Intended Audience

The proposed Software Requirements Specification (SRRS) document targeted to be distributed amongst a wide variety of stakeholders who have an interest or who are affected by the proposed system production, assessment, and sustainability of that system. The document is structured in such a way that it can be understood by the whole targeted audience of the document, which includes the requirements of the system, the objectives, and the constraints of the system, irrespective of their position they have in the project.

The usage of detailed information and terminology in this document will be targeted towards being detailed enough in order to be down-to-earth, whilst still providing technical and non-technical stakeholders with an opportunity to use the SRS document as a reference point of the software development life cycle.

The intended audience for this document includes, but is not limited to, the following groups:

- **Academic Supervisors and Assessors:**
Responsible for reviewing the SRS in order to assess the feasibility of the project, scope definition, and analysis of compliance with Capstone assessment criteria.
- **Development Team Members:**
Use the SRS as one main means of understanding the functionality, constraints, and requirements of the system during design, implementation, and testing phases.
- **Project Stakeholders:**
People or groups who have an interest in the outcomes of the system, who need a good high-level understanding of the capabilities and limitations of the system.
- **Maintainers and Developers of the Future:**
Refer to the SRS to know existing system requirements when undertaking maintenance, updates, or enhancements after initial deployment.
- **Quality Assurance and Testers:**

Use the documented requirements as a foundation for verifying, validating, and testing system behavior against requirements.

This SRS is a way to ensure that all the intended readers are on the same page with respect to the system requirements, and it also serves as a constant point of reference for communication, decision-making, and evaluation throughout the project life cycle.

1.4 Scope of the System

This section defines the boundaries of the proposed system by describing how it is supposed to work, its boundaries, and constraints. The scope also describes what the system will incorporate and what it will not incorporate to reassure the stakeholders that the coverage of the system is fully comprehended and helps in preventing further coverage and controlled scope creep in the system development.

The proposed system will be a group project planning and management system that is university-level oriented. It provides a centralized platform, on which the users may arrange their tasks, track progress, and manage deadlines in a focused, transparent space. The system also focuses on the enhancement of organization, responsibility of contribution, and transparency in team-based studies.

The core capabilities within the scope of the system are:

- User account administration, authentication, and role access.
- Group and project management- This enables the user to make and administer project groups in academics.
- Creation and assigning of tasks, which allows giving out duties to members of the group.
- Progress reports, such as status reports on tasks and completion.
- Management of deadlines and timelines, which helps with scheduling and monitoring of milestones.
- Raw reporting and summaries, which give a macro-level understanding of the project progress.

The system will be operating within the following established limits:

- The system should be used in academic contexts and is not aimed at serving commercial project management processes.
- Connecting with other external enterprise solutions (e.g., commercial ERP or advanced analytics applications) is out of scope.
- The system lacks institutional assessment and automated grading.

- Considerations: Advanced artificial intelligence or predictive analytics capabilities are not a part of the first scope.

By clearly defining these boundaries, the scope ensures that development remains focused, achievable, and aligned with the project timeline and academic objectives. Any future extensions beyond this scope would be considered enhancements and are not part of the current system requirements.

1.5 Definitions, Acronyms, and Abbreviations

This section gives detailed definitions of the terms, acronym, and abbreviations that are used in this Software Requirements Specification (SRS). The main point is to avoid ambiguity and promote the similarity of understanding the technical and operational concepts of the system among all the stakeholders such as developers, academic supervisors and end users.

1.5.1 Definitions

Table 3 defines the specific technical terms and project-specific concepts used to describe the **TeamTrack** system.

Table 19: Definitions of Terms

Term	Definition
Activity Logging	The generation of audit trails and contribution metrics through automated documentation of user activities in the system like task updates, file uploads and changing of status.
Contribution Analytics	One of the fundamental characteristics of TeamTrack which consolidates data about user activity (e.g., time logs, task completions) to measure, graphically display and give a score to each individual member contribution with respect to the group.
Contextual Chat	Real-time communication functionality in which discussion threads are attached directly to project tasks or milestones so that conversation history is always maintained with the work item.

Dashboard	The main graphical user interface (GUI) that gives a high-level summary of a project status, the next deadlines, and lists of tasks of each individual user at the moment of their login.
Inactivity Detection	An algorithmic function that determines long durations of zero activity of a group member according to the log information and alerts or suggestions to the team leader.
Instructor View	An academic supervisor Read-Only permission level, which is specifically created to help academic supervisors audit project progress and see summary contributions, but cannot change project data.
Machine Learning-Ready	A system architecture design that organizes the data (logs, timestamps, user inputs) in standard format that can be used in the future to train predictive AI models.
Rule-Based Logic	A pre-programmed process of data handling that is involved in the first part of AI wherein set criteria (e.g., "task overdue exceeds 3 days) stimulate certain insight in the system.
Stakeholder	Any party or person who has a vested interest in the system such as students who are part of the team, academic assessors and people who will maintain the system in the future.
Time Tracking	A working module that enables the user to start/stop timers by hand or time duration against individual tasks to measure the effort expended by the task correctly.
Uneven Workload	An identified state of the system in which the allocation of the tasks or the number of the hours recorded by a group member differ considerably compared to the average of the group.

1.5.2 Acronyms and Abbreviations

Table 4 lists the abbreviations and acronyms used in this document to maintain brevity and consistency.

Table 20: Acronyms and Abbreviations

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
DBMS	Database Management System
FR	Functional Requirement
HTTP/S	Hypertext Transfer Protocol / Secure
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
SC	System Constraint
SRS	Software Requirements Specification
SQL	Structured Query Language
UI/UX	User Interface / User Experience
URL	Uniform Resource Locator

1.6 Competitor / Existing Tools Analysis

The existing project management software market is already controlled by complex commercial products aimed at enterprise flexibility and teamwork within the corporation. The particular demands of academic group projects however, including the evaluation of individual contribution, instructor supervision, and free-of-charge availability, form a distinct set of requirements that these Commercial Off-The-Shelf (COTS) tools do not fulfill [1].

The section contains a critical examination of the most popular solutions in the market (Trello, Jira, and Asana) and popular communication platforms (Slack/Discord). These tools are analyzed using three major criteria:

- Accessibility: Ease of use and cost barriers for students [4].
- Accountability: Mechanisms for tracking individual performance versus group velocity [2].
- Academic Suitability: Support for the specific workflows of university capstone and group assessments [1].

1.6.1 Analysis of Commercial Project Management Solutions

1. Trello (Kanban-Based Management) The Trello application is a common tool among student groups because of its graphical simplicity and free version. It employs Kanban board (Boards, Lists, and Cards) technique to picture workflow.

- Operational Limitation: Although useful in high-level visualization of tasks, Trello does not have built-in support of time tracking or contribution analytics. It works based on a team-first framework in which tasks are transferred between columns, but does not necessarily track the amount of time a task took and who put effort in it.
- Academic Gap: Trello is not able to provide historical data reports that can be evaluated by peers. The instructors are not able to retrospective to check whether a student is not active in three weeks; the instructors can observe the present situation of the board only [2].

2. Jira Software (Agile/Scrum Management) Jira is the standard software development industry application and it has a powerful scrum, Sprint planning as well as backlog support.

- Operational Limitation: Jira is specifically created to support large-scale engineering teams that have full-time project managers. The complexity of the setting up (configurations of workflows, issue types, and permissions) is usually inappropriate to the project size in the case of a standard university group. This causes the so-called tool fatigue, when students use more time to operate Jira than to create their project [1].
- Academic Gap: Jira emphasizes Product Velocity (how fast a feature is shipped), rather than Student Contribution. It does not have an inbuilt position of an Academic Supervisor that can see several projects at once without using complicated administrative setups.

3. Asana / Monday.com (Task and Timeline Management) These applications offer a compromise, including list view, Gantt framework timelines and workload administration capabilities.

- Operational Limitation: The most applicable features to achieve equity in group work, including the following features, are nearly exclusive to Premium or Enterprise subscriptions, and have no lock: Workload Reporting, and Timeline Views. This renders them unaffordable to student teams that have zero budget [4].
- Academic Gap: These tools, similar to Trello, presuppose a professional situation, in which employees are economically driven. They do not have the accountability triggers (like the alert of inactivity) to present the free-rider problem that is inherent in the academic group work [2].

1.6.2 The Problem of Fragmented Workflows (Communication Tools)

One of the major problems of the modern student processes is the lack of integration of Task Management (e.g., Trello) and Communication (e.g., WhatsApp, Discord, Slack) [3].

- **Context Switching:** The students talk about the project requirements via WhatsApp, yet work on Trello. This causes the history of making critical decisions to be lost. As a task is executed or postponed, the reasons and the manner of doing this are lost in a chat log that is not relevant to the project board.
- **Lack of Traceability:** A teacher who visits a project board cannot identify who really worked on a project because they do not know the discussion that identifies an author of the problem solution.

1.6.3 Comparative Feature Matrix

Table 5 summarizes the functional disparity between market leaders and the proposed TeamTrack system, specifically regarding academic requirements.

Table 21: Comparative Analysis of Existing Tools vs. TeamTrack

Feature Category	Feature	Trello	Jira	Asana	TeamTrack (Proposed)
Core Management	Task Creation & Assignment	✓	✓	✓	✓
	Real-Time Contextual Chat	✗	✗	✗	✓ (Task-Linked)
Accountability	Individual Contribution Metrics	✗	✗	✗	✓ (Grading-Focus)
	Native Time Tracking	✗	✗	✗	✓
	Inactivity / Dropout Detection	✗	✗	✗	✓ (AI-Driven)
Accessibility	Learning Curve	Low	High	Medium	Low
	Instructor / Supervisor View	✗	✗	✗	✓
	Cost for Advanced Features	Free	Free*	Paid	Free (Open Source)

**Jira is free for small teams but retains high configuration complexity.*

1.6.4 Conclusion and Gap Analysis

The analysis establishes the fact that there are indeed powerful tools in the management of work, but there exists a dire dearth of tools that can be applied to manage student groups. The TeamTrack system is designed to fill three gaps that are present in the existing environment:

1. The Black Box of Participation: Current technologies hide individual effort behind organizational development. The Contribution Analytics in TeamTrack bring the individual effort into the limelight [2].
2. Disconnected Chat Problem: TeamTrack will incorporate communication into the project board so that the discussions can be archived into the project documentation [3].
3. The Lack of Pedagogical Insight: In contrast to Jira or Asana, TeamTrack applies AI-based reasoning specifically to detect the academic risks, including the uneven workload distribution or student disengagement, and offers the actionable insights to both the students and supervisors [1].

1.7 Proposed System Overview

TeamTrack is the proposed solution that is an academic-specific, AI-assisted, web-based collaborative project management platform. In contrast to the general-purpose commercial solutions, TeamTrack is designed to serve the fundamental pedagogical needs of group work in universities, specifically, fairness, transparency, and accountability.

The section is a high-level description of the system architecture, the main functional modules of the system, and the mechanisms in particular that are employed to improve the coordination of students and instructor control.

1.7.1 System Concept and Objectives

TeamTrack is a focus group management system that integrates traditional project planning systems with high-performance contribution tracking and live communication. By making individual participation visible and measurable, the main aim of the system is to alleviate typical areas of friction in the group work, particularly unequal workload distribution, absence of communication context, and the free-rider phenomenon.

The system will be based on Role-Based Access Control (RBAC) architecture, which will offer different interfaces to:

- **Student Team Members:** Are task oriented, collaborative and time conscious.
- **Student Team Leaders:** Authorized to organize project environments, delegate responsibilities and monitor group development.
- **Academic Supervisors (Instructors):** Given a read-only Audit View to see over-level progress reports and individual contribution summaries but not to intervene in daily operations.

1.7.2 Core Functional Modules

TeamTrack system consists of four interconnected functional modules that cooperate to aid the project lifecycle:

Collaborative Task Management Module In essence, TeamTrack enables users to establish projects, milestones, and subdivide the requirements into small tasks.

- **Feature:** Each task has its timeframes and faces its owner.
- **Differentiation:** In contrast to generic to-do lists, in TeamTrack artifact check-in or status updates ensure the completion of the tasks is recorded and not merely checked.

2. Contextual Communication Module To clarify the problem of disintegrated processes (the change between project boards and external chat applications), TeamTrack integrates a live Contextual Chat System.

- **Feature:** Discussion threads will be integrated directly into individual tasks or project milestones.
- **Benefit:** This will make sure that all decisions, clarifications, and updates are linked to the work item in question historically so that the decision trail can be referenced in future.

3. Contribution Analytics and Time Tracking This module is the foundation of data to establish single performance.

- **Feature:** Tasks are registered against time by the user and the system keeps track of the user activity (e.g. tasks performed, comments posted, files uploaded).
- **Benefit:** This is a compilation of data that will be presented as Contribution Dashboards, which will provide a visual breakdown of who did what. This removes the confusion on individual effort and offers factual information on peer reviews or grading.

1.7.3 The AI-Assisted Component

One of the major innovations of TeamTrack is that it applies intelligent logic to take the process of reporting to the next level of active assistance instead of passive one. The system uses rule-based logic and machine-learn ready logic to process the above generated data.

These automated suggestions are Smart Insights, but not automatic decisions, which put human agency in the first place. Key capabilities include:

- **Uneven Workload Detection:** The system compares work assignments and approximated hours to alert that one of the members is overworked, and the other is not worked to capacity.
- **Inactivity Alerts:** The system monitors the data in logs and since the team leader could see that there was a period of silence or inactivity among certain members, the team leader would intervene before the deadline is threatened.
- **Risk Prediction:** It is a proactive prioritization approach which involves consideration of the historical progress patterns towards identifying the task, which is likely to default within its deadline and allows re-prioritization of the team.

1.7.4 Conclusion

In summary, TeamTrack is the only thing that will allow small task lists to be converted into complicated enterprise software. It offers an organized platform that allows students to finish their projects and at the same time build effective teamwork habits and to be graded fairly through the incorporation of planning, communication, and analytics into one platform.

2. Overall Description

2.1 Product Perspective

2.1.1 System Overview

Team Track is a new collaborative project management web based platform which is a stand alone application that we are developing with a view to supporting university group projects. The design is aimed at overcoming typical problems faced by students in teams, including the problem of unequal participation, the inability to see the efforts made by someone, and the disunity of communication with various tools [5], [7]. We have modeled Team Track specifically to suit the academic environment rather than the general purpose type of project management software where fairness, accountability and transparency are necessary.

2.1.2 Standalone Nature and Core Integration

The system is not an addition or a customization of an existing platform. Rather, we will conceptually inspire ourselves by professional task management and collaboration tools and modify these concepts to the assessment oriented group work settings. We will make Team Track a solution to combine task management, team communication, time tracking, and contribution analytics in one centralized workspace, eliminating use of a variety of unconnected applications [5], [6].

2.1.3 Positioning in Educational Technology Context

Team Track will be placed in an advanced niche between conventional productivity tools and institutional learning management systems in a wider educational technology context. Although the available tools facilitate teamwork, they are mostly intended to be used in the workplace as opposed to a group evaluation in a learning institution. Team Track will be designed not to replace the current university systems, but to be a complementary system, with a flexible structure that can be integrated with it in the future, though is completely independent in its first release [5].

Table 22: System Context and Scope (Section 2.1 – Product Perspective)

Aspect	Description
System Name	Team Track
System Type	Web based collaborative project management system
Primary Purpose	Support coordination, transparency, and accountability in university group projects
Target Domain	Academic group work within universities
Standalone or Integrated	Standalone system (initial release)
Relationship to Existing Systems	Complements existing tools (e.g., LMS, productivity platforms) but does not replace them
AI Role	Advisory decision-support only (no automated judgments)

2.2 Product Functions

These are the major core functions that Team Track will offer to users operating in a project/workspace. These functions will aim to facilitate an organized groupwork, responsibility distribution, and openness in evaluation based groupwork settings [5], [6], [7].

2.2.1 User Authentication and Accounts

We will also enable users to sign-up and log-in safely. Once authenticated, the users can access only projects/workspaces that they are members of, and the project data will not be accessible to unauthorized users [5], [6].

2.2.2 Project/Workspace Management

We will enable users to create or join a workspace, workspace detail and team membership control. The workspace environments and administrative options will be role based (e.g. the visibility of the administrator / team leader) in order to facilitate guided project coordination [5], [7].

2.2.3 Task Management (Board and List Views)

We are going to assist in the creation, viewing, updating and closing of tasks. The tasks may be managed by users using a Kanban-style board (e.g., Backlog, In Progress, Review, Done) and a list view, which may be filtered/sorted by due date, assignee, priority, and status. This is to make sure that tasks are organized, trackable and well assigned throughout the project lifecycle [5], [6].

2.2.4 Task Collaboration and Activity History

We are going to supplement task-level collaboration and transparency with the help of task comments and an activity log. The most important changes in the tasks like status updates, reassignment and due date changes will be displayed to the users alongside the name of the member who has made a particular change and the timestamp of that change. We will also preserve this history, to enhance accountability and confusion, particularly in cases where dual or more members are engaged in the same task at the same time [5], [7].

2.2.5 Milestones and Calendar Visibility

We will also show milestone deadlines and task due dates in order to enable teams to plan the workload and track the process. The milestones/calendar view will allow users to see future deadlines and open related tasks so as to assist in completing them in time [5], [6].

2.2.6 Team Communication (Chat)

We shall do project-level communication in support to coordination. Messages will have time stamps and unread markers to enable the user to follow through efficiently. Near-real-time refresh will support real-time behavior in the first release, and may be upgraded to live updates where needed [5], [7].

2.2.7 Time Tracking and Timesheets (CORE)

We would assist in individual time tracking of tasks, i.e. start / stop timers and manual time records. Logged time will be summarized in timesheet-like formats (e.g. weekly totals) to help facilitate transparency of effort as well as reinforce contribution analytics without imposing productivity targets [5], [7].

2.2.8 Contribution Analytics and AI-Assisted Insights

The contribution summaries will be created based on the recorded task activity (e.g., completions, updates, comments) and time logs. In the instances where AI-assisted insights are involved, we will make them more of a recommendation than an automated decision. The insights could bring to the fore the patterns like uneven workload distribution, extended inactivity or tasks that could get delayed on the basis of observed progress patterns [5], [7].

2.2.9 Planned Enhancements (Future Releases)

To avoid over-claiming, we will treat the following as optional future enhancements unless implemented in the current release:

- File attachments/storage
- Real-time live updates via WebSocket's (beyond near-real-time refresh)
- Advanced AI insights beyond basic summaries

2.2.10 Task Activity and Engagement Indicators

We shall give task activity indicators to ensure accountability and early work stalling detection. With the help of task updates and time logs, as well as interaction history, we will show whether a task is active or not (e.g., no activity during a specified time frame or deadline is approaching, and little progress has been made). These indicators will facilitate effective follow-ups and awareness within the team in time and may activate optional reminders to the member designated or flags of visibility by the team leader. All indicators will be informational and can be edited by a user (e.g.,

indicating that a task is blocked with a reason) so that the system could allow collaboration without making automatic decisions regarding performance [5], [7].

2.2.11 Artificially Intelligent Query and Report Assistant.

We will add an AI-powered chat assistant that will enable users to ask questions about project data and reports in natural language. In answering questions like finding out the contribution of a certain member of the team to a certain section or task, the assistant will use task records, contribution log, history of activity, and report sections to give their answers. This characteristic will facilitate transparency, lessen manual work on the contribution analysis, and help instructors and fellow team members to comprehend the pattern of participation in an environment of a project.

2.2.12 Management of Resources and Documents.

We will offer resource and document management facilities to enable centralized access to the project-related files in every workspace. Users can post as well as have access to shared resources and manage them collaboratively. Moreover, we will also facilitate integration with external cloud storage system like Google drive whereby the user of the application can link and access files directly without having to duplicate data. The control of access to resources will be made according to the membership of workspace and the user role.

Table 23: Core Product Functions

Function Category	Description
User Authentication and Accounts	Secure registration/login; access limited to authorized workspaces
Project/Workspace Management	Create/join workspaces; manage membership and role-based settings
Task Management	Create/assign/update/close tasks with deadlines; Board/List views
Task Collaboration and History	Comments and activity log showing key changes and responsible user
Milestones/Calendar	Display milestone deadlines and due dates; link deadlines to tasks
Communication (Chat)	Project level messaging with timestamps and unread indicators
Time Tracking/Timesheets	Track time on tasks (timer/manual); generate timesheet summaries
Contribution Analytics	Summaries of progress and participation indicators
AI-Assisted Insights	Recommendation-only insights (no automated judgement)

Planned Enhancements	File storage, full WebSocket live updates, advanced AI (future)
----------------------	---

2.3 User Classes and Characteristics

Team Track will be designed to have several classes of users who have different roles and patterns of interaction.

2.3.1 Student Team Members

The main users of the system will be the student team members. We presume that they would be digital-literate and conversant with web-based platforms. Their primary inter-relationships will involve fulfilling assigned duties, recording the time spent in activities, and being part of the project communication, as well as going through the summary of individual contribution as a reminder of their role and at-team progress [7].

2.3.2 Team Leaders

Student leaders will be the team leaders that will organize group activities. They will also have the ability to control project settings, allocate and give priority to tasks, and view high-level contribution analytics to facilitate balanced participation and effective coordination during the project life cycle besides their normal member capabilities [7].

2.3.3 Instructors or Supervisors (Read-Only Access)

Teachers or managers can be provided with optional read-only permission to see the general progress of the project and summary of project contributions. We will delineate this position to offer supervision and transparency without being able to directly interfere with workforce within teams to promote equitable evaluation in respect of student autonomy [6], [7].

2.3.4 System Administrators

The administration of the platform, user accounts, as well as the availability and security of the system will be the responsibilities of system administrators. We will make sure that this user group does not participate in the project implementation or academic evaluation, and there is no ambiguity between system administration and educational processes [6], [7].

Table 24: User Classes and Roles (Section 2.3 – User Classes & Characteristics)

User Class	Description	Key Responsibilities
Student Team Member	Primary system user	Complete tasks, log time, communicate, view contribution summaries
Team Leader	Student coordinator	Assign tasks, manage project settings, review team analytics
Instructor / Supervisor	Optional read-only user	View project progress and contribution summaries
System Administrator	Technical user	Maintain system, manage accounts, ensure security and availability

2.4 Operating Environment

2.4.1 Client Environment (Browser and Device Access)

Team Track will be developed as a web based web application that is available on modern desktop and laptops. The browsers that will be supported will be Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari of the newer versions. The first release will not need a native mobile application since it will be designed to work best on bigger screens and interact with a keyboard [8].

2.4.2 Server Environment (Backend and Architecture)

The backend server will be running on web server which will perform the user authentication, store data, process analytics and provide real time communication services. The standard client-server architecture will be followed to ensure that it scales and is maintainable, with the last project milestones decided on the implementation technologies. It is assumed that the backend will be hosted on a typical web server platform (e.g. Linux-based hosting) and relational database to store structured data about projects and users [5], [8].

2.4.3 Network and Deployment Constraints

The system will need to have an active internet connection to access the system and have real-time support features like communication and notifications. We will tailor Team Track in such a way that it will operate within the normal network environment of the university and will be used only by the academicians. In the first release, we will not address mass commercial implementation, which will enable the system scope and performance expectations to stay in line with the educational goals [5].

2.4.4 Operating System Support

Team Track will be provided as a web application, which is based on a browser, and thus will be compatible with a big part of operating systems. Customers can use the system in standard desktop and laptop operating systems such as:

- Windows 7 to Windows 11 Microsoft windows.
- macOS: Modern web browsers: modern versions of the operating system.
- Linux: Distributions which can support supported web browsers.
- Windows 10-11 Windows 7-8.1 Support Windows 7-8.1 will be supported via browser compatibility.

Since one of the interfaces of the system will be fully implemented in the form of a web-browser, we will not need any installation of the operating-system.

2.4.5 Minimum Hardware Requirements

These are the minimum hardware requirements that are suggested to have a smooth and responsive user experience when accessing Team Track:

- Processor: Dual-core processor or its level of performance.
- RAM (memory): 4 GB will be the minimum required.
- Storage: Does not need much local storage, this data is handled at the server level.
- **Display:** Screen resolution of 1280 × 720 pixels or higher
- **Input Devices:** Physical keyboard and mouse or trackpad for efficient interaction

These features are meant to assist in the normal academic application situations like long term task management, data entry and interactive interaction.

2.5 Design and Implementation Constraints

This section identifies the key constraints that will affect the design, development, and implementation of the Team Track system. These limitations allow a feasible solution within the academic Capstone time frame while following the rules of technical, usability, and ethical requirements.

Team Track is being built as a university-level project management and accountability platform. Therefore, the implementation has to be realistic in scope, achievable within the time available for development, and consistent with academic expectations regarding assessment.

2.5.1 Academic Time and Resource Constraints

Since Team Track is being implemented as part of a Capstone project, the system has to be finished within a limited time frame (approximately four months). Development will therefore focus

on key functionality, including task tracking, time tracking, communication, and contribution analytics.

- Advanced enterprise-scale features will be available in the future.
- The development team will concentrate on making a stable MVP before adding optional modules.

2.5.2 Technology Stack Constraints

The web-based architecture of the project is going to be followed using widely supported technologies to ensure maintainability and accessibility.

Key constraints include:

- The backend implementation will be based on Python (Django + Django REST framework).
- Frontend will communicate using the cipe API through a sequential process through the web (HTTPS).
- A Django ORM relational database (PostgreSQL/SQLite) will be used.

These constraints guarantee that the system can be implemented with technologies appropriate to students developing the system.

2.5.3 Security and Role-Based Constraints

Due to the sensitive academic contribution data that Team Track handles, access control to the data is essential.

- Users can only access projects to which they belong
- Role-Based Access Control (RBAC) should be enforced for leaders, members, instructors, and administrators.
- Contribution analytics should not be judgmental; instead, need to be informational.

This ensures that no misuse occurs, and it is fair in academic environments.

2.5.4 AI Feature Constraints

Although the system includes AI-assisted insights, the AI component is constrained by ethical and academic boundaries:

- AI will only provide recommendations and summaries for users timing and work.
- AI will not automate grading or make evaluative decisions this depends on the team leader.
- AI insights must remain transparent and explainable.

This constraint avoids over-claiming automation beyond the project scope.

2.5.5 Deployment and Scalability Constraints

The initial release is designed for small-to-medium university teams rather than large commercial environments.

- Expected concurrent usage: 10-20 active users.
- Hosting will remain within typical web server environments.
- Full real-time WebSocket support may be deferred to future releases.

2.6 Assumptions and Dependencies

This section defines assumptions made during requirements planning and external dependencies that may affect system performance, system development progress, and the deployment feasibility.

These assumptions assist in defining the circumstances under which Team Track is expected to be successful.

2.6.1 User Assumptions

The system assumes that the primary users (students and instructors):

- Have basic digital literacy and familiarity with web platforms.
- Access the system using modern browsers such as Chrome, Edge, or Firefox.
- Will regularly log tasks and time entries for accurate analytics.

The system also assumes that student teams are motivated to use the platform for accountability and coordination.

2.6.2 Environment Assumptions

Team Track is designed as a browser-based platform, therefore it assumes:

- Users have access to a stable internet connection.
- Devices meet minimum hardware requirements (4GB RAM, modern dual-core CPU).
- The platform will be used mainly on desktops/laptops rather than mobile-first devices.

2.6.3 Data and System Dependencies

The system depends on several internal and external components:

- Database availability for storing tasks, logs, and activity history.
- Backend API stability for handling authentication, analytics, and communication.
- Optional integration dependencies such as: Email services for verification and password recovery and file storage services for report exports or attachments (future scope).

2.6.4 Development Dependencies

The successful completion of the project depends on:

- Continuous collaboration among team members.
- Access to development tools such as GitHub, IDEs, and testing environments.
- Timely implementation of core modules before optional enhancements.

2.6.5 Assumptions About Scope Control

To avoid uncontrolled expansion, it is assumed that:

- The first release will focus only on academic group project management.
- Commercial project management features (ERP integration, grading automation) are out of scope.
- Advanced AI prediction features will remain future improvements.

2.7 System Workflow Overview

This part gives an overview of the interaction between users and Team Track at a high level of understanding how a group project is taken from beginning to end. The sequence of actions from account creation to the execution of a task, collaboration, time tracking, and contribution analytics is covered by the workflow.

Team Track is aimed at centralizing academic teamwork into one structured environment.

2.7.1 Workflow Stages

Stage 1: User Registration and Login

1. A student or instructor registers using their university email.
2. The system verifies credentials and grants authenticated access.
3. Users are redirected to their dashboard upon successful login.

Stage 2: Workspace and Team Setup

1. A Team Leader creates a new project workspace.
2. A unique join code or invitation is generated.
3. Team members join the workspace and roles are assigned.

This establishes the collaborative project environment.

Stage 3: Task Planning and Assignment

1. Tasks are created with deadlines, priority, and descriptions.
2. Tasks are assigned to specific members.
3. Tasks appear in Kanban Board or List View.

This ensures structured workload distribution.

Stage 4: Collaboration and Communication

1. Members communicate using the built-in project chat.
2. Task-level comments allow clarification and updates.
3. All major actions are stored in the activity log for transparency.

Stage 5: Time Tracking and Effort Logging

1. Users start/stop timers while working on tasks.
2. Manual logs can be added for offline work.
3. Weekly timesheets summarize effort contributions.

Time tracking supports accountability without enforcing productivity targets.

Stage 6: Contribution Analytics and Insights

1. The system generates contribution dashboards using task activity and time logs.
2. Leaders and instructors can view workload distribution.
3. AI-assisted insights may highlight patterns such as imbalance or inactivity.

Analytics remain supportive rather than evaluative.

Stage 7: Project Completion and Archiving

1. Once the project ends, the workspace can be archived.
2. Data remains available for review and academic assessment.
3. No further edits are allowed after archiving.

2.7.2 Workflow Summary Table

Workflow Step	User Action	System Output
Registration/Login	User signs in	Secure access granted
Workspace Setup	Leader creates project	Workspace + join code
Task Assignment	Tasks allocated	Kanban/List updated
Collaboration	Chat + comments	Activity history stored
Time Tracking	Timer/manual logs	Timesheet summaries
Analytics Review	Dashboard accessed	Contribution insights
Completion	Archive project	Read-only final record

Table 25: Work Flow Table

3. System Features

3.1 User Registration and Authentication

3.1.1 Description and Priority

This will be the gateway to the Team Track platform that is secure. It is also configured to specifically fit an academic setting where all the contributions, time logs, and communications are credited with verified university identities. It is a High Priority feature since it is a requirement to all the other system functionalities.

3.1.2 Stimulus/Response Sequences

- Stimulus: A new student or instructor is enrolling into the university by means of registering with an email address and a valid password.
- Response: The system checks the email domain, verifies that the email address does not exist, and mails a time based activation link to the inbox of the user.
- Stimulus: Clicking at the activation link in university email.
- Response: The system provides the account with the user profile and redirects the user to the log in page.
- Stimulus: User clicks the Logout button and is an authenticated user.
- Response: The system will end the session token, clear the local project data cache and redirect the user to the landing page.

3.1.3 Functional Requirements

- FR 3.1.3.1: The system will also provide the ability to open an account through the entry of full name, university email, and the use of a secure password.
- FR 3.1.3.2: The system will ensure that the given email address uses the standard university domain format (i.e. @university.edu).
- FR 3.1.3.3: The system will send an automated account activation email with a unique (time-sensitive) link (which will have a life span of 24 hours) to the registered email address.
- FR 3.1.3.4: The system will solely permit activation of an account and further log-in once the user has managed to click the activation link.
- FR 3.1.3.5: The system will have a secure log-in interface where a user will have to enter his/her registered e-mail and an encrypted password to enter.
- FR 3.1.3.6: The system will have a functionality of Forgot Password that will enable the user to request a password reset through his registered university email.
- FR 3.1.3.7: The system will enforce password complexity policy of at least 8 character with at least one upper case letter, one lower case letter, one number, and one special character.
- FR 3.1.3.8: In case the user is denied access because of an incorrect set of credentials or because the account is not activated, an error message will be displayed clearly to the user in the system.
- FR 3.1.3.9: The system will automatically end the session of a user after a time of idleness (e.g. 60 minutes) to avoid access to unauthorized information in open-to-the-world laboratory facilities.
- FR 3.1.3.10: The system will enable users to effectively log out of the platform with the data of all the sessions being wiped off the browser.

3.2 Role-Based Access Control (RBAC)

3.2.1 Description and Priority

This option determines the groups of authorization and permissions by the various user classes within the Team Track platform. The use of RBAC will allow integrity of data and avoid illegal changes to project settings, task assignment, and contribution logs. This is a High Priority characteristic because it will uphold the core of the system, which is fairness and accountability within a university environment.

3.2.2 Stimulus/Response Sequences

- Stimulus: The user in the role of Student Team Member tries to access the menu of Project Settings to delete another user.
- Response: The system detects the unauthenticated role and denies entry by concealing the menu or showing an Access Denied notification.
- Stimulus: A Team Leader picks one of the members and transforms a member into a Leader.
- Response: The system will update the role in project attribute in the project member entity and will renew the permissions of the designated user.
- Stimulus: An "Instructor" enters a given workspace.
- Response: The system detects the read-only role and turns off all the "Create," "Edit" or "Delete" buttons on the task board and chat module.

3.2.3 Functional Requirements

- FR 3.2.3.1: The system will have four major user roles: Student Team Member, Team Leader, Instructor (Supervisor), and System Administrator.
- FR 3.2.3.2: Team Leaders will be able to control project-level environments, such as workspace names, project description, and milestone deadline in the system.
- FR 3.2.3.3: The system will give Team Leaders the unique right to add people to a workspace and allot or re-allot work.
- FR 3.2.3.4: The system will permit Student Team Members to access all project data but will only permit them to write updates to their own Time Log entries and Task status updates to which they are assigned.
- FR 3.2.3.5: The system will impose read-only permissions on the role of an Instructor, allowing only seeing the project progress, contribution analytics, and logs of activities.

- FR 3.2.3.6: The system should only enable the System Administrator to have access to the entire platform in terms of management (e.g. approvals of accounts and technical maintenance) but not in terms of editing project-specific academic content.
- FR 3.2.3.7: The system will automatically update a user interface (e.g. dashboard buttons and menu options) according to his/her active role in a particular project.
- FR 3.2.3.8: The system will have a role-management interface that will allow the incumbent Team Leader to demote a Member to Leader position or relinquish leadership.
- FR 3.2.3.9: The system will reject any data modification request made by a user within a user session that lacks the necessary write permissions on that particular entity.
- FR 3.2.3.10: The system will show a standardized error prompt in case a user tries to overcome role restrictions through direct manipulation with URL.
- FR 3.2.3.11: The system will maintain a role-change history log that records all role assignments, modifications, and removals within a project workspace.

3.3 Project and Team Management

3.3.1 Description and Priority

This option allows the formation of collaborative workspaces (projects) that are centralized and the control of the team structures within these spaces. It enables users to set project boundaries, membership management and hierarchy that is necessary to set in academic accountability. This is a High Priority feature considering that this is the main container of all the other activities (tasks, logs and communication).

3.3.2 Stimulus/Response Sequences

- Stimulus: When a user clicks the option to create a new project and provides a project name and description.
- Response: The system generates a new workspace record to which the creator is granted the role of the Team Leader, and a special Project Join Code is created.
- Stimulus: A Team Leader goes to the email address of a university and writes to a peer asking them to join.
- Response: When the email is found, the system gives a workspace invitation notification and adds the user to the list of pending members.
- Stimulus: A student types a valid Project Join Code into the interface of the Join Project.
- Response: The system has confirmed the code, created an account of the user within the project as a student team member, and provided the user with the project dashboard.
- Stimulus: A Team Leader chooses the action of the Archive Project.
- Response: The system changes the workspace to a read only position and leaves all the data intact to be assessed by the instructor, but does not allow any additional editing.

3.3.3 Functional Requirements

- FR 3.3.3.1: The system will enable authorized users to create a new workspace by filling out a project title, a short description, and an estimated completion date.
- FR 3.3.3.2: a system should also automatically give the role of Team Leader to the user that creates the workspace.
- FR 3.3.3.3: The system will create a unique Project Join Code in alphanumeric that will be assigned to each new workspace to make it easy to enroll students.
- FR 3.3.3.4: The system will enable Team Leaders to add other registered users of the university into a workspace through their university email address.
- FR 3.3.3.5: There will be a Team Membership List of each workspace in the system, with the names, university emails, and designated role of all participants.
- FR 3.3.3.6: The system will enable the Team Leaders to change metadata of projects (e.g., make changes to the description or a deadline) any time prior to the archiving of the project.
- FR 3.3.3.7: The system will provide the team leaders to demote a student team member to a team leader to facilitate the common leadership models.
- FR 3.3.3.8: The system will enable Team Leaders to fire members out of a workspace, which will strip such a user of access to the project data and retain his/her history of activities to ensure transparency.
- FR 3.3.3.9: Within the system, each workspace will have a Project Dashboard, which shows a top-level overview of current work and a timeline of the most recent activity on the team.
- FR 3.3.3.10 The system will have a Project Archiving feature, which will prevent any further editing to the workspace, but leave all the analytics of contributions and communication history in the assessable state.

3.4 Task Management System

3.4.1 Description and Priority

This is the main feature that drives the group work in a workspace. It enables the issue, delegation and status monitoring of individual units of work. It has both Board and List views thus assuring the visibility of the team members and leaders on the progress of the project. It is a High Priority feature because it produces the "Activity Log" and "Engagement Indicators " required by the analytics of the system.

3.4.2 Stimulus/Response Sequences

- Stimulus: A Team Leader, by clicking on create task, provides a title, description and a due date.

- Response: The system stores the task and gives it a distinct task-id and it appears under the Backlog or To-Do column of the board.
- Stimulus: A user can drag a task card out of the column In Progress to Review on the Kanban board.
- Response: The system logs the change in database and logs the time where the task changes and the action in the project Activity Log.
- Stimulus: A user makes a filter that allows only the tasks assigned to him or her to be displayed.
- Response: The system resorts to displaying the view of only a subset of tasks that matches the ID of the user.
- Stimulus: A Team Member leaves a comment regarding the given task.
- Response: The system is appended with the comment to the task record and informs the assignee and Team Leader.

3.4.3 Functional Requirements

- FR 3.4.3.1: The system will enable Team Leaders and the Student Team Members to add tasks to an active workspace by specifying a title, description, a priority (Low, Medium, High), and a specific deadline.
- FR 3.4.3.2: The system will enable Team Leaders to delegate to a member only one task to improve the level of individual accountability.
- FR 3.4.3.3: The system will have a Kanban Board that will show columns of tasks organized according to their status: Backlog, In Progress, Review and Done.
- FR 3.4.3.4: The system will include a List view where the user can sort tasks based on due date, priority or assignee.
- FR 3.4.3.5: The system would automatically refresh the Activity Log whenever a task is created, edited, re-allocated or modified status.
- FR 3.4.3.6: The system will enable users to add comments to certain tasks to enable the context-related communication.
- FR 3.4.3.7: The system will include the Task Activity Indicators to identify stalling through highlighting of tasks that are not updated in a designated time period (e.g. in 48 hours).
- FR 3.4.3.8: The system will have the ability to designate a task as being blocked, and have an obligatory text box to describe what has caused the blocking.
- FR 3.4.3.9: The system will allow search functionality which will search specific tasks depending on the words in the title or description.
- FR 3.4.3.10: The system will have a warning displayed on the dashboard regarding any task that has a deadline of less than 24 hours and has not been done so.
- FR 3.4.3.11: The system will allow task dependencies to be defined, enabling tasks to be linked such that one task cannot begin until another is completed.

3.5 Time Tracking and Effort Logging (CORE)

3.5.1 Description and Priority

This option enables each user of the team to document actual time used on particular activities in a workspace. The factual foundation of the analytics of the system by recording effort data using both live timers and manual logs will form the basis of the analytics of the system in terms of "Contribution Analytics" (Section 3.7), and "AI-Assisted Insights" (Section 3.8). A feature of this type of High Priority is crucial to address the issue of unequal participation that was determined in the System Overview.

3.5.2 Stimulus/Response Sequences

- Stimulus: Clicked: A user navigates to a task assigned to him and presses the button named Start Timer.
- Response: The system starts a running clock, switches the button to the Stop Timer position and starts showing a visual clue that there is an active session.
- Stimulus: When a user clicks on Stop Timer, he/she does it.
- Response: The system stops the clock and counts the total time, asks the user a short description of the Work Performed and stores the record to the Time Log entity.
- Stimulus: When a user clicks the manual log, date, start time and end time are entered in which work had been done offline.
- Response: The system checks the validity of the length of time (cannot be less than zero), makes sure that the entry is associated with a given task_id, and modifies a weekly timesheet of the user.
- Stimulus: A Team Leader looks at the summary of the Team Timesheet.
- Response: The system summarizes all the logs related to the project and presents a table of the hours spent per member.

3.5.3 Functional Requirements

- FR 3.5.3.1: The system will also have an automated Start/Stop Timer of all tasks allocated to the user that has logged into the system.
- FR 3.5.3.2: It will provide the system that will enable the users to manually add Manual Time Records (Date, Start Time, End Time) to record project work done out of the web platform.

- FR 3.5.3.3: The system will enforce a mandatory textual description of work (minimum 10 characters) to every time log entry prior to it being saved in the database.
- FR 3.5.3.4: The system will not allow a user to have multiple running active timers on various tasks at the same time.
- FR 3.5.3.5: The system will be able to automatically estimate the length of every log and associate it with the user id and task id in question.
- FR 3.5.3.6: The system will include a Personal Timesheet view where users are able to view a summary of the logged hours with each week and project displayed separately.
- FR 3.5.3.7: The system will enable Team Leaders and Instructors to access Aggregated Timesheets that indicate aggregate effort put in by all the project members.
- FR 3.5.3.8: The system will enable users to amend or remove their personal time logs in a 24-hour period (Grace Period) after which the time log will become frozen to ensure the integrity of analytics.
- FR 3.5.3.9: The system will show a live Effort Indicator (e.g., a progress bar or pie chart) of how much time has been logged against how complicated the task should be.
- FR 3.5.3.10: The system will produce an error message in case a manual time log has an overlap with an existing record of the same user.

3.6 Team Chat and Communication Module

3.6.1 Description and Priority

The Team Chat and Communication Module allows project members to communicate with each other within a project. This feature gives users functionalities such as sending messages related to tasks, giving updates and progress checks, and overall collaborating with each other, and ensures all the communication stays relevant to the project. Messages are stored and can be reviewed later.

Priority: High

3.6.2 Stimulus/ Response Sequences

- **Stimulus:** A project member opens the project chat.
- **Response:** The system loads the project chat interface and displays the message history.
- **Stimulus:** A project member types a message and clicks “Send”.
- **Response:** The system makes sure the message isn’t empty, stores it with the sender, timestamp, and project ID, and then displays it in the chat.
- **Stimulus:** Another member is currently viewing the same project chat.
- **Response:** The system displays the new message to the other member in near real-time.
- **Stimulus:** A user attempts to send a message that is only spaces or invalid input.

- **Response:** The system rejects the submission and prompts the user to enter a valid message.

3.6.3 Functional Requirements

- FR 3.6.3.1: The system shall allow users to send text messages within a project-specific chat
- FR 3.6.3.2: The system shall associate each message with a sender, timestamp, and project
- FR 3.6.3.3: The system shall display message history for each project in chronological order
- FR 3.6.3.4: The system shall only allow access to project chat messages to project members
- FR 3.6.3.5: The system shall validate messages to prevent empty or invalid submissions
- FR 3.6.3.6: The system shall allow project members to view messages after logging out and logging back in (persistent messages).
- FR 3.6.3.7: The system shall support basic message formatting safety by sanitising message content before storing or displaying it.
- FR 3.6.3.8: The system shall provide a clear indication of who sent each message (e.g. showing the name or role of sender).

3.7 Contribution Analytics Dashboard

3.7.1 Description and Priority

The Contribution Analytics Dashboard provides information on individual and team participation within a project. The dashboard aims to improve accountability as well as showing workload between members by visualizing metrics (charts etc.) like time spent on tasks, task completion, and frequency of activity. This feature assists team leaders and instructors in finding imbalances in contribution between members and monitoring the overall advancement of the project

Priority: High

3.7.2 Stimulus/ Response Sequences

- **Stimulus:** New task is updated or time logs are added by members.
- **Response:** The system recalculates totals and updates the dashboard values so that the analytics stay accurate.
- **Stimulus:** The leader selects a specific member to view their contribution breakdown.
- **Response:** The system filters the dashboard to show that member's time logged, task statuses, and activity.
- **Stimulus:** The system has insufficient data to show analytics (e.g. nothing has been logged yet).

- **Response:** The system displays a clear message (e.g., “No data recorded yet”) instead of showing empty charts.
- **Stimulus:** A project leader opens the Contribution Analytics Dashboard for their project.
- **Response:** The system retrieves task, time log, and activity data and displays the analytics view.

3.7.3 Functional Requirements

- FR 37-01: The system shall calculate total time spent by each user on project tasks.
- FR 37-02: The system shall present contribution data using visual summaries such as charts or tables
- FR 37-03: The system shall display task completion or status statistics per user per project
- FR 37-04: The system shall allow project leaders to view contribution analytics for all team members (or can all team members see the contribution)
- FR 37-05: The system shall derive analytics data from tasks, time log, and activity records.
- FR 37-06: The system shall update contribution analytics automatically when new task or time log data is recorded.
- FR 37-07: The system shall provide a comprehensive overall project summary display showing totals for the entire team (e.g., total hours, tasks completed).
- FR 37-08: The system shall allow the dashboard to display contribution data for the chosen time range (e.g., weekly view for the whole project timeline).
- FR 37-09: The system shall highlight any signs of contribution imbalance within the team (e.g., unusually low logged time compared to the team average) as something to review
- FR 37-10: The system shall allow authorised roles to export or download a simple summary report (e.g. CSV/PDF style output) of contribution analytics.
- FR 37-11: The system shall provide comparative contribution indicators that display each user’s contribution relative to team averages within the project.

3.8 AI-Driven Project Assistant

Description: The AI-Driven Project Assistant is an intelligent support agent that is included within the TeamTrack platform. In contrast to regular passive help systems, this assistant works on two planes reactive and proactive. In response, it is an interface (chatbot) enabling students to query project information (in natural language) by asking questions like when is the next deadline or what has been happening lately.

The system performs as a background analytics engine proactively. It constantly tracks project health indicators, such as the rate of task completion, time spent by individuals, and frequency of communication. Through rule-based logic on this data, the assistant detects possible risks, including imbalanced workload, so-called free-riding, or presented deadline misses. These are communicated as Smart Recommendations to Team Leaders and Instructors, so that an issue in academics is identified before it turns into a serious failure.

Stimulus/Response Sequence:

- **Stimulus:** Student user enters query What are my high-priority tasks this week? into assistant chat window.
- **Response:** The system interprets the natural language query, gets the tasks in the database allocated to the user of type High priority and with a due date that is within 7 days, and presents it as an interactive list.
- **Stimulus:** The background analysis engine identifies the fact that student A has recorded 80% of the total time spent on the projects and student B has recorded 5% within 2 weeks.
- **Response:** The system triggers an "Workload Imbalance Alert to the dashboard of the Team Leader and proposes the re-distribution of the pending tasks to balance the effort.
- **Stimulus:** An Instructor requests a "Project Health Summary" for a specific group.
- **Response:** The system combines the activity logs, completion speed and the deadlines that are missed, to come up with a text-based summary (e.g., a group 4 is on track but there are indicators of siloed communication).

Functional Requirements:

- **FR 3.8.1:** The system will include a natural language processing (NLP) interface that can be used to understand user queries about the project status, deadlines, and team activity.
- **FR 3.8.2:** The system will be using a rule-based algorithm that will continually scan the distribution of completed tasks and recorded hours of group members to produce a score under the term of a Workload Variance.
- **FR 3.8.3:** The system will send an automatic message to the Team Leader in case a group member has no documented activities (login, task update, chat) within a defined number of hours (default: 72 hours).
- **FR 3.8.4:** The system will also include Smart Recommendations of task assignment which will suggest particular members of a new task, basing the recommendation on the availability and past speed of completion.
- **FR 3.8.5:** The system will compute a Deadline Risk Probability of active tasks that will be computed by calculating the remaining time and time average of similar past tasks.
- **FR 3.8.6:** The system will produce "Contribution Insight" reports to the Instructors which explicitly point out gaps between self-reported time logs and actual system activity (i.e. long hours logged and few artifacts produced).
- **FR 3.8.7:** In order to be transparent, all AI generated suggestions should be explicitly labeled as System Generated Suggestions and that human approval (Team Leader or Instructor) must be received before any data is altered.

- **FR 3.8.8:** The system will allow users to request explanations for AI-generated insights, providing a breakdown of the data and logic used to produce the recommendation.

3.9 Notifications and Reminder System

Description: The Notifications and Reminder System is the proactive interactive platform of the TeamTrack platform. It is mainly used to counter the out of sight, out of mind phenomenon prevalent in academic projects that are long term. This characteristic does not provide alerts on the fly, but rather it uses a smart "Escalation Engine" that focuses the information according to urgency

The system difference between Passive Notifications (e.g., file upload, an overall comment) and Actionable Alerts (e.g., approach deadlines, an @mention, a change in assignment). The system will help to avoid the issue of notification fatigue by allowing to batch minor updates (low-priority) and deliver high-priority notification through various channels (In-App, Email) to ensure that the user is informed.

Stimulus/Response Sequence:

- **Stimulus:** A Team Leader alters the urgency of a task, switching it to the value of Medium to the value of Urgent.
- **Response:** The system instantly recognizes the Owner of the Task and sends a push notification of high priority to their device and shows the task on their personal dashboard in red.
- **Stimulus:** A task deadline is reached but the status is not noted as a Completed one.
- **Response:** The system will activate a Missed Deadlines Protocol: it notifies the assignee of the failure and forwards an email to the Team Leader, causes a late event to be recorded in the project analytics.
- **Stimulus:** A user has been offline in 5 days and has messages to read.
- **Response:** The system generates a mail known as the Weekly Digest that summarizes all the missed activity, mentions of all the chat messages that have not been read, and the various due dates and sends them to the registered university email of the user.

Functional Requirements:

- **FR 3.9.1:** This system will have a centralized area called the Notification Center, which will be available in the global navigation bar and will be divided into three categories: Tasks, Mentions, and System Alerts.
- **FR 3.9.2:** The system will implement a Multi-Channel Alerting mechanism, allowing users to configure whether they receive specific alerts via Email, In-App Popups, or both.

- **FR 3.9.3:** The system will also use Deep Linking of all the notification whereby clicking an alert will take the user directly to the task card or chat thread of the project and not the home page.
- **FR 3.9.4:** The system will issue the reminders about the Procrastination Prevention at the set schedule (e.g., 7 days, 3 days, 24 hours) before a milestone deadline.
- **FR 3.9.5:** The system will automatically combine high volume low priority events (e.g., 5 users updated 10 tasks) into one summary notification so that inbox is not flooded.
- **FR 3.9.6:** The system will make sure that urgent tasks with high priority are prioritized above user settings regarding the Do Not Disturb feature to provide urgent deadline alerts.
- **FR 3.9.7:** The system will have a Mark as Read feature that will synchronize with the session and in case a user reads a notification on mobile, it must be flagged in read on desktop.

3.10 Activity Log and Audit Trail

Description: The Activity Log and Audit Trail is the backbone of the forensic component of the TeamTrack system. Conflicts are common when it comes to academic group work, and they are seen to be based on one of the following: contribution equality, data loss, or unauthorized changes. It is a characteristic that captures a recorded history of each state change in the project environment in granular and immutable form.

As opposed to a plain “Recent Activity feed, this module records the Before and After states of the data, the exact identity of the actor as well as the time to the second. This information is essential to the "Contribution Analytics" integration and can give Instructors an indisputable evidence of attendance (or absence) to grade.

Stimulus/Response Sequence:

- **Stimulus:** A user is trying to delete a shared document in the folder of Final Report.
- **Response:** The system is deleting and at the same time, a permanent record is written in the AuditLog table with: Action:DELETE, Object:FinalReport.docx, Actor:User_ID, and RecoveryID.
- **Stimulus:** An Instructor should check whether a student actually submitted code prior to a deadline.
- **Response:** the Instructor uses the Audit Log filter of "User: StudentName" and Date Range: Last 2 Weeks. It gives a chronological list of all of the particular edits, uploads, and comments left by that student with the system.
- **Stimulus:** A user tries to do some editing to the description of a task done last month.

- **Response:** The system records this Post-Completion Edit as a flagged event and the Team Leader is warned of the possibility of historical revisionism.

Functional Requirements:

- **FR 3.10.1:** The system will capture an unaltered audit history of all tasks, files, and project settings Create, Read, Update, and Delete (CRUD) operations.
- **FR 3.10.2:** Every entry in the logs ought to permanently record the following metadata: Actor Identity (user ID), Timestamp (UTCM format) and IP Address, Type of action and the actual Data Field that was altered.
- **FR 3.10.3:** The system will also have Version Control logic of text fields (e.g., Task Descriptions) where one can see older versions of the text to retrieve information overwritten by another member.
- **FR 3.10.4:** The system will enable Team Leaders and Instructors to use a combination of various criteria to filter the Activity Log (e.g. Show all DELETIONS by USER X in PROJECT Y).
- **FR 3.10.5:** To maintain the integrity of data when appealing to the academic authorities, the system will not allow any user role, including Administrators, to manipulate or delete the entries of the Audit Log.
- **FR 3.10.6:** The system will offer an “Export Audit Trail” option, which will enable the Instructors to export the complete project history in the form of a time-stamped PDF or CSV file to be used in external grade documentations.
- **FR 3.10.7:** The system will automatically store Activity Logs of at least one academic year (or university-established period) of project activities before deleting data.

3.11 Updated Functional Requirements (TIS Report)

3.11.1. Functional Requirements

Functional requirements describe the specific services and behaviours that the system shall provide. These requirements define how the system shall support teamwork, project coordination, task management, contribution tracking, and AI-assisted support. The requirements are organised into functional areas to ensure clarity and traceability.

3.11.2. User Authentication and Account Management

The system shall provide secure user authentication and account management to ensure that only authorised users can access system resources.

This functional area includes user registration, login, session management, and access control.

Requirement ID Description

FR-1 The system shall allow users to register using valid credentials.

FR-2 The system shall allow registered users to log in securely.

FR-3 The system shall manage authenticated user sessions.

FR-4 The system shall restrict access to system features to authenticated users only.

This functionality ensures controlled access and supports role-based permission enforcement.

3.11.3 Project Management

The system shall provide project management functionality to support structured collaboration between users. Projects shall serve as containers for tasks and assigned users.

Requirement ID Description

FR-5 The system shall allow authorised users to create new projects.

FR-6 The system shall allow users to view projects to which they are assigned.

FR-7 The system shall allow permitted users to update project details.

FR-8 The system shall associate users with projects based on defined roles.

This functionality supports organisation, accountability, and role-based access control.

3.11.4 Task Management

The system shall provide task management functionality to support planning, assignment, and tracking of project activities.

Requirement ID Description

FR-9 The system shall allow users to create tasks within a project.

FR-10 The system shall allow tasks to be assigned to users.

FR-11 The system shall allow users to update task status and details.

FR-12 The system shall allow users to view task progress and deadlines.

This functionality supports responsibility allocation and progress monitoring.

3.11.5 Contribution Tracking and Activity Logging

The system shall provide contribution tracking and activity logging to enhance transparency and accountability.

Requirement ID Description

FR-13 The system shall log user actions related to tasks and projects.

FR-14 The system shall store contribution data for analysis and reporting.

FR-15 The system shall allow authorised users to view contribution records.

This functionality supports fair evaluation and collaboration transparency.

3.11.6 AI-Assisted Support Features

The system shall provide AI-assisted support features to help users interact with the system and receive guidance related to project activities.

Requirement ID Description

FR-16 The system shall accept user queries for AI-assisted support.

FR-17 The system shall generate AI-based responses for supported queries.

FR-18 The system shall restrict AI functionality to approved use cases.

This functionality enhances user experience while maintaining controlled AI behaviour.

3.11.7 User Roles and Permissions

TeamTrack system has Role-Based Access Control (RBAC) model to manage access to system functionality and data. RBAC will assist in offering security, accountability, and fair co-operation in a scholarly community to make sure that each user can access the system within the stipulated scope of the responsibility.

Access controls go both over the frontend interface, and at the API layer to the backend, and unauthorised actions cannot be performed regardless of how a request is spawned.

3.11.8 Defined User Roles

The system identifies four key user roles and each one of them is created to facilitate a particular pattern of interaction in the platform.

Student Team Member

Student Team Members are the main actors in the system, as they have to complete project work and document personal input.

Allowed Actions:

- Projects and dashboards are assigned.
- See tasks, deadlines and milestones.
- Mark the status of tasks and comments on the tasks.
- Time in the books working on something.
- See the personal contribution history and summary.

Access Limitations:

- Is not able to alter project level settings or milestones.
- The user has no ability to delegate.
- Not able to control team membership and user prepositions.

This role allows active involvement and retains the structure and control of the project.

Team Leader

Team Leaders are in charge of arranging the activities of the project and being able to keep track of the progress of the team.

Allowed Actions:

- Full Member Permissions of the Student Team.
- Form and shape project work spaces.
- Assign and reassign tasks Establish and revive milestones and deadlines.
- See team contribution analytics.
- add/ delete project members.

Responsibilities:

- Keep balanced distribution of work load.
- Follow up progress and involvement.
- Make sure project deliverables are done on time.

The position makes coordination and accountability a possibility without automated appraisal systems.

Instructor / Supervisor (Read-Only Role)

The teachers and supervisors will have a read-only access to provide transparency and academic supervision.

Allowed Actions:

- See project status, tasks and milestones.
- Is there access contribution analytics and summaries?
- Look at the logs of activity and history of communication.

Restricted Actions:

- None of the task creation or change.
- None of the project setups are accessible.
- None involved in project communication.
- No user or role management

This position maintains the autonomy of students and allows them to be reviewed academically.

System Administrator

SA is in charge of ensuring the technical integrity of the platform.

Allowed Actions:

- Administer user accounts and configuration of platform.
- Gather performance and health of the monitor systems.
- Uphold security measures and infrastructures.

Restricted Actions:

- No entry to academic project information.
- None of the tasks or records of contribution changed.

This distance does not allow administrative control over the academic content.

3.11.9 Role–Permission Summary

Table 19: Role-Permission summary table

System Function	Student Member	Team Leader	Instructor	Admin
View Projects & Tasks	✓	✓	✓	✗
Create / Edit Tasks	✓ (assigned only)		✗	✗
Assign Tasks	✗	✓	✗	✗
Log Time	✓	✓	✗	✗
View Contribution Analytics	✓ (self)	✓ (team)	✓	✗
Manage Project Settings	✗	✓	✗	✗
Manage Users & Roles	✗	✓	✗	✓ (platform only)

3.11.10 Permission Enforcement Strategy

User permissions have been enforced in the TeamTrack system through a layered security scheme to provide a uniform, reliable, and secure access control to all components of the system. Checks of permission are implemented on several levels to exclude the unauthorised actions despite the manner of a request initiation.

Role validation is applied on every secured endpoint of the API at the backend level. Every request made is verified to have a valid security token and the role of the user is verified and then no operation is done. Any requests failing to satisfy the permissions criteria are in effect denied, and it is impossible to execute restricted actions using the direct API access or by hand manipulating the request.

On the frontend, there is dynamism of the user interface, which considers the role of the authenticated user. Elements of the interface like buttons, menus and configuration options are either visible or invisible based on the permissions that a user has. This minimizes the chance of unintentional misuse and enhances usability through showing only relevant actions to the user depending on his or her role.

Moreover, attempt of unauthorised access is handled in a safe manner and logged to help in auditing and monitoring of a system. These logs will provide an insight into the breaches of access and help trace the trends of the potential security violations or abuse during the system uses.

This stratified policy implies that not only is permission enforcement strong, but also steady and resistant to unintentional and intentional abuse.

3.11.11 Design Rationale

The TeamTrack system academic and collaborative aspects have been selected to resort to Role-Based Access Control (RBAC) model to guarantee the system security and integrity. RBAC also provides a more structured and scaled approach to access control through grouping permissions according to well defined user roles and not by user.

In terms of academics, RBAC can contribute to the effort of ensuring that the project data cannot be manipulated by a third party and only the approved users can implement structural changes such as employee assignment or deadline changes. This assists in establishing equitableness and responsibility in group tasks based course work by preventing unintentional or non-acceptable amendments to joint artefacts of work.

RBAC also enables the contribution to be transparent since the actions are associated with the user roles and user identities. This openness helps in adequate documentation of activities, contribution analytics, which are the primary concern of the system to help improve accountability in shared academic tasks.

The primary reason is the presence of a read-only role of an instructor in that this role enables academic supervisors to check summaries of project progress and contribution without disrupting the daily team activities. This alternative of design will retain student autonomy but will provide a significant control to the instructors.

Lastly, RBAC also facilitates scalability and maintainability in the future. The access control model is flexible and can accommodate the new features of the future as the system advances; new privileges or roles may be added without making significant changes in the system architecture, which implies that the access control model will be flexible and adaptable to the upcoming changes.

3.11.12 Use Cases

The use cases are the description of the interaction between the users and the TeamTrack system to fulfill a certain goal. They give practical and user oriented perspective into the way in which the system functionality is accessed and implemented; they are a complementary to the functional requirements as identified in Section 4.1.

The use cases specified in this section represent realistic work processes involved in academic projects, are consistent with the role definitions in Section 4.3, and act as a base of system design, UI development, and testing in subsequent sections of this report.

3.11.13 UC-01: User Registration and Authentication

Primary Actor: Student / Instructor

Precondition: It is not the case that the user is an authenticated user.

Postcondition: The user logs in and is taken to his or her dashboard.

Description:

This use case determines the access to the system by a user in a secure manner.

Main Flow:

1. Valid credentials are used to register the user.
2. The information is authenticated by the system.
3. The user is authenticated and logs in.
4. An encrypted session is created.
5. The user-specific dashboard is presented.

Alternate Flow:

- In case of the invalidity of credentials, a system shows an error message and denies access.

3.11.14 UC-02: Create Project Workspace

Primary Actor: Team Leader

Precondition: The user is logged in and possesses the privilege of a Team Leader. **Postcondition:** New project workspace is created.

Description:

This application case will imply the development of an organized project setting.

Main Flow:

1. The Team Leader chooses the alternative to develop a project.
2. Project details are entered.
3. The system is a validation of the input.
4. The working environment is established.
5. Team Leader will be assigned ownership permissions.

3.11.15 UC-03: Join Project Workspace

Primary Actor: Student Team Member

Precondition: This project is there and the user is already authenticated.

Postcondition: A user joins the project workspace.

Description:

This use case enables students to become part of a project that is in place.

Main Flow:

1. The user provides acceptable project join identifier.
2. The system authenticates the identifier.
3. The user is enrolled in the project.

4. Project activities and communications facilities get available.

Exception Flow:

In case of invalidity of the identifier, access is denied and feedback is given.

3.11.16 UC-04: Task Creation and Assignment

Primary Actor: Team Leader

Prerequisite: There is an existence of a project workspace.

Postcondition A task is assigned and created.

Description:

This use case determines the way of planning and delegating tasks.

Main Flow:

1. The Team Leader develops a task that has the necessary information.
2. One of the members of the team is given responsibility.
3. The project task view presents the task.

3.11.17 UC-05: Task Update and Collaboration

Primary Actor: Student Team Member

Precondition: The user is allotted the task.

Postcondition: Task changes are logged.

Description:

This application case assists in cooperative task performance.

Main Flow:

1. Task status or content is updated by the user.
2. The comments are provided where necessary.
3. The update is logged in the system with timestamps.

3.11.18 UC-06: Time Tracking and Activity Logging

Primary Actor: Student Team Member

Precondition: The task is there and it is allocated to the user.

Postcondition: Time and activities information have been stored.

Description:

In this application, personal work is held to account.

Main Flow:

1. The user will activate a timer of a task or deactivate it or input time.
2. Time entry is recorded in the system.
3. The contribution summaries are kept up to date.

3.11.19 UC-07: Contribution Analytics and AI-Assisted Insights

Primary Actor: Team Leader / Instructor

Precondition: There is the contribution data.

Postcondition: Analytics are presented.

Description:

This application case assists the workload monitoring and transparency.

Main Flow:

1. The analytics dashboard is accessed by the user.
2. Summaries and indicators of contributions are shown.
3. Intelligence insights with the help of AI are expressed as suggestions.

Constraint:

- The outputs of AI are informational and do not conduct an automated assessment.

3.11.20 UC-08: Instructor Project Review (Read-Only)

Primary Actor: Instructor / Supervisor

Precondition: The instructor can read-only.

Postcondition: The data in the project is examined, but not edited.

Description:

This application is useful in academic supervision.

Main Flow:

1. The teacher gets access to a work station.
2. Tasks, progress and analytics are checked.
3. There are no actions of modification.

3.11.21 UC-09: Activity Audit and History Review

Primary Actor: Team Leader / Instructor

Precondition: There are activity logs.

Postcondition: The review of historical activity is carried out.

Description:

This use case enables one to view task and project history.

Main Flow:

1. The user gets access to activity logs.
2. Updates on schedules and tasks are shown.
3. Past activities are assessed in terms of transparency.

3.11.22 Use Case Coverage Summary

The specified use cases all show the support provided by TeamTrack to:

- Formal project and activity management.
- Open donation monitoring.
- The non-evaluative insights with the help of AI.
- Lecturer supervision that is not intrusive.

Such applications are directly used to determine system design, UI flows, and test cases as shown in subsequent parts of this report.

4. External Interface Requirements

4.1 User Interface Requirements

Team Track is built as a web application that will be optimised to work in desktops and laptops as project collaboration will be done with a lot of typing, updating tasks, creating time cards and clicking various displays (tasks, chat, deadlines, and analytics). [4]. We are going to make the interface uniform and compatible with the following efficient browsers, Google Chrome, Microsoft Edge, Mozilla Firefox, and Apple Safari, among others [8], [9].

4.1.1 General layout and navigation

We shall adopt the same layout on all the pages so that confusion and time taken to learn is minimized. [8].

Top navigation bar (persistent)

- Team Track logo (returns to the Dashboard)
- Project/Workspace selector
- Global search (tasks, milestones, members, messages)
- Notifications (updates, mentions, due dates)
- User menu (Profile, Settings, Logout)

Side navigation menu (workspace navigation)

- Dashboard
- Tasks (Board / List)
- Milestones / Calendar
- Team / Members
- Chat (project communication)
- Time Tracking / Timesheets
- Analytics / Contributions
- Settings (visible based on role permissions)

Table 26: Primary Navigation Structure

Navigation Area	Item	Purpose	Visible to
Top Bar	Logo	Return to Dashboard	All users
Top Bar	Workspace selector	Switch current project/workspace	All users
Top Bar	Global search	Find tasks/milestones/members/messages	All users
Top Bar	Notifications	View mentions, assignments, due date alerts	All users
Top Bar	User menu	Profile/settings/logout	All users
Sidebar	Dashboard	Project overview	All users
Sidebar	Tasks (Board/List)	Track and manage tasks	All users
Sidebar	Milestones/Calendar	View deadlines and milestone plan	All users
Sidebar	Team/Members	View roles and membership	All users
Sidebar	Chat	Project communication	All users
Sidebar	Time Tracking/Timesheets	Log time and view summaries	All users
Sidebar	Analytics/Contributions	View progress and contribution summaries	All users
Sidebar	Settings	Configure workspace rules and roles	Role-based

4.1.2 Key screens and UI behaviour

The following are the key user screens that we will provide.

Login / Authentication screen

- Email/username and password fields having clear validation feedback. [11].
- Incorrect password or email digit error messages, like session expired, log in again. [11]

Dashboard

- Status cards/tiles (update on project overview) (pending milestones, tasks to be done, outstanding tasks, project activity)
- “Assigned to me” panel listing current tasks with due dates and priority
- Task shortcuts, Calendar, Chat, Time Tracking, and Analytics.

Tasks page (Board and List view)

- Board view (Kanban- style) and its columns can be configured like Backlog, In Progress, Review and Done.
- List view with sorting/filtering by due date, assignee, priority, and status
- Users may open task details on a card/row and customize task fields.

Task detail's view (panel or page)

- Title, description, status, priority, due date, assignee(s), labels/tags, and checklist/subtasks
- Discussion and clarification thread of comments.
- Activity log (audit trail) of that which was changed and by whom.
- Entry point in time tracking (e.g. start/stop timer or add manual entry point to task)

Milestones / Calendar

- Shows milestone and task due dates.
- Offers visibility of deadlines to allow the teams to plan the workload and prevent failures to submit it.

Chat

- Communication area of the team messages and coordination on the project level.
- Missing indicators and message timestamps.
- Close real-time behaviour in the first release (refresh/polling), and a live upgrade path in case it is needed. [9].

Time Tracking / Timesheets

- Manual and start/stop timer of tasks.
- Summary of weekly timesheet per user.

- Clear validation for incorrect inputs (e.g., “End time cannot be earlier than start time.”)

Analytics / Contributions

- Reports on completion of tasks, time, and participation rates on the basis of recorded activity.
- In case the assistance of AI is provided as a aspect of scope, it will not present itself as an automatic decision but rather as a suggestion (ex: "Workload looks unbalanced- consider reassigning tasks.)

Table 27: Core Screens and Primary User Actions

Screen	Main Purpose	Key Actions Supported
Login	Secure user access	Log in, validation errors, session recovery
Dashboard	Project overview	View deadlines, assigned tasks, quick navigation
Tasks (Board/List)	Task tracking	Create/edit tasks, assign members, filter/sort
Task Details	Task collaboration	Comment, update fields, view history, log time
Milestones/Calendar	Deadline visibility	View milestones/due dates, open linked tasks
Team/Members	Team structure	View roles and members (manage if role permits)
Chat	Communication	Send messages, reply, view unread updates
Time Tracking/Timesheets	Effort visibility	Start/stop timer, manual entry, weekly summary
Analytics	Contribution summaries	View trends, workload distribution, recommendations (if enabled)

4.1.3 Usability and accessibility requirements

We will ensure the UI is readable, predictable, and accessible:

- Conspicuous font and standardized labelling of buttons (e.g. Create Task, Save, Submit) [6].

- Colour-based feedback entails text labels (not colour-only meaning). [10].
- Keyboard support of key flows (tab order and visible focus state). [10].
- Icon-only actions (e.g. Notifications, Edit task) have tooltips. [10].
- Clear loading indicators and success feedback (e.g., “Task updated successfully.”). [10]
- Clear error feedback for inputs (e.g., “Due date cannot be in the past.”, “Title is required.”), [10]

Table 28: UI Feedback and Error Message Examples

Scenario	Feedback Type	Example Message
Wrong login credentials	Inline error	“Incorrect email or password.”
Session expired	System prompt	“Session expired, please log in again.”
Missing required field	Inline validation	“Title is required.”
Invalid date	Inline validation	“Due date cannot be in the past.”
Invalid time log	Inline validation	“End time cannot be earlier than start time.”
Save successful	Toast notification	“Changes saved successfully.”
Network issue	Banner/toast	“Something went wrong. Please check your connection and try again.”

Table 29: Example user navigation flow

Screen	Purpose	Trigger	Next step
Login	Authenticate the user	Open Team Track URL	Redirect to Dashboard on success
Dashboard	Show overview of project status	Successful login or menu click	Navigate to Tasks, Chat, Time Tracking, Analytics
Tasks (Board/List)	Create/update tasks and track progress	Click “Tasks”	Open task details and edit fields

Task Details	View task info and collaborate	Click a task card/row	Save changes, comment, update status, log time
Milestones/Calendar	View deadlines and milestone plan	Click “Milestones/Calendar”	Check due dates, adjust planning
Chat	Communicate with team members	Click “Chat”	Post messages and respond
Time Tracking/Timesheets	Track time and view summaries	Click “Time Tracking”	Start timer, submit entries, view weekly totals
Analytics	Review contribution summaries	Click “Analytics”	View trends and participation indicators

4.2 Hardware Interfaces

TeamTrack does not have any hardware interfaces that are specialised. Team Track will be provided as a web-based browser app, which can be executed on usual desktop and laptop computers with the use of a typical input device (keyboard, mouse/trackpad). One needs a consistent internet connection in order to use the system and features like updates of tasks, messaging, and analytics views. Tablets might be capable of connecting to simple functions, but as we will be maximizing the interface on devices that have physical keyboards, they can support the use of texts and task management. It does not need any external devices, drivers, plug-ins or explicit hardware components.

4.3 Software Interface Requirements

Team Track will incorporate several software elements that are collaborative to provide the platform. We will make interfaces modular in order to allow the system to be maintained and extended without having to redesign significant portions of it. [9], [10].

4.3.1 Backend API interface (Python)

The backend will be implemented with Python (Django) and visible functionality with Django Rest Framework (DRF). The frontend will make calls to backend endpoints to: [9], [10].

- authenticate user and verify sessions [9].
- create/read/ update/ delete tasks and milestones [9], [10].
- keep comment and project activity logs [9], [10].

- post/ retrieve time logs, timesheet summaries [9], [10].
- get dashboard sums and analytics results [9], [10].

4.3.2 Authentication interface

We shall promote secure user authentication in one of the following ways (depending on the final implementation decision): [11].

- JWT authentication (e.g. SimpleJWT) and access to API by a token, or [11].
- Session-based authentication in case the UI is closely bound to Django templates [9].

On secured activities, we shall need to authenticate using valid credentials (token or session) before project data are returned. [11].

4.3.3 Database interface

The backend will be linked to a relational database where the records about the system will be stored as:

- users and roles
- projects/workspaces and memberships.
- activities, sub activities, remarks, and work-in-progress.
- milestones and due dates
- summaries of time sheets and time logs.
- analytics report and activity logs.

To control conformity in the way we operate the database and to minimize errors, we will apply an ORM-based interface (Django ORM).

4.3.4 Optional external interfaces (scope-dependent)

Included within our project scope, Team Track could interface with:

- email service (recovery of the password, verification, notification)
- file storage (attachments, exported reports)
- advanced AI insight module (beyond basic summaries)

When these are not put in place in the present milestone, then we will refer to them as proposed improvements and not existing functionality.

Table 30: Key Software Interfaces

Interface	Direction	Data Type	Purpose
Browser UI ↔ Backend API	Two-way	JSON over HTTPS	Tasks, chat, milestones, time logs, analytics

Backend API ↔ Database	Two-way	SQL via ORM	Persist and retrieve system records
Backend API ↔ Auth service	Two-way	Tokens/sessions	Login, session validation, access control
Backend API ↔ Email service	Outbound	Email payloads	Password reset, verification, alerts
Backend API ↔ File storage	Two-way	File + metadata	Attachments and exports
Backend API ↔ Advanced AI	Two-way	Structured I/O	Enhanced recommendations (future)

4.3.5 Software interface summary table

Component	Purpose	Technology / Interface
Web Client (Browser UI)	User interaction	HTTPS requests, JSON payloads
Backend API	Business logic and validation	Django + DRF endpoints
Authentication	Secure access control	JWT (SimpleJWT) or Django sessions
Database	Persistent storage	Relational DB via Django ORM (PostgreSQL/SQLite)
Time Tracking Module	Time logs and timesheets	Internal service/module (core)
Analytics Module	Contribution summaries	Internal service/module (core)
Optional Email Service	User notifications/reset	SMTP/API service (scope-based)
Optional File Service	Attachments/exports	Storage API (scope-based)

Table 31: Software Interface Summary table

4.4 Communication Interface Requirements

Team Track will employ safe communication technologies to secure data and guaranteed interactions among the user interface and the backend services. [9].

4.4.1 Client-server communication

- All communications will be on HTTPS. [9]
- The system will be based on request/response in which API requests issued by the browser and returned in the form of JSON. [9], [10].
- Standard HTTP methods will be used: GET (retrieve), POST (create), Patch/PUT (update), DELETE (remove) will be used. [9]

4.4.2 Data formats and response handling

- JSON will be used as the main data transfer format. [10].
- The server will respond with common status codes 200/201 success, 400 invalid request, 401/403 unauthorised, 404 not found, 500 server error. [9].
- The response to errors will contain understandable messages to allow the UI to provide user-friendly displays (e.g., "You are not allowed to edit this task.).

4.4.3 Authentication in communication

For protected requests, we will ensure the user's credentials are included: [11]

- JWT method: token sent via an Authorization header [11].
- Session approach: secure authenticated session cookie communicated with the use of HTTPS [9].

In case of failed authentication, we will give a plain response to user and take the user back to the login point where necessary. [11].

4.4.4 Reliability and failure handling

When the network or server malfunctions in carrying out vital functions (saving work, making a post, recording time) we will: [9].

- show a vivid message like something has gone wrong. You have to check your connection and reset it again. [9]
- avoid so-called silent failures by displaying the unsaved changes [9].
- allow non-destructive action safe retry (when it is safe) [9].

4.4.5 Real time communication

In the first release we will make the chat and notifications near-real-time. In case of need of live real-time updates, we can adopt: [9].

- Live updates on WebSocket's, or [9].

- polling/periodic refresh as a less complex initial way [9].

In case the current milestone does not have full WebSocket real-time capabilities, we will list it as a future enhancement.

5. Non-Functional Requirements

Non-Functional Requirements (TIS REPORT)

5.0.1 Non-Functional Requirements.

Non-Functional Requirements determine the quality features, limitations, and working environment of the system. These needs give directions of how the system will work and not what it will do. They make the system secure, efficient and reliable and at the same time scalable and maintainable as its use and functionality increases. The non-functional requirements mentioned in this section are based on the SRS and enhanced to meet events that are to be put in place as discussed later in this report.

5.0.2 Performance Requirements

We will design the system to provide reasonable response times in the normal operating conditions and supporting several users at once. Performance requirements are found to leave user interactions, API requests and system responses responsive and stable to the normal usage conditions. They are required to optimise API, database design, and backends architecture to avoid performance bottlenecks [3].

Requirement ID Description:

NFR-1 We will ensure timely responses to standard user requests

NFR-2 We will support concurrent user access without significant performance degradation
Meeting these requirements helps maintain a smooth user experience and prevents delays during critical system interactions.

5.0.3 Security Requirements

The non-functional requirement of the system is security. Several security controls will be applied to secure system data, deny unauthorised access, and to secure communication between system components. The security will be implemented throughout the frontend and backend as well as AI services [2].

Requirement ID Description:

NFR-3 We will encrypt data transmitted between client and server

NFR-4 We will enforce role-based access control

NFR-5 We will validate user input to reduce security risks These measures protect data confidentiality, integrity, and system access, reducing the risk of data breaches, misuse, and malicious activity.

5.0.4 Scalability Requirements

We will build the system in such a way that it will accommodate the future expansion in the number of users, data that is stored and the functionality of the system without necessitating any significant change in the architecture. The problems of scalability determine the strategy of deployment, the modularity of the system, and the choice of technology [3].

Requirement ID Description:

NFR-6 We will support scalable system deployment.

NFR-7 We will use modular system components to enable future expansion.

These requirements ensure that the system can evolve over time to accommodate increased demand and new features.

5.0.5 Reliability and Maintainability

We will make sure that the system will have a consistent functioning in various environments and that it will be maintainable to last its lifecycle. The requirements of reliability and maintainability facilitate stability of the systems, simplicity of updates, and sustainability.

Requirement ID Description:

NFR-8 We will maintain consistent system behaviour across environments.

NFR-9 We will support maintainable and modular system design.

These requirements reduce system downtime, simplify debugging and updates, and ensure that future modifications can be implemented with minimal disruption.

5.1 Security Requirements

This section defines the security requirements for the system. The requirements focus on protecting user accounts, data on projects, and system functionality from unauthorised access or misuse.

5.5.1 Authentication and Access Control

NFR 5.1.1.1: The system shall require authenticated access for all users before permitting access.

This NFR ensures that only registered users are able to access projects, tasks, and collaborate with members in the system.

NFR 5.1.1.2: The system shall ensure that user authentication credentials are securely stored and not retained in plain-text form.

This reduces the risk of exposed credentials and violation of confidentiality, moreover, reduces this risk of unauthorised access to stored data.

NFR 5.1.1.3: The system shall restrict access to project data to authorised project members only.

This prevents users from viewing or interacting with projects they aren't assigned to and keeps things simple.

NFR 5.1.1.4: The system shall enforce role-based access control to limit actions and visibility based on user roles.

Gives access to project leaders and project members according to the features relative to their role (least privilege).

5.1.2 Data Protection and Activity Control

NFR 5.1.2.1: The system shall protect sensitive user and project data during storage and during transmission *in deployment environments*.

This requirement supports protecting confidentiality and integrity of stored and transmitted data.

NFR 5.1.2.2: The system shall log security-related events such as authentication attempts and role changes.

These logs are for basic monitoring and accountability withing the system.

NFR 5.1.2.3: The system shall ensure that time logs can only be created or modified by authorised project members and project leaders.

This ensured that time tracking data stays reliable while allowing flexibility for contributors and leaders to make necessary corrections.

5.1.3 Session and Input Security

NFR 5.1.3.1: The system shall expire user sessions after a period of inactivity and require reauthentication.

Reduces the risk of unauthorised access from unattended user sessions.

NFR 5.1.3.2: The system shall validate and sanitise all user inputs to reduce common web security risks.

This helps prevent any invalid, inconsistent or malicious input from affecting the systems behaviour.

5.2 Performance Requirements

This section outlines the performance requirements for the Team Track system. These requirements focus on ensuring that the system remains responsive, reliable, accessible and usable under normal operating conditions.

5.2.1 System Responsiveness and Load Handling

NFR 5.2.1.1: The system shall respond to user actions (loading page, updating tasks, sending messages) within 2 seconds under normal operation.

This ensures that users experience minimal delay when interacting with the system during regular usage.

NFR 5.2.1.2: User authentication actions including login and logout shall complete within two seconds of submission.

Fast authentication improves usability and avoids delays during access to the system.

NFR 5.2.1.3: The system shall be able to support at least 30 active users at the same time without noticeable issues.

This requirement ensures acceptable performance for a typical university group project usage.

NFR 5.2.1.4: The system shall maintain stable performance without significant degradation when handling simultaneous operations across multiple modules.

5.2.2 Feature-Specific Performance

NFR 5.2.2.1: Messages sent through the Team Chat feature shall appear for the other project members within 1–2 seconds of being sent.

This supports fast and efficient communication between project members.

NFR 5.2.2.2: Changes made to tasks and time logs shall be reflected in the system interface within 1 second of submission or page refresh.

This ensures that updates are not delayed, reducing confusion or any duplicate actions.

NFR 5.2.2.3: The Contribution Analytics Dashboard shall load and display summary information within 3 seconds or page refresh for a project.

This allows users to see contribution insights without significant waiting time.

5.2.3 Scalability and AI Performance

NFR 5.2.3.1: The system shall continue to perform reliably even as the amount of stored data increases.

This requirement supports usability as projects, tasks, and logs accumulate over time.

NFR 5.2.3.2: AI-generated summaries shall be returned within a reasonable response time under normal usage conditions.

This ensures that AI-assisted features remain practical and useful within the system workflow.

5.3 Usability Requirements

This section defines usability requirements for Team Track. The requirements aim to make the system easy to learn, user friendly, and visually consistent, allowing users to efficiently complete their tasks without any complexity.

5.3.1 Interface Design and Consistency

NFR 5.3.1.1: The system shall provide a clear and simple user interface that allows users to understand and navigate the main features simply.

This helps reduce the complexity and having to learn the platform for new users.

NFR 5.3.1.2: The system shall have a consistent layout and format including visuals, colours, and terminology across all the pages.

Consistency improves usability and reduces user confusion.

5.3.2 Task Efficiency and Feedback

NFR 5.3.2.1: The system shall allow users to complete tasks such as creating tasks, logging time, and sending messages within a few steps.

This ensures that common actions can be completed quickly and efficiently.

NFR 5.3.2.2: The system shall provide clear feedback messages for user actions, including successful submissions or any errors.

Feedback helps users understand the outcome of their actions and correct mistakes when needed.

NFR 5.3.2.3: The system shall minimise the number of steps required to perform frequent actions such as task updates, time logging, and communication.

5.3.3 Accessibility and Readability

NFR 5.3.3.1: The system shall function on typical desktop and laptop displays.

This ensures compatibility with commonly used devices in an academic environment.

NFR 5.3.3.2: The system shall display all readable text and appropriately sized interface features.

This supports readability and overall ease of use.

5.4 Availability and Reliability Requirements

This section allows the availability and reliability requirements of the system to be defined to ensure consistent operation, fault tolerance, and dependable access for the users. These requirements are critical to ensuring trust from the user and ensuring that system services are available during normal use and during peak academic use.

5.4.1 Availability Requirements

NFR 5.4.1.1: The system will be accessible at least 99 percent of the time without taking into consideration the planned maintenance.

This is necessary so that the system is available most of the time to facilitate its constant use on academic life. Scheduled downtimes are not factored in ensuring there is space of updating and enhancing the system without compromising the availability requirements.

NFR 5.4.1.2: Scheduled maintenance actions will be announced in advance (more than 24 hours).

Prior information of the maintenance activities assists the users to plan their own working schedule and minimize interference with the existing work and project schedules.

NFR 5.4.1.3: The system will enable graceful degradation which ensures that core features are made available when the system is partially unavailable.

This is necessary to guarantee that the most vital functions of the system are maintained when non-critical components fail hence reduction of service disruption.

5.4.2 Reliability Requirements

NFR 5.4.2.1: The system will be able to withstand unforeseen failures in less than 5 minutes without any data loss.

Failure recovery is necessary within just a few seconds to maintain service continuity and avert data loss to users in unforeseen system failure.

NFR 5.4.2.2: The system should have automated data backups at least twice in 24 hours.

Frequent automated backups minimize the chances of loss of data permanently and the likelihood of effective restoration of the system in those cases where systems fail or the data is corrupted.

NFR 5.4.2.3: The system will record system failures and critical mistakes to assist in diagnostics and recovery.

System logging facilitates efficient monitoring, troubleshooting and analysis of incident afterward and it helps in the overall maintenance and reliability of the system.

These requirements align with internationally recognized **software quality attributes**, which emphasize **availability** and **reliability** as core characteristics of dependable software systems [12].

NFR 5.4.2.4: The system shall ensure that no user data is lost during unexpected failures by maintaining transactional consistency.

5.5 Data Integrity and Privacy

This section specifies the requirements related to protecting user data from unauthorized access, modification, or loss, while ensuring compliance with data protection and privacy principles. Given that the system handles user accounts, group data, and task-related information, maintaining **data integrity** and **data privacy** is essential.

5.5.1 Data Integrity Requirements

NFR 5.5.1.1 The system must also guarantee that the information held in the system is precise, comprehensive, and congruent throughout the life of the system.

This is required to ensure that user and project data are not involuntarily distorted, deformed, or divergent in their state during storage, processing, and retrieval.

NFR 5.5.1.2: The system will have access controls that will set it to follow that user and project data are not modified by unauthorized individuals.

Only authorized users can get access to data manipulation privilege, hence the probability of possible errors, like intentional or malicious manipulation of data, is reduced.

NFR 5.5.1.3: The system will be capable of verifying all the user inputs to reduce any chances of data corruption or inconsistency.

Input validation helps in ensuring that only well-structured and good data is processed and stored via the system, as it improves the quality of the overall data.

5.5.2 Data Privacy Requirements

NFR 5.5.2.1: The system will provide usage controls on personal and project-specific data defined by user-identity and permissions.

Role-based access helps in providing privacy and confidentiality because the users can only see or edit data that is within their duties.

NFR 5.5.2.2: Authentication details of the user will be stored in an encrypted or hashed form in the system.

Storing the credentials in a secure way minimizes the chances of credential breach and unauthorized access to the system.

NFR 5.5.2.3: The system will process the personal data by complying with the principles of data protection and privacy, such as minimization and limitation of purposes. This is in accordance with the need to only collect and process a limited amount of personal data that is required, and only for the genuine use of the legitimate system.

Such requirements are guided by data protection regulations and best practices that are acceptable, such as the General Data Protection Regulation (GDPR), in which the ultimate importance is given to lawful, secure, and transparent data processing [13].

This section is a summary of the ethical and responsible usage requirements in regards to any intelligent or automated capabilities that would be part of the system. Although the system is not

based on advanced principles of artificial intelligence in decision-making, the ethical principles of AI are used to be fair, transparent and responsible in behavior of the system.

5.6 Ethical and Responsible Use Requirements

NFR 5.6.1 Stated in such a way that the system should not automatically make decisions that affect users adversely in the absence of human supervision.

This is important so that the users are held responsible for making significant actions, and automated capabilities would not forcefully deliver undesirable results alone.

NFR 5.6.2: The system will provide transparency to automated recommendations or system-generated insights that are available to users.

Transparency gives knowledge to users on how recommendations or outputs are produced, which enhances confidence in the system's behavior.

NFR 5.6.3: The system will not have any biased behavior; resources will be made to use the same logic for all users.

The even-handed treatment will aid in avoiding unreasonable treatment and unexpected bias in automated system behavior.

NFR 5.6.4: Where it is appropriate, the system will ensure that the user has a clear explanation of how the system-generated output is obtained.

The explanations enhance the understanding of users and serve to encourage responsible usage of automated functions.

NFR 5.6.5: The system shall provide users with the ability to review and override AI-generated suggestions before any action is applied.

Those requirements meet ethical principles of AI as defined on an international level, where intelligent systems must focus on transparency, ethical accountability, and be human-centered [14], [15].

6. Data Requirements

6.1 Data Entities Description

6.1.1. User

Description:

Represents a person registered on the Team Track platform. Users are able to be involved in projects, assigned tasks, track time, and interact with other team members. Every user has a role assigned to it which determines their level of access and permissions.

Key Attributes:

Attribute Name	Description
user_id	Uniquely identifies a registered user
name	Full name of the user
role	Role of the user (member/ leader)
email	Email address used for identification and login
account_status	Indicates whether the account is active, inactive, or suspended
created_at	Date and time when the user account was created

Table 32: Key Attributes User Table

6.1.2. Project

Description:

Project entity represents a group project created by users. A project contains tasks, team members, activity logs, and analytics data.

Key Attributes:

Attribute Name	Description
project_id	Unique identifier for each project
project_name	Name of the project
description	Short description of the project as defined by user
start_date	Project start date
end_date	Project end date
created_by	Identifier of the user who created the project

Table 33: Key Attributes Project Table

6.1.3. Task

Description:

Refers to a single work unit of a project. Tasks are allocated to users and monitored on progress and time taken on each. Every task also has a status and a deadline to help track progress and for ensuring its completion on time, and are linked to time logs and activity records to track effort and analytics.

Key Attributes:

Attribute Name	Description
task_id	Unique identifier for each task
assigned_user_id	User assigned to complete the task
project_id	Project to which the task belongs
title	Short title describing the task
description	Detailed description of the task
status	Current task state (e.g. pending, in progress, completed)
due_date	Deadline for task completion

Table 34: Key Attributes Task Table

6.1.4. Time Log

Description:

The amount of time a user spends on particular task is recorded as a Time Log. Capture when work starts and ends. Time logs are used for effort tracking and contribution analytics.

Key Attributes:

Attribute Name	Description
timelog_id	Unique identifier for each time log entry
user_id	User who recorded the time
task_id	Which task the time log is associated with
start_time	Time when work on the task started
end_time	Time when work on the task ended
duration	Total time spent on the task

Table 35: Key Attributes Time Log Table

6.1.5. ActivityLog

Description:

Represents a system generated record of meaningful user activities such as task updates, status change, and project changes. It tracks changes made in the projects in the past.

Key Attributes:

Attribute Name	Description
activity_id	Unique identifier for each activity record
user_id	User who performed the action
project_id	Project in which the action occurred
action_type	Type of action performed (example: task update)
timestamp	Date and time when the action occurred

Table 36 : Key Attributes Activity Log Table

6.1.6. Message

Description:

Represents a communication message exchanged between project members within the team chat module.

Key Attributes:

Attribute Name	Description
message_id	Unique identifier for each message
project_id	Project where the message was sent
sender_id	User who sent the message
content	Text content of the message
timestamp	Date and time when the message was sent

Table 37: Key Attributes Message Table

6.1.7. Notification

Description:

Represents system generated alerts to alert users of significant events like a deadline or task allocation.

Key Attributes:

Attribute Name	Description
notification_id	Unique identifier for each notification
user_id	User receiving the notification
notification_type	Type of notification (example: deadline reminder)
is_read	Shows whether the notification has been viewed
timestamp	Date and time when the notification was generated

Table 38: Key Attributes Notification table

6.1.8. Project Member

Description:

Represents the membership relationship between a user and a project. It contains information on who the users are on what projects, and can hold role information on the project.

Key Attributes:

Attribute Name	Description
project_id	Project associated with the membership
user_id	User associated with the project
role_in_project	Role of the user within the specific project

Table 39: Key Attributes Project Member table

6.2 Data Relationships

This part explains the connection between the core data entities in TeamTrack, and the expected cardinality of each relationship between them.

User-Project (Membership), (M:N)

A user can be a member of multiple projects and a Project can have multiple users. This relationship is project membership and shows importance of roles in a project.

Project-Task (Contains), (1:N)

A project can contain many tasks, and each task belongs to exactly one project. This ensures that the tasks are always tracked and contributing in the right project.

User-Task (Assigned To), (1:N)

A user can be assigned to many tasks, but each task is assigned to one user.

Task-TimeLog (Has Time Logs), (1:N)

A task can have many time log entries each time a log entry is made by one user. This links the contributor and the time spent on a task.

Project-ActivityLog (Has Activity Records), (1:N)

A project can have many activity log entries, and each activity log entry belongs to one project. Gives a history of the activities and modifications in the project as an account.

User-ActivityLog (Performs Action), (1:N)

A user can generate many activity log entries by actions like creating tasks, or updating or modifying project details. Each activity log entry is linked to the user who performed the action.

Project-Message (Contains Messages), (1:N)

A project can contain many chat messages, and each message belongs to one project. This keeps communication related to the relevant project.

User-Message (Sends),(1:N)

A user can send many messages and each message has one sender user.

User-Notification (Receives), (1:N)

A user can receive many notifications, and each notification is linked to one user.

The data relationships are illustrated in Appendix B.

6.3 Data Storage and Retention

In this section, logical requirements of the TeamTrack database are defined, including the way data is stored, secured, and finally put out of service.

6.3.1 Logical Database Requirements

Persistence Layer: The system shall utilize a Relational Database Management System (RDBMS) to ensure data persistence and support the complex \$1:N\$ and \$M:N\$ relationships defined in Section 6.2.

The **integrity and constraints of the data obtainable from the system** can be expressed as follows:

- **Referential Integrity:** The system shall enforce foreign key constraints to ensure that no Task or TimeLog exists without a corresponding Project or User.
- **Atomic Transactions:** All updates to Time Log and Activity Log must be atomic to ensure that partial logs do not occur during system or network failures.

Access Capabilities: Data access shall be restricted at the database level using the role attribute defined in Section 6.1, ensuring that "Instructors" have read-only views while "Team Leaders" have write access to project metadata.

6.3.2 Data Retention and Archiving

In line with the university project life cycles and the requirement of Contribution Analytics, the system will follow the following rules of retention:

- **Active Retention:** All data related to active projects (Users, Tasks, Time Logs, and Messages) shall be retained in primary storage for the duration of the academic semester.
- **Archival Policy:** Upon project completion or the end of the semester, project data shall be moved to an "Archived" state. Data stored in archives will last at least two academic years so that they can be accessed in case of audit of grades or student portfolio, and then transferred to cold storage.
- **Activity Log Lifespan:** To support the AI-Assisted Insights feature, granular Activity Log entries must be retained in an accessible format for the entire duration of the project to allow the machine-learning logic to analyse historical progress patterns.
- **Notification Purging:** To optimize storage performance, Notification records with an `is_read` status set to "True" shall be automatically purged from the database after 90 days.

6.3.3 Data Privacy and Disposal

- **Personally Identifiable Information (PII):** In alignment with the Ethical and Responsible AI Considerations (Section 5.6), if a user deletes their account, the system shall scrub PII from the User entity within 30 days.
- **Anonymization:** To preserve the accuracy of the Contribution Analytics Dashboard for remaining team members, any TimeLog or ActivityLog associated with a deleted user shall be anonymized (linked to a "Deleted User" system ID) rather than permanently removed.

- **Backups:** The system shall perform incremental backups every 24 hours and a full database backup weekly. Such backups will be stored in an encrypted off-site cloud storage.

7. Appendices

7.1 Appendix A: Proposed Technology Stack and Deployment Overview (Informative)

This appendix gives a representative technology stack and deployment strategy of TeamTrack. It is added to explain the way how the external interfaces of the system (browser access, HTTPS communication, authentication, and API integration) can be facilitated in deployment. Implementation can be perfected in subsequent milestones.

Table 40: Proposed Technology Stack Summary

Layer / Component	Proposed Technology	Purpose in TeamTrack
Client Access	Modern web browsers (Chrome, Edge, Firefox, Safari)	Access Team Track through a browser based UI
Frontend UI	React (Vite)	User interface for tasks, chat, milestones, analytics, and time tracking
UI Styling	Tailwind CSS + component library (e.g., shadcn/ui)	Consistent design system and reusable UI components
Data Visualization	Recharts (or equivalent)	Charts for contribution analytics and progress summaries
Backend API (Python)	Django + Django REST Framework (DRF)	REST API endpoints and core business logic
Authentication	JWT (access/refresh)	Secure token based access to protected endpoints
Database	PostgreSQL	Persistent storage for users, projects, tasks, time logs, and messages

ORM	Django ORM	Structured model-based access to relational data
Reverse Proxy	Nginx	Routes requests to frontend and /Api to backend; enforces HTTPS
HTTPS/TLS	Let's Encrypt (Certbot)	Encrypts all client-server communication
Containerization (optional)	Docker + Docker Compose	Repeatable deployments of backend, database, and services
Background Jobs (optional)	Celery + Redis	Async tasks such as analytics generation and scheduled summaries
Real-time Updates (optional)	WebSocket's (or polling fallback)	Live chat/notifications if enabled; polling as initial approach
Source Control	Git + GitHub	Version control and team collaboration workflow
CI/CD (optional)	GitHub Actions	Automated testing and deployment pipeline
API Testing	Postman / Hopscotch	Testing authentication, task, chat, time logging endpoints

7.2 Appendix B – Entity Relationship Diagram (ERD)

This appendix presents the Entity Relationship Diagram (ERD) for the TeamTrack system. The diagram illustrates the core data entities and the relationships between them as described in section 6.

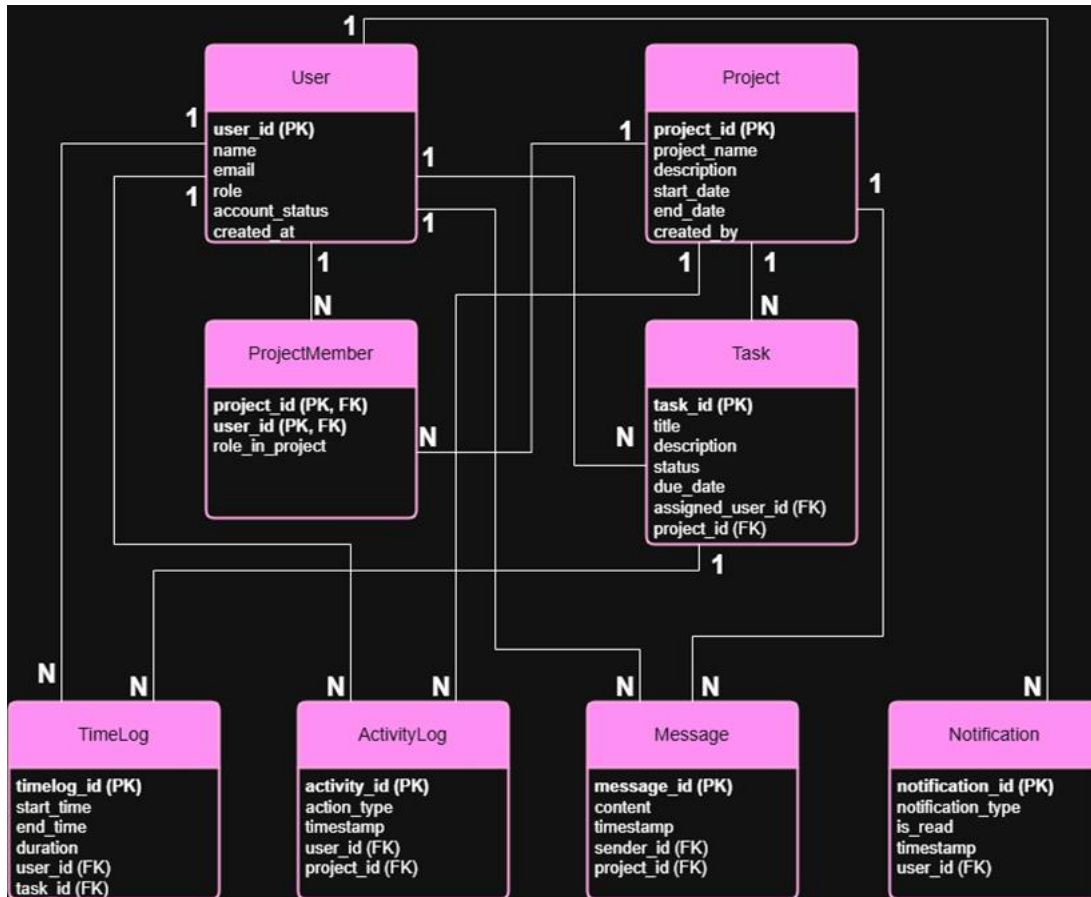


Figure 1: Entity Relationship Diagram for the TeamTrack system

7.3 Appendix C - Use Case Diagram

This appendix shows the Use Case Diagram for the TeamTrack system. The diagram shows the main actors interacting with the system and the core functionalities for each role. It provides a high-level view of how Project Members and Leaders interact with the system to perform various tasks.

The use case diagram is intended to support the functional requirements described in section 3 by visualising user interactions with the system.

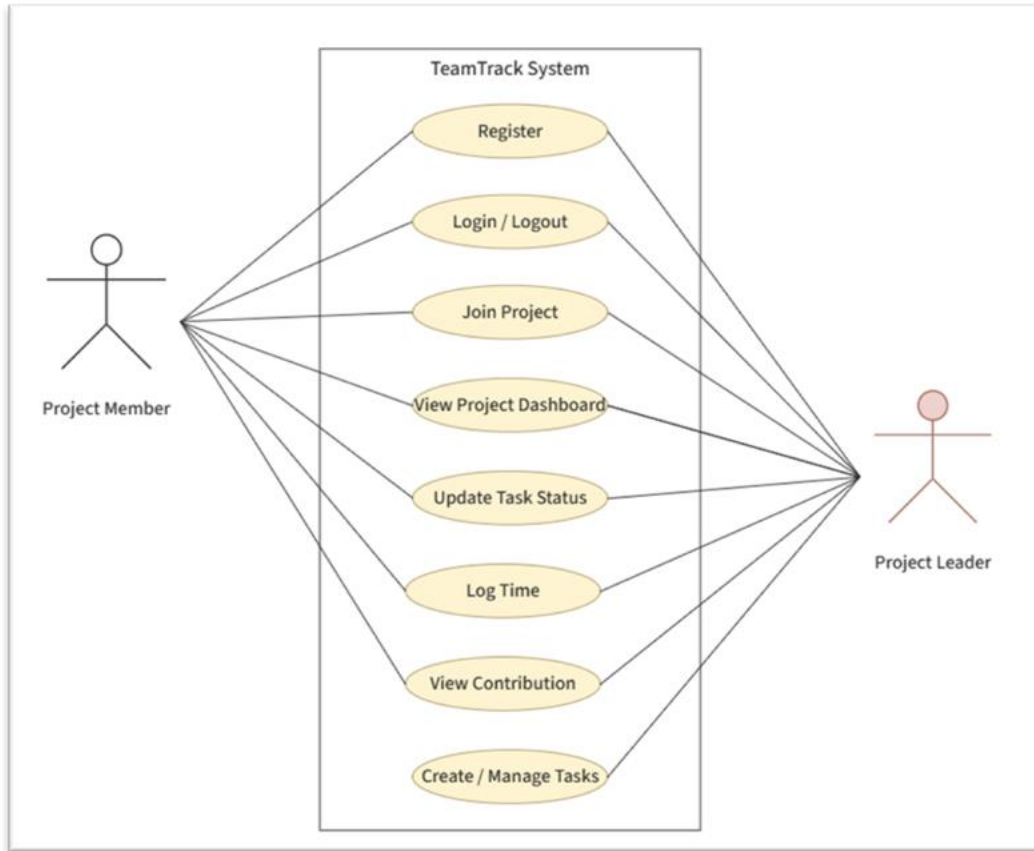


Figure 2: Use Case Diagram for the TeamTrack System

7.4 Appendix D – Glossary

AI-Assisted Insights: Automatically suggested best practices from time spent working on tasks, and logging of work hours.

API (Application Programming Interface): A set of rules which enable different software to interact with each other, such as frontend and backend.

CI/CD (Continuous Integration / Continuous Deployment): Development practices that automatically test and deploy changes to software.

Data Entity: A structured representation of stored information in the system, such as users, projects, tasks, or time logs.

ERD (Entity Relationship Diagram): A diagram used to visually represent data entities within a process and how they interact with one another.

FR (Functional Requirement): A requirement specifying what the system must do or how it should behave.

NFR (Non-Functional Requirement): A requirement that specifies the quality characteristics of a system (e.g. performance, security, usability).

JWT (JSON Web Token): A secure, tokenized way of authenticating users and granting them access to secured system resources.

Kanban Board: A visual task management approach where tasks are displayed in columns representing workflow stages.

RBAC (Role-Based Access Control): An access control method in which the permissions of users are specified based on their role within the system.

REST API: A web service architecture that employs standard HTTP methods (GET, POST, PUT, DELETE) to facilitate data exchange between systems.

Time Log: A record of time spent by a user on a specific task, used to support effort tracking and contribution analysis.

Timesheet: A summary view of logged time, typically grouped by user and time period.

7.5 Appendix E – Software Quality Attributes

Table 41: Software Quality Attributes Table

Quality Attribute	Definition (Meaning)	Importance in TeamTrack	System Design Approach / Implementation Strategy	Measurable Requirement / Example Metric
Usability	The degree to which the system is easy to learn, navigate, and operate for users.	TeamTrack is used by students, so the interface must remain simple and intuitive without technical complexity.	Clean UI layout, consistent navigation, Kanban boards, dashboards, clear task actions.	Users should complete task creation within < 1 minute without training.
Accessibility	The ability of the system to support users with different needs or disabilities.	Ensures inclusivity for university environments with diverse	Responsive design, readable fonts, high contrast mode, keyboard	Meets WCAG basic compliance (e.g., readable text, proper labels).

		student populations.	navigation support.	
Performance	The system's responsiveness and efficiency under normal workload.	Students expect fast interaction when logging tasks, time, or messages.	Optimized backend queries, caching dashboards, efficient REST API calls.	Dashboard loads within < 2 seconds for typical project size.
Scalability	The ability of the system to handle growth in users, projects, or activity logs.	Future expansion may include multiple teams, instructors, or departments.	Modular architecture, database indexing, scalable hosting.	Supports at least 200 concurrent users without degradation.
Reliability	The probability that the system functions correctly without failure.	TeamTrack must not lose contribution or time tracking data.	Transaction-safe database operations, error recovery, validation checks.	System uptime of ≥ 99% during semester usage.
Availability	The degree to which the system remains accessible when required.	Students may work late nights, weekends, and near deadlines.	Cloud deployment with monitoring, failover support.	System accessible 24/7 with minimal downtime.
Security	Protection against unauthorized access and data breaches.	Contribution analytics and academic records must remain private and role-protected.	RBAC enforcement, JWT authentication, encrypted passwords, secure sessions.	Unauthorized users cannot access workspace data (0 tolerance) .
Privacy	Ensuring user data is stored and handled ethically.	TeamTrack stores sensitive participation metrics which must not be misused.	Data minimization, restricted instructor view, no public exposure of logs.	Logs visible only to authorized members and supervisors.

Maintainability	Ease of fixing bugs, updating features, and improving the system.	TeamTrack is a Capstone project and may evolve beyond MVP.	Modular Django apps, clean API structure, documented code.	New feature integration possible within one sprint cycle.
Extensibility	The ability to add new modules or enhancements in future releases.	Future scope includes file uploads, AI predictions, integrations.	Plugin-ready architecture, service separation, reusable components.	AI module can be upgraded without rewriting core features.
Portability	The ability of the system to run across environments and devices.	Students use Windows, macOS, and Linux systems.	Browser-based platform with cross-platform support.	Runs successfully on Chrome, Edge, Firefox, Safari.
Interoperability	Ability to integrate with external systems or APIs.	Future integration may involve LMS systems or cloud storage.	REST API architecture, optional Google Drive linking.	External file system integration supported in later releases.
Data Integrity	Ensuring stored data remains accurate, consistent, and not corrupted.	Time logs and contribution data must remain trustworthy.	Input validation, audit trails, frozen logs after grace period.	No duplicate or conflicting time entries allowed.
Traceability	Ability to track actions back to specific users and timestamps.	Critical for fairness in academic contribution evaluation.	Activity logs, task history, timestamped chat records.	Every task update stored with user + time reference.
Accountability Support	Ensuring effort distribution is transparent across members.	Core goal is solving the “free-rider	Contribution dashboards, workload imbalance	System highlights unusually low

		problem” in group projects.	detection, analytics export.	participation automatically.
Responsiveness (UI Feedback)	System provides immediate confirmation of user actions.	Students need clarity when tasks are updated or timers started.	Notifications, progress indicators, success/error messages.	Actions confirmed within < 0.5 sec response feedback.
Fault Tolerance	System continues functioning even if minor failures occur.	Prevents loss of work if connectivity drops mid-session.	Autosave drafts, retry mechanisms, graceful error pages.	No data loss during brief disconnect (<30s).
Ethical AI Support	AI recommendations must remain transparent and non-judgmental.	AI should not grade students or make unfair decisions.	Recommendation-only insights, explainable workload alerts.	AI provides suggestions, not automated scoring decisions.
Compliance with Academic Context	Suitability for university project workflows rather than enterprise workflows.	TeamTrack is designed specifically for Capstone group accountability.	Instructor read-only role, academic contribution focus, lightweight design.	Supports academic supervision without intrusive control.
Documentation Quality	System requirements and workflows must be clearly documented.	Essential for Capstone assessment and future maintenance.	SRS-based development, requirement traceability tables, Agile records.	All major modules mapped to FR/NFR requirements.

7.6 Appendix F – Agile Development

The development of the Team Track project has been created on the basis of the Agile software development approach, which should facilitate iteration-based delivery, constant stakeholder engagement, and structured monitoring of progress. Agile practices have been chosen as the needs of the system requirements have changed, and they need to release incremental functionality within the time limit of the Capstone development.

This section captures the Agile processes that were used during the project lifecycle, such as the interaction among stakeholders, the coordination of the sprint, the mechanism of tracking the progress, and the responsibility distribution among the team members.

7.6.1 Client Interaction and Feedback

The stakeholder involvement was upheld during the development process to make sure that the system requirements were in line with the desired academic collaboration goals.

7.6.2 Meeting Documentation

The client was formally engaged in meetings to review functional expectations, validate feature priorities, and validate project scope. Significant deliverables of these meetings were documented in organized meeting minutes, and they consisted of:

- Verified system objectives and deliverables.
- Clarification of the features (task management, time logging, analytics)
- Listed action items to be taken in further sprints.
- Revision Feedback on usability and applicability to academics.

The records of meetings were kept in the common project documentation repository so that they can be traced and be transparent.

7.6.3 Communication and Feedback Records

Client and supervisor feedback was also supported through formal email communication. Written correspondence was maintained to document requirement confirmations, milestone updates, and agreed deliverable expectations. This ensured that project decisions remained verifiable throughout the development lifecycle.

7.6.4 Internal Team Communication

Effective internal coordination was required to ensure consistent implementation across system modules and equitable workload distribution.

7.6.5 Sprint Planning Sessions

Sprint planning sessions were held at the beginning of each development iteration. During these sessions, the team:

- Reviewed the completion status of previous sprint objectives
- Defined sprint-level implementation targets
- Assigned responsibilities based on feature ownership
- Identified potential development risks and constraints

This process ensured that sprint deliverables remained measurable and aligned with the Software Requirements Specification.

7.6.6 Stand-Up Coordination

Short progress coordination discussions were conducted throughout development to identify blockers and synchronise implementation tasks. These discussions supported timely resolution of issues and maintained continuity between sprint milestones.

7.6.7 Progress Tracking and Schedule Control

Progress tracking was applied as a formal Agile control mechanism to ensure that project implementation remained consistent with planned delivery timelines.

7.6.8 Metrics and Monitoring

Project progress was monitored using defined tracking measures, including:

- Sprint milestone completion checklists
- Task allocation boards
- Time tracking logs for effort visibility
- Feature completion mapping against documented requirements

These measures ensured that system development remained verifiable and systematically controlled.

7.6.9 Milestone Tracking

Sprint milestones were compared to the planned schedules to ensure timely completion. Slippages in anticipated schedules were detected in time, and corrective measures were taken in time before the significant dates of deliveries.

7.6.10 Corrective Actions

Where deviations occurred, corrective actions were applied through Agile reprioritisation. Examples included:

- Reallocation of tasks between team members
- Prioritisation of core MVP requirements
- Deferral of optional enhancements (future scope features)

This ensured that the project remained feasible within the academic development constraints defined in the SRS.

7.6.11 Sprint Review and Reflection

At the conclusion of each sprint cycle, sprint reviews were conducted to evaluate:

- Completed deliverables
- Implementation challenges
- Coordination effectiveness
- Adjustments required for the next iteration

This reflective process supported continuous improvement and ensured that sprint planning remained evidence-driven.

7.6.12 Continuous Improvement Commitment

The Agile methodology supported ongoing refinement throughout the project lifecycle. Estimation accuracy and workload balancing improved over successive iterations through structured sprint evaluation and controlled task distribution.

7.7 Appendix G –Attributions

Table 42: Attribution Table

TASKS		TEAM MEMBERS				
Main Part	Sub Part	Omar Waheed Khan	Muhammad Hashir	Maria Waheed	Zernish Shahid Nazir	Mohammad Siraj Hussain

Introduction	Purpose					
	Report Structure and Formatting					
	Intended Audience					
	Scope of the System					
	Definitions, Acronyms, and Abbreviations					
	Competitor / Existing Tools Analysis					
	Proposed System Overview					
Overall Description	Product Perspective					
	Product Functions					
	User Classes and Characteristics					
	Operating Environment					
	Design and Implementation Constraints					
	Assumptions and Dependencies					
	System Workflow Overview					
System Features (Functional Requirements)	User Registration and Authentication					
	Role-Based Access Control					
	Project and Team Management					
	Task Management System					

	Time Tracking and Effort Logging					
	Team Chat and Communication Module					
	Contribution Analytics Dashboard					
	AI-Driven Project Assistant					
	Notifications and Reminder System					
	Activity Log and Audit Trail					
External Interface Requirements	User Interface Requirements					
	Software Interface Requirements					
	Communication Interface Requirements					
Non-Functional Requirements	Security Requirements					
	Performance Requirements					
	Usability Requirements					
	Availability and Reliability					
	Data Integrity and Privacy					
	Ethical and Responsible AI Considerations					
Data Requirements	Data Entities Description					
	Data Relationships					

	Data Storage and Retention					
Appendices	Purposed technology stack					
	Entity Relationship Diagram					
	User case diagram					
	Glossary					
	References					
	Agile Development					
	Meeting Minutes					
	toggl reports					
Formatting						

7.8 Appendix H – References

- [1] M. Paasivaara, S. Durasiewicz, and C. Lassenius, "Using Scrum in Distributed Agile Development: A Multiple Case Study," in *Proc. 4th IEEE Int. Conf. on Global Software Engineering (ICGSE)*, Limerick, Ireland, 2009, pp. 195–204. DOI: <https://doi.org/10.1109/ICGSE.2009.25>
- [2] R. M. Brooks and J. L. Ammons, "Free Riding in Group Projects and the Effects of Timing, Frequency, and Specificity of Criteria in Peer Assessments," *Journal of Education for Business*, vol. 78, no. 5, pp. 268–272, 2003. DOI: <https://doi.org/10.1080/08832320309598613>
- [3] S. C. Herring, "Computer-Mediated Communication on the Internet," in *Annual Review of Information Science and Technology*, vol. 36, no. 1, B. Cronin, Ed. Medford, NJ: Information Today Inc., 2002, pp. 109–168. DOI: <https://doi.org/10.1002/aris.1440360104>
- [4] P. Ralph *et al.*, "Pandemic Programming: How COVID-19 Affects Software Developers and How Their Organizations Can Help," *Empirical Software Engineering*, vol. 25, no. 6, pp. 4927–4961, 2020. DOI: <https://doi.org/10.1007/s10664-020-09875-y>
- [5] I. Sommerville, *Software Engineering*, 10th ed., Pearson Education, 2016.
- [6] *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.
- [7] B. Oakley, R. M. Felder, R. Brent, and I. Elhajj, "Turning student groups into effective teams," *Journal of Student-Centered Learning*, vol. 2, no. 1, pp. 9–34, 2004.

- [8] World Wide Web Consortium (W3C), "Architecture of the World Wide Web, Volume One," W3C Recommendation, Dec. 15, 2004. <https://www.w3.org/TR/webarch/>
- [9] R. Fielding, M. Nottingham, and J. Reschke, "HTTP Semantics," RFC 9110, IETF, Jun. 2022. <https://www.rfc-editor.org/rfc/rfc9110.html>
- [10] T. Bray, "The JavaScript Object Notation (JSON) Data Interchange Format," RFC 8259, IETF, Dec. 2017. <https://datatracker.ietf.org/doc/html/rfc8259>
- [11] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015. <https://datatracker.ietf.org/doc/html/rfc7519>
- [12] F. Febrero, C. Calero, and M. Á. Moraga, "Software reliability modeling based on ISO/IEC SQuaRE," *Information and Software Technology*, vol. 70, pp. 18–29, Feb. 2016. DOI: <https://doi.org/10.1016/j.infsof.2015.09.006>
- [13] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 (General Data Protection Regulation)," *Official Journal of the European Union*, 2016. <https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng>
- [14] B. D. Mittelstadt, P. Allo, M. Taddeo, S. Wachter, and L. Floridi, "The ethics of algorithms: Mapping the debate," *Big Data & Society*, vol. 3, no. 2, 2016. DOI: <https://doi.org/10.1177/2053951716679679>
- [15] L. Floridi *et al.*, "AI4People - An Ethical Framework for a Good AI Society: Opportunities, Risks, Principles, and Recommendations," *Minds and Machines*, vol. 28, no. 4, pp. 689–707, 2018. DOI: <https://doi.org/10.1007/s11023-018-9482-5>
- [16] FastAPI, "Security and Authentication," FastAPI Documentation, 2024. <https://fastapi.tiangolo.com/tutorial/security/>
- [17] OWASP Foundation, "OWASP Top Ten Web Application Security Risks," OWASP, 2023. <https://owasp.org/www-project-top-ten/>
- [18] PostgreSQL Global Development Group, "PostgreSQL Performance Tips," PostgreSQL, 2024. <https://www.postgresql.org/docs/current/performance-tips/>

7.9 Appendix I – Meeting Minutes

Meeting minutes are formal written documents that summarize the important discussions, decisions, and actions to be taken that were agreed upon by project meetings. They are a significant tool of communication and accountability throughout the software development lifecycle.

For the TeamTrack project, I led and maintained meeting minutes to make sure that all stakeholder interactions and internal team discussions were tracked and traceable. These minutes were used mostly to:

- Confirm and refine system requirements with the client
- Record feedback and scope adjustments discussed during meetings
- Track decisions related to feature prioritisation and sprint planning
- Assign action items and responsibilities to team members
- Maintain transparency and consistency across the development process

By recording the outcomes of meetings in an organised way, I ensured the project requirements were maintained in line with client expectations and all team members had a reference point of agreed objectives and next steps.

Meeting minutes were kept in the shared project documentation repository and referenced throughout Milestone 1 in support of the preparation of the Software Requirements Specification (SRS).

Table 43: Meeting Minutes Summary

Date	Description	Link
9/01/26	Deciding on idea	Link
13/01/26	Capstone Project Confirmation	Link
14/01/26	Work breakdown Meeting and Agile methods	Link
20/01/26	SRS Requirement Updates – From Client	Link

7.10 Appendix J – toggl reports

Table 44: Toggl Report Table

Member Name	Toggl Report (Link)
Muhammad Hashir	Link
Omar Waheed Khan	Link
Maria Waheed	Link
Zernish Shahid Nazir	Link
Mohammed Siraj Hussain	Link

Work Distribution Summary for Milestone 1

Ownership of the Software Requirements Specification (SRS) for Milestone 1 was divided according to section and subsection. The goal was to make sure each team member took responsibility for areas that matched their technical understanding, research focus, and expected development roles. Distribution followed the structure from the Milestone 1 Table of Contents, covering key parts like the Introduction, Overall Description, System Features (Functional Requirements), External Interface Requirements, Non-Functional Requirements, Data Requirements, and Appendices.

Defining system functionality, user interactions, and data structures relevant to TeamTrack was something each member contributed to. User roles, task management, system workflows, these elements were all addressed. The final SRS takes those individual contributions and consolidates them into one cohesive document. The structure stays the same throughout, but requirements have been refined for clarity, consistency, and alignment with what the project is trying to achieve.

Section-Wise Contribution Tables

1. Introduction

Section / Subsection	Primary Contributor	Contribution Details
1. Introduction	Muhammad Hashir, Omar Waheed Khan	Shared ownership of the Introduction section, with Hashir leading the opening subsections and Omar leading the analytical subsections.
1.1 Purpose	Muhammad Hashir	Drafted the purpose statement of the SRS, outlining its objectives, scope, and intended role in the project lifecycle.
1.2 Report Structure and Formatting	Muhammad Hashir	Wrote the document formatting standards, section numbering conventions, requirement identification format, writing guidelines, and use of visual aids.
1.2.1 Document Formatting Standards	Muhammad Hashir	Defined formatting rules for fonts, styles, and visual consistency across the document.

1.2.2 Section Numbering and Organization	Muhammad Hashir	Documented the section numbering scheme and organisational hierarchy of the SRS.
1.2.3 Requirement Representation and Identification	Muhammad Hashir	Wrote the requirement identification format and coding conventions.
1.2.4 Requirement Writing Guidelines	Muhammad Hashir	Documented the writing style guidelines for functional and non-functional requirements.
1.2.5 Use of Emphasis and Visual Aids	Muhammad Hashir	Described use of tables, diagrams, and emphasis conventions throughout the document.
1.3 Intended Audience	Muhammad Hashir	Identified and described the target audiences for the SRS, including developers, instructors, and clients.
1.4 Scope of the System	Muhammad Hashir	Defined the boundaries and scope of the TeamTrack system, including what is and is not included.
1.5 Definitions, Acronyms, and Abbreviations	Omar Waheed Khan	Compiled the full glossary of terms, definitions, and abbreviations used throughout the SRS.
1.5.1 Definitions	Omar Waheed Khan	Wrote and organised the definitions of domain-specific terms used in the document.
1.5.2 Acronyms and Abbreviations	Omar Waheed Khan	Listed and defined all acronyms and abbreviations referenced in the SRS.
1.6 Competitor / Existing Tools Analysis	Omar Waheed Khan	Conducted the analysis of commercial project management solutions and existing tools, including the comparative feature matrix and gap analysis.
1.6.1 Analysis of Commercial Project Management Solutions	Omar Waheed Khan	Reviewed and documented existing tools such as Trello, Jira, and Asana relative to TeamTrack's target context.

1.6.2 The Problem of Fragmented Workflows	Omar Waheed Khan	Documented the fragmented communication tool problem in academic group work contexts.
1.6.3 Comparative Feature Matrix	Omar Waheed Khan	Constructed the feature comparison table contrasting TeamTrack against existing solutions.
1.6.4 Conclusion and Gap Analysis	Omar Waheed Khan	Wrote the conclusion identifying the gap in the market that TeamTrack addresses.
1.7 Proposed System Overview	Omar Waheed Khan	Drafted the high-level overview of the TeamTrack system, its concept, objectives, and core modules.
1.7.1 System Concept and Objectives	Omar Waheed Khan	Wrote the system concept statement and primary objectives.
1.7.2 Core Functional Modules	Omar Waheed Khan	Described the core functional modules of the proposed system.
1.7.3 The AI-Assisted Component	Omar Waheed Khan	Wrote the section describing the AI-assisted features and their role in the system.
1.7.4 Conclusion	Omar Waheed Khan	Drafted the concluding remarks for the Introduction section.

These section assignments are reflected in the Milestone 1 ToC.

2. Overall Description

Section / Subsection	Primary Contributor	Contribution Details
2. Overall Description	Mohammad Siraj Hussain, Muhammad Hashir	Shared ownership across the Overall Description section, with Siraj leading the product context subsections and Hashir covering constraints and workflow.
2.1 Product Perspective	Mohammad Siraj Hussain	Drafted the product perspective, including system overview, standalone

		nature, and positioning in the educational technology context.
2.1.1 System Overview	Mohammad Siraj Hussain	Wrote the high-level system context description for TeamTrack.
2.1.2 Standalone Nature and Core Integration	Mohammad Siraj Hussain	Described TeamTrack as a standalone platform and its integration approach.
2.1.3 Positioning in Educational Technology Context	Mohammad Siraj Hussain	Contextualised TeamTrack within the academic project management landscape.
2.2 Product Functions	Mohammad Siraj Hussain	Documented all core product functions including authentication, task management, time tracking, communication, and analytics.
2.3 User Classes and Characteristics	Mohammad Siraj Hussain	Wrote the user class definitions including Student Team Members, Team Leaders, Instructors, and System Administrators.
2.4 Operating Environment	Mohammad Siraj Hussain	Documented the client and server environment, network constraints, OS support, and hardware requirements.
2.5 Design and Implementation Constraints	Muhammad Hashir	Wrote the design and implementation constraints covering academic timelines, technology stack, security, AI features, and deployment limitations.
2.6 Assumptions and Dependencies	Muhammad Hashir	Documented the project's user, environment, data, development, and scope assumptions and dependencies.
2.7 System Workflow Overview	Muhammad Hashir	Drafted the system workflow overview including workflow stages and the workflow summary table.

These allocations are explicitly listed in the Milestone 1 ToC.

3. System Features (Functional Requirements)

Section / Subsection	Primary Contributor	Contribution Details
3. System Features	Maria Waheed, Zernish Shahid Nazir, Omar Waheed Khan	Shared ownership across the functional requirements, with Maria leading the core user-facing features, Zernish leading communication and analytics, and Omar leading AI and notification features.
3.1 User Registration and Authentication	Maria Waheed	Wrote the description, priority, stimulus/response sequences, and functional requirements for user registration and authentication.
3.2 Role-Based Access Control (RBAC)	Maria Waheed	Documented the RBAC feature including priority, stimulus/response sequences, and functional requirements for role-based permissions.
3.3 Project and Team Management	Maria Waheed	Wrote the project and team management feature section, covering creation, membership, and project lifecycle management requirements.
3.4 Task Management System	Maria Waheed	Drafted the task management system feature, including task creation, assignment, status tracking, and board/list view requirements.
3.5 Time Tracking and Effort Logging (CORE)	Maria Waheed	Documented the core time tracking and effort logging feature, its priority, and detailed functional requirements.
3.6 Team Chat and Communication Module	Zernish Shahid Nazir	Wrote the team chat and communication module feature requirements, including real-time messaging and channel management.
3.7 Contribution Analytics Dashboard	Zernish Shahid Nazir	Documented the contribution analytics dashboard feature, including data

		visualisation and individual contribution tracking requirements.
3.8 AI-Driven Project Assistant	Omar Waheed Khan	Wrote the AI-driven project assistant feature requirements, covering intelligent query handling and AI-generated insights.
3.9 Notifications and Reminder System	Omar Waheed Khan	Documented the notification and reminder system requirements, including trigger conditions and delivery mechanisms.
3.10 Activity Log and Audit Trail	Omar Waheed Khan	Wrote the activity log and audit trail feature requirements, covering logged events and access controls.

This ownership is shown in the Milestone 1 ToC.

4. External Interface Requirements

Section / Subsection	Primary Contributor	Contribution Details
4. External Interface Requirements	Mohammad Siraj Hussain	Led the external interface requirements section across user, hardware, software, and communication interfaces.
4.1 User Interface Requirements	Mohammad Siraj Hussain	Wrote the general layout and navigation, key screens and UI behaviour, and usability and accessibility requirements.
4.1.1 General Layout and Navigation	Mohammad Siraj Hussain	Documented the layout principles and navigation structure of the TeamTrack interface.
4.1.2 Key Screens and UI Behaviour	Mohammad Siraj Hussain	Wrote requirements for core application screens and their interactive behaviour.

4.1.3 Usability and Accessibility Requirements	Mohammad Siraj Hussain	Documented usability standards and accessibility compliance requirements.
4.3 Software Interface Requirements	Mohammad Siraj Hussain	Wrote the backend API, authentication, database, and optional external interface requirements, including the software interface summary table.
4.4 Communication Interface Requirements	Mohammad Siraj Hussain	Documented client-server communication protocols, data formats, authentication in communication, reliability handling, and real-time communication requirements.

These responsibilities are directly assigned in the Milestone 1 ToC.

5. Non-Functional Requirements

Section / Subsection	Primary Contributor	Contribution Details
5. Non-Functional Requirements	Zernish Shahid Nazir, Muhammad Hashir	Shared ownership, with Zernish leading the security, performance, and usability requirements and Hashir covering availability, reliability, data integrity, and ethical AI considerations.
5.1 Security Requirements	Zernish Shahid Nazir	Wrote the security requirements covering authentication and access control, data protection, and session and input security.
5.2 Performance Requirements	Zernish Shahid Nazir	Documented system responsiveness, feature-specific performance targets, and scalability and AI performance requirements.
5.3 Usability Requirements	Zernish Shahid Nazir	Wrote the usability requirements covering interface design and consistency, task efficiency and

		feedback, and accessibility and readability.
5.4 Availability and Reliability Requirements	Muhammad Hashir	Documented the availability and reliability requirements for the TeamTrack system, including uptime targets and failure recovery.
5.5 Data Integrity and Privacy	Muhammad Hashir	Wrote the data integrity and privacy requirements ensuring consistent, accurate, and GDPR-aligned data handling.
5.6 Ethical and Responsible AI Considerations	Muhammad Hashir	Documented ethical constraints and transparency requirements for the AI-assisted components of TeamTrack.

This ownership is shown in the Milestone 1 ToC.

6. Data Requirements

Section / Subsection	Primary Contributor	Contribution Details
6. Data Requirements	Zernish Shahid Nazir, Maria Waheed	Shared ownership, with Zernish leading data entities and relationships and Maria handling data storage and retention.
6.1 Data Entities Description	Zernish Shahid Nazir	Documented all core data entities including User, Project, Task, Time Log, Activity Log, Message, Notification, and Project Member with their key attributes.
6.2 Data Relationships	Zernish Shahid Nazir	Wrote the data relationship definitions and entity associations between all core system entities.
6.3 Data Storage and Retention	Maria Waheed	Documented logical database requirements, data retention and archiving policies, and data privacy and disposal requirements.

These assignments are directly shown in the Milestone 1 ToC.

7. Appendices

Appendix	Primary Contributor	Contribution Details
Appendix A: Proposed Technology Stack	Omar Waheed Khan	Documented the proposed technology stack and deployment overview, including frontend, backend, database, and AI component selections.
Appendix B: Entity Relationship Diagram (ERD)	Zernish Shahid Nazir	Prepared the Entity Relationship Diagram illustrating the data model and entity associations for TeamTrack.
Appendix C: Use Case Diagram	Zernish Shahid Nazir	Prepared the Use Case Diagram capturing the interactions between system actors and key system features.
Appendix D: Glossary	Zernish Shahid Nazir	Compiled the glossary of terms and definitions used throughout the SRS.
Appendix E: Software Quality Attributes	Zernish Shahid Nazir	Documented the software quality attributes table covering usability, reliability, performance, security, and maintainability with measurable targets.
Appendix F: Agile Development	Muhammad Hashir	Documented the Agile methodology, sprint planning, client interaction, internal communication, progress tracking, and continuous improvement practices applied during Milestone 1.
Appendix G: Attributions	All Members	All team members contributed their names and section ownership to the attributions table, which records section-level authorship across the full SRS.

Appendix H: References	Omar Waheed Khan	Compiled and formatted all references cited throughout the SRS in IEEE referencing style.
Appendix I: Meeting Minutes	Muhammad Hashir	Documented and maintained the meeting minutes for all team and client meetings conducted during Milestone 1.
Appendix J: Toggl Reports	All Members	Each team member individually submitted their own Toggl time tracking report as evidence of contribution effort.
Document Formatting	Omar Waheed Khan, Muhammad Hashir	Shared responsibility for overall document formatting, consistency, and visual presentation of the SRS.

These appendix assignments are listed in the Milestone 1 ToC.

Overall Contribution Percentage Summary

Team Member	Primary Milestone 1 Areas	Approx. Ownership
Muhammad Hashir	Introduction (Purpose, Structure, Audience, Scope), Design & Implementation Constraints, Assumptions & Dependencies, System Workflow Overview, Non-Functional Requirements (Availability, Data Integrity, Ethical AI), Agile Development Appendix, Meeting Minutes, Document Formatting	20%–24%
Mohammad Siraj Hussain	Product Perspective, Product Functions, User Classes, Operating Environment, External Interface Requirements (UI, Software, Communication)	18%–22%
Omar Waheed Khan	Definitions & Abbreviations, Competitor Analysis, Proposed System Overview, AI-Driven Project Assistant, Notifications & Reminder System, Activity Log & Audit	20%–24%

	Trail, Technology Stack Appendix, References, Document Formatting	
Maria Waheed	User Registration & Authentication, RBAC, Project & Team Management, Task Management System, Time Tracking & Effort Logging, Data Storage & Retention	18%–22%
Zernish Shahid Nazir	Team Chat & Communication Module, Contribution Analytics Dashboard, Non-Functional Requirements (Security, Performance, Usability), Data Entities & Relationships, ERD, Use Case Diagram, Glossary, Software Quality Attributes	18%–22%

Closing Statement

Primary ownership of specific sections and subsections in Milestone 1 was assigned to individual team members. That said, the Software Requirements Specification was still submitted as a collaborative group effort. Discussion, review, refinement, and final consolidation before submission, everyone contributed through those activities. The requirement that the final report clearly attributes who wrote each part of the document is satisfied by this structure.

SRS Versions

SRS Version 2.0 Update (15/02/26)

The second version of the SRS was revised on 15 February 2026 after an internal assessment of the initial requirement set. The emphasis at this step was on the expansion and organization of the system requirements to be more representative of the desired functionality and workflow of TeamTrack.

Based on this, a number of areas where the requirements needed more clarification and formalization were identified. Specifically, the functional scope of the system, non-functional expectations, roles and interaction flows had to be clearly delineated to facilitate the subsequent design and development phases.

To overcome this the functional requirements have been greatly extended to offer better coverage of fundamental system functions like task management, time tracking, communication and reporting work flows. Besides this, the non-functional requirements were presented and organized to establish the expectation of performance, usability, reliability, and system behavior in the normal conditions of use.

The introduction of user roles and permissions was also formal to introduce a clear access control model in the system. This made sure that the various classes of users like team members, team leaders and instructors would have corresponding scoped capabilities that matched with academic project processes.

In addition, an organized system of use cases was created to portray the main user interactions with the system. These user scenarios gave a better insight into how people would interact with functionality like task updates, time logging, and collaboration to bridge the gap between abstract system requirements and real-world behavior of a system.

This version saw an overall improvement of the SRS structure, although the basic scope of the system was the same. The changes were mainly aimed to make it more clear, complete and closer to the desired system design, which would provide a better basis on which later iterations would be built.

SRS Version 3.0 Update (08/03/26)

The third release of the SRS was revised on 08 March 2026, based on the client feedback on enhancing the usability of the documents and their navigation. The main purpose of this update was to improve the interaction of the users with the document and not to change the system requirements.

According to this feedback it was possible to make modifications to enhance the accessibility of important sections in the SRS. Specifically, the fast navigation links were added and improved so that readers could navigate more effectively between various parts of the text, particularly in the requirements, roles and use cases.

These alterations were meant to facilitate easier readability and ease of reference especially to the stakeholders who were going to look into certain sections of the document when discussing or in the course of evaluation.

It did not introduce any new functional or non-functional requirements. The update did not change the scope, behaviour or direction of design of the system and was confined to bettering the presentation and usability of the SRS document.

SRS Version 4.0 Update (29/03/26)

The fourth and final version of the SRS was updated by 29 March 2026 following a review with the client ahead of the final milestone submission. That led to the fourth and final version of the SRS being updated. The client pointed out that accountability around role management and AI transparency were areas worth revisiting. Making sure those were sufficiently covered was the concern.

From that the team also identified several further areas where the requirements could be made more precise. Task dependency handling and contribution fairness visibility were two of them. Both had emerged through the later architecture and design work, and the level of detail needed to be better reflected. To reflect this, four new functional requirements were added, FR-22, FR-43, FR-72, and FR-80, along with four new non-functional requirements, NFR-14, NFR-24, NFR-33, and NFR-44.

FR-22 was introduced to enforce transparency around role changes within project workspaces. All permission modifications should be logged and traceable. FR-43 addressed a gap in the task management requirements by formalizing support for task dependencies, which shows better how academic project workflows actually operate. FR-72 makes the contribution analytics section stronger with comparative indicators, giving users contextual insight into contribution balance rather than just raw totals. FR-80 supported responsible AI usage, keeping the system transparent and trustworthy in an academic setting.

Non-functionals were also touched on. NFR-14 was added to maintain stable performance across the platform's many simultaneously active modules. NFR-24 focused on workflow efficiency, requiring that frequent actions like task updates and time logging stay consistent for daily use. NFR-33 enforced transactional consistency to protect logged records, including time logs, contribution data, and activity history, against unexpected data loss. NFR-44 reinforced responsible AI use by making sure users can review and override AI generated suggestions before any action is applied.

The main fundamental structure or scope of the SRS did not change with these additions it only aligned with the finalized system design. Nothing critical was left underspecified heading into implementation.

7. TIS Report



List of Tables

Table 1: Stakeholder Identification Table	147
Table 2: Team Members	148
Table 3: Deliverables Overview	150
Table 4: System Structure	157
Table 5: Frontend Criteria and Comparison	157
Table 6: Backend Criteria and Comparison	158
Table 7: Database Criteria and Comparison	159
Table 8: AI Criteria and Comparison	160
Table 9: Security Stack	161
Table 10: Final Stack	161
Table 11: User Authentication FR (UPDATED)	162
Table 12: Project Management FR (UPDATED)	163
Table 13: Task Management FR (UPDATED)	163
Table 14: Contribution Tracking FR (UPDATED)	164
Table 15: AI-Assisted FR (UPDATED)	165
Table 16: Performance Requirements FR (UPDATED)	165
Table 17: Security Requirements FR (UPDATED)	166
Table 18: Reliability and Maintainability FR (UPDATED)	167
Table 19: Role-Permission summary table	169
Table 20: System Modules and Responsibilities	184
Table 21: Version Control and Environment Management Tools	185
Table 22: Implementation and Testing Stages	186
Table 23: Operating Environment Summary	187
Table 24: Backend Technology Stack	188
Table 25: Frontend Technology Overview	189
Table 26: User Interface Design Elements	190
Table 27: AI Module Architecture Components	191
Table 28: AI Interaction and Data Flow	192
Table 29: AI Data Flow Process	192
Table 30: Authentication and Access Control Mechanisms	193
Table 31: Application and Infrastructure Security Measures	194
Table 32: Test Scenario Matrix	197

Table 33: Communication Governance Structure	205
Table 34: Agile Roles and Contributions	207
Table 35: Meeting Logs	208
Table 36: Meeting Minutes Summary	209
Table 37: Final Stack Overview	212
Table 38: Comparison Table Frontend	220
Table 39: Comparison summary	227
Table 40 : Comparison Databases	232
Table 41: Alternatives Evaluated for AI Integration	234
Table 42: Expanded Comparison AI Model	235
Table 43: Requirements Comparison	236
Table 44: Backend Framework Alternatives Evaluated	238
Table 45: Comparison API's	238
Table 46: Firehose vs Traditional Storage	240
Table 47: Criteria Databases	240
Table 48: Authentication Comparisons	242
Table 49: Technologies Selection Summary	243
Table 50: Criteria for Authentication	243
Table 51: Glossary	254
Table 52: WBS Structure	264
Table 53: ToggI Report Table	267

List of Figurines

Figure 1: System Architecture Diagram	176
Figure 2: Entity Relationship Diagram (ERD).	179
Figure 3: User Flow Diagram	182
Figure 4: System Flow Diagram	183
Figure 5: Trello Kanban	210
Figure 6 : Wireframe	257
Figure 7: Logo	257

1. Executive Summary

TeamTrack Technology Infrastructure Specification (TIS) introduces their academic project management system, by which teamwork can be accomplished by the university students working on group assignments together. The system creates a unified digital workplace which works with AI-enhanced knowledge to maximize task management and individual contribution measurement and develops open lines of communication.

The TIS extends the defined needs and conceptual planning process by developing the technical direction of the system, the choice of technology stack to use, and an architecture foundation needed to implement the system. The document outlines the infrastructure operations in the platform that include processing user interface components, backend, database structure, artificial intelligence systems, as well as security.

The project has reached where it is today since it has completed the first phase of conceptual design to the actual development of technical specification. The team was able to identify fundamental system modules, find out functional and non-functional requirements and organize architectural decisions that would facilitate the real system development. The suggested architecture explains a web based frontend that will be linked with API-based back end services and will rely on relational data storage along with an artificial intelligence system that creates recommendations and insights and safeguards academic integrity via human review.

The report records the procedure that tested and chose technologies based on its usability, feasibility, performance of the system, and long-term usability. System design documents, implementation plans, testing methods and project coordination documentation ensures accountability and traceability is done through the development process.

TIS system provides critical technical support that allows TeamTrack to provide a system that interrelates between conceptual planning and real system implementation. The system defines a clear development direction that involves test processes and deployment plans in the future that considers the scalability of the system and operational feasibility and compliance with the real-life academic collation needs.

2 Introduction

2.1 Background / Context

TeamTrack is an academic project management and accountability platform that aims to help student teams working on group project and assignments. In contrast to general purpose workplace tools, TeamTrack is designed to support academic collaboration needs, making individual contributions clear and to ensure that communication remains work related, as well as helping teams stay organised around deadlines. The platform combined task management, project

chat, time tracking, and contribution analytics in a single workspace, and reduces the need of separate tools and simplifies transparency throughout projects.

Group assessments are common in university settings across courses. However, student teams work under different pressures and constraints than workplace teams, they have a set semester deadline, grades, working with many different students and different teams, and limited oversight from instructors. Many teams rely on a mix of tools like Trello, WhatsApp, Google Docs, or email threads. Although each of the tools has its own purpose, none of them are designed to manage contributions accountability within academic environments.

This results in inconsistent teamwork. Task boards show what is there at any given moment but not the way it developed, meaning the context behind decisions, changes in responsibilities, and effort over time is lost. Chat threads include significant decisions that can be hardly followed. This creates mismatch between the visible performance and the actual effort.

This leads to a lack of consistent teamwork, since Task boards show what is there at any moment, but doesn't show the way it developed. This means that the context behind decisions or any changed in responsibilities, and effort put in is lost over time. Chat threads contain important decisions which can hardly be tracked, and this creates mismatches between performance, and the actual effort put in.

2.2 Problem Statement

Student teams often face unequal workload distribution, unclear responsibility boundaries, and limited evidence of individual effort.

University group projects often face problems like unequal workload distribution, unclear responsibility boundaries, and limited evidence of individual effort. It's not usually an issue with planning, its about effort being traceable, and fair. Usually, teams use many different tools for different project needs, like task boards, different messaging apps, and different streams of communication, but this way, decisions and context are at risk of being lost in chat history and task boards only show current states. Other than that, instructors and students also find it difficult to justify the distribution effort, especially when you consider lack of evidence about contributions.

In cases where there is a disagreement about participation, there is not much data trusted data or evidence that can be referred to. Lecturers usually rely on peer assessments which are subjective, and this could be unfair or unreliable, especially when each student has their own account of team contribution. The current tools cover productivity, but they don't support requirements of academic accountability, like time logs, contribution analytics, or any history reviews specific to support assessment contexts.

TeamTrack is a solution to these problems as it has the core teamwork functions in a single system. It gives students the ability to manage their tasks through board or list views, communicate within a project environment, and be able to time themselves on work either through timers or manual logs, and with this information in turn contributes to contribution summaries that help

keep track of workload balance. As for the AI features, they are more like support insights and suggestions.

2.3 Aim and Objectives

2.3.1 Aim

TeamTrack's goal is to develop an online platform for academic project management that will improve university group work's accountability, transparency, and cooperation, remove all the bits and pieces of necessary functionalities from multiple environments and merge them into one, to remove any inconsistencies when trying to work with, and promoting fairness in evaluation and collaboration between students. TeamTrack aims to help students by making teamwork more structured, traceable, and evidence based.

2.3.2 Objectives

These goals ensured that TeamTrack is a tool designed to help structured teamwork in providing fairness and accountability for academic activities.

1. Make sure that only authorized users can access project workspaces by having secure user authentication and account management procedures in place.
2. Role-based access control (RBAC) to ensure that all user and project-related visibility is restricted to roles for which they have been assigned.
3. Allow project and workspace management features like creating teams, controlled member access, and archiving on the end of the project.
5. Task management functions including board, list views, assignment management, task deadlines, task status and comments.
6. Generate structured timesheet summaries for each user and project, using time tracking system (timers and manual recordings).
7. Make activity monitoring trails to ensure a history of significant system events and changes.
8. Keep an activity trail to have transparency and history of significant system events and changes.
9. Provide AI-assisted data in a recommendation, recommending workload observations or inactivity alerts.

2.4 Scope (In-Scope / Out-of-Scope)

2.4.1 In-Scope

The scope of this project is focused mainly on core features that are needed to make TeamTrack function according to the goals set here. Since the system is created during a timeframe where features which have a direct impact on improving transparency, traceability and collaboration, in a more structured manner, are intended to be prioritized over others. The following are some of their

reasons. The features are included because they are directly addressing issues identified (in the problem statement).

- Secure user registration and authentication, to ensure that only individuals part of a project can access the project.
- Have Role-based access control (RBAC), so that permissions are specific to whichever role the user has.
- Creating projects, managing members, and archiving features once the project is completed.
- Task management through board and list views, including assignment, deadlines, status tracking, and comments
- Built-in project chat that stores communication history to preserve decision context
- Time tracking with timers and manual log entries
- Contribution analytics dashboards generated from task and time log data
- Activity logging to maintain a structured store of key changes
- AI-assisted insights limited to summaries and for recommendations

2.4.2 Out-of-Scope

These are some features excluded from the system. The following features are not included because of feasibility and time limitations, also considering the alignment with the actual requirements of group work in universities.

They are also excluded because they would either a, demand more work to develop than what's practical or even needed for the project, or b, they just go beyond what was planned and specified (and time consuming). The goal is to be realistic. They could be considered in the future, but they aren't really needed to achieve the current objectives.

- Enterprise-level integrations, meaning nothing for large-scale corporate use
- No evaluation or grading related features
- Advanced predictive AI or chat models
- Large-scale deployment assumptions beyond typical university group sizes
- Complex real-time infrastructure
- Extensive document managing or hugely larger file storage systems

2.5 Assumptions and Constraints

2.5.1 Assumptions

The following assumptions guide the design of the system. These assumptions ensure the system works for realistic patterns of academic usage.

- User are expected to have basic digital literacy and access the system through modern web browsers.
- For the statistics to be accurate, teams will maintain tasks and time logs proactively.

- Internet connectivity will be stable on user side, so they can access the web apps.
- Access to the platform will be done mainly through desktop or laptop environments.
- The insights are to help supervisors, but supervisors will continue to have independent grading authority.

2.5.2 Constraints

The following limitations apply to TeamTrack's development and deployment. Throughout the project, these limitations influence how features are prioritized and also the choices for technology.

- The depth of each feature is limited to its core functionality, considering the fact that the project must be completed within an academic capstone timeframe.
- The system architecture is going to follow a web-based client–server structure and use a Python backend framework (FastAPI) and a relational database handled with ORM.
- Database design must enforce referential integrity and transactional consistency as specified in the SRS.
- Strict RBAC enforcement and restricted access to data are required by security regulations.
- AI-assisted features have to be transparent, and must only provide suggestions/insights, not make decisions or anything else.
- The system is not intended for enterprise-scale implementations, but rather for small/medium academic teams.

3. Project Overview

The project involves the development of an artificial intelligence-based team-based project management and time tracking system in the hope of serving university students who are required to complete group based assignments. The typical issues of coordination can manifest itself in the group projects, e.g., the workload allocation disposition, the invisibility of the contribution of each party, inability to hit the milestones, and inconsistency of the communication practices. All these can negatively affect the performance of the teams and make the academic staff even more complicated in case of attempting to evaluate the effort put in by individuals in a reliable and equal manner.

The primary objective will be to design and deploy a system that will ease the process of assigning tasks and monitoring them, tracking activities at the time-level, monitoring the progress, and generating reports depending on the contributions. These core functions, along with AI-driven analytics to produce structured progress summaries, detect participation inactivity, and produce participation insights, which indicate individual participation over time, are the solution as well. The system will give the power of consolidation of project coordination mechanisms, intelligent

tracking capabilities within a unified system and consequently accountability, visibility of operation as well as increase efficiency between student teams will be heightened.

The project is implemented on the basis of TIS module and system development lifecycle. This includes the requirements elicitation, design, architectural design, interface design, implementation of prototypes, verification and validation and assessment of the stipulated academic use cases. Additionally, the process of development would be coupled with continuous interaction with the clients as it would ensure that the final design choice would rely on the current academic needs and expectations of the stakeholders.

The final output will be the available prototype of a working system and documentation outlining the purpose and use of the system and anticipated value to bring about the changes in collaboration dynamics and permit more unbiased assessment processes to take place within the group-based academic environment.

3.1 Stakeholders

Stakeholders would be individuals or groups with an interest in the successful completion and implementation of the project. Identifying stakeholders ensures that their requirements and expectations are taken into account throughout the system development process.

Table 45: Stakeholder Identification Table

Stakeholder	Role in Project	Interest in System	Level of Influence
Dr. Benita Veronica (Client)	Project client and academic supervisor	Ensures system meets academic needs and improves group project evaluation	High
Students (End Users)	Primary users of system	Use system for task tracking, time tracking, and collaboration	High
Lecturers	Secondary users	Monitor student progress and assess contributions	Medium

Project Team Members	System developers	Responsible for design, development, and testing	High
University	Institutional stakeholder	Potential adoption of system for academic use	Medium

3.1.1 Stakeholder Descriptions

Client - Dr. Benita Veronica:

Dr. Benita Veronica is the primary individual whom we have as a client in this project. She provides project requirements, direction, and feedback in the development. Her needs and expectations must be addressed, and her academic needs should be met to make the project successful.

Students (Primary Users):

The system will enable students to handle group projects, allocate school work, and track the amount of time spent on work. The system must be easier and simpler to use and efficient enough that students embrace the system.

Lecturers:

The system will be used by lecturers to monitor the progress of students and evaluate contributions. They require transparent dashboards and reports that provide information on the performance of people and teams.

Project Development Team:

Designing and construction of the system is assigned to the development team. Team members should work well to deliver on time and produce quality results. This will allow the project to devise a system that will meet the expectations of the stakeholders involved and is capable of impacting all the users in an academic and technical sense.

3.2 Roles and Responsibilities

Table 46: Team Members

Team Member	Role	Key Responsibilities
Muhammad Hashir	Project Manager & System Analyst	Project planning, documentation, requirement

		analysis, coordination with client
Omar Waheed Khan	Backend Developer	System architecture, technology selection, backend support
Maria Waheed	Documentation & Research	Research, report writing, testing documentation
Zernish Shahid	Testing & Support	System testing, debugging, user feedback collection
Mohammad Siraj	Frontend Developer	UI design, user interface development, usability improvements

3.2.1 Role Explanation

Project Manager:

Overall plan of the project, milestone monitoring, communication to the stakeholders, and report writing. Make sure that every one of the deliverables is completed on schedule and to the academic specifications.

Technical Lead:

In charge of selecting the appropriate technologies, designing the systems, and developing. Means that the system architecture is scalable and operational.

Frontend Developer:

Creates user interface and designs. Provides that the system is not only attractive, but also navigable and easy to use.

Documentation & Research:

Manages research processes, prepares reports, and aids in the preparation of reports. Make sure that referencing and structured documentation are accurate.

Testing & Support:

Their duty will be to test the functionality of the system, identify bugs, and guarantee system quality. Gathers the feedback and facilitates the improvements.

3.3 Project Deliverables

Project deliverables represent the outputs that will be produced throughout the project lifecycle. These deliverables demonstrate progress and ensure that project objectives are achieved.

Table 47: Deliverables Overview

Deliverable	Description	Responsible Members	Timeline
Requirement Analysis Document	Detailed system requirements and stakeholder needs	Hashir, Maria	Completed
System Design Document	Architecture, database design, UI design	Omar, Hashir	Ongoing
UI Prototype	Initial interface design and layout	Zernish	Ongoing
Functional Prototype	Working version of system with core features	Entire Team	Upcoming
Testing Report	System testing results and feedback	Siraj, Maria	Upcoming
Final Report & Presentation	Complete documentation and demonstration	Entire Team	Final Milestone

3.3.1 Description of Deliverables

Project deliverables refer to the quantifiable items that are going to be created in the project lifecycle. These deliverables show how the project is progressing, and all the requirements and

objectives that were identified in the previous stages are implemented. All the deliverables are well planned to facilitate the technical development process as well as the academic documentation process.

The items sent in the deliverables are aimed at ensuring that the system has been well planned, designed, implemented, tested, and assessed before delivering. They are also good examples of teamwork, research, and practical application of the knowledge acquired during the course. The project deliverables may be grouped into documentation deliverables and technical deliverables, which would lead to the overall success of the system.

3.3.2 Key Project Deliverables

1. Requirement Analysis Document

This report offers an in-depth insight into the problem statement, stakeholder requirements, system goals, and functional requirements. It forms the basis of any further development work and makes the team well aware of what should be constructed.

Purpose:

- Specify system goals and scope of the project.
- Expectations of document stakeholders.
- Elaborate functional and non-functional requirements.
- Offer an organizational scheme of development.

Importance:

This deliverable will make sure this project is not off track with the needs of the user, and there is no confusion later in the development.

2. System Design Document

The system design document describes the technical organization and design of the proposed solution. It contains the diagrams, workflows, database design, and interface layouts explaining the way the system is going to work.

Contents include:

- System architecture diagram.
- Database design and data movement.
- Design and layout of user interface.
- Technology stack and technologies used.

Purpose:

To give a clear roadmap on which the development should be made, and to make sure that every member of the team is aware of the interaction of the system components.

3. UI Prototype

The UI prototype is the visual and interactive diagram of the system prior to actual development. It enables the stakeholders and users to see what the system would look like and how it will work.

Key features:

- Initial interface layouts
- Navigation structure
- User interaction flow
- Feedback for improvements

Importance:

With the help of this prototype, it is possible to diagnose the usability problems at the initial stage and make sure that the system will be convenient and easy to use.

4. Functional Prototype

This is the working version of the system, which is the functional prototype that illustrates core functionality and features. It enables the team to test the work of a system in a simulated environment.

Includes:

- Core system modules
- Effort management and teamwork functions.
- Data storage and retrieval capabilities.
- Included security and log-on system.

Purpose:

To confirm that the system is performing as specified in the requirements, or to know the necessary technical advancements before the system is finally deployed.

5. Testing and Evaluation Report.

This is a deliverable that records all the activities carried out in the project with regard to testing. It contains testing plans, test scenarios, test results, and system analysis.

Contents include:

- Testing methodology
- Test scenarios and outcomes
- Bug fixes and bug identification.
- User feedback analysis

Importance:

Make sure that the system is stable, reliable, and prepared to be implemented.

6. Final Report and Presentation.

The final report will have a detailed documentation of all the project, such as research, design, development, testing, and evaluation. The presentation will show the system and emphasize major accomplishments.

- Final report includes:
- Full system documentation.
- Explanation of design and development.
- Results of tests and validation.
- Future recommendations and improvements.

Presentation includes:

- System Architecture simulation.
- Project journey and outcomes
- Problems encountered and measures taken.

Purpose:

To give a full picture of the project, as well as prove how the offered solution helps to solve the problem identified successfully.

3.4 Tools and Technologies Used

The TeamTrack system uses a Role-Based Access Control (RBAC) model to control system access to system functionality and system data. RBAC provides a system with security, accountability, and equitable interaction ensuring each user can only interact with the system in the perimeters of his or her assigned duties, thereby supporting security, accountability, and equitable collaboration within an academic setting.

The permissions are applied consistently both in the front end interface and the backend API layer so that unauthorised actions are not permitted irrespective of how a request is generated.

3.4.1 Development Tools and Environment

The process of its development is premised on the industry-standard tools that facilitate collaboration, version control, and consistency of environments:

- **Bitbucket**
The primary version control system, where to regulate the source code, maintain a history of changes done and oversee cooperation on the source code. controlled and branch mergers.
- **Docker**
The system components will be containerised using Docker to ensure that the environments will be identical whenever running the components in both the development and testing stages. Containerisation reduces the issues associated with the environment and facilitates control over dependency.
- **Container-Based Architecture**
The major system components (frontend, backend API, AI module, databases) are deployed separately as containers which further enables them to be isolated, trustworthy and needs less maintenance.

3.4.2 Backend Technologies

The most central layer of the system is the backend layer which administers the business logic, data processing and coordination of the system:

- **Python**
The primary reason why Python was selected as the primary backend programming language is that the language is readable, has an excellent ecosystem, and can be intertwined with web services and technologies related to AI.
- **FastAPI**
Fast API implements RESTful backend services. It is effective in asynchronous handling of requests, request validation is automatically carried out and is written in a manner that makes easy creation of APIs which makes it able to serve many users interacting with it in a specific time.
- **RESTful API Architecture**
The backend supports the usage of RESTful endpoints to make the process of communicating with the frontend, databases, and AI services safe and organized.

3.4.3 Frontend Technologies

The frontend provides the system with the user interface and deals with the system being user-friendly, responsive, and clear:

- **Nuxt.js**
The frontend application is implemented through Nuxt.js which is a Vuj-based platform. It is also component-based, inbuilt routing, also improves on performance and render serverside.
- **Web-Based User Interface**
The system is accessed through the normal web browsers on the desktop and laptop computer hence wide level of access is achieved without the need of installing the native applications.

3.4.4 Database Technologies

The system has been built upon a dual-database strategy in order to isolate the transactional workloads and analytical workloads:

- **PostgreSQL**
Its primary relational database is PostgreSQL in which the structured transaction data, including users, projects, tasks, access permission, and the records of the core system are stored. It offers data integrity and complex data relation.
- **Firehose Database**
The firehose database is logged to record the high rate activity data such as user activity logs and time tracking activities. This division removes the processing overhead that would be experienced in the main transactional database.

3.4.5 AI and Analytics Technologies

The system will include an element that can be assisted by AI to enhance the usability of the system and create an insight:

- **FastAPI-Based AI Service**
The AI module is an independent FastAPI service to ensure that it can be deployed independently and does not affect the main performance of the backend with AI-related processing.
- **LLaMA 3.1 Language Model**
LLaMA 3.1 is integrated with a natural language query and AI-based insights. The applications of AI are restricted to guided and limited purposes to ensure reliability and safety of the systems.

3.4.6 Security and Communication Technologies

The security is maintained within all the components of the system using standardised mechanisms:

- **HTTPS with SSL Encryption**
Data confidentiality and integrity are ensured by using HTTPS to protect all communication between the frontend and the backend and databases and AI services.
- **Token-Based Authentication and RBAC**
Role based access control and use of secure authentication tokens are used to make sure that system features and data is only availed to authorised users according to their assigned roles.

3.4.7 Summary

The chosen tools and technologies offer reflective balance between performance, security, and scalability and at the same time, be suitable in an academic development scenario. The containerised architecture and modular architecture makes system development controllable, maintains system growth and facilitates future upgrades without the need to make significant architecture alterations.

4. Technology Stack Selection

Another important aspect of the successful development of the suggested system is the choice of a suitable technology stack. The technologies selected should be able to facilitate scalability of the system, have a security measure, provide functionality to integrate AI, and should be viable under the development constraints of time, cost, and technical expertise. The section describes the system procedure taken to identify, appraise and choose the most appropriate technologies to implement the system.

The choice of technology was informed by both the functional and non-functional requirements as outlined in [Section 5](#) and the architectural considerations as outlined in [Section 6](#). The various technologies were evaluated on performance, compatibility, security, complexity of development and long maintainability. Instead of using the tools randomly, comparative evaluation method was taken. The final decision was made by considering several options that were available to each system layer.

4.1 Overview of Required Technologies

The suggested system will need technologies on more than one layer to facilitate user interaction, data processing, automation by AI, and secure storage of information. Such requirements are related to the necessity to develop a scalable and intelligent platform that can be used to provide collaborative academic environments.

The system architecture consists of presentation, application, AI services, database, and security layers, which necessitate special technologies.

Table 48: System Structure

Technology Layer	Requirement	Role in System
Frontend	Interactive interface	Enables user interaction
Backend	API & logic handling	Processes requests
AI Module	Automation & intelligence	Chatbot, summarization
Database	Data storage	Stores users and system data
Security	Authentication & access	Protects system usage
Infrastructure	Hosting environment	System availability

Further system structure is illustrated in [Section 6.1 System Architecture](#).

4.2 Frontend Technologies Criteria

The criterion of frontend technologies included usability, responsiveness, integration capability, and long-term sustainability. The interface was required to be lightweight but dynamic because the system presupposes the use of dashboards, collaboration tools, and AI-assisted interaction.

Table 49: Frontend Criteria and Comparison

Criteria	Description
Performance	Efficient rendering and loading

Responsiveness	Mobile and desktop compatibility
Integration	REST API compatibility
Maintainability	Ease of updates
Community Support	Availability of documentation

Comparison of Alternatives

Technology	Advantages	Limitations	Decision
Nuxt.js	Lightweight, easy learning curve, flexible	Smaller ecosystem than React	Selected
React.js	Large ecosystem, scalable	Higher complexity for beginners	Not selected
Angular	Enterprise-level architecture	Heavy framework, steep learning curve	Not selected

Nuxt.js was selected because it provides a middle ground between simplicity and scalability, and can be tolerated when it comes to developing an academic system. React and Angular were not selected in terms of complicatedness and overhead of development in the context of the project.

Frontend implementation details are discussed in [Section 7.3 Frontend Implementation](#)

4.3 Backend Technologies Criteria

Backend technologies were tested on their capacity or the backend to support the API driven architecture, AI processing, and the performance of the system.

Table 50: Backend Criteria and Comparison

Criteria	Description
Scalability	Multi-user handling
AI Integration	ML compatibility
Security	Authentication and access control
Performance	Fast request handling

Comparison of Alternatives

Technology	Advantages	Limitations	Decision
FastAPI	High performance, async support, Python ecosystem	Newer framework	Selected
Django	Mature, full-stack support	Heavy for API-only system	Not selected
Node.js	Event-driven, scalable	Complex dependency management	Not selected

The reason why FastAPI was chosen was that it has great support for AI libraries and the development of APIs. The adoption of Django and Node.js was not done due to the unwanted complexities in terms of system requirements introduced by these frameworks.

Backend development approach is further detailed in [Section 7.2 Backend Implementation](#)

4.4 Database Technologies Criteria

The database layer plays a crucial role in maintaining system data integrity and performance.

Table 51: Database Criteria and Comparison

Criteria	Description
Data consistency	Accurate storage
Scalability	Ability to handle growth
Query performance	Efficient retrieval
Security	Data protection

Comparison of Alternatives

Database	Advantages	Limitations	Decision
PostgreSQL	Reliable, structured data support	Requires schema planning	Selected
MySQL	Easy setup, popular	Limited advanced analytics	Not selected

MongoDB	Flexible schema	Less suitable for structured academic systems	Not selected
---------	-----------------	---	--------------

PostgreSQL was chosen because of its consistency and the possibility of working with structured data. MongoDB was also an option that was discarded since the system needs high-quality data management in terms of relational data management.

Database structure is defined in [Section 6.2 Database Design](#)

4.5 AI Technology Evaluation

AI integration is a key component of the system, supporting automation and intelligent recommendations.

Table 52: AI Criteria and Comparison

Criteria	Description
NLP capability	Chatbot support
Deployment	Local vs cloud
Privacy	Data protection
Cost	Operational feasibility

Comparison of Alternatives

AI Technology	Advantages	Limitations	Decision
LLaMA 3.1 (Ollama)	Local deployment, privacy	Hardware dependency	Selected
OpenAI GPT	High accuracy	API cost, internet dependency	Not selected
Google Gemini	Cloud scalability	Data privacy concerns	Not selected

LLaMA was selected due to privacy, cost efficiency, and offline capability.

Detailed analysis appears in [Section 10.5 External Services & APIs](#)

4.6 Security and Performance Considerations

Security and system performance were integrated into the technology selection process to ensure reliability and protection.

Table 53: Security Stack

Area	Approach
Authentication	Firebase / OAuth
Authorization	Role-based access
Data Security	Encryption
Performance	Lightweight frameworks

Implementation is described in [Section 7.5 Security Controls Implemented](#)

4.7 Final Outcome / Chosen Stack

Based on the evaluation and comparisons conducted, the final technology stack was selected.

Table 54: Final Stack

Layer	Selected Technology	Justification
Frontend	Vue.js / Nuxt.js	Balance of simplicity and scalability
Backend	FastAPI	Efficient API and AI integration
AI	LLaMA via Ollama	Privacy-focused and local deployment
Database	PostgreSQL	Reliable structured storage
Security	Firebase / OAuth	Secure authentication

The selection aligns with system requirements and supports scalability, security, and intelligent automation.

Further technical justification and experimentation outcomes are presented in [Section 10: Technology Investigation & Justification](#)

5. Software Requirements (UPDATED)

The following section shows the revised Functional Requirements of the system as a component of the Technical Implementation Specification (TIS) report. These requirements have been based on the Software Requirements Specification (SRS) and have been narrowed down to the present knowledge of system scope, design factors, and intended technologies of implementation. These requirements give a clear and systematic background to system architecture, design choices and implementation of actions that will be addressed in the further sections of this report.

[Click this link to access the updated SRS Document Link](#)

***Note Changes done in SRS report are included in this report as well for readability.**

5.1 Functional Requirements

The functional requirements determine what the system will do as well as the fundamental services and behaviors that will be provided to the users. The system will support teamwork to project management, coordination of work, tracking contributions, and AI-related assistance. The functional requirements are also grouped into rational functional areas in a bid to improve the clarity, traceability and adjustment with elements of implementation.

5.1.1 User Authentication and Account Management

To ensure that only authorised users can access system features and data, we will introduce safe user authentication and account management capability. Users will be required to be authenticated before connecting with projects, tasks or AI-assisted facilities. The secure session handling mechanism will be adopted to maintain the integrity of the system and protect sensitive information of the users [42][43].

This is a functional area that is contained in registering user, managing user log in/sessions and implementing access controls.

Table 55: User Authentication FR (UPDATED)

Requirement ID	Description
FR-1	We will allow users to register using valid credentials
FR-2	We will allow registered users to log in securely

FR-3	We will manage authenticated user sessions
FR-4	We will restrict access to system features to authenticated users only

This feature will provide controlled access to the system, avoid unauthorised usage and provide the foundation of enforcing role-based permissions in the system.

5.1.2 Project Management

We will also provide the project management feature to allow the organized cooperation among users. The packages of activities, users, and activities involved will be in the form of projects. Access, creation, and management of the projects will be done by the approved users according to their positions.

The project management assists in giving support to the organization, accountability, and ownership in the system.

Table 56: Project Management FR (UPDATED)

Requirement ID	Description
FR-5	We will allow authorized users to create new projects
FR-6	We will allow users to view projects they are assigned to
FR-7	We will allow project details to be updated where permitted
FR-8	We will associate users with projects based on roles

5.1.3 Task Management

Task management capability will also be used to assist in planning, implementing, and controlling project activities. Tasks will be linked with specific projects and assigned to individual users. People will be able to update the data of a task as the work goes on, hence will give real time data on the state of the project.

This functional area helps in the responsibility of allocation, monitoring of the workload, and progress.

Table 57: Task Management FR (UPDATED)

Requirement ID	Description
FR-9	We will allow users to create tasks within a project
FR-10	We will allow tasks to be assigned to users
FR-11	We will allow users to update task status and details
FR-12	We will allow users to view task progress and deadlines

Task management features allow proper coordination and contribute to achieving the objectives of the project within the set-out timelines.

5.1.4 Contribution Tracking and Activity Logging

In order to support transparency and accountability in collaboration, we will initiate the implementation of contribution tracking systems, and activity logging systems. The system will track the user activities on the projects and tasks involving updates and interactions. This data will also be helpful in making contribution, reporting and analysis.

Table 58: Contribution Tracking FR (UPDATED)

Requirement ID	Description
FR-13	We will log user actions related to tasks and projects
FR-14	We will store contribution data for later analysis
FR-15	We will allow authorised users to view contribution records

Tracking contribution can be used to equally assess the personal contribution and can offer an overview of system usage and collaboration trends.

5.1.5 AI-Assisted Support Features

We are going to introduce AI-assisted support options to assist the users in communicating with the system and receiving guidance connected to using the system and project activities. Predefined and approved assistance tasks will be the only AI functionality because they ensure that the use of AI is feasible, reliable, and ethical during the project period. The AI aspect will serve the client and not impersonate the main system decision-making.

Table 59: AI-Assisted FR (UPDATED)

Requirement ID	Description
FR-16	We will accept user queries for AI-assisted support
FR-17	We will generate AI-based responses to supported queries
FR-18	We will restrict AI functionality to approved use cases

This functionality enhances user experience while maintaining control over system behaviour and AI-generated outputs.

5.2 Non-Functional Requirements

Non-Functional Requirements. Non-Functional Requirements determine the quality features, limitations, and working environment of the system. These needs give directions of how the system will work and not what it will do. They make the system secure, efficient and reliable and at the same time scalable and maintainable as its use and functionality increases. The non-functional requirements mentioned in this section are based on the SRS and enhanced to meet events that are to be put in place as discussed later in this report.

5.2.1 Performance Requirements

We will design the system to provide reasonable response times in the normal operating conditions and supporting several users at once. User are found to leave performance requirements. interactions, API calls and response of the system reactive and consistent to the usual operations.

To prevent them, they must optimize API, database design, and backend architecture. performance bottlenecks [44].

Table 60: Performance Requirements FR (UPDATED)

Requirement ID	Description
NFR-1	We will ensure timely responses to standard user requests
NFR-2	We will support concurrent user access without significant performance degradation

Satisfaction of these requirements ensures a seamless user experience and there is no hold-up during. important interreligious interactions

5.2.2 Security Requirements

Security is a non-functional requirement of the system. There are a couple of security controls that will be implemented. to safeguard system information, reject unauthorised access, and to safeguard communication amongst system. components. The security will be enforced both in the frontend and the backend and AI. services [43].

Table 61: Security Requirements FR (UPDATED)

Requirement ID	Description
NFR-3	We will encrypt data transmitted between client and server
NFR-4	We will enforce role-based access control
NFR-5	We will validate user input to reduce security risks

These are implemented to ensure data confidentiality, integrity and access to the system and minimize the chances of data. violations, abuse and bad faith.

5.2.3 Scalability Requirements

We will develop the system in a manner that will facilitate growth in the future. of users, data stored and system functionality without requiring any. radical redesign in the architecture. The strategy depends on the issues of scalability. deployment, system modularity, and technology selection [44].

Table 17: Scalability Requirements FR (UPDATED)

Requirement ID	Description
NFR-6	We will support scalable system deployment
NFR-7	We will use modular system components to enable future expansion

These requirements make sure that the system can scale with time to support more. demand and new features

5.2.4 Reliability and Maintainability

We will ensure that the system will be able to work in different environments in a consistent manner. that it will be sustainable to serve its life. The reliability and maintainability requirements. enable stability of the systems, ease of updates, and maintainability.

Table 62: Reliability and Maintainability FR (UPDATED)

Requirement ID	Description
NFR-8	We will maintain consistent system behaviour across environments
NFR-9	We will support maintainable and modular system design

These requirements reduce system downtime, simplify debugging and updates, and ensure that future modifications can be implemented with minimal disruption.

5.3 User Roles and Permissions

TeamTrack system has Role-Based Access Control (RBAC) model to manage access to system functionality and data. RBAC will assist in offering security, accountability, and fair co-operation in a scholarly community to make sure that each user can access the system within the stipulated scope of the responsibility.

Access controls go both over the frontend interface, and at the API layer to the backend, and unauthorised actions cannot be performed regardless of how a request is spawned.

5.3.1 Defined User Roles

The system identifies four key user roles and each one of them is created to facilitate a particular pattern of interaction in the platform.

Student Team Member

Student Team Members are the main actors in the system, as they have to complete project work and document personal input.

Allowed Actions:

- Projects and dashboards are assigned.
- See tasks, deadlines and milestones.

- Mark the status of tasks and comments on the tasks.
- Time in the books working on something.
- See the personal contribution history and summary.

Access Limitations:

- Is not able to alter project level settings or milestones.
- The user has no ability to delegate.
- Not able to control team membership and user prepositions.

This role allows active involvement and retains the structure and control of the project.

Team Leader

Team Leaders are in charge of arranging the activities of the project and being able to keep track of the progress of the team.

Allowed Actions:

- Full Member Permissions of the Student Team.
- Form and shape project work spaces.
- Assign and reassign tasks Establish and revive milestones and deadlines.
- See team contribution analytics.
- add/ delete project members.

Responsibilities:

- Keep balanced distribution of work load.
- Follow up progress and involvement.
- Make sure project deliverables are done on time.

The position makes coordination and accountability a possibility without automated appraisal systems.

Instructor / Supervisor (Read-Only Role)

The teachers and supervisors will have a read-only access to provide transparency and academic supervision.

Allowed Actions:

- See project status, tasks and milestones.
- Is there access contribution analytics and summaries?
- Look at the logs of activity and history of communication.

Restricted Actions:

- None of the task creation or change.
- None of the project setups are accessible.
- None involved in project communication.
- No user or role management

This position maintains the autonomy of students and allows them to be reviewed academically.

System Administrator

SA is in charge of ensuring the technical integrity of the platform.

Allowed Actions:

- Administer user accounts and configuration of platform.
- Gather performance and health of the monitor systems.
- Uphold security measures and infrastructures.

Restricted Actions:

- No entry to academic project information.
- None of the tasks or records of contribution changed.

This distance does not allow administrative control over the academic content.

5.3.2 Role–Permission Summary

Table 63: Role-Permission summary table

System Function	Student Member	Team Leader	Instructor	Admin
View Projects & Tasks	✓	✓	✓	✗
Create / Edit Tasks	✓ (assigned only)		✗	✗
Assign Tasks	✗	✓	✗	✗
Log Time	✓	✓	✗	✗

View Contribution Analytics	✓ (self)	✓ (team)	✓	✗
Manage Project Settings	✗	✓	✗	✗
Manage Users & Roles	✗	✓	✗	✓ (platform only)

5.3.3 Permission Enforcement Strategy

User permissions have been enforced in the TeamTrack system through a layered security scheme to provide a uniform, reliable, and secure access control to all components of the system. Checks of permission are implemented on several levels to exclude the unauthorised actions despite the manner of a request initiation.

Role validation is applied on every secured endpoint of the API at the backend level. Every request made is verified to have a valid security token and the role of the user is verified and then no operation is done. Any requests failing to satisfy the permissions criteria are in effect denied, and it is impossible to execute restricted actions using the direct API access or by hand manipulating the request.

On the frontend, there is dynamism of the user interface, which considers the role of the authenticated user. Elements of the interface like buttons, menus and configuration options are either visible or invisible based on the permissions that a user has. This minimizes the chance of unintentional misuse and enhances usability through showing only relevant actions to the user depending on his or her role.

Moreover, attempt of unauthorised access is handled in a safe manner and logged to help in auditing and monitoring of a system. These logs will provide an insight into the breaches of access and help trace the trends of the potential security violations or abuse during the system uses.

This stratified policy implies that not only is permission enforcement strong, but also steady and resistant to unintentional and intentional abuse.

5.3.4 Design Rationale

The TeamTrack system academic and collaborative aspects have been selected to resort to Role-Based Access Control (RBAC) model to guarantee the system security and integrity. RBAC also provides a more structured and scaled approach to access control through grouping permissions according to well defined user roles and not by user.

In terms of academics, RBAC can contribute to the effort of ensuring that the project data cannot be manipulated by a third party and only the approved users can implement structural changes such as employee assignment or deadline changes. This assists in establishing equitableness and responsibility in group tasks based course work by preventing unintentional or non-acceptable amendments to joint artefacts of work.

RBAC also enables the contribution to be transparent since the actions are associated with the user roles and user identities. This openness helps in adequate documentation of activities, contribution analytics, which are the primary concern of the system to help improve accountability in shared academic tasks.

The primary reason is the presence of a read-only role of an instructor in that this role enables academic supervisors to check summaries of project progress and contribution without disrupting the daily team activities. This alternative of design will retain student autonomy but will provide a significant control to the instructors.

Lastly, RBAC also facilitates scalability and maintainability in the future. The access control model is flexible and can accommodate the new features of the future as the system advances; new privileges or roles may be added without making significant changes in the system architecture, which implies that the access control model will be flexible and adaptable to the upcoming changes.

5.4 Use Cases

The use cases are the description of the interaction between the users and the TeamTrack system to fulfill a certain goal. They give practical and user oriented perspective into the way in which the system functionality is accessed and implemented; they are a complementary to the functional requirements as identified in Section 4.1.

The use cases specified in this section represent realistic work processes involved in academic projects, are consistent with the role definitions in Section 4.3, and act as a base of system design, UI development, and testing in subsequent sections of this report.

5.4.1 UC-01: User Registration and Authentication

Primary Actor: Student / Instructor

Precondition: It is not the case that the user is an authenticated user.

Postcondition: The user logs in and is taken to his or her dashboard.

Description:

This use case determines the access to the system by a user in a secure manner.

Main Flow:

1. Valid credentials are used to register the user.
2. The information is authenticated by the system.
3. The user is authenticated and logs in.
4. An encrypted session is created.
5. The user-specific dashboard is presented.

Alternate Flow:

- In case of the invalidity of credentials, a system shows an error message and denies access.

5.4.2 UC-02: Create Project Workspace

Primary Actor: Team Leader

Precondition: The user is logged in and possesses the privilege of a Team Leader. **Postcondition:** New project workspace is created.

Description:

This application case will imply the development of an organized project setting.

Main Flow:

1. The Team Leader chooses the alternative to develop a project.
2. Project details are entered.
3. The system is a validation of the input.
4. The working environment is established.
5. Team Leader will be assigned ownership permissions.

5.4.3 UC-03: Join Project Workspace

Primary Actor: Student Team Member

Precondition: This project is there and the user is already authenticated.

Postcondition: A user joins the project workspace.

Description:

This use case enables students to become part of a project that is in place.

Main Flow:

1. The user provides acceptable project join identifier.
2. The system authenticates the identifier.
3. The user is enrolled in the project.
4. Project activities and communications facilities get available.

Exception Flow:

In case of invalidity of the identifier, access is denied and feedback is given.

5.4.4 UC-04: Task Creation and Assignment

Primary Actor: Team Leader

Prerequisite: There is an existence of a project workspace.

Postcondition A task is assigned and created.

Description:

This use case determines the way of planning and delegating tasks.

Main Flow:

1. The Team Leader develops a task that has the necessary information.
2. One of the members of the team is given responsibility.
3. The project task view presents the task.

5.4.5 UC-05: Task Update and Collaboration

Primary Actor: Student Team Member

Precondition: The user is allotted the task.

Postcondition: Task changes are logged.

Description:

This application case assists in cooperative task performance.

Main Flow:

1. Task status or content is updated by the user.
2. The comments are provided where necessary.
3. The update is logged in the system with timestamps.

5.4.6 UC-06: Time Tracking and Activity Logging

Primary Actor: Student Team Member

Precondition: The task is there and it is allocated to the user.

Postcondition: Time and activities information have been stored.

Description:

In this application, personal work is held to account.

Main Flow:

1. The user will activate a timer of a task or deactivate it or input time.
2. Time entry is recorded in the system.
3. The contribution summaries are kept up to date.

5.4.7 UC-07: Contribution Analytics and AI-Assisted Insights

Primary Actor: Team Leader / Instructor

Precondition: There is the contribution data.

Postcondition: Analytics are presented.

Description:

This application case assists the workload monitoring and transparency.

Main Flow:

1. The analytics dashboard is accessed by the user.
2. Summaries and indicators of contributions are shown.
3. Intelligence insights with the help of AI are expressed as suggestions.

Constraint:

- The outputs of AI are informational and do not conduct an automated assessment.

5.4.8 UC-08: Instructor Project Review (Read-Only)

Primary Actor: Instructor / Supervisor

Precondition: The instructor can read-only.

Postcondition: The data in the project is examined, but not edited.

Description:

This application is useful in academic supervision.

Main Flow:

1. The teacher gets access to a work station.
2. Tasks, progress and analytics are checked.
3. There are no actions of modification.

5.4.9 UC-09: Activity Audit and History Review

Primary Actor: Team Leader / Instructor

Precondition: There are activity logs.

Postcondition: The review of historical activity is carried out.

Description:

This use case enables one to view task and project history.

Main Flow:

1. The user gets access to activity logs.
2. Updates on schedules and tasks are shown.
3. Past activities are assessed in terms of transparency.

5.4.10 Use Case Coverage Summary

The specified use cases all show the support provided by TeamTrack to:

- Formal project and activity management.
- Open donation monitoring.
- The non-evaluative insights with the help of AI.
- Lecturer supervision that is not intrusive.

Such applications are directly used to determine system design, UI flows, and test cases as shown in subsequent parts of this report.

6 System Design

6.1 System Architecture

6.1.1 System Architecture Diagram

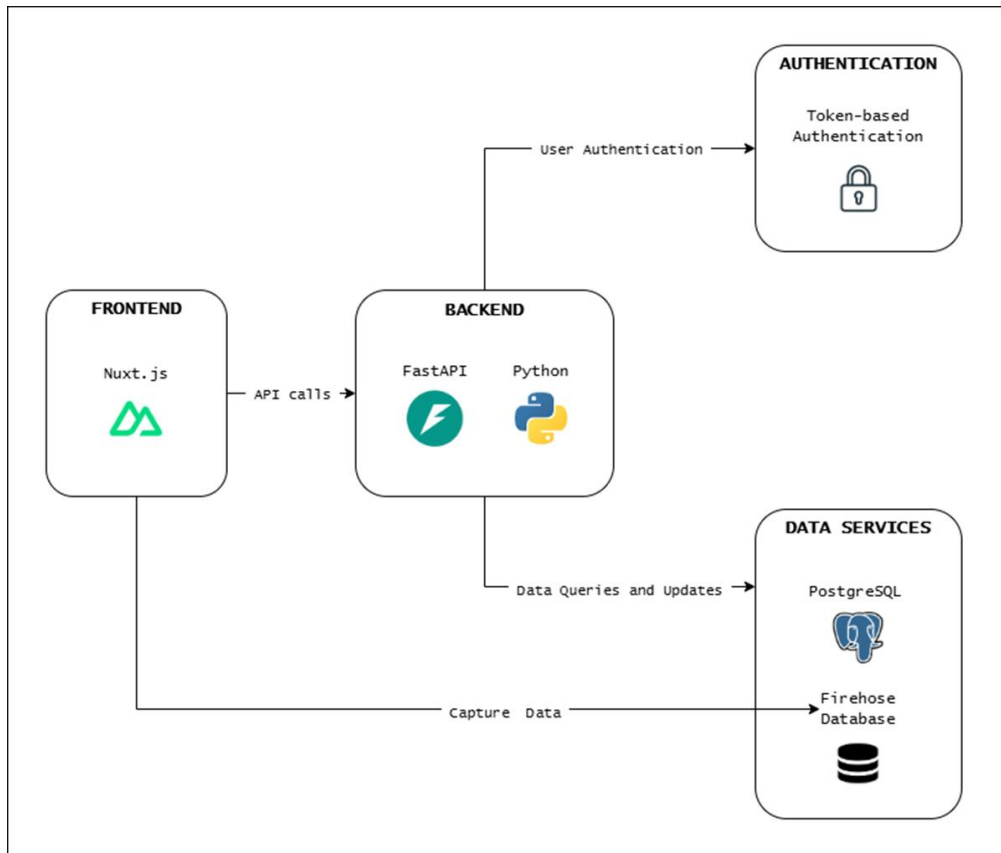


Figure 3: System Architecture Diagram

6.1.2 Architectural Overview

TeamTrack system adheres to Web-based client server development model which is meant to provide isolation of concern, scaling and maintainability. The system has three main layers as described by the high-level architecture diagram, namely frontend client layer, the backend application layer and the data services layer. Such a hierarchical design facilitates trusted communication, modularized development, and restricted system resource access. The architecture fits the technical requirements that have been stated in the project documentation, which stipulates the use of a Python based backend and a relational database management system.

6.1.3 Frontend Layer

The frontend is created with the help of the Nuxt.js, a modern web framework that is chosen because of its high performance, design, and maintainability.

The frontend represents the major user interface point where the students and the instructors will be able to handle the projects, assigning tasks and time, and viewing dashboards of project contributions. Any actions performed by users, including task creation, time registration, chat messages, and dashboard visits, are sent to the backend with structured API calls over HTTP/S. Such a design means that the business logic and data processing is centralized in the backend, and not done at client level.

6.1.4 Backend Layer

Python and FastAPI are used to implement the backend, which is an API-driven service layer with high performance.

The role of the backend is:

- Processing authentication requests
- Role-based access control (RBAC) enforcement.
- Executing business logic
- Task workflow and time logs management.
- Creating the contribution analytics.
- Handling activity logging

This layer is used to validate all frontend API calls and process them before they are sent to the database. This facilitates secure access control, ordered handling of requests and imposing constraints on the system as defined in the SRS.

6.1.5 Authentication Mechanism

The system is configured to do authentication by use of tokens to ensure that only authorized users can access project workspaces. When a user successfully logs in, authentication tokens are created and verified by the backend after which the user is allowed to access secured resources. This mechanism facilitates the secure session management and it conforms with security requirements as in the SRS specifically those relating to authentication, access control as well as session management.

6.1.6 Data Services Layer

The data services layer will be comprised of two different storage mechanisms:

1. PostgreSQL: The main relational database, which will hold the data of transactional system users, projects, tasks, time logs, messages, and also role assignment. The database guarantees referential integrity and transactional consistency because of the system requirements.
2. Firehose Database: A secondary logging and analytics database that stores high frequency activity data (like event of user interaction and time tracking records). This isolation enables the analytical workloads to be handled without the involvement of the essential transactional processes and thus maintains the performance and scalability of the system.

All access to the database is made through the backend layer and access to data is controlled and security enforced.

6.1.7 Data Flow Explanation

Whenever a user performs an action on the site the frontend will make a request to the backend such as logging in, opening a workspace or updating a task. Then the important part is done by the backend: verifying who the user is, implementing the rules of the system, the decision whether the action can be done is made.

PostgreSQL retains regular system data in the form of users, projects, tasks and milestones to ensure that it remains structured and can be recalled whenever the need arises in a short period. Meanwhile, frequently occurring activities (such as updates to activity and time-related logs) are instead sent to Firehose database. This prevents the primary database overload with the constant log-like information.

The system checks the authentication token before any private information is revealed or modified. This ensures that only persons who are supposed to have access to protected areas are allowed. In general, this flow assists the system in maintaining a clean history of actions and contributions, which contributes to transparency and facilitates the way to track the participation within the team.

6.1.8 Architectural Rationale

This design was chosen as it makes the system simple to expand, secure to utilize and manage as the project advances. The work is divided between the frontend, the backend and the databases instead of being contained in one place. That makes all the parts easier to construct, test and modify without disrupting the others.

It is also upgradeable in future. In case the team is interested in enhancing analytics or introducing more sophisticated AI functions, they can be introduced on top of the ready-made system, without replacing the entire system. All in all, this arrangement is on the academic scope of the project, but it should also help to achieve the primary objective of organized collaboration and effective contribution monitoring.

6.2 Database Design (ERD / Schema)

This section explains the TeamTrack system's database architecture and explains how the schema satisfies the functional and non-functional requirements. The design is grounded on a relational data model, it models the entities, relationship between them and constraints that support the features it has (collaboration, task tracking, time logging, activity monitoring). The schema maintains relational integrity, ensures the transactions and handling are dependable, and clears separation between the main system entities like users, projects, and tasks.

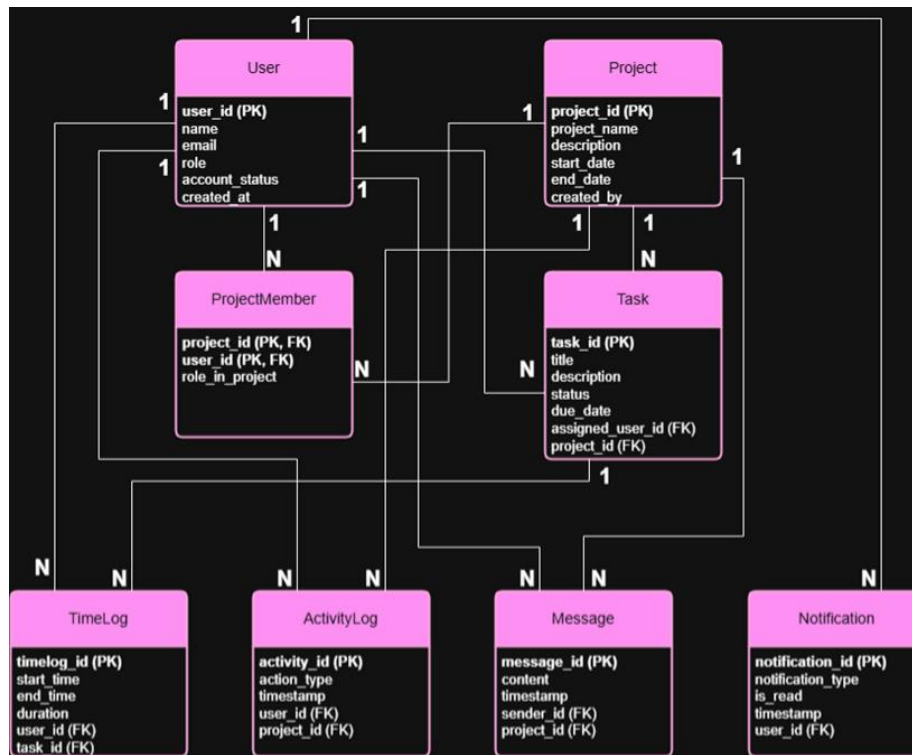


Figure 4: Entity Relationship Diagram (ERD).

6.2.1 Design Rationale

These are the relational modelling principles used by the TeamTrack database to ensure consistency in its design. To keep the data consistent, the database connects the entities that are related to one another with primary and foreign keys. It also prevents incorrect records from getting into the database in addition to keeping the system consistent in the sense that tasks or time logs and any messages must be linked to valid user or project.

- User, Project, and Task are core entities, which reflect the main interactions within the system.
- Since a user can be part of multiple projects and a project can have many users, this relationship is shown handled by the ProjectMember table, this table connects the users to projects, and it also stores each user's specific role in that project (e.g., member or leader).

- Every task is associated with a single user, and it only has a single project, this makes it easy to trace responsibility, timelines and progress of any task clearly.
- TimeLog and ActivityLog are stored as separate tables so that historical records are saved separate from any task data, which supports contribution analysis and keeps the past activity from being deleted if the tasks get updated.
- Message and Notification are stored separately from task and project data as this supports communication and any alert features while keeping the main task formats clean and organised.

6.2.2 Support for System Requirements

The database architecture has been created to accommodate both the functional and non-functional requirements.

- The way time logs and activity records are stored ensures that entries are either saved completely or not saved at all, and this is to support the system requirement that time tracking data remains reliable, and that no inconsistent records are created when multiple users interact with the system at the same
- The relationships that exist between the Users, Projects, Tasks, and Memberships are all maintained through the use of the foreign keys. The use of the foreign keys ensures that the task is always linked to the correct project, the time logs are linked to the correct user and task, and that the project information is never standalone but is always linked to other information.
- Having historical records like time logs and activity logs separately from task data is to ensure that the Contribution Analytics Dashboard and the Activity Log/Audit Trail are supported. The reason for this is to ensure that the information regarding the contributions is traceable and cannot be accidentally modified with the updates to the task information.
- Attributes about role stored within the User and ProjectMember entities support the Role-Based Access Control requirement, because these attributes allow the backend to enforce permissions consistently, so the members, leaders /instructors, can only perform actions permitted by their role.
- The structure also supports performance and reliability requirements since it keeps frequently accessed data (tasks and memberships) separate from historical logs. When retrieving dashboards, analytics reports, or conversation history, this keeps things responsive.

The database structure has overall been able to provide TeamTrack with a reliable foundation as all the information regarding the long-term reliability, the analytics production, the contribution tracking, the authentication, and the access are all provided as required.

6.3 User Flow Diagram

6.3.1 Overview of User Flow

In this subsection, the entire navigation system of the TeamTrack is provided in a user manner perspective. The user flow diagram is used to explain the way users will use the application interface, commencing with the authentication process and continuing with accessing the dashboard, project selection, and functionalities of core systems. The diagram only concentrates on the interface level interactions and leaves out any processing information at the back end.

6.3.2 Authentication and Entry Flow

System access and authentication is the starting point of the user flow. The application is launched by its users, and they submit credentials, and are redirected to the dashboard on successful authentication. If authentication is not performed, and the user is redirected to the login or registration screen. This step ensures limited access to the system until it can be used.

6.3.3 Project Interaction and Core Features

Once authenticated, a user gets access to the dashboard and picks a project or a workspace. From the The project overview page, the users are able to do some basic actions such as creating the task, viewing the task, and performing the task. team communication, contribution analytics, time tracking, and resource management. Every operation will take the user back to the project overview, and there is a structured loop of navigation within the system.

6.3.4 Logout and Session Termination

The final step of user flow is the operation of the logout, to end active session and allow returning the system to its original condition. This will guarantee secure session control and adequate user exit out of the application.

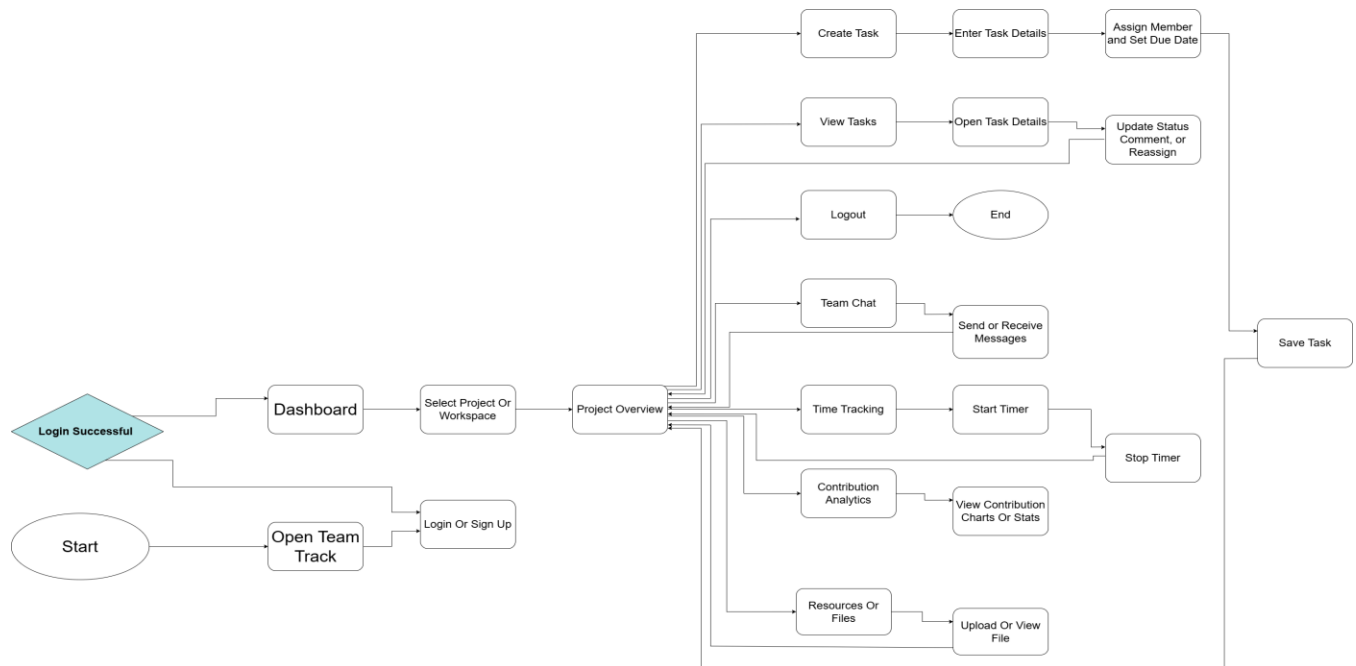


Figure 5: User Flow Diagram

6.4 System Flow Diagram

6.4.1 Overview of System Architecture

TeamTrack internal system architecture will be included in this sub section. The system flow diagram reveals the request and data flow among the system components, including the frontend, backend API, database layer, AI module and file storage services. The scheme provides a general overview. Presentation of the architecture which will be utilized in implementing the implementation strategy in Section 7.

6.4.2 Frontend–Backend Communication

The website developed using Nuxt.js is the frontend and communicates with the backend API through secure HTTPS request. The authentication credentials are authenticated by the backend, which then gives out JWT. credentialed access to further authentication. This is what enables this secured and controlled. system resources to be accessible.

6.4.3 Backend Processing and Database Interaction

The business logic is done in the back-end API, coded in Python and will control the communication with the PostgreSQL database. The fundamental activities of authentication validation, project management, task updates and contribution recording are handled via formal API endpoints. Database communication makes sure that there is an enduring storage and retrieval of structured transactional data.

6.4.4 AI Service Integration

The AI module is a independent fastapi based service which is combined with LLaMA 3.1 language model. The backend puts a user AI request to the AI service that obtains the necessary contextual information and produces structured responses. This is the production that is sent back safely to the front-end.

6.4.5 File Storage and Secure Communication

Objects of storage services are operated with file upload and retrieval operations. The metadata is kept in the database, and the contents of the files are kept in independent management to ensure that the database is scalable and responsive. All communication among components of the system is secured through the use of the encryption of the communication system (SSL/TLS).

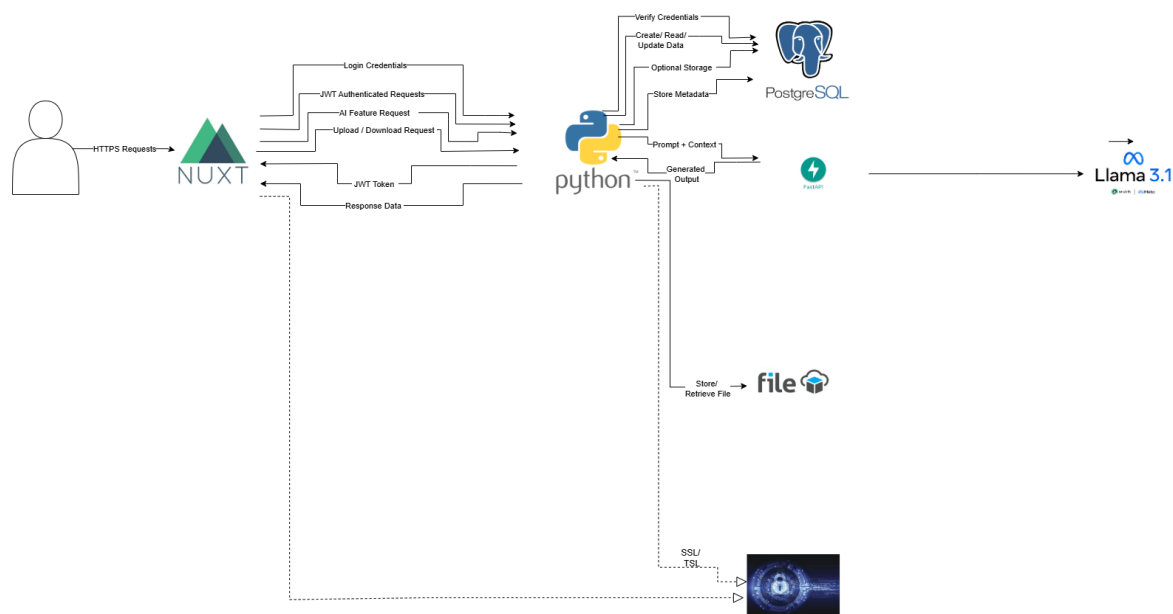


Figure 6: System Flow Diagram

7. Implementation

This section details the implementation strategy which was used in the system. It describes the development procedure, back and front implementation approach, AI implementation, and security measures. Modular system design, scalability, and secure operation are considered because we would also like to make sure that the system is capable of being developed effectively and can be expanded in the following phases [45], [46], [47].

7.1 Development Approach / Methodology

7.1.1 Development Model

We shall be working with an iterative development model, which is guided by Agile principles to implement the system [49]. The development will be done in stages with the initial core system functionality being implemented, and then other features and refinements added to the core system. The latter enables the creation of a start point in the system, which can be extended later with functionality.

System components will be constantly evaluated in each development iteration, and this allows them to detect early problems and take corrective measures in a timely manner [49]. Repetitive development also enables us to integrate the improvement based on the testing results and changing requirements of the project and facilitates flexibility during the implementation process. This type of development model assists in controlled development of systems, minimizes development risk, and enhances the overall quality of the system by keeping every increment tested and validated prior to any further development [49].

7.1.2 Modular Implementation Strategy

We will also use a modular implementation plan where the system is broken down into individual and interconnected modules [49]. This is done by way of developing and testing various system components simultaneously and having well-defined boundaries between responsibilities. Modular design also enhances maintainability and eases system updates because it reduces the effects of modifying particular components. All the modules in the system will be designed to interface with the other components in the system with well defined interfaces, which will facilitate easy integration and system reliability [45]. Scalability is also improved by this strategy, because single modules can be altered or added without making many adjustments on the system architecture [49].

Table 64: System Modules and Responsibilities

Module	Description
Frontend	Handles user interface rendering and client-side interactions
Backend	Administers business reasoning and reveals RESTful API services.
AI Module	Gives natural language processing and analysis.
Database Layer	Stores organized information and records and event system.
Security Layer	Authentication and encrypted communication.

7.1.3 Version Control and Environment Management

The main control tool that we will use to facilitate the work of the team and the management of the source code is Bitbucket. Bitbucket will allow us to follow the changes in the code, have many branches of development, and have a clear history of alterations during the development process. This facilitates regulated work and minimizes the chances of code clash during implementation [50].

The development and execution environment will be standardised with the help of Docker, so that the configurations remain the same at the development, testing, and deployment phases [47]. All the system components such as the frontend, backend services, databases and AI module will run in isolated containers whose dependencies, configurations and environment variables are clearly defined.

The union of both strategies enhances the reliability of the development process because it reduces any environment-related problems, eases the dependency management and facilitates the integration process of components of the system with ease [47], [50]. It also improves maintainability since the system will act in the same way using various development machines.

Table 65: Version Control and Environment Management Tools

Tool	Purpose	Description
Bitbucket	Version Control	Manages source code versions, tracks changes, and supports collaborative development
Docker	Environment Management	Provides containerised environments with consistent configurations
Containers	Component Isolation	Separates frontend, backend, database, and AI services for reliable integration
Environmental Variables	Configuration Management	Manages service configurations without hardcoding sensitive values

7.1.4 Implementation and Testing Progression

We will implement the system in a progressive and gradual manner, with the initial system being the simplest. functionality and afterwards we can always add more [8]. The first one will be the development of. focused to develop the API architecture of the backend, database connectivity, and required. frontend views have a solid foundation on which the development will be implemented. As soon since these core features are already in existence, we will slowly start to add other features such as activity logging. and AI-assisted support. It will entail cross-module integration and module development. Every module, like frontend, back-end services, AI module and database layer will be created and developed. tested on individual basis and then inter-integrated with the other modules [4], [7]. The modular approach reduces the development complexity, increases maintainability, and helps in the early identification and correction of implementing issues. The testing activities will be conducted in the process of implementing the system to provide system behaviour. and functional integrity [4]. This will entail the verification of backend API endpoints, verification of database. operations, checking the proper frontend interactions, and checking the reliable system communications. The primary test in this stage will be a functional validation test and integration stability test. because exhaustive performance or stress test can be absorbed during the further development. Early on, the system needs to be constantly tested which helps to get the system refined as it progresses. contributing to the verification of such integrated components to work effectively as the system grows up [8]

Table 66: Implementation and Testing Stages

Phase	Focus Area	Description
Initial Setup	Core System Foundation	The backend API framework, database connectivity, and simple frontend views will be put in place to create a stable development base.
Module Development	Independent Components	We will create and test single modules, i.e., frontend parts, backend services, AI functionality, and database activities.
Integration	Cross-Module Interaction	We shall combine system components step by step to give a satisfactory communication between the frontend, backend, AI module, and databases.
Functional Testing	Behaviour Verification	We will test API responses, database interactions, frontend interactions, and system interactions to be functionally correct.

Refinement	Issue Resolution	As the development moves forward, we will resolve identified issues and optimize the functionality of the system.
------------	------------------	---

7.1.5 Operating Environment

The system will be developed and run in a containerized operating environment to provide similarity in the development and testing phases [47]. The system components will be encapsulated with Docker, and the application is going to be runnable without being tied to the underlying operating system of the host and minimize compatibility issues with the environment. We will also deploy back-end services that consist of the FastAPI application and the AI module into Linux-based Docker containers [45], [47].

Database services like PostgreSQL and firehose database will also run in sealed containers allowing it to be configured to make things very simple to manage and ensure communication between system components is reliable [48]. The frontend application implemented with Nuxt.js will be served with the help of regular web browsers and will be compatible with the modern operating systems, such as Windows, macOS, and Linux [46].

The frontend and backend communication will be secured using HTTPS that allows the transmission of the data safely when the system is in operation [47]. This operating environment facilitates modular system design, scalability, and maintainability with a clear separation of the application layers without any interaction with the overall system architecture being compromised [47].

Table 67: Operating Environment Summary

Component	Operating Environment	Description
Frontend	Web Browser (Windows, macOS, Linux)	User-facing interface accessed via modern browsers
Backend API	Linux-based Docker Container	Handles RESTful requests and business logic using FastAPI
AI Module	Linux-based Docker Container	Processes AI queries and generates system insights
PostgreSQL Database	Docker Container	Stores structured transactional system data
Firehose Database	Docker Container	Captures high-frequency activity and event data

Security Layer	HTTPS with SSL Certificates	Ensures encrypted communication across system components
----------------	-----------------------------	--

7.2 Backend Implementation

7.2.1 Backend Technology Stack

Python and FastAPI will be used to implement the backend because they are high-performance, scaled, and have excellent support of the modern API development [45]. The business logic, data processing and communication between the frontend, databases and AI module will be the main part of the system on this backend layer.

Fast API provides asynchronous processing of requests, which means that it supports the efficient processing of numerous simultaneous requests [45]. This would be appropriate in supporting real-time interactions like user actions, update of tasks, and logging of activities. Python can also be easily integrated with AI services, which can facilitate the scalability of the future systems [45].

Table 68: Backend Technology Stack

Component	Technology Used
Programming Language	Python
API Framework	FastAPI
Relational Database	PostgreSQL
Event Storage	Firehose Database
Containerisation	Docker

7.2.2 API and Business Logic Implementation

FastAPI will be used to provide RESTful API endpoints to core system functionalities, such as user authentication, project management, task handling, and contribution tracking [45]. These endpoints will be based on the principles of REST in order to have a consistent structure of requests, predictable responses, and easy integration with the frontend elements. A structured service layer will be used to perform business logic, with API request processing and core application logic being separated [45]. Such separation of concerns enhances maintainability, increases the debugging and testing of systems and enables system functionality to be extended

without having to make substantial modifications to the existing API endpoints. The input validation mechanisms and controlled error handling mechanisms will be used to ensure reliable and predictable behaviour of the system [45]. Success and failure states will be communicated clearly to the frontend with the help of standardised responses, which will help facilitate healthy interaction of system components.

7.2.3 Data Management and Storage

PostgreSQL will be our main relational database containing structured transactional information like user accounts, projects, tasks and permission to access information [48]. PostgreSQL is chosen due to its dependability, data integrity and complex relationships. Moreover, high-frequency data on activities such as time tracking records and user interaction events will be captured using a Firehose database. This isolation enables the analytical and logging workloads to be processed without affecting the response of the core transaction database [48]. This compound arrangement of these databases assists in operational functionality as well as scalable analytics processing.

7.3 Frontend Implementation

7.3.1 Frontend Framework Selection

The frontend will be built with Nuxt.js, a Vue-based framework with support of components development and server-side rendering [46]. Nuxt.js is chosen because it enhances the work of the applications, allows them to handle the efficient routing, and promises the organized and well-maintained frontend structure. The model enables us to create user interface elements that are scalable and at the same time provides easy navigation and quick loading of pages [46]. Its modular design allows maintaining a clear separation of concerns in the frontend, which allows expansion of features in the future and improvements of the interface without significant changes to existing codebase.

Table 69: Frontend Technology Overview

Component	Technology	Purpose
Frontend Framework	Nuxt.js	Builds scalable, component-based user interfaces
Rendering Strategy	Server-Side Rendering (SSR)	Improves performance and initial page load
Routing	Nuxt Built-in Router	Manages page navigation and view structure

UI Components	Vue Components	Ensures reusable and consistent interface design
---------------	----------------	--

7.3.2 User Interface Design

The user interface will be designed in such a way that it is highly focused on the clarity, usability and consistency to provide a positive user experience [46]. The system will offer user-friendly dashboards, project views, and analytics summaries which enable individuals to view the progress of the project, the status of tasks and individual contributions with ease. User interface components that can be reused will be put into place to ensure the desired visual look throughout the application [46]. This component model minimizes repetition and maintaining it is also not difficult. and even the experience to user is standardized in different pages of a system too. interactions of the user.

Table 70: User Interface Design Elements

UI Element	Description
Dashboards	Display project status, task progress, and user contributions
Project Views	Present detailed information about projects and associated tasks
Analytics Summaries	Provide visual insights into performance and activity
Reusable Components	Maintain consistent layout, styling, and interaction patterns

7.3.3 Frontend–Backend Communication

The frontend will be linked to the back end services through the RESTful APIs through HTTPS that is safe and trustworthy concerning data interchange [45], [47]. The API requests will be structured in a manner that they will ease the effective retrieval of the data and real-time update where applicable. To avoid the use of the system resources by the unauthorised users we will also apply token based authentication in ensuring that only the authorised users may access the system resources [45], [47]. The system will be secured with the help of the mechanism, and the interaction between the frontend and the backend parts will be efficient.

7.4 AI / Chatbot Module

7.4.1 AI Architecture Overview

The AI module will exist as an autonomous service, which will be developed with FastAPI. independently of the core backend system. This geometric division is easier to maintain, promotes modularity and allows functionality to do with AI to be upgraded or scaled without impacting basic application services. We can ensure the computationally-intensive AI processes do not get in the way by isolating the AI service. Responsiveness in back-end, or responsiveness in front-end. The AI module will communicate with the backend through the secure RESTful APIs, which will enable the controlled access to. data regarding the systems of interest without compromising the overall stability and security of the system. The design facilitates the clear architecture of AI service and core functions. system, such that interaction can be assured, and system evolution may be controlled.

Table 71: AI Module Architecture Components

Component	Technology	Description
AI Service	FastAPI (Python)	Hosts AI logic and exposes AI-related API endpoints
Language Model	LLaMA 3.1	Processes natural language queries and generates responses
Backend Interface	RESTful APIs	Enables controlled communication with the core backend
Containerisation	Docker	Isolates AI services for stability and scalability

7.4.2 Language Model Integration

We are going to incorporate LLaMA 3.1 in order to make the system to be able to interact with natural language. This language model will help the users to make queries regarding the data of the project including task progress, contribution breakdown and activity insight by providing a user-friendly input in the form of natural language. The AI service will use backend APIs [45], [48] to retrieve relevant information stored in system databases, the data will be processed in the language model and provide context-aware responses to users. To maintain the reliability of the system behaviour and exclude cases of unauthorised access to sensitive information, AI interactions will be confined to a set and controlled use cases [47]. By integrating the solution, the

user experience is enhanced with easy-to-understand AI-generated information and does not affect interactions, data security, and system integrity [47].

Table 72: AI Interaction and Data Flow

Stage	Description
User Query	User submits a natural language query through the frontend
API Request	Frontend forwards the query to the AI service via secure APIs
Data Retrieval	AI service requests relevant data from the backend
AI Processing	LLaMA 3.1 processes the query and contextual data
Response Delivery	AI-generated response is returned to the user

7.4.3 AI Data Flow

We will take a systematic AI data flow so that the natural language queries are handled safely, effectively and in a manner that is controlled [47]. The AI module will have a definite set of steps, whereby user input will be processed and finally the AI-generated response will be forwarded to the frontend. Upon a user putting in a query in natural language, the query will be sent to the AI service using secure APIs [47]. The AI module will then only retrieve the relevant and authorised data only needed to process the query [48]. The language model will analyse this data and come up with a contextual response of the given information. The resulting response will in turn be sent back to the frontend to be displayed to the user. Such a data flow guarantees the quality interaction of AIs without affecting system performance, data integrity, and security [47].

Table 73: AI Data Flow Process

Step	Description
User Query	We will allow users to submit natural language requests through the frontend
Data Retrieval	The AI service will fetch relevant and authorised data from system databases
AI Processing	LLaMA 3.1 will analyse the query and generate a contextual response

Response Delivery	The AI-generated answer will be returned securely to the frontend
-------------------	---

7.5 Security Controls Implemented

7.5.1 Secure Communication

We will protect all system communications with the help of the SSL certificate, and the data that is conveyed between the frontend and the backend and the AI services will be encrypted [47]. This prevents the possibility of interception of sensitive information such as authentication credentials and user-related data or their access by un-authorised persons during transmission. The HTTPS will provide integrity and confidentiality of data in addition to creating trusted communication channels among all the parts of the system [47]. All API interactions will be enforced with secure communications to ensure uniform system protection.

7.5.2 Authentication and Access Control

To control user sessions and to ensure authorised access to system resources, we will use token-based authentication to control system resources [45]. Upon successful log in, authentication tokens will be issued that will be checked each time an API request is made to the protected API and only authenticated users will have access to restricted functionality. Besides that, we will use the role-based access control (RBAC) to limit the system activities depending on the user role [47]. This will make sure that only the features and data that the user is assigned to will be visible to the user and this will make the system much safe as far as unauthorized actions are avoided.

Table 74: Authentication and Access Control Mechanisms

Mechanism	Description
Token-Based Authentication	Verifies user identity for each protected request
Role-Based Access Control	Restricts functionality based on user roles
Session Validation	Ensures active and authorised user sessions
Access Policies	Defines permissions for system resources

7.5.3 Application and Infrastructure Security

To minimize security risks, we will use several application level and infrastructure level security measures [47]. These are aimed at scrutinizing the inputs of the system, handling responses to errors safely, and isolating the components of the system to reduce the effects of the possible vulnerabilities. The isolation of system components will be done via container isolation through Docker that minimizes the attack surface and eliminates the potential impacts of a system failure on other services [47]. The services will be restricted through controlled access regulations that will ensure that only authorised operations are executed.

Table 75: Application and Infrastructure Security Measures

Security Measure	Purpose
Input Validation	Prevents malformed or malicious requests
Error Handling	Avoids unintended information leakage
Docker Isolation	Reduces attack surface and limits impact of faults
Access Restrictions	Limits unauthorised actions across system components

8. Testing and Validation

The proposed project needs to be tested and proven because this will prove that the proposed system will be used accordingly, deliver the necessary outcomes, and be stable in the field. Regarding the fact that the system is expected to help the collaborative use, manage the tasks, and provide the systematic interaction between the users and stakeholders, it is suggested that the system should not only be tested in the context of the technical functionality but also in the context of usability, performance, and reliability.

Here, the systematic testing that will be adopted within the project is detailed, and real test conditions are provided, which are intended to check the behaviour of the system under different conditions of operation.

8.1 Proposed Testing Approach

Testing this project is systematic and repeated, and guarantees the continuous enhancement of the results and the recognition of the issues as quickly as possible. Rather than testing at the final phase, the process of validation will be performed at the development life cycle. This will reduce

risks, enhance the stability of the system, and it will also be palatable to the demands of the stakeholders.

The proposed testing approach focuses on verifying:

- Functional correctness of the system
- User interaction and usability
- System performance and responsiveness
- Data accuracy and integrity
- Security and access control

Testing will be conducted using both technical evaluation and user-based validation to ensure the system performs effectively in realistic scenarios.

8.1.1 Testing Strategy Phases

The project will adopt the following structured testing phases:

1. Unit Testing

The individual modules will be unit tested in a bid to possess a way of determining that all the features are working well individually. This comprises of system elements of verification of authentication, task management, data processing and interface interaction. This will fix all the reasoning mistakes or inconsistencies existing and integrate them.

2. Integration Testing

Integration testing will follow after every module has been tested in order to ensure that no communication error exists between modules, such as the user interface, the backend services, and the database systems. This is to ensure that the workflows of the systems run freely in case there is an instance of two or more modules interacting with each other.

3. System Testing

The working solution will be the application, which will be established by using system testing. This is to conclude that all is running in tandem as it is expected, and the system is meeting the overall project requirements. This consists of performance testing, stability testing and workflow validation.

4. User Acceptance Testing (UAT)

User Acceptance Testing will engage the real users, the team members, and the stakeholders of the system. The response that will be received at this stage will be the basis of the usability, practicality, and relevancy of the solution. This will make sure that the system is not out of date with the real-life expectations, but only the technical assumptions.

5. Performance Testing

The system shall be put into test by simulation workloads to verify its capacity in supporting other users and a huge amount of data. Measures of the performance indicators will comprise the response time, speed of processing and system stability.

6. Security Testing

The aspects related to security validation will be the access control, the dependability of the authentication, and the safety of the user data. The privacy and integrity of information will be assessed by measuring the attacks of unauthorized access and vulnerability to the system.

Key Objectives of the Testing Approach

- Ensure system functionality meets defined requirements
- Identify defects and usability issues early
- Validate user experience and workflow efficiency
- Maintain system reliability under different conditions
- Protect system data through security mechanisms
- Support continuous improvement through iterative feedback

This structured approach ensures the system evolves into a stable and reliable platform before final deployment.

8.2 Test Scenarios and Outcomes

It has been constructed to put the system into test on various test situations that are realistic to determine the effectiveness of the system. These are normal interactions that will be undertaken by the users of the system. This is to ensure that the behavior, reliability, and user experience of the system have been established to be accurate in a number of conditions in which the system is used.

The test scenarios are composed of the description of the scenario, the expected outcome, and the validation of the scenario. These scenarios will be used to educate the evaluation process and give calculable data on the performance of a system.

8.2.1 Test Scenario Matrix

Table 76: Test Scenario Matrix

Test ID	Test Scenario	Description	Expected Outcome	Actual Outcome	Status
TS-01	User Registration	A new user registers on the platform using valid credentials	User account successfully created and stored in system database	To be tested	Pending
TS-02	Login Authentication	Registered user attempts to log in using credentials	System verifies identity and grants secure access	To be tested	Pending
TS-03	Task Creation	User creates and assigns a task within the system	Task stored and visible to relevant users	To be tested	Pending
TS-04	Collaboration Activity	Multiple users interact and update project tasks	Updates reflected without data conflict	To be tested	Pending
TS-05	Data Retrieval	User retrieves stored project information	Accurate and timely retrieval of data	To be tested	Pending
TS-06	Error Handling	User inputs invalid data	System generates warning and prevents incorrect action	To be tested	Pending
TS-07	Performance Load	Several users access system simultaneously	System remains responsive and stable	To be tested	Pending

TS-08	Security Validation	Unauthorized access attempt	Access denied and logged securely	To be tested	Pending
-------	---------------------	-----------------------------	-----------------------------------	--------------	---------

8.2.2 Expected Outcomes

The testing phase will produce a certain amount of measurability, which will justify the availability and stability of the systems. The findings will present information that the suggested solution has the capability of upholding real-life project completion and achievement of objectives.

Expected results include:

- Predictable system reliability in various user conditions.
- Proper implementation of each functional need.
- Better interaction with the end user and easy navigation.
- Storage and retrieval of system information are reliable.
- Proper management of wrong inputs and unanticipated user behaviors.
- Authentication and security on user information.

Also, the performance testing is supposed to indicate that the system is capable of accommodating various users without any serious delays or failures. This is necessary to provide scalability and longevity.

User feedback collected during testing will also play a significant role in refining system features and improving overall experience.

8.2.3 Validation Strategy

This will be validated to ensure that the solution that is developed can address the problem that was identified in the earlier stages of the project. Whereas the purpose of testing is to ascertain system functionality, the purpose of validation is to determine the usefulness, utility, and effects of the system. This shall be through a validation process, both technically and in regard to the stakeholders.

Validation Activities

- Comparison of system functionality with initial project requirements.
- Assessment of the effectiveness of the system in addressing the problem identified.
- Daimler-Gathering feedback with users, team members, and stakeholders.
- Evaluating the growth of task efficiency and teamwork.
- Evaluating accessibility and system usability.

- Determining shortcomings and improvement opportunities in the future.

Validation Methods

The solution will be validated with the help of the following methods:

- User Feedback Sessions: - It involves real user interaction and getting suggestions.
- Performance Metrics: The response time, efficiency in completing the tasks, and reliability.
- Requirement Traceability: System features to the initial project requirements.
- Stakeholder Review: system usefulness judgement in terms of the client and the project supervisor.

Validation Outcomes

Validation of the process is supposed to ensure that:

- The system fulfills project requirements and goals.
- The solution enhances efficiency and collaboration at work.
- The system is convenient to the users.
- The platform is reliable and scalable.
- The solution is appropriate to be developed further and implemented.

The validation outcomes would be used to make amendments and modifications in future project milestones, especially the improvement of the system, adding features, and deployment preparedness.

8.3 Issue Resolution and Refinement Strategy

TeamTrack system is based on a methodical and repetitive process of issue resolution and refinement so that defects, usability issues, and functional inconsistencies identified in the process of testing are resolved in a controlled and systematic way. This is a strategy that contributes to the constant improvement of the system and corresponds to the iterative approach to development, outlined in Section 6.

The activities of issue resolution are implemented throughout the development lifecycle instead of being a part of one test phase. This method allows to detect the issues early, minimize the possibility of lack of their resolution, and contribute to gradual improvement of system functionality and usability.

8.3.1 Sources of Issue Identification

Problems can be found in the course of several testing and review procedures, such as:

- Implementation of pre-established functional and integration test scenarios.
- Walkthroughs in a manual system by members of the development team.
- Representative user feedback on User Acceptance Testing (UAT).
- The test of cross modules between frontend, backend, database, and AI modules.
- Development and testing Review of system logs and error responses.

The system has the advantage of seeing both technical and user oriented views in the refinement process by gathering the issues on several sources.

8.3.2 Issue Documentation and Logging

All issues identified are recorded in form of documents in order to be traceable and accountable.

Issue records include:

- A brief definition of the problem observed.
- The component of the system that is impacted (e.g., frontend, backend API, database, AI module).
- Guidelines needed to reproduce the problem.
- The behaviour of the system expected and actual behaviour.
- The role under which the issue occurred of the user.
- The date of identification and status presently.

This documentation ensures that problems are well comprehended and the solution to these problems is well monitored throughout the process of problem resolution.

8.3.3 Issue Classification and Severity Levels

The issues are divided in accordance with their effect on the system functionality and user experience to aid the priorities:

- **Critical Issues:**
Problems which deny entry to the system or essential functioning, including authentication errors, information loss, or improper permission checks. These are issues prioritised that should be resolved at hand.
- **Major Issues:**
Problems that have serious impact on usability or correctness, but do not completely

prevent system functionality, e.g., false updates to task state or false or incorrect contribution summaries.

- **Minor Issues:**

Cosmetic or low-impact problems, such as layout inconsistency or non-critical error messages, that have no impact on the fundamental system behaviour.

This classification enables development effort to be concentrated on solving the high-impact issues first whilst minor refinements are done where time allows.

8.3.4 Resolution Workflow

When an issue is prioritised it has a workflow of resolving:

1. **Root Cause Analysis:**

The root cause of the problem is examined to determine whether it is caused by logic errors, integration problems, configuration problems or limitations to usability design.

2. **Corrective Implementation:**

Specific fix is introduced in the affected module, with less effects on other unrelated parts of the system.

3. **Verification Testing:**

This problem is tested again with the same reproduction steps to make sure that the problem has been solved.

4. **Regression Testing:**

Other related functionality is also tested to confirm that the fix has not brought about new defects or undesired behaviour.

5. **Issue Closure:**

After being confirmed, the problem is checked as solved and registered.

This workflow makes sure that the fixes are valid, tested and do not cause harm to the stability of the system.

8.3.5 Iterative Refinement and Improvement

Besides correction of defects, issue management process aids in the continuous refinement of the system. The refinement activities are concerned with:

- Enhancing workflow and usability.
- Improving uniformity between system interfaces.
- Improving the handling of errors and user feedback messages.

- Enhancing system responsiveness and stability.

Feedback obtained in the process of testing will be analyzed and included in the next round of development. This process of constant refinement guarantees that the system will change according to the actual usage data and testing results as opposed to being fixed as it was originally.

8.3.6 Change Tracking and Documentation

Any issues solved and refinement measures taken are recorded in order to have a clear record of system change. This documentation is in support of:

- Openness in decisions made in development.
- Issue to fix to system behaviour traceability.
- Simpler maintenance and subsequent improvement planning.

This record allows keeping track of the evolution of systems because this aspect can be checked and explained in the course of assessment and development further.

8.4 User Acceptance Testing (UAT)

To test how the TeamTrack system performs under the demands of the realistic project workflow in academia, the User Acceptance Testing (UAT) is performed to determine whether the system can meet the expectations of the targeted users and supports realistic academic project workflows. Contrary to technical testing activities, which are concerned with being correct and integrated into the system, UAT is concerned with usability, clarity of interaction and overall fitness of purpose in the eyes of end users.

The main purpose of UAT is to confirm that the users are able to execute the necessary functions like managing projects, doing the assigned work, monitoring the contributions and reviewing the progress without facing critical problems or misunderstanding. This makes sure that the system is demonstrably, assessment and academic friendly.

8.4.1 UAT Participants and Context

The testing of User Acceptance is conducted by the representatives of the leading user roles in the system, namely the Student Team Members, Team Leaders, and Instructors who work in the read-only mode. Each participant does their interaction with the system with permissions that are valid according to the role they are doing so that the testing is realistic in terms of usage.

Testing is done under controlled environment which resembles the way a system is normally operated. The participants are also given access to sample projects and tasks where they can assess the system behaviour by using scenarios that are similar to the real coursework tasks.

8.4.2 UAT Scope and Coverage

User Acceptance Testing aims at all the basic features to be used in collaborative academic project management. These are system access and authentication, role-based permission enforcement, project and tasks management workflow, time tracking and activity log, and contribution analytic presentation.

AI-assisted support features are also assessed where necessary to make sure that responses are pertinent, comprehensible, and limited to predetermined applications. Low-level technical testing is also avoided, but its purpose is to guarantee that the system meets user interaction, comprehension of workflow, and general system usability.

8.4.3 UAT Scenario Execution

The scenarios of User Acceptance Testings are based on the use cases specified previously in the report and are performed in a serial order by participants. Such situations involve the creation and management of project workspaces, assigning and updating tasks, time on activities recorded, and summaries on contributions provided by the student and instructor sides.

In the execution, the participants monitor system behaviour and record deviations with regards to the anticipated results. Special emphasis is put on the question of the workflow intuitive nature, the clarity of system feedback, and the ability of users to perform tasks without extra instructions.

8.4.4 Acceptance Criteria

The team track system is said to be good when all the required user processes can successfully be done without critical errors. Role-based access controls should be strictly adhered to in order to make sure that a user can only perform the actions that are relevant to the specified role.

Information typed in testing should be properly stored and should be shown in system views and system response should be understandable and foreseeable.

Any problems we find which prevent us to successfully complete the task or which seriously affect usability are considered acceptance failures and resolved by the issue resolution process outlined in Section 7.3.

8.4.5 Feedback Collection and Refinement

The feedback gathered in the course of User Acceptance Testing is assessed to find out the usability issues, non-efficiency, or small functional enhancements. Instead of viewing UAT as a

terminal stage, feedback is used in the iterative refinement of the system where possible, so that the latter keeps becoming better without losing the ability to remain stable.

The refinements brought about through a UAT are recorded, applied and tested to ensure that the refinements eliminate the concerns raised and that they do not create new problems in other parts of the system.

8.4.6 UAT Documentation and Evidence

All the User Acceptance testing activities, observations, and results are recorded in order to promote transparency and traceability. This involves documentation of implemented situations, detected problems, and concurrence of settled issues. Appendix C provides supporting evidence including screenshots and completed test records to prove that the system has been validated at the user-level in an ordered way.

9. Project Management

The Technology Infrastructure Specification (TIS) stage of project management was managed using formal stakeholder engagement and structured coordination coupled with Agile documentation practices. Since the TIS was in need of constant improvement of infrastructure elements, architecture of the system, and supplementary technologies, structured communication and recorded decision-making were necessary.

In this section, the management practices that have been applied in the preparation of the TIS will be outlined in terms of communication planning, Agile coordination, and formal meeting documentation. The activities made sure that there was consistency throughout the document, traces to technical decisions, and consistency with client expectations.

9.1 Communication Plan

The drafting of the Technology Infrastructure Specification had to be in a coordinated manner among the team members and the client to the extent that there was clarity in the infrastructure planning and precision in documentation. The architecture of communication was designed to facilitate the decision-making process, tracking of the documentation, and ensuring the architectural elements.

The communication plan made sure that all the technical responses, infrastructure selection, and documentation updates were kept in order and traceable through the entire TIS stage.

Communication Objectives

The communication strategy was formulated so as to accomplish the following:

- Keep the infrastructure planning and client expectations in line.
- Make sure that technology decisions are properly understood by team members.
- Support coordination: Co-ordination of documentation effort amongst numerous contributors.
- Empower clarification of architectural conditions and constraints.
- Ensure openness to project development and task responsibility.

Communication Channels

Formal and informal channels of communication were used to carry out communication to achieve efficiency and responsiveness.

- The physical client meetings took place in Curtin University during architectural validation.
- Internal planning and coordination through online meetings through Google Meet.
- Real-time updates and clarifications in WhatsApp group.
- Common repository of documentation of versions and joint editing.

Table 77: Communication Governance Structure

Stakeholder Group	Communication Medium	Primary Purpose	Communication Nature
Client	Physical meetings	Validation of infrastructure and architecture	Formal
Development Team	Google Meet	Planning, coordination, progress monitoring	Structured
Development Team	WhatsApp Group	Quick clarifications and updates	Informal
Project Team	Shared repository	Documentation sharing and revision control	Continuous

This form of organized communication has guaranteed that all the planning activities in the infrastructure remained synchronized and the decisions were always conveyed throughout the TIS document. The communication plan allowed the stakeholders and team members to work with the required clarity to minimize the ambiguity of infrastructure decisions and help develop the documentation evenly over the course of the TIS phase.

9.2 Agile Development

The Technology Infrastructure Specification was prepared by adopting agile coordination practices to facilitate iterative development of documentation, flexible technology assessment, and ongoing team members collaboration. Agile approaches are what provided flexibility without losing the form of progress, as the planning of infrastructure was further developed according to the research and the feedback of the client. Instead of viewing the TIS as a static document, it was created on a continuous basis, with the infrastructure components continuously being refined and proven.

Agile Implementation Approach

The Agile framework applied during the TIS phase focused on documentation-driven development rather than software implementation. The following principles guided the process:

- Incremental development of documentation sections.
- Continuous evaluation of infrastructure technologies.
- Iterative refinement of architectural design.
- Integration of feedback from team discussions and client meetings.
- Monitoring of progress through structured coordination.

Sprint Planning and Task Structuring

Documentation activities were organised in sprint-style cycles to ensure measurable progress and balanced workload distribution.

Key sprint planning practices included:

- Identification of documentation deliverables for each iteration.
- Allocation of responsibilities based on expertise and section ownership.
- Setting of short-term completion targets.
- Identification of dependencies between infrastructure components.
- Review of previously completed sections.

Coordination and Synchronization Practices

Regular coordination ensured continuity between documentation activities and technology planning.

- Weekly coordination meetings conducted via Google Meet.

- Continuous progress updates through WhatsApp communication.
- Resolution of documentation and research blockers.
- Alignment of architectural decisions across team members.

Table 78: Agile Roles and Contributions

Role	Responsibilities	Contribution to TIS Development
Team Lead	Coordination of documentation and planning	Ensured alignment across infrastructure decisions
Documentation Contributors	Preparation of TIS sections	Maintained content accuracy and structure
Research Contributors	Technology evaluation and comparison	Supported justification of infrastructure choices
Client	Feedback on architecture and feasibility	Validated planning direction

Monitoring and Progress Control

Progress was monitored using structured tracking mechanisms to maintain accountability.

- Section completion checkpoints
- Monitoring of assigned responsibilities
- Tracking of meeting action items
- Continuous updates following coordination discussions

Agile helped to plan and improve in an adaptive way throughout the preparation process of the TIS. The process of refining infrastructure decisions numerically led to the validation of decisions and a systematic documentation of infrastructure decisions, which leads to a coherent and technically sensible specification.

9.3 Meeting Minutes

Formal documentation of discussions, decisions, and action items that emanated during stakeholder and team interactions was done through meeting notes that were meticulously kept during the TIS phase. These inventories acted as a very critical source of reference to infrastructure planning and documentation congruency. Documentation of meetings made the project management activities transparent, accountable, and continuous.

Purpose of Meeting Documentation

Meeting minutes supported the project by:

- Recording infrastructure and technology discussions.
- Capturing decisions related to architecture planning.
- Tracking assigned tasks and responsibilities.
- Maintaining traceability of stakeholder inputs.
- Supporting preparation of the TIS documentation.

Meeting Categories

The following types of meetings were conducted:

- Client consultation sessions
- Internal technology evaluation meetings
- Documentation planning discussions
- Progress monitoring sessions
- Architecture validation meetings

9.3.1 Meeting Log

Table 79: Meeting Logs

Date	Meeting Title	Participants	Key Focus	Outcome
27/01/26	Technology consultation with client	Client + Team	Feasibility and constraints	Initial infrastructure direction defined
27/01/26	Technology evaluation meeting	Internal team	Review of tools and frameworks	Technologies shortlisted
29/01/26	Technology stack finalisation	Internal team	Decision-making and planning	Documentation responsibilities allocated

3/2/2026	Progress coordination meeting	Internal team	Monitoring workload and alignment	Adjustments to task distribution
10/2/2026	Architecture presentation with client	Client + Team	Validation of system architecture	Infrastructure planning confirmed

Meeting Evidence and Traceability

To ensure accountability and provide verifiable documentation of project coordination, records of conducted meetings were maintained and referenced throughout the TIS preparation.

Table 80: Meeting Minutes Summary

Date	Description	Reference
27/01/26	Client consultation on infrastructure technologies	Documented meeting record
27/01/26	Internal discussion on technology selection	Documented meeting record
29/01/26	Technology stack finalisation discussion	Documented meeting record
3/2/2026	Team progress and coordination meeting	Documented meeting record
10/2/2026	Client architecture validation session	Documented meeting record

Key Outcomes from Meetings

The meetings conducted during the TIS phase resulted in:

- Confirmation of infrastructure technologies.
- Validation of architectural direction.
- Allocation of documentation responsibilities.
- Continuous alignment between team and client.
- Identification of technical research requirements.

Meeting notes were essential in keeping the project under control and keeping the infrastructure planning as well as documented and traceable. These records facilitated continuity of decisions and made sure that the TIS was a reflection of the approved project talks and agreed technical directions.

9.4 Integrating Kanban

9.4.1 Trello Kanban Board

This is a screenshot of the kanban board at the end of week 2.

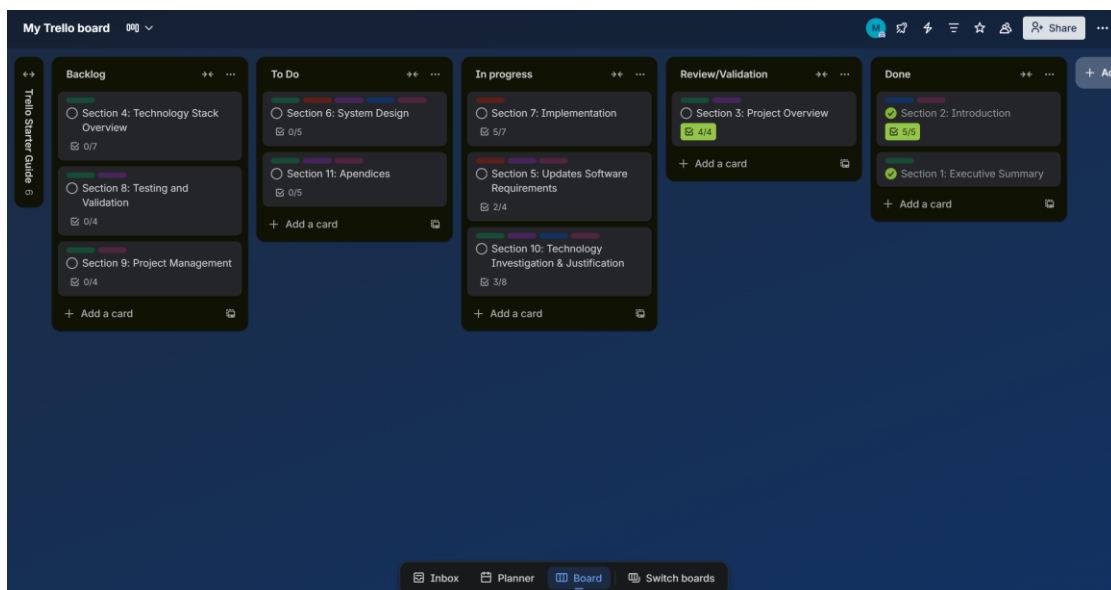


Figure 7: Trello Kanban

Trello Link:

<https://trello.com/invite/b/6990aec858b573fa19e11afa/ATTle2e7839ac9b426009e963c8958956098AFB99C18/my-trello-board>

9.4.2 Kanban Board Workflow

To manage Milestone 2, the team combined a Kanban workflow with the help of Trello. The board was designed in such a manner that it had a proper documentation lifecycle, where all parts of the report could be traced back to the planning stage, up to the final approval of the report. The workflow was structured into five columns, including Backlog, To Do, In Progress,

Review/Validation and Done. This organization enabled the team to have a clear view of tasks and control them, minimize misunderstandings on what to work on next, and make sure that the completed parts were properly checked before being declared done.

9.4.3 Task Movement and Workflow

The Backlog column was a temporary storage place of sections or tasks that were determined, but not yet ready to begin (such as tasks that required input by other sections or that required clarification). When a task was actionable, it was transferred to To Do where it was deemed as being ready to be initiated. When one of the members actively started writing or updating a section, the card was transferred to In Progress, which allowed the team to prevent duplication of work and the activity to be visible at a glance. On completion, the cards were transferred to Review/Validation to check consistency, correctness, quality of formatting and conformity to Milestone 2 documentation requirements. The card was not moved to Done until the review was finished and any feedback was implemented, and it was not to mean that the card was finalised and validated but only drafted.

9.4.4 Clear Ownership Through Colour Coding

To ensure that ownership and accountability is easily visible, the colour-coded indicators (called labels in Trello) were added to each Trello card to reflect the team members who would be responsible in that section of the documentation. The colour scheme was: subtle green = Hashir, subtle purple = Omar, subtle pink = Maria, subtle red = Siraj and subtle blue = Zernish. This allowed it to be easy to determine who was in charge of each section, and it would also be easy to filter quickly during a meeting, and it would also be easy to coordinate the sections when there was a dependency between sections (such as in situations where one section would depend on the information written in another section). Altogether, the implementation of Kanban in Trello was an easy yet efficient means of monitoring the progress throughout the milestone timeline and ensuring consistent progress toward the ultimate submission.

10. Technology Investigation & Justification

10.1 Technology Investigation & Justification

This part defines the last group of technologies that are selected to use in TeamTrack and the reason why they are suitable in the project. The choice of the stack was made to fit what the system has to do, how it should perform, and what is feasible to create in an academic project. It is also

aligned with the requirements enumerated in the SRS, particularly in the areas of security, scalability and ensuring that the system is easy to maintain.

In general, these tools facilitate effective collaboration within a common workspace, effective monitoring of contributions, and analysis of activity by AI. The options also make the system modular and thus everything can be enhanced or expanded later without the need to redesign everything.

10.1.1 Overview of Final Stack

Table 81: Final Stack Overview

Layer	Technology Selected	Purpose
Frontend	Nuxt.js (Vue.js Framework)	User interface and client-side rendering
Backend	Python + FastAPI	API development and business logic
Database (Primary)	PostgreSQL	Relational data storage
Event / Activity Storage	Firehose Database	High-frequency event logging
AI Module	LLaMA 3.1	AI-assisted insights and reporting
Authentication	JWT (Token-Based Auth)	Secure session management
Containerisation	Docker	Environment standardisation
Communication Protocol	HTTPS (SSL/TLS)	Secure data transmission

10.1.2 Frontend Technology – Nuxt.js

Nuxt.js was chosen as a frontend framework because of its design structure, scalability, and performance optimisation features. It promotes component creation and server-side rendering and enhances performance and user experience.

This is in line with the SRS usability, accessibility and systematized navigation.

Key reasons for selection:

- Reusable and serviceable architecture.
- Efficient routing system
- Scalable components of UI.

- Powerful ecosystem and sustainability.
- Designed to fit dashboard display.

Nuxt.js allows to implement:

- Kanban-style task boards
- Contribution dashboards
- Project chat interface
- AI query interface

10.1.3 Backend Technology Python and FastAPI

Python was chosen because it has a good ecosystem, can be read, and is able to integrate AI.

FastAPI has been chosen as a backend framework because:

- High performance (asynchronous support)
- RESTful API compatibility
- Auto-generated Open API documentation.
- Strong validation system

This supports:

- User authentication (FR-1 to FR-4)
- Project management attributes (FR-5 to FR-8)
- Task management (FR-9 to FR-12)
- Tracking of contribution (FR-13 to FR-15)

FastAPI enables the ability to clearly separate concerns between:

- API routing
- Business logic
- Database operations

This facilitates maintainability and scalability.

10.1.4 Database Technology- PostgreSQL

The major relational database management system is chosen as postgresQL.

Reasons for selection:

- ACID compliance (guarantees the integrity of data)
- Good relational support (foreign keys, constraints)
- Optimal indexing and query optimisation.

- Mature ecosystem

This is in line with SRS requirements on:

- Data integrity
- Role-based access control
- Contribution analytics
- Audit trails

PostgreSQL will provide a stable storage of:

- Users
- Projects
- Tasks
- Time Logs
- Activity Logs

10.1.5 Event Logging-Firehose Database

The Firehose-style database architecture is utilized to work with the data of high-frequency logging (e.g.):

- Task updates
- Time tracking entries
- Activity logs
- User interaction events

By decoupling transactional data (PostgreSQL) and high-volume logs, it is better:

- Performance
- Scalability
- Analytical processing

This architecture will make sure that the system is able to produce contribution analytics dashboards without affecting the primary system performance.

10.1.6 AI Technology – LLaMA 3.1

LLaMA 3.1 is chosen to generate insights with the use of AI.

Its role is limited to:

- Workload analysis
- Inactivity detection
- Natural language query response.

- Explanations of contribution summary.

The AI is a recommendation system which cannot and does not automate grading decisions, which is consistent with system constraints specified in the SRS.

This ensures:

- Ethical AI usage
- Human oversight
- Openness of decision-making in systems.

10.1.7 Security Technologies

Mechanisms that have been put in place regarding security:

- HTTPS with SSL encryption
- JWT-based authentication
- Role-Based Access Control (RBAC)
- Input validation
- Secure session management

These are technologies which directly provide the security and access control requirements that are outlined in the SRS.

10.1.8 Containerization – Docker

Docker is used to:

- Standardise development environments
- Avoid dependency wars.
- Make sure it is regularly deployed.
- Isolate services (frontend, backend, and database) and AI.

That helps to maintain and ensure reproducibility during development and testing.

10.2 Frontend Technology Evaluation

10.2.1 Context: Frontend Requirements of TeamTrack

The frontend features are all accessed through a browser interface since the platform is full web based. Therefore, the frontend has to present information clearly, respond quickly to user actions, and ensure that its enforcing role-based visibility based on the role of the user logged in. All the functionalities that the chosen software should have should help us follow the system

requirements and achieve them in the best way possible, and the frontend should support these modules.

- User registration, login, and secure session handling
- Interfaces for project and workspace management
- Task management using list views and a board (Kanban-style)
- Time tracking with start/stop timers and manual logging option
- Team chat with near real-time updates
- Contribution analytics dashboards that have visual summaries
- Notifications and activity log views

Performance and Responsiveness

Common operations include sending messages, switching tasks, recording time, and displaying dashboards, among others. These are part of the requirements of the responsiveness criteria. The frontend should be able to perform these operations quickly under normal conditions. The frontend should support these following features, so it meets requirements.

- Efficient state handling
- Optimized component rendering
- Fast page routing
- Smooth REST API integration
- (Near real-time updates for chat messages and task changes should also be handled)

Usability and Interface Consistency

Since the target users of the TeamTrack are students and instructors, it should be simple and consistent across all the modules. All the navigation elements including dashboard, task board, analytics, and timesheets should all have a predictable layout. RBAC is also important, certain actions and interface features are meant to be restricted depending on the users' permissions (leader/ member). The frontend must provide:

- Clear form validation feedback
- Understandable error messages
- Consistent navigation
- Readable layouts for desktop and laptop devices

Security and System Integration

The frontend communicates with backend services over HTTPS and should handle authentication tokens and sessions validation correctly. Routes should only be accessible with valid credentials. So, the interface must also handle:

- Session expiration
- Network interruptions

- API error responses

10.2.2 Nuxt.js

Overview

Nuxt.js is a progressive frontend framework built on top of Vue.js. It combines project configuration, server-side rendering (SSR), and routing into a single framework, it offers an organized method for creating online applications [1]. It makes Large scale frontend development easier using its built-in rules for file organization, page routing, data fetching, setting it apart from plain Vue.js.

One of the most important features of it is that it supports several rendering techniques, which include server-side rendering (SSR), static site generation (SSG), and client-side rendering. This helps developers optimize performance and how long it takes a page to load, depending on the system requirements [1]. It also includes a CLI, as well as a modular structure, which eases scalability as well as developing maintainable applications. Based on TeamTrack's needs, which include a structured routing system, API communication, reusing UI components, as well as optimizing performance, Nuxt.js is a good balance based on its flexibility and structure.

Strengths

- Using a clean component-based architecture, built on top of Vue.js
- Included system for routing, based on a file structure, which means there is no need for any form of configuration
- Includes support for server-side rendering (SSR), which increases speed as well as SEO [1][2].
- The design is modular, easy to integrate state management, API processing, as well as authentication
- Automatic code splitting and optimized rendering help the app run faster and more smoothly [2], it helps the performance.

Nuxt.js satisfies the needs for user interface components, systematic navigation across many system modules, and fast dashboard loading.

Weaknesses

- It can be a bit more complex compared to Vue.js
- Expects knowledge of SSR concepts, may increase learning curve.
- A smaller ecosystem compared to React-based frameworks

Even though it simplifies architectural decisions, developers need to understand its rendering modes and behaviour to use it effectively.

10.2.3 React (with Next.js)

Overview

React is a JavaScript library by Meta for making user interfaces using reusable components, meaning the interface is built from small, reusable pieces that can be combined and reused in different parts of the application. It focuses mainly on displaying efficiently and updating user interface components using a virtual DOM, but it does not include built-in routing, server-side rendering, or a complete project structure, so you usually need other tools to build full applications. [3]

Next.js is the framework build on React which extends its capabilities by adding features like file-based routing, server-side rendering (SSR), static site generation (SSG), and built-in features. It lets programmers to create complete front-end apps with better performance and a structured architecture [3]. It also simplifies any SSR configuration and page-based routing, which would otherwise require manual setup when using only React.

For TeamTrack, the combination of React and Next.js could be a highly powerful and versatile solution for the front end, all because it supports dynamic dashboards, code reusability, secure API calls, and performance tuning and those are things that are needed. But the cons are that it involves some extra architectural considerations, i.e., it involves a few more architectural and structural decisions, although slightly fewer than the ones involved in the frameworks based on Vue.

Strengths

- Finding libraries and solutions is made easier by the extensive ecosystem and community support.
- Architecture is based on components that encourages reuse user interface design [2]
- Virtual DOM improves performance and efficient UI updates [2]
- The automatic code splitting and page-based routing features of Next.js
- Good for long term industry implementation and enterprise used

The enterprise feature is not really necessary for TeamTrack, and it's good to note that React is a highly scalable solution and best suited for complex systems.

Weaknesses

- React is not a complete framework on its own, and it needs extra libraries for state management and routing [2].
- The architecture is complex and may increase if used alongside Next.js
- Learning curve considering SSR and rendering concepts
- More configuration decisions compared to opinionated frameworks like Nuxt

10.2.4 Angular

Angular is a front-end framework which has various functionalities and is used, maintained by and Google. It's primarily built with Typescript and takes a structured, opinion based approach for developing complex web applications. In contrast to React libraries, Angular provides built-in functionality for routing, state control, form managing, HTTP requests communication or other domains in the same framework.

It follows a modular build system which means you can scale the application efficiently by dividing it into small chunks and reuse them, known as components, wherever needed. So, each component has its own layout, logic, and data, that makes the app more organized and it's also easier to maintain as it develops, other than that it also uses two-way data binding, which means that when the data changes, the interface updates automatically, and this is good devs because it keeps the system consistent automatically, following a modular structure making it suitable for enterprise level apps that focus on organization and being maintainable [2][4].

For TeamTrack, Angular would be helpful in handling the dashboards, securing API communication, and also the RBAC behaviors discussed, without needing many third-party libraries, but it's also comprehensive and can be more complex compared to simpler frameworks.

Strengths

- A complete framework with built-in routing, HTTP services, and form handling [4]
- Strong structure that supports consistency in extensive applications
- Two-way binding make synchronization between UI and data a lot simpler
- Ideal for intricate, large-scale systems
- Large community support and long-term backing by Google

Offers a well-organized environment that can appropriately support TeamTrack's structured models.

Weaknesses

- More difficult to learn than frameworks that are lightweight or built on Vue [2].
- Complex syntax because of TypeScript and other framework norms.
- The framework is larger than alternatives such as Nuxt.
- May add extra complexity.

It is well-organized, but it is also large and complex, which could be beyond the needs of TeamTrack.

10.2.5 Comparison Table

Criteria	Nuxt.js (Vue-based)	React (with Next.js)	Angular
----------	---------------------	----------------------	---------

Framework Type	Vue-based full framework	React library + framework extension	Full frontend framework
Architecture Style	Component-based, opinionated structure	Component-based, flexible and modular	Component-based, structured and opinionated
Routing	Built-in file-based routing	File-based routing via Next.js	Built-in router module
Rendering Support	SSR, SSG, CSR supported	SSR, SSG via Next.js	Primarily CSR, SSR via Angular Universal
Configuration Level	Moderate, convention-driven	Flexible but requires decisions	Highly structured, configuration-heavy
Learning Curve	Moderate (Vue + SSR concepts)	Moderate to high (React + Next concepts)	High (TypeScript, RxJS, full framework features)
Ecosystem Size	Growing Vue ecosystem	Very large global ecosystem	Large enterprise ecosystem
Performance Optimisation	Automatic code splitting, SSR support	Virtual DOM, automatic code splitting	Ahead-of-time compilation, optimisation tools
Best Fit	Structured dashboards and modular web apps	Large-scale scalable applications	Enterprise-grade complex systems

Table 82: Comparison Table Frontend

10.2.6 Final Justification: Why Nuxt.js

Based on the evaluation, Nuxt.js is selected as the best suitable framework for TeamTrack's frontend because TeamTrack requires a frontend system that is not only organized and flexible but also capable of supporting a dashboard, task board, analytics visualization, API communication, and modular navigation through different system modules. So, the overly complex options that are more for enterprise architectures aren't needed but what is required is a system that can support organizational routing, reuse of UI components, and optimization of performance.

This is exactly what Nuxt.js can offer, as compared to using React as a frontend framework, as it not only includes routing and rendering configuration out of the box, which means there is no need

for manually configuring additional libraries, but it also simplifies the development and maintenance of the software by not only providing fewer architectural decisions but also by maintaining a clearer structure compared to using React with Next.js. React does work well with **Next.js and also offers many features and flexibility and scalability but the level of flexibility isn't** needed here.

Another suitable option for building a frontend system is using Angular, yes, it is a powerful and enterprise-level frontend framework, but what it also offers is a higher level of complexity due to a steeper learning curve, and what is not required is this level of strictness for a framework, especially for a simpler software system like TeamTrack. What is also important is that it meets all of TeamTrack's functional requirements as a system:

- Dynamic dashboards and task views
- Structured navigation between modules
- Secure REST API communication
- Reusable and maintainable UI components
- Optimized performance through SSR and code splitting

Summary

After assessing a few different frontend technologies, we chose Nuxt.js as the most suitable choice for TeamTrack's platform since it offers the best balance between performance, scalability, and its maintainable. The framework is made as a component-based architecture, along with its support for server-side rendering and a robust ecosystem, all these made it a particularly good choice for developing features like interactive dashboards, task boards, contribution analytics interfaces that are all used by TeamTrack. Relative to the alternatives like React and Angular, Nuxt.js provided a more streamlined development, while still enabling scalability and a well-structured user interface, hence it was chosen as the frontend framework for TeamTrack.

10.3 Backend Technology Evaluation (Python, FastAPI, Alternatives, Security Considerations)

10.3.1 Context: Backend Requirements of TeamTrack

TeamTrack is an academic collaboration solution, which is actually designed to avail project spaces, task management, time track, contribution analytics and role-based access control. The backend must therefore provide (1) APIs of all the major workflows of high quality, (2) secure authentication and authorization capabilities across roles, (2) free of contamination integration with a relational database layer, and (4) scalability to analytics and optional AI-supported functions. In addition, a student development team and in-service should be able to support the backend on the basis of iterative limit on testing response.

As the system relies on the frontend- backend architecture through the API, the latter will be API-first, with business logic and security actions being concentrated at transparent endpoints. The new trend in the world of web systems is the use of web API-based systems and the selection of the backend framework can significantly influence the speed of development, maintainability and the performance of the operations. The recent benchmark study of Python web frameworks in API applications (Django/DRF, Flask, FastAPI) has also shown that the trends of the overheads of frameworks, concurrency model, and ORM interaction have substantial impacts on throughput and response time [11].

10.3.2 Evaluation Criteria

The evaluation criteria that were used to ensure that the selected technology of the backend was compatible with the objectives of TeamTrack included:

1. Performance and Scalability: The ability to handle parallel requests and be responsive to API request when the classes/teams are in their normal use.
2. Development Productivity: velocity of code, code structure articulateness, learning, and making the desired implementation fast.
3. Maintainability and Robustness: Support of code quality (typing, validation, modularity), testability, and long-term maintainability.
4. Security and Access Control Support: Support of secure authentication and authorization patterns (token-based auth, RBAC), input verification, and secure API practices.
5. Ecosystem and Integration: Database access, validation, API documentation, and deployment patterns are available.

10.3.3 Selected Backend Language: Python

The reason to choose Python as the back-end language is due to its ability to facilitate quick development and readability and an advanced ecosystem in web APIs and data-centric workflows. In the case of TeamTrack, the strengths of Python can be especially applicable due to the fact that the project implies the work with structured data processing, analytics, and non-obligatory AI-related integration (where Python is rather popular in real life).

Nonetheless, Python too has trade-offs: when used as a compute-heavy workload, its runtime performance may be less than compiled languages. In the case of TeamTrack, the majority of the backend work is I/O-bound (processing of HTTP requests, reads/writes to the database), with the framework architecture and an async support having a more significant effect on the perceived

performance than bare compute throughput. Empirical studies on a set of high-productivity languages such as Python indicate that Python may be slower in certain performance metrics, but has considerably higher benefits in the programmability and productivity, which validate its appropriateness in the context of rapid development [19].

Moreover, the Python ecosystem also uses type annotations to achieve maintainability and reduce defects. The results of large-scale empirical studies indicate that type annotations are increasingly becoming widespread and can assist maintainability and tooling [17]. This is significant since TeamTrack also needs role-based maintainable logic and endpoint-to-endpoint regular request and response schemas.

10.3.4 FastAPI as the Chosen Backend Framework

FastAPI was chosen as the backend framework since it fits well with the API-first nature of TeamTrack and focuses on the best practices of modern backends: request validation, schema-based development, and asynchronous endpoint support. Fast API is particularly appropriate when one wants to create REST-style APIs in which both correctness and maintainability require uniform data models across the frontend-backend interface.

10.3.4.1 Performance and Concurrency

FastAPI is often considered to be a high-performance Python framework, especially in API loads. The Journal of Computer Sciences Institute peer-reviewed benchmark study reviewed FastAPI, Flask, and Django (including Django Rest Framework) on common API endpoints and CRUD operations and discovered that FastAPI achieved higher results in most situations, particularly in request throughput and base API handling [11].

In the case of TeamTrack, where frequent updates of tasks are required, time logs are submitted, and queries to the dashboard are made, these performance features will enable a reactive user experience.

Notably, not only is the framework called performance, but also the manner in which the data access layers (use of ORM and query patterns) are carried out. The same benchmarking study points out that an overhead caused by ORM may have an impact on performance even in the context of operations that are not heavy database usage, which supports the idea that the database interactions must be efficient and unwarranted abstractions must not exist in critical endpoints [11].

10.3.4.2 Developer Productivity and Maintainability

The FastAPI design encourages the development of APIs in a highly structured manner: the endpoints are being described along with request/response data models, which help reduce ambiguity in addition to ensuring easier standardized validation. This reduces integration friction, and improves coordination of the team: in the case of a system being built by students, the backend developers can provide schemas and fail on invalid input rather than letting inconsistent information propagate.

The empirical evidence to this strengths is at least by empirical work on the value of type annotations in Python project using annotations allows tooling and maybe helpful in removing defect introduction due to type-related faults by identifying the faults at an earlier stage and enhancing maintainability [17]. Because TeamTrack requires dynamic data structures (tasks, projects, time logs, user roles), the schema-based API development is a reasonable approach to augmenting the accuracy level and reducing the quantity of rework in the subsequent testing periods.

10.3.4.3 Fit for TeamTrack's Architecture

FastAPI is also appropriate to TeamTrack since the system is not a server-rendered and monolithic one. TeamTrack, in turn, follows a frontend design with a backend and communicates with the latter via a set of specific API points. Here, it will not be suitable to use full-stack systems with intensive templating strata. A more API-oriented structure would allow a better separation of concerns as the backend would remain used solely by authentication, authorisation, business logic processing, and data persistence.

10.3.5 Alternative Backend Frameworks

The available background frameworks were reviewed in line with the chosen criteria to support the implementation of FastAPI in TeamTrack.

10.3.5.1 Django (with Django Rest Framework)

Django is a full-stack web application framework that is normally used in large-scale web applications and is typified by a philosophy of batteries-included. It offers a built-in administration tooling, Object-Relational Mapper (ORM), authentication implementation, and a fully developed ecosystem. These features would allow cutting down on development costs by providing ready-to-use components of shared backend needs.

Nonetheless, since TeamTrack is API-first, Django can be an extraneous overhead. Comparisons between Django/DRF and Flask as well as FastAPI performed on the benchmark show that Django/DRF is able to pay framework overheads in most scenarios involving API requests especially when compared to the less bulky or API-friendly frameworks [11]. Besides that, the complete stack design of Django can potentially add complexity to implementing the software

used by student groups in situations where large parts of the framework (e.g. templating, server-render views) are not being used.

Conclusion: Django/DRF would be a good fit with a monolithic or full-stack implementation, whereas its integration with TeamTrack would be less firm.

10.3.5.2 Flask

Flask is a simple microframework that has been characterized by flexibility and simplicity. It is also applicable in small applications where the developer is interested in having control of components fully.

The problem with it is that the Flask framework frequently has to be assembled manually (validation, structured schema processing, regular error responses, authentication pattern). Extensions have been known to fill gaps, although this practice may introduce integration risk to student teams and may create inconsistencies with expansion of the project. Flask is capable of competing with FastAPI in similar loads on the API benchmarks, but not always faster [11].

Conclusion: Flask is good to use with small prototypes, whereas FastAPI offers more robust structure with a reduced number of integration risks.

10.3.5.3 Node.js / Express

The event-driven concurrent model of node.js (with Express) is popular as a web API because it has a large ecosystem and is event-driven. It is able to do well with I/O-bound workloads.

But in the case of TeamTrack, it will create a cross-language divide when AI/data components are still in Python. This adds complexity of development and integration. On the contrary, Python offers a common language platform in both data analytics and backend logic. Also, the performance profile of FastAPI is competitive in the current API environment, which avoids the necessity to change the ecosystem only due to the perceived performance advantages [11].

NB: Node.js/Express is an option, but Python offers a more unified stack to a system that has analytics/AI-related capabilities.

10.3.5.4 Java / Spring Boot

Spring boot is an enterprise level framework that provides a robust structure, scalability and a wide array of security tooling. It is widely used as a selection of production enterprise systems.

In the case of TeamTrack, the trade-off is in the development overhead: Spring Boot generally demands greater configuration effort and is more challenging to learn by smaller academic teams. Although it is powerful, it can slow the iteration speed and raise the time of implementing features that are relatively simple to implement in Python frameworks.

Conclusion: Spring Boot is very robust, but not best in a student project where speedy delivery and continuous improvement are required.

10.3.6 Security Considerations for the Backend

Security is one of the key assessment criteria since TeamTrack contains project and contribution data and applies role-differentiated access. The API-based systems are generally trusted by a proper implementation of authentication, authorization and the secure API practices, and are not merely trusted based on the framework selection. However, frameworks have an impact on the ease with which these controls can be established in a proper and uniform manner.

10.3.6.1 Authorization Model: Role-Based Access Control (RBAC)

TeamTrack is a system where RBAC is used to manage activities of Student Members, Team Leaders and Instructors. RBAC is a commonly known model that allows the administration of permissions to be simplified by associating rights with positions and not with specific users. The initial RBAC reference models proposed in IEEE literature indicate the way RBAC enhances the ease of controlling the assignments of permissions and simplifies over ad hoc management of the permission assignments [12].

It reinforces the objectives of TeamTrack: open access policies, regular implementation, and the possibility of increasing the authorization range as the system expands.

10.3.6.2 Token-Based Authentication and Session Handling

Contemporary API services are frequently based on the token-based authentication. Identity claims are normally transmitted in a secure way with the help of JSON Web Tokens (JWTs) between the client and the server. The literature on peer-reviewed journals which talk about token-based security in REST API show how JWT-based mechanisms are applied to stateless authentication patterns which fit web services [16].

TeamTrack Token-based auth enables the frontend to be separated, and the backend to authenticate tokens and provide role-based permissions to each request.

10.3.6.3 OAuth 2.0 Security Considerations

OAuth 2.0 patterns or other authorization patterns are applied (e.g. to later integrate with other identity providers or services), then special care is needed. The study on security evaluation was published in Information and Computer Security and it outlines possible security problems with OAuth 2.0, and the need to implement it properly against the threat model [13]. More so, peer-reviewed ACM publication of formal security analysis proves that OAuth security is only reliable in

the case of appropriate deployment decisions made and points to the fact that the vulnerabilities may occur without the most appropriate mitigation measures [14].

Although TeamTrack may not use full OAuth flows within the current milestone, such works inform secure design assumptions and minimize the risk in subsequent integrations.

10.3.6.4 Secure API Practices and Microservices-Oriented Risks

Although API systems are vulnerable to common risks in a single-backend architecture, they include broken authentication, incorrect authorization validation, injection vulnerability and insecure dependency usage. A case study carried out by the Journal of Systems and Software on security practices to use in microservices-based systems supports the necessity of systematic security controls (secure defaults, regular enforcement of authentication, validation, and monitoring) [15].

Whereas, TeamTrack does not always have to be broken down into microservices, the underlying principles of API-security are relevant: the backend should control access at all points and uphold a safe operational culture.

10.3.7 Comparison Summary

The qualitative overview of the reviewed backend frameworks is presented below. The comparison highlights the fitness of the API-first academic workflow system of TeamTrack.

Table 83: Comparison summary

Criteria	FastAPI (Python)	Django/DRF (Python)	Flask (Python)	Node/Express (JS)	Spring Boot (Java)
API Performance (typical REST endpoints)	Strong; benchmarked high in studies [11]	Moderate; more framework overhead [11]	Competitive but less structured [11]	Strong for I/O-bound APIs	Strong, enterprise-grade
Development Speed (student team)	High (structured schemas, validation)	Medium (heavier stack)	High initially; complexity grows	Medium; ecosystem large	Lower; more configuration

Maintainability	High (typing + schema-driven) [17]	High but heavier conventions	Medium; depends on team discipline	Medium; depends on patterns	High; strong conventions
Security Integration	Strong fit for RBAC + token auth; backend-enforced	Strong built-ins; but heavier stack	Requires more manual setup	Strong ecosystem; more moving parts	Very strong security framework
Fit for TeamTrack	Best overall balance	Possible but less aligned	OK for small scope	Viable but increases stack split	Overkill for scope

10.3.8 Final Justification

FastAPI (Python) was chosen since it is the most suitable in terms of performance, productivity, and maintainability to an API-first academic project management platform. The benchmarking as demonstrated by peer-reviewed results reveals that API workloads perform well as compared to other mainstream Python frameworks [11].

Python is good to execute fast, and it is compatible with data-centric attributes, whereas the research of type-annotation adoption shows the benefits of Python systems in terms of maintenance through schema-driven development [17].

Lastly, the security model of the backend is enhanced with RBAC base which is rooted on the literature of IEEE alongside token based authentication systems with the help of literature [12], [16].

Summary

The backend technology evaluation ultimately resulted in choosing Python with FastAPI as the most appropriate foundation for the TeamTrack system. The main thing about FastAPI that aligned with TeamTrack's goals was its strong performance capabilities and support for asynchronous request handling, both of which aligned well with the system's architectural goals reliant on REST APIs and modular services. Some options that were considered are Django, Flask, and a range of Frameworks based on Node.js, but FastAPI stood out because of its lightweight but flexible nature, since it provides the necessary support for both security and scalability without the constraints of more rigid frameworks, while still enabling fast development. With all these benefits, Python and FastAPI have been chosen to be used as the backend services and business logic in TeamTrack.

10.4 Database Technology Evaluation

10.4.1 Context: Data requirements of TeamTrack

This section focuses on the data characteristics that need to be considered when choosing a tool for the database. Users, teams, projects, tasks, time logs, notifications, and activity records are some of the relational data sources that TeamTrack depends on. Since they're so closely connected, the entities need to have consistent reinforcement of relationships to make sure that data is accurate during concurrent operations, which are operations that may be happening at the same time.

System priorities

- **Transactional consistency**, especially when there are many users creating activity records, tracking time, or updating tasks at the same time. For example if there are many people updating tasks, creating time logs, or creating activity records, all happening at the same time, it should be able to save all those changes correctly, without one user overwriting another user's changes or making incomplete records.
- **Strong relationships between all related data.** For example, a record for a Task, a TimeLog, should be related to a valid User and a valid Project, and it should not be allowed for a TimeLog to exist without a User, a Task, or a valid Project, to organize this data and to not have any broken or invalid lines between entities.
- **Access control aligned with user roles:**
Not every user should be able to see or modify all data. Leaders have more privilege, and the database should support these restrictions based on roles and permissions for each role, making sure that only the authorized users can access or update certain things; everyone has the privileges only they need.
- **Reliable historical data.** The system should be able to retrieve the past information, the contributions' history, the activity logs, and the archived projects, and so on, as it would be important for analytics and other things.

In order to avoid the possibility of incomplete and corrupted records in the database in the event of a system crash or other network problems, the database should be able to handle these changes with care, and the time logs and activity changes should be saved either entirely or not at all, without partial saving, because it's important to ensure the accuracy of the contributions and the projects, so that they can be reviewed whenever needed.

The retention of the data should be taken into consideration as well, and the projects should be easily retrievable if they are not yet completed, but the projects should be archived if they are already completed, so the information can be referred to in the future if needed.

But the system should be able to remove certain information if it is no longer needed, like the notification if it has already been read by the users.

As the system of TeamTrack mostly deals with structured data, the scale of the system is not really big, so it would not really be needed. In other words, there would be no need for schema-less databases and sharding, as the traditional structured database setup would be enough for the system.

10.4.2 PostgreSQL

Overview

PostgreSQL is an open-source relational database management system, and its primary goal is to offer a reliable and scalable database system for the storage of data. It is well-known and used in web and enterprise applications where structured data management, consistency, and strong query support are required [5].

Strengths

- ACID compliance, hence making it reliable and consistent in the processing of transactions
- Strongly supports relational data with constraints, foreign keys, and complex queries
- Advanced security support, including support for role-based access control and SSL support
- Supports both structured and semi-structured data through JSON and JSONB data types [5][6]

Weaknesses

- More complicated setup and configuration compared to other database systems, especially those that are light weight
- A more difficult learning curve for developers and administrators who are new to relational databases
- It may be slightly slower for very simple or mainly read-based applications where its advanced features are not really needed [6].

10.4.3 MySQL

Overview

MySQL is an open-source relational database management system that utilizes structured tables and SQL to store and retrieve information. Due to its ease of use, performance, and language support, it is frequently applied in web-based applications [7]. Well-known.

Strengths

- Open-source and widely used, with strong community support
- Strong performance for workloads involving a lot of reading and basic transactional systems
- Compatible across platforms and supports many programming languages
- Supports ACID-compliant transactions when using storage engines such as InnoDB [7][8]

Weaknesses

- Fewer advanced SQL features compared to PostgreSQL, limits complex query support
- Less flexible handling of semi-structured data compared to NoSQL databases
- Can be an issue to scale and maintain in large systems [8]

10.4.4 MongoDB

Overview

MongoDB is a NoSQL database that is document-oriented. In MongoDB, information is stored as flexible documents that are similar to JSON. MongoDB is normally applied to applications that require the handling of a lot of unstructured or semi-structured information and require more scalability and availability [9].

Strengths

- The schema design is flexible as it allows fast development without strict table structures
- It can handle massive volumes of data effectively by scaling across multiple servers (sharding).
- Works well for systems with lots of reads and writes happening frequently
- Can store complex nested data easily without needing table joins [9][10]

Weaknesses

- Does not offer the same level of transaction security as other databases, especially when it is dealing with a lot of documents.
- Higher memory and resource usage due to document-based storage
- Doesn't naturally support joins like SQL databases, which makes relational queries harder
- Uses the SSPL license, which may not suit all organizations [10]

10.4.5 Comparison Table

Criteria	PostgreSQL	MySQL	MongoDB
Database Type	Relational (SQL)	Relational (SQL)	Document-based (NoSQL)
Data Model	Structured tables with strong relationships	Structured tables with simpler relational features	Flexible, JSON-like documents
ACID Compliance	Full ACID compliance	ACID support via InnoDB engine	Limited (mainly document-level)

Schema Flexibility	Medium (fixed schema with JSON support)	Low (rigid schema)	High (schema-less design)
Scalability Approach	Vertical scaling; horizontal via extensions	Primarily vertical scaling	Native horizontal scaling (sharding)
Query Complexity Support	Advanced queries, joins, constraints	Basic to moderate query support	Limited relational querying, no native joins
Performance Profile	Balanced performance for complex transactions	Optimised for read-heavy workloads	Optimised for high-volume read/write operations
Security Features	Strong role-based access, SSL, fine-grained controls	Standard access controls and authentication	Role-based access; depends on deployment setup
Typical Use Cases	Enterprise systems, structured data, transactional applications	Web applications with simpler data models	Real-time systems, unstructured or rapidly changing data

Table 84 : Comparison Databases

10.4.6 Final Justification: Why PostgreSQL

Based on the evaluations and research, PostgreSQL has been selected as the most appropriate database technology for TeamTrack. The system has structured and highly related data, all the entities that were discussed earlier, and therefore it requires a powerful relational integrity and transaction-oriented database. A relational database is simply the most logical choice for the system.

The SRS clearly outlines the importance the system places on the accuracy, reliability, and security of the data. Since PostgreSQL is an ACID-compliant database, it ensures that operations, like ones for modifying tasks or adding roles, are performed correctly, even in some case if multiple users are accessing the system at the same time, and that is exactly what TeamTrack needs.

When considering the system that TeamTrack needs, it becomes very important that the data remains consistent, especially in a collaborative environment in which there are likely to be multiple users making changes at the same time.

If we compare it MySQL, MySQL is a lot simpler, and PostgreSQL is better because it provides more advanced querying capabilities, stronger constraints, and better overall support for complex relationships. It's true that MySQL performs well for simpler applications that mostly handle read-

heavy workloads, it is not as well suited for systems with many interconnected entities like TeamTrack.

MongoDB was also considered because it offers powerful scalability, which could be good for the system, but it turns out that it is not really something that TeamTrack needs. The system doesn't require the schema-less data storage that MongoDB offers, and it doesn't require the horizontal scalability that it offers because it doesn't require the use of multiple servers. The system simply requires structured data and relationships, and using a NoSQL database would make it more complicated than it needs to be.

Summary

The most relevant objective as far as database technologies evaluation was concerned was to identify a system which can handle structured academic collaboration data required to support TeamTrack in a reliable manner. PostgreSQL was the best candidate due to its ability to provide solid support of relational integrity, complex querying and transactional consistency. These features were especially important to TeamTrack where project activities, time logs, and user identities have connected relationships.

The alternatives (MongoDB, MySQL) were evaluated too but PostgreSQL had more advanced relational capabilities and better data consistency assured. Based on all those facts, PostgreSQL was selected as the primary database for the TeamTrack platform.

10.5 External Services & APIs

Executive summary

In this section, the previous consideration of external services and API is further elaborated and justified by research and expanded to a more precise and clear justification, maintaining the same technology options: LLaMA 3.1 (through Ollama) as the local LLM inference system, FastAPI as the API layer, Firehose and IAM-based authentication as a logging and event ingestion service, PostgreSQL as the structured data store, and the use of tokens as an application access controller.

The stack is not very bulky within the general academic prototype. It puts privacy and offline capability first (runs the LLM locally), can be scaled to an orchestration API (asynchronous service design), and has a strict separation between transactional data (PostgreSQL) and high-volume telemetry (Firehose). It also makes the system scalable to grow in the future: the local-first architecture could later be converted to a hybrid or cloud architecture without any significant changes in architectural elements, provided that practices of REST interfaces, authentication scopes, and observability are maintained throughout the system [28].

10.5.1 AI Model Integration (LLaMA 3.1 via Ollama)

Large Language Models (LLMs) are also used in modern intelligent systems to aid in summarization, automated assistance, and context-dependent generation. Language models based on transformers have been shown to be highly competent at general-purpose language tasks, and can frequently be prompted to learn a new task and adapt to it without task-specific model retraining in prototype environments [25]. The survey literature also forms a general enabling technology layer, but also shows that non-architectural factors like cost, control, safety, and deployment considerations can affect architectural choice, particularly in academic settings [27].

In this project, the LLM layer needs to meet three primary requirements that are inferred by the section, namely, privacy, easy integration, and affordability of experiments. The fact of privacy is especially a point of influence since as soon as an application uses an external hosted model API, it has to send prompts and (this can be sensitive) user or system context across the local trust boundary.

The models built on LLaMA are explicitly designed to be deployable and efficient based on their size, and thus, they can be used in local inference cases when relying on the cloud is unwanted [26]. By selecting LLaMA 3.1 through Ollama, the local-first pattern is thus applied, in which the AI capability is a service endpoint, and on the same architectural plane as the backend API, and not a third-party requirement. This fits an academic prototype requirement profile: iterative testing, predictability of cost, and manageability of data exposure.

Table 85: Alternatives Evaluated for AI Integration

Technology	Type	Advantages	Limitations
Cloud LLM APIs	Cloud-based	High performance, no local setup	Cost per usage, internet dependency, privacy concerns
GPT-style hosted APIs	Proprietary cloud	Reliable and scalable	Expensive for continuous testing
Open-source local models	Local deployment	Privacy, no API cost	Hardware requirements
LLaMA 3.1 via Ollama	Local deployment	Free, private, easy integration	Requires moderate hardware

Comparison of LLM Deployment Approaches

Criteria	Cloud APIs	Local LLM (Ollama + LLaMA 3.1)
----------	------------	--------------------------------

Cost	Recurring API cost	Free after setup
Privacy	Data sent externally	Fully local processing
Performance	High but network dependent	Stable and offline capable
Customization	Limited	Highly customizable
Academic Use	Limited free usage	Ideal for research/testing

Expanded comparison: Cloud vs Local vs Hybrid LLM deployment

The initial comparison is furthered below to involve hybrid deployment since it is a typical way of future migration of academic prototypes: most local-first systems ultimately embrace cloud augmentation as burst capacity or managed operations, but retain local processing of sensitive workflows.

Table 86: Expanded Comparison AI Model

Attribute	Cloud LLM deployment	Local LLM deployment (Ollama + LLaMA 3.1)	Hybrid deployment (local + hosted)
Performance & scalability	Typically high scalability; dependent on network and provider throttling	Bounded by local compute; latency stable and predictable within local environment	Can route high-load to cloud while keeping sensitive workloads local
Security & privacy boundary	Prompts/contexts leave local boundary; privacy depends on provider controls	Prompts/contexts remain local by design; boundary is controlled by the project team	Requires policy-based routing to prevent accidental cloud leakage
Cost profile	Ongoing variable cost correlated with usage	Upfront/maintenance cost; marginal cost low for repeated testing	Mixed model; can optimize cost by using cloud only when needed
Offline capability	Not available (internet dependency)	Available (fully offline inference)	Partial (local retains minimal capability offline)
Customization & n	Constrained by API features and provider policy	High controllability (prompting, model choice, version pinning)	Supports experimentation; added

experimentation			complexity in routing and monitoring
Operational overhead	Low (managed)	Moderate (local provisioning, updates, monitoring)	Highest (two environments + consistency management)
Suitability for academic prototype constraints	Good for rapid start but may be cost-limiting at scale	Strong fit when budget/privacy constraints dominate	Best when scaling must be demonstrated, but design/ops maturity is available

This expanded framing supports the chosen local-first approach as the most coherent baseline under the stated prototype constraints, while explicitly mapping a credible path to future scaling.

LLM training vs inference suitability

An essential technical difference, as far as academic systems are concerned, is the fact that the use of an LLM may mean training or inference. The original technology choice implies inference rather than training. That is in line with the studies that indicate that strong task performance could be realized with prompting and limited-shot approaches without training a new model end-to-end [25], and the rationale of applying open foundation models that are more efficient to deploy [26].

Table 87: Requirements Comparison

Dimension	Training (pretraining/fine-tuning)	Inference (serving/generation)	Fit for this project (as specified)
Compute requirements	High and specialized; sustained resource demand	Moderate-to-high depending on model size; can be bounded by selecting/quantizing models	Inference-oriented; training requirements are not stated and are treated as out-of-scope/unspecified
Data requirements	Requires curated datasets, governance, and evaluation	Requires prompts, optionally retrieval context, and output evaluation	Prototype can iterate via prompt design and evaluation criteria [25], [27]

Operational complexity	Experiment tracking, retraining loops, model registry, reproducibility controls	Model serving, caching, rate limiting, and failure handling	More feasible for academic prototypes: serve a pinned model version locally
Risk & governance	Higher risk (data leakage, bias amplification, reproducibility)	Still requires safety/output controls, but fewer ML lifecycle dependencies	Consistent with a local-first privacy posture and iteration needs
Maintainability	Requires ML ops pipeline	Requires service ops + integration tests	Aligns with “AI as a service endpoint” architecture

As no training datasets, training objectives, or model-training pipeline are described in the document, the best interpretation of the document is that the system is not based on training models to adapt it. Rather, it applies interaction-based mechanisms like prompting, structured inputs and outputs, as well as evaluation mechanisms to steer system behaviour [25], [27].

Architecturally, Ollama and FastAPI are independent and dependent backend services, respectively, with Ollama being the local runtime and API layer to LLaMA 3.1. The structure enables the application to code LLM inference as a component instead of integrating model logic within a single, large body of code. Consequently, there is an increase in maintainability since there are no significant changes required with respect to backend services when switching or updating models, which is common with open-source models when comparing research and development [27].

The privacy and control are also enhanced with this separation. The boundary of integration is very well defined: model inference may be simply local (the approach that is chosen) or deliberately pushed to external services in case it is necessary in the future.

10.5.2 Backend API Service – FastAPI

The central point of the system is the backend API layer: the layer receives requests from frontends, performs authentication, coordinates AI inference calls, and carries out organized persistence operations. The API boundary is not just a communication interface in REST-oriented systems, but also a contract: resources, representations, and stateless interactions can be evolved and can also facilitate the decoupling of the client-server [28]. Therefore, the framework selection must be justified at the architectural level, and not solely because of the comfort of the developers.

The first one is API-based communication and asynchronous processing as the drivers of selections. This type of emphasis is correlated with the present-day rationales of network-program

design, asynchronous I/O, and event-driven execution, able to deliver large numbers of parallel connections at economical expense to workloads limited by I/O bottlenecks and increase load throughput [29]. Though [29] explains how it can be applied to Node.js, the architectural principle of non-blocking network patterns that can be used to support high concurrency is applicable in the selection of the backend architecture whenever it makes external calls (e.g., to an LLM service) and database calls.

Table 88: Backend Framework Alternatives Evaluated

Framework	Language	Advantages	Limitations
Flask	Python	Lightweight and simple	Limited scalability
Django	Python	Full-featured framework	Heavy for API-only systems
Node.js (Express)	JavaScript	Fast and scalable	Different ecosystem
FastAPI	Python	High performance and async support	Newer framework

Comparison at scale: FastAPI vs Django vs Node.js under load.

Since the original document does not specify explicit load behaviors (concurrent users, average request time, p95 latency, etc.), the following comparison is made qualitatively around architectural behaviors under I/O-heavy service patterns (DB + LLM calls), which is in line with the principles of asynchronous network program framework [29] and the design of RESTful APIs as an interface contract [28].

Table 89: Comparison API's

Criteria (load-relevant)	FastAPI (Python, async-capable)	Django (Python, full-stack)	Node.js + Express (JavaScript)
Concurrency model under I/O	Async endpoints can reduce blocking when awaiting DB/LLM calls; design aligns with service orchestration	Generally heavier stack; async support exists but often not the primary mode for API-only microservices	Event-loop model is explicitly designed for high concurrency; strong fit for I/O-bound APIs [29]

Latency sensitivity	Potentially stable if service boundaries are local (local LLM) and endpoints are non-blocking	Can be stable but may carry additional framework overhead in API-only use	Often strong for I/O-bound work; must manage CPU-bound tasks to avoid event loop stalls
AI integration friction	Same ecosystem as common AI tooling; reduces glue code and cross-language operational burden	Also Python, but full-stack patterns may exceed scope for service-only layer	Different ecosystem from Python AI stack; integration via network boundaries is clean but increases operational diversity
Maintainability in academic prototype	Clean separation: API-only service with automatic interface documentation can reduce integration errors (implementation detail; unspecified in original)	Full-stack structure may add complexity if only an API is needed	Common and well-understood, but increases polyglot complexity with a Python-based AI subsystem
Failure isolation	Works well when AI runtime is a separate service: API can degrade gracefully if LLM unavailable	Similar potential, but structure may encourage monolith patterns	Similar; service boundaries must still be designed explicitly

Justification of Fast API in this architecture.

Retaining the original choice of FastAPI, the most reasonable architecture motivation is its correspondence with a single-language (Python) orchestration layer, which may combine closely with AI-proximate workflows without being unable to sustain current asynchronous service patterns. This minimizes the surface area of integration: even fewer language runtimes, fewer deployment artifacts, and less complicated debugging are important in an academic prototype with little operational support (specified in 10.5.6). REST contract layer as defined in [28] is evolvable: the system can be later evolved to use hybrid or cloud LLM inference, without altering the semantics of the client, as long as the resource/representation contract does not change.

A practical consequence of the chosen stack is that request orchestration can be structured as:

1. Authenticate request (token validation).
2. Perform DB operations (PostgreSQL).
3. Call local LLM inference endpoint (Ollama).

4. Return response.
5. Emit structured events/logs to Firehose.

10.5.3 Data Pipeline and Logging – Firehose

There are two types of data that are isolated by the system: application data and operational or analytical telemetry data. Firehose is used in event ingestion and streaming, such as logs, activity records, and analytics events, and is not regarded as a conventional database. Such a difference is a universal law of scalable architectures: transactional systems do possess a structured store to be consulted, but telemetry systems are low-volume and append-only and target monitoring and analysis.

The research in the big-data systems highlights the importance of pipelines and distributed processing in changing the massive volumes of event data into useful insights. It further isolates storage systems that are built on active queries and those that are optimized on big scale ingestion and processing [30]. This work is of a more general scope than the one under investigation, but it explains why a special ingestion layer would be used, on which the logs and events would be loaded, instead of loading the main relational database with such a heavy load.

Table 90: Firehose vs Traditional Storage

Feature	Traditional Database	Firehose Pipeline
Data storage	Primary structured storage	Event and log streaming
Real-time processing	Limited	High
Scalability	Moderate	Very High
Monitoring support	Basic	Advanced
Use case	Core system data	Logs and analytics

Expanded comparison: Event pipeline vs ETL vs traditional database

The project’s selected combination (PostgreSQL + Firehose) is best understood by distinguishing three data-handling paradigms:

Table 91: Criteria Databases

Attribute	Traditional database (OLTP)	Batch ETL (periodic processing)	Event pipeline (streaming ingestion)
Primary goal	Serve application state transactions	Transform and load data periodically for analytics	Collect and route events/logs continuously
Latency	Low for queries/transactions	High (minutes/hours) depending on schedule	Low to near-real-time ingestion/forwarding
Schema flexibility	Moderate; schema changes require planning	Often transforms into analytics schemas	Often semi-structured; schema-on-read common (downstream)
Scaling pattern	Vertical + replication; careful indexing	Horizontally scalable compute jobs [30]	Horizontally scalable ingestion/partitioning conceptually [30]
Best fit for	User data, relationships, constraints	Periodic analytics, large transformations	Observability, clickstreams, audit logs, AI interaction telemetry
Why not use “only this”	Can be overloaded by high-volume logs	Not real-time; not ideal for alerting/monitoring	Not suitable as authoritative transactional store

10.5.4 Authentication Services

The authentication mechanism in the system is based on the use of tokens to confine the authorized access and also perform scalable session management in distributed applications. The scholarly and architectural rationale is that token mechanisms typically work well with stateless service interactions, and it is true to the principles of the rest that clients are decoupled and requests are stateless [28].

OAuth 2.0 is an authorized framework that is formal and presented in the reference set [31]. In this system, OAuth 2.0 is rather likely to be used in cases of the incorporation of third-party identity providers or delegating authorization capabilities, which are not required here. The fact that the original part does not specify identity provider integration, delegated authorization requirements, and federation requirements means that they should be considered unspecified, and not assumed.

The following table will be supplied to make sure that there is the required range of comparison and is possible to locate the position of token-based authentication options within a viable trade space within the architectural framework. It should be pointed out that the decision concerning the project is auth (chosen) token-based, which may be a jwt or an opaque token, and this point is not mentioned in the text.

Table 92: Authentication Comparisons

Approach	What it is (conceptually)	Strengths	Trade-offs / risks	Fit for this project (as specified)
Session-based auth	Server stores session state; client holds session ID	Simple mental model; easy revocation	Harder to scale across services; requires shared session store	Possible, but less aligned with stateless service scaling patterns [28]
JWT bearer tokens	Self-contained signed tokens validated by services	Stateless validation; good for distributed APIs	Revocation harder; token leakage risk; careful expiry needed	Compatible with “token-based auth,” but JWT specifically is unspecified
OAuth 2.0	Authorization framework for delegated access	Standardized flows; supports external IdPs	Higher integration complexity; more moving parts	Useful if third-party login or delegated authorization is required; requirement unspecified

Security and maintainability trade-offs

A rigorous authentication design is not only about choosing “tokens” but also about lifecycle and boundary controls. Even within token-based approaches, security quality depends on:

- **Token lifetime and refresh strategy** (unspecified).
- **Revocation strategy** (unspecified; especially relevant for JWT-like tokens).
- **Role-based access control scope** (mentioned in the original section but details unspecified;).
- **Separation of user auth vs service auth** (user tokens for API access; IAM for Firehose ingestion as stated).

10.5.5 Technology Selection Summary

The selected stack is best interpreted as a coherent set of complementary roles rather than isolated technology choices:

- **FastAPI** provides the service boundary and orchestration logic (REST contract, auth enforcement, request handling) [28].

- **LLaMA 3.1 via Ollama** provides local inference so the AI capability is available under privacy and offline constraints [26], with prototype adaptation supported by few-shot/prompting strategies [25].
- **PostgreSQL** serves as the structured system-of-record for transactional data (selection stated; detailed DB rationale is minimal in the original section and thus remains high-level).
- **Firehose** ingests operational telemetry and analytics events to support monitoring and scale, aligning with pipeline-oriented data handling patterns [30].
- **Token-based authentication** enforces access control at the API boundary, with OAuth 2.0 positioned as a formal standard option when delegated authorization is required [31].

Table 93: Technologies Selection Summary

Component	Selected Technology	Purpose
AI Processing	LLaMA 3.1 via Ollama	Intelligent system assistance
Backend API	FastAPI	Core application processing
Database	PostgreSQL	Structured data storage
Data Pipeline	Firehose	Logging and event streaming
Authentication	Token-based auth	Secure access control

Concise criteria-to-selection alignment

Because quantitative benchmarks and target performance metrics are **unspecified**, alignment is stated qualitatively based on the explicit selection criteria in the original section

Table 94: Criteria for Authentication

Selection criterion	How the chosen stack addresses it
Performance & scalability	Async-oriented API orchestration principles [29]; separation of OLTP state (PostgreSQL) and telemetry ingestion (Firehose) [30]
Security & privacy	Local LLM inference limits external data exposure [26]; token boundary at API layer [28]; service-to-service IAM boundary for telemetry (stated; details unspecified)
Cost efficiency	Local inference reduces recurring API costs; open model access supports academic experimentation [26], [27]

Ease of integration	RESTful API boundary supports modular integration [28]; Python-based backend aligns with AI integration
Suitability for academic/prototype environments	Few-shot/prompting reduces training requirements [25]; open model and survey context point to accessibility benefits [26], [27]
Long-term maintainability & extensibility	Clear service boundaries and contracts [28] allow swapping implementation details without rewriting clients

10.5.6 Assumptions & Constraints

In this subsection, there are clearly mentioned assumptions and limitations that influence the validity of the technology decisions. It is the responsibility of the user request to point out items in the list that are assumptions, or implied by an academic prototype framing; where a technical detail is not explicitly in the original section, then it is described as unspecified and not assumed.

Operational assumptions

The operational assumptions include the following:

- No financial assistance/encouragement by a holding organization. This increases the appeal of open-source and cost-stable elements and decreases the use of paid hosted LLM APIs.
- Academic prototype context. The architecture does not focus on enterprise-grade operational maturity but rather on demonstrability, modularity, and research justification.
- No dedicated DevOps/SRE team. The complexity of operation should be minimized; local first and single language integration minimizes the overhead on deployment.

Financial assumptions

Budget constraints. Continuous per-call LLM API billing is looked down upon, preferring local inference and open models [26], [27].

Preference for open-source. Conforms to the choice of LLaMA-based models [26] and the academic accessibility narrative in general in LLM surveys [27].

Technical assumptions

- Scarce GPU/ compute. This system does not use training, but instead uses LLM inference, in line with the requirement of using prompting/few-shot patterns instead of entire training

pipelines [25]. Hardware specification (CPU, RAM, GPU model, VRAM, storage) is not specified.

- Local deployment to provide privacy. LLM inference is also maintained locally without transmitting data to the outside environment; it fits the adopted methodology and aligns with the privacy concerns mentioned in the initial review.
- LLM configuration (model size variant, quantization level, maximum context length, safety filtering) is not specified.

Summary

TeamTrack incorporated a set of external services and APIs to facilitate the key system capacities, such as event logging, AI assisted insights, and secure authentication. To process the logs of high frequency activities, Firehose was included to process such stream of data effectively without affecting and slowing the main database. The LLaMA 3.1 model was used to drive the analytical capabilities of the platform and provide the AI assistance in the interview of the contribution analytics, which provides users with an additional understanding of their collaboration trends. With the other system components as its neighbours, FastAPI acts as the backend API layer, which serves as the communication nexus between them.

10.6 Evaluation Against Objectives

This section evaluates how the final selected technology stack supports the achievement of the project objectives defined in Section 2.3 of the TIS report.

The analysis shows consistency between implementation feasibility, system goals and the technical architecture.

The primary objective of TeamTrack is to enhance transparency, accountability, and structured collaboration in university group projects. The technologies selected were formulated to fulfil these aims but are attainable considering the academic limitations.

10.6.1 Objective 1: Secure User Authentication and Controlled Access

The objective requires secure account management and authorised access control .

Technology Support:

- FastAPI with JWT authentication [32].
- HTTPS encryption [33].
- Role-Based Access Control (RBAC) [34].
- Enforced relational constraints in PostgreSQL [35].

These technologies ensure:

- Only authorised users may access project work-spaces.
- Authorization is done according to specified roles.
- Secure session management
- Securing contribution information.

This directly meets the security and access control requirements which are outlined in the SRS.

10.6.2 Objective 2: Implement Role-Based Access Control (RBAC)

It should distinguish among the student members, team leaders, instructors and administrators.

Technology Support:

- FastAPI backend role validation [32].
- Role associations at the database level (Foreign key enforcement at PostgreSQL level) [34].
- Role verification by use of tokens in secured endpoints [35].

The architecture will guarantee that:

- Instructors can be given read-only audit views.
- The management rights are controlled by team leaders.
- Limited modifying skills by the members.

This provides academic equity and systematic responsibility.

10.6.3 Objective 3: Structured Project and Task Management

The system should be able to aid in project development, allocation of tasks, time limits and status management.

Technology Support:

- Nuxt.js dynamic Kanban-style board interface [36].
- FastAPI REST endpoints of operation on the project/tasks [37].
- Task-to-project mapping using PostgreSQL relational schema [35].

The loose coupling between the frontend and the backend guarantees:

- Real-time task updates
- Clear status tracking
- Management of workspace organisation.
- Scalable project structure

This facilitates formalised teamwork and work transparency.

10.6.4 Objective 4: Time Tracking and Contribution Transparency

Among the primary objectives of TeamTrack is to ensure the clarity of contributions and their simple provability, rather than guessing (or remembering) them. To accomplish that, the system maintains structured time logs on PostgreSQL, and activities such as updates and other fast and frequent ones are stored in the Firehose database. The backend subsequently fetches this information and combines them to generate useful summaries and contribution views.

Separating event logging and the regular transactional data is also a way of making the system run well. PostgreSQL remains management oriented on the main records, whereas Firehose works with the intensive flow of the activity events. This minimizes delays, keeps the data cleaner and results in analytics are more reliable. Ultimately, it helps in fostering the actual problem that the project is attempting to address and that is, the unequal division of labor that can occur in group projects.

10.6.5 Objective 5: Contribution Analytics Dashboard

TeamTrack also requires a dashboard that is simple to participate in at a glance rather than a list of bare logs. The visual part is done using the Nuxt.js dashboard components, whereas the backend is done by running aggregation queries using the stored data. And that is not all, one can use LLaMA 3.1 to transform the numbers into short summaries that can tell the reader what is happening in simple words.

Since the system retains a record of the activities and subsequently compiles them into sensible outputs, the dashboard is able to display the distribution of work within the team, members who are idle and give justifiable contribution summaries. It comes in handy more so in university projects where employees require clear indication whenever evaluating teamwork and individual effort.

10.6.6 Objective 6: AI-Assisted Support

The part of the AI in TeamTrack is not intended to judge the user but assist him. This is aimed at providing insights and recommendations without taking any decision on grading or final decision completely out of human hands. That is why the system can treat AI as a support tool but not a tool that is able to evaluate performance on its own.

LLaMA 3.1 is interlinked by controlled API calls, and can only be accessed by particular endpoints which are locked down. The backend also uses filtering based on rules in such a way that the AI

only gets the information that it requires and no more. The result is maintained transparent and understandable, thus, users can see what the AI is making its recommendations on.

These controls ensure that the AI is advisory, not making decisions and is transparent. This aligns with the ethical AI boundaries in the SRS and will prevent the possibility of the system becoming an automated grader.

10.6.7 Objective 7: Maintain System Integrity and Performance

TeamTrack must be made smooth and reliable even in the case of multiple users being online simultaneously. When the system begins dragging or freezing at the peak of the busy seasons it is counterproductive to the entire idea of utilizing it in coordination and tracking.

FastAPI should be considered to back this up, since it can allow requests to be asynchronous thus a long task does not halt all the other tasks. PostgreSQL is also important in this regard, particularly, when it comes to indexing and simple optimisation as most of the activities within the application are associated with reading and updating stored data. Docker is also helpful, as it maintains the environment the same and eliminates problems with various configurations or unmet requirements.

The other critical decision is to separate analytics and high frequency logging in the firehose database. This does not allow the heavy tracking workloads to compete with the core features such as tasks and milestones. All these choices combined make the system stable, less likely to have bottlenecks and yet remain realistic to the scope of an academic project.

10.6.8 Overall Evaluation

On the whole, the completed technology stack is what TeamTrack is attempting to do and within the constraints of an academic construction. The system brings about transparency by documenting the activity in a uniform manner and accountability by transforming the records into outright contribution analytics. Other security measures include role-based access control and encryption of communication, whereas the frontend is user-friendly since it is designed in a modular manner which is easier to extend and maintain.

The AI feature is also done with caution. It will be there to provide suggestions and summaries, but not to make grading decisions, and this makes the system fair and in accordance to the SRS requirements of ethical nature. Regarding the performance component, the division of the frontend, the backend, the databases and the AI module helps to keep the responsibilities apart and prevents the slowing of the other side.

Due to such separation, every goal is supported with a particular section of the architecture, which makes the system simpler to operate as it expands. Eventually, a sensible choice to make in this

project is the stack, which helps maintain the primary objective of equitable teamwork, enhanced accountability, and a more organized collaboration.

10.7 Limitations & Assumptions

10.7.1 Context

This section records the main limitations and assumptions of operation of TeamTrack as applied in the current milestone. This is to clarify what the system is meant to support at this point, what risks are not covered by the scope and resources, and what must be true to have the system running properly. Providing limitations and assumptions enhances readability and allows the technology evaluation and validation results in Section 9 to be in line with plausible constraints.

10.7.2 Limitations

(1) Academic-scope deployment and testing coverage.

TeamTrack has been created and tested on the academic milestone. Consequently, the system has not been subjected to scale of traffic conditions, non-homogeneous deployment conditions or adversarial conditions as would be experienced in production systems. Whereas functional testing and user-level validation will allow the project to verify that workflows work as expected with representative users, the scope of the project excludes long-term reliability testing, penetration testing, or large-scale stress testing.

(2) Performance depends on implementation and data-access patterns.

In spite of the fact that framework choice can affect baseline API performance, responsiveness is observed to also depend on database interaction Gestures and application design. Empirical studies of Python web-framework performance have demonstrated that coding overhead and CRUD/data-access characteristics of frameworks have a significant impact on the performance of API endpoints, so that even with a framework implementation of choice, inappropriate query patterns or poor endpoint design can significantly worsen the performance of endpoints [11]. This is very specific to the endpoints which consolidate contribution summaries or analytics views.

(3) RBAC scalability and administrative complexity.

Role-Based Access Control is easier to manage since it links permissions to roles, however, it may become complex due to increasing the number of roles, permissions, and exceptions. RBAC models can be used to provide a framework based approach to access control, but in practice, role engineering issues can occur due to the proliferation of access rules on a fine-grained basis [12]. In

TeamTrack, RBAC has deliberately been made simple (Student, Team Leader, Instructor) to ensure easy understanding and maintenance in an academic context, and increment in role granularity can be cautiously redesigned to ensure too much administrative complexity is not added.

(4) Token-based authentication trade-offs.

Token authentication facilitates a strict frontend backend isolation of API-based systems and is common when verifying stateless requests [16]. Nevertheless, token-based implementation may give rise to viable trade-offs including the policies of lifecycle of tokens, expiration and method of revocation. Such trade-offs are normally dealt with by conservative assumptions of expiry and safe storage in an academic prototype but complete hardening of production would necessitate more extended session and revocation model.

(5) OAuth-related security guarantees depend on correct deployment assumptions.

In places where OAuth-style flows of authorization are already present or are under consideration, such security properties are known to be sensitive to properly configured and implemented details of security-related properties (such as redirect handling, state/CSRF protection and endpoint assumptions) [14]. Although a system may not directly follow full OAuth flows in the current milestone, the adoption of OAuth in future would need cautious attention to the threat model and suggested mitigations that would be found in the literature [13], [14]. This weakness is thus posed as a risk in integrating in the future instead of a functional blemish.

(6) Service-based security risks remain relevant even without full microservice decomposition.

TeamTrack need not be entirely microservice-based, and API-based architectures have the same security risks in service boundaries (authentication consistency, validation, monitoring, and secure defaults). Empirical research on security practice in service based systems underlines that the systematic controls and operational discipline have a huge impact on the actual security practice results [15]. This constraint emphasizes the fact that security is not merely a design intent, but it is also a matter of enforced adherence and engineering disciplines.

(7) AI-assisted outputs are not guaranteed to be correct.

In case the system has AI-assisted information, the results must be seen as supportive and informational, but not authoritative. Hallucination and factual unreliability are always cited in the literature as persistent problems in large language models, especially when the prompts are underspecified or the model is not grounded [20]. In the case of TeamTrack, it implies that AI capabilities should be limited to low-stakes help (e.g., guidance, summaries, suggestions), and they should not be regarded as a source of truth to be assessed, make policy, or factual assertions.

10.7.3 Assumptions

(1) Stable user environment and access conditions.

The system is based on the assumption that the users can use a modern browser and a stable network connection that is good enough to facilitate the API-based communication between the frontend and the back-end. The interface is created based on the common usage patterns of the students that presuppose a minimum familiarity with the web-based dashboards and task management workflows.

(2) Correct user identity and role assignment.

TeamTrack presupposes that the user accounts are linked to the appropriate role, and that role assignment is an accurate reflection of the actual project responsibilities (e.g., Team Leaders are legitimate coordinators, Instructors are read-only reviewers). Access control depends on proper role configuration and thus the improper role assignment may ruin the permission enforcement even when the logic of RBAC is coded in an appropriate way [12].

(3) Trust boundaries aligned with an academic deployment.

The system presupposes a common academic threat scenario in which the users are authenticated as part of a class/team and they are not deliberate and motivated to exploit the system in a sophisticated manner. Although there are simple security measures that are made and applied, the system does not presuppose the working capability of a production security team. Such an assumption is in line with the scope of the project and its validation mechanism as well as delimiting the current security posture [15].

(4) Consistent backend configuration and secure defaults.

The system assumes that there is a shared academic threat case where the users are authenticated as members of a team/class and they do not have a deliberate and motivated initiative to misuse the system in an advanced fashion. Even though it has some uncomplicated security measures generated and implemented, the system does not assume the operability of a production security personnel. This assumption conforms to the project scope and mechanism of its validation and also limiting the security posture [15].

(5) Data integrity and database availability.

TeamTrack also presupposes that databases (both transactional and logging/analytics in case of their use) can be accessed and are reliable. The workflow of the system should be corrected by the stable storage and retrieval of the tasks, time logs, and contribution summaries. Unanticipated

database failures or inconsistent configuration may consequently affect the behavior of the system beyond what the application layer will inhibit.

10.7.4 Summary

In general, the limitations of TeamTrack which exist today are mainly due to scope of milestones, restrictions of academic deployment and focus on core functionality rather than production hardening. The assumptions presented in this section determine the predicted operating environment (stable user access, proper role allocation, and dependable database access) and give a context to the results of testing. These limitations also guide the improvement roadmap in Section 9.9 where the such improvements as the increased role granularity, the enhanced session management, and the expanded security validation may be discussed in the further iterations.

10.8 Future Enhancements

10.8.1 Context

The existing implementation of TeamTrack provides the fundamental workflows of academic collaboration though a number of improvements can be made to make the system more robust, secure, scaled, and easier to use as the system continues to be refined. The following improvements have been offered to fit into the API-based architecture of the system (frontend + backend + database) and to cope with the most widespread risks and shortcomings of the current RESTful platforms.

10.8.2 Security Hardening Enhancements

One of the key future actions would be to enhance API security measures beyond the baseline authentication and RBAC. The most recent polls of RESTful API vulnerability reveal that an authorization, authentication, and injection-based weaknesses are recurrent, which supports the idea that systematic and layered defense instead of a single control is necessary [21]. It should then be improved in future by: (i) tighter object-level authorization controls at all sensitive endpoints, (ii) standardized input verification and output encoding and (iii) automated security testing as part of the development pipeline (e.g., static analysis and targeted API security testing to known REST vulnerability taxonomies) [21].

One can also enhance the security of the session with increased token lifecycle management. JWT-based authentication has no state support, whereas JWT-based authentication supports stateless. the lifecycle policies of verification, token lifecycle policies (expiry, refresh, revocation handling) must be officially enforced to minimize exposure in case of a token leakage [16]. This consists of

temporary access tokens and renewal behaviour that is controlled and limit access to detection of abuse patterns by the server.

Lastly, security auditability, through more secure tamper-resistant logging and more event-sensitive auditing (authentication events, permission failures, administrative actions), needs to be enhanced. Audit-log research points out that logs are essential to forensic investigation, but are susceptible to manipulation (including by privileged users), encouraging the need to improve them (integrity-protected or append-only storage patterns of key audit events) [23].

10.8.3 Reliability, Abuse Prevention, and Availability

Availability protection increases with the number of teams that use TeamTrack and the volume of usage. A rapidly known control method is known as rate limiting which helps in curbing the abuse and alleviates denial-of-service type pressures at the API layer. The detailed discussion of API rate limiting mechanisms presents popular strategies (e.g., token bucket, leaky bucket, sliding window) and their applicability to DDoS-resilience and traffic shaping in API-related settings [22]. At the API gateway or middleware level (that backs up endpoints like time logs, analytics queries) rate limiting can be used to help defend critical endpoints whilst service quality remains intact to legitimate users [22].

Parallel The operation observability can be extended using structured logging, performance metrics (latency/error rates), and critical workflow traceability. This is directly contributing to quicker diagnosis and more trusted resolution of the issue in the event of defects or configuration errors in subsequent iterations (that is closely connected to your Section 7 approach of refinement).

10.8.4 Data and Analytics Enhancements

The further versions can build analytics beyond simple contribution tracing by adding: (i) more detailed reporting dashboards (trends per project, workload distribution, patterns of milestones accomplishment), (ii) customizable instructor perceptions to follow progress at cohort scale, and (iii) more powerful rule-based data validation to enhance the credibility of calculated metrics. These improvements must be consistent with the existing separation of transactional and high-frequency activity/log data, without impairing the performance of the main database but allowing scalable processing of analytics.

10.8.5 UI/UX and Workflow Improvements

Regarding usability, it is possible to add quality-of-life features to TeamTrack that include: better task filtering and search, bulk operations on team leads, better visualisations of progress, and

better onboarding (guided creation of projects, assigning roles, and time logs). The suggested improvements are intended to ease the friction of students and add transparency to instructors, but do not alter the basic permission model of Section 4.3.

10.8.6 AI/Assistance Enhancements

In case of AI-based additions, the features should be optimized in terms of reliability and safety. Since one of the drawbacks in the output of LLM is hallucinations, future AI capabilities should include protections, like constrained prompting, prompting users explicitly before following the recommendation, and displaying clear UIs that signal that the output is advice and not a command [20]. Also, it has been mentioned that a fast injection is a known risk in LLM-based applications; a study has shown that injected instructions may cause system behaviour contrary to the request given by the user, which prompts the adoption of layered mitigations (input sanitisation, content/data separation, and limited permissions on tools/actions) [24].

10.8.7 Summary

In summary, the proposed enhancements focus on strengthening TeamTrack’s long-term reliability and security while preserving its API-first architecture. Priority improvements include layered API security controls and lifecycle-safe authentication handling [21], stronger auditability through integrity-aware logging [23], and resilience measures such as rate limiting and improved observability to protect availability under increased usage [22]. As the platform evolves, analytics, workflow usability, and guided onboarding can further improve transparency and reduce friction for student teams, while any AI-assisted capability should be expanded cautiously with safeguards due to known reliability and prompt-injection risks in LLM-integrated systems [20], [24].

11. Appendices

Appendix A: Glossary

This glossary defines the key technical and system-related terms used throughout the TeamTrack TIS document. The terms reflect the technologies, architecture, and development practices referenced in the main report and supporting communication.

Table 95: Glossary

Term	Definition
------	------------

API (Application Programming Interface)	A structured interface that allows communication between frontend, backend, and external services.
Activity Log	System record that tracks user actions such as task updates, logins, and interactions.
AI Module	System component responsible for AI-assisted insights, summaries, and recommendations.
Authentication	Process of verifying user identity before allowing access to the system.
Authorization	Mechanism that controls what actions a user can perform based on assigned roles.
Backend	Server-side layer responsible for processing logic, APIs, and database communication.
CLI (Command Line Interface)	Text-based interface used by developers to interact with systems, run commands, and manage environments.
Contribution Analytics	Dashboard and reports that visualise user participation and workload distribution.
Database	Structured storage layer used to manage system data such as users, projects, and tasks.
DOM (Document Object Model)	Programming interface representing the structure of web pages, enabling interaction with UI elements.
FastAPI	Python framework used to build high-performance APIs for backend services.
Firehose Database	Logging system used to capture high-frequency activity and event data.
Frontend	User-facing interface of the system, built using Nuxt.js and Vue components.
JWT (JSON Web Token)	Secure token used for session authentication between frontend and backend.
LLaMA	Language model used in the AI module for natural language processing and insights.
MVP (Minimum Viable Product)	Initial version of the system containing core functionality for validation and testing.

Nuxt.js	Vue-based frontend framework used to develop the system interface and routing.
ORM (Object Relational Mapping)	Technique that allows backend applications to interact with relational databases using programming objects.
PostgreSQL	Relational database used to store structured system data.
RBAC (Role-Based Access Control)	Security model that restricts system actions based on user roles.
REST API	Standardised interface enabling communication between system components over HTTP.
SSR (Server-Side Rendering)	Rendering technique where pages are generated on the server before being delivered to the browser.
Session Management	Handling of user login sessions to maintain secure access.
SSPL License (Server Side Public License)	License governing the use and distribution of certain server-side software technologies.
System Architecture	Structural design showing interaction between frontend, backend, database, and AI components.
Task Management	Core system feature used to assign, track, and update project tasks.
Time Logging	Feature used to record work duration for contribution tracking.
UI (User Interface)	Visual and interactive components of the system through which users interact with TeamTrack.
User Flow	Representation of the steps a user follows when interacting with the system.
Web Application	Browser-based software system accessible through internet connectivity.

Appendix B: UI Design Artefacts

During the initial design phase of TeamTrack, the team developed the first user interface design entities to possess an infrastructure that would give them the manner in which the system will run and appear like before the developers commenced with the work. The designers created a prototype wireframe in order to show how the users would navigate through the system at the same time showing the most important elements of the main interface in the form of dashboards, task

boards, and user interfaces. In order to design a graphic image of the platform that will serve as the system of collaboration and project management in academics, the designers had to make an experimental design of a logo.

The latter artifacts enabled the team to transform their conceptual requirements into visual representations that they used in the usability, layout clarity, and interface structure consistency.

Figure 8 : Wireframe

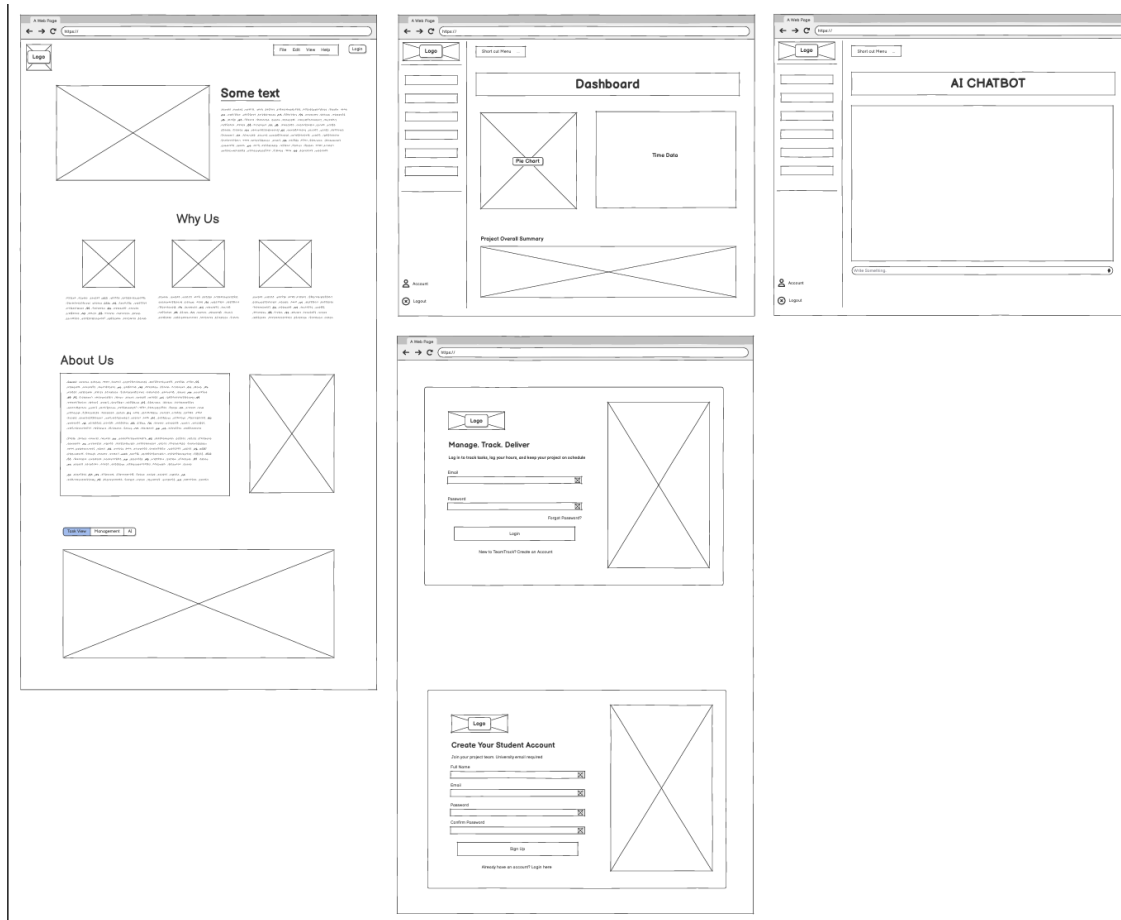


Figure 9: Logo



The first interpretation of the interface to the system and the course of the branding was initial wireframes and logos. The early period artifacts can be utilized to get an impression of how the system will be interacted with by the users and how the platform will be visually located. These designs will be improved further in the future and transformed into structured Low-Fidelity and High-Fidelity prototypes. The development will enable consideration of better usability, interface consistency, and more detailed visual development to be evaluated before full implementation.

Appendix C: User Manual

1. Purpose of the User Manual

This is a rough guideline of how to navigate and use the TeamTrack system documentation and interface. It will be targeted at ensuring that the reader/evaluator can easily access important sections of the document, to get to know and understand the components of the system, and to find the relevant supporting materials that are included in the document.

The guide will be aimed at document navigation, structural features, and usability amenities, including the table of contents, figure references, and quick navigation provision.

2. Document Navigation Structure

The document has been designed to enable it to navigate easily and find components of the system, design artefacts, and implementation details in a short time.

The following navigation elements are included:

- Table of Contents (TOC)
- List of Tables
- List of Figures
- Section Reference Links
- “Back to Top” navigation button

These elements improve readability, accessibility, and document usability for reviewers and stakeholders.

3. Table of Contents

At the onset of the document, the Table of Contents is given, which contains all the major sections, subsections, and appendices. All the entries are connected to their respective sections so that one

can navigate fast.

Any headings in the TOC can be clicked by the users to directly get to the required part of the report.

4. List of Tables

The List of Tables presents all tabular content used throughout the document like:

- Stakeholder tables
- Requirements tables
- Technology comparison tables
- Testing matrices

5. List of Figures

The List of Figures includes all visual artefacts and diagrams used within the document, such as:

- System architecture diagrams
- User flow diagrams
- Database schema diagrams
- UI wireframes and logo artefacts

Each figure is numbered and referenced for easy navigation and traceability.

6. Section Reference Links

Hyperlinked references are embedded throughout the document to allow quick navigation between related sections. These links guide readers to supporting content such as:

- Technology justification sections
- System architecture explanations
- Testing and validation details
- Implementation descriptions

This ensures continuity and prevents duplication of explanations across sections.

7. “Back to Top” Navigation Feature

To enhance usability and document accessibility, a “Back to Top” navigation feature is included at the bottom-left of each page.

This feature allows readers to:

- Instantly return to the beginning of the document
- Access the Table of Contents without manual scrolling
- Improve navigation efficiency during review

This is particularly useful for long technical documents where frequent movement between sections is required.

8. How to Use This Document Efficiently

Readers are encouraged to follow these steps when using the document:

1. Begin with the Table of Contents to determine the needed part.
2. Search visual or structured content using the List of Tables or List of Figures.
3. See internal reference explanations on related explanations.
4. The button known as the Back to Top will help you get back to the top of the document easily.

The method can guarantee easy navigation and understanding of the system documentation.

9. Intended Users of the Guide

This quick guide is intended for:

- Academic evaluators
- Project supervisors
- Team members
- Stakeholders reviewing the system documentation

It supports efficient access to project information and enhances overall document usability.

Appendix D: References

- [1] Nuxt Labs, “Nuxt Documentation – Introduction.” <https://nuxt.com/docs/4.x/getting-started/introduction>
- [2] RisingStack Engineering, “The Best JavaScript Frameworks: Pros and Cons Explained (2024).” <https://blog.risingstack.com/best-javascript-frameworks/>
- [3] Vercel, “What is React and Next.js? – Official Documentation.” <https://nextjs.org/learn/react-foundations/what-is-react-and-nextjs>
- [4] Angular Team, “Angular Documentation – Overview.” <https://angular.dev/overview>
- [5] PostgreSQL Global Development Group, “PostgreSQL 18.1 Documentation.” <https://www.postgresql.org/docs/current/>
- [6] R. Chidambaram, “PostgreSQL: Advantages and Disadvantages in 2025.” <https://www.ralantech.com/resources/postgresql-advantages-and-disadvantages/>
- [7] Oracle Corporation, “MySQL Documentation.” <https://dev.mysql.com/doc/>
- [8] RemotePlatz, “The Advantages and Disadvantages of MySQL Database [Updated 2024].” <https://www.remoteplatz.com/en/blog/advantages-and-disadvantages-of-mysql>
- [9] MongoDB Inc., “MongoDB Documentation.” <https://www.mongodb.com/docs/>
- [10] GeeksforGeeks, “MongoDB Advantages & Disadvantages.” <https://www.geeksforgeeks.org/mongodb/mongodb-advantages-disadvantages/>
- [11] B. Bednarz and M. Miłosz, “Benchmarking the performance of Python web frameworks,” *J. Comput. Sci. Inst.*, vol. 36, pp. 336–341, 2025, doi: 10.35784/jcsi.7738. <https://doi.org/10.35784/jcsi.7738>
- [12] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996, doi: 10.1109/2.485845. <https://doi.org/10.1109/2.485845>
- [13] E. Ferry, J. O’Raw, and K. Curran, “Security evaluation of the OAuth 2.0 framework,” *Inf. Comput. Secur.*, vol. 23, no. 1, pp. 73–101, 2015, doi: 10.1108/ICS-12-2013-0089. <https://doi.org/10.1108/ICS-12-2013-0089>
- [14] D. Fett, R. Küsters, and G. Schmitz, “A comprehensive formal security analysis of OAuth 2.0,” in *Proc. 2016 ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2016, pp. 1204–1215, doi: 10.1145/2976749.2978385. <https://doi.org/10.1145/2976749.2978385>

- [15] A. Rezaei Nasab, M. Shahin, S. A. Hoseyni Raviz, P. Liang, A. Mashmool, and V. Lenarduzzi, “An empirical study of security practices for microservices systems,” *J. Syst. Softw.*, vol. 198, art. no. 111563, Apr. 2023, doi: 10.1016/j.jss.2022.111563. <https://doi.org/10.1016/j.jss.2022.111563>
- [16] S. Dalimunthe, E. Hasri Putra, and M. A. F. Ridha, “RESTful API security using JSON Web Token (JWT) with HMAC-SHA512 algorithm in session management,” *IT J. Res. Develop. (ITJRD)*, vol. 8, no. 1, pp. 81–94, Aug. 2023, doi: 10.25299/itjrd.2023.12029. <https://doi.org/10.25299/itjrd.2023.12029>
- [17] L. Di Grazia and M. Pradel, “The evolution of type annotations in Python: An empirical study,” in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE ’22)*, Nov. 2022, pp. 209–220, doi: 10.1145/3540250.3549114. <https://doi.org/10.1145/3540250.3549114>
- [18] F. Khan, B. Chen, D. Varró, and S. McIntosh, “An empirical study of type-related defects in Python projects,” *IEEE Trans. Softw. Eng.*, vol. 48, no. 8, pp. 3145–3158, Aug. 2022, doi: 10.1109/TSE.2021.3082068. <https://doi.org/10.1109/TSE.2021.3082068>
- [19] J. Gmys, T. Carneiro, N. Melab, and D. Tuytens, “A comparative study of high-productivity high-performance programming languages for parallel metaheuristics,” *Swarm Evol. Comput.*, vol. 57, art. no. 100720, Sep. 2020, doi: 10.1016/j.swevo.2020.100720. <https://doi.org/10.1016/j.swevo.2020.100720>
- [20] S. Abdullahi, K. U. Danyaro, and H. Chiroma, “The rise of hallucination in large language models: systematic reviews, performance analysis and challenges,” *Cluster Computing*, vol. 29, art. no. 124, Feb. 2026, doi: 10.1007/s10586-025-05891-z. <https://doi.org/10.1007/s10586-025-05891-z>
- [21] F. Tanveer, F. Iradat, W. Iqbal, and A. Ahmad, “Towards Secure APIs: A Survey on RESTful API Vulnerability Detection,” *Computers, Materials & Continua*, vol. 84, no. 3, pp. 4223–4257, Jul. 2025, doi: 10.32604/cmc.2025.067536. <https://doi.org/10.32604/cmc.2025.067536>
- [22] M. Manoharan, “API Rate Limiting Mechanisms in SaaS Applications: A Systematic Analysis of DDoS Protection Strategies,” *Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol.*, vol. 10, no. 6, pp. 1787–1798, Dec. 2024, doi: 10.32628/CSEIT241061223. <https://doi.org/10.32628/CSEIT241061223>
- [23] A. Ali, M. Ahmed, and A. Khan, “Audit Logs Management and Security – A Survey,” *Kuwait J. Sci.*, vol. 48, no. 3, Jun. 2021, doi: 10.48129/kjs.v48i3.10624. <https://doi.org/10.48129/kjs.v48i3.10624>
- [24] X. Liu, Z. Yu, Y. Zhang, N. Zhang, and C. Xiao, “Automatic and Universal Prompt Injection Attacks against Large Language Models,” *arXiv*, Mar. 2024, doi: 10.48550/arXiv.2403.04957. <https://doi.org/10.48550/arXiv.2403.04957>
- [25] T. Brown et al., “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems*, 2020. <https://doi.org/10.48550/arXiv.2005.14165>
- [26] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models,” *Meta AI*, 2023. <https://doi.org/10.48550/arXiv.2302.13971>

- [27] W. X. Zhao et al., “A Survey of Large Language Models,” *ACM Computing Surveys*, 2023. <https://doi.org/10.48550/arXiv.2303.18223>
- [28] L. Richardson and S. Ruby, *RESTful Web Services*. O’Reilly Media, 2007. <https://www.oreilly.com/library/view/restful-web-services/9780596529260/>
- [29] S. Tilkov and S. Vinoski, “Node.js: Using JavaScript to Build High-Performance Network Programs,” *IEEE Internet Computing*, 2010. <https://doi.org/10.1109/WICOM.2010.5601098>
- [30] T. White, *Hadoop: The Definitive Guide*. O’Reilly Media, 2012. <https://www.oreilly.com/library/view/hadoop-the-definitive/9781449328917/>
- [31] D. Hardt, “The OAuth 2.0 Authorization Framework,” IETF RFC 6749, 2012. <https://datatracker.ietf.org/doc/html/rfc6749>
- [32] S. Ramírez, “FastAPI Documentation,” FastAPI, 2024. <https://fastapi.tiangolo.com/>
- [33] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” IETF RFC 8446, 2018. <https://datatracker.ietf.org/doc/html/rfc8446>
- [34] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-Based Access Control Models,” *Computer*, vol. 29, no. 2, pp. 38–47, 1996, doi: 10.1109/2.485845. <https://doi.org/10.1109/2.485845>
- [35] PostgreSQL Global Development Group, “PostgreSQL Documentation – Constraints and Indexes,” 2024. <https://www.postgresql.org/docs/current/ddl-constraints.html>
- [36] Nuxt Labs, “Nuxt Documentation,” 2024. <https://nuxt.com/docs>
- [37] M. Fielding, “Architectural Styles and the Design of Network-Based Software Architectures,” Ph.D. dissertation, Univ. of California, Irvine, 2000. https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm
- [38] Amazon Web Services, “Amazon Kinesis Data Firehose Documentation,” 2024. <https://docs.aws.amazon.com/firehose/>
- [39] Meta AI, “Introducing LLaMA,” 2023. <https://ai.meta.com/llama/>
- [40] European Commission, “Ethics Guidelines for Trustworthy AI,” 2019. <https://digital-strategy.ec.europa.eu/en/library/ethics-guidelines-trustworthy-ai>
- [41] Docker Inc., “Docker Documentation,” 2024. <https://docs.docker.com/>
- [42] FastAPI, “Security and Authentication,” FastAPI Documentation, 2024. <https://fastapi.tiangolo.com/tutorial/security/>
- [43] OWASP Foundation, “OWASP Top Ten Web Application Security Risks,” OWASP, 2023. <https://owasp.org/www-project-top-ten/>

[44] PostgreSQL Global Development Group, “PostgreSQL Performance Tips,” PostgreSQL, 2024. <https://www.postgresql.org/docs/current/performance-tips/>

[45] FastAPI, “FastAPI Documentation – High-Performance API Framework for Python,” 2024. <https://fastapi.tiangolo.com/>

[46] Nuxt, “Nuxt.js Documentation – Vue-Based Frontend Framework,” 2024. <https://nuxt.com/docs>

[47] Docker Inc., “Docker Documentation – Containerization and Application Environment Management,” 2024. <https://docs.docker.com/>

[48] PostgreSQL Global Development Group, “PostgreSQL Documentation – Relational Database Architecture and Management,” 2024. <https://www.postgresql.org/docs/>

[49] K. Schwaber and J. Sutherland, “The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game,” Scrumguides.org, Nov. 2020. <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>

[50] Atlassian, “Bitbucket Documentation – Git Code Management and Version Control,” 2024. <https://support.atlassian.com/bitbucket-cloud/>

Appendix E: Attribution Matrix (WBS)

Table 96: WBS Structure

TASKS		TEAM MEMBERS				
Main Part	Sub Part	Omar Waheed Khan	Muhammad Hashir	Maria Waheed	Zernish Shahid Nazir	Mohammad Siraj Hussain
Executive Summary	N/A					
Introduction	Background / Context					
	Problem Statement					
	Aim and Objectives					
	Scope (In-Scope / Out-of-Scope)					
	Assumptions and Constraints					

Project Overview	Stakeholders					
	Roles and Responsibilities					
	Project Deliverables					
	Tools and Technologies Used					
Technology Stack Overview	Overview of Technologies					
	Frontend Technologies					
	Backend Technologies					
	Database Technologies					
	AI Technologies					
	Security and Performance					
	Final Outcomes					
Updated Software Requirements	Functional Requirements					
	Non-Functional Requirements					
	User Roles and Permissions					
	Use Cases					
System Design	System Architecture					
	Database Design (ERD / Schema)					
	UI/UX Design Overview					
	User Flow Diagram					

	System Flow Diagram					
Implementation	Development Approach / Methodology					
	Backend Implementation					
	Frontend Implementation					
	AI/Chatbot Module					
	Security Controls Implemented					
Testing and Validation	Proposed Testing Approach					
	Test Scenarios and Outcomes					
	Issue Resolution and Refinement Strategy					
	User Acceptance Testing (UAT)					
Project Management	Communication Plan					
	Agile Development					
	Meeting Minutes					
	Integrating Kanban					
Technology Investigation & Justification	Final Technology Stack					
	Frontend Technology Evaluation					
	Backend Technology Evaluation					

	Database Technology Evaluation					
	External Services & APIs					
	Evaluation Against Objectives					
	Limitations & Assumptions					
	Future Enhancements					
Appendices	Glossary					
	UI Design Artefacts					
	User Manual / Quick Guide					
	References					
	Work Breakdown Structure					
	toggl reports					
Formatting	N/A					

Appendix F: Toggl Reports

Table 97: Toggl Report Table

Member Name	Toggl Report (Links)
Muhammad Hashir	Link
Omar Waheed Khan	Link
Maria Waheed	Link
Zernish Shahid Nazir	Link
Mohammed Siraj Hussain	Link

Work Distribution Summary for Milestone 2

Ownership of the Technical Implementation Specification (TIS) was distributed across team members based on section relevance, technical expertise, and alignment with development responsibilities. Each section and subsection was assigned to specific contributors to ensure clarity of ownership and accountability. The distribution followed the official TIS Table of Contents, ensuring that all aspects including system design, implementation, testing, and project management were comprehensively addressed. Individual contributions were later consolidated into a cohesive and structured document, maintaining consistency in formatting, terminology, and technical depth.

Section-Wise Contribution Tables

1. Executive Summary

Table 1.1: Work Distribution for Executive Summary

Section / Subsection	Primary Contributor	Contribution Details
Executive Summary	Muhammad Hashir	Drafted the executive overview of the TIS, summarising system purpose, implementation approach, and key technical decisions aligned with TeamTrack objectives.

2. Introduction

Table 2.1: Work Distribution for Introduction

Section / Subsection	Primary Contributor	Contribution Details
2. Introduction	Zernish Shahid Nazir, Maria Waheed	Shared ownership of the Introduction section, with Zernish leading foundational subsections and Maria contributing to scope and constraints refinement.
2.1 Background / Context	Zernish Shahid Nazir	Drafted the project background, contextualising TeamTrack

		within academic collaboration challenges.
2.2 Problem Statement	Zernish Shahid Nazir	Defined the core problem addressed by the system, focusing on inefficiencies in team collaboration and tracking.
2.3 Aim and Objectives	Zernish Shahid Nazir	Documented system aims and measurable objectives aligned with project goals.
2.4 Scope	Zernish Shahid Nazir, Maria Waheed	Defined system boundaries, including in-scope and out-of-scope functionalities.
2.5 Assumptions and Constraints	Zernish Shahid Nazir, Maria Waheed	Documented technical, operational, and academic constraints along with key assumptions.

3. Project Overview

Table 3.1: Work Distribution for Project Overview

Section / Subsection	Primary Contributor	Contribution Details
3. Project Overview	Muhammad Hashir, Omar Waheed Khan	Shared ownership, with Hashir leading structural definitions and Omar contributing to technical tools.
3.1 Stakeholders	Muhammad Hashir	Identified and documented all stakeholders including students, instructors, and administrators.
3.2 Roles and Responsibilities	Muhammad Hashir	Defined responsibilities of each stakeholder within the system lifecycle.
3.3 Project Deliverables	Muhammad Hashir	Documented all project deliverables across development phases.
3.4 Tools and Technologies Used	Omar Waheed Khan	Evaluated and documented development tools,

		frameworks, and supporting technologies.
--	--	--

4. Technology Stack Overview

Table 4.1: Work Distribution for Technology Stack Overview

Section / Subsection	Primary Contributor	Contribution Details
4. Technology Stack Overview	Muhammad Hashir	Documented the complete system technology stack including frontend, backend, database, AI components, and performance considerations across all subsections.

5. Updated Software Requirements

Table 5.1: Work Distribution for Updated Software Requirements

Section / Subsection	Primary Contributor	Contribution Details
5. Updated Software Requirements	Siraj Hussain, Omar Waheed Khan, Maria Waheed	Shared ownership across requirements refinement and documentation.
5.1 Functional Requirements	Mohammad Siraj Hussain	Updated and refined system functional requirements based on implementation needs.
5.2 Non-Functional Requirements	Mohammad Siraj Hussain	Documented updated system constraints including performance, scalability, and security.
5.3 User Roles and Permissions	Omar Waheed Khan	Defined role-based access control and permission structures.
5.4 Use Cases	Omar Waheed Khan, Maria Waheed	Developed detailed use cases describing system interactions and workflows.

6. System Design

Table 6.1: Work Distribution for System Design

Section / Subsection	Primary Contributor	Contribution Details
6. System Design	Mixed Contributions	Collaborative effort across architecture, database, UI, and system flows.
6.1 System Architecture	Maria Waheed	Designed system architecture including component interactions and deployment structure.
6.2 Database Design	Zernish Shahid Nazir	Developed database schema, ERD, and data relationships.
6.3 UI/UX Design Overview	Muhammad Hashir, Omar Waheed Khan	Designed interface structure and user interaction principles.
6.4 User Flow Diagram	Mohammad Siraj Hussain	Created user flow diagrams representing user journeys within the system.
6.5 System Flow Diagram	Mohammad Siraj Hussain	Developed system-level flow diagrams illustrating data and process flow.

7. Implementation

Table 7.1: Work Distribution for Implementation

Section / Subsection	Primary Contributor	Contribution Details
7. Implementation	Mohammad Siraj Hussain	Led all implementation-related sections including development approach, backend and frontend implementation, AI integration, and security controls.

8. Testing and Validation

Table 8.1: Work Distribution for Testing and Validation

Section / Subsection	Primary Contributor	Contribution Details
----------------------	---------------------	----------------------

8. Testing and Validation	Omar Waheed Khan, Muhammad Hashir	Shared ownership of testing strategy and validation processes.
8.1 Proposed Testing Approach	Muhammad Hashir	Defined testing methodologies and validation strategies.
8.2 Test Scenarios and Outcomes	Muhammad Hashir	Documented test cases, expected results, and evaluation outcomes.
8.3 Issue Resolution and Refinement Strategy	Omar Waheed Khan	Defined debugging, issue tracking, and refinement workflows.
8.4 User Acceptance Testing	Omar Waheed Khan	Documented UAT procedures and validation criteria.

9. Project Management

Table 9.1: Work Distribution for Project Management

Section / Subsection	Primary Contributor	Contribution Details
9. Project Management	Maria Waheed, Muhammad Hashir	Shared ownership of management practices and coordination strategies.
9.1 Communication Plan	Muhammad Hashir	Defined internal and external communication strategies.
9.2 Agile Development	Muhammad Hashir	Documented Agile methodology, sprint cycles, and workflow.
9.3 Meeting Minutes	Muhammad Hashir	Recorded and maintained meeting documentation.
9.4 Integrating Kanban	Maria Waheed	Documented Kanban implementation for workflow tracking.

10. Technology Investigation & Justification

Table 10.1: Work Distribution for Technology Investigation & Justification

Section / Subsection	Primary Contributor	Contribution Details
----------------------	---------------------	----------------------

10. Technology Investigation & Justification	All Contributors	Collaborative evaluation and justification of technologies used.
10.1 Final Technology Stack	Maria Waheed	Documented final selected technologies and rationale.
10.2 Frontend Technology Evaluation	Zernish Shahid Nazir	Evaluated frontend frameworks and tools.
10.3 Backend Technology Evaluation	Omar Waheed Khan	Evaluated backend technologies and frameworks.
10.4 Database Technology Evaluation	Zernish Shahid Nazir	Evaluated database solutions and selection criteria.
10.5 External Services & APIs	Muhammad Hashir	Documented external integrations and APIs.
10.6 Evaluation Against Objectives	Maria Waheed	Assessed technologies against project goals.
10.7 Limitations & Assumptions	Omar Waheed Khan	Identified system limitations and constraints.
10.8 Future Enhancements	Omar Waheed Khan	Proposed potential system improvements and scalability options.

Appendices

Table A.1: Work Distribution for Appendices

Appendix	Primary Contributor	Contribution Details
A. Glossary	Muhammad Hashir	Compiled definitions and terminology used in TIS.
B. UI Design Artefacts	Muhammad Hashir	Documented UI designs and visual artefacts.
C. User Manual / Quick Guide	Muhammad Hashir	Prepared user guidance documentation.
D. References	Omar Waheed Khan, Maria Waheed	Compiled and formatted references.
E. Work Breakdown Structure	Omar Waheed Khan	Documented WBS structure.
F. Toggl Reports	All Members	Submitted individual time tracking reports.

Formatting

Table F.1: Work Distribution for Document Formatting

Section	Primary Contributor	Contribution Details
Document Formatting	Omar Waheed Khan, Muhammad Hashir	Ensured consistency in formatting, structure, and visual presentation of the TIS document.

Final Contribution Percentage Summary

Team Member	Primary Contribution Areas	Approx. Ownership
Muhammad Hashir	Executive Summary, Stakeholders, Technology Stack, Testing Strategy, Project Management, Appendices, Formatting	22%–26%
Omar Waheed Khan	Tools & Technologies, User Roles, Use Cases, Testing Refinement, Technology Evaluation, References, Formatting	20%–24%
Maria Waheed	Scope, Architecture, UI/UX, Kanban, Technology Evaluation	16%–20%
Zernish Shahid Nazir	Introduction, Database Design, Frontend & Database Evaluation	16%–20%
Mohammad Siraj Hussain	Requirements, System Flows, Implementation	20%–24%

Closing Statement

Primary ownership of specific sections and subsections in the TIS was assigned to individual team members to ensure accountability and clarity in contribution. Despite this structured distribution, the Technical Implementation Specification remained a fully collaborative effort. All members participated in review, refinement, and consolidation of the document prior to submission. This approach ensured both individual responsibility and collective ownership, while clearly documenting contribution attribution in alignment with project requirements.

TIS Version 2.0 Update (04/03/26)

Client feedback led to an update of the Technology Investigation Study (TIS) on 4 March 2026. The client noted that the technology evaluation sections were thorough in their comparisons, but the reasoning behind each final selection was not clearly written in one place and requested that each of the major evaluation section should include a concise summary that explicitly states why the chosen technology was selected over the alternatives considered that we mentioned.

Making the document easier to follow was one goal and also to ensure that architectural decisions documented in the Software Architecture Specification (SAS) could be clearly traced back to the research conducted in the TIS. As a result, summary paragraphs were written and added to four sections. Section 10.2 covered the frontend evaluation. Section 10.3 covered the backend. Section 10.5 addressed the database assessment. Section 10.6 dealt with API architecture and external services.

1. For the frontend summary, Nuxt.js was confirmed as the selected framework. The reasoning cited its component based architecture, server-side rendering support, and suitability for building the interactive dashboards and contribution analytics interfaces that are central to TeamTrack.
2. The backend summary put together what was mentioned and confirmed Python with FastAPI as the chosen foundation. Asynchronous request handling, lightweight flexibility, and strong alignment with the system's REST API and modular service architecture were the main reasons.
3. PostgreSQL was confirmed as the primary database in the database summary. Went over relational integrity, complex querying support, and transactional consistency as the critical factors and how all of these matter given the interconnected nature of TeamTrack's project activities, time logs, and user data.
4. The external services summary outlined the role of AWS Firehose for high frequency activity log processing. LLaMA 3.1 was included for AI assisted contribution analytics, FastAPI remained the central API communication layer tying all components together.

None of the technology selections changed as a result of these updates. The purpose was solely to improve readability and strengthen traceability between the investigative research process and the final architectural decisions made for TeamTrack.

8. SAS Report



Executive Summary

The current milestone report shows the Software Architecture Specification (SAS) of the proposed project an AI-assisted collaborative project management and time tracking system to be used by the university students during group assignments. The report is a continuation of the previous milestone since it is based on problem identification and initial planning, shifting to the more framework-based architectural and design-oriented phase. It is primarily aimed at defining the way the proposed system will be structured, what huge components it will consist of, how these components will interact and how the entire solution will serve both students and lecturers in a useful academic setting.

The project is driven by the universal challenges that are encountered when group work among students takes place. Poor communication, uneven allocation, and inability to see contributions, as well as accountability, make collaboration ineffective in many academic teams. The problems also tend to interfere with the performance of the team as well as equitable evaluation by the lecturer. The solutions to these issues presented in the proposed system include the implementation of a centralized system through which project task management, time tracking, contribution monitoring, and AI-assisted analysis are combined in the same environment.

On a higher level, the **system is planned to** accommodate the following **objectives**:

- Enhance student group work transparency.
- Enhance better task ownership and progress.
- Help maintain a more just contribution monitoring and academic review.
- Increase accountability by developing work logging.
- Create AI-based summaries and insights to make improved decisions.

Establish a better organized and accessible working atmosphere of academic projects.

This SAS report describes how the system will be designed and why the key design decisions have been made. It defines the frontend and database architecture, purpose of the database, purpose of the authentication and access control, and external service usage and AI-based processing. This report also demonstrates the flow of information as it is likely to work through the platform after the interaction with the user is made and finally to the back end processing, storage and presentation. This way, the document acts as a technical design plan of the subsequent development and implementation stages of the capstone project.

The system architecture has been broken down into a number of key areas in such a way that the project can be designed in a manageable and modular way.

These areas include:

- **Architectural overview**, which determines the general organization and the limits of the system.
- **Component and module design**, which determines the duty of the significant subsystems.
- **Data architecture, how the project**, task, user and contribution data are going to be stored.
- **Integration architecture**, which takes into account APIs, AI services and external tools.
- **Security and ethical issues** that deal with privacy, access control and responsible system behavior.
- **Detailed system design**, which breaks the architecture down into pages, services, endpoints and internal flows.

Besides architecture, other sections incorporated in this report involve aspects of design validation, prototyping, agile planning and project risks. These paragraphs are significant as they demonstrate that the project is not merely being handled as a technical idea, it is a designed development project. The fact that the user interface design, interaction diagrams, validation activities, meeting records, and development planning are included shows that the team is experiencing a realistic systems development process. This enables the report to bridge the gap between technical design and practical implementation, as it makes the development and testing of the proposed system to be monitored and meaningful.

The other significant contribution of such milestone is that it assists in transforming previous project concepts to a more tangible implementation path. The report does not just stay at the level of general objectives, but it starts formulating the actual system structure that will help the objectives. These involve the determination of the planned modules, potential service roles, anticipated user interactions, and the place of AI generated outputs in this platform. It also gives a firm basis to be used in the future steps like prototype development, testing, refining and final evaluation.

Generally, this milestone is a significant milestone in the project life cycle. It documentalizes the technical design of the proposed solution and gives a clear architectural grounds to the further work. The report also enables consistency in the team and aids in making decisions in a more effective way by the way of documenting the structure, responsibilities, interactions, and design considerations of the system. Finally, the intention of this project is to provide a practical and intelligent platform of academic collaboration enhancing the transparency of teamwork, accountability, and better management of university group projects.

1. Overview

1.1 Background and Context

TeamTrack is a project being built as a web based collaboration platform that is aimed at supporting student project teams to enhance coordination of tasks, transparency, and monitoring of contribution. The team in the past set the system requirements in the form of the SRS and the technical direction in the form of the TIS, including the target stack, the security strategy and the workflows.

Milestone 3 is a continuation of that with the transition of requirements definition to design and architecture specification. It is not simply a list of system functions but instead a record of how the system will be made up of components, how these components are going to communicate with each other and how the UI is going to help achieve the necessary user journeys. This milestone also makes sure that design decisions are also consistent with the fine-tuned SRS and the implementation strategy outlined in the TIS.

Summary: Milestone 3 is interim whereby requirements are transformed into design-ready artefacts, the implementation is more organised, fewer reworks in the future.

1.2 Purpose of This Milestone Report

This Milestone 3 report is intended to have a design-level specification that is understandable enough to make development not be too complicated, and be understandable enough to be evaluated by academics. It conveys the desired architecture, component interaction and UI design intent in such a fashion that implementation can be done with a reduced number of unknowns and reduced document to document discrepancies.

This milestone report aims to:

- **Define an implementation-ready design baseline** by specifying an architectural style, system boundaries and distribution of responsibilities between modules.
- **Explain how the system works as a whole** by explaining the main interaction flows (e.g. authentication, task updates, contribution logging, and analytics viewing).
- **Provide UI evidence and design reasoning** using wireframes/prototypes and user journey coverage in such a way that the interface is aligned to actual use cases.
- **Demonstrate security-by-design thinking**, especially the RBAC implementation sites, the clarity of the authentication flow, and the responsible processing of AI-aided features.
- **Maintain traceability across deliverables** through demonstrating consistency between SRS requirements, TIS planning and SAS/design artefacts in the documents presented in this milestone.

Summary: The Milestone 3 report is basically the blueprint stage- it is the one that displays how the system will come out to be before it is finally built.

1.3 Scope and Key Objectives

The Milestone 3 is concentrated on architecture and design-level specification but not on full development. The sphere is restricted to the definition of the structure, interface, and direction of the UI needed to proceed to the implementation with a certain level of certainty.

Scope includes:

- **System context and boundaries**, including what is inside TeamTrack and what external dependencies exist (tools/APIs/services).
- **Architecture and module planning**, How the frontend, backend services, databases, and AI support components are decoupled and what each has to do.
- **Data model and data flow design**, the way in which important objects (users, projects, tasks, time logs, and contribution records) are represented in the system and the way in which they are transferred.
- **Component interaction descriptions**, concentrating only on the key flows in order to have interfaces clear without excessive documentation of implementation detail.
- **UI artefacts and screen flows**, low and high-fidelity prototypes, as well as the rationale of the improvements realized via feedback.
- **Design-level validation planning**, whereby the design is reviewed to ensure that it can be reviewed against the requirements before any implementation takes place.

Key objectives:

1. **Convert requirements into a clear design structure** such that every functional area has an architectural home and ownership is apparent.
2. **Strengthen traceability** between among SRS - TIS - SAS, whereby requirements and technical planning are documented and used to justify the design artefacts.
3. **Improve implementation readiness** Endpoints, data modeling, responsibilities, and UI flows visible enough to accomplish sprint labor.
4. **Reduce integration and scope risk early** the more it is to establish interfaces and boundaries at the earliest and avoid such during development.

Summary: The scoping is architectural and focuses on the design of a stable architecture and a path of the UI that may be implemented by the team gradually and controlled, in a gradual and controlled way, using sprints.

1.4 Summary of Changes Since Previous Milestone

This section will provide the updates and changes that are of utmost importance to the Software Requirements Specification (SRS) since the last submission of the milestone. The revised features are more clear, have more compliance with the Technical Implementation Specification (TIS), and the refinement of the system design decision following the technical planning and review.

1.4.1 Refinement of Functional Requirements

The Functional Requirements (Section 4) were rearranged and re-read to increase the clarity, consistency, and traceability. The following were the improvements that were done:

- **Clear separation of functional areas** in such a way that similar requirements are grouped on a rationalized fashion (e.g., authentication, project/workspace management, task handling, contribution tracking and AI-assisted support). This does away with redundancy and simplifies the document.
- **Consistent use of formal “shall” statements** in order to enhance the quality of the academic work and maintain the requirements as measurable, to enable later testing and validation.
- **Stronger mapping across roles → use cases → system behaviours**, and the actions and permissions of each role have a clear connection to what the system is required to do.
- **Removal of ambiguity in wording** through narrowing down of phrases that might have been taken differently, enhances reliability in the translation of requirements into design choices.

Summary: The functional requirements are now more understandable, role-oriented, and designed in a manner that enables the creation of the direct test cases.

1.4.2 Addition of TIS-Aligned Non-Functional Requirements Summary

It added another subsection known as 5.0 Non-Functional Requirements (TIS REPORT). In this section, the implementation-oriented summary at high level will be presented covering:

- **Performance strategy** by connecting anticipated system responsiveness to intended architectural decisions (e.g. layered design, scalable services).
- **Security enforcement approach** by authentication, authorisation, and access control will be facilitated using the tech stack that will be implemented.
- **Scalability considerations** by describing how the design is capable of scaling up as user/projects increase without significant redesign.

- **Reliability and maintainability planning** through linking the quality requirements of the system to realistic implementation decisions e.g. the modularity and responsible separation.

Summary: Under this section, the traceability is enhanced by literally closing the gap between the quality requirements in the SRS and the practical technical direction in the TIS.

1.4.3 Expansion of Detailed Non-Functional Requirements

The Non-Functional Requirements had been elaborate and elaborated to cover the complete system quality attributes. Improvement was done in:

- **Security:** more specifications regarding authentication behaviour, RBAC implementation, audit logging anticipations, and session/token policies to curb abuse and aid responsibility.
- **Performance:** better definition of response-time expectations, concurrent usage assumption, and realistic performance targets which can be tested.
- **Usability:** enhanced expectations of interface clarity, feedback messages, and consistency in order to enable users to follow through with workflow without being confused.
- **Availability and reliability:** more concrete planning regarding uptime anticipations, recovery concepts, and backup deliberations in order to prevent the risk of data loss.
- **Data integrity and privacy:** sophisticated phrasing of data protection guidelines, the management of sensitive data, and regularity regulations of the information stored.
- **Ethical and responsible AI use:** increased focus on scope of AI-aided features to help users, but not create misuse or damage academic integrity.

Summary: Non-functional requirements are now more comprehensive, measurable and capture what is practically intended to be achieved by the system.

1.4.4 Strengthening of Role-Based Access Control (RBAC)

The RBAC model was reduced to make it more clear and easier to enforce. Updates include:

- **Clearer role definitions** for Student Member, Team Leader, Instructor, and System Administrator on what they can see, create, edit, and manage.
- **Layered enforcement strategy** In which access control is enforced at the UI level (the unauthorised options are not displayed) and the backend level (the unauthorised requests are rejected even when typed in manually).
- **Better alignment with use cases**, including that the permissions are consistent with the system workflows and that the system is not conflicting with what the users are permitted to do as opposed to what the system can permit them to do.

- **Security and integrity improvement**, and this attribute is of paramount importance in an academic context where the tracking of the contributions and accountability are the paramount outcomes.

Summary: RBAC is now easier to comprehend, enforce and make related to actual workflow, thus contributing to the security of the entire system.

1.4.5 Structural and Consistency Improvements

Document structure was also verified and fixed to a better version:

- **Logical flow between sections**, reduction of jumps and enabling the reader to follow the requirements - quality attributes - supporting design direction.
- **Consistency in terminology**, so the fundamental words (role names, modules, system features) may be the same throughout the SRS and they are aligned with the TIS language.
- **Alignment between SRS and TIS**, where one stipulates that it is required based on the approach to doing it, which has already been documented.
- **Clear separation of requirement-level vs implementation-level detail**, so that the SRS does not get too technical but can be followed to the implementation plan.

Summary: Non-functional requirements are now more comprehensive, measurable and capture what is practically intended to be achieved by the system.

1.4.6 Overall Summary

In general, the modifications, which have occurred since the last milestone, enhance the quality of the SRS by increasing the clarity of the requirements, their traceability, and compatibility with the TIS. These improvements enhance the project to prepare Milestone 3 design documentation and minimise the risk of contradictions as the project enters implementation.

2. Project Description and Goals

2.1 Problem Statement

Academic group projects are one of the primary components of any university course and education, yet, even considering their importance, the tools that the existing platforms offer to facilitate collaboration between students are deficient in many ways related to the lack of the fairness, accountability, and transparency, mainly because the existing platforms did not consider the unique needs of academic teamwork.

In many cases students rely on a mix of different apps like Trello for task management, WhatsApp or any other apps for communication. Even though these tools are useful individually and fulfil their purposes, they aren't designed specifically for academic group work meaning that it makes it

difficult to clearly track who did what, how much effort each member has contributed, and whether responsibilities were distributed and taken care of properly. This is fragmented and the fragmented combination of task management leads to issues like separation and changing of contexts, losing history of decisions, poor traceability of contributions. This also increases the possibility that some critical discussions take place outside the project board and so it is not easy to establish who made certain decisions.

Issues created by the fragmented workflow in academic environments include:

- History of conversation and decision making is not directly associated with project activities directly.
- Important updates can get buried in chat threads.
- Instructors can hardly see the real work of individual students.
- Contribution levels are often judged subjectively instead of based on structured data.

What university projects actually require is a collection of functionalities that can allow contribution transparency, meaningful supervision, and early identification of any disengagement of students. Because of this fragmentation the workload in projects is often distributed unequally and also there is the so called free-ride behaviour where some members might have a higher burden of work than others might seem to have, but in reality, they contribute very little. Another limitation is that existing project management tools are mainly designed for corporate environments and they usually focus on productivity and completing tasks instead of any academic group needs.

The possibility to observe teamwork in a systematic and reliable way should be considered as a priority characteristic. TeamTrack bridges this gap directly with a unified environment that enables task management, and contribution tracking, and communication to co-exist with the specific aim of academic use. When these attributes are distributed to different tools, there is disjointed collaboration, unclear individual tasks, and the fairness of any evaluation becomes even harder to maintain.

2.2 Proposed Solution Overview

TeamTrack's plan is to be developed as a web-based platform with a straightforward goal: improving fairness, transparency, and accountability within university group projects.

When working in an academic team, it can be difficult to determine who actually contributed to which parts of the final submission. Work distribution can become confusing, and individual efforts may get lost in the collective output. The point is that through the connection of tasks, discussions, and records of activities, teams can organize their work and have a stable history of personal input over time.

PostgreSQL is the central transactional database used as a storage, which contains the main entities like users, projects, tasks, and records of activity. Various high frequency events may be handled using a Firehose style of logging, which prevents the slowness of the main operational database by analytical processing. The security aspect is handled through JWT based

authentication + role-based access control, ensuring sessions remain secure, and permissions are consistently enforced.

The architecture is modular. The frontend is built with Nuxt.js, it is a Vue based framework, the backend exposes a REST API implemented in Python using FastAPI.

Interconnected capabilities supporting the solution:

1. Joint management of tasks where projects are divided into various tasks with each having a deadline.
2. Time tracking and contribution analytics, which together make summaries and dashboards from time logs and activity records, gives visibility into team participation
3. Contextual communication ensuring discussions are linked directly to the relevant project work instead of just existing alone
4. An AI assisted component designed to provide advisory insights

2.3 Target Users and Use Cases

TeamTrack is intended to be used on academic group projects whereby the transparency of contributions and coordination is important. The main users are student team members, who use the platform on a day-to-day basis to fulfill their given assignments, communicate with their team and update their time/effort logs. Team leaders are a second user group core, as they manage the workspace and organize the project environment by establishing the project environment, assigning tasks, and tracking patterns of progress/participation within the team. Teachers/ managers get accounts by the optional audit view which is read only and enables them to see the overall progress and contribution summaries without disrupting the day to day workflow of the team. Lastly, the administrators of the system ensure the maintenance of the platform on a technical basis (accounts, availability, and security) and do not get involved in the academic evaluation processes.

These users will be able to interact with TeamTrack as follows:

2.3.1 UC-01: Registering and Authenticating a user.

A user (either students or instructors) register an account and log in with valid credentials. On authentication, the system creates a secure session which then takes the user to a dashboard. In case credentials are wrong, we will deny access the user will get an error message.

2.3.2 UC-02: Create Project Workspace.

A project leader can form a new project workspace by typing in the relevant project information. A system authenticates the input, establishes the workspace and provides ownership rights to the team leader in order to be able to manage the project environment.

2.3.3 UC-03: Join Project Workspace.

A student group member enters an existing workspace, with a sound join identifier/code. The system checks the identifier, registers the student in the project, and opens access to characteristics of project activities and teams communication areas. In case of an invalid identifier, the request is rejected and a feedback is returned.

2.3.4 UC-04: Creation and Assignment of Tasks.

A team leader will provide tasks within an already existing workspace, add the necessary details of the task and assign a particular member to it. The task can then be viewed in the task view of the project, which allows the team to monitor what is required to be done.

2.3.5 UC-05: Update and Collaboration in Task.

A team member is a student who will update an assigned task (change of status, change of content), and can also comment where necessary. All updates are time-stamped to allow the team to have a clear history of work on the work item and how they collaborated on it.

2.3.6 UC-06: Tracking of Time and Activities.

Timing a task by a student team member can be achieved by either starting/stopping a timer or inputting time. The system captures the time record and maintains the contribution summaries current to ensure that participation may be reviewed in the future based on evidence and not on guesswork.

2.3.7 UC-07: Contribution Analytics and AI-Assisted Insights.

When there are data on contributions, the team leaders and instructors will gain entry to an analytics dashboard. The dashboard will allow one to see trends on participation and workload, and the insights that AI provided will be displayed in the form of guidance (not automated grading) to see the imbalance or inactivity early.

2.3.8 UC-08: Instructor Project Review (Read-Only).

A project is reviewed by an instructor/supervisor who reads only, reviews tasks, progress, and analytics without any privileges to make changes to the workspace. This facilitates control and openness and retains student control over daily work.

2.3.9 UC-09: Activity Audit and History Review.

A team leader or an instructor looks at the history of activities to learn what went wrong or what was wrong, when it went wrong and the way the project changed with time. This assists in accountability, dispute management and validating patterns of contribution with the help of logged evidence.

2.4 Assumptions and Constraints

TeamTrack is an intended project management and accountability platform at the university level so the system will be realistic in scope and within the achievable academic time cross-checked with core expectations of usability, security and ethical standards. These are the assumptions and constraints that outline the circumstances under which TeamTrack should run and the boundaries that mold the design and implementation processes.

2.4.1 Assumptions

TeamTrack presupposes that its primary clientele (students and instructors) is fairly digitally literate, access the application via modern web browsers (e.g., Chrome, Edge, Firefox), and will record all items as completed tasks/time entries to ensure that the analytics of contributions will be significant. The assumption also implies that student teams are ready to use the platform on coordination and accountability instead of considering it as an add-on.

Since TeamTrack is web based, it presupposes the presence of a stable internet connection, desktop/laptop meeting the minimum hardware standards (e.g., approximately 4GB RAM and a current dual-core processor), and a desktop/laptop-based use case, rather than a mobile-based one.

At the system level, TeamTrack presupposes the presence and resilience of the internal system elements (database to store the tasks/logs/activity history and a backend API to perform the authentication (and analytics and communication)). It can also be based on optional services later in the scope like email services to verify/retrieve password and file storage support to export reports or attachments.

Operationally and project-wise, the technology choices presuppose a small-scale academic prototype, minimally supported by external (no full-fledged DevOps/SRE team) and financially constrained (no need to pay per call to LLM APIs, prefer in-house inference)

circumstances.

2.4.2 Constraints

2.4.2.1 Time and resource limitation in academics.

TeamTrack is required to be developed within a short time frame (around four months). Consequently, implementation emphasises the main capabilities (task tracking, time tracking, communication, contribution analytics) and aims at the delivery of the stable MVP prior to the optional modules.

2.4.2.2 Technology stack limitations.

The system is based on a web-based architecture with the popular technologies that would be used in the development of the students. The server is limited to Python-based stack (Django + Django REST Framework), the web talks to it via web APIs over HTTPS, and storage is managed with the help of a Django ORM-powered database (e.g., PostgreSQL/SQLite).

2.4.2.3 Constraints of security and roles.

Given that TeamTrack stores sensitive academic contribution information, only projects a user belongs to can be accessed, and RBAC should be imposed on leaders, members, instructors, and administrators. To be fair in the academic settings, contribution analytics should not be judgmental.

2.4.2.4 AI feature constraints.

The use of AI is limited by both ethical and academic considerations. The AI component must give recommendations and summaries only and should not make grading and evaluative decisions automatically and should be the one that is transparent/explainable.

2.4.2.5 Limitations on deployment and scalability.

The first version is focused on the small-to-mid university teams and not enterprise usage. Projected concurrency is small (approximately 1020 active users), it is hosted on standard web server platforms and extensive support of real-time WebSockets may be postponed to future releases.

2.4.2.6 Scope control constraints.

To avoid uncontrollable growth, the initial release is limited to academic group project management. Any features that are commercial grade (e.g. ERP integrations, automation of grading, etc.) are not included in the scope, and any future advanced predictive AI features are not considered a core deliverable.

3. TIS Updates + SRS Document

The Technology Investigation Study (TIS) was updated many times throughout this milestone. The main purpose of this was to explain the final technology choices and improve the framework of the technology evaluation sections and elaborate slightly more. At the end of each of the major technology evaluations, short summaries were written which were used to clearly point out the reasoning behind the final decision of each decision. These added parts are supposed to establish a better relationship between the investigative and research process and the architectural decisions leading to the subsequent system design.

The following parts of the TIS included summaries:

- **Section 10.2 Frontend Technology Evaluation.**
- **Section 10.3 Backend Technology Evaluation**
- **Section 10.5 Database Technology Assessment.**
- **Section 10.6 API Architecture and External Services.**

Each summary is a summary of the results of the comparisons and evaluations and explains the reason why the selected technology was selected over the other options that were considered. This improvement improves readability and also makes the decisions of technology as recorded in the Software Architecture Specification (SAS) traceable to previous research.

It should be noted that these updates do not change the earlier chosen technologies, they reinforce the rationale behind the architectural decisions that were chosen in the TeamTrack system.

Link provided below for Updated TIS Report:

[Click this link to access the Updated TIS Report](#)

Updated SRS Report link is attached in this section for viewing the Software Requirements, this section is to show the updated SRS made in order for the milestone 2 and milestone 3.

Updates done in SRS:

A simple section created to easily navigate through each of the Requirements in a simple and straightforward way.

Further updation of the SRS may be done in the future, if requirements are changed by client or team for better performance or system stability.

Link provided below for Updated SRS Report:

[Click this link to access the Updated SRS Report](#)

4. Software Architecture Specification (SAS)

4.1 Architectural Overview and Style

4.1.1 Architectural Overview

TeamTrack is a structured web-based system which is implemented on client-server architecture model. The platform is a centralized solution, which brings the management of academic projects together, the monitoring of contributions, analytics reporting, internal communication, and advisory AI support, all on a single platform. [14]

Architecture isolates the system responsibilities into clear and maintainable logical layers to provide clarity, maintainability, and controlled scalability. The frontend layer deals with presentation logic and user interaction whereas the backend application layer is used to enforce authentication, authorization, validation and business rules. Any data that is stored in a relational database where there are consistency and organization of records. [14]

It is an API-driven system design. The system has the backend endpoints on which any user activity is computed as the control line. The client will not be allowed to access direct databases. [14]

An AI advisory module is also installed as another outer part. It is designed to do nothing other than suggest and make no automated grading or decision-making. It is controlled integration that assures academic integrity but assists users to be more productive. [14]

The principles of modularity, the secure data processing and further expansibility, and fitting into the milestone-based development flow are the features of architecture. [14]

4.1.2 Architectural Style

TeamTrack is a Layered Architecture that is carried out in a Client-Server model. [14]

The architecture will be broken down into four logical layers:

- **Presentation Layer** – It deals with the presentation of the user interface and user interaction.
- **Application Layer** – The application layer is concerned with business logic, API processing, validation and role-based access control.
- **Data Layer** – Data Layer is used to manage structured data that is persistent.
- **AI Advisory Layer** – Offers external advisory functionality as controlled.

This type of architecture was chosen as it: [14]

- Encourages segregation of responsibilities.
- Improves scalability and maintainability.
- Simplifies debugging and testing.

- Concentrates security enforcement over the application layer.
- Enables modules of system development to be developed independently.

This layered design, in contrast to a monolithic design, does not allow tight integration of UI components and backend logic, in the long term making the structure simpler to understand. [14]

4.1.3 Runtime Interaction Flow

The system is run at runtime in a secure way of communication via request-response based on the principles of the HTTPS according to the postulates of REST. [14]

A sequence of actions is performed by the system when a user carries out an action:

1. The frontend is an API request in structured fashion.
2. The backend validates authentication tokens.
3. Permission checks are conducted on roles. [15]
4. Business logic is executed.
5. Data Base procedures are being carried out.
6. A JSON response is returned.
7. Interactive interface is dynamically built on the frontend.

This kind of ordered flow will ensure a controlled access to the data, centralized validation and tracing the audit to its complete length. [14]

4.1.4 Major Architectural Components

The backend is designed into functional units based on functional requirements. [14]

Table 4.1 : Architectural Component Table

Component	Primary Responsibility
Authentication Module	Secure login, token issuance, session validation
RBAC Module	Enforces permissions based on user roles [15]
Project Management Module	Manages project lifecycle
Task Management Module	Handles task creation and updates
Time Logging Module	Records contribution evidence
Analytics Module	Generates dashboard summaries
Chat Module	Manages internal communication
Activity Logging Module	Maintains audit trail
AI Advisory Module	Provides summaries and task suggestions

The modules are independent of each other and communicate via the designated interfaces in the application layer. [14]

4.1.5 System Architecture Diagram

Figure : TeamTrack Layered System Architecture



The diagram graphically illustrates the layered clientserver design of TeamTrack, such as the Presentation Layer, Application Layer, Data Layer and External AI Advisory Module and its interaction paths. [16]

4.1.6 Architectural Diagram Explanation

The architectural design presents the stratified structure of TeamTrack and the interaction routes between the components of the system. [16]

On the upper part of the architecture is the Presentation Layer which is the user interface on the client side which is accessed by a web browser. The layer does the visualization of dashboards, forms, project boards, and analytics visualizations. It does not get into the database directly. Rather, any communication is implemented with secure API requests. [14]

The Application Layer (Backend API) is the heart of the processing of the system. It has global components such as authentication, RBAC implementation, project management, task management, time logging, analytics computation, chat management, and activity logging. Any incoming request undergoes first validation at this layer to verify compliance of authentication and authorization any business logic is run. [14]

The RBAC module is a gatekeeper in the application layer. It permits users to only get resources that are in line with the roles assigned to them. As an example, the lecturers can access analytics

dashboards but cannot edit task records, whereas students can access tasks in their respective projects only. [15]

The relational database system containing structured records of user accounts and project, tasks, time logs, chat history and audit logs are the Data Layer. The database is not accessible by external clients who can just receive it by the backend services that enhance security boundary. [14]

AI Advisory Module represents an example of an external, but related element. It is only responsive to filtered information provided in the form of backend endpoints that provide suggestion-based output such as summaries or productivity insights. Neither is it direct and automatic in changing the records of databases nor is automated in grading and evaluation of performance. [14]

The flows of communication will be controlled in the diagram:

- Frontend ↔ Backend (HTTPS API communication)
- Backend ↔ Database (Structured queries via ORM)
- Backend ↔ AI Service (Filtered advisory requests)

This architectural isolation makes certain that security is enforced at the API boundary, data integrity is ensured by centralized persistence, maintainability is achieved on a component-by-component basis, and scalability is achieved through slackly coupled service interactions.

In general, the diagram provides a clear separation of concerns with a tight control over integration between system modules. [14], [16].

4.2 System Context and Boundaries

4.2.1 System Context

TeamTrack operates within a well-structured university teaching and learning environment, in which teamwork in project activity is a core component of the student assessment. It is planned to have a system that will facilitate regulated communication between students, lecturers, and administrative staff and remain transparent and accountable to contributions. [14]

Such setting requires students to collaborate in groups accomplishing educational assignments, which frequently entails assigning tasks, monitoring progress, communicating and providing evidence of contribution. To observe the engagement and determine fairness, lecturers need to see such activities without directly interfering with project processes. Administrators make sure that systems are configured, that roles are assigned, and that systems are stable. [14]

The key stakeholders who will be involved with the system are: [14]

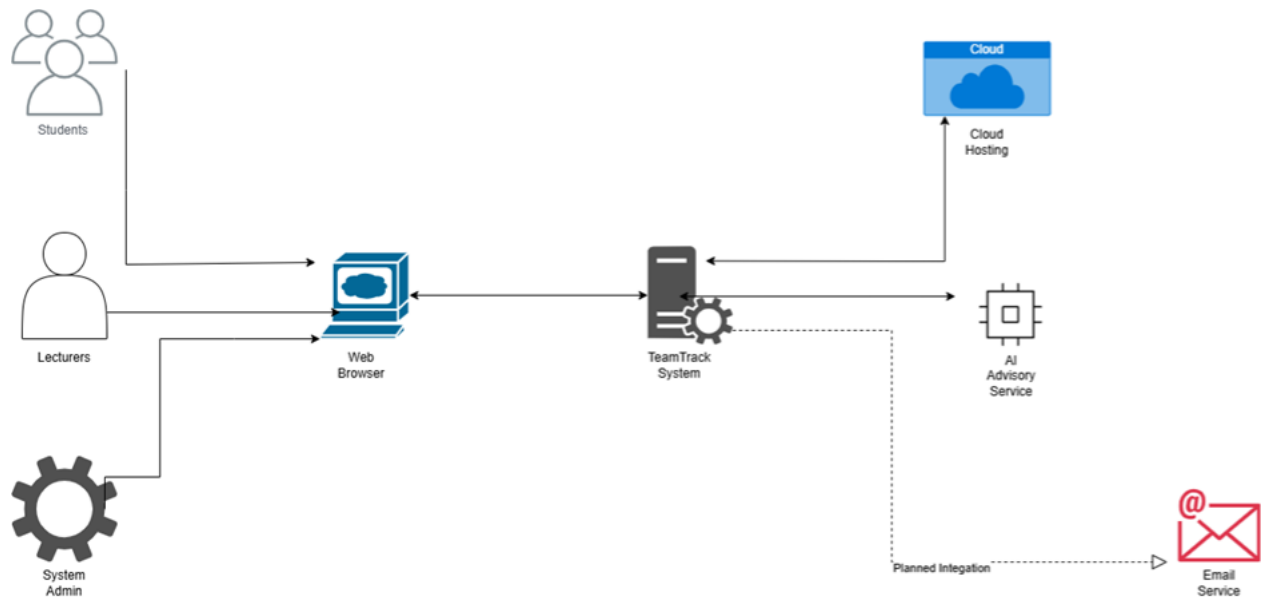
- **Students** – Assign tasks, create and maintain projects, manage project time, commute with other members of the team, and see analytics dashboards that show the participation metrics. [14]

- **Lecturers** – View read-only analytics dashboards to track the project involvement and share contribution amongst the team members. [14]
- **System Administrators** – Assign user privileges, system policies and configuration. [15]
- **Developers / Technical Maintainers** – Watch over deployment, updates, monitoring, and maintenance of the system. [14]

TeamTrack engages all the stakeholders through the regular web browsers. It is platform neutral and available on the internet making it device and operating system neutral. The interaction between client and server components is provided by secure HTTPS channels to secure data integrity and confidentiality.

The system is not a vacuum because it is a part of an academic governance system in which human judgment is still dominant. Institutional decision-making processes are not substituted by TeamTrack that brings transparency and analytics support.

Figure 4.2.1 : TeamTrack System Context Diagram



4.2.2 System Boundary Definition

It is important to define the system boundary to understand what is architecturally under the control of TeamTrack, and what falls outside the scope of its operation. This helps to limit the scope creep and ensures that they meet the milestone deliverables. [14]

Internal System Scope (In-Boundary Components)

The TeamTrack architecture is entirely managed by the following components: [14]

- Authentication and session management [14]

- Role based access control (RBAC) [15]
- Lifecycle management of the project. [14]
- Fourth, task creation, modification and tracking. [14]
- Time creating and sharing contribution. [14]
- Generation of analytics based on archived logs. [14]
- Internal chat persistence [14]
- Activity audit logging [14]
- Artificial intelligence-generated consulting overviews and recommendations.

All these components are the main functionality of the system and reflect the features promised in the SRS and being implemented in the TIS.

Notably, the analytics dashboards are based solely on data on task and time log records. The system does not allow calculated contribution metrics to be changed by any manual override or subjective input.

External Scope (Out-of-Boundary Components)

The following elements are explicitly excluded from the system boundary:

- Computerized scoring/educational analysis. [14]
- Enterprise/corporate integration.
- High volume document management or file storage services. [14]
- The high-order predictive or decision-making AI models.
- Basic authentication is not sufficient in external institutional databases. [14]

These omissions are deliberate. TeamTrack has not been created to substitute academy, institutional infrastructure, or enterprise level platforms; it is intended to facilitate collaboration and transparency. [14]

Through the clear definition of these exclusions, the complexity of architecture is controlled and in line with the academic project requirements. [14]

4.2.3 External Systems and Dependencies

TeamTrack relies on various external organizations to support the infrastructure and provide longer functionality though core logic is internal.

Table 2: External Entity Table

External Entity	Role in Context	Dependency Nature
-----------------	-----------------	-------------------

User Browser	Interface rendering and user interaction	Required
Internet Network	Communication channel between client and server	Required
Cloud Hosting Infrastructure	Application deployment and runtime environment	Required
AI Service Provider	Advisory suggestion and summary processing	Optional but integrated
Email Service (Future Scope)	Account verification and password recovery	Optional

These bodies are the ones that offer infrastructure or additional services. Nevertheless, they neither hold nor manage the business essence of the system.

Any sensitive processing is kept in application layer residing in the controlled backend environment. [14]

4.2.4 Trust Boundaries and Security Segmentation

To protect academic records and contribution data, the architecture establishes clear trust zones.

Trust Segmentation Model

1. **Client Zone (Unprotected Area)**

The browsers and user devices are assumed to be untrusted. Any incoming traffic in this zone is checked in terms of validation and authentication prior to processing. [14]

2. **Application Layer (Primary Trusted Boundary)**

The first secure boundary is the API in the back end. Before any communication with the database, all authentication validation, RBAC enforcement, input validation and business rule execution takes place. [14], [15]

3. **Data Layer (Highly Trusted Zone)**

The authoritative records of academics are stored in the relational database. It cannot be accessed by outside clients but can only be accessed using validated back-end services. [14]

4. **AI Advisory Layer (Controlled Semi-Trusted Zone)**

The AI module is treated as semi-trusted. It receives filtered, non-sensitive data for suggestion generation.

5. It makes no direct changes or executions to records or database operations. [14]

This segmentation ensures:

- Every user input is checked and then processed.
- Attempts of unauthorized access are blocked on the API boundary. [15]

- There is integrity of database. [14]
- AI recommendations should not violate the fundamental data integrity. [14]

4.2.5 Contextual Data Flow Summary

Within the defined boundary:

- User actions originate in the client zone. [14]
- Requests are validated and authorized at the application layer. [14], [15]
- Persistent data operations occur exclusively within the data layer.
- Advisory insights are generated through controlled interaction with the AI layer.
- Final responses are returned to the presentation layer for rendering.

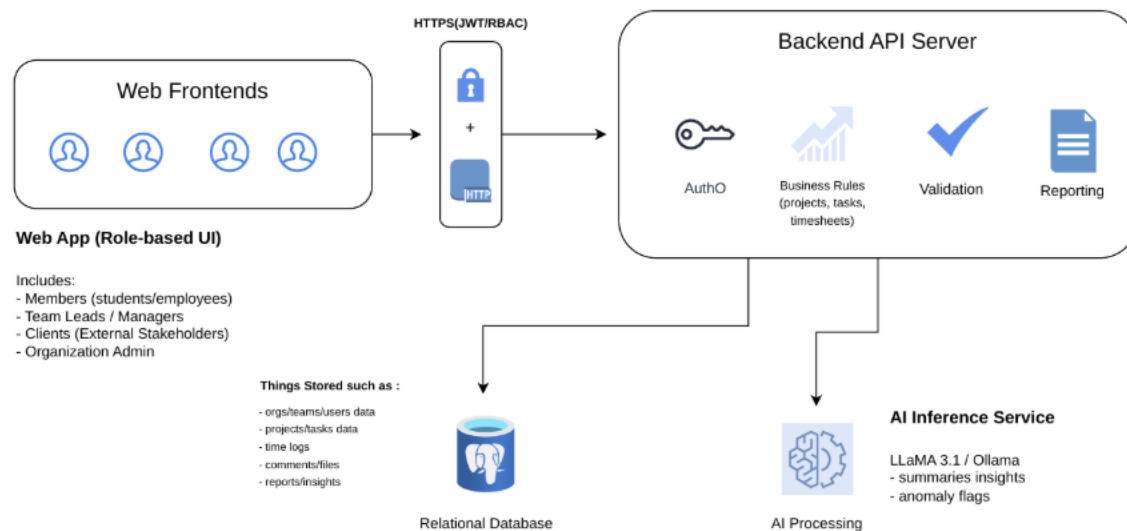
This managed data flow implements security and architectural transparency and upholds traceability by use of mechanisms of activity logging. [14].

4.3 Major Components / Modules

This part explains the key software elements of the proposed system and outlines the explicit roles of every module. The system is an artificial intelligence-based collaborative project management and time tracking system tailored towards university group projects, whereby fairness in contribution and objective progress tracking are crucial. In contrast to the general project tools, the system should meet the requirements of the academic assessment: it should be able to capture the structured work evidence, offer clear records of the activity, and enable the lecturers to monitor the progress and the participation in the team without being dependent on the subjective arguments of the team members.

In order to do this, the architecture is engineered on a bounded basis. The frontend is concerned with the interaction and visualization, the backend executes business rules and permissions, and the database is the single source of truth in terms of task execution, time logs, audit evidence, and generated AI insights. The capability of AI is provided with a special orchestration layer, which oversees the data shared with outside parties, prompt building, and result storage to trace it. The aim is not simply to work, but to work in a manner that can be audited, secured, and scaled to further milestones.

Figure 4.3.1 : System Components and Primary Interactions



4.3.1 Component Overview and Rationale

The implementation is done in the form of a web application so that student teams and lecturers can access it easily without doing a local installation. The frontend works with various user roles yet it operates on the same principle: showing information and gathering users actions. The backend will ensure that all the workflow rules are adhered to and that there will be no inconsistent or unfair usage like that of a student modifying the time logs of another student or a lecturer accessing teams that they do not supervise. The significance of this separation lies in the fact that the academic systems should be trustworthy and predictable: the computation of contributions sums and metrics of reports should be provided based on the verified data of the backend and not on the client-side algorithm.

Strong evidence layer is also a part of the system. As an illustration, an activity evidence module takes note of an event when a task status is altered or a time log is created with the name of the actor, the date and time and the reference of the entity. This offers a reliable journal that can substantiate lecturer evaluation and group responsibility. AI characteristics exist on compiled trusted records such that analysis is founded on real evidence as opposed to uncritical textual details.

4.3.2 Presentation Component (Frontend Responsibilities)

Frontend Web Application (Student and Lecturer Interfaces)

User visualization and workflows are done by the frontend. It is deliberately maintained as a thin layer to ensure that the business logic is in place and does not compromise security in the backend. The front-end will accommodate various views, which are in line with the academic group work, including student task performance, team coordination, and lecturer monitoring.

Key responsibilities:

- Display Student workspace in views such task board, task underway, due dates and progressing indicators.
- Render Time track views such as timer control, manual entry forms and time history.
- Team collaboration features such as activity feed, team progress charts and deadlines.
- Display Lecturer dashboards such as team comparison, progress snapshots and contribution evidence views.
- Record time actions such as start timer, stop timer, edit log entries and add manual logs.
- Show results of reporting like totals of each member, totals of each task, number of overdue and export buttons.
- Show AI responses, including summaries and inactivity indicators and suggestions on the next steps.

Input validation (format and completeness): Acceptance of input is performed at the front-end to reduce the load on the backend.

- User session behavior include maintaining behaviour like persistence of a user session on login, user session logout, and user session expiry.

Boundary rules:

- The frontend should not hold permanent and authoritative information, it eats backend responses.
- The frontend is not allowed to make any final contribution or reporting calculations but shows metrics provided by the backend.
- The frontend should not make direct calls to external services but make backend end calls only.

4.3.3 Backend Component (API Layer Responsibilities)

Backend API Server (Validation, Permissions, Coordination)

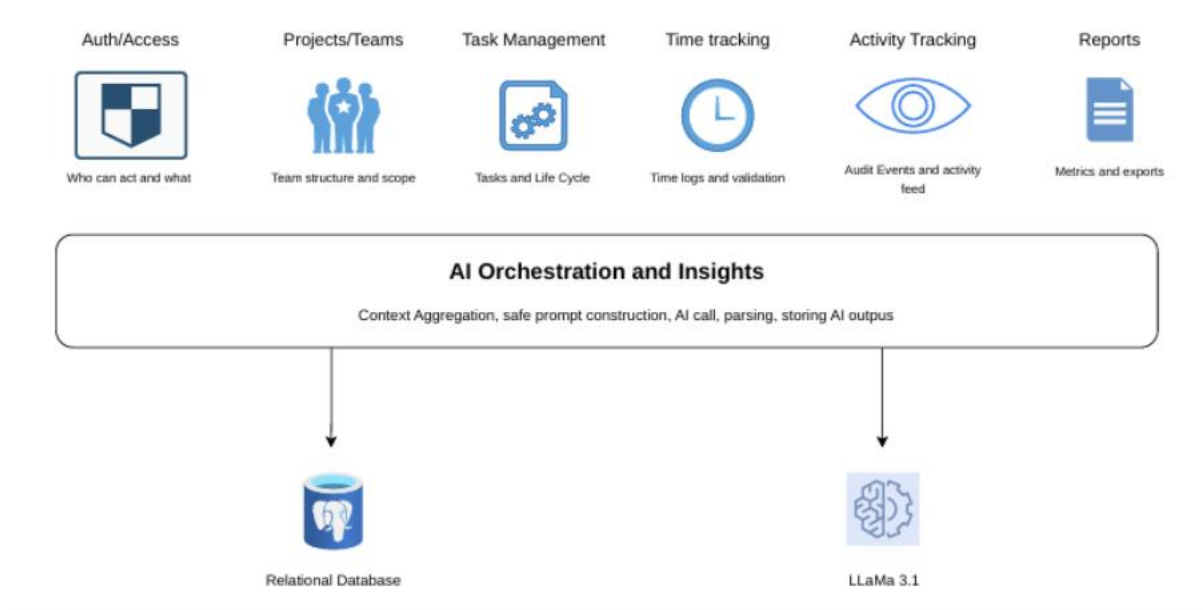
Correctness and security The main point of enforcement is the backend API server. All client requests are verified, verified through role permissions, and run through business rules and finally

arrive at the database. This guarantees that there are uniform results and safeguards the contribution evidence integrity.

Key responsibilities:

- Authenticate requests and enforce role-based access control for Student and Lecturer.
- Validate entity ownership and scope such as project membership and team membership.
- Imposed task workflow rules are authorized status transitions and required fields.
- Enforce time tracking rules such as positive durations, valid task linkage, and correct ownership.
- Provide stable REST endpoints for tasks, time logs, teams, reports, and AI insights.
- Aggregate metrics needed by reports and dashboards (server-generated).
- Coordinate AI orchestration requests and store resulting insights.
- Standardize error responses and log significant events for debugging and audit needs.

Figure : Backend Module Decomposition



4.3.4 Core Business Modules (Detailed Responsibilities)

Authentication and Authorization Module

This module ensures that everything is done in the right identity and position. The platform has sensitive contribution evidence and therefore, the authorization should be rigid and uniform.

Responsibilities:

- Manage account registration and login.
- Manage session or token lifecycle and expiry.
- Enforce role permissions for Student and Lecturer.
- Enforce project/team scope checks for every request.
- Provide a single permission-check mechanism reused across endpoints.

Project and Team Management Module

The module determines the context of collaboration in which there are the tasks and time logs.

Responsibilities:

- Create and manage projects and teams.
- Manage membership and membership roles.
- Control which teams a lecturer can access.
- Maintain project settings such as dates and milestone labels if required.
- Prevent cross-team access to tasks and logs.

Task Management Module

This module organizes the work in assessable trackable tasks.

Responsibilities:

- Create, update, and remove tasks and subtasks.
- Assign tasks to members and record assignment history.
- Maintain task lifecycle state and ensure consistent transitions.
- Handle due dates, priorities, and categorization.
- Provide task history for evidence and lecturer review.

Time Tracking Module

This module relates organized time information about tasks and membership within a team.

Responsibilities:

- Start/stop timer operations linked to a task and user.
- Manual time entry with date, duration, and description.
- Validate time entries for correct ownership and project scope.
- Prevent invalid entries such as negative durations.
- Generate aggregates used for dashboards and reports.

Activity Evidence Module

This module captures significant activities as transparency record.

Responsibilities:

- Record task creation, updates, status changes.
- Record time log creation and edits.
- Record membership changes and report generation events if required.
- Provide a chronological feed by project/team.
- Provide lecturer review views for accountability.

Reporting and Analytics Module

This module produces quantifiable results which are required in academic assessment.

Responsibilities:

- Contribution Compute per period and per member.
- Calculate workload imbalance indicators like an imbalanced task completion.
- Create snapshots in terms of overdue work and ratio of completion.
- Export in structured formats i.e.: CSV and optional PDF.
- Make team lecturer comparisons.

AI Orchestration and Insights Module

This module is AI-assisted without, apparently, losing control and reproducibility.

Responsibilities:

- Extract aggregated context from tasks, time logs, and activity events.
- Build consistent prompt templates and apply guardrails.
- Call the AI inference service and parse responses.
- Store AI results with timestamp and reporting window.
- Provide an endpoint to retrieve stored AI insights for dashboards.

4.3.5 Module Inputs/Outputs Summary Table

Module	Inputs	Outputs
Task Management	task details, assignments, status updates	task records, task history
Time Tracking	timer actions, manual entries	time logs, time totals
Activity Evidence	module events	audit events, activity feed
Reporting & Analytics	tasks, logs, activity	metrics, comparisons, exports
AI Orchestration	aggregated metrics	AI summaries, flags, recommendations

Table 3: Modules Summary Table

4.3.6 Conclusion

Finally, the modules in the system are designed in such a way that they foster productivity and academic accountability. The frontend offers easy workflows to the students and lecturers whereas the backend ensures that roles, rules and evidence integrity is upheld. The fundamental contribution data are task management and time tracking, and the activity evidence module is used to provide traceability as to who did what and when. Transforming raw evidence into lecturer-readable summaries and exports is done through reporting. AI orchestration is built on these trusted records to create more visibility with summaries and participation insights and maintain security, privacy, and reproducibility. Such modular structure also allows the growth in future since the major functions are independent of each other and able to develop without shaking the whole system.

4.4 Data Architecture (Storage + Flow)

TeamTrack divides its data into two layers model. The core structured transactional data is the focus of the first layer. Persistent entities are stored within the first layer. This includes users, projects, tasks, contribution records etc. The second layer handles high frequency activity and event data that captures system interactions and operational logs as they happen throughout the platform.

Separating this way follows the goals of TeamTrack including scalable analysis, clear contributions tracking, and reliable academic collaboration. In contrast to most group tools which default to informal chat, TeamTrack has been built to formally store academic work, and therefore, the storage model is about consistency in data, restrictive access controls, and complete traceability of all user-actions within the system.

Transactional integrity of the core entities is preserved. Having this separation ensures that the system is dependable and reliable in terms of records and performs well.

4.4.1 Core Relational Data Layer (PostgreSQL)

PostgreSQL became the main relational database management system (RDBMS) for TeamTrack because of its good support for structured schemas, reliable transaction handling, and flexible querying capabilities. It is an open-source object relational database system, it is known for reliability and consistency with data [7]. PostgreSQL is used by TeamTrack to store all essential academic records that have to be kept track of.

- User accounts and information related to authenticating data
- Definitions of projects and workspaces
- Role based memberships
- Deadlines, tasks, and milestones

- Time logs that are organized
- Contribution metrics and summaries

The model makes sure that data relationships are defined explicitly. The structure of the database is meant to guarantee that relationships between records will always be valid, so a task has to be associated with a project, for instance. Time logs should always reference both a task and a user. User permissions are linked to what kind of member a user is for each project. Through constraints in a database, it makes sure that a record never exists without a parent, showcasing the other goal of TeamTrack which is referential integrity at the database level, not merely relying on application logics.

Because TeamTrack places a lot of importance on transactional reliability (especially since contribution history is tied to accountability), PostgreSQL was selected as the database technology, because it also does support indexing and more advanced read operations which can help dashboards remain responsive even if multiple users are interacting with the system at the same time. PostgreSQL is also ACID compliance which is useful when the system is under heavier usage, since it ensures that updates to tasks, time logs, or team memberships remain consistent and reliable [7].

4.4.2 Activity and Event Logging Layer (Firehose)

Other than structured records, TeamTrack also has other data in large volumes, including activity data. Events that come under this activity data-

- Changing a task's status
- Time tracking updates
- Contribution edits
- Rapid interaction logs
- System generated activity records

Storing all of these inside a relational database would increase load and reduce performance so high frequency data is handled separately using Firehose. Firehose is a streaming data infrastructure which is made to handle data on large scales efficiently [8], because it allows high throughput logging but doesn't compete with the main transactional workloads, which is good for the system.

TeamTrack's architecture

- PostgreSQL remains focused on structured academic records
- Firehose deals with the streams of activities.
- The backend combines and processes both sources of data in order to generate contribution analytics.

To avoid performance problems, the system maintains high-volume activity logs independent of the storage of transaction, in turn, it does not allow large volumes of activity data to slow down

core project and task update operations. When the high-performance applications and database are separated, the system is scalable and reliable even under load.

4.4.3 Data Flow Between System Components

The frontend does not connect to the database in TeamTrack. All data requests instead go through the backend layer, and following this layer you perform validation, authentication and permission before executing any data operation. Unauthorized access is prevented, and the risk of client-side manipulation affecting the database records is reduced.

Function flow:

1. The HTTPS requests sent by the frontend to the backend API.
2. Before granting access, authentication credentials are verified.
3. After this FastAPI processes the business logic of the operation.
4. PostgreSQL is used in structured data operations.
5. High-frequency action events are sent to Firehose for logging.
6. The frontend receives the aggregated results for display.

4.4.4 Data Preparation for Analytics and AI

In creating the contribution dashboards, the system pulls together transactional data and activity events through time. To summaries metrics, backend processes raw logs that are later displayed instead of raw logs. These summaries help to see how much work each person had to do, who left what, and how well everyone worked together overall. Give a clearer insight of:

- How workload is distributed across the team
- How often each member participates
- Patterns in how tasks get completed
- Overall contribution trends over time

When the AI is used to help generate summaries, only some contextual info is passed to the AI module when generating summaries.

The LLaMA 3.1 models from Meta, which specialize in comprehending and producing natural text, have also been integrated into TeamTrack [9]. It's used in an advisory role to help explain what the data shows. So, the backend generates an analysis, and the AI component (which only provides advice) must interpret it. It does not make any decisions or act with authority within the platform.

To maintain this supportive role, the AI service does not have direct access to the full database. It supports interpretation. The AI service doesn't access the full database it only does this:

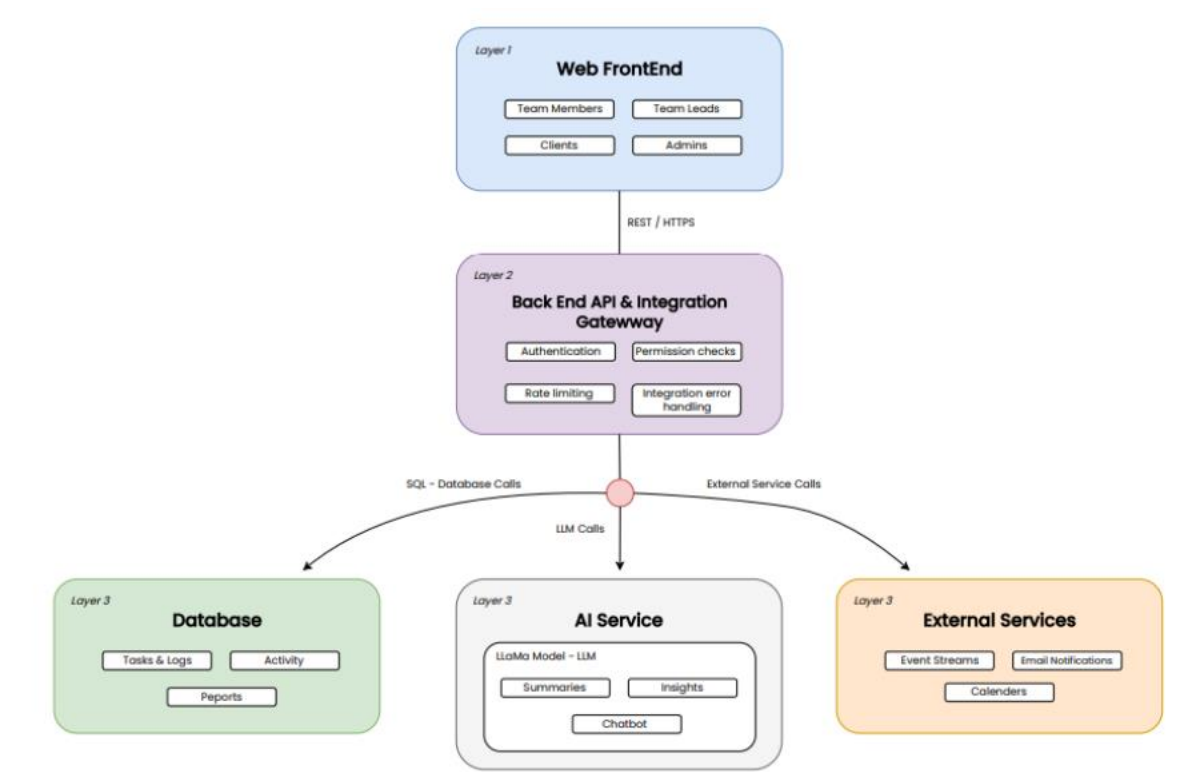
- Backend picks the relevant summary (e.g. for a certain task or member)
- Only what is necessary gets sent to the AI
- The AI produces readable explanations of analytical results

4.5 Integration Architecture (External Services/APIs)

This part explains the interaction between the system and the external services and APIs with regard to interface boundaries, data exchange design, operational reliability and security issues. The integration architecture is also very crucial as the system will be introducing AI-based capabilities, and this will need to be communicated to an inference service. In educational institutions, the integration has to be regulated in order to guarantee the privacy, prevent the incidental release of confidential data, and provide the ability to trace AI-generated results.

The platform is tailored in such a manner that the backend acts as an integration gateway. Only the backend is in contact with the frontend. The backend in turn calls out to all the external calls, implements authentication and permission policy, formats and packages safe payloads, implements a time out and results storage. This will provide consistency, secure credentials, and even allow the system to be operational even when outside services are unavailable.

Figure 4.5 : Integration Architecture Diagram (Backend Gateway)



4.5.1 Integration Types and Purpose

Various integration categories are supported in the architecture with a different purpose and risk profile.

Integration categories:

- AI inference integration for summaries, insights, and inactivity detection.
- Optional event streaming integration for analytics and evaluation evidence.
- Optional notification integration for reminders and alerts.
- Optional future academic integration such as calendar and file storage if required.

The architecture treats AI as a value-added feature rather than a dependency for core operations. As an example, when the AI service is not working, the system will still offer regular reporting of past saved information and ensure that the student processes and lecturer tracking continue.

4.5.2 AI Inference Integration (Primary External Service)

Purpose and Functional Scope

The AI inference service applies to produce human readable summaries and insights of trusted platform evidence. This assists in decreasing the amount of time required by the lecturers and team leaders to analyze big collections of tasks and logs.

AI features supported:

- Summary of the project progress weekly depending on task progress and recent activity.
- Inactivity detection based on measurable evidence (task updates, time logs, last activity).
- Recommended actions such as redistributing tasks or focusing on overdue work.
- Lecturer-style insights such as “team workload imbalance indicators” in narrative form.

Data Exchange Design (Why Structured Payloads)

The backend formulates a payload that is aggregative of raw data to:

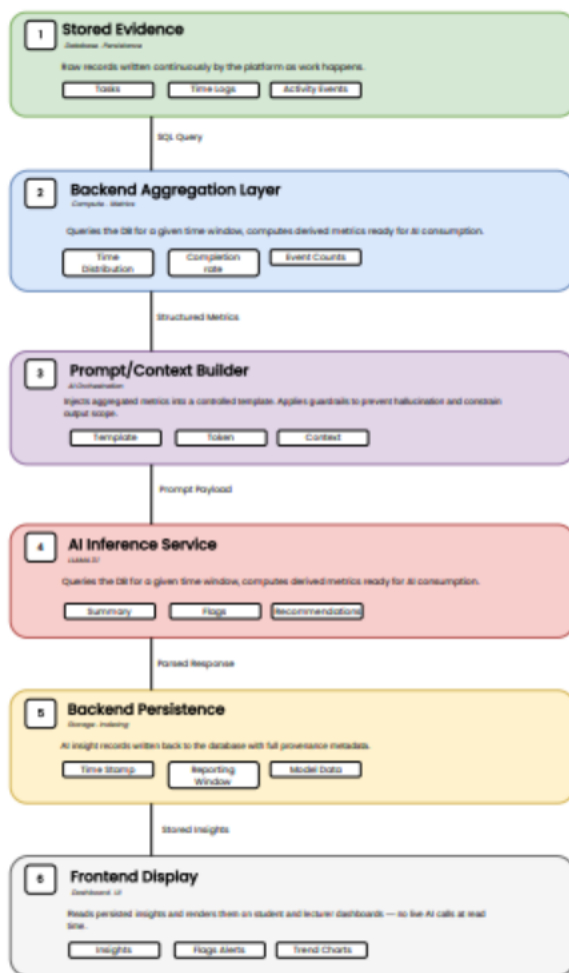
- reduce privacy risk
- reduce token usage and processing time
- maintain consistent output formats
- improve stability and reduce hallucination risk

The platform evidence to metrics converted by the backend includes:

- task totals by status
- overdue task counts
- completion ratio per member
- total time logged per member

- last activity timestamp per member
- major events count in the reporting window

Figure 4.5.2 : AI Data Flow Diagram



4.5.3 Integration Interfaces and Endpoint Responsibilities

In this section, the interfaces have been recorded in a manner that accommodates an SAS.

Frontend Backend Internal API endpoints (Frontend Backend).

Typical endpoints include:

- POST /auth/login
- GET /projects/{id}/dashboard
- CRUD /tasks
- CRUD /time-logs
- GET /reports/contribution?project={id}&period=weekly
- POST /ai/insights/generate?project={id}&window=7d
- GET /ai/insights?project={id}&window=7d

The backend exposes stable endpoints so the frontend remains simple and does not depend on external API changes.

External service endpoints (Backend to External)

Typical AI endpoint design:

- POST /infer or /generate
- Payload contains aggregated metrics and required context
- Response contains structured insight fields

4.5.4 Integration Table

Integration	Trigger	Payload Type	Expected Response	Storage Strategy
AI inference	User request or scheduler	Aggregated metrics	Summary + flags + recommended actions	Stored per project and reporting window
Firehose/event store (Phase 2)	Platform events	Event messages	ACK or receipt	Optional receipt logs
Notifications (Phase 2)	Rule triggers	Notification payload	Delivery status	Optional delivery logs

Table : Integration Table

4.5.5 Reliability, Failure Handling, and Operational Behaviour

External integrations bring about uncertainty hence the system has specified operational behaviors.

Reliability controls:

- Timeouts for AI calls so requests do not hang.
- Rate limiting to prevent excessive AI requests from one project.
- Retries for non-critical integrations such as event publishing.

- Circuit-breaker behavior (recommended) to temporarily stop AI calls if repeated failures occur.

Failure cases and responses:

- AI unavailable: show a clear message and allow retry; continue showing normal reporting metrics.
- AI slow response: return a “processing” state or timeout and ask the user to retry later.
- AI low-quality output: allow regeneration and store the latest version as current.
- Event store failure: log and retry later; do not block tasks/time logging.
- Notification failure: log failure; if possible show in-app alert fallback.

4.5.6 Security and Privacy Controls for Integrations

Student contribution evidence is sensitive hence the integration security is explicit.

Controls applied:

- API keys stored only in backend environment variables.
- No external calls are made by the frontend.
- Outbound payload is minimized and structured.
- Access control is checked before AI generation so only authorized roles can trigger “official” insights.
- Logging excludes sensitive content; logs store only metadata such as timing and status codes.
- The result of AI is stored in a format to allow traceability (review by lecturer) and to prevent repeated calls.

4.5.7 Conclusion

Conclusively, it is mentioned that the integration architecture will create value through AI that will not compromise system reliability and academic trust. The backend is a gateway that is also controlled making permission checks, privacy-preserving payloads, timeouts, rate limits, and outputs and stores with traces. Trusted platform evidence is used to produce AI insights not loose text by users, and this enhances consistency and less risk. Phase 2 extensions like event streaming and notifications can be added and provided with optional features to ensure the core workflows are not affected, since tasks and time tracking are internal-first and can be accessed all the time when the external services are degraded.

4.6 Security Architecture and Ethical Considerations

Within TeamTrack, security is a fundamental element of how the system is built, and not something simply put on afterwards. Because the platform deals with academic contribution records, role-based permissions, and AI-assisted insights, the system needs to minimise the risk of misuse while still supporting transparency and fairness.

Security controls are not handled only on the frontend. Instead, they are enforced mainly at the backend level, where every request must pass validation checks before any operation is allowed. This means that even if someone attempts to manipulate the client-side behavior, the backend will reject requests that do not follow the system's rules.

So security mechanisms are implemented across several connected layers within the architecture. These protections are built directly into the system design rather than being treated as separate add-on components.

- **Access control and authentication enforcement:** making sure users are who they say they are and only see what they're supposed to (least privilege)
- **Data protection and secure communication:** keeping information safe in transit and at rest
- **Ethical and responsible system behavior:** ensuring the system acts in ways that align with academic integrity and fairness

4.6.1 Authentication and Access Control Architecture

TeamTrack implements authentication using JSON Web Tokens (JWT). The process flows like this: TeamTrack login --> JWT token generated --> token included in API request --> backend validates token --> access granted or denied. After the user has successfully logged in, the system generates a secure token, which should be included with all subsequent API calls. Every request is verified by the backend before any action is performed to verify the identity of the user.

Using FastAPI for this implementation is better because it provides built-in security tools to perform token validation and dependency-based access control, and with this authentication can be applied consistently over all the API endpoints, which is important because TeamTrack supports multiple user roles and each has different levels of the permissions. A well-structured request processing is also offered by FastAPI and assists in keeping API interactions on the platform in order and safe [11].

So after the authentication is done the system also needs to check the permissions allowed for the user to see the actions they can perform, which explains the Role-Based Access Control (RBAC) is implemented. Access permissions are managed centrally because we want it to stay consistent across the application rather than being handled separately in different parts of the code. For every request that retrieves data or functionality, the backend checks:

- Is the token actually valid?
- What role does this user have: member, leader, instructor, or admin?

- Does that role have permission to perform this specific action?

This layered approach addresses one of the more common and dangerous vulnerabilities in web applications today according to OWASP Top 10 2025 [10], which is broken access control. Before touching the database, checks happen first and doing this reduces chances of wrong changes or twisted data slipping through.

Applying these rules early on is good because it just makes sure that contribution insights stay protected. The front end never decides who gets in on top of that control stays at the back, where every request is measured against permissions. Requests missing proper clearance get turned away without exception.

4.6.2 Secure Communication and Data Handling

When the TeamTrack frontend communicates with the backend, the data is exchanged securely as it uses HTTPS instead of any regular unsecure connection so this would definitely help keep the communication between client and server safe.

- Frontend request secures HTTP connection and the connect with the Backend API
- Data formatted as JSON formatted images is sent and received
- Sensitive data like login credentials and user profile information is kept safe through encryption as it is being sent.
- Since this is an encrypted connection, the data that is in transit over the network cannot be easily intercepted or read by outsiders

Likelihood of injection vulnerabilities and malformed data attacks that are among the top application security concerns identified by OWASP [10] are attempted to be filtered out. This is because before processing any data the backend also runs it through strict validation checks.

Database access restrictions

- The frontend never communicates directly with storage layers
- All reads and writes pass through FastAPI
- Permission validation occurs before data operations

This layered format means that the core database isn't exposed to direct manipulation and the same rules are applied consistently no matter where the request is coming from.

Logging mechanisms: while Firehose handles high frequency event logging, it doesn't override any relational integrity in PostgreSQL.

4.6.3 Protection Against Common Web Risks

System architecture is designed with awareness of common web application risks outlines in the OWASP Top 10: 2025 [10]. The aim was to consider these risks as we design in design phase itself instead of catching them later, to do this we had to shape the architecture in a way that would

reduce possible attack surfaces on its own, rather than reacting to the issues once the code already exists.

Thinking about common security threats from the start makes our overall structure harder to exploit, more resilient. The aim was to establish a foundation which would already constrain the possible attack openings, and the architecture would be aligned with established web security issues. Some of the major risks taken into consideration in this process are:

- Broken access control, through backend-enforced RBAC
- Authentication failures, through token validation
- Injection risks, through structured request validation
- Security logging failures, through centralised activity tracking

4.6.4 Ethical Considerations in Academic Context

Right from the beginning, security was considered from both technical and ethical perspectives, how the system worked technically, and how it handled ethical concerns. Avoiding bias and misuse was important from the start of the design process, especially considering that the platform tracks contribution behavior and generates insights from that data. So, preventing unfair outcomes was treated just as important as protecting against issues like data leaks or any errors with access, ideas related to trust and responsible use of the system influenced some design decisions in the structure of TeamTrack.

Architectural safeguards:

- Structured records, not subjective interpretation, are the source of contribution measurements
- The AI module does not access the full database directly
- Only necessary summarised data is passed to the AI service

AI LLaMA 3.1 [9] plays a supportive role, interpreting patterns in the data and offering explanations, but not producing any outcomes/ marking. For clarity, all moves users make get recorded, keeping contributions visible and accounted for, group evaluations are traceable and less ambiguous, but sensitive user information stays protected behind access controls that only allow certain data through when needed.

4.7 Architecture Decisions and Alternatives Considered

TeamTrack's structure came together using two key pieces: what the SRS laid out for functionality as well as what the TIS explained for technical reviews. Instead of picking tools first then designing around them, choices for structure were designed aiming to make the platform maintainable, scalable, and secure, but still staying true to its main goal of making academic collaboration more transparent.

During the design stage, although multiple designs got evaluated early on, one was chosen due to clean separation of duties among parts. That setup made tracking contributions easier while keeping analysis tools functional, but its real strength came from balancing clarity with practical oversight needs in shared research work.

4.7.1 Choice of Layered System Architecture

The architectural foundation of TeamTrack is organised around a layered structure, where system is divided into several functional layers. With this design the frontend interface interacts with the backend API services, which, in their turn, process business logic and organize operations related to database. The various layers get responsibility for different concerns and only interact with the nearest layers without being a threat to the other layers and maintain a clear boundary across the system.

The reason why layered architectures are highly used in the software systems is that it ensures there is separation of responsibilities and that maintenance is made easy due to the reduction and removal of tight coupling between various sections of the system [12]. In the case of TeamTrack, this architecture allows the user interface, application logic and data storage layers to develop autonomously and yet interact with each other via well-defined interfaces.

Benefits of this architecture:

- There is a strong separation of the user interface and the services at the back-end.
- Validation and security enforcement may be centrally stored in the backend layer.
- The maintenance and scalability activities in the future are made significantly easier.
- The threat of the client-side access to the database is completely removed.

During the design work other types of architecture were considered. Frontend-heavy design, in which a significant application logic was located in the client, was eventually removed as a result of the possible failure to have control over validation and enforcement of security. It was also considered a microservices architecture, but it was found to be too complex to fit the scope of this system and would create an extra deployment and maintenance burden. With that in mind, the layered architecture was the most suitable structural design to use to implement TeamTrack.

4.7.2 Database Technology Selection

The choice of a suitable database system was a critical architecture decision of TeamTrack, as the platform was required to handle organized academic collaboration information such as tasks, projects, time logs, and user roles. These entities are defined by the strong interrelationships and have to keep effective historical records to be able to support meaningful contribution analysis.

PostgreSQL was selected as the main relational database after thorough consideration. This was mostly influenced by the fact that it strongly supported relational integrity and high querying abilities as well as being transactionally reliable [7]. All these aspects combined assist in keeping

the contribution records in line and any changes made to the tasks, memberships and activity logs to always be updated throughout the whole system.

A few of other database technologies were also considered during the evaluation process. Even though it has a flexible schema design, MongoDB was not a good fit in a system with high relational dependencies. MySQL was a potential candidate which had credible relational functions though some of the advanced features and extensions that PostgreSQL has were not available, so it was dropped. Comparing PostgreSQL with the system requirements identified in the SRS, the PostgreSQL was suitable to store the structured collaboration data of the TeamTrack in academic collaboration.

4.7.3 Separation of Transactional Data and Activity Events

Another architectural choice that was highly important was the separation of structured transactional data and high frequency activity events. In this design, the PostgreSQL will be used to store structured data which includes tasks, users, and contribution logs, and the activity streams will be sent to Firehose to be processed.

This separation was added because of the system providing a large number of events updates such as task information, time tracking updates, and interaction records. Saving all these events in the relational database would have meant an increase in load and so possible performance degradation. A single database method that was taken into consideration during the design stage but was rejected eventually because it would strain the relational database too much and would make the process of real time analytics harder. Using PostgreSQL and Firehose, the architecture can provide a more scalable and maintainable configuration.

Especially considering Firehose is specifically created to scale to high volumes of streaming data efficiently, and continuous activity events can be done without disrupting the workload of the relational database [8], so the separation between event logging and transactional storage is scalable and minimises any possible development of issues in system performance as the system is used.

4.7.4 AI Integration Strategy

The architecture also includes an AI assisted component that is meant to understand collaboration analytics. TeamTrack also incorporates LLaMA 3.1 language model to make explanations of contribution patterns and analysis insights of system data.

Introduced by Meta, LLaMA 3.1 is a large language model that is trained on natural language understanding and generation tasks [9]. It's part of TeamTrack but it does not decide outcomes or hand out scores it only it helps users understand data by highlighting trends found in contributions and how team members interact.

As for ethical considerations in an academic environment, the AI system has limited data access by design because it's not pulling straight from the main storage, it relies on inputs shaped and given to it beforehand. The backend prepares summarised analytical information and only this

Once the diagram is introduced, the architecture can be described as a collection of responsibility boundaries: the frontend is concerned with user experience and with the flow of interactions, the backend is concerned with the constant enforcing of the rules (particularly, security and business logic) and the data services are concerned with long-term storage. This is important in TeamTrack because the system must support the student workflows (task updates, time logging, activity evidence) and lecturer workflows (monitoring contributions, fairness, reporting) and also possess access rules which are properly enforced on the server-side and hiding is not performed by a client-side UI. The API Security advice published by OWASP lists authorization (e.g. broken access control) failures as some of the most common and damaging API threats, which is why TeamTrack sets a dedicated security boundary before every of the domain modules [20].

5.1.1 Presentation Layer (Layer 1)

The Presentation Layer is constructed on NuxtJS to provide a web application interface with a structure to students and lecturers. Nuxt works well with these types of systems that are dashboard-like, since it offers a common project structure, routing/navigation assistance, reusable components, and rendering tactics to enhance performance and user experience of interactive programs. The presentation layer is intended to be thin, i.e. it should not have critical business rules or permission logic but should be concerned with user workflows, validation to enhance his/her usability and simple displays of system state returned via the API. The documentation of Nuxt is focused on the organization of the applications and routing patterns, which is appropriate in the case of TeamTrack that has to handle several role-based pages (projects, teams, tasks, reports) [22].

It is worth explaining the interaction of the UI and the backend first, before enumerating the features: UI does not interact with databases directly. The UI interacts with backend endpoints instead of rendering, and calls them. This will make sure that the backend is able to impose the same policies and audit actions. Practically, the UI will add an access token to every request (i.e. an Authorization: Bearer ... header), and the backend is responsible to determine what data the user is allowed to (or can) see. This request pattern is familiar to standard API security practices of web applications and is equivalent to OAuth2 bearer token implementation [27].

What the Presentation Layer provides (Student UI + Lecturer UI)

- Role based navigation (role of a student vs. role of a lecturer)
- Status, deadlines, assignees Task board / task list views.
- Activity feed (who, when) UI.
- Team pages (members, team roles, membership status)
- Status reports (progress reports, contribution summary reports)
- AI insights See (visages, tips, flags shown at the end of processing by the back end)

Why the UI is intentionally thin (important for fairness + security)

- Eliminates security by UI (covering buttons with cloth and blanket is not authorization)
- No chance of exposure of lecturer-only data accidentally.
- The validation and access control is centralized in the backend modules.
- Facilitates regular audit recording (backend is able to document the changes at uniform rates)

5.1.2 Application / API Layer (Layer 2)

Application/ API Layer is the central implementation layer of TeamTrack. It has the duty of exposing a documented API surface, executing business rules on modules like projects, tasks, time logs, activity tracking and reports and functioning as the sole control point in terms of authentication and authorization. TeamTrack is written in FastAPI + Python, since FastAPI is a framework that is oriented to API creation and has automatic generation of OpenAPI schemas, robust request validation using typed models, and popular security patterns (e.g. OAuth2 with JWT bearer tokens). This allows TeamTrack to specify consistent API contracts at an early stage (helpful in prototyping) and keep them consistent as new endpoints are introduced [25], [26].

One significant design choice that is demonstrated in the diagram is that the API layer is not a monolithic block of code logic. **Rather it is organized into:**

1. A security boundary,
2. Domain modules, and
3. An integration gateway.

This modular design is supposed to maintain the clarity of responsibilities and eliminate the chances of unequal enforcement of the rules across the endpoints.

Basic roles of Application/API Layer

- REST endpoints processing (JSON-based) (HTTPS).
- Identification (authenticate identity)
Validation (check an access control list)
- Domain workflow(task/time logs/ reports logic)

5.1.3 Security Boundary (Authentication, Authorization, Validation, Rate Limiting)

Security boundary is a special control layer, which evaluates each request prior to domain logic. It is significant to TeamTrack since the same system has various user roles (students and lecturers) and is integrated with sensitive evaluation indicators (contributions, time logs, and evidence). The security boundary is a constant to ensure unintentional increase in privileges or data leakage.

Authentication design (token-based)

The TeamTrack will be modeled using token-based authentication whereby users will be authenticated and will be issued with an access token that he/she can present in the further requests. OAuth 2.0 specifies the general pattern of authorization, which can be used in most web systems based on the concept of bearer tokens to access secured resources. The thing is that in TeamTrack, the front-end identifies a token associated with each API request and the backend simply checks the validity and expiration of this token and then moves on. [27], [28].

Bearer token usage

Bearer tokens are those that anyone who obtains the token can make use of and that is why they need to be secured during transmission and validated thoroughly on the server. RFC 6750 is the standard practice of bearer tokens in HTTP requests (e.g. authorization header). TeamTrack has this architecture that explicitly directs all requests by verifying the tokens, whereby only authorized sessions can reach the secure endpoints [27].

JWT for signed claims

A convenient format of token that can be used with TeamTrack is that of JWT whose data can be embedded in the backend with claims of identity and roles in a small signed token. RFC 7519 defines JWT which is extensively used to perform stateless authentication between API services. With the JWT signature and expiry (exp claim) the backend can verify session validity quickly by performing a validation check and then go on to perform more in-depth authorization. Notably, even JWT, TeamTrack continues to provide authorization based on server-side checks of resource ownership and role access; it does not consider JWT permission itself, but rather as identity context [18].

Authorization design (role-aware + object-level checks)

TeamTrack is not the only student vs lecturer. It should also be object-level: a student should not be able to see tasks and reports in projects he/she is not a member of, and a lecturer should not be able to see projects he/she does not supervise. When it comes to the API Security top 10, OWASP clearly mentions that one of its top risks is the so-called Broken Object Level Authorization (BOLA)

and that is precisely what happened, when an attacker alters the ID in the URL and enters the data of another team. TeamTrack is doing this by carrying out membership checks and role checks in the security boundary, and in the domain modules of each resource read/write [20].

Validation and rate limiting

The requests should be checked both in terms of schema correctness (types, required fields) and business correctness (e.g., time log duration not negative, task is the project of task). It has rate limiting as an element of securing login endpoints, report generation endpoints, and AI endpoints to minimize abuse and maintain availability. Throttling and consistent validation facilitates typical API hardening guidelines suggested by safety groups and documentation on how to create secure API designs [20].

One more architectural advantage that should be mentioned before the summary is that a security boundary also decreases the duplication of the security code in all endpoints and enforces uniformity. This proves to be particularly useful with a capstone system where various modules would be developed simultaneously.

Security Boundary functions (as shown in the diagram)

- Authentication with the aid of a token (verify bearer token / JWT)
- Role based access control (student/lecturer privileges)
- Membership checks (per project/task/log) Object-level authorization
- Membership checks (per project/task/log)
- Schema + business rule enforcement Initialization request validation
- Reduce availability + abuse control: rate limiting.

5.1.4 Domain Modules

A request making it across security boundary is then forwarded to domain modules. The business capabilities of TeamTrack are such modules. This form of a backend structure is noteworthy in maintainability: every module possesses its logic, has its own validations and possesses its own formats of data access but has common cross cutting rules (security, error handling, activity logging).

The core functional areas shown by your diagram would be your domain modules of TeamTrack which would be aligned with the usual project management and contribution-tracking requirements of academic group work. As the system may be employed in ensuring equity and transparency, it must have the capacity of storing data, and also the connection between the entities of data.

Planned domain modules (mapped to diagram)

- Auth / Access
- Projects / Teams
- Task Management
- Time Tracking
- Activity Tracking
- Reports

Module purpose summary

- **Projects/ Teams:** collaboration membership and boundaries.
- **Task Management:** It establishes the work items, ownership, and status transition, and due dates.
- **Time Tracking:** logs effort evidence (time logs) which are task/project-associated.
- **Activity Tracking:** provides records (audit trail), and evidences.
- **Reports:** Micrometric and insight into lecturer-friendly dashboards.

5.1.5 Integration Gateway

In addition to the fundamental CRUD workflows, TeamTrack has an integration gateway section where cross-cutting, asynchronous, and cross-service-based features are found. This is a general practice in contemporary systems: hold the core domain logic fixed and channel out other capabilities over a gateway layer with the ability to slowly evolve without necessarily upsetting the leading modules.

Event publishing (append-only activity history)

TeamTrack can gain advantages out of an append-only activity/event model since the issues of collaboration dispute and fairness often demand traceability: who changed status of the task, who logged time, when are the updates made, etc. Recording major actions as events allows to provide the reliable feed of activities, audit logs, reporting pipelines, etc. This is in line with the overall system design principles of ensuring auditability and accountability of the multi-user systems. (Although TeamTrack can do this bare minimum in the capstone, the architectural slot is significant in future expansion) [20].

AI orchestration (controlled summaries and insights)

The existence of the AI orchestration block is due to the fact that it is not a good idea to ask AI functions to read everything in the database or bypass access controls. Rather, the backend creates a context window (e.g. past 7 days of activity on a project, aggregate measures and most active anomalies) that the guardrails operate on, before invoking the AI inference component.

This ensures:

1. The AI can only view pertinent, sparse data, and
2. The outputs are saved and reproducibly displayed (e.g. insights associated with a time window and project).

The security model and OpenAPI contracts of FastAPI can ensure the security of AI endpoints and align with the rest of the system [25], [20].

Integration Gateway responsibilities

- Publish audit feeds + audit activities.
- Snapshot generation of trigger report (Optional)
- Memorize AI activity knowledge (rather than re-calculate) later on.

5.1.6 Data Services Layer (Layer 3)

Data Services Layer: It is the layer that is tasked with the role of maintaining the storage of TeamTrack data and regular retrieval of data. Two intended storage technologies are employed in the architecture and each was selected to suit various data characteristics.

PostgreSQL (relational transactional store)

PostgreSQL will be used as the main relational data warehouse of structured, relational objects, like users, projects, teams, tasks, assignments, and time logs. These entities are well related and enjoy the advantages of relational integrity as well as transactional guarantees. The documentation provided by PostgreSQL describes its concurrency control model (MVCC) that is capable of ensuring the safety of operations concurrently when multiple users are working on tasks and logs. This is relevant in TeamTrack since students are able to update tasks simultaneously, and lecturers can create reports simultaneously [29].

Firestore (document store for flexible artifacts)

Document-like data, which is flexible and evolves over the time, e.g. evidence attachments, activity event payloads, or AI insight documents can be served in Firestore. Firestore is an object of scalable storage of documents and provides the possibilities of transactional/batched operation in its model. This can be used to supplement PostgreSQL by eliminating the load on relational schemas of document-based contents [30].

One of them is that the frontend does not directly access PostgreSQL or Firestore. The backend mediates all access to the database, and imposes authorization policies and provides a stable audit trail. This will be necessary in order to avoid data leakage and is also in line with API security

practices [20].

Data layer responsibilities

- Put retailer data (tasks, memberships, time logs) in PostgreSQL.
- Firestore Store flex data (evidence docs, optional events, AI insights, etc).
- Aided query and dashboard aggregation reporting. and be reliable, and constant in concurrent use.

5.1.7 End-to-End Flow Examples

Example 1: Student creates a task

It is a new task made by a student in the Nuxt UI. The UI sends a HTTPS request to the backend in the form of a JSON. The token is requested in the backend server and whether the student belongs to the project is verified in the backend. The business rules (e.g. required fields, deadline format) are also enforced by the backend that inserts the new task into the PostgreSQL using a transaction and an activity event is logged by the integration gateway. This end-to-end control has reduced the chances of having state inconsistency and accountability is maintained.

Example 2: Student logs time

In the case of the time logging of a student the request is processed in an identical way: token validation, membership checks, validation (duration rules, time window rules) and then written to PostgreSQL. The system also logs the activity in the activity stream to enable the lecturer to view time evidence within a specific reporting period. The MVCC model in PostgreSQL offers safe concurrent inserts/updates that can be made with the provision of multiple students making logs at the same time [29].

Example 3: Lecturer views contribution report

It opens a contribution report dashboard by a lecturer. UI makes a request to the endpoint of reporting. The backend checks lecturer authorization and project supervision area, and executes aggregation queries on PostgreSQL and, when document artifacts appear to be displayed by evidence or AI insight documents, retrieve artifacts when documents on firestore (when evidence or AI insight documents are included in the report). Backclass is manifested with measurements and summaries giving a consistent format of the reaction schema of metrics and summaries. This makes the lecturer reports constant and cannot be edited in the client front.

5.1.8 Scalability, Maintainability, and Evolution

This architecture has been made in such a way that TeamTrack can begin as a prototype and then evolve into a full fledged implementation without redesigning the architecture.

Scalability

Since the backend is API-based and is capable of operating as stateless services (tokens contain session context), it can scale horizontally. The frontend is still an independent deployable layer.

Maintainability

The domain module structure allows the development and testing of the different areas of the system (tasks, time logs, reporting, AI) independently and, nevertheless, conform to common security and error-handling standards. The generation of Open API using Fast API assists in sustaining the contracts and preventing the non-drift of the contracts among front-end anticipations and backend answers [25], [26].

Security posture

The clear security boundary and focus on object-level authorization is simply the result of typical failure modes in APIs. According to OWASP API Security Top 10, broken authorization and too much data exposure are common; the architecture of TeamTrack does not place sensitive logic in the UI and instead implements it on the server-side. [20].

Extensibility (AI as additive capability)

AI knowledge is explicitly summoned behind the integration gateway to ensure that core task / time workflows are operational with or without the unavailability of AI. This allows the system to not be weak or reliant on AI presence to offer essential functionality.

5.2 Frontend Design (Pages, Navigation, Components)

TeamTrack frontend is a web GUI that has been built with Nuxt.js (Vue) due to its ability to have a structured routing system, and reusable user interfaces elements, and high performance associated with dashboard applications. The frontend will strive to provide consistency in navigation between modules (Dashboard, Tasks, Chat, Time Tracking, Analytics) as well as to meet responsiveness needs including effective state management and routing, ease of integration of REST API, and optimised component layout [22], [23].

5.2.1 Pages and core user flows

The primary user flow of TeamTrack begins with the Login page where a successful user goes through the Dashboard. Core areas can be moved to by clicking on Tasks, Chat, Time Tracking, and

Analytics on the dashboard. Task work is managed by use of Tasks (Board/List) views and Task Details screen where a user can update status, comment, and time. Milestones/Calendar is used to manage planning and deadlines and Chat is used to communicate.

The most important pages intended on TeamTrack will be:

- **Authentication pages:** Login (and other related flows, including registration/verification, enabled).
- **Dashboard:** overview of the project and the rapid access to the main modules.
- **Activities:** Board/List views to monitor the progress.
- **Task Information:** commenting, status, editing and time recording.
- **Milestones/ calendar:** visibility of deadline and planning.
- **Chat:** team communication and responses.
- **Analytics + AI query interface:** contribution views and insight interactions (enabled).

5.2.2 Navigation design

The navigation will be made to be predictable in all modules to ensure that students and tutors acquire the system in a short time. The frontend layout is designed with consideration of uniform navigation and readable layouts on desktop/laptop computers and also RBAC in which certain actions/UI options can be limited based on role (leader vs member). The file-based routing of Nuxt is implemented in such a way that the pages are arranged in a structured way (e.g. /dashboard, /tasks, /tasks/:id, /calendar, /chat, /analytics) and the user can switch between modules without being confused with different layouts [22].

5.2.3 Reusable UI components

The frontend employs reusable elements like: to make the UI more consistent and eliminate repetition of certain code.

- **Top-level layout shell:** header + navigation area + main content container (common to all pages).
- **Project/workspace selector:** allows changing context in case several workspaces are available.
- **Task card:** on Board and List views: Task card and task row components.
- **Task content panel/modal:** task content edit, status, comments, and time log functions.
- **Chat thread + message bubble elements:** message list and compose box.
- **Analytics widgets:** contribution patterns/workload patterns charts/ summary tiles.
- **Elements of the form:** texts, dropdowns, date pickers, and blocks of validation messages.

5.2.4 UI Feedback, validation and error management

The front-end should be able to offer transparent validation and comprehensible errors, particularly those related to authentication, task updates, and time recording. Some of the planned feedback patterns would include a pattern of inline errors when using invalid to log-in details, field and invalid date/time log validation, a pattern of successful save with messages of a smiley in a toast, and a pattern of network problems with messages of a smiley on a banner/toast.

5.2.5 Expectations of frontend-backend integration

The frontend interacts with the backend services through the REST APIs over HTTPS and should not use it improperly, i.e., it should be capable of processing authentication tokens and session validation. It should also handle life cycle of the sessions, network failures and API error responses in a graceful manner to assure that users do not lose work or to get trapped in incomplete states [19], [20].

5.3 Backend Design (Planned Services + Endpoint List)

TeamTrack has a backend implementation as an API-first application layer which presents a stable set of endpoints to the frontend and has a uniform security, validation, and role-based authorization implementation across all modules. The backend is done in FastAPI, as it provides schema-based request/response validation as well as OpenAPI generation, which allows the project to have an accessible API contract as new features are added. This is a contract-first method which minimizes integration uncertainties between frontend and backend and facilitates systematic testing and documentation during development [25], [26].

Once the role of the backend has been defined, the design can be described as a series of services planned to be implemented in a single codebase, but designed in such a way that this will act like discrete services at logical level. This modularity helps in maintainability and it helps to minimize the chances of mixed responsibility in which task logic, time logging logic and authorization logic becomes mixed. Security is also considered as a cross-cutting requirement: all endpoints are secured through the use of token authentication and authorization checks to prevent most traditional API threats like Broken Object Level Authorization (BOLA), overexposing too much data, and not controlling endpoint inventory well as defined by OWASP API Security [20].

5.3.1 Backend Design Objectives

The backend has been developed in such a way that it allows both student productivity and lecturer evaluation requirements as well as maintaining integrity and traceability on contribution records. Bearer tokens of OAuth2 style are required to accomplish the requirement to ensure that every request is authenticated, and JWT is a good format of a token that can bear signed identity and

session claims, and some expiry and role context. This ensures utmost efficiency of backend in request validation and server-side enforcement of stringent object-level authorization checks [18], [27], [28].

Backend objectives

- Offer a consistent REST API to all of the core features (tasks, teams, time logs, reporting)
- Programmatic authentication and role aware authorization should be enforced on a regular basis.
- Keep a track of activities in a recordable manner in respect of accountability and fairness.
- Support successful aggregation of reporting and lecturer dashboards.
- Keep artificial intelligence non-blocking (not core system) school of thought.

5.3.2 Planned Backend Services (Logical Modules)

The backend will be designed as a set of services/ modules. Although they may be implemented as a single application to deliver capstones, the internal organization considers them as discrete modules and they have distinct responsibilities. The routing patterns and dependency patterns of FastAPI can facilitate this style by enabling a consistent style of applying security dependencies (token verification + role checks) across groups of endpoints [25].

Table 5.3.1 : Planned backend services/modules

Service / Module	Primary Responsibility	Typical Data	Key Security Concern
Identity & Access	Authentication, session identity, role context	users, roles	token validation + secure role checks
Projects & Teams	Project creation, membership, team roles	projects, memberships	object-level authorization (membership scope) [20]
Task Management	Task CRUD, assignment, status transitions	tasks, assignments	BOLA prevention for task IDs
Time Logging	Time logs, edits, approvals (optional)	time_logs	integrity + ownership checks
Activity & Evidence	Activity feed, audit events, evidence metadata	events, evidence	avoid excessive exposure + tamper resistance
Reporting & Analytics	Contribution dashboards, progress summaries	aggregates, snapshots	lecturer scope restrictions
Integration Gateway	Event publishing, AI orchestration workflows	event pipeline, AI outputs	rate limiting + controlled access

The design of the module is in favor of a security-first model in which the endpoints do not trust the client. Access controls are implemented even when the UI does not display any lecturer feature to the students. OWASP API Security plainly points out that authorization should be done on API level, especially on object-level access, which directly applies to TeamTrack where a project ID and a task ID are in the URLs [20].

5.3.3 Authentication & Authorization Model (Backend Perspective)

Its backend is supported by a model that requires attaching an access token to every request to be made by the frontend and validation of the corresponding token by the backend before running any domain logic. OAuth 2.0 is a standard authorization model which can be used to implement this style, and bearer tokens in HTTP are defined in RFC 6750 (e.g. Authorization header). TeamTrack implements the token on each secured request to minimize the notion of session confusion and unauthorized access [27], [28].

JWT is employed as the practical token storage due to its abilities in supporting the signed claims and expiry checks (e.g., exp). RFC 7519 is used to specify JWT format and it is the responsibility of the backend to verify the signature and expiry and then proceed. Although JWT can implement role claims, TeamTrack continues to do server checks on project membership and role permissions to avoid privilege escalation and object misuse on an object-level [18], [20].

Table 5.3: Authorization rules (high-level policy examples)

Action	Student	Lecturer	Notes
View own projects	✓	✓	Student must be a member; lecturer must supervise
View other team's project	✗	✓	Prevent cross-team leakage (BOLA) [20]
Create/edit tasks in own project	✓	✓	Membership check required
Approve time logs	✗	✓	Optional workflow, lecturer-controlled
View contribution analytics	Limited (own/team view)	✓	Lecturer has deeper visibility

5.3.4 API Design Conventions and Principles

The backend API is conventionally based to ensure that endpoints are predictable, testable and well documented.

Principle 1: Visibility of contract in Open API.

Supported FastAPI The automatic OpenAPI generation is supported, which allows the use of similar endpoint schema and documentation. This is in line with the open API specification that assists in maintaining proper API inventories and minimizes un-documented endpoints [25], [26].

Principle 2: Standardized authentication

RFC 6750 provides that bearer tokens are always employed throughout protected endpoints [27].

Principle 3: Authorization everywhere at the objects.

Any endpoint which receives IDs (projectId, taskId, logId) should ensure that the user who is making the request has permission to access that item. This is a direct mitigation of the OWASP BOLA risks [20].

Principle 4: Semantics of consistency of errors.

The endpoints have predictable HTTP status codes (400/401/403/404/409/429) in order to ensure predictable response by the frontend.

5.3.5 Endpoint List Planned (Service Grouped)

The following is the list of the planned backend surface of the milestone. Implementation can have minor variations in the form of path names, but coverage does not change. Module-based grouping of endpoints with each endpoint being designated with a presence or absence of the Auth indicator of whether a bearer token is necessary.

Table 7: Identity & Access endpoints

Method	Endpoint	Auth	Purpose	Notes
--------	----------	------	---------	-------

POST	/auth/login	No	Authenticate and issue token	Token strategy aligns with OAuth2-style patterns [28]
POST	/auth/logout	Yes	End session	May be client-side only unless revocation implemented
GET	/auth/me	Yes	Retrieve current user info	Avoid excessive exposure; least privilege [20]

Table 17: Projects & Teams endpoints

Method	Endpoint	Auth	Purpose	Notes
POST	/projects	Yes	Create project	Lecturer-restricted or controlled
GET	/projects	Yes	List accessible projects	Filtered by membership/supervision
GET	/projects/{projectId}	Yes	Project detail	Membership/supervision check (BOLA control) [20]
POST	/projects/{projectId}/members	Yes	Add/request member	Role rules apply
GET	/projects/{projectId}/members	Yes	List members	Output filtered by role visibility

Table 18: Task Management endpoints

Method	Endpoint	Auth	Purpose	Notes
POST	/projects/{projectId}/tasks	Yes	Create task	Must confirm membership
GET	/projects/{projectId}/tasks	Yes	List tasks	Filters: status, assignee, due date
GET	/tasks/{taskId}	Yes	Get task detail	Prevent ID tampering
PATCH	/tasks/{taskId}	Yes	Update task	Validate status transitions
POST	/tasks/{taskId}/assign	Yes	Assign task	Enforce role rules

Table 19: Time Logging endpoints

Method	Endpoint	Auth	Purpose	Notes
POST	/time-logs	Yes	Create time log	Validate duration + task membership

GET	/time-logs	Yes	Query time logs	Filter by project, user, date range
GET	/time-logs/{logId}	Yes	View log	Owner or lecturer authorization
PATCH	/time-logs/{logId}	Yes	Edit time log	Audit required
POST	/time-logs/{logId}/approve	Yes	Approve log	Lecturer only (optional workflow)

Table 11: Activity & Evidence endpoints

Method	Endpoint	Auth	Purpose	Notes
GET	/projects/{projectId}/activity	Yes	Activity feed	Useful for accountability and reporting
POST	/evidence	Yes	Add evidence artifact	Store metadata + links
GET	/evidence	Yes	Search evidence	Filter by project/task/user

Table 12: Reporting & Analytics endpoints

Method	Endpoint	Auth	Purpose	Notes
GET	/projects/{projectId}/reports/progress	Yes	Team progress overview	Combines tasks + recent activity
GET	/projects/{projectId}/reports/contribution	Yes	Contribution analytics	Lecturer-only deep view
POST	/projects/{projectId}/reports/snapshot	Yes	Create report snapshot	Stores computed results

Table 13: AI Orchestration endpoints

Method	Endpoint	Auth	Purpose	Notes
POST	/ai/insights/generate	Yes	Generate AI insights	Strong rate limiting + guardrails [20]
GET	/ai/insights	Yes	List insights	Filter by project + window
GET	/ai/insights/{insightId}	Yes	View insight	Lecturer-authorized

5.3.6 Validation, Data Integrity, and Error Handling

Fast API allows structured validation of request bodies, query parameters, as well as response models, making APIs more consistent and minimizing errors in integration. TeamTrack should

consider this as it is significant since various modules are connected to each other (e.g., tasks and time logs should refer to the same project membership rules). [25]

The risks highlighted by the OWASP API Security include the fact that unstable validation, overexposure, and ineffective authorization functions may result in data leaks or manipulations. The design of TeamTrack thus makes validation obligatory to all write endpoints, and makes sure that error response does not spill unauthorized object existence [20].

Table 14: Error behavior conventions

Status	Meaning	Example scenario
400	Bad Request	Missing required field, invalid time duration
401	Unauthorized	No token / invalid token
403	Forbidden	Student trying to access lecturer report
404	Not Found	Object not found or hidden due to authorization
409	Conflict	Concurrent update conflict (optional)
429	Too Many Requests	Rate limit triggered

5.3.7 Persistence Strategy

The primary relational entity and coherent reporting queries transactional store is going to be postgresSQL. The MVCC behavior of postgresSQL can be used to ensure safe concurrency in the multi-user workload which applies when multiple students access work and logs at the same time [29].

Firestore is intended to be used with flexible data in the form of documents in which schema modification and pattern retrieval of documents are useful (evidence metadata, event payloads of activities, AI insight documents). Firestore documentation gives details on how it can be used as a document store with decent scalability and how it can be used with transactional/batched updates where necessary [30].

This division assists TeamTrack to maintain relational integrity in the areas where it counts (tasks/time logs/teams) and enable the storage of artifacts that might change rapidly during prototyping and iterative development to be more flexible.

5.3.8 Backend Maintainability and API Inventory Control

OpenAPI is a living schema and a collection of routes that are being exposed, and is used by the backend in order to prevent drift in the endpoints. The OpenAPI gives a commonized method to define endpoints, request bodies, and responses that enhance the capability to test and validate frontend-backend integration [26].

It is also in line with the problem of OWASP in inefficient inventory control in which unknown endpoints and hidden debugging paths become a possible hacking hazard. TeamTrack is able to view the whole API surface as the application is being built through the use of structured routing per-module and with FastAPI OpenAPI generation enabled [20], [25].

5.4 Database / Data Model Design (ERD + Schema Summary)

The structured collaboration features of TeamTrack are built around a relational data model. The system has support for many core functions like

- Project management
- Time tracking
- Messaging
- Contribution analytics

And all of these rely on structured relationships between the data entities. The architecture has PostgreSQL as the primary database to keep all the info on transactional data including the users, projects, tasks and time logs, because this relational structure helps keep consistency between the entities and also ensures that operations on important records like any updates stay reliable.

Referential integrity is enforced keeping data linked properly through careful use of primary and foreign keys constraints spread across the structure.

This ensures that the dependent records cannot exist without their respective parent entities. An example is that a time log should never be made without legitimate task and user, and the fact that tasks need to be part of a legitimate project. These limits are important in preserving the reliability and traceability of the contribution data, as it is of high importance.

In relational database design, referential integrity is typically implemented through primary - foreign key relationships that preserve consistent links between tables, and having this principle within TeamTrack ensures that key entities such as **User, Project, Task, and TimeLog** stay logically connected throughout the system and improves reliability of collaboration data and supports the platform's goal of being transparent and auditable histories of project activity [13].

5.4.1 Entity Relationship Diagram (ERD)

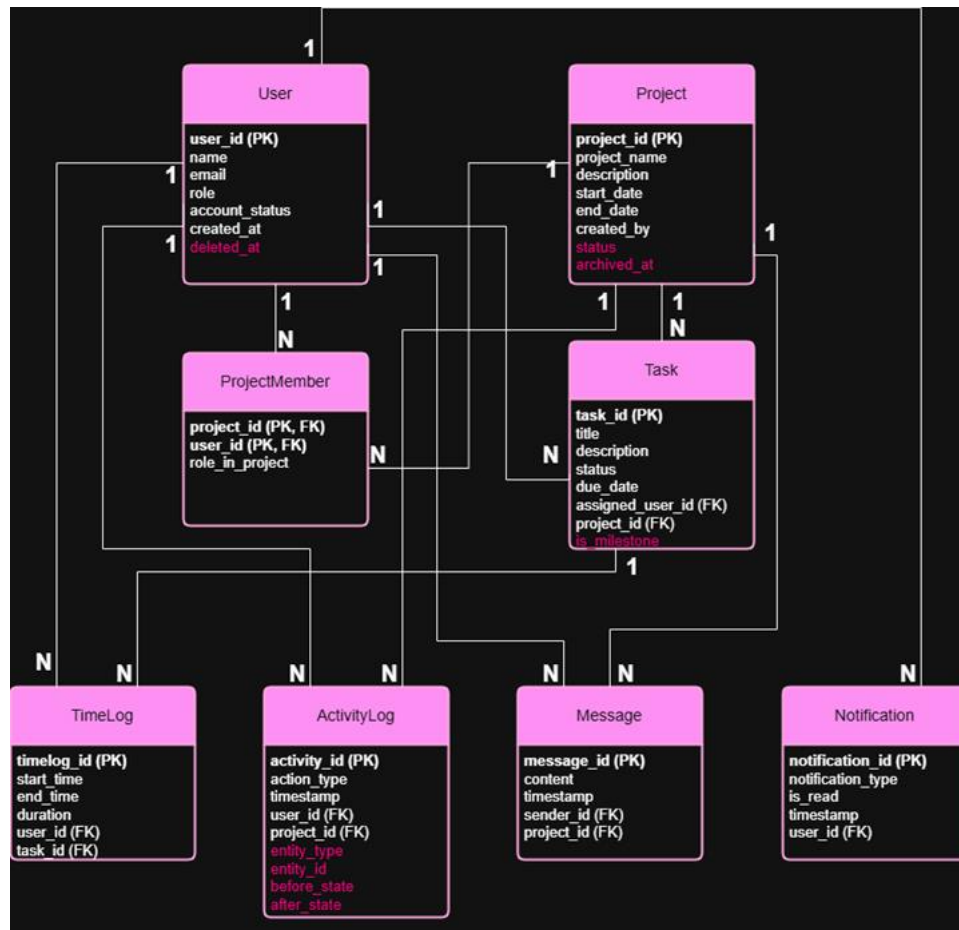


Figure 5.4.1 : Entity Relationship Diagram (ERD).

The diagram shown below is the Entity Relationship Diagram of the TeamTrack system, including its fundamental entities, and relationship between them. The core of this structure is made up of a few main objects **User**, **Project**, **Task**, **ProjectMember**, **TimeLog**, **ActivityLog**, **Message** and **Notification**. All these are the primary operational data that can be used to have the collaboration and contribution monitoring in the platform.

This structure makes it possible for a system to have consistent and unambiguous relationships between users, tasks, and contribution records and larger objectives of academic collaboration and accountability are considered by the organization and traceability of collaboration data.

The diagram shows some important relationships:

- A User can have multiple projects, and many projects can have many users. This many to many is shown through the **ProjectMember** entity which is a table associated between the two.

- Projects serve as lists of Tasks, i.e. a Project may contain many Tasks, but a Task is constrained to belonging to exactly one Project.
- Tasks themselves can have several **TimeLog** entries attached to them, and it offers a way of tracking the work done by individual users over time.
- Projects have messages, not tasks so collaboration within a team can be seem related to the project.
- Notifications are directly associated with Users and are as reminders or system alerts through a targeted delivery model.
- **ActivityLog** is also a valuable auditing tool it captures the major user activities in a project and provides a history that supports transparency and analysis.

5.4.2 Schema Design Considerations

In creating the database schema, relational modelling principles guided the overall approach ensuring data in the system was consistent and could be analysed. The structure produced places core operational entities such as **User**, **Project** and **Task** representations of the main collaborative interactions that are taking place in the platform. These are complemented by supporting entities that include **TimeLog**, **ActivityLog** and **Notification** that record operational history and system events.

Separating the main project data from the historical and activity-related information was one of the design decisions. The two purposes of this division are to maintain performance and be able to execute any analytical queries. Time logs and activity logs, like them, are stored independently of task data. Without changing or interfering with the original task records itself, this framework provides the opportunity to produce contribution analytics and project history.

5.4.3 ERD Attribute Extensions and Improvements

The updated Entity Relationship Diagram introduced several additional attributes to make the data model more consistent with system requirements as specified in both the SRS and TIS. These extensions improve the capabilities of the schema to support deletion, project life cycle management, task specialisation and activity detailing.

User

(soft) Deletion of user accounts was added by adding a timestamp attribute **deleted_at**. Instead of permanently deleting a user and possibly breaking historical references, accounts can be deleted but time logs, activity records and other contribution data remain. The design contributes to privacy needs and the necessity to preserve the correct project history.

Project

Two attributes were added to reflect the project lifecycle in a more detailed way.

- **status:** This is a field that shows the state of the project. (active, completed, archived)

- **archived_at:** Recording of the time the project was archived.

These attributes allow projects to move out of an active phase into an archived phase when they are completed or when an academic term is ended. This enables historical project data to be preserved to be reviewed later or to be used in analytics, but active project data must be easily available.

Task

Milestone is a boolean attribute that was added to depict the milestone type tasks. This design does not introduce an independent milestone entity (that was considered) instead it considers milestones as a type of specialised task, which represents any significant project deadlines or milestones. This keeps the database schema simpler but still allows the system to recognise task related to milestone. The app can then use this when needed, for example when highlighting important deadlines in the interface or having milestone-related notifications.

ActivityLog

The ActivityLog entity got a few more attributes to handle the detailed change tracking:

- **entity_type:** The type of entity that was changed (e.g. Task or Project).
- **before_state:** Records a data structure of the data before the change.
- **after_state:** Archives a data structure of the post-change data.

The fields allow them to record the changes between the different sorts of entities in a flexible and standard style and are accomplished with the use of JSON structures, these features collectively improve system traceability and serve as the foundation for the platform's audit trail capability.

5.4.4 Alignment with System Architecture

The relational schema that is introduced all through this section forms the main transactional data layer of TeamTrack system. The backend API mediates all the database operations and is responsible for the task of user permission verification and system rules enforcement prior to any modifications to the underlying data store.

It is worth noting that although PostgreSQL is used to store documented system records, other categories of high frequency event data may be routed through separate logging or event processing components within the whole system architecture. This isolation is to avoid the imposing of performance costs on the main transactional database by heavy logging of the activity, yet to be able to maintain full records of system activity to be revisited later by analysis or audit.

5.5 Authentication and Role-Based Access (Planned)

TeamTrack will incorporate a token-based authentication model, in which the user will have to authenticate to the backend and be issued a JWT (JSON Web Token), which they will present on future requests in order to access features that require authentication [18]. This is in line with the choice of security strategy to use in the system (JWT-based authentication + RBAC), or in the secure handling of sessions and validation of inputs as part of the layer of system security [17].

5.5.1 Flow of authentication (planned)

In the FastAPI backend, the users will be registered and will log in using special authentication endpoints. Upon successful authentication, the back-end grants a signed JWT with user identifier and appropriate access claims (e.g., role and membership in the project) [18], [24]. The user interface allows the frontend to store and add a token when making secured endpoints calls. The token expiry will be implemented to minimize the exposure to the token compromise and logout will nullify the active session on the client-side (by deleting the stored token) [17]. The same token mechanism would also be aimed to assist in secure session management within the web based interface.

5.5.2 Role model (system- wide + project-level)

TeamTrack is an academic workload, and therefore access control will operate on two levels: a system role (what the user is in the platform) and a role within a particular project (what the user is capable of doing in that workspace). The SRS data model already contains a project membership concept and a role in project attribute, which would permit decisions based on a project instead of providing the one global permission level.

5.5.3 Core roles (planned)

The system will accommodate at least: Student Team Member, Team Leader, Instructor/Supervisor (read only) and System Administrator. The Instructor view is clearly designed to enable auditing and contribution summaries viewing without modifying project data, whereas Team Leaders need more write permissions to administer project metadata and oversight capabilities.

5.5.4 Enforcement of permission (UI second, backend-first)

RBAC will be applied in the backend at the API layer where all protected endpoints verify the JWT and validate role / project membership before the desired operation is performed. This discourages the escalation of privilege even in instances where the user goes around the UI [20], [17]. On the frontend, features inaccessible to the user (e.g. instructor accounts will not have access to the edit

controls) will be hidden or disabled by use of route guards and conditional rendering, but this is considered a usability feature, rather than the security boundary. Because the selected security stack involves token-based authentication and RBAC, the checks should be uniform in each of the modules (projects, tasks, time logs, activity logs, analytics and administration features).

5.5.5 Database-level alignment

Besides API enforcement, stored role and membership systems will be used to facilitate pattern of restricted data access particularly sensitive academic evidences like time logs and activity logs. The SRS specifically defines the capabilities of restricting access according to role such as read-only access to instructors and higher access write access to team leaders in the corresponding domains of project control.

5.6 Design Constraints and Scalability Notes

The section outlines the key design constraints that influenced the system architecture of TeamTrack and captures the scalability considerations that have been included in the suggested design. The requirements are grounded on the topic of Software Requirements Specification (SRS) and Technical Infrastructure Specification (TIS) to achieve consistency between the requirements, architectural decisions, and implementation planning.

5.6.1 Design Constraints

The system design of TeamTrack is affected by the academic, technical, security, and deployment constraints. These limitations form architectural choices and give realistic limits to be followed when implementing under the milestone scope.

(1) Academic Time and Resource Constraints

TeamTrack is being designed within a stipulated academic capstone timeframe. The creation of features should be possible on small time and staffing. As a result:

- The system lays special emphasis on key modules e.g. authentication, role-based access control (RBAC), task management, time tracking, contribution analytics and activity logging.
- Advanced improvements, integrations into the enterprise, and sophisticated real-time infrastructure are deliberately out of scope at this point.
- Design complexity is mitigated so that the system can have an implementation, test; and demonstrating schedule within the academic schedule.

Summary: The system is aimed to provide the basic accountability and transparency capabilities and will not overstep the academic limitations.

(2) Technology Stack and Architectural Constraints

It is a web-based client-server architecture with a Python-based backend application (FastAPI), relational database (PostgreSQL), and structured frontend application (Nuxt.js). Such decisions have a direct impact on system structure by:

- Implementing an API-first architecture, which has all the frontend interactions as RESTful endpoints [4].
- The need to have structured relational data models of core entities like users, projects, tasks, time logs and activity records.
- Constraining architecture designs like distributed microservices or serverless deployment in this milestone.

The design is thus made in line with the chosen technology stack to ensure there is a consistency in requirements, infrastructure planning and implementation of the system.

Summary: Architectural choices are made to be consistent with the stack chosen to minimize the implementation risk and ensure documents are consistent.

(3) Security and Role-Based Access Control (RBAC) Constraints

The enforceability of RBAC and access controls to system data are mandatory restrictions [5]. This has a heavy impact on the design:

- The validation of permission should be done both at the frontend and the backend levels.
- The verification of the authentication tokens and role checks are mandatory in all the safe API endpoints.
- Role-based restrictions must be enforced on sensitive activities like project management, member control and access to analytics.
- To maintain academic integrity, activity logging should be Security enforcement is embedded into system workflows rather than treated as an add-on feature.
- The enforcement of security is integrated with the workflows within systems as opposed to being an add-on feature.

Summary: RBAC is a structural constraint of interface design, API logic, and system behaviour.

(4) AI Feature and Ethical Constraints

AI-assisted functionality is limited to advisory services to summarise activities, detect workload imbalance, or make insight recommendations. The AI module:

- Should not take independent decisions or dictate.
- Should be open in its deliverables.
- Should not supersede the student control or the academic grading.
- Is an autonomous module to reduce the interference with the stability of core systems.
- This ensures ethical use of AI within an academic setting.

It is realistic and not overengineered architecture.

Summary: This system is scalable as per the academic expansion but is not intended to be on the enterprise-level level.

(5) Deployment and Scale Constraints

TeamTrack is not targeted to be used in large scale implementation in enterprises, but small-sale academic teams. As such:

- It is not based on complex enterprise infrastructure.
- Currently it cannot be multi-region deployed and orchestrated.
- The performance will be aligned with the general university project usage patterns.

The architecture is realistic and scalable without being over engineered.

Summary: This system is scalable as per the academic expansion but is not intended to be on the enterprise-level level.

5.6.2 Scalability Considerations

Scalability has been implemented in the architecture although it is not mandatory since it is required in the future as the enterprise scale is reached.

(1) Modular Layered Architecture

The system is designed into distinct layers, which are frontend, backend API, database services, and AI module. This separation:

- Minimizes interconnection among parts.
- Provides freedom of modifications or enlargement of modules.
- Allows expansion without redesigning.

This modular structure supports maintainability and incremental scalability.

(2) Stateless RESTful Backend Design

The FastAPI server is a stateless RESTful server [4]. This facilitates scalability by:

- Possibility to have more than one back end to execute at the same time in case of increased demand.
- Allowing load balancing during subsequent deployments.
- Avoiding server level session bottlenecks.

Stateless request handling forms a scalable foundation.

(3) Container-Based Deployment Readiness

Docker is used to containerise system components [6]. Containerisation:

- Provides similar environments between development and deployment.
- Scales replications of backend or AI services in case of a growth in user load.
- Facilitates easier transition to cloud infrastructure at later stages.

This improves portability and scalability flexibility.

(4) Database Workload Separation

The architecture is based on the PostgreSQL as a structured transactional data storage and another database Firehose as a logging database. This separation:

- Eliminates performance load in the transactional database.
- Prohibits logging business in the retardation of task operations.
- Gets responsive with increasing usage.

Database separation supports performance stability under increased activity.

(5) Alignment with Performance and Scalability Requirements

The design helps in supported performance expectations with:

- Organized relational schema design: referential integrity.
- Well-performing API request processing.
- Possible scaling to pagination, filtering, and indexing when the data volumes grow.
- Future preparedness of caching or analytics summarisation improvements.

Scalability is therefore considered structurally, even within academic scope.

5.6.3 Constraint–Impact–Scalability Mapping

The effects of constraints on the design decisions and the scalability in the constraints decisions are summarized in the table below.

Table 15: Constraint-Impact Mapping Table

Constraint Category	Design Impact	Scalability Handling Strategy
Academic Time & Scope	Central feature prioritisation; elimination of enterprise complexity.	Modular construction is capable of elemental growth.
Technology Stack (FastAPI + Relational DB)	Layered architecture API-first structured schema.	Stateless backend provides horizontal scaling; database indexing provides expansion.
Security & RBAC Enforcement	Lightweight authentication; role-based endpoint protection.	RBAC logic is always endpoint scaled.
AI Ethical Constraints	AI confined to advisory knowledge.	AI module are independent and separable.
Deployment Scale Limitations	Designed for academic teams, not enterprise	Containerisation will facilitate replication and subsequent migration to the cloud.
Activity Logging Volume	Academic team designed, not enterprise.	In dual-database design, workload of logging is isolated.

5.6.4 Conclusion

The well-known academic, technical, security and ethical constraints affect the system design of TeamTrack. These constraints can ensure viability in scope of milestones and ensure transparency and accountability as the underlying system objectives. In spite of such limitations, modular layering, stateless API design that is RESTful and containerised deployment and separation of database workloads have been considered to be scalable. This provides a sustainable platform that is able to support the increasing academic load and resultant upgrading without necessarily making radical changes to the architecture.

6. CID

6.1 Component Interaction Overview

The Component Interaction Diagram (CID) is the description of how the key parts of TeamTrack are interacting in the process of the system functioning. The System Architecture section explains what components exist with their functions. Interaction patterns are shown as users interact with features.

TeamTrack is like a dynamic system rather than a static one. When a user sets up a new task, the system must relay that across multiple areas at once, and considering that, different components work together continuously to process requests and update the system state. Components:

- User-initiated actions
- Request and response cycles of API.
- Authentication and role validation
- Database operations
- Activity logging
- AI processing and response return structured.

The figure 8 in section 6.2 shows the Backend API acting like the main point of coordination, all the interactions outside of it pass through it, ensuring that everything is consistent including enforcement of permissions, structured data updates, and traceable contribution logging. This interaction design supports the following system goals defined in the SRS:

- Clear contribution monitoring.
- Reliably enforcing RBAC
- Secure communication
- Controlled AI-assisted support

Instead of having components interact freely with each other, this model is more structured and layered, because the frontend never directly communicates with storage of AI runtime components. All the operations have to go through the backend before reaching other components.

6.2 CID Diagram (Interaction Flow)

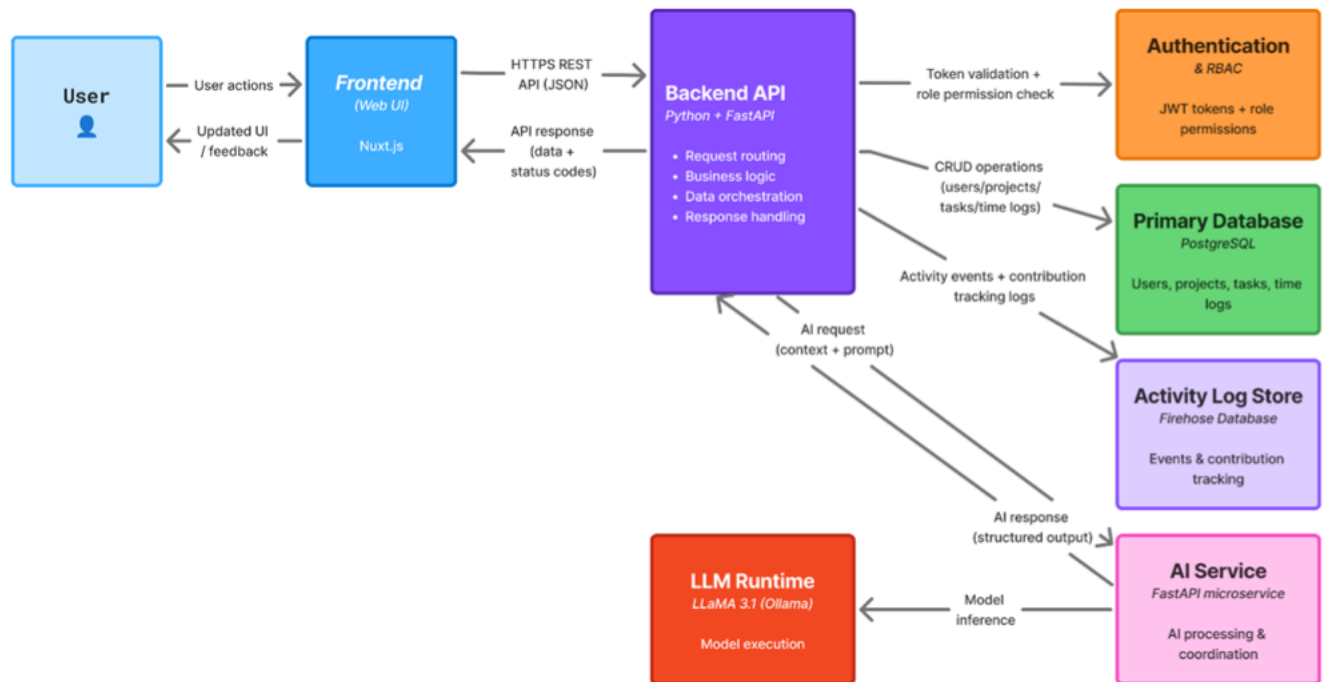


Figure 6.2 : Component Interaction Diagram

The CID shows the interaction sequence that starts with the user and then flows down to the other components of the system, the overall structure:

1. A user performs an action in the frontend interface.
2. The frontend sends a structured request to the backend as HTTPS REST API request.
3. Authentication and role permissions are validated by the backend.
4. If they're permitted, the backend performs the required data operations.
5. The backend optionally logs contribution events.
6. In case AI functionality is requested, the back-end directs processing to the AI service.
7. The frontend receives a structured response from the backend.
8. The user interface is updated in the frontend.

Three main things are also highlighted by the diagram, architectural decisions:

- There is authentication and RBAC that is implemented at the backend boundary before any data operation.
- The process of logging activity is managed independently of relational storage to avoid any decline in performance.
- All communication between frontend and backend is over HTTPS using JSON payloads so that data exchange is secure and structured.

6.3 Interface / Method Call Specifications (Key Flows Only)

This part mentions about primary interaction flows that happen the most frequently in the TeamTrack system. The goal isn't to define every endpoint, but to represent, at a high level, how the system's components coordinate during its most important operations.

6.3.1 Task Creation and Update Flow

When a team member creates or updates a task, the interaction flow is as follows:

- The frontend gets information about the task like title, description, due date, priority, assigned members, etc.
- The HTTPS API request is sent to the backend.
- The backend authenticates the user and ensures he/she is allowed to create or make changes to tasks of that project.
- The task record is either created or updated in PostgreSQL.
- Action is logged and kept as record
- A structured response is returned to the frontend.
- The UI is updated in order to include the new or changed task it should reflect it.

Other than supporting the need that users be able to create, assign, and change tasks within a project workspace, this interaction flow also aligns with the requirement that important task-related actions are documented so that contributions can be easily traced. The system can track participation and uphold transparency throughout project operations by keeping tasks organized and recording updates.

6.3.2 Contribution Logging and Analytics Flow

The following is the system flow of how contribution data is recorded or displayed.

- An action is performed by a user like recording time, completing a task or sending a team message.
- Frontend sends the action details to the backend
- Backend validates permissions
- The project data is stored or updated in PostgreSQL.
- An activity log store entry is written with corresponding event.
- When the analytics dashboard is called, the backend gets both structured project data and the activity data logged.
- Backend merges and processes this data
- The processed analytics is sent to the frontend that presents the contribution summaries.

The flow follows the requirements that the system should have a contribution analytics dashboard to display the measured participation data. It reflects the systems' purpose of improving fairness in group work, since it makes contributions visible.

6.3.3 Authentication and Role Enforcement Flow

The steps and flow when a user logs in or attempts to perform an action:

- The user submits their login credentials
- Backend verified the credentials
- An authentication that is successful results in a secure JWT token.
- This token is attached to future API requests
- Each of the secured requests is validated by the backend.
- The role of the user is established by the backend (member, leader, etc)
- The backend performs a check of whether the role is authorised to execute the requested action or not.
- In case it is valid, the request goes through. Otherwise, an error response is provided.

This flow supports the requirement that the system must have secure authentication and role-based access control. It also guarantees that project data, tasks assignment, administrative controls, and analytics are limited based on user roles to make sure the project workspaces are fair and there's no misuse.

6.3.4 AI-Assisted Insight Flow

The interaction flow for when AI-based insight is requested:

- The frontend makes a request to fetch the contribution insights or project feedback.
- The backend gathers the relevant project data using PostgreSQL
- The backend retrieves related activity logs from the logging store
- This information is organized by the backend into a contextual prompt.
- The prompt is sent to the AI service
- The AI service forwards the request to the language model runtime
- The model generates a structured response.
- The AI service sends the processed result to the backend.
- The final structured response is relayed to the frontend by the backend.
- The UI displays the AI-generated insight.
-

The flow supports the updated requirements introduced in the TIS where AI-assisted insights enhance the contribution analytics. The AI component doesn't replace the structured tracking but rather, based on the recorded data of activities, it creates an overview of the context and provides intelligent feedback to users.

6.4 Component Responsibility Allocation (Team Ownership)

In the development effort for TeamTrack, each team individual member is allocated key system components determined by each person's technical role in the project. In this organisation, members focus on specific system components while also making contributions to the platform's overall integration.

Hashir - Project Manager and Backend Operations

Hashir is in charge of project management and backend operations. He is in charge of overseeing the backend architecture, managing the project process, and making sure that backend operations and the API that connects all of the system's components are operating as intended.

Omar - Backend Engineer

Omar is responsible for implementing the backend operations that the system relies on, so this includes things like the FastAPI service layer, managing the requests that are processed, and implementing the business logic that is required for things like task management, contribution tracking, and data processing activities.

Siraj - Frontend Engineer

Siraj focuses on the development of the front-end application that enables the user to interact with the application. The responsibilities of the role also extend to the integration of the frontend application with the backend API, apart from the development of the application interface.

Maria - UI/UX Designer and Interface Implementation

Maria focuses on the user interface of the application, as well as the structure of the application interface, in order to maintain the usability of the application interface for the target users of the application.

Zernish - Data and Security Analyst

Zernish is responsible for both security and data architecture of the system. This includes designing database schema, managing data structures of PostgreSQL, and designing data storage and processing of activity data, and participating in security components of other parts of the system too.

7. UI Design and Prototyping

7.1 UI Design Approach and Standards

Here, the design strategy of UI that was applied in TeamTrack and the rules that were applied throughout the screens to ensure that they were cohesive, convenient, and understandable are outlined. The interface was designed to suit the requirements of the university project teams who require managing tasks, tracking the contribution rates, and receiving the insights on AI assistance. Due to this fact, the design has been focused on fast understanding of project status, easy navigation among modules without the need to put much effort and simple and clean design that makes the project easy to understand when using it on a regular basis.

7.1.1 UI Design Goals

The UI has been developed in such a way that users can know their position and what they can do in a few seconds of opening the platform. The design ensures that all screens look the same with similar layout, typography, spacing and similar button designs hence the user does not have to retrain the system to traverse features. One of the objectives was enhancing transparency, which is, the information regarding progress and the contribution is displayed through the dashboard graphs instead of being concealed behind several clicks.

The interface is also designed to be lightweight and only one purpose is shown on the screens at a time, whether it is authentication, tracking, or AI interaction. This approach aligns with the concepts of usability that imply the minimization of the unnecessary complexity so that the users could focus on the effective execution of the work. The usability principles that have been implemented highlight the necessity of possessing a definite system condition, predictable navigation and recognisability interaction rather than remembering how the system works (as a requirement) [1]. The prototype was developed based on these principles and the location of the interface elements in the system.

Finally, the design enables the real student processes to be supported by aligning the screens to the real world academic operations like recording time, contribution summary and task progress. This will ensure that the interface is grounded on how the users will use the system in the actual project work.

7.1.2 Layout Structure and Navigation Standard

TeamTrack is persisting with a classic layout approach to make the navigation predictable. The top navigation bar of the home page and other public pages such as the landing page has a basic top navigation bar with page anchors (Home, About, Features, Support), and direct access (sign up, log in). This must help the new users understand the role of the platform comparatively fast and get into the onboarding process without confusion.

Following the authentication, the interface changes to an application-like interface with a permanent sidebar on the left side to all essential features such as tracking, reports, AI chat, organisation tools, integrations, and user settings. This navigation model gives the user the ability to navigate through different modules of the system in the shortest time possible and provide an awareness of their location in the system on a constant basis.

Having regular navigation structures is highly advisable in interface design models on the basis that familiar layouts minimize user mistakes and enhance learnability. The interface design canons proposed by Shneiderman highlight the significance of consistency and the ability of the system to be properly organised to make the complex systems easier to interact with by the user [2]. These ideas played a role in the choice of a consistent navigation system on the platform.

7.1.3 Visual Hierarchy and Information Prioritisation

The UI uses easy visual hierarchy to enable users to scan and understand information. Both pages have a large content area in the center, and the navigation is reserved to the sides of the interface. The system also ranks indicators of the project that are of utmost importance towards the top of the dashboard and tracking pages. These metrics are the percentage of completion of the project, the number of hours worked by the team members, and the charts of the workload distribution.

Representing the information with the help of visual summaries will contribute to quicker interpretation of the project status and will save the user time when analysing enormous amounts of numerical data manually. The fact that similar information can be put in segments also enhances the level of readability and will enable the user to first concentrate on the most important information.

The presence of the definite hierarchy and information groupings conforms to the general user interface design guidelines that promote the more essential material to be positioned on the forefront and the secondary information to be structured and available to the user [1].

7.1.4 Component Consistency Standards

The consistency of the interface is very important so that a single user experience is created. TeamTrack contains universal rules of styling of buttons, input fields, navigation items, and content containers on all screens. The main actions are performed with the help of visually differentiated call-to-action buttons, and the secondary controls are intentionally kept light in order not to

interfere with the primary actions on a page. The authentication screens are also made in a similar format which has well defined input boxes and controls in the application such as password visibility options and remember me buttons. Predictability of component behaviour will ensure the users quickly get acquainted with the interface and reduce interaction mistakes.

Uniformity in interface behaviour is highly advised in the usability specifications since the users can apply their knowledge in one section of the system in another without re-educating themselves [2]. This is due to the ability to have a uniform layout and component behaviour making the interface simpler to learn and predictable when using it regularly.

7.1.5 Accessibility and Readability Standards

Some fundamental considerations of accessibility were made on the prototype. The interface is well contrasted in terms of text and other background to enable the readability in the different displays. There is also the consistency in the use of spacing between content sections to prevent visual clutter and scan information quickly.

The terms of navigation and interface were also made simple to enable the users to understand what each part of the system is all about with ease. Clearly marked sections such as Reports, Tracking and Settings reduce confusion and increase system learnability.

The aspects of accessibility may be contrasted to the web accessibility guidelines that facilitate readable contrasts, sensible hierarchy and readable navigation to accommodate a wide range of users [3].

7.1.6 Feedback, Guidance, and Error Prevention

The UI design has the elements which help in the guidance of the user and reduces the risk of the interaction error. The authentication forms also possess such convenient features as password visibility options and remembered log-in options, which reduce the amount of frustration in making frequent visits.

It also has distinct call-to-action buttons and this will enable the user to understand what is required next in routine processes such as registering, logging in or using the features of the systems. In the AI chatbot interface, there are suggested prompts that guide a first time user who may not be familiar with how to use the conversational systems. These prompts show the potential questions and prompt an investigation into the capabilities of the AI assistant.

Giving instructions and avoiding typical interaction mistakes is a highly known principle of usability that enhances more user confidence and minimises errors in dealing with digital systems [1].

7.1.7 AI Interaction UI Standard

The interface of the AI assistant is created more as a supportive tool rather than the control mechanism of the platform. The chatbot page has a bare bone design that gives priority to the chat interface. It has suggested prompts, which are employed to guide the interactions and reduce confusion when the users want to obtain information of the system.

The separation of the AI interface and the more noticeable project management features will also be used to maintain the platform clarity. This kind of design is supposed to alert the users that the AI assistant may provide insights and suggestions, yet does not determine how the projects are proceeding and change the system data automatically.

7.1.8 Summary

Overall, the layout of the UI in TeamTrack is based on the utilization of the frequent patterns of layouts, anticipated navigation, high visual hierarchy, and legibility of the interface elements. The prototype will also strive to create the system that will be easy to use by the student teams, as well as capable of supporting more complex capabilities such as contribution analytics and AI-supported insights with the assistance of the established principles of usability and accessibility.

These are the conventions that provide a level platform on which the implementation stage is founded and which also might ensure that the further stages of the design will be capable of further developing the interface and remain useful and understandable at the same time.

7.2 Wireframes and Screen Flow (Early Layouts)

Overview

The TeamTrack wireframes are the first formal visualisation of the interface of the platform that was developed before low-fidelity or high-fidelity prototyping. Created to provide the overall structure and layout of screens, these wireframes were concentrated on determining the positioning of the main components, the layout of screens between each other and the logical flow of user actions without using visual effects or colour. The wireframes were created in Figma, and mentioned as artefacts in the Appendix.

The wireframe set includes five major screens, including the Landing Page, Login, Sign Up, Dashboard, and the AI ChatBot. They chart the full journey that a user might follow when joining the platform all the way to authorized use of the internal functionality of the system.

7.2.1 Screen Descriptions

Screen 1: Landing Page

The Landing Page is considered the front door to TeamTrack. The wireframe also defines a top nav bar with the platform logo in the left section, a horizontal nav menu (Why Us, Task View, Management, AI, About Us), and a login button on the right. The platform tagline appears in the hero section. The delivery was focused on the screen to convey the purpose of the product instantly. There is a shortcut menu and an account indicator, which is found at the upper side, to offer quick access to returning users. The display is deliberately minimalist and guides the user to one key activity, which is to log in.

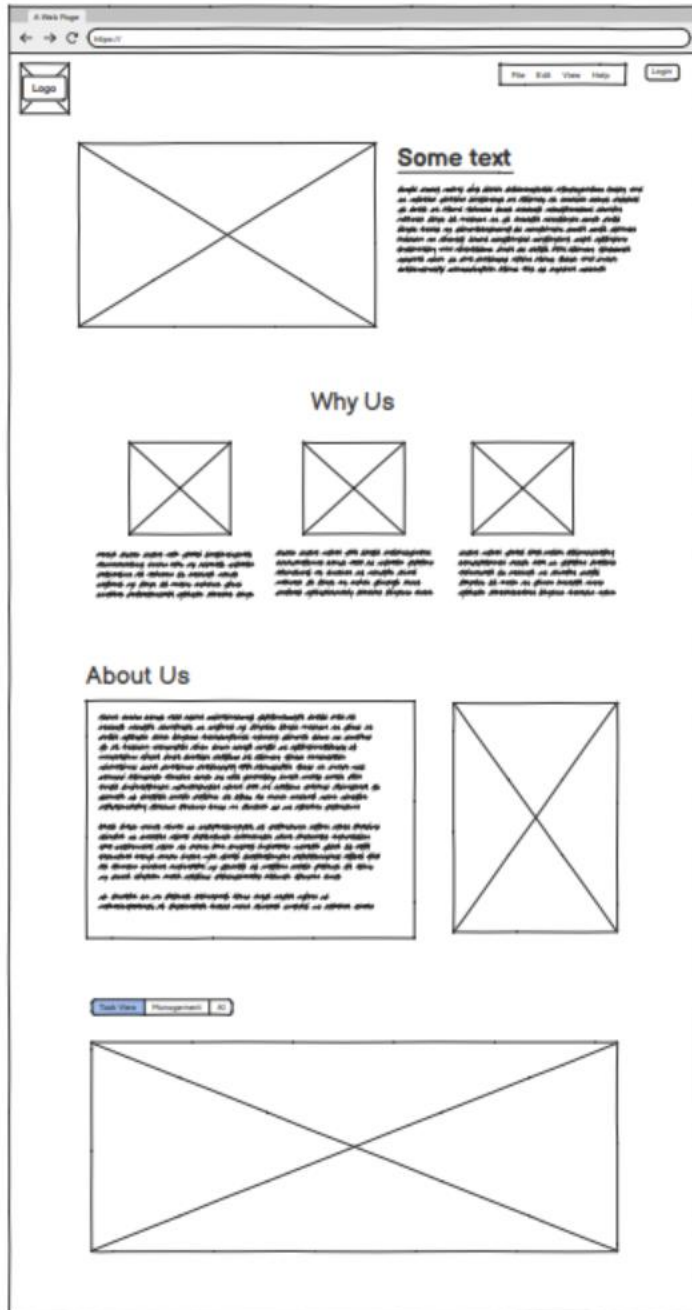


Figure 10: Wireframe Landing Page

In this wireframe, the following are important aspects:

- Logo placement (top left)
- Five items on the navigation menu.
- Account area and shortcut menu.
- Login button as the major call to action.

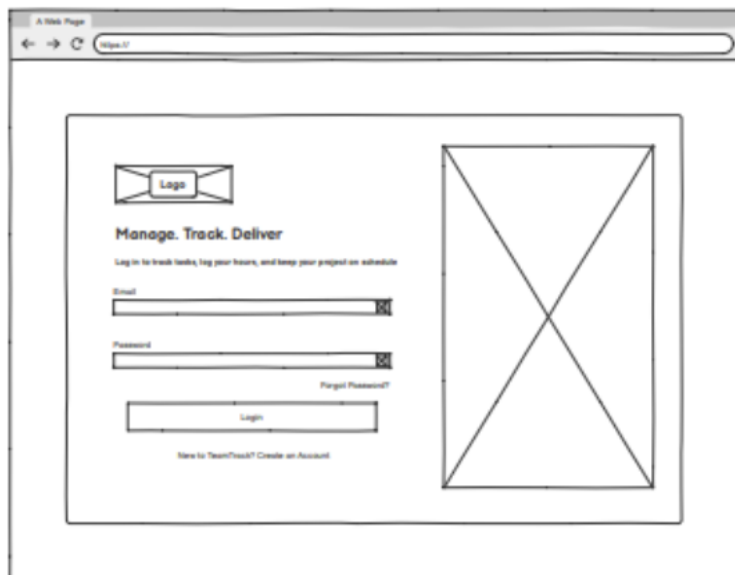
Screen 2: Login Page

The Login wireframe includes a centered authentication card with a brief instructional subheading “Log in to track tasks, log your hours, and keep your project on schedule” and an Email field, a Password field, Forgot Password link and a primary Login button. In the bottom, there is a secondary connection that reads “New to TeamTrack? Create an Account”, which offers a straight forward route to registration devoid of any clutter to the main form. The logo is not removed in the header because it is necessary to have a continuity of the brand in all the public screens.

The important items to this wireframe:

- Context instructional subheading
- Input fields of email and password.
- Forgot Password link
- Primary Login button
- Link to Sign Up for new users

Figure 11: Wireframe Login Page



Screen 3: Sign Up Page

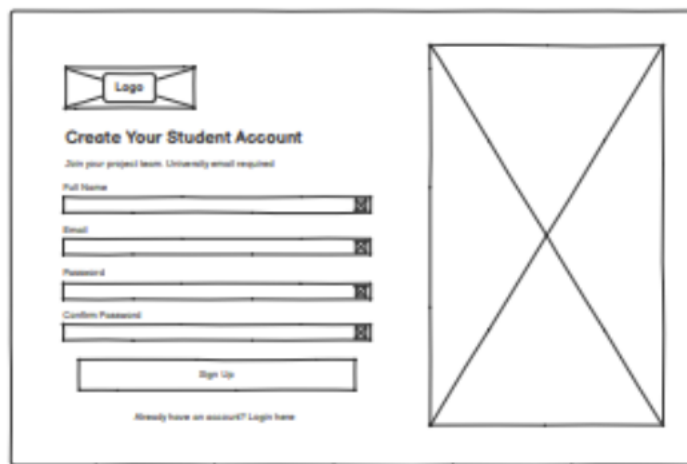
The Sign Up wireframe reflects the form of the Login design. The form gathers four items including Full Name, Email, Password and Confirm Password. In the bottom of the fields is a Sign Up button, which is the main action, and a link indicating the presence of an account already: Already have an account? Backing home: Login here gives the user the option of moving back. The header note is defining an academic access restriction upon registration, indicating that it is obligatory to have the

University email. This limitation is the direct mapping of the system role based access control needs.

The major components of this wireframe include:

- Email, Confirm Password, Full Name, Password, and Confirm Password.
- University email mandatory notice.
- Primary Sign Up button
- Back-link to Login

Figure 12: Wireframe Sign up Page



The wireframe shows a sign-up page layout. On the left, there is a 'Logo' placeholder, followed by the title 'Create Your Student Account' and a note 'Join your project team. University email required'. Below this are input fields for 'Full Name', 'Email', 'Password', and 'Confirm Password', each with a small 'x' icon on the right. A 'Sign Up' button is positioned below the fields. At the bottom, a link reads 'Already have an account? Login here'. On the right side of the page, there is a large rectangular area with a diagonal 'X' across it, indicating a placeholder for an image or graphic.

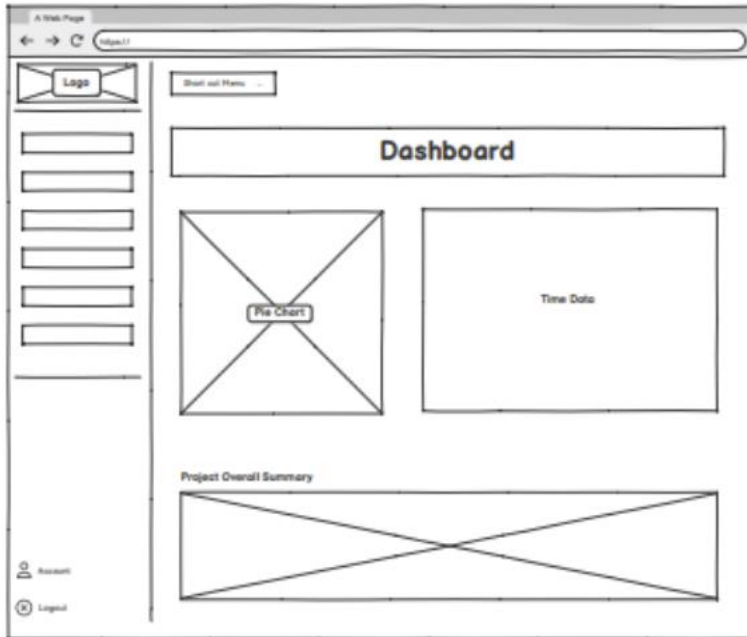
Screen 4: Dashboard

The dashboard wireframe is a screen that is structurally the most complex of the set. It brings about a consistent sidebar design that is extended into the low-fidelity prototype. The shortcut menu, logout control, and account section are on the sidebar along with the logo. The primary content panel is separated into three, namely, Pie Chart to visualise the contribution, Time Data panel to record logged hours, and a Project Overall Summary panel to give an overall status. The dashboard has a Logout button that is always available in case of controlling a session.

The primary aspects of this wireframe:

- Logo, shortcut menu, account, and logout sidebar.
- Pie chart panel of contribution data.
- Time log of registered hours.
- Overall summary panel of the project.

Figure 13: Wireframe Dashboard Page



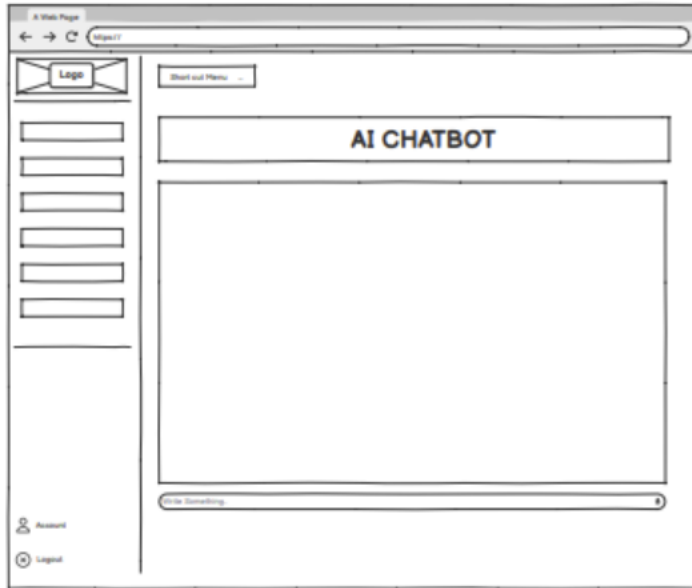
Screen 5: AI ChatBot

The wireframe of the AI ChatBot is made sparse. It also carries the same sidebar layout as the Dashboard, having the logo, shortcut menu, logout, and account section all there, which creates visual consistency across all the authenticated screens. The chat interface with which the pivot is devoted is located fully in the main content area and is called AI CHATBOT, where a single text input (Write Something..) is placed at the bottom. Such a design choice is a result of the design choice to keep the AI interaction interface clean and uncluttered, minimizing cognitive load and making the user purely focused on the conversational input.

The major aspects of this wireframe are:

- Logo with sidebar that does not disappear, shortcut, and logout.
- Full-width chat area
- Lower based text entry field.

Figure 14: Wireframe AI Chatbot Page



7.2.2 Screen Flow

The wireframes follow a clear and linear screen flow that maps to the authenticated and unauthenticated states of the system:

Step	Screen	Transition Trigger
1	Landing Page	User opens TeamTrack
2	Login	User clicks Login or is a returning user
3	Sign Up	User clicks "Create an Account" from Login
4	Dashboard	Successful authentication
5	AI ChatBot	User selects ChatBot from the sidebar navigation

Table 16 : Screen Flow Table

The flow has a stringent authentication gate. None of the authenticated screens are displayed without going through either the Login or Sign Up screen. This is directly mapped with security and role based access control requirements which were defined in the system specification.

7.2.3 Design Rationale

The wireframe phase was used in the design process to achieve three main functions. To begin with, it enabled the team to confirm the information hierarchy of every screen and spend time on visual design since the most vital activity, i.e., the ones like the Login, Start Tracking, and Open

ChatBot, were always the most visible elements on the screen. Second, it created a standard layout language through the authenticated screens, and there is always a sidebar with a logo, a set of navigational links, and a logout button that ensures a user does not lose his/her way when using the platform. Third, it revealed structural choices very early on, like the university email requirement on Sign Up or the three-panel structure of the Dashboard, that would have been difficult to modify later in a later prototyping or development phase.

The wireframes deliberately do not contain colour, imagery and detailed typography. Their worth is layout rationality and interaction sequence as opposed to visual attractiveness, which is tackled by the following low-fidelity and high-fidelity prototyping stages. The appendix provides the links in full Figma artefact.

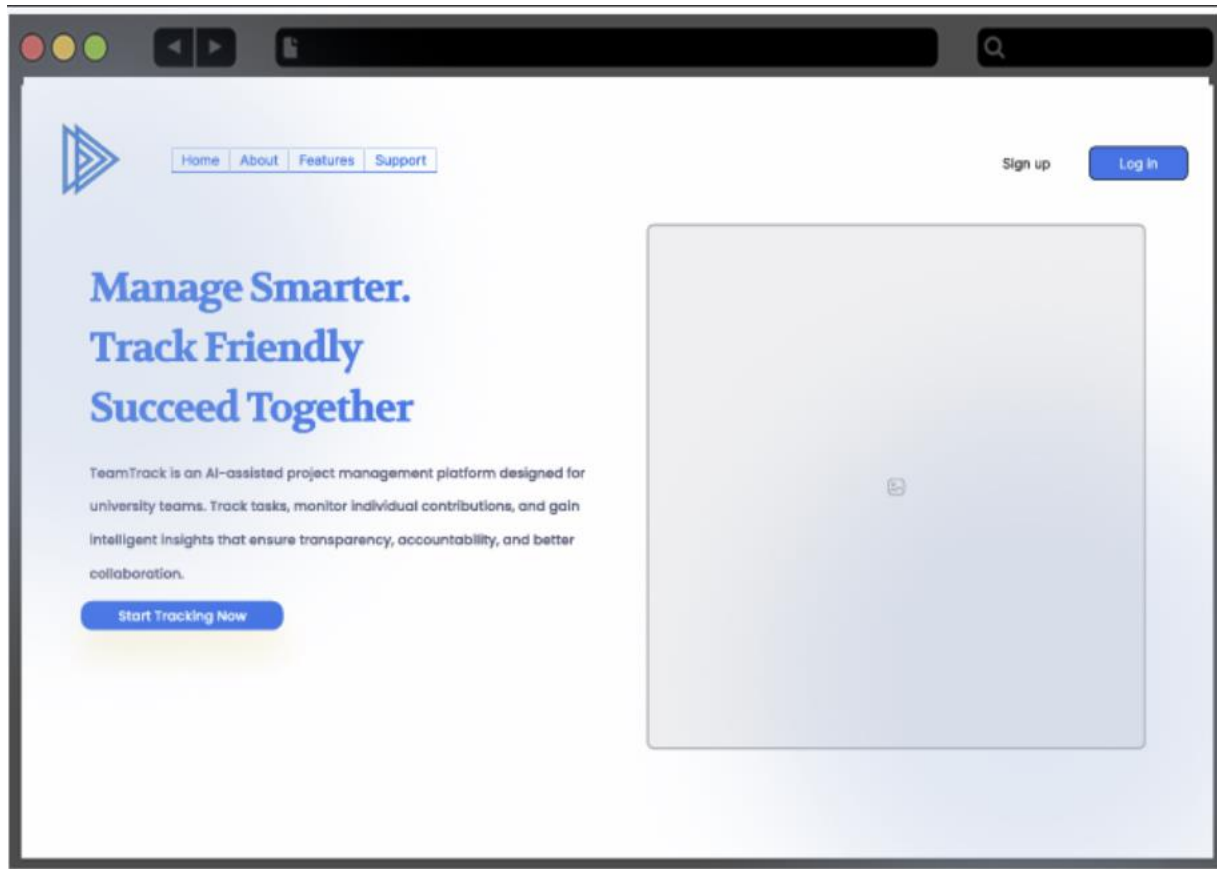
7.3 Low-Fidelity Prototype (Screens + User Journeys)

The TeamTrack low-fidelity prototype was created with Figma, and it generated a set of wireframe-based screens that define the fundamental layout, navigation flow and interaction structure of the platform. The prototype has not been made to look pixel perfect, but rather it places a high value on information hierarchy, usability and logical flow with which a user would navigate in managing a university group project. The prototype will include 5 screens, such as the Landing Page, the Login, Signup, Student Dashboard and the interface of the AI ChatBot. Each screen was created to mirror the functional requirements stated in the requirements gathering stage which makes the wireframes a significant guide to the high-fidelity design and further development. The Figma artefacts are fully interactively linked in Appendix.

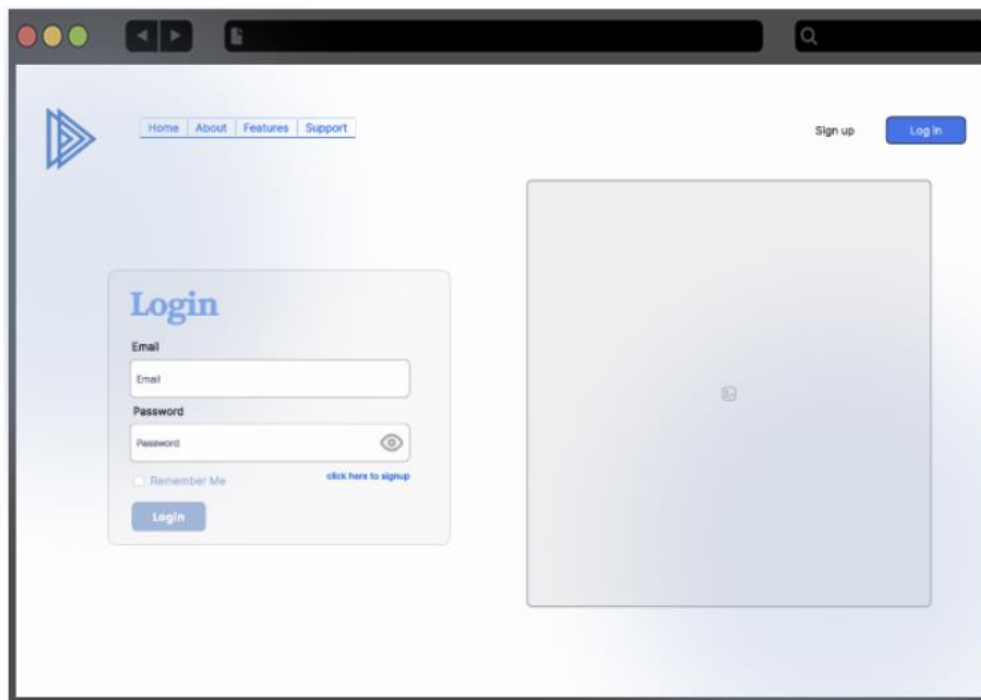
7.3.1 Prototype Screens

All the five screens made in Figma as the low-fidelity prototype are shown in the following figure. All screens will be annotated with the most important interface elements that will be brought into the high-fidelity and development stages.

Figure 15: Low-Fidelity Designs



Landing Page



Home About Features Support

Sign up Log in

Sign Up

Email

Password

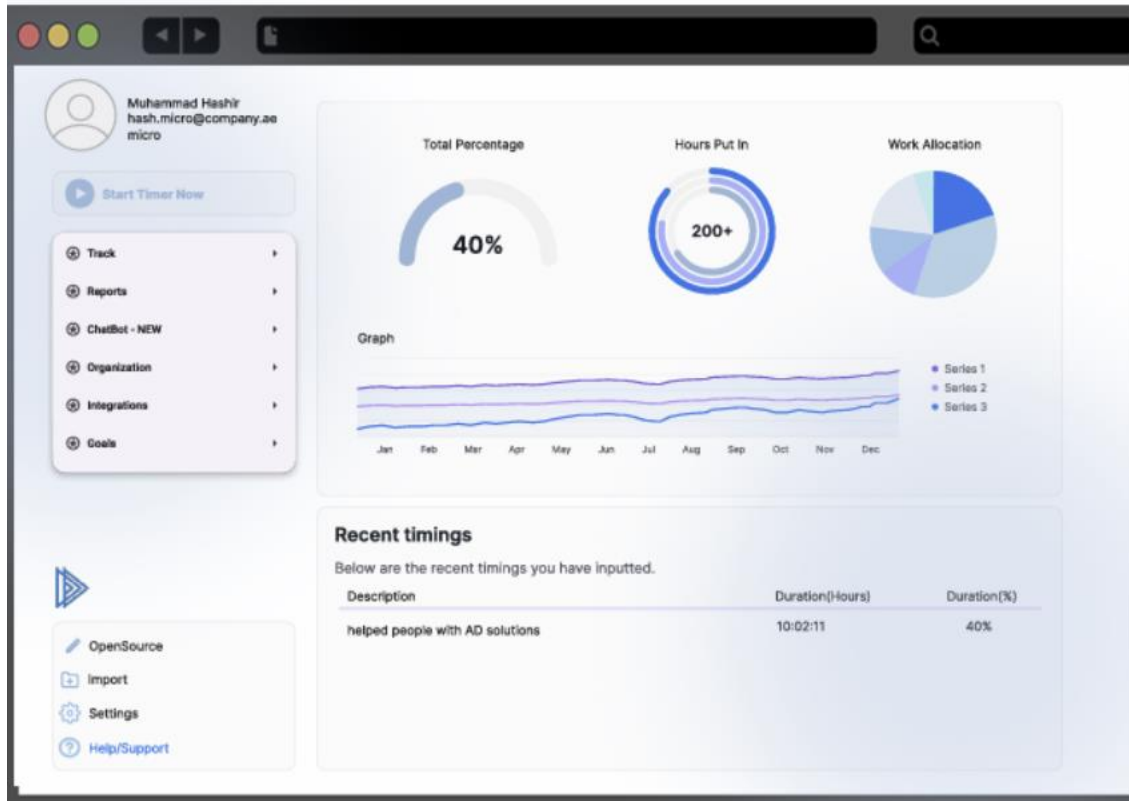
Retype-Password

☐ I agree to terms and conditions

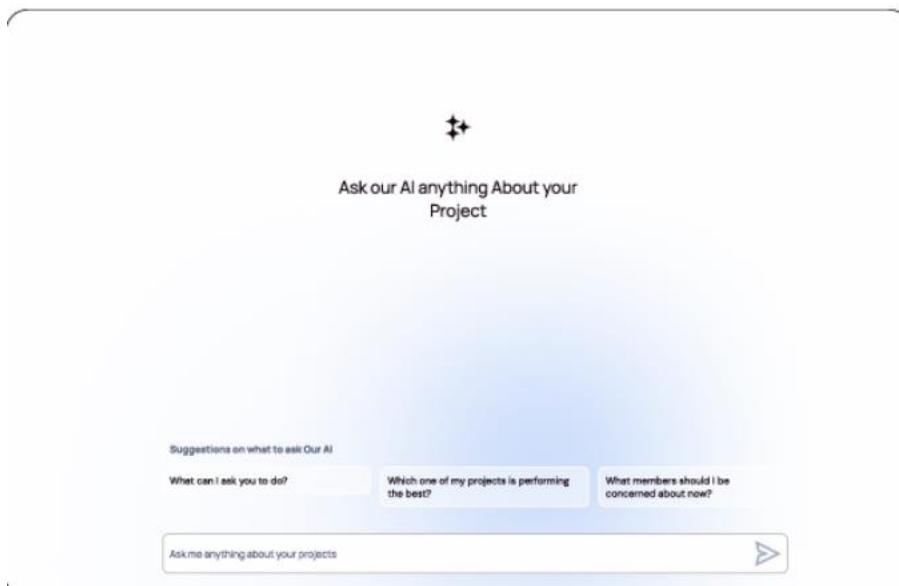
Sign Up

Already got an account? [Click here to login](#)

Login/Signup Page



Main Dashboard



AI Chatbot page

An outline of each of the screens with their navigation path and the important UI components are briefly represented in the table in table 7.3.1 below.

Table 17: Prototype Summary Table

Screen	Navigation Path	Key UI Elements
Landing Page	Home → Landing	Hero headline, platform tagline, CTA button, navigation bar
Login Page	Landing → Login	Email field, password field, Remember Me toggle, login button, link to Sign Up
Sign Up Page	Landing → Sign Up	Email, password, retype-password fields, T&C checkbox, Sign Up button, link back to Login
Student Dashboard	Login → Dashboard	User profile panel, sidebar navigation, KPI widgets (percentage, hours), time-series graph, recent timings table, timer start
AI Chatbot	Dashboard → ChatBot	Prompt input, AI response area, suggested prompts panel, project-aware question examples

7.3.2 Explanation of Each Screen

Screen 1: Landing Page

The Public Facing Interface to TeamTrack is the Landing Page. It will mainly be used to convey the value proposition of the platform and push new users or returning ones to the correct authentication path. The screen has a very big hero headline, which is, "Manage Smarter. Track Friendly. Succeed Together," a description of the short platform and a noticeable call-to-action button, which is "Start Tracking Now." There is a navigation bar in the top that has links to Home, About, Features, and Support, Sign Up and Log In buttons. The high-fidelity version has a blank image place on the right hand side of the hero section where a product illustration or promotional image can be placed.

The major design choices on this screen are:

- Having a CTA button in the upper section of the hero area to minimize the friction of the new users.
- Maintaining navigation very simple so as not to overload the layman visitors.

- Separating textual information and visual media using two column layout.

Screen 2: Login Page

The Returning User Page gives returning users an easy authentication form. The wireframe consists of an email field, a password field with a visibility option, a checkbox that allows persisting a session, and a major button labelled as Login. There is a secondary connection at the bottom, which states Click here to Sign Up, so that the registration flow can be found, but the main view is not overloaded. There is a dummy panel at the right hand side that replicates the structure that has been adopted on the Landing Page consistent with visual consistency on any screen accessible by the public.

The major design choices on this screen are:

- Centralisation of the form to get the instant attention to authentication fields.
- Similar to the Sage Products, adding a password visibility option to minimize typing mistakes.
- Direct linking to Sign Up to cater to the users who get into the mode of login without an account.

Screen 3: Sign Up Page

New users can access Sign Up Page where they can create a TeamTrack account. The form asks the email address, a password, and a password confirmation field so that the user cannot enter the information twice. The fields also have a terms and conditions checkbox below them as a compliance component. The primary action is made to look like the "Sign Up" button and there is a footer link that says; Already got an account? Click here to login" has a definite way out. The screen is a reflection of the Login Page design, he does not change the two-column composition and preserves the visual of the placeholders on the left.

The major design choices on this screen are:

- To prevent typing errors prior to creating an account, have a retype-password field.
- Other: Adding a T&C checkbox in order to make legal and academic integrity requirements met.

When moving between the two pages, the symmetry in layout has to be maintained with the Login Page so that the cognitive load is reduced.

Screen 4: Student Dashboard

Student user pivot Once authenticated, student users are redirected to the Student Dashboard. The prototype includes the most sophisticated screen that serves as the central point where all the key features can be accessed. The design is categorized into a left side bar and content area. The sidebar includes the profile information of user (name, organisation, email) as well as links to navigation to Track, Reports, ChatBot (marked as NEW) and Organisation, Integrations, and Goals.

The bottom of the sidebar is occupied by secondary utilities like OpenSource, Import, Settings and Help/Support.

The central content area is further broken down into three KPI metric displays at the top total percentage (40%), hours put in (200+), and work allocation (pie chart), and a time-series contribution graph during the entire calendar year. Under the graph, there is a table-like section named Recent Timings which shows an entry of time as a table which is divided into columns of Description, Duration (Hours) and Duration (%). An always-present, Start Timer Now button can be found in the sidebar that allows one to track time in a hurry without leaving the dashboard view.

The main design considerations on this screen will be:

- Sidebar navigation in order to enable continuous access to all major sections without reloading of page.
- KPI widgets on the top of the content area to reveal the most important data as soon as one logs in.
- A time-series of one year to avail the students and lecturers with a visual representation of the patterns of contribution.
- A table of recent timings log to have traceable evidence-based contribution records.
- Adding the ChatBot link named NEW to make the AI feature visible and get people engaged.

Screen 5: AI ChatBot

The AI ChatBot display offers a conversational interface, which features AI inference layer of the platform. The screen has a very minimal and focused design: there is a central prompt box with the message “Ask our AI anything about your Project” written in it, a text input box at the bottom with a send button, and a panel with suggestions on what to ask AI completed of prompts written in cards. Examples suggestion cards would be questions like, which one of my projects is doing the best? And “Who knows what members I ought to be so much concerned about at the moment? — directly due to the AI use cases that were identified in the system requirements.

Some of the major design choices adopted on this screen are:

- Timely recommendation cards reduce the entry barricade to individuals who are not accustomed to AI assistants.
- It has a clean and distraction free design that ensures the focus is on the conversational interface.

The screen will be placed on the sidebar in ChatBot, and it will be fully included in the primary navigation framework.

7.3.3 User Journey Diagram

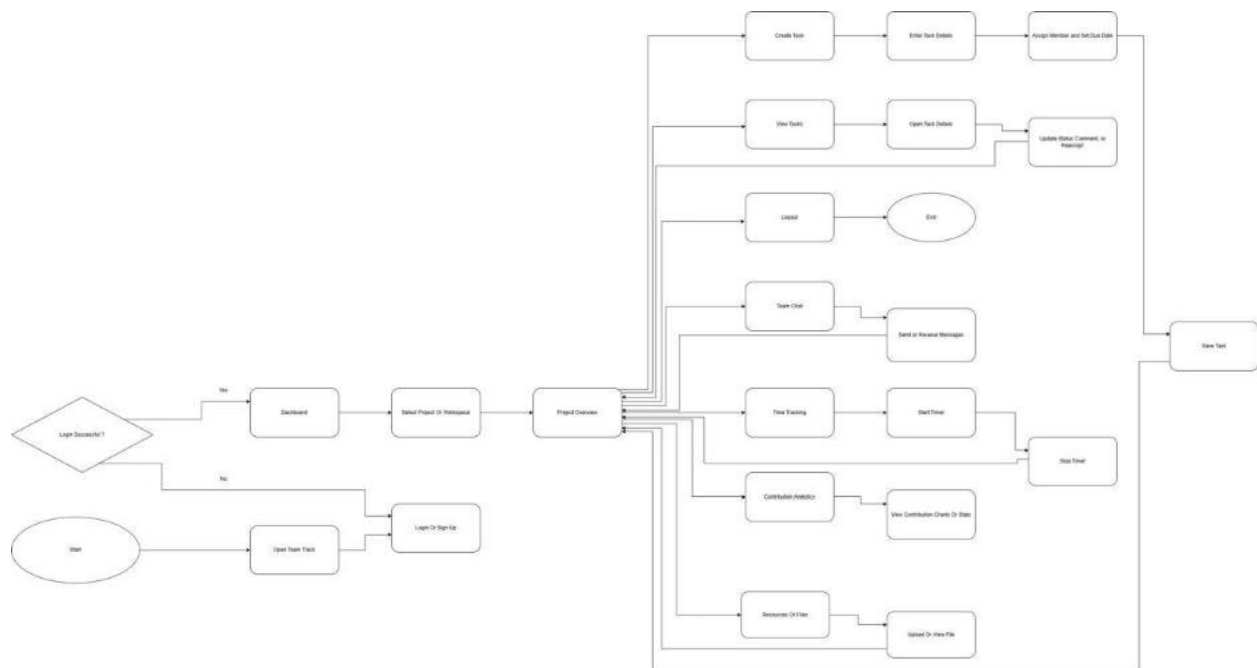
The user journey diagram below illustrates the entire flow of interaction between a user and the TeamTrack system, beginning with first access to the system and continuing onwards to the full range of authenticated actions accessible to a user in a project. The diagram was created in

conjunction with the low-fidelity screens so that all screens and paths of navigation are taken into consideration, and the prototype is representative of the actual user behaviour patterns.

Start is a point where the user clicks on TeamTrack and gets to the Login or Sign Up screen. When authenticated successfully, what appears to the user is the Dashboard, where he/she chooses a project or a workspace, which takes them to the Project Overview. Based on the Project Overview, the user can split into six major paths of activities:

- Create Task: Add the task information, allocate a member, a due date, and save the task.
- View tasks: Open task details and change the status, add comment or reassign.
- Team Chat: Send or receive messages in the project team.
- Time Tracking: Start and stop a timer that is associated with a particular task.
- Contribution Analytics: See contribution charts or statistics of the project.
- Resources or Files Upload or view project related documents and files.

Figure 16: User Journey Diagram



There is also a Logout path, which ends at End, which is termination of the session. As can be seen in the diagram, every one of the five prototype screens is directly associated with this journey: the pre-authentication flow includes the Landing, Login, and Sign Up screens, whereas the authenticated workspace includes the Dashboard and ChatBot screens.

7.3.4 User-Friendly Design Principles and Colour Palette

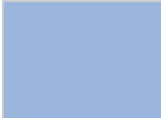
The design principle of the TeamTrack low-fidelity prototype revolves around the ideas of clarity, accessibility and consistency. Each screen was designed in such a way that it only required a user to make very few decisions at any given moment, minimised cognitive load because of established layout conventions and made sure that the most important of actions are always at hand right there. The prototype will be based on a set of established UX principles, which will be implemented to the high-fidelity and implementation phases of the project.

User-friendly design principles were used as follows in the prototype:

- Consistency: There is a similar structure of navigation bar, typography scale, and hierarchy of the buttons within all the screens, and that is why the users do not need to re-learn the interface when switching between screens.
- Progressive Disclosure: Only when the user clicks on the corresponding section, complicated functionality (like analytics and AI insights) is revealed, so that information is not overloaded on the initial visit.
- Affordance and Feedback: Presentation is easy to understand, and interactive components like buttons, toggles, and timer controls are indicated and labelled in a fashion that allows users to anticipate the result of an act even before performing it.
- Error Prevention: Confirmatory fields (retype-password) and validation checkboxes (T&C agreement) can be used to ensure that errors are prevented during the early stages of spreading to the system.
- Accessibility: The text and background combinations on all screens have adequate contrast ratios that are compatible with the WCAG guidelines and make everything readable.

The choice of colour palette taken in TeamTrack is mindful. The colours have a specified purpose within the interface, and as a result, there are visual consistency and harmony, making the platform look professional and credible as an academic resource. All the palette is given in Table 7.3.2 below.

Table 18: Colour Palette

Swatch	Hex Code	Colour Name	Usage in Design
	#9AB6DC	Steel Blue	Secondary backgrounds, sidebar accents, chart fills
	#1570EF	Primary Blue	Primary buttons, CTAs, active navigation links, key headings
	#FFFFFF	White	Page backgrounds, card surfaces, modal overlays

	#5E6282	Muted Slate	Body text, labels, placeholder text, secondary icons
	#000000	Black	Primary headings, high-contrast text elements

The interface is anchored with the Primary Blue (#1570EF) as the main brand colour and it is used in all primary actions and active states. Steel Blue (#9AB6DC) offers a more relaxed complementing color of backgrounds and minor objects without creating distraction. White (FFFFFF) is used to guarantee uncluttered content sections, whereas Muted Slate (#5E6282) is used to deal with the secondary content and labels, where contrast should be guaranteed without interfering with the main content. The color black (#000000) is used on the most important headings to have a high typography hierarchy in all screens.

Combined, these design decisions along with colour choices make sure that the TeamTrack low-fidelity prototype is not merely a working blueprint to build upon, but a solid groundwork for an intuitive and accessible user experience that is coherent visually. The detailed Figma artefacts, such as the links to interacting prototypes and annotated screen exports, are presented in the Appendix.

7.4 Feedback and Improvements Made

Survey Summary:

After the development of the basic prototype of UI we had a short-term feedback survey to get suggestions on how to make it better and also to determine whether the design direction is understandable and pleasing to the target audience. The reactions were also received especially among the students who had limited representation of other universities. The respondents rated the overall look of the design, their chances of referring a platform with this kind of UI, and their thoughts of how the design can be better.

Key Findings from Feedback:

The overall responses were very positive. Most of the respondents rated the interface as Good or Excellent and would be Likely or Very Likely to recommend such interface design. It means that the layout is easy to read, the navigation is simple and layout design presents the purpose of the platform in a favorable light. A small group responded more neutrally (e.g. satisfactory or neither likely nor unlikely) and this brings out the fact that even though the UI is generally well adopted, it still needs some polishing to suit the broader taste and to be more visually fined.

Improvements Identified and Applied/Planned:

One was of the palette refinement repetition. The recommendations that the respondents made include that colour scheme ought to be improved and more colours of blue ought to be applied in order to improve the consistency and visual balance. This will be addressed by making the accent colours of the navigation, dashboard cards, and other interactive parts the same and increasing the contrast between the background and content containers to make reading more comfortable and stifle the impression of similarity across screens.

The other similarity was the similarity between the Home, Login and Sign Up pages. According to one of the respondents, these screens look nearly the same except that form items, which might be confusing to new users. To enhance this, an onboarding experience will be visually distinguished through variations in layout structures, perfection of headings and supportive text blocks, and rebalanced spacing of each page to provide a clear step on what is needed on that particular page. Another recommendation that was directly connected to this one was to make the size of the hero image smaller and place more informative text below it to make the landing page more understandable about the value of the platform and not to be overwhelmed by the visuals.

Another improvement that was mentioned in feedback was the improvement of the AI assistant experience. A single respondent suggested the addition of a chat history feature in order to allow revisiting past traffic and keep track of the context. This addition is very pertinent since AI support will aim at supporting teams within a long project scope. The panel of chat history (or a session list) would enable users to see their previous recommendations, to carry on a discussion thread, to prevent repeating questions, enhance its ease of use and practicality of the AI element.

Another comment made by one of the respondents was that the interface is too user friendly. This, even though possibly not bad, means that the UI may appear oversimplified to certain users. The solution to the interface can be achieved through more controls in optional advanced views, collapsible panels, or secondary settings pages instead of complicating it. The minimalistic design is preserved in this strategy, though it provides more degree of depth to the users such as team leaders or instructors who may require higher degrees of control or analysis.

Implementation Priority:

The enhancements will appear as follows, according to their frequency and impact: first of all, the colour palette and the visual consistency will be optimised, secondly, the onboarding pages will be separated in order to avoid confusion, and finally, the AI assistant will be improved with the help of the chat history. Such are the refinements that will be incorporated in the later design to make the

visual aspect more professional and of user of the device without losing the overall simplicity and usability of prototype.

7.5 Conclusion

This section outlined the design and the prototyping which was carried out on UI of teamtrack, how the functional requirements of the system are translated to a feasible and a real system interface. The prototype illustrates how the users would go about the platform to manage collaborative academic projects through the coming up with well-structured layouts, a defined pattern of navigation and space of visual organisation. The interface design is so simplified, understandable, and transparent that users can simply find valuable project information which entails the progress of the project work, contribution statistics, summary of time tracking among other valuable information of the project.

UI strategy concentrates on screen consistency to the extent that they can guarantee that their users know to expect to find relatively similar patterns of navigation and expected interaction throughout the platform. Those redirected to public pages such as the landing page are dedicated to the introduction of the system and the instruction of the user about the initial steps to be followed; the authenticated pages are provided as an application with the permanent navigation and the information systems shown on the dashboard. This separation of onboarding and workspace environments is part of a more smooth user experience and the project teams to be more interested in monitoring progress and organization of work.

The dashboard interface is one of the main elements of the design, and it shows the most important indicators in the form of visual summaries and charts project progress, allocation of work, and time spent. By placing these metrics in the main workspace, the system allows the user to visually comprehend the situation of projects and detect possible problems like the lack of equal distribution of the workload or the unfinished work. Another layer of support introduced by the incorporation of an AI chatbot interface is that a user is set to ask the project to provide insights or recommendations in the form of a conversation, but without any misunderstanding with the program as the primary tool of project management.

Feedback obtained by the usability survey also aids the effectiveness of the existing design direction. The design was mostly rated well by the majority of participants and they suggested that they would recommend a system that uses this type of interface. Simultaneously, the survey also revealed some areas of improvement, namely the colour palette, better distinction between onboarding pages, and possible extension of AI options by adding such features as chat history.

The same knowledge can be applied to provide good instructions regarding the future development of the design and ensure that the final interface will continue to be updated in respect to the requirement of the users.

Altogether, Figma prototype is a visual representation of the required user experience and a potent foundation of the system implementation phase. The design options mentioned in this section give a consistent design in terms of interface that helps in interaction, openness to contribute, and easy accessibility of the system features. As the development progresses, more in-service upgrades will be done, and results of the feedback of the user and the usability validation will be provided in order to ensure that the final platform is user-friendly, has a visually pleasing look, and corresponds to the end-purpose of the project.

8. Testing and Validation (Design-Level)

8.1 Testing Objective and Scope

8.1.1 Objective

In this milestone, we will confirm that our system design is right, complete, and is in line with what the system will be providing. As this is at the level of SAS + CID, our testing is not about running automated scripts or testing run-time performance but is mostly design-level validation.

We will confirm that:

- We still have the right requirements, which can be tested.
- our UI flows support the intended user journeys,
- We can implement and make our API contracts consistent, [19], [20].
- We have our access control expectations (authentication + RBAC), [17], [18], [20].
- And typical cases of errors are expected such that the system is predictable. [19], [20].

This validation makes less work later at the integration stage since we will be assured at an early stage that the blueprint is accurate before we produce more.

8.1.2 Scope of Validation

We will validate our artefacts of design through checking and witnessing design artefacts and checking their correspondence with the anticipated system behavior.

Table 18: Validation Scope and Boundaries

Validation Area	What We Will Validate in This Milestone	What We Will Not Execute Yet
Requirements	Coverage, consistency, and acceptance criteria	Full user acceptance execution with real users
UI / Prototype	Screen flow correctness, navigation, input rules, error states	Large-scale usability testing and UI experiments
API Contracts (CID)	Request/response format, status codes, RBAC rules, error cases	Automated API testing on a deployed environment [19], [20]
Security (Design)	Auth approach, RBAC decisions, validation rules, safe errors	Penetration testing, vulnerability scanning [17], [18], [20]
Performance (Design)	Design choices that support responsiveness (pagination, limits, caching plan)	Load testing, benchmarking, tuning

8.1.3 Validation Approach

To ensure that design readiness is achieved upon this milestone, we will conduct design-level validation in an organized and traceable manner. This validation is geared towards a consistency of requirements, UI flows, and API contracts and also predictable failure handling as well as security expectations.

The design will be validated by us through:

1. Going through requirement groups to ensure that every requirement is addressed by at least one design artefact (UI flow and/or API contract).
2. Using wireframes/prototype screens to walk through important user journeys to verify that the steps, inputs, navigation, and expected results are correct.
3. The API contracts can be validated against the CID tables to ensure their request/response formats, status codes, RBAC rules, and cases of errors are consistent across the board. [19], [20].
4. Enforcing the expectations of RBAC is consistently applied to both the UI controls and the API permissions. That is, restricted actions are prevented at the interface and enforced at the backend. [17], [20].
5. Ensuring that failure modes are specified in clear-cut situations to ensure the system can be predicted, safe, and consistent in the event of failure. [19], [20].

8.1.4 Validation Deliverables (Design Evidence)

To show design readiness, we will create systematic evidence in the form of tables and diagrams that indicate that validation has been undertaken, and results have been considered.

Table 20: Design-Level Validation Outputs

Output	Purpose
Requirements Validation Table (RTM-style)	Proves requirements are covered by UI/API decisions
UI Walkthrough Scenarios Table	Proves the user journeys are complete and logical
API Contract Table	Proves endpoints are defined and implementable
Error Handling Matrix	Proves the system will behave safely and consistently during failures [19], [20]
RBAC Validation Table	Proves restricted actions are protected by role/permission rules [17], [20]

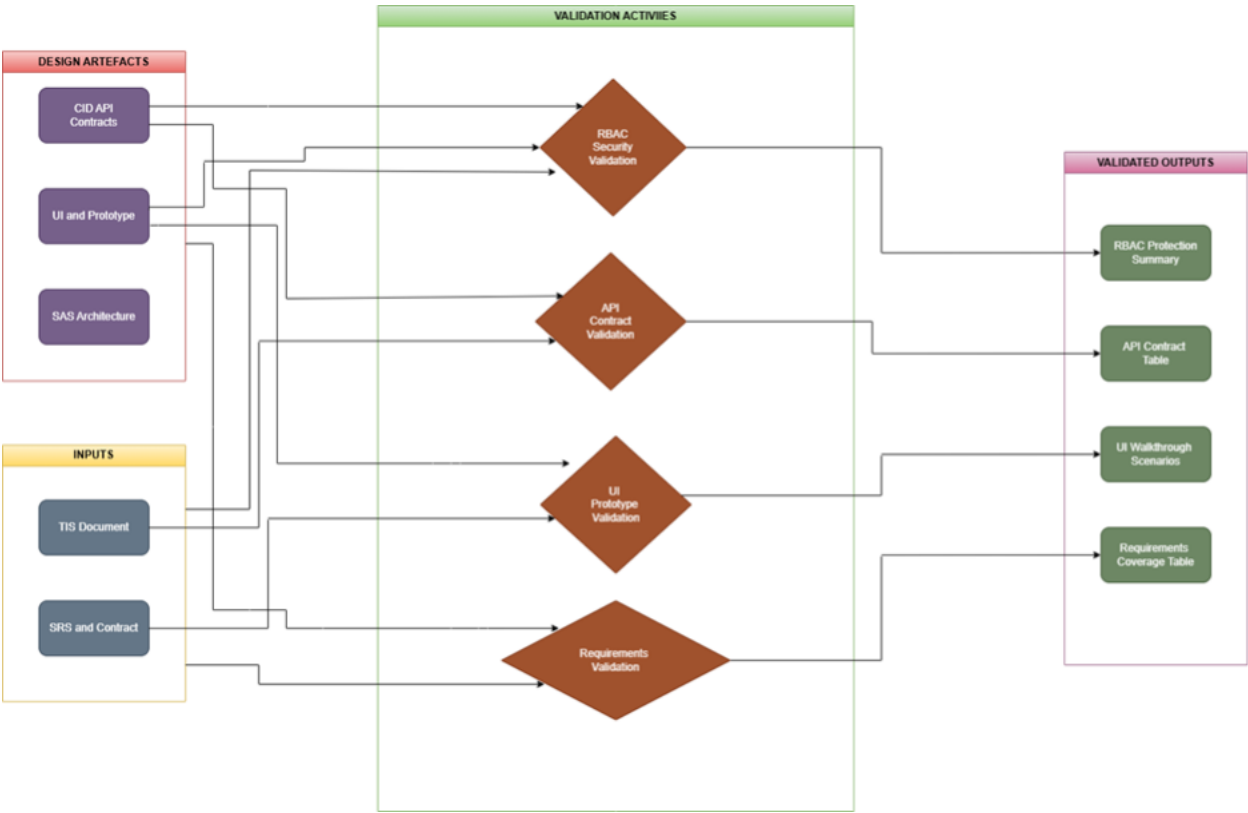


Figure 17: Validation Overview Diagram

Design-Level Validation Overview

This diagram summarizes how Milestone 3 design artefacts (SAS architecture, UI prototype, and CID API contracts) and project inputs (SRS/Contract and TIS) are validated through requirements, UI, API, and RBAC checks, producing documented validation outputs such as the requirements coverage table, UI walkthrough scenarios, API contract table, RBAC summary, and error handling matrix.

8.2 Requirements Validation (Review + Acceptance Criteria)

8.2.1 Validation Purpose

To make sure that requirements are:

- **complete** (there are no missing edge cases or flows),
- **consistent** (no conflict among modules),
- **measurable** (there are acceptance criteria that are testable), and
- **implementable** (the design reflects the desired behavior).

At this milestone we will avert the frequent issues like:

- that is a requirement which lacks a supporting screen flow,
- a screen flow which does not support API contract,
- an API, whose role permissions are unclear, or

features that cannot be checked since the conditions

8.2.1.1 Validation Method

We will validate requirements using a structured review approach. We will (17) group requirements into major feature areas, (18) map each requirement to at least one UI screen flow and at least one planned API endpoint in the CID, (19) confirm that role-based access rules are clear for restricted actions, and (20) define acceptance criteria that can later be reused directly as test cases. Where gaps are identified, we will record the issue and update the relevant design artefacts before further implementation proceeds.

8.2.2 Requirements Review Checklist

We will apply the following checklist when validating each requirement group.

Table 21: Requirements Validation Checklist

Check Item	What We Will Check	Example Evidence
------------	--------------------	------------------

Design coverage	Each requirement is supported by UI and/or API contract	UI journey + endpoint existence
Testability	Clear pass/fail outcome exists	Acceptance criteria statement
Role clarity	Restricted actions specify required roles	RBAC table + API role column
Data feasibility	Required data is identifiable and collectable	Request fields + response fields
Failure handling	Common errors are described	400/401/403/404 coverage [19], [20]

Table 28: Traceability Rules Used in Validation

Rule	What We Will Enforce	Why It Matters
R1	Every functional requirement maps to at least one UI flow	Prevents “requirements without screens”
R2	Every UI flow maps to at least one API endpoint	Prevents “screens without backend support”
R3	Restricted actions must specify role/permission	Prevents access control ambiguity [17], [20]
R4	Each requirement includes at least one negative case	Ensures predictable failure handling [19], [20]
R5	Acceptance criteria are written as pass/fail checks	Converts design into future test cases

8.2.3 Acceptance Criteria Guidelines (Design-Level)

Acceptance criteria will be written in such a way that they can be used in future as test cases without re-interpretation. Each criterion will include:

- **Actor** (who performs it and role),
- **Action** (what they do),
- **Expected result** (what must happen), and
- What will happen upon failure (what needs to occur).

Example acceptance criteria style

- “Upon a valid user logging in we will reply with a token/session and redirect to the users dashboard. In case of invalid credentials we will send out 401 and display a safe error message without exposing sensitive information”. [17], [18], [19], [20]

Additional acceptance criteria examples

- A team leader will be permitted to give tasks and deadlines under a project. In case a non-leader tries the same we will prevent the UI action and the API layer by sending back 403. [17], [19], [20].
- We will then make time entries against a project/task and aggregate summatives to contribution analytics. We will also send 400 and provide clear field-level instructions, in case missing or invalid required fields are found. [19], [20].
- The AI insights generated by us will be in the form of project activity and time tracking as recommendations and task history. The tasks and roles will not be automatically changed on the basis of AI output. In the case of unavailability of the AI service, we will fallback safely and the user can continue working on the project normally as well. [21].

8.2.4 Requirements-to-Design Validation Summary

The following is the requirements to design validation summary we expect to keep and continuously grow as requirements are resolved. As shown in this table, UI and API design choices support every key requirement area, such as access control and expectations of error-handling.

Table 23: Requirements-to-Design Validation Summary (Design-Level)

Requirement Area	UI Coverage (Wireframes/F low)	API Coverage (CID)	Acceptance Criteria (Design-Level)	Key Error / Negative Case	Status
Authentication & Account Access	Login screen, role-based dashboard landing	POST /auth/login, POST /auth/logout	Valid login returns token/session and redirects to correct dashboard; invalid login returns 401 with safe message	Invalid credentials → 401; missing fields → 400 [18], [19], [20]	Covered
Role-Based	Leader-only controls; member-only	Role-protected endpoints across modules	Unauthorised actions are blocked in UI	Member attempts leader	Covered

Access (RBAC)	project views; restricted admin settings		and denied by API with 403	action → 403 [17], [19], [20]	
Project Setup & Membership	Create project, invite/join screens, member list	POST /projects, GET /projects, POST /projects/{id}/members	Users can create projects and manage membership according to role; only authorised users can manage roles	Project not found → 404; forbidden role change → 403 [19], [20]	Covered
Task Management	Task list/board, create/edit task, assign, update status	GET /tasks, POST /tasks, PATCH /tasks/{id}	Tasks can be created, assigned, and updated; invalid status/state changes are blocked	Invalid update → 400; task not found → 404 [19], [20]	Covered
Time Tracking & Timesheets	Timer/manual entry, timesheet summary screens	POST /time-entries, GET /time-entries/me, GET /timesheets/summary	Time entries are recorded and reflected in summaries; invalid fields return 400	Missing/invalid fields → 400 [19], [20]	Covered
Contribution Analytics	Analytics dashboard, filters and summaries	GET /analytics/contributions	System shows contribution summaries based on tasks, activity, and time; restricted views follow RBAC	Unauthorised access → 403 [17], [20]	Covered

Activity Logging / Audit Trail	Activity feed view (project timeline)	GET /activity, logging hooks/events	Key actions create activity entries with actor + timestamp; entries are visible per permissions	No permission → 403 [17], [20]	Planned/Covered (as per design)
Team Chat (Real-Time)	Chat screen within project context	GET /projects/{id}/messages, POST /projects/{id}/messages	Messages are posted and retrieved within correct project; non-members cannot access	Non-member access → 403; project not found → 404 [19], [20]	Covered
AI-Assisted Insights	AI insights panel, recommendation cards, “why” explanations, fallback state	POST /ai/insights or GET /ai/insights?project_id=	AI generates recommendations using project data; insights are suggestions only; failures return safe fallback	No data → 200 with “insufficient data”; AI down → 503 fallback [19], [21]	Covered
Data Integrity	Form validation states across modules	Validation rules across endpoints	Invalid input returns 400 with field guidance; data consistency is preserved	Invalid fields → 400 [19], [20]	Covered

8.2.5 Review Findings and Actions

In validation, we shall note gaps that are detected between requirements and design artefacts. Such gaps can be missing error states, ambiguous permissions in any role, inconsistent names in UI and API, or missing flows. When we find such gaps, we will refreeze the corresponding UI flow,

API contract or acceptance criteria and then the design will always be consistent and implementation ready.

Table 24: Requirements Review Findings

Finding	Impact	Action We Will Take
Leader vs member permissions unclear for some actions	RBAC risk and inconsistent behaviour	We will document permissions in CID and reflect them in UI controls [17], [20]
Contribution analytics data sources not explicit	Dashboard may not match tracked metrics	We will define which logs/time entries feed each metric
AI insight conditions need clarity (e.g., inactivity threshold)	AI outputs may look arbitrary	We will define rule thresholds and return “insufficient data” where applicable [21]
Error states inconsistent across modules	Poor UX and integration confusion	We will standardise error format and UI error messaging [19], [20]

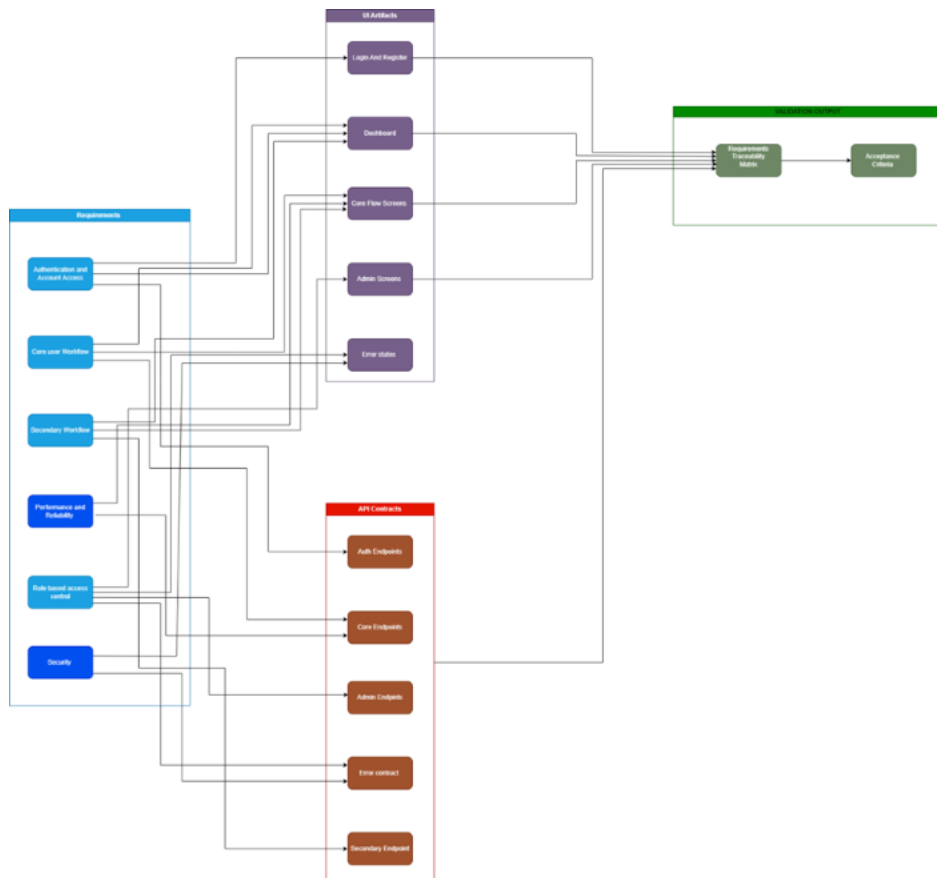


Figure 18 : Requirements Traceability Diagram

Note: Role-based access control (RBAC) and error-handling consistency are validated through the API contract review and RBAC protection tables (Table 30–Table 32). Design-level performance and reliability considerations are validated through architecture decisions such as pagination, limits, and consistent error responses, rather than runtime benchmarking at this stage.

8.3 UI / Prototype Validation (Walkthrough + Feedback)

8.3.1 UI Validation Goal

To test the UI design, we will make sure that the users are able to complete the most important processes of the work without any hassles, in a clear and foreseeable way. At this point, the validation of UI is done by means of structured walkthroughs with wireframes / prototype screens (design-level evidence) as opposed to usability experimentation or large-scale user validation. [19], [20].

UI validation will confirm:

- navigation consistency (there are no dead ends and there are return paths),
- accurate sequencing of screens (none are skipped in critical processes),

- understandable input requirements (forms, constraints, and guidance text),
- consistent and understandable UI states (success, loading, empty-state, and errors), [19], [4].
- role-based visibility (Leader vs Member vs Admin access and controls), [17], [20] and
- AI experience quality (insights are in the form of recommendations with explanation and safe fallback states). [21].

The problems, where detected, will be recorded in the table in the walkthrough findings, and the wireframes/prototype will be revised to align itself with the requirements and API contracts further before going ahead with the implementation.

8.3.1.1 Walkthrough Method (Design-Level)

Our walkthrough sessions will be done by choosing high-priority user journeys and going through each step screen by screen. In each case, we shall establish that:

1. the user will be able to access the goal without discontinuities,
2. the UI is clear in giving system feedback (success/loading/error).
3. actions that are restricted are not subjected to unauthorized roles, [17], [20] and
4. AI results are well-articulated in the form of recommendations, and not in the form of automatic decisions. [21].

Where problems are found, we shall document the results and make the wireframes/ prototype more specific to the problem and then move on to the implementation process.

8.3.2 Walkthrough Scenarios

We will use scenario-based walkthroughs to validate the most important user journeys across roles.

Table 25: UI Walkthrough Scenarios

Scenario ID	User Journey	What We Will Validate	Expected Result
UI-1	Login → Role Check → Dashboard	Correct landing by role + session behaviour	Member lands on Member Dashboard; Leader/Admin lands on Leader/Admin Dashboard
UI-2	Browse/Search Projects → Select Project → Project Dashboard	Navigation + context preservation	Project context is maintained (project name, members, summary visible)

UI-3	Create Task → Assign Member → Set Deadline	Form validation + role rules	Task created successfully and appears in task list/board [20]
UI-4	Update Task Status → Add Comment/Update	State changes + feedback	Updates save correctly; activity updates reflect change
UI-5	Start Timer / Add Manual Time → Timesheet Summary	Time tracking usability + correctness	Time entry saves and appears in summary calculations
UI-6	View Contribution Analytics → Filters/Range	Insight clarity + filtering	Analytics page shows metrics and updates based on filters
UI-7	Team Chat → Send Message → View Thread	Real-time flow + correct project linkage	Message posts under correct project and is visible to members
UI-8	Permissions Test (Member attempts Leader/Admin action)	UI hiding + denial handling	Restricted actions hidden; if accessed, clear “Access Denied” shown [17], [20]
UI-9	AI Insights → View Recommendations → “Why” Explanation	AI transparency + explanation	Insights shown as recommendations with explanation and no auto-changes [21]
UI-10	AI Unavailable / Low Data → Fallback State	Safe AI failure handling	UI displays fallback (“insufficient data” or “service unavailable”) without blocking normal work [21]
UI-11	Error State Validation (invalid form + not found)	Consistent error UI	Field errors shown for invalid input; not found screens shown where appropriate [19], [20]

8.3.3 UI Validation Checklist

To keep walkthroughs consistent, we will apply the following checklist to each scenario.

Table 26: UI Validation Checklist

Check	What We Will Look For
Flow completeness	All steps exist from start to finish with no missing screens
Navigation	Clear back/home paths and consistent project context

Input rules	Required fields, formats, and constraints are visible and understandable
Feedback states	Success, loading, empty-state, and error-state screens exist
Role handling	Restricted actions are hidden/blocked and clearly explained [17], [20]
Accessibility basics	Readable layout, consistent controls, and clear labels
AI clarity	AI outputs are labelled as recommendations with “why” explanations [21]
AI safety/fallback	Insufficient data/unavailable service handled without disrupting core workflows [21]

Table 27: UI State Validation (What We Will Confirm Appears in the UI)

UI State Type	Example Where It Appears	What We Will Validate
Loading state	Dashboard, analytics, chat history	Clear loading indicator and no frozen UI
Empty state	No tasks, no chat messages, no analytics data	Clear message + recommended next action
Validation error	Create task, login, time entry	Field-level guidance, not generic failure [20]
Access denied	Member tries leader-only action	Clear explanation and safe navigation back [17], [20]
Not found	Invalid project/task link	Friendly “not found” screen and return option [19], [20]
AI fallback	AI unavailable or insufficient data	Recommendations replaced with safe fallback text [21]

8.3.4 Feedback and Improvements

We will document feedbacks and enhancements as indications of the iterative refinement as opposed to a single design. The feedback will be taken in the form of internal team reviews and supervisor discussion points which will be translated into refinements on the UI.

Table 28: UI Feedback Log

Feedback Theme	Observation	What We Will Improve
Navigation clarity	Users may not know where they are within a project	We will add breadcrumbs and consistent “Project Home” navigation
Error messaging	Some screens may not explain errors clearly	We will standardise error messages and add field-level hints [19], [20]
Role visibility	Some actions may appear available to all users	We will adjust visibility and add clear permission messages [19], [20]
Flow efficiency	Too many steps for common actions	We will streamline workflows using modals/shortcuts where appropriate
Analytics readability	Metrics may look confusing without context	We will add labels/tooltips and simple explanations for metrics
AI transparency	Users may not trust AI outputs	We will show “why” explanations and present insights as recommendations only [21]
AI fallback handling	AI downtime/low data could confuse users	We will add “insufficient data” and “service unavailable” fallback states [21]

8.3.5 UI Artefacts and Diagram Reference

We will include a screen flow diagram that visually represents the main navigation paths validated in Table 25. This ensures our walkthrough scenarios are traceable to a single source of truth for UI flow.

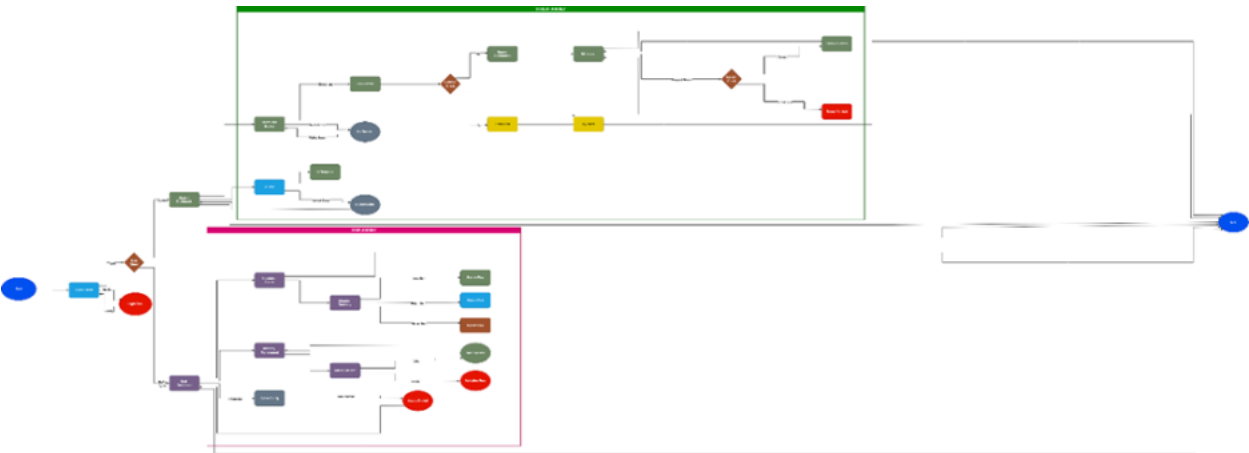


Figure 18 : Screen Flow Diagram (User Journey Walkthrough)

summarizes the primary navigation paths across roles and modules; detailed step-by-step validation is documented in the walkthrough scenarios (Table 25) and UI state checklist (Table 26).

8.4 API Contract Validation (Request/Response + Error Cases)

8.4.1 Purpose

We will test API contracts to make sure that the frontend and the backend can be joined without misunderstanding. At this point, it is essential to have clearly defined, consistent, and role/permission rules endpoint so that the implementation and integration work can be carried out with minimum rework. [19], [20].

The validation of API contract will assure:

- There is a consistency in the endpoint naming and module grouping,
- request fields and types of data are easily understandable and implementable,
- All modules have the same response structures,
- it contains standard status codes,
- and failure cases are characterized in such a way that failures become predictable and safe. [19], [20].

TeamTrack finds this validation significant since various modules (projects, tasks, time tracking, chat, analytics, AI insights) are required to interact and use similar identity/permission logic. [18], [19], [20].

8.4.1.1 Validation Method (Design-Level)

We will check API contracts through a systematic CID check procedure. For each endpoint, we will:

1. confirm the endpoint supports a real UI journey,
2. RBAC rules are articulate, confirm authentication and RBAC rules are explicit, [17], [18], [20].
3. validate request/response shapes are complete (required vs optional fields),
4. common error cases are given (400/401/403/404/409) [19], [20] and
5. define the naming and response forms in a standard way such that the frontend can apply predictable action. [19].

8.4.2 Contract Standards We Will Follow

We will use the following standards to maintain the consistency of all APIs:

- **Reliable naming:** Endpoints will be named in a straightforward structure, e.g., /projects, /tasks, /time-entries, /analytics, /chat and /ai to make the API structured and simple to comprehend.
- **Authentication enforcement:** Authentication control: the authenticated endpoints will need a valid token/session. [17], [18].
- **RBAC enforcement:** Endpoints with roles will indicate the role they require (e.g. Leader/Admin). [17], [20].
- **Pagination:** List endpoints will be able to operate pagination and filtering where needed, e.g. in case of a task, activity log or message, to make retrieving data more efficient and manageable.
- **Consistent responses:** This will be in a consistent format (e.g. data, message and optional meta in the instance of pagination).
- **Consistent error:** The messages will be the standard status messages and safety messages, which will not disclose any sensitive information. [19], [20].
- **Input validation:** Server side validation of mandatory fields will be carried out with error. feedback that is given to the user as clear as possible in case of a invalid input. [20].

Table 29: API Contract Standards Checklist

Standard	What We Will Enforce	Benefit
S1: Naming consistency	Clear module prefixes and predictable resource paths	Easier frontend integration
S2: Auth required	Token/session required for protected endpoints	Prevents unauthorised access [17], [18]
S3: RBAC required	Leader/Admin-only actions clearly marked	Prevents permission confusion [17], [20]
S4: Pagination	Page + limit + filters for list endpoints	Performance and usability
S5: Response format	Standard data/message/meta style	Predictable client handling
S6: Safe errors	No sensitive details in errors	Better security posture [20]

8.4.3 API Contract Validation Table (CID Summary)

The CID-style validation table which we will use is as follows. The endpoint selected is indicative of the fundamental modules of TeamTrack, including authentication, projects/membership, tasks, time tracking, analytics, activity logging, chat, and AI insights. [18], [19], [20].

Table 36: API Contract Validation (CID Summary)

Endpoint	Method	Auth Required	Role Required	Request (Key Fields)	Success Response	Key Error Cases
/auth/login	POST	No	Public	email, password	200 OK (token/session + user + role)	400 invalid format, 401 invalid credentials [17], [18], [19]
/auth/logout	POST	Yes	Any logged-in	token/session	200 OK (logged out)	401 invalid/expired [17], [18]
/projects	GET	Yes	Member/Leader /Admin	filters, pagination	200 OK (project list + meta)	401 unauth [17], [18]
/projects	POST	Yes	Member/Leader /Admin	name, description	201 Created (project object)	400 invalid input [19], [20]
/projects/{id}	GET	Yes	Member/Leader /Admin	id	200 OK (project details)	403 not member, 404 not found [17], [20]
/projects/{id}/members	POST	Yes	Leader/Admin	user_id, role	200 OK (member	403 forbidden, 404 not

					added/updated)	found, 409 conflict [17], [20]
/tasks	GET	Yes	Member/Leader /Admin	project_id, filters, pagination	200 OK (task list + meta)	400 invalid filters, 401 unauth [19]
/tasks	POST	Yes	Member/Leader /Admin	project_id, title, assignee_id, due_date	201 Created (task object)	400 invalid, 403 not member [17], [20]
/tasks/{id}	PATCH	Yes	Member/Leader /Admin	status, title, due_date, assignee_id	200 OK (updated task)	400 invalid state, 403 forbidden, 404 not found [19], [20]
/time-entries	POST	Yes	Member/Leader /Admin	project_id, task_id (optional), minutes, note	201 Created (time entry)	400 invalid, 403 forbidden [17], [20]
/time-entries/me	GET	Yes	Member/Leader /Admin	filters, pagination	200 OK (my entries + meta)	401 unauth [17], [18]
/analytics/contributions	GET	Yes	Member/Leader /Admin (or Leader-only if policy)	project_id, date_range, filters	200 OK (summary metrics)	403 forbidden, 404 not found [17], [20]

/activity	GET	Yes	Member/Leader /Admin	project_id, pagination	200 OK (activity feed + meta)	403 not member, 404 not found [17], [20]
/projects/{id}/me ssages	GET	Yes	Member/Leader /Admin	pagination	200 OK (message list + meta)	403 not member, 404 not found [17], [20]
/projects/{id}/me ssages	POST	Yes	Member/Leader /Admin	message_t ext	201 Created (message object)	400 invalid, 403 not member [19], [20]
/ai/insights	POST	Yes	Member/Leader /Admin	project_id, optional_co ntext	200 OK (recommend ations + “why”)	400 invalid, 403 forbidden, 503 unavailabl e [21]
/ai/insights	GET (<i>optio nal</i>)	Yes	Member/Leader /Admin	project_id	200 OK (latest insights)	404 not found/no ne, 503 unavailabl e [21]

8.4.4 Standard Error Handling Matrix

We will also ensure that every endpoint contains standard cases of errors so that the frontend can be able to respond uniformly and safely to their occurrence. [19], [20].

Table 31: Error Handling Matrix

Status Code	Meaning	When We Will Use It
----------------	---------	---------------------

400	Bad Request	Missing required fields, invalid formats, invalid state transitions [19], [20]
401	Unauthorized	Token missing/expired/invalid [17], [18]
403	Forbidden	Authenticated but lacks permission or is not a project member [17], [20]
404	Not Found	Project/task/resource does not exist or is not accessible [19]
409	Conflict	Duplicate action, invalid transition, rule-based conflict (e.g., role assignment conflict) [19], [20]
500	Internal Server Error	Unexpected backend failure
503	Service Unavailable	External AI service unavailable or temporary outage [21]

8.4.4.1 Standard Error Response Format (Consistency Rule)

To have predictable frontend processing, we will have the same error response format across endpoints (e.g. error code, message, and optional field errors on validation errors). We will make sure that error messages are not dangerous, and they are not exposing any top-secret information about the backend and authentication. [19], [20].

8.4.5 RBAC Contract Validation

We will ensure that RBAC is always implemented when it comes to sensitive operations. This makes sure that our security rules, besides being described, are also represented in the API contract such that unauthorized actions are prevented not only in UI, but also at the API layer. [17], [20].

Table 38: RBAC Protection Validation

Sensitive Operation	Allowed Roles	Expected Behaviour for Other Roles
Add/remove members, assign roles	Leader/Admin	We will return 403 and deny the action [17], [20]
Edit project settings	Leader/Admin	We will return 403 for non-leaders [17], [20]
Create/update tasks across the project	Member/Leader/Admin (policy-based)	If restricted, we will return 403 and block UI controls [17], [20]

View contribution analytics (team-wide)	Leader/Admin (<i>or Member if allowed</i>)	If restricted, we will return 403 and show access message [17], [20]
Moderate/delete chat messages (if supported)	Leader/Admin	We will return 403 for normal members [17], [20]
View another user's private time entries	Leader/Admin (policy-based)	We will return 403/404 depending on privacy policy [17], [20]
Trigger AI insights generation	Member/Leader/Admin	If not member, we will return 403; if service down, return 503 [17], [20], [21]

8.4.6 Diagram Reference

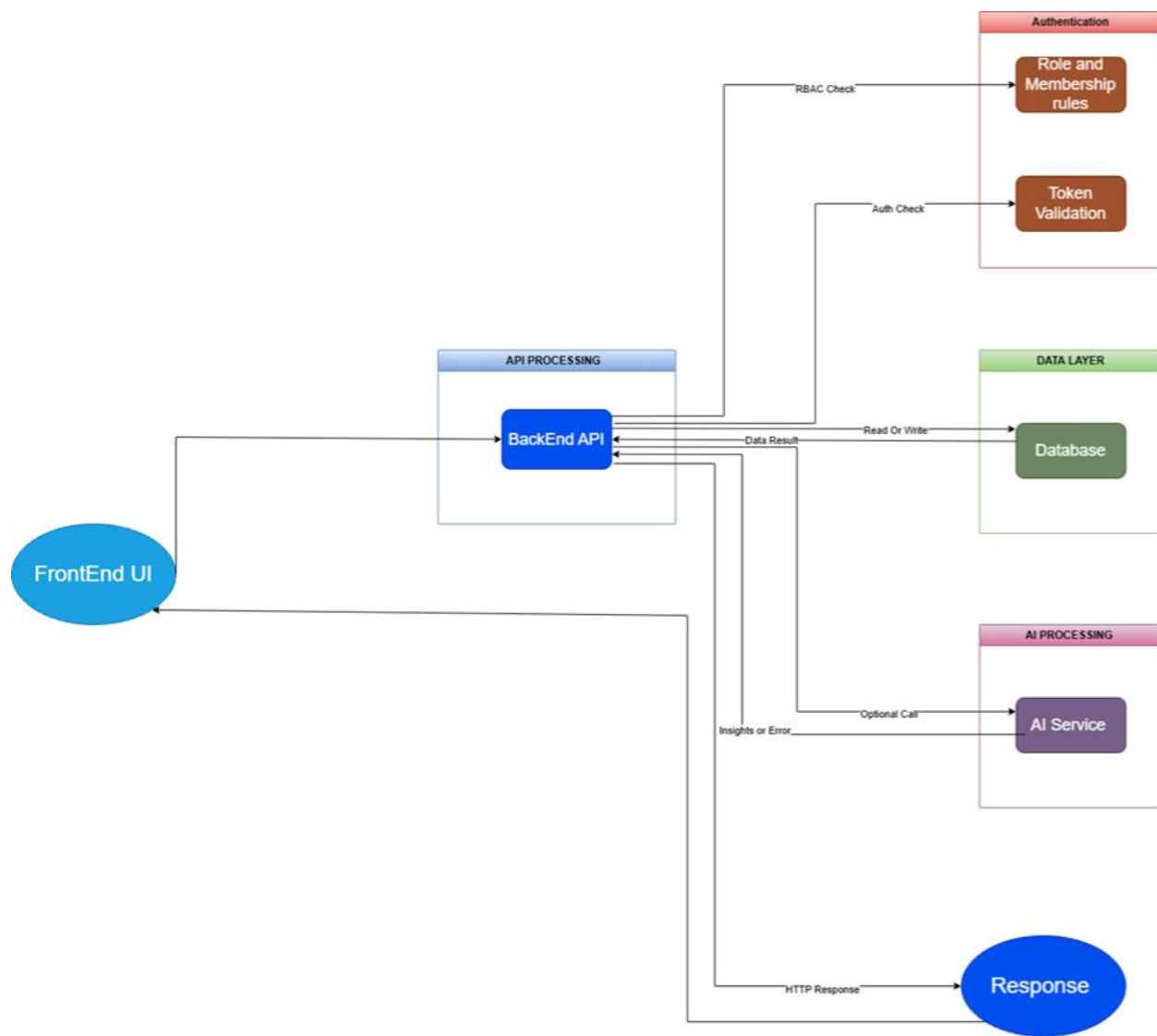


Figure 20: API Interaction Diagram

API Interaction Overview. This diagram illustrates how the Frontend UI communicates with the Backend API, including authentication and RBAC checks, database read/write operations, and optional AI service calls. The backend returns a standardised HTTP response to ensure consistent handling of success and error cases across modules.

8.5 Database Validation (Entity Relationships + Constraints)

8.5.1 Purpose

We test the database design, in such a way that the entity relationships, the structural construction and data integrity rules are competent enough to assist the functional and non-functional requirements of the system. Such a milestone phase is referred to as database validation; this is accomplished with the rigorous understanding of design level correctness and structural integrity but not with regards to performance benchmarking or index optimisation or issues with production deployments.

Database testing has established that:

- all the necessary entities are present and are directly related to defined system modules,
- relationships are a correct representation of real-life interactions (projects, tasks, users, time tracking, chat, AI insights),
- referential integrity is formally enforced by foreign key relationships,
- constraints eliminate invalid, duplicate or inconsistent data states, and
- stored data relationships are structurally correct as far as role-based access assumptions are concerned.

This verification ensures the full consistency between the backend logic, API contracts and the database schema and gives the impression that the database schema is wholly valid as well as internally consistent before any additional implementation takes place. [19], [20].

8.5.1.1 Validation Method (Design-Level)

We validate the database schema using a structured and traceable review process:

1. Each significant requirement area is layered to 1 or more database entities.
2. Foreign key relations are verified to create logical connections (e.g., task to project to user).
3. Primary keys and uniqueness constraints are verified to avoid the conflict of duplication.
4. Optional and required fields are matched against rules of contract validation on API.

5. The structures (role assignments, ownership, membership mappings) related to the RBAC concept are validated to enable secure enforcement at query and application levels. [17], [20].

In case of any inconsistencies found, the schema definition and the ERD will be adjusted and then the process continues further with implementation being reconsidered to be structurally correct.

8.5.2 Core Entity Validation

The following core entities exist to support TeamTrack’s functional scope:

- Users
- Projects
- ProjectMembership
- Tasks
- TimeEntries
- ActivityLogs
- ChatMessages
- AllInsights
- ContributionAnalytics (derived or computed view)

These structures denote the lowest and adequate level of information necessary to facilitate authentication, collaboration, or structured tasks and time tracking, contribution analytics, audit log, communication, and AI-based recommendations.

Table 35: Core Entity Validation Summary

Entity	Purpose	Key Relationships	Key Constraints
Users	Stores user identity and system-level role	Linked to ProjectMembership, Tasks, TimeEntries, ActivityLogs	Unique email; role field required
Projects	Represents a collaborative workspace	Linked to Users (via membership), Tasks, ChatMessages	Name required; owner must exist
ProjectMembership	Maps users to projects	FK → Users, FK → Projects	Unique (user_id + project_id)
Tasks	Stores project work items	FK → Projects, FK → Users (assignee)	Title required; valid status values only
TimeEntries	Tracks time spent on tasks/projects	FK → Users, FK → Projects, FK → Tasks (optional)	Minutes > 0; valid project

			membership required
ActivityLogs	Records system actions	FK → Users, FK → Projects	Timestamp required
ChatMessages	Stores team communication	FK → Projects, FK → Users	Message text required
AllInsights	Stores generated AI recommendations	FK → Projects	Must include explanation field
ContributionAnalytics	Computed summary view	Derived from Tasks + TimeEntries + ActivityLogs	Read-only derived structure

8.5.3 Relationship Validation

We confirm that relationships enforce correct system logic and prevent invalid states.

1. User ↔ Project (Many-to-Many)

The implementation through Project Membership table.

- A user can be a member of several projects.
- There can be more than one user in a project.
- Status Leader/Admin/Member is determined by the role field within membership.

This framework directly aids in enforcing RBAC at application and database layer. [17], [20].

2. Project → Tasks (One-to-Many)

- There is no task that is assigned to more than one project.
- Within a project, there can be several tasks.

Foreign key:

tasks.project_id → projects.id

This avoids orphaning tasks as well as maintaining task visibility within project scope and logical bounds.

3. User → TimeEntries (One-to-Many)

- Every entry is assigned to a single user.
- A user can have more than one entry.

Foreign key:

time_entries.user_id → users.id

Also, time records are compared to membership of a project to prevent cross-project data contamination.

4. Project → ChatMessages (One-to-Many)

- Messages should be part of a given project.
- Messages cannot be accessed or posted by non-members.

Foreign key:

messages.project_id → projects.id

This makes sure that communication is kept within the project's scope.

5. Project → AllInsights (One-to-Many)

- AI insights are project based.
 - Every insight has a field of explanation and time of generation.
 - The insights do not necessarily change the records of the tasks, role, and membership. [5].
- This maintains stability of the system and is in line with the principles of responsible AI governance.

8.5.4 Constraint Validation

Constraints ensure data integrity, making sure it does not enter an invalid state.

Table 34: Constraint Validation Matrix

Constraint Type	Example	Why It Matters
Primary Key	id fields (UUID or integer)	Unique entity identification
Unique Constraint	users.email	Prevent duplicate accounts
Foreign Key	task.project_id	Prevent orphan records
Check Constraint	time_entries.minutes > 0	Prevent invalid time logging
Enum/Status Constraint	task.status IN (todo, in_progress, done)	Prevent invalid task states
Composite Unique	(user_id, project_id) in membership	Prevent duplicate memberships
Not Null	required fields (title, project_id, role)	Enforce required data

These limitations go hand in hand with predictable API behaviour and standardised error responses. [3], [4].

8.5.5 RBAC Data-Level Validation

We confirm that database structure fully supports RBAC enforcement logic.

Table 35: RBAC Data Support Validation

Sensitive Operation	Required Data Structure	Validation Approach
Assign project roles	Role field in ProjectMembership	Role stored and validated at query level
Edit project settings	Project owner/leader role	Only authorised roles allowed
View analytics	Membership + role validation	Leader-only if policy restricts
Delete chat message	Membership + role	Restrict to Leader/Admin
Trigger AI insight	Valid project membership	Non-members denied

All these characteristics allow auditing and traceability to be available, and recovery predictable. [3], [4].

8.5.6 AI Data Validation Considerations

Given the inclusion of AI-assisted insights, we validate:

- Table AllInsights contains the field explanation/why of transparency. [21].
- The insights are recorded in the form of recommendations only.
- Outputs of AI do not directly transform task, membership, and role data.
- In the absence of AI service, there will be no database to write anomalies or incompleteness.

This is in line with responsible system design of AI and safe failure management. [21].

8.5.7 Data Integrity & Consistency Rules

Other structural protection measures are:

- Cascading deletes are tediously managed (e.g. deleting a project can cascade tasks and chat messages, but not user records).
- Soft-delete policy can be implemented in which case audit preservation needs some form of retention (e.g. activity logs).
- Raw data regarding time and tasks are not overwritten or mutated into derived analytics data.

These characteristics enable auditing and traceability to be accessible besides the recovery to be predictable. [19], [20].

9. Agile Development and Implementation Plan

9.1 Development Approach (Iterations / Sprints)

Development of TeamTrack will take an iterative form of Agile development whereby the system shall be provided in small bits so that progress can be checked periodically and modifications be done early enough instead of at the end. This is in line with the unit expectation that the work is to be done according to the sprint and every milestone represents the results of the sprint cycle. The implementation process follows Agile principles whereby the essential functionality is first implemented and further developments and refinements as well as the process of constantly reviewing components and taking corrective actions in case of problems during implementation are employed.

In this project, the team will have three (3) sprints in which each sprint takes one week. A short sprint planning session on the first day of each sprint ensures that the sprint goal is confirmed and, together with selecting backlog items, to coordinate the progress made daily (lightweight check-ins), and a short reflection at the end of the sprint review (a sprint) to identify what was good and what should be improved. The transparency and consistent delivery of this sprint format is due to the fact that every week becomes a more visible and testable increment, not leaving integration and validation to the last week.

A product backlog contains work, which is pulled into a sprint backlog every week in a priority and dependencies based manner. Short sprints may not work when the tasks are too large, hence the backlog items are split into small bits, which can actually be finished within the sprint window. Estimation is done with a rough-but-right attitude (relative sizing and effort at the task level) to ensure that the team does not make commitments to too accurate but unreliable plans.

Sprints are also based on modular implementation mindset with frontend, backend, database, AI/analytics, and security work developing in parallel with defined interfaces and areas of

integration. This minimizes system bottlenecks and helps in controlled system development, since every increment is tested/validated, before proceeding to the next increment.

9.2 Module Breakdown and Team Allocation

TeamTrack is structured into a range of core modules that represent the key functionality of the system: task management, contextual communication, time tracking + contribution analytics, and an AI-assisted element of the generation of insights. The stack (Nuxt.js frontend, FastAPI backend, PostgreSQL storage, Firehose-style event logging, JWT-based authentication, and the LLaMA 3.1 AI service) supports these modules. The module ownership assigns the work needed to develop the module in order to have tasks run parallel during each sprint and have a clear responsibility on the integration points.

9.2.1 Module breakdown (what the team is building)

9.2.1.1 Frontend UI / UX module (Nuxt.js pages + components)

In this module, the designer discusses all the visual screens (dashboard, tasks views, task details, chat, calendar/milestones, analytics and authentication screens) and shared UI elements and client-side validation/feedback. It ensures that there is smooth navigation and the implementation of role based visibility in the interface level.

9.2.1.2 FastAPI-based backend API + business logic module

This module contains the REST API operations of authentication, creating / joining a project/workspace, CRUD on tasks, and chat operations, time logging, analytics queries, and any integration endpoints that the AI assistant uses. The choice of FastAPI is due to the high-performance API routing, validation, and the resolution of the separation between the routing and business logic and database operations.

9.2.1.3 Database and transaction data module (PostgreSQL)

The basic entities of the system (users, projects, tasks, time logs, activity logs) are stored in this module with relational integrity in such a way that the contribution evidence is credible. PostgreSQL has been chosen because of its ACID compliance, constraints and support of indexing which directly tolerate the requirement of integrity and audit trail.

9.2.1.4 Firehose-style logging/high frequency activity logging

It is a module that manages common events (task updates, time tracking events, activity logs, interaction events) in a fashion that enables analytics without impacting on the main operations. Logging transactional information, and the logs with high volumes separately, facilitates scalability

and processing of logs.

9.2.1.5 Contribution analytics + dashboard module

In this module, the data stored and the logs are changed into the visibility of who did what by producing analytics outputs on the team leaders and the read-only reviewers. The analytics and time tracking package is one of the fundamental system features that are meant to promote fairness and accountability.

9.2.1.6 AI-assisted insights module (LLaMA 3.1 service)

The module offers the workload analysis, detection of inactivity, natural language query responses and summaries of contribution explanation. The AI is framed as a decision support and does not in any way automate the grading decisions.

9.2.1.7 Access control module (RBAC + validation) / security.

The module imposes secure authentication, role-based access control, secure session management and input validation throughout the application. These controls restrict the project data and only the authorised users have access to the protected operations.

9.2.2 The allocation of teams (who owns what)

9.2.2.1 Hashir (Team Manager)- Backend Operations + Project Manager

Has sprint coordination, backlog prioritisation, large-scale backend planning/integration, so that core API and module interfaces are consistent between sprints.

9.2.2.2 Omar Backend Engineer

Has implementation support of the backend API and business logic (authentication endpoints, project/task logic, logging endpoints and integration stability), collaborates with team lead on the backend operations.

9.2.2.3 Siraj- Frontend Engineer

Has performed frontend implementation in Nuxt.js, including routing, key screens, state of the UI, and integrating back-end endpoints.

9.2.2.4 Maria- UI/UX Designer and Implementation.

Responsible of UI/UX design fidelity and implementation of frontend components/pages and is responsible to make sure that the navigation, page layout, and interaction flows are in line with the planned user journeys.

9.2.2.5 Zernish- Data and Security Analyst.

Has database/data integrity (entity constraints, storage decisions, retention alignment) and security /risk-oriented implementation support, such as validation or access control tests.

This breakdown permits the co-locating frontend, backend and data/security tasks on the three one-week sprints without giving up time to integration, and takes integration ownership to avoid late merge conflicts or incompatible interfaces.

9.3 Timeline and Deliverables

This section will detail the intended timeline and significant deliverables in the proposed project. As the system is being developed as a part of a capstone project, one should adhere to a structured schedule that will make sure that every step will be performed in the logical sequence and within a reasonable amount of academic time. Clarity of the timeline makes the team work together in a coordinated manner, tracking the activities, the dependencies among activities, and minimizing chances of delays. It also gives a visibility of the way the project is likely to proceed out of the initial planning and requirement analysis to design, prototyping, implementation, testing and final submission.

The schedule of this project has been structured to indicate a development cycle. The project is not viewed as a large task but rather the work is broken into small manageable stages with each stage producing a particular output to the total development of the system. This enables the team to review the progress periodically, take into consideration feedback on clients and refine the solution as the project progresses. Timeline also allows the team to have improved allocation of roles since various deliverables can be allocated depending on the ownership of sections, technical duties, and milestones.

The time frame of the project is supposed to facilitate the following:

- Make sure to make sure that important academic milestones are achieved within the required time.
- Divide the project into smaller stages that can be monitored.
- Coordinate technical tasks with submission of reports and client response.
- Give a methodical journey between the requirements collection and the eventual analysis.
- Assist the team in tracking the relationships between design, development and testing processes.

Minimize chances of hasty work towards meeting deadlines of submission.

The bulk products of the project are allocated to the various stages such that the documentation and the system development proceed in tandem. The above deliverables are not restricted to written reports alone, but also design artefacts, diagrams, prototype screens, technical planning products, and materials related to the implementation. The reason behind this is that the capstone project demands both project management and technical system design evidence.

The following are the expected deliverables:

- Project proposal and preliminary planning documents, such as preliminary definition and scope clarification of a project.
- Deliverables that are related to requirements like problem definition, stakeholder understanding, and system objectives.
- TIS milestone deliverables such as project planning project methodology and initial technical direction.

SAS report deliverables such as architecture design, module definition, database planning, integration design and security considerations.

- Designs artefacts related to UI/UX, including low-fidelity wireframes, high-fidelity screens of a prototype, and structure.
- Component and interaction diagrams, such as architecture diagrams, CID diagrams and planned system flow representations.
- Validation and testing plans, such as acceptance criteria, walkthrough-based validation and design-level review activity.
- Meeting minutes, communication documents that record team organization and interaction with the client during the project.
- Final implementation and demonstration delivery, such as the created prototype, the supporting explanation, and final report material.

In order to make sure that such deliverables are created successfully, the project schedule must take a chronological order where the preliminary work is done and followed by the technical work in detail. It could be used as an example, that requirements and system scope need to be comprehended before architecture design can be completed, and architecture design needs to be specified before implementation and validation can be conducted appropriately. In the same manner, prototyping and interface planning needs to be done early enough so as to facilitate feedbacks and refinements prior to the actual development processes. This sequence will help in keeping consistency throughout the project and later outputs will be grounded on well documented previous decisions.

The basic project schedule can be seen as follows:

Phase 1: Requirement understanding and Project initiation.

Emphasize on the background research, problem definition, understanding of the stakeholders, scope definition, and project goals.

Phase 2 Planning and technical investigation.

Emphasize the choice of methodology, first technology choices, feasibility debate, and preparation of milestones.

Phase 3: Software architecture specification.

Concentrate on system architecture, component identification, and integration planning, database design, UI design and interaction modelling.

Phase 4: Preparation of prototype and development.

Concentrate on the design improvement, planning implementation activities, interfaces preparation, and technical direction verification.

Phase 5 Implementation and testing.

Emphasize the prototype construction, verification of functionality expected, user flows and technical consistency. This is a stage during which the final version is refined and submitted.

Time is also significant in team responsibility. The association of deliverables and certain steps and tasks allows the team to monitor better what has been done, what is outstanding and where further efforts might be required. It comes in handy particularly in a group academic project whereby the effort is shared among many members and coordination is required to maintain the quality of work output. It also aids communication with the client, since the progress updates can be associated with specific deliverables instead of the vague descriptions of the status.

In general, the timeline and deliverables plan offers a systematic way of handling the project to the end. It makes sure that the process of development is organized, transparent and in accordance with the academic expectations. Having the key stages of work outlined and the results that could be obtained in each stage, the team stands better chances to work with time efficiently, organize the tasks, and provide a complete and properly documented system.

9.4 Tools and Collaboration (Trello, Meetings, WhatsApp)

To ensure that TeamTrack remains on track throughout three one-week sprints, the team embraced a lean collaboration model which focused on visibility, quick communications and ownership. Trello (task tracking and accountability), Google Meet (structured discussions and decision-making), and a WhatsApp group (quick coordination between meetings), were the primary tools. Such combination made a work always synchronised even in case tasks were divided between frontend, backend, UI/UX and data/security tasks.

9.4.1 Trello (task tracking and synchronisation)

The core of the work organisation and control was a Trello board. The board used a Kanban-based flow where the columns had been arranged to indicate a regular lifecycle of the task: Backlog, To Do, In Progress, Review/Validation and Done. This enabled the team to monitor what was yet to be done, what was being done and what had to be reviewed before it could be finalised. Notably, assignments were not considered as being finished just because they were written, they needed to be taken through the review/validation processes before they would be considered as having been done.

Better sprint planning was also supported by the board. Items with high priority were transferred out of Backlog into To Do at the beginning of each week sprint so the sprint backlog could be seen. Tasks that were blocked by dependencies (e.g. one section required the information available in another section) remained in Backlog until the dependency was addressed, which stopped the team wasting time to get work started that could not be completed.

The team applied labels (colour coding) to determine who was responsible on a task and to physically distinguish between work areas to enhance accountability. There was an owner assigned to each card (according to section/module responsibility) which meant that everyone knew who was leading which piece of work and who to consult in case of the need to change anything. This minimized the duplication (two individuals accidentally designing the same thing) and facilitated the progress tracking of the work during meetings.

9.4.2 Meetings (Google Meet planning and progress)

The team relied on Google meet to collaborate formally, primarily planning, sprint progress, misunderstandings and finalisation of decision that impacted on several members of the team. It was particularly notable that meetings were necessary in those cases when tasks were cross-dependent (e.g. frontend required to confirm API payloads, or documentation sections required to be aligned on terminology). During the meetings, the team would go through Trello cards, talk about blockers, and make decisions on the next steps so that the work can be followed up and properly distributed.

To make meetings effective, the team used the following format: a brief progress round (“what is done / what is next / what is blocked”), review of high-priority Trello cards, and a final review of what

needs to be done before the next check-in. This contributed to consistent improvement in the short one-week sprints in which the timing factor can easily affect the timeline.

9.4.3 WhatsApp (fast daily coordination)

In-between meetings, a WhatsApp group was employed to coordinate the rapid coordination. It helped with supporting short updates like confirming small decisions, asking quick questions (e.g., is this part of the wording right?), informing others of changes, and marking blockers early so that they could be dealt with before they slowed down the sprint. This assisted the team to avoid having to wait to the next scheduled meeting to make some few clarifications.

9.4.4 Documentation collaboration approach

The team used shared document workflows as opposed to version control through Git with regard to milestone documentation and report writing. This made the process easy and easy to work on by all members, yet the collaborative editing, consistent formatting and last-minute incorporation of sections were supported. Consolidated coordination was done either by Trello ownership tags or WhatsApp messages that showed a person or a group that was having the main document under editing at that time.

9.4.5 Constraints of practical tools and their management

The group had practical problems whereby tools failed at the time of maximum cooperation. An illustration is that Figma was at times buggy in low-fidelity prototyping, and that Microsoft Word was at times prone to create formatting issues or glitches when multiple users were using the same main report document. To handle this, the team minimized simultaneous editing, applied short editing turns (one person works on the document and the rest do a review), and used Trello/WhatsApp to coordinate changes to ensure that the primary document remained stable. This helped to minimize chances of lost progress or formatting issues that were experienced at last minute when the submission deadlines were near.

9.5 Meeting Minutes

A record of discussions, decisions, and action items that were created during the internal and client interactions was formalized in the form of meeting minutes that were prepared during the whole project. These records were a valuable source of information in project coordination, design planning, communication tracking and documentation consistency. The preservation of meeting minutes also helped the project management process to be more transparent, accountable, and continuous as well as ensuring key updates and agreed actions were well captured to be reviewed in the future.

Purpose of Meeting Documentation

The project was supported by meeting minutes by:

- Documenting significant design and development talks that were related to the system design.
- Capturing the decisions reached in the planning and review exercises.

Following the assigned tasks and team duties.

- Traceability of the client feedback and stakeholder input.
- Facilitating coordination in report writing, design and implementation activities.

9.5.1 Meeting Log

Table 36: Meeting logs

Date	Meeting Title	Participants	Key Focus	Outcome
17/02/26	Distributing Sections for SAS Report	Internal team	Section allocation, system architecture responsibilities, dataset discussion, figure estimates	Initial SAS work distribution finalized
24/02/26	Presenting System Architecture (Client)	Client + team representative	Discussion of system architecture, design suggestions, explanation of system modules	Client feedback on architectural direction recorded
24/02/26	Team Work Follow Up	Internal team	Follow-up on team progress, discussion of client suggestions, finalizing sections	Team aligned on revisions and section completion
4/3/2026	Presenting System Architecture (Client)	Client + team representative	Presentation of proposed prototype, system architecture, ER diagram, and user journey diagrams	Design direction and presented artefacts reviewed with client
5/3/2026	Team Work Follow Up	Internal team	Final document formatting, minor updates, final review	Final document compilation and review completed

Meeting Evidence and Traceability

In order to have accountability and offer verifiable records of project coordination, documentation of all the major meetings was kept and used in all the milestones of the SAS. These meeting reports

facilitated section allocation tracing, client feedback, design meeting, monitoring of progress, and final document review work.

Link Pasted Below (for meeting minutes documents)

[Drive Link](#)

Table 37: Meeting Minutes Summary

Date	Description	Reference
17/02/26	Internal meeting on SAS section distribution, architecture responsibility allocation, and figure planning	Documented meeting record
24/02/26	Client consultation on system architecture, module explanation, and design suggestions	Documented meeting record
24/02/26	Internal team follow-up on progress, client suggestions, and section finalization	Documented meeting record
4/3/2026	Client review of proposed prototype, system architecture, ER diagram, and user journey diagrams	Documented meeting record
5/3/2026	Internal meeting on final formatting, minor updates, and overall document review	Documented meeting record

Key Outcomes from Meetings

The meetings conducted during the SAS milestone resulted in:

- confirmation of section ownership and report responsibilities.
- discussion and refinement of the proposed system architecture.
- incorporation of client suggestions into the design direction.
- review of prototype-related artefacts including ER and user journey diagrams.
- alignment of the team on final formatting, revisions, and document completion.

These meeting notes were important in keeping the milestone structured and coordinated. They ensured that client input was captured, team responsibilities were clearly followed up, and the final SAS document remained aligned with the agreed technical direction of the project.

10. Risks and Mitigation

10.1 Technical Risks

TeamTrack is an online system, based on Nuxt.js frontend, Python/FastAPI backend, PostgreSQL as a transactional data store, a Firehose-style store of high-frequency activity logs, a JWT based

authentication scheme, and an AI component (LLaMA 3.1) to provide contribution insights and summaries. Due to the strong reliance on accurate time monitoring, sound activity records and access control based on the role, the primary technical risks are associated with the work of the system, the correctness of the data, the enforcement of access control and system cross-linking.

Risk 1: API bottlenecks when used in bursty mode

Fast API is built to be very high-performance and asynchronous and request processing is achievable, although TeamTrack is still prone to slowdowns during the busiest times (lots of users updating work, recording time, and refreshing dashboards at the same time). In case of a decrease in response times, users can have delays in saving logs or loading analytics, which can influence trust and usability. Performance testing needs to be performed early to reduce this risk with core endpoints (task update, timelog creation, writing activitylogs) and analytics queries need to be optimised with indexing and pagination where necessary.

Risk 2: Lack of data consistency between the PostgreSQL and Firehose logs

To enhance the performance and analytics power, the architecture isolates transactional data (PostgreSQL) and high-rate logs (Firehose-style logging). Nonetheless, when a task update is successful in PostgreSQL, but the event carrying out the activity does not give an update (or the reverse occurs), the dashboards and audit trails might be out of step with each other, which results in conflict during peer review. Mitigation The useful mitigation is to treat log writes as a unit of work or, at least partially, and to use retry/queue logic to ensure that failed event inserts are recorded without re-creating an entry.

Risk 3: Risk of partial or corrupted TimeLog/ActivityLog records in case of failures

TeamTrack specifically states that updates to Time Log and Activity Log must be atomic to avoid the occurrence of partial logs in case of system and network failure. These records are the main evidence of contribution analytics, and unfinished ones can literally weaken equity and responsibility. The mitigation actions to take include transactional time log creation (must be start/end/duration valid), log failure handling designed to leave a log either fully saved or not saved at all and not half saved.

Risk 4: RBAC enforcement (authorization bugs)

TeamTrack has role-based access control with the instructor having audit access to project metadata and team leaders having write access to project metadata. One example of technical risk is unstable endpoint and UI route permission checks that may reveal data or grant the incorrect roles. This danger becomes greater with the increase in features (analytics, chat, admin tools).

Mitigation must comprise of centralised permission checks (middleware/dependencies), role-based test accounts, and endpoint tests, which assert that all the secured routes reject unauthorised roles.

Risk 5: JWT token leak and poor session lifecycle security

The system has a JWT-based authentication as a fundamental security control. When the tokens are not logged out of the browser in a secure way, stored in the browser in an insecure manner or are not expired, attackers might use a stolen token to gain access to secure resources. Mitigation involves brief token lifetime, safe storage patterns (not unsafe persistence), server-side verification of high-risk operations (e.g. re-check role access to sensitive updates).

Risk 6: Risk of input validation vulnerabilities to injection or unsafe rendering

The type of text that can be entered in TeamTrack includes task, message and search query and AI prompt. The validation features of FastAPI does not ensure the exposure of the system when the sanitisation of various parts of the system is not consistent (backend validation, database constraints, frontend rendering). This may lead to things like unsafe content showages or malicious codes on the record. One of the mitigation measures is to impose validation rules on the backend, escape output on the frontend, and test popular injection patterns during QA.

Risk 7: AI insight quality (hallucination or false summaries)

The AI element is restricted to workload analysis, inactivity, natural language queries, and contribution summary clarifications, and shall not be used to provide grades. In case of inaccuracy of AI outputs or being too certain, team leaders or instructors would interpret the insights of the system wrongly and draw unfair conclusions. Mitigation In the case of AI, it would benefit to label AI results as advisory, display supporting evidence (e.g. which logs/events did an AI summary depend on), and make the raw time/activity history behind the result visible to users.

Risk 8: AI performance and resource limitations

Operating an AI component of the system (or even as a side service) may add latency and demand, particularly when a number of users demand summaries or analytics at the same time. In case API responsiveness is adversely impacted by AI requests, fundamental project-management activities are impaired. One mitigation is to isolate AI workloads (separate service/container), rate-limit heavy requests and cache the results of repeated dashboard views.

Risk 9: Risk of ingesting events that are not available or delayed.

The contribution dashboards require full and up-to-date activity logs (task updates, status changes, and time logs). In case event ingestion places records under load, or retries and events

get duplicated, analytics will not be realistic. This goes against the objective of making contributions visible and measurable. Mitigation consists of deduplication techniques, sound ingestion (queue/retry), and reconciliation between transactional data and event logs.

Risk 10 Backup / restore failures, and retention misconfiguration.

The system should be backed up in an incremental manner after every 24 hours and full backup after each week to be kept in an encrypted offsite cloud. Failure to test backups or restores may imply that the system may lose the key audit evidence (TimeLogs/ActivityLogs), one of the greatest risks of an academic contribution-tracking platform. Mitigation involves the seasonal restore testing (at least against a small sample), documented recovery procedures, and checking backups that are not only of PostgreSQL data but also any log-store data that is required to complete the analytics.

Risk 11: Environment drift and deployment differences throughout the team.

Docker will aim at standardising development environments and isolating services (frontend, backend, database, AI module). When Docker setups drift (they are different, they do not have the required env variables, there are issues in secret handling), then the team can run into the issues of the “works on my machine” which can slow down the progress and conceal bugs until later testing. Such mitigation measures as having only one source-of-truth compose file, a setup checklist, and regular pull-and-run checks by various team members are implemented.

Risk 12: Browser-side UI performance and role-visibility concerns.

TeamTrack is availed via browser and it will need to control role visibility in the UI as well as the API tier. When the UI components are big (large boards, analytics charts, lists of messages), they may appear laggy or slow to interact with. This danger is magnified as the datasets are expanded throughout the semester. One mitigation is to use pagination/virtual lists on large pages, keep re-rendering to a minimum and ensure that restricted UI components have been not only hidden visually but also restricted by backend authorization.

10.2 Schedule / Team Risks

TeamTrack is being developed in a capstone academic timeline, which introduces a fixed-deadline situation in which scope and quality have to be weighed against each other. The SRS also acknowledges that a stable MVP should get priority during development first then optional features added later, which implies that time pressure, inter-section/inter-module dependencies, and coordination among multiple contributors are the primary risks to development. In previous milestone work, the team had already implemented sprint-type cycles, section ownership, and

progress tracking coordination activities, thus, the same risks and controls are applicable in moving into the implementation planning.

Risk 1: Underestimating the amount of work because of academic time squeeze.

The project is limited in time (about four months) and has to concentrate on the core features (task tracking, time tracking, communication, analytics). When the team does not work hard enough, they will be accumulating work close to deadlines and create defects and deliverables that are not complete. This risk is greater when the attempt to add the features which are in the category of nice to have is done before the MVP has been locked.

Risk 2: During sprints, scope creep can be experienced.

Sprint like and agile planning allow iteration, although they can also facilitate the addition of ideas during the cycle, particularly when research or other feedback suggests new opportunities. Unplanned scope changes may postpone the delivery of core and lead to the incomplete modules. In order to minimise last-minute overload, it is useful to keep sprint targets small and to base new work on backlog prioritisation.

Risk 3: Dependency blockers between sections / modules

The documentation workflow of the team already understands dependencies (some tasks are maintained in the backlog until other elements can clarify the inputs). When a major part of the system (e.g., endpoint backends, database structure, authentication rules, etc.) is changed later, it may wait until the UI is blocked, analytics configurations, and testing strategies are finalized. Sprint planning must clearly follow the dependencies and ensure that prerequisite decisions are made prior to the commencement of implementation.

Risk 4: Workload imbalance and work bottlenecks on an individual.

Section ownership and expertise ensured that each section had responsibilities, thereby enhancing accountability but may cause bottlenecks due to work overload or absence of one of the members. When a high-dependency part comes to a halt, the whole sprint becomes slow. This can be reduced by cross-checking the work during sprint planning and working on high priority items in pairs.

Risk 5: Missed alignment and communication delays.

The project will be based on the various communication channels (formal client meetings, Google Meet, WhatsApp, shared repository). This helps to keep updates fast but may also result in the vital decisions being spread out in various tools or not comprehended (e.g. something agreed in WhatsApp not translated into Trello/tasks). Even the governance strategy already focuses on the minimization of ambiguity, but alignment remains reliant on regular updates and recording decisions.

Risk 6: Poor handoffs amongst team members.

Due to the division of the work into modules, handoffs are common (frontend ↔ backend ↔ database ↔ AI). In case of ambiguity in handoff details (payloads, role permissions, field names), integration will be slow and rework will be more. This is particularly dangerous with a modular architecture where maintainability is expected to have clear boundaries.

Risk 7: Left too late review/validation.

The workflow of the team on Trello has a Review/Validation stage and does not push the work to the Done stage until the checks and feedback are exercised. When the review is not carried out in time before submission, then the team might present with formatting problems, missing parts or sections, or discrepant content. Early checkpoint review at every iteration is useful to maintain the quality.

Risk 8: Meeting time not translating into action items.

Meeting notes have been in use where decisions formerly are traceable and continuity is maintained such as the assignment of documentation duties and progress control. Whilst time is being spent, any meetings that are discussion-heavy with no action items will delay the progress of these meetings. This risk can be minimized by maintaining activities associated with Trello cards and assigning owners.

Risk 9: Tool/process friction (tool fatigue).

According to the SRS, complex tools (such as Jira) may cause students teams to experience tool fatigue, in which they deteriorate time using the tool instead of the project. Trello is liable to overhead as well when boards are not maintained. It needs to remain a light process- just enough monitoring to ensure accountability without having too much where it slows down delivery.

Risk 10: Milestone window Attendances/availability change.

Due to the small scale of this student project and a lack of time and specific DevOps/SRE-like coverage, there are few schedule slip opportunities due to unanticipated availability loss (illness, exams, personal conflict). The presence of this risk is increased where a single individual

possesses all the specialised knowledge (single point of failure). The effect of pairing and shared documentation is lessened.

Risk 11: Unrealistic iteration planning and errors in estimation.

Agile planning is based on realistic estimates of iterations and avoiding of realistic and incorrect estimates; big tasks must be increased into small and quantifiable units. Sprint commitments result in a missed deadline in case the tasks are not broken down or estimated optimistically.

Risk 12: Risk of compliance and submission (version, completeness, formatting).

Capstone papers must be professionally formatted, correctly structured and meet milestone standards. In case the team hurries up in submitting assembly, then it may result in omission of evidence parts, uneven headings, or unfinished appendices, despite the technical work being done. This can be minimized by treating final packaging as a scheduled activity (not a last minute activity).

10.3 Risk of Data Privacy and Security.

TeamTrack accepts user accounts, project/workspace data, task content, chat messages and evidence of contribution including Time Logs and Activity Logs. Since the platform is constructed in a way that allows contributions to be transparent (and in particular, perspective of a team leader and instructor by simply observing the session), the risks of privacy and security are primarily due to the improper control of access, inadequate protection of sessions, unsecured data processing, and unintentional exposure of logs/analytics/AI capabilities.

Risk 1: There is risk of unauthorized access caused by RBAC misconfiguration.

TeamTrack has other types of users (student members, team leaders, instructors/supervisors (read only), and system administrators). In case role checks are not consistent among endpoints, a user can access private project information or update records that they are not supposed to access (e.g., editing logs, changing tasks or accessing the data of another team). This is a high impact risk as it impacts privacy as well as faith in contribution evidence.

Risk 2: JWT/session compromise and account takeover.

Authentication and access control of TeamTrack is a requirement in the system. The attackers can end up using tokens to access secured features in case it is stored insecurely in the browser, leaked in logs, or not deleted/revoked appropriately. This is more severe in case of the

compromised privileged positions (leader/admin) as this is the ones that are generally more visible.

Risk 3: Weak password storage policies or absence of hardening.

One of the fundamental features is user authentication which should be done securely. When password hashing and salting are not done correctly, the leakage of a database may reveal credentials. This is particularly dangerous in student settings where the use of common passwords across services is frequent.

Risk 4: Insecure transport (TLS/ HTTPS misconfiguration).

TeamTrack is a web-based client-server application that is used on browsers. Unless HTTPS is applied on an end-to-end basis (although API calls are included), tokens and user information may be compromised over a common network. The login and session security is impacted by this risk as well.

Risk 5: The exposure of sensitive data via logs and analytics.

TeamTrack is based on the use of Activity Logs and Time Logs as the source of accountability and analytics. When sensitive content is captured in logs (e.g., in private chat snippets, tokens, emails, detailed information on who did what more than is required), the damage to privacy is greater. Traceability is supposed to be enabled by logging, and excessive logging is a security vulnerability.

Risk 6: Data integrity and contribution evidence tampering.

Time Logs and Activity Logs are required to be reliable evidence to the value of the system. Provided that users are free to edit previous logs or the tooling of the administrator is not safeguarded, it is possible to alter the contribution records and this is not fair or accountable. This is a security risk (unauthorised modification) as well as an academic integrity risk.

Risk 7: AI misuse or breach of personal information into prompts/outputs.

TeamTrack is comprised of AI-assisted knowledge and an intelligent query/report assistant. When individuals enter into the AI prompts sensitive material or when the system automatically subjects raw private messages/logs to AI, without restrictions, the personal information may be exposed in generated summaries. It is a governance risk as well: AI outputs have to remain consistent with the assistive insights instead of revealing unneeded personal information.

Risk 8: Cross project data leakage (tenant isolation failure).

TeamTrack works with several projects/ workspaces. Without proper scoping of database queries or caching (checks project_id / workspace_id not in place), a user can view tasks, logs or analytics

of another team. It can occur easily when developing dashboard/analytics in case there is inconsistency in filtering.

Risk 9: Input-based attacks (XSS/ SQL injection/ unsafe rendering).

The platform takes user generated content through tasks, messages and AI queries. In case validation/sanitisation is not performed fully, an attacker can insert scripts (XSS) or make unsafe queries. The damage goes between session hijacking or relentless defacement or information breach.

Risk 10: Brute force and enumeration attacks on auth endpoints.

The style flows are frequently targeted in registration, login and password reset. Absence of rate limiting and lockout policies means that attackers can try credential stuffing or will find valid emails/user names through error responses. This augments account takeover risk.

Risk 11: Backup, retention and disposal risks.

In the SRS, there are data retention/archiving and privacy-conscious disposal requirements. When sensitive records (user accounts, logs) are stored in the backups (which are not encrypted or stored permanently), a subsequent leak will have a significantly greater impact. Live data is not the only source of privacy threat, there are also old archived data that were never tidied.

Risk 12: Overexposure of monitoring/telemetry data.

This is critical to performance and reliability monitoring, however accidental request payloads, headers (including tokens), or private message content may be used as diagnostic dashboards or traces of errors. This is an everyday real life problem in which the support tooling is the leak path.

Risk 13: Third-party dependency vulnerabilities.

The selected stack consists of several frameworks and services (frontend framework, backend framework, database, containerisation, and AI tooling). An opening is provided by any vulnerable dependency which is not updated, or which is updated with insecure default settings.

10.4 Mitigation Strategies

TeamTrack mitigation strategy is aimed at alleviating the probability and consequence of the technical, schedule/team, and privacy/security risks that were observed in Sections 10.1-10.3 by implementing mitigation controls in three tiers: architecture + implementation safeguards, security/privacy controls, and project management controls. One of the principles is that both

contribution evidence (TimeLog and ActivityLog) should be trustworthy, and it is facilitated by imposing atomic updates and strong integrity constraints on the data layer.

The team will focus on ensuring that the most write-heavy endpoints (task updates, time logging, activity logging) are performance tested initially in development and require pagination and indexed query as the route analytics heavy routes. In areas where traffic peaks or abuse signatures are experienced, rate limiting or code bans will be used as API middleware to ensure that critical endpoints are not blocked to legitimate end users.

The system will view task/time updates and audit records of the same, as a synchronous work unit to avoid data inconsistency between transactional data and event/activity logging. This consists of write retry of event/logs and integrity verification so that analytics does not silently drift out of the source-of-truth records. Moreover, TimeLog and ActivityLog operations should be structured in such a way that when failures happen, there is no partial write that can happen.

Access control will be implemented at more than one layer to minimize the vulnerabilities of RBAC enforcement and cross-project leakage: role checks at the API level and database access controls that align with the project membership and role properties (e.g. instructor read-only, team leader write privileges). It will also be verified with role-based test accounts to make sure that the operation with protection will not be carried out outside of the authorized roles according to the planned token-based authentication and RBAC strategy.

To mitigate the risks of session/token compromise, TeamTrack will enforce severe token lifecycle policies (expiry and refresh handling, revocation, etc) to ensure that compromised tokens are not very valuable and suspicious behaviour is monitored at the server level. Each request made with protection will have to be session-validated, and privileged operations will have to be performed with the latest valid role state (not stale client assumptions).

Validation will be considered a backend responsibility initially (payloads scheme validation, and strict restrictions on permitted fields), and then safe frontend rendering will be practiced. Input validation and safe error handling are application security control measures expressly designed to stop unintentional information disclosure in case of failures.

To ensure data-in-transit confidentiality and integrity, TeamTrack will place URLs over HTTPS/TLS between clients and servers and between service-to-service calls (internal, 2 service-to-service) between internal and AI service to avoid exposing credentials and user information on unsecured networks. The deployment model as well facilitates safe routing and enforcement of HTTPS through a reverse proxy setup.

In order to enhance audit reliability and minimize the risk of tampering, critical security and permission events (authentication attempts, permission failures, administrative actions) will be recorded in a more event-sensitive fashion, and key audit trails will assume tamper-resistant formats (e.g. append-only or integrity-protected storage of high-value audit records). This enhances forensic traceability and minimizes the possibility of silent changes to contribution evidence.

To cut down AI-related privacy exposure and inaccurate outputs the AI service will not be able to produce any insight before retrieving authorised data (limited to the role and project assigned to the user), and will not provide responses to the frontend in an unsecured way. Since one of the limitations is hallucination, the responses provided by AI will be framed as recommendations (not orders), and UI prompts that encourage verification and discourage excessive trust in summary generated by AI will be displayed.

To promote the backup/restore resiliency and decrease the catastrophic loss of data, TeamTrack will adhere to the scheduled backup requirements: the incremental backups will be performed every 24 hours, and full backups will be performed 1 time per week with the help of encrypted backup data that will be stored in the off-site cloud storage. Practically, the backups will be accompanied with regular testing of the restores (despite any limitation) to make sure that the recovery is possible within the deadlines.

To ensure the privacy, retention, and disposal, TeamTrack will utilize established retention policies: data of ongoing projects should be stored in primary storage during an academic semester and data of finished projects should be stored in an archived form at least two academic years (and then cold storage). In case a user cancels their account, personal identifiable information will be scrubbed in 30 days and time logs/activity logs will be anonymised but not deleted to maintain fair contribution analytics to the team that remains.

Standardisation of services through containerisation will be employed to minimise the effects of drift in deployment and inconsistency between the environment to minimise the blow radius of faults. This facilitates repeatable deployment and reduces the issues of works on my machine during the integration and testing.

Lastly, in order to address schedule/team risks (scope creep, dependency blockers, late review), sprint planning will maintain work in small, verifiable chunks and impose a definition of done that incorporates review/validation instead of investing QA to the finish. The realistic estimation practice will also be used in planning whereby it will not give a promise that is correct but flawed through breaking down big work into small items in a backlog and monitor progress in completing them.

11. Appendices

Appendix A: Glossary

This glossary defines all the technical and system-related terms that are employed during the TeamTrack report. The terminologies contain the technologies, architecture and development practices mentioned in the body report and the supporting sections.

Term	Definition
API (Application Programming Interface)	An organized interface that enables communication among frontend, backend as well as external services.
Activity Log	A system record which monitors user activities like updating tasks, changing status, logging in, and activities.
AI Module	One of the system components that produce insights, summaries, and recommendations based on AI support.
Analytics Dashboard	A graphical interface with statistics on projects, summary of contributions, trends on work and participation data.
Authentication	The identification of the user and granting access to the system.
Authorisation	The system which regulates the things that a user can do depending on his/her role and permissions.
Backend	The server component of the system that handles the requests, implements the business logic and communicates with databases.
Client-Server Architecture	A type of system where the client requests a server and the server responds and processes requests.
Component Interaction	The exchange and synchronization that takes place within the various modules or sections of the system.

Containerisation	A system architecture: A deployment system where system components are encapsulated in isolated containers so that there is consistency across environments.
Contribution Analytics	The analysis of the task activity, time records and the information about the project participation to assess the individual and team contributions
Dashboard	The primary display that allows the user to have a high-level view of the project progress, activity and contribution information.
Data Flow	Flow of data between the system elements like the frontend to the backend and to the database.
ERD (Entity Relationship Diagram)	A visual representation of system entities, attributes and relationship among them in the database design.
FastAPI	The Python RESTful API services Backend Framework that is employed at TeamTrack to develop secure and structured services using RESTful API.
Figma Prototype	The prototype of design which was designed on Figma to mirror the visual layout, navigation, and user flow of the system.
Firehose	The event-stream layer The logging layer is a high-frequency layer storing activity and time-tracking events separately, not the same as the main database.
Frontend	The user interface component of the system consisting of pages, forms, dash boards, and other interface components.
JWT (JSON Web Token)	A safe token mechanism to store authenticated sessions and authenticate users on API calls.
Layered Architecture	A design type where the system is divided into logical layers including presentation, application as well as data services.
Modular Design	These include a design approach in which system can be divided into separate

	components with each having its own responsibility.
Nuxt.js	The frontend frame-work applied in TeamTrack to create structured and component-based web pages and navigation.
PostgreSQL	The database system that enables the storage of structured information about users, project, tasks and time logs as a relational database management system.
RBAC (Role-Based Access Control)	A security model where access and permission of different users is regulated by assigned roles.
RESTful API	The API style enabling the communication between the client and the server based on the standard HTTP requests and on the structured endpoints.
Scalability	The system was designed to support increased number of users, workload or data growth without significant redesign.
System Boundary	The demarcation of the boundaries of what should be covered in the TeamTrack system and what is considered as outside or out of scope.
Task Management	The system feature whereby one can create, assign, update and monitor the project tasks.
Time Log	An entry of the time that a user was occupied performing a task or activity.
Traceability	The capability to trace the actions, changes and requirements evidently throughout the system and documentation.
UI (User Interface)	The graphical aspect of the system in which the users interface with the pages, controls, and features.
Usability	How simple the system is to master, how effective to operate, how a system can be comprehended by the target users.
Workspace	A teamtrack project area that users can work in, manage tasks and track progress.

Appendix B: Artefacts (Links)

Application	Links
Figma (Low-Fidelity)	Link
Draw.io (Diagrams)	Link1 , Link2
Wireframe	Link
Low Fidelity (pdf)	Link
Survey	Link
Trello Board Link	Link
Version History Document (WORD)	Link

Appendix C: References

- [1] J. Nielsen, “10 Usability Heuristics for User Interface Design.” Nielsen Norman Group, 1994.
<https://www.nngroup.com/articles/ten-usability-heuristics/>
- [2] B. Shneiderman et al., *Designing the User Interface: Strategies for Effective Human-Computer Interaction*, 6th ed. Pearson, 2016.
- [3] W3C, “Web Content Accessibility Guidelines (WCAG) 2.2.” World Wide Web Consortium, 2023.
<https://www.w3.org/TR/WCAG22/>
- [4] R. T. Fielding, “Architectural Styles and the Design of Network-Based Software Architectures.” Ph.D. dissertation, University of California, Irvine, 2000.
<https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [5] D. Ferraiolo, D. Kuhn, and R. Chandramouli, *Role-Based Access Control*. Artech House, 2003.
- [6] D. Merkel, “Docker: Lightweight Linux Containers for Consistent Development and Deployment.” *Linux Journal*, no. 239, 2014. <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>
- [7] PostgreSQL Global Development Group, “About PostgreSQL.”
<https://www.postgresql.org/about/>
- [8] StreamingFast, “Firehose Overview.” <https://firehose.streamingfast.io/introduction/firehose-overview>
- [9] Meta AI, “Introducing Meta Llama 3.1.” <https://ai.meta.com/blog/meta-llama-3-1/>
- [10] OWASP Foundation, “OWASP Top 10:2025.” <https://owasp.org/Top10/2025/>
- [11] S. Ramírez, “FastAPI Security Tutorial.” <https://fastapi.tiangolo.com/tutorial/security/>
- [12] GeeksforGeeks, “Software Architectural Patterns in System Design.”
<https://www.geeksforgeeks.org/system-design/design-patterns-architecture/>

- [13] M. Blaha, “Referential Integrity is Important for Databases.” Modelsoft Consulting Corp, 2005. <https://www.odbms.org/wp-content/uploads/2005/11/007.02-Blaha-Referential-Integrity-Is-Important-For-Databases-November-2005.pdf>
- [14] ISO/IEC/IEEE, “ISO/IEC/IEEE 42010:2022 — Software, systems and enterprise — Architecture description.” IEC Webstore, Nov. 2022. <https://webstore.iec.ch/en/publication/80194>
- [15] R. S. Sandhu, D. F. Ferraiolo, and R. Kuhn, “The NIST Model for Role-Based Access Control: Toward a Unified Standard.” NIST, 2000. <https://csrc.nist.gov/projects/role-based-access-control>
- [16] S. Brown, “The C4 model for visualising software architecture.” C4 Model, 2018. <https://c4model.com/>
- [17] National Institute of Standards and Technology, “Digital Identity Guidelines: Authentication and Authenticator Management (NIST SP 800-63B-4).” NIST, Jul. 2025. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-63B-4.pdf>
- [18] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT).” IETF RFC 7519, May 2015. <https://datatracker.ietf.org/doc/html/rfc7519>
- [19] R. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics.” IETF RFC 9110, Jun. 2022. <https://www.rfc-editor.org/rfc/rfc9110.html>
- [20] OWASP Foundation, “OWASP Top 10 API Security Risks – 2023.” OWASP, 2023. <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- [21] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0) (NIST AI 100-1).” NIST, Jan. 2023. <https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf>
- [22] Nuxt Team, “Nuxt 3 Documentation.” Nuxt. <https://nuxt.com/docs>
- [23] Vue.js, “Vue.js Documentation.” <https://vuejs.org/guide/>
- [24] S. Ramírez, “FastAPI Documentation.” FastAPI. <https://fastapi.tiangolo.com/>
- [25] FastAPI Documentation, “Security OAuth2 and JWT.” <https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>
- [26] OpenAPI Initiative, “OpenAPI Specification (v3.1).” <https://spec.openapis.org/oas/v3.1.0.html>
- [27] M. Jones and D. Hardt, “The OAuth 2.0 Authorization Framework: Bearer Token Usage.” RFC 6750, IETF, Oct. 2012. <https://www.rfc-editor.org/rfc/rfc6750>
- [28] D. Hardt, “The OAuth 2.0 Authorization Framework.” RFC 6749, IETF, Oct. 2012. <https://www.rfc-editor.org/rfc/rfc6749>
- [29] PostgreSQL Global Development Group, “Concurrency Control - MVCC.” <https://www.postgresql.org/docs/current/mvcc.html>

[30] Google Cloud, “Cloud Firestore Documentation - Overview.”

<https://cloud.google.com/firestore/docs/overview>

Appendix D: Attribution Table (WBS)

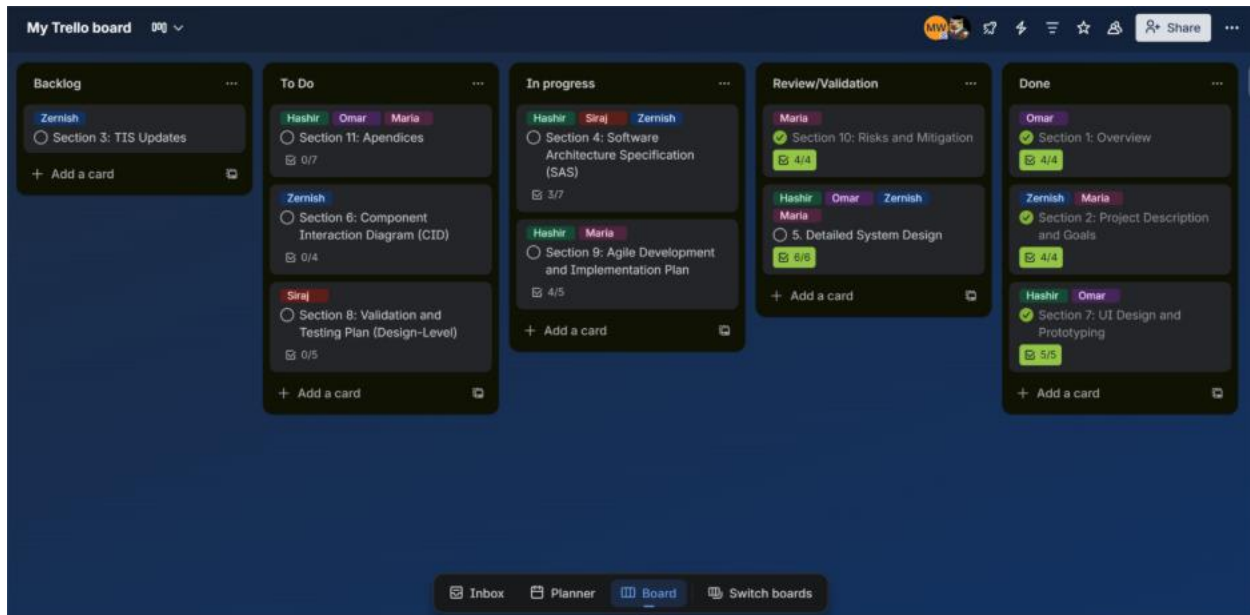
TASKS		TEAM MEMBERS				
Main Part	Sub Part	Omar Waheed Khan	Muhammad Hashir	Maria Waheed	Zernish Shahid Nazir	Mohammad Siraj Hussain
Executive Summary	N/A					
Overview	Background and Context					
	Purpose of This Milestone Report					
	Scope and Key Objectives					
	Summary of Changes Since Previous Milestone					
Project Description and Goals	Problem Statement					
	Proposed Solution Overview					
	Target Users and Use Cases					
	Assumptions and Constraints					
TIS Updates	N/A					
SRS Updates	N/a					
Software Architecture Specification (SAS)	Architectural Overview and Style					
	System Context and Boundaries					
	Major Components / Modules (Responsibilities)					
	Data Architecture (Storage + Flow)					
	Integration Architecture					

	(External Services/APIs)					
	Security Architecture and Ethical Considerations					
	Architecture Decisions and Alternatives Considered					
Detailed System Design	High-Level Architecture Diagram (Overview)					
	Frontend Design (Pages, Navigation, Components Integration)					
	Backend Design (Planned Services + Endpoint List)					
	Database / Data Model Design (ERD + Schema Summary)					
	Authentication and Role-Based Access (Planned)					
	Design Constraints and Scalability Notes					
UI Design and Prototyping	UI Design Approach and Standards					
	Wireframes and Screen Flow (Early Layouts)					
	Low-Fidelity Prototype (Screens + User Journeys)					
	Improvements Made					
	Conclusion (of section)					

Validation and Testing Plan (Design-Level)	Validation Objectives and Scope					
	Requirements Validation (Acceptance Criteria Review)					
	UI / Prototype Validation (Walkthrough + Findings)					
	API Specification Review (Planned Payloads + Error Cases)					
	Database Validation (Entity Relationships + Constraints)					
Agile Development and Implementation Plan	Development Approach (Iterations / Sprints)					
	Module Breakdown and Team Allocation					
	Timeline and Deliverables					
	Tools and Collaboration (Trello, Meetings, WhatsApp)					
	Meeting Minutes					
Risks and Mitigation	Technical Risks					
	Schedule / Team Risks					
	Data Privacy and Security Risks					
	Mitigation Strategies					
Appendices	Glossary					
	UI Artefacts (Links)					
	References					
	Attribution table (WBS)					

	Trello / Task Board Evidence					
	Toggl					
Formatting	N/A					

Appendix E: Trello / Task Board Evidence



Appendix F: User Manual

1. Purpose of the User Manual

This is a rough guideline of how to navigate and use the TeamTrack system documentation and interface. It will be targeted at ensuring that the reader/evaluator can easily access important sections of the document, to get to know and understand the components of the system, and to find the relevant supporting materials that are included in the document.

The guide will be aimed at document navigation, structural features, and usability amenities, including the table of contents, figure references, and quick navigation provision.

2. Document Navigation Structure

The document has been designed to enable it to navigate easily and find components of the system, design artefacts, and implementation details in a short time.

The following navigation elements are included:

- Table of Contents (TOC)
- List of Tables
- List of Figures
- Section Reference Links
- “Back to Top” navigation button

These elements improve readability, accessibility, and document usability for reviewers and stakeholders.

3. Table of Contents

At the onset of the document, the Table of Contents is given, which contains all the major sections, subsections, and appendices. All the entries are connected to their respective sections so that one can navigate fast.

Any headings in the TOC can be clicked by the users to directly get to the required part of the report.

4. List of Tables

The List of Tables presents all tabular content used throughout the document like:

- Stakeholder tables
- Requirements tables
- Technology comparison tables
- Testing matrices

5. List of Figures

The List of Figures includes all visual artefacts and diagrams used within the document, such as:

- System architecture diagrams
- User flow diagrams
- Database schema diagrams
- UI wireframes and logo artefacts

Each figure is numbered and referenced for easy navigation and traceability.

6. Section Reference Links

Hyperlinked references are embedded throughout the document to allow quick navigation between related sections. These links guide readers to supporting content such as:

- Technology justification sections
- System architecture explanations
- Testing and validation details
- Implementation descriptions

This ensures continuity and prevents duplication of explanations across sections.

7. “Back to Top” Navigation Feature

To enhance usability and document accessibility, a “Back to Top” navigation feature is included at the bottom-left of each page.

This feature allows readers to:

- Instantly return to the beginning of the document
- Access the Table of Contents without manual scrolling
- Improve navigation efficiency during review

This is particularly useful for long technical documents where frequent movement between sections is required.

8. How to Use This Document Efficiently

Readers are encouraged to follow these steps when using the document:

1. Begin with the Table of Contents to determine the needed part.
2. Search visual or structured content using the List of Tables or List of Figures.
3. See internal reference explanations on related explanations.
4. The button known as the Back to Top will help you get back to the top of the document easily.

The method can guarantee easy navigation and understanding of the system documentation.

9. Intended Users of the Guide

This quick guide is intended for:

- Academic evaluators
- Project supervisors
- Team members

- Stakeholders reviewing the system documentation

It supports efficient access to project information and enhances overall document usability.

Appendix G: Toggl

Table: Toggl Report Table

Member Name	Toggl Report (Links)
Muhammad Hashir	Link
Omar Waheed Khan	Link
Zernish Nazir	Link
Maria Waheed	Link
Mohammed Siraj Hussain	Link

Work Distribution Summary for Software Architecture Specification (SAS)

Ownership of the Software Architecture Specification (SAS) was distributed across team members based on section relevance, technical expertise, and alignment with architectural and design responsibilities. Each section and subsection was assigned to specific contributors to ensure accountability and clarity of ownership. The distribution strictly followed the SAS Table of Contents, ensuring that architectural design, system modeling, validation planning, and implementation strategy were comprehensively covered. Individual contributions were consolidated into a unified document, maintaining consistency in structure, terminology, and technical depth across all sections.

Section-Wise Contribution Tables

1. Executive Summary

Table 1.1: Work Distribution for Executive Summary

Section / Subsection	Primary Contributor	Contribution Details
Executive Summary	Omar Waheed Khan	Drafted the executive summary, providing an overview of the SAS, highlighting architectural direction, design scope, and key updates since the previous milestone.

2. Project Description and Goals

Table 2.1: Work Distribution for Project Description and Goals

Section / Subsection	Primary Contributor	Contribution Details
2. Project Description and Goals	Maria Waheed, Zernish Shahid Nazir	Shared ownership of the section, with Zernish leading problem definition and solution framing, and Maria contributing to user-focused and constraint-related aspects.
2.1 Problem Statement	Zernish Shahid Nazir	Defined the core problem addressed by TeamTrack within the architectural context, focusing on inefficiencies in collaboration and contribution tracking.
2.2 Proposed Solution Overview	Zernish Shahid Nazir	Documented the proposed system solution, outlining architectural intent, system capabilities, and high-level design direction.
2.3 Target Users and Use Cases	Maria Waheed	Identified target users and documented key use cases representing system interaction scenarios.
2.4 Assumptions and Constraints	Maria Waheed	Documented system assumptions and constraints impacting architectural decisions, scalability, and implementation feasibility.

3. TIS Updates

Table 3.1: Work Distribution for TIS Updates

Section / Subsection	Primary Contributor	Contribution Details
3. TIS Updates	Zernish Shahid Nazir	Documented updates made to the TIS and analysed their impact on the Software Architecture Specification, ensuring alignment between design and implementation.

4. Software Architecture Specification (SAS)

Table 4.1: Work Distribution for Software Architecture Specification

Section / Subsection	Primary Contributor	Contribution Details
----------------------	---------------------	----------------------

4. Software Architecture Specification	Mohammad Siraj Hussain, Muhammad Hashir, Zernish Shahid Nazir	Shared ownership across architectural design, with contributors collaborating on system structure, data flow, and integration strategies.
4.1 Architectural Overview and Style	Mohammad Siraj Hussain	Defined the architectural style, including system layering, modular design principles, and overall structural approach.
4.2 System Context and Boundaries	Mohammad Siraj Hussain	Documented system boundaries, external actors, and interactions between the system and external environments.
4.3 Major Components / Modules (Responsibilities)	Muhammad Hashir	Identified system components and defined their responsibilities within the architecture.
4.4 Data Architecture (Storage + Flow)	Zernish Shahid Nazir	Designed data storage structure and defined data flow across system components.
4.5 Integration Architecture (External Services/APIs)	Muhammad Hashir	Documented integration with external services and APIs, including communication mechanisms and data exchange.
4.6 Security Architecture and Ethical Considerations	Zernish Shahid Nazir	Defined system security architecture, including access control, data protection, and ethical considerations.
4.7 Architecture Decisions and Alternatives Considered	Zernish Shahid Nazir	Documented key architectural decisions and evaluated alternative approaches with justification.

5. Detailed System Design

Table 5.1: Work Distribution for Detailed System Design

Section / Subsection	Primary Contributor	Contribution Details
5. Detailed System Design	Maria Waheed, Muhammad Hashir, Zernish Shahid Nazir, Omar Waheed Khan	Collaborative effort across system design, ensuring alignment between frontend, backend, database, and architectural components.
5.1 High-Level Architecture Diagram (Overview)	Muhammad Hashir	Designed the high-level system architecture diagram illustrating component relationships and system flow.

5.2 Frontend Design (Pages, Navigation, Components Integration)	Maria Waheed	Defined frontend structure, page hierarchy, navigation flow, and component integration.
5.3 Backend Design (Planned Services + Endpoint List)	Muhammad Hashir	Designed backend services, API endpoints, and interaction logic between system modules.
5.4 Database / Data Model Design (ERD + Schema Summary)	Zernish Shahid Nazir	Developed the Entity Relationship Diagram (ERD) and database schema structure.
5.5 Authentication and Role-Based Access (Planned)	Maria Waheed	Designed authentication mechanisms and role-based access control model.
5.6 Design Constraints and Scalability Notes	Omar Waheed Khan	Documented system constraints and scalability considerations affecting system growth and performance.

6. Component Interaction Diagram (CID)

Table 6.1: Work Distribution for Component Interaction Diagram

Section / Subsection	Primary Contributor	Contribution Details
6. Component Interaction Diagram (CID)	Zernish Shahid Nazir	Led all subsections, designing and documenting component interactions, communication flows, and responsibility allocation across system modules.

7. UI Design and Prototyping

Table 7.1: Work Distribution for UI Design and Prototyping

Section / Subsection	Primary Contributor	Contribution Details
7. UI Design and Prototyping	Omar Waheed Khan, Muhammad Hashir	Shared ownership of UI design and prototyping activities.
7.1 UI Design Approach and Standards	Omar Waheed Khan	Defined UI design principles, standards, and usability guidelines.

7.2 Wireframes and Screen Flow (Early Layouts)	Muhammad Hashir	Created wireframes and initial screen flow layouts.
7.3 Low-Fidelity Prototype (Screens + User Journeys)	Muhammad Hashir	Developed low-fidelity prototypes representing user journeys and interface interactions.
7.4 Improvements Made	Omar Waheed Khan	Documented iterative improvements based on design evaluation and feedback.
7.5 Conclusion (of section)	Omar Waheed Khan	Drafted the concluding remarks summarising UI design outcomes.

8. Validation and Testing Plan (Design-Level)

Table 8.1: Work Distribution for Validation and Testing Plan

Section / Subsection	Primary Contributor	Contribution Details
8. Validation and Testing Plan	Mohammad Siraj Hussain	Led all subsections, defining validation objectives, testing scope, and review methodologies for system design.

9. Agile Development and Implementation Plan

Table 9.1: Work Distribution for Agile Development and Implementation Plan

Section / Subsection	Primary Contributor	Contribution Details
9. Agile Development and Implementation Plan	Maria Waheed, Muhammad Hashir	Shared ownership of development planning and implementation strategy.
9.1 Development Approach (Iterations / Sprints)	Maria Waheed	Defined iterative development approach and sprint planning methodology.
9.2 Module Breakdown and Team Allocation	Maria Waheed	Allocated system modules and responsibilities across team members.
9.3 Timeline and Deliverables	Muhammad Hashir	Developed project timeline and defined milestone deliverables.
9.4 Tools and Collaboration (Trello, Meetings, WhatsApp)	Maria Waheed	Documented collaboration tools and communication strategies.

9.5 Meeting Minutes	Muhammad Hashir	Recorded and maintained meeting minutes and discussion outcomes.
---------------------	-----------------	--

10. Risks and Mitigation

Table 10.1: Work Distribution for Risks and Mitigation

Section / Subsection	Primary Contributor	Contribution Details
10. Risks and Mitigation	Maria Waheed	Led all subsections, identifying technical, schedule, and security risks and defining appropriate mitigation strategies.

Appendices

Table A.1: Work Distribution for Appendices

Appendix	Primary Contributor	Contribution Details
A. Glossary	Omar Waheed Khan	Compiled glossary of terms and definitions used throughout the SAS.
B. UI Artefacts (Links)	Omar Waheed Khan, Muhammad Hashir	Documented UI artefacts and provided links to design resources.
C. References	Omar Waheed Khan	Compiled and formatted references according to academic standards.
D. Attribution Table (WBS)	Omar Waheed Khan	Prepared and documented the Work Breakdown Structure attribution table.
E. Trello / Task Board Evidence	Maria Waheed	Documented Trello board evidence demonstrating task allocation and progress tracking.
F. Toggl Reports	All Members	Submitted individual time tracking reports reflecting contribution effort.

Formatting

Table F.1: Work Distribution for Document Formatting

Section	Primary Contributor	Contribution Details
Document Formatting	Omar Waheed Khan, Muhammad Hashir	Ensured consistency in formatting, structure, and presentation of the SAS document.

Final Contribution Percentage Summary

Team Member	Primary Contribution Areas	Approx. Ownership
Muhammad Hashir	Architecture, Backend Design, Agile Planning, UI Wireframes, Timeline, Formatting	22%–26%
Omar Waheed Khan	Executive Summary, UI Design, Appendices, Constraints, Formatting	20%–24%
Maria Waheed	Frontend Design, Agile Planning, Risk Management, Assumptions & Constraints	18%–22%
Zernish Shahid Nazir	Data Architecture, CID, TIS Updates, Architecture Decisions, Database Design	18%–22%
Mohammad Siraj Hussain	Architectural Overview, System Context, Validation and Testing Plan	18%–22%

Closing Statement

Primary ownership of specific sections and subsections in the Software Architecture Specification (SAS) was assigned to individual team members to ensure accountability and clarity in contribution. Despite this structured distribution, the SAS remained a fully collaborative effort. All members participated in review, refinement, and consolidation of the document prior to submission. This approach ensured both individual responsibility and collective ownership, while clearly documenting contribution attribution in alignment with project requirements.

9. Handover Report



Table of Contents

Executive Summary.....	443
2. Introduction.....	444
2.1 Purpose of This Document.....	444
2.1.1 Review and Feedback Purpose	444
2.1.2 Continuity and Reference Purpose	444
2.1.3 Quality and Readiness Purpose.....	445
2.1.4 Purpose Summary.....	445
2.2 What This Handover Document Contains	445
2.2.1 Requirements and Scope Materials	446
2.2.2 User Interface and Design Artefacts	446
2.2.3 Prototype Evidence	446
2.2.4 Supporting Design and Validation Artefacts	446
2.2.5 Agile and Process Evidence	447
2.2.6 Documents Included in This Handover	447
2.2.7 Key Objectives Reflected in the Handover	448
2.2.8 Contents Summary	448
2.3 Out-of-Scope for This Document	449
2.3.1 Full Implementation Package.....	449
2.3.2 Full Repetition of Previous Reports	449
2.3.3 Final System Completion Judgment.....	449
2.3.4 Individual Contribution Submission	449
2.3.5 Out-of-Scope Summary.....	449
3. Project Overview	450
3.1 Problem Statement.....	450
3.1.1 Fragmented Academic Collaboration Tools	450
3.1.2 Lack of Individual Contribution Tracking	451
3.1.3 Limited Transparency and Accountability	451
3.1.4 Absence of AI-Assisted Insights	452
3.1.5 Summary	452

3.2 Solution Overview	452
3.2.1 TeamTrack Platform Vision.....	453
3.2.2 Key Features and Capabilities.....	453
3.2.3 High-Level Architecture	454
3.3 Objectives and Success Criteria.....	455
3.3.1 Primary Objectives	455
3.3.2 Functional Objectives.....	457
3.3.3 Non-Functional Objectives.....	458
3.3.4 Success Criteria and KPIs	460
4. Software Requirements Specification (UPDATED SRS).....	462
4.1 Functional Requirements	462
4.1.1 User Registration and Authentication.....	462
4.1.2 Role-Based Access Control (RBAC)	463
4.1.3 Project and Team Management.....	464
4.1.4 Task Management System	465
4.1.5 Time Tracking and Effort Logging (CORE).....	466
4.1.6 Team Chat and Communication Module	467
4.1.7 Contribution Analytics Dashboard.....	468
4.1.8 AI-Driven Project Assistant.....	469
4.1.9 Notifications and Reminder System.....	469
4.1.10 Activity Log and Audit Trail.....	470
4.2 Non-Functional Requirements	471
4.2.1 Security Requirements	471
4.2.2 Performance Requirements.....	472
4.2.3 Usability Requirements.....	473
4.2.4 Availability and Reliability Requirements	474
4.2.5 Data Integrity and Privacy	474
4.2.6 Ethical and Responsible Use Requirements	475
4.3 System Constraints.....	475
4.3.1 Network and Deployment Constraints	475
4.3.2 Operating System Constraints	476

4.3.3 Hardware Constraints	476
4.3.4 Academic Time and Resource Constraints	476
4.3.5 Technology Stack Constraints	477
4.3.6 Security and Role-Based Constraints	477
4.3.7 AI Feature Constraints	477
4.3.8 Deployment and Scalability Constraints.....	477
4.4 User Classes and Roles	477
4.4.1 Student Team Members.....	478
4.4.2 Team Leaders.....	478
4.4.3 Instructors or Supervisors (Read-Only Access)	478
4.4.4 System Administrators	478
4.5 SRS Updates and Revisions	479
4.5.1 Functional Requirements Revisions.....	479
4.5.2 Non-Functional Requirements Revisions	480
5. Technology Infrastructure Specification (TIS UPDATION).....	482
5.1 Technology Stack Selection	482
5.1.1 Frontend: Vue.js / Nuxt.js	482
5.1.2 Backend: FastAPI.....	483
5.1.3 Database: PostgreSQL.....	484
5.1.4 AI Module: LLaMA via Ollama	485
5.1.5 Authentication: Firebase / OAuth	486
5.1.6 Overall Stack Summary	487
5.2 System Architecture Design	488
5.2.1 System Architecture Diagram	488
5.2.2 Database Design (ERD / Schema)	489
5.2.3 User Flow Diagram.....	490
5.2.4 System Flow Diagram.....	491
5.2.5 Architecture Design Summary	492
5.3 Security and Performance Considerations.....	492
5.3.1 Security Considerations	492
5.3.2 Authentication and Access Control	493

5.3.3 Data Protection and Secure Handling	493
5.3.4 Performance Considerations	494
5.3.5 Scalability and Maintainability Considerations	494
5.3.6 Security and Performance Summary.....	495
5.4 Technology Investigation and Justification	495
5.4.1 Frontend Technology Justification	496
5.4.2 Backend Technology Justification.....	496
5.4.3 Database Technology Justification	497
5.4.4 AI Technology Justification	497
5.4.5 Authentication Technology Justification	498
5.4.6 Overall Stack Justification	498
5.4.7 Justification Summary	498
6. Deployment and Hosting Overview.....	499
6.1 Deployment Strategy and Environment Setup	499
6.1.1 Deployment Strategy Overview	500
6.1.2 Containerized Deployment Environment	500
6.1.3 Environment Setup (Development, Testing, Production).....	501
6.1.4 Backend and Database Deployment.....	501
6.1.5 AI Module Deployment	502
6.1.6 Security and Communication Setup.....	502
6.1.7 Deployment Summary	503
6.2 Frontend Hosting	503
6.2.1 Nuxt.js Build and Deployment Process	504
5. Updates and Maintenance	504
6.2.2 Selection and Justification of Hosting Platform.....	505
6.2.3 Frontend Hosting Architecture Summary.....	506
6.2.4 Summary	506
6.3 Backend Hosting.....	506
6.3.1 FastAPI Server Deployment.....	507
6.3.2 Environment Variables and Configuration Management	508
6.4 Database Hosting.....	510

6.4.1 PostgreSQL Deployment and Access	510
6.4.2 Data Migration Plan	511
6.5 AI Module Deployment	513
6.5.1 LLaMA via Ollama Local Deployment	514
6.5.2 AI Service Integration with Backend	515
6.6 CI/CD Workflow	516
6.6.1 Purpose of the CI/CD Workflow	516
6.6.2 Bitbucket-Based Deployment Pipeline	517
6.6.3 Build Process and Automation	519
6.6.4 Validation Before Delivery	521
6.6.5 Benefits of Our Planned Workflow	521
6.6.6 CI/CD Workflow Summary	522
6.7 Product Delivery Plan to Client	523
6.7.1 Purpose of the Product Delivery Plan	523
6.7.2 Repository Access and Handover Method	524
6.7.3 Delivery Artefacts Provided to the Client	525
6.7.4 Client Acceptance of the Deployment Approach	526
6.7.5 Benefits of the Planned Delivery Approach	527
6.7.6 Product Delivery Plan Summary	528
7. Technical Readiness	529
7.1 Development Environment Setup	529
7.1.1 Overall Setup Approach	529
7.1.2 Tools and Technologies Setup	530
7.1.3 Environment Consistency and Configuration	530
7.2 Planned Spike Testing Strategy	531
7.2.1 Purpose of Spike Testing	531
7.2.2 Key Areas to be Tested	531
7.2.3 Expected Outcome	531
7.3 Codebase Skeleton	532
7.3.1 Concept and Purpose	532
7.3.2 Expected Project Structure (Assumed)	532

7.3.3 Readiness for Implementation	533
8. Task Backlog and Development Estimates.....	533
8.1 Backlog Snapshot	533
8.1.1 Effort Summary by Component.....	539
8.2 How Effort and Priority Were Decided	539
8.2.1 Priority Framework	539
8.2.2 Effort Estimation Approach	540
8.2.3 Estimation Confidence and Refinement Plan	541
8.3 Sprint Allocation and Delivery Timeline	541
8.3.1 Sprint 1: Foundation and Core Skeleton	541
8.3.2 Sprint 2: Core Features and Analytics	542
8.3.3 Sprint 3: AI, Deployment and Acceptance Testing.....	542
8.4 Dependencies, Risk Control and Definition of "Done"	543
8.4.1 Task-Level Dependency Overview.....	543
8.4.2 Risk Controls.....	544
8.4.3 Definition of "Done".....	545
9. Client Communication and Acceptance	546
9.1 UI Feedback and Sign-Off	546
9.1.1 UI Feedback Process	547
9.1.2 UI Feedback Findings and Improvements	547
9.1.3 UI Validation and Acceptance	548
9.1.4 UI Sign-Off	548
9.2 Product Delivery Plan.....	549
9.2.1 Deliverables Included in Product Delivery.....	549
9.2.2 Delivery Process	550
9.2.3 Acceptance Criteria	550
9.2.4 Final Delivery Summary	551
9.4 Technology and Development Readiness	551
9.4.1 System and Environment Readiness.....	551
9.4.2 Development Approach and Implementation Readiness	551
9.5 Summary of Client Alignment	552

9.5.1 Alignment with Project Objectives	552
9.5.2 Consistency Across Deliverables	552
9.5.3 Communication and Alignment Confirmation	552
9.6 Meeting Minutes	553
9.6.1 Meeting Log.....	553
10. Appendices.....	555
Appendix A: Glossary.....	555
Appendix B: Updated Links.....	558
Appendix C: References.....	559
Appendix D: Contribution Matrix.....	560
Appendix E: User Manual	562
Appendix F: Toggl Reports	564

List of Tables

Table 1: Purpose of This Document Summary	445
Table 2: Documents Included in This Handover	447
Table 3: Objective-to-Document Mapping	448
Table 4: Out-of-Scope Summary	449
Table 5: User Classes and Roles	479
Table 6: New Functional Requirements	480
Table 7: Frontend Technology Selection Summary	483
Table 8: Backend Technology Selection Summary	484
Table 9: Database Technology Selection Summary	485
Table 10: AI Technology Selection Summary	486
Table 11: Authentication Technology Selection Summary	487
Table 12: Overall Technology Stack Overview	488
Table 13: Architecture Artefacts Reused in Milestone 4	492
Table 14: Security and Performance Considerations Summary	495
Table 15: Technology Investigation and Justification Summary	498
Table 16: Main Objectives of the CI/CD Workflow	517
Table 17: Bitbucket-Based Pipeline Stages	518
Table 18: Build and Automation Activities	519
Table 19: Validation Focus Areas.....	521
Table 20: Benefits of the Planned CI/CD Workflow	522
Table 21: Overall CI/CD Workflow Summary	522

Table 22: Main Objectives of the Product Delivery Plan	524
Table 23: Repository Access and Handover Components.....	525
Table 24: Delivery Artefacts Summary	526
Table 25: Client Acceptance Considerations	527
Table 26: Benefits of the Planned Delivery Approach.....	528
Table 27: Overall Product Delivery Plan Summary	528
Table 28: Proposed Codebase Structure (Initial Setup).....	532
Table 29: Full Task Backlog with Effort Estimates and Sprint Allocation.....	539
Table 30: Effort Summary by Component.....	539
Table 31: Task-Level Dependency Matrix	544
Table 1: Meeting logs	553
Table 2: Meeting Minutes Summary	554

List of Figures

Figure 1: System Architecture	488
Figure 2: Database Schema	489
Figure 3: System Flow Diagram	491
Figure 4: CI/CD Workflow Diagram	520

Executive Summary

This milestone report provides the handover and implementation preparedness of TeamTrack, a web-based academic project management and accountability system that aims to assist university students in working on group projects. Following the previous milestones that constituted the system requirements, technical direction, and software architecture, this paper incorporates the above into a viable handover document for development, deployment, and transition to the client. This is to demonstrate not just what TeamTrack is, but how the project is ready to be implemented and delivered.

TeamTrack has been created to address general issues with academic group work, in which students tend to use tools that do not interact with each other, resulting in substandard quality of communication, inability to track contributions, and accountability issues. The proposed system will solve these problems, as it will offer a single integrated system, which will unify project and task management, secure authentication, role-based access control, team communication, time tracking, activity logging, contribution analytics, and AI-assisted support. The platform will enhance transparency, fairness, coordination, and effectiveness of teams in university projects by integrating these functions within the same environment.

The project development process has also been recorded in this document, where planning has been changed to execution preparation. It unifies the software requirements, technology infrastructure decisions, deployment and hosting strategy, technical readiness, backlog planning, and client communication records in a structured format of handover. This gives a coherent overview of the current system, the decisions that the team has made, and the action plans needed to transfer the system design to the implementation.

Technically, TeamTrack is based on a scalable and recent web architecture. It will be based on a planned solution with Nuxt.js / Vue.js as the frontend, FastAPI as the backend services, PostgreSQL as the relational data storage, Firebase / OAuth as the authentication service, and LLaMA through Ollama as the AI-assisted functionality. These technologies were chosen because they help in supporting performance, maintainability, security, and future scalability, and remain within the scope of the project in terms of academics as well as the purpose of the system. The report also reinforced this base with the description of deployment planning, environment setup, codebase preparations, and implementation supporting processes.

In general, this milestone is an intermediary between design and delivery. It demonstrates that TeamTrack is not just a visionary academic solution anymore, but a structured and technically ready system with a clear direction towards development, implementation, and handing the client. The combination of well-defined requirements, technical decisions, delivery planning, and alignment of the stakeholders reveals that the project is ready to enter the next phase of implementation and that this goal will pursue the primary purpose of the project, which is to enhance transparency, accountability, and collaboration when working in academic groups.

2. Introduction

2.1 Purpose of This Document

The present handover document is prepared to show the current state of planning and design of TeamTrack before it is submitted during the final stage. There is no need to discuss the project requirements definition, technology research, and software architecture and interface design since all these stages have already been accomplished at this stage of the capstone. Hence, this document aims at consolidating the most pertinent review documents in a single location to ensure that the client or supervisor can evaluate the direction of the project, ensure compliance with the proposed scope, and give feedback to finalize it.

TeamTrack is a proposed AI-assisted collaborative project management app that is being created to assist in university group work. Throughout the previous milestones, the system has been characterized by the ability to create project workspaces, assign tasks, have conversations within the team, log time, history of activities, analytics of contributions, AI-assisted insights, and access control based on roles to students, team leaders and instructors.

This is to be a structured handover package and not the re-submission of all the past reports. It emphasizes the artefacts that the reviewer needs the most at Milestone 4: the agreed-upon system direction, the artefacts of the user-facing design, and evidence of the prototype of significant or uncertain features, as well as the evidence of the supporting process, i.e., Agile progress. By so doing, the handover acts as a control point between the planning work done and the last phase of refinement in implementation.

2.1.1 Review and Feedback Purpose

Formal review and feedback are one of the key objectives of this document. The handover enables the client or supervisor to determine whether the project is still on track with the approved scope and whether the design direction is befitting the purpose of the intended users.

This is significant since TeamTrack is not merely a technical system, but a platform, which should be practical and comprehensible to students and instructors. The reviewer can provide comments on the clarity of the system, utility and practicability of the system by providing comments on the requirements of summary, interface design, and prototype evidence presented jointly before the system is finally submitted.

2.1.2 Continuity and Reference Purpose

The other objective of this document is to establish continuity between the previous milestone reports and the final deliverables that will come thereafter. The SRS determined what should be done in the system, the TIS defended the technical direction, and the SAS planned the systems architecture, modules, interfaces and the UI based on this plan.

Those strands are made available as a more reviewable form in this handover, allowing the project to proceed without confusion, duplication, or gaps among documents.

2.1.3 Quality and Readiness Purpose

This document also shows that the project is in a reasonable level of design readiness. It demonstrates that requirements have been converted into interface flows, architectural choices, and prototype artefacts as opposed to just being a conceptual notion.

This preparedness is particularly critical in Milestone 4 since the handover will demonstrate that the group possesses sufficient material on validated planning to allow the group to have meaningful client feedback and an easier transition to the final stage.

2.1.4 Purpose Summary

In general, this document is designed to assist in conducting the review, keeping continuity between milestones, and showing that TeamTrack is prepared to be refined. It is a concentrated package of handover which makes the most pertinent planning and design artefacts available in an organized manner.

Table 98: Purpose of This Document Summary

Subsection	Purpose Area	Explanation
2.1.1	Review and Feedback Purpose	Enables the client or supervisor to review the current project direction and provide final-stage feedback.
2.1.2	Continuity and Reference Purpose	Connects the SRS, TIS, and SAS into one handover package for easier use and traceability.
2.1.3	Quality and Readiness Purpose	Shows that the project has reached a meaningful planning and design readiness level.
2.1.4	Purpose Summary	Summarizes the overall reason this handover document has been prepared.

2.2 What This Handover Document Contains

This handover document is the most relevant of the materials to be reviewed by clients and supervisors at this point of the project. It is not necessary to recreate all the technical aspects of

the past milestones, but to show the most efficient artefacts that can be used to analyze the direction, readiness, and quality of TeamTrack prior to the final submission.

2.2.1 Requirements and Scope Materials

The handover will also contain the materials of requirements which stipulates the purpose of the entire system, purpose of the users and the anticipated functionality. In the case of TeamTrack, such materials prepare the platform as a university-oriented collaborative system, which aims to enhance accountability, visibility of tasks, communication, and contribution monitoring of group projects. Other key functional areas, including authentication, project and team management, task workflows, team chat, time tracking, contribution analytics, and AI-assisted support, are defined by the requirements as well.

2.2.2 User Interface and Design Artefacts

The interface and design artefacts are also found in the handover, as Milestone 4 focuses on the user-facing component of the project specifically. These documents consist of wireframes, screen designs, navigation structure, user journeys, and other visual planning artefacts which can be used to communicate how users will use TeamTrack.

This is particularly crucial to TeamTrack since the usability of dashboards, task boards, chat spaces, analytics pages, and instructor views are directly linked to whether the system will be effective in achieving its purpose of enhancing collaboration and equity in academic group work. Proposed prototype screens and user journey flows were mentioned as a part of the handover prior to the design documents.

2.2.3 Prototype Evidence

The handover encompasses prototype evidence of features that require to be demonstrated as opposed to being described only. Milestone 4 anticipates the fact that features that might remain unclear, difficult, or await validation should be embodied via prototype work.

In the case of TeamTrack, some of these areas might be AI-enhanced insights, contribution analytics presentation, time logging interactions, and flow charts of task collaboration. These prototypes, even when not completely in use, can assist the reviewer in realizing how the system is supposed to act in practice by the team.

2.2.4 Supporting Design and Validation Artefacts

There are also selected supporting artefacts where they reinforce the handover. They can be the ERD, the references to the data structure in the domain level, the interface validation artifacts, or the arrangements to test key flows available to the user.

The usefulness of these artefacts is that it indicates that there is a logic in the design of the project and not just the surface-level work on the UI. Previous milestones already captured architecture,

database design, validation design and interface behaviour and the said elements were also pertinent to the handover phase.

2.2.5 Agile and Process Evidence

The handover also provides evidence that the team still operates under an Agile process. This involves tracking progress, meetings, task allocation, and collaboration evidence indicating that a group has been working responsibly across milestones in an iterative manner. Sprint planning, task coordination or Trello, meetings, and ownership breakdowns, previously mentioned in earlier reports, are supportive of this section of the handover.

2.2.6 Documents Included in This Handover

The handover package is grounded on the previous TeamTrack milestone documents with some modifications on their client-related segments. This handover has the core documents as indicated in the table below.

Table 99: Documents Included in This Handover

Document / Artefact	Main Purpose in Milestone 4	What It Contributes
Software Requirements Specification (SRS)	Defines agreed system scope and requirements	Functional requirements, non-functional requirements, user roles, interfaces, constraints
Technology Investigation Summary (TIS)	Explains technical direction	Chosen stack, technology justification, implementation approach, security and testing direction
Software Architecture Specification (SAS)	Defines system structure and design	Architecture, components, data flow, UI design basis, validation planning, prototypes
UI/UX Artefacts	Shows user-facing design	Wireframes, screen flow, layout direction, navigation and interaction logic
Prototype Evidence	Demonstrates uncertain or important features	Early proof of usability, interaction direction, feasibility of key features
Supporting Design Artefacts	Strengthens technical understanding	ERD, interface plans, validation materials, design support references
Agile Evidence	Shows organized team process	Sprint planning, task ownership, meeting logs, collaboration tracking

2.2.7 Key Objectives Reflected in the Handover

The handover also reflects the main objectives of TeamTrack that have been carried across the previous milestones. These objectives guide what is included and what the reviewer should be able to confirm from this package.

- Improve transparency in student group collaboration
- Support clearer task ownership and deadline tracking
- Provide a central place for communication and project coordination
- Strengthen accountability through time logs and activity history
- Offer contribution analytics and AI-assisted recommendations
- Support role-based access for team leaders, members, and instructors
- Present a design direction that is practical for final implementation

Table 100: Objective-to-Document Mapping

Project Objective	Main Supporting Document(s)	Reason
Clear functional scope	SRS	Defines required system behaviour and boundaries
Suitable technical direction	TIS	Justifies tools, frameworks, and implementation choices
Coherent architecture and data flow	SAS	Explains system structure, modules, and interactions
Usable interface and navigation	UI/UX Artefacts, SAS	Shows how users will interact with the system
Demonstration of complex features	Prototype Evidence	Makes uncertain or high-value features more concrete
Evidence of organized progress	Agile Evidence	Shows project management, iteration, and team coordination

2.2.8 Contents Summary

All in all, this handover has had the requirements view, interface artefacts, prototype materials, supporting design references, and Agile evidence that can be reviewed with informed entreaty. All these materials combine to project TeamTrack in such a manner that is organized, can be reviewed, and ultimately used to give final feedback.

2.3 Out-of-Scope for This Document

Though the given handover document has the primary materials required to be reviewed, it is purposely reduced in terms of scope. It exists to deliver a narrowly scoped package of feedback as opposed to recreating the entire capstone project in its final form.

2.3.1 Full Implementation Package

The full implementation package of TeamTrack is not present in this document. Full backend code, full frontend code, deployment configuration, full database scripting, and production level set up is not in scope of this handover except in small portions as prototype evidence.

2.3.2 Full Repetition of Previous Reports

This handover also fails to reproduce all the sections of the SRS, TIS and SAS in detail. The requirements, technologies, architecture, testing direction, and planning already have broad coverage in those reports. Rather, this document selects the sections that are most likely to be of interest to the clients and supervisors.

2.3.3 Final System Completion Judgment

The handover does not purport that the system is complete, well tested, and production ready. There are some features that might be done at prototype level or might be refined later after the feedback has been given.

2.3.4 Individual Contribution Submission

This form also fails to substitute the individual contribution form that each member of the group is supposed to submit. Handover is a collective group product and individual reflection and contribution reporting are independent.

2.3.5 Out-of-Scope Summary

The out of scope areas primarily consist of full technical implementation, complete duplication of previous milestone material, final system completion claims and individual submission material. These limits are useful in defining the handover in order to make it focused.

Table 101: Out-of-Scope Summary

Out-of-Scope Area	Explanation
Full implementation package	Not a complete code, deployment, or production handoff
Full repetition of earlier reports	Only the most review-relevant material is included

Final readiness evaluation	This is a feedback-stage handover, not the final proof of completion
Individual contribution work	Separate from the group handover document

3. Project Overview

3.1 Problem Statement

TeamTrack system is developed under the influence of various difficulties that are typical of group projects at school and group work. Current collaboration tools and processes do not usually offer an integrated space that allows to manage tasks, communicate, track contributions, and analyze performance in a well-organized and transparent way. Consequently, learners and lecturers have challenges in monitoring progress, appraisal and accountability within the group work setup. These problems adversely affect the efficiency of collaboration, equality in grading, and the results of the project, in general.

As per the system analysis and the project documentation completed in previous milestones, the common issues found in academic teams include a lack of coordinated communication tools, role ambiguity, low presence of individual input in the team and how to use smart tools to guide the project and project report. TeamTrack platform seeks to resolve the said issues by offering a central academic collaboration platform that allows integrating project management, communication, time tracking, contribution analytics, and AI-assisted insights under one platform.

The key problem areas, which are covered by this project are explained below.

3.1.1 Fragmented Academic Collaboration Tools

Among the major issues revealed during the academic collaboration, there is the usage of various tools that are not connected to perform various functions of group work. Students normally rely on independent communication platforms (e.g. messaging applications), document sharing, task management and time management. Such fragmentation results in unproductive work processes, information loss, insufficiency of coordination, and the inability to monitor the progress of the project on various platforms.

The integration is not present across tools, so that students have to manually monitor all of the tasks, contributors, and communication channels among various systems, which makes it more likely to fail to meet the deadline, communicate inconsistently, and have inconsistent documentation of the project. This issue is particularly crucial in the academic settings as the group

work involves a formal organization and set of coordination and documentation that can be evaluated.

TeamTrack system resolves this problem by offering a centralized solution that brings task management, communication, activity logging and project tracking issues into one system eliminating the need to use many external tools enhancing the efficiency of workflow.

3.1.2 Lack of Individual Contribution Tracking

The second problem associated with academic group projects is the fact that individual contributions cannot be properly tracked. In most group assignments, the instructors have a hard time finding out the amount of each student contributed to the project thus grading them unfairly and causing conflict among team members.

Conventional collaboration tools lack any form of organized contribution, time tracking, and activity tracking. Consequently, a few members of the team might do much more work than others, and all the team members will have equal assessment. Such lack of contribution transparency makes teams less accountable, and this may have an adverse impact on team performance and motivation.

TeamTrack system is designed to address this issue, through the introduction of activity logging, time recording, and contribution analytics which will capture the user activities, changes in activities, and time spent on activities. This enables the instructor and team leaders to assess the contributions in a more balanced way and makes every team member responsible in what he or she does.

3.1.3 Limited Transparency and Accountability

Effective collaboration requires transparency and accountability as this is necessary within the academic setting where group performance and individual contributions need to be considered. Nevertheless, most of the available systems of collaboration lack vision of the progress of the work, history of activities, and engagement in the project.

In the absence of appropriate transparency, team members might not know the project status, due dates, and tasks hence tasks fail to be done properly and there is lack of coordination. Equally, there is a possibility that teachers do not get a detailed record of the project activities and as such, it is not easy to assess project development and student involvement.

TeamTrack platform helps to solve this problem because it offers dashboards, logs of activities, the ability to track the project history, and reporting in order to enable both team members and instructors track the progress and participation during the project life cycle. This enhances accountability and makes it apparent to all the stakeholders how the project is going.

3.1.4 Absence of AI-Assisted Insights

Typical project management tools do not include built-in smart information and automated project data analysis. Users have to read the progress of tasks, activity logs, and performance indicators manually to know the project status and performance of the team.

This is without intelligent support, and both the students and the instructors have a hard time determining risks of the project, a workload imbalance, a delay, or even the trend of performance. Moreover, when it comes to creating project summaries or project reports, manual work and documentation may be needed.

To maximize this limitation, the TeamTrack system includes AI-supported functions that are able to create summaries, analyze contributions, offer project insights as well as support users in reporting and decision making. This increases productivity, better monitoring of projects and data-driven insights to the students and the instructors.

3.1.5 Summary

Overall, it can be claimed that the critical issues concerning academic group collaboration are decoherent tools, the absence of tracking contributions, minimal levels of transparency and accountability, and the absence of AI-aided insights. Such problems lower the level of cooperation among employees, establish inequitable conditions of evaluation, and complicate the task of organizing project management both to the students and the instructors.

The treatment put forward is the TeamTrack system to address these issues since it offers a comprehensive academic collaboration platform integrating project management, communication, time tracking, contribution analytics, and artificial intelligence-aided insights in a centralized system that is specifically structured to handle an academic group project.

3.2 Solution Overview

To overcome the issues found in the problem statement, the proposed solution is the TeamTrack system, which is an AI-assisted academic collaboration and project management system tailored to the university group project. The purpose of the system is to create a centralized space where both students and instructors are able to manage project, monitor tasks, follow contributions and communicate as well as derive insights with the help of AI-assisted analysis. The platform combines various collaboration capacity into one platform to enhance transparency, accountability, organization of projects, performance of the team in general.

The solution is aimed at integrating project and task management, time and project tracking, communication, contribution analytics, and AI-supported reporting into one system. This combined method saves the usage of several external tools, and it offers a systematic setting to conduct academic cooperation. The system will be intended to support students in a team setting,

team leaders in taking care of project tasks and instructors in tracking the progress of the project and project contributions.

The TeamTrack solution aims at the general goal of increasing the efficiency of collaboration, as well as offering equitable contribution monitoring, enhance project visibility, and the academic evaluation with the help of the organized tracking of activities and analytics. The system will also focus on assisting insights and summaries on project performance and progress with the assistance of AI to ensure the users know more about the respective performance and project advancement.

3.2.1 TeamTrack Platform Vision

The mission of the TeamTrack platform is to develop a unified academic teamwork platform (under one roof) that enhances the group project management of the university students. The platform will be created to offer an organized platform through which all the project related functions are handled like task management, communication, time tracking and contribution monitoring are handled at a central location.

The system will benefit both the students and the instructors because it will enhance transparency, accountability and organization of projects. Students will be able to organize tasks, track time, get in touch with the team members and track the progress of their project whereas the instructors will be able to see the activity of the projects, track the contributions and assess the participation of the students more objectively.

The long-term vision of the platform consists of enhancing the academic collaboration workflows, satisfying the assessment of contributions fairly, and offering smart insights into the project performance with the help of AI-assisted analytics. The platform will be scalable and modular in order to have other features and improvements incorporated in the further stages of development.

On the whole, the TeamTrack platform will offer highly structured, clear and smart collaboration platform that is academic group project-specific.

3.2.2 Key Features and Capabilities

TeamTrack system has some main features that are meant to facilitate the work of the academic cooperation and project management. These features will be in accordance with the functional requirements and system modules as outlined in the project documentation.

The most prominent characteristics of the system are:

User Authentication and Account Management

Using the system, users create accounts and use them to safekeep their account information. Authentication and access control is provided to prevent access of project data and system features by unauthorized persons.

Project and Workspace Management

Users are allowed to form project work spaces, members in the team and settings of a project. Any projectwork is a focal node of project tasks, communication, and project information.

Task Management System

Using the system, one can create tasks, assign tasks to staff, update on tasks, and monitor the tasks. The task boards and lists would assist the teams to arrange the project work and keep track of the deadlines.

Time Tracking and Activity Logging

Users are able to record the hours taken in the task and make updates regarding their activity. The system has logs of activity and time records that are utilized in tracking contributions and analyze achievements.

Team Communication Module

The platform has a team communication option where users are enabled to send messages, communicate, and collaborate in the project environment.

Contribution Analytics Dashboard

The system has analytics and reporting capabilities enabling it to monitor user contribution, history and progress of the project. Such analytics assist instructors and team leaders in measuring the work of individuals and the project itself.

AI-Assisted Insights and Reporting

The system will incorporate an AI component, which is capable of producing summaries, insights, and reports in accordance with the data related to project activity and contributions. This assists users in knowing the progress in project, detecting problems and creating reports more effectively.

All these features interact to create a balanced academic collaboration platform, which enhances the organization of the project, tracking of contributions and the general teamwork.

3.2.3 High-Level Architecture

The TeamTrack system is based on a system architecture comprising of a frontend layer, a backend application layer, a data storage layer, and an AI integration layer. This architecture divides the system into various layers in order to enhance scalability, maintainability and organization of the system.

Frontend Technology– Nuxt.js

The front-end layer has to deal with the user interface and interaction. It enables users to Web surf and log on to the system, handle tasks, look at dashboards, interact with team members, and interact with the system.

Backend Technology- Python with FastAPI

The business logic, API services, authentication, and role-based access control tasks, logic of task management, and communication between the frontend and database are handled by the backend layer. The backend gathers the user requests and controls system.

Database – PostgreSQL

Data is filed in its data layer that includes users, projects, tasks, time logs, activity logs, project messages, and project contribution analytics. The database ensures an organized data storage and retrieval.

Event Logging – Firehose Database

The activity logs and event data are stored in a separate logging database. This enhances analytics of the systems and tracking of the contribution without tampering with the main business database.

AI Technology – LLaMA 3.1 via Ollama

The platform combines an AI model to deliver project data-based summaries, analytics and insights. The AI module serves in a containerized environment where it operates the project activity data to give smart insights.

Containerization – Docker

The components of the system that are containerized with Docker include the backend, AI module, and services. This enhances the deployment, scalability and consistency of the environment.

Altogether, the chosen technology stack facilitates the system needs in terms of scalability, performance, and AI integration, as well as the modular system design.

3.3 Objectives and Success Criteria

3.3.1 Primary Objectives

The main goals of TeamTrack accord the essence of purposes that the system is likely to serve in the case of managing group projects in academia. The above goals are based on the identified issues in the previous sections of this report, specifically, the utilization of incomplete collaboration tools, the untrustworthiness of contribution monitoring, the low level of transparency in group activity, and the unintelligible assistance of AI. To address these concerns, TeamTrack is intended as a combination of academic collaboration that facilitates better coordination of

projects, increases accountability and helps provide a deeper evaluation of project participation by students.

The initial main goal of TeamTrack is to unify the academic cooperation within one system. Communication, task management, file sharing, and tracking of the progress are often handled by students relying on several independent tools in most university projects. This usually results in confusion, duplication of effort and low visibility among the team members. TeamTrack will help minimize such problems by making critical collaboration functions interact in a single environment. Centralizing these activities, the system will be able to offer a more organized workflow, less time wastage in changing between platforms, and the efficiency of the project execution.

The second overall goal is to enhance transparency and accountability within the team members. One of the most significant issues of cooperative academic work is that the activities and duties of individuals are not always visible, thus it is hard to decide who did what and how regularly each of the members worked. TeamTrack helps solve this issue by capturing the activities of projects, task update, time recording, and interaction that is related to contributions in a well-organized manner. This will enable students and the instructors to have a better idea of project development and individual involvement. This leads to a more transparent and accountable working environment, which is supported by the platform.

The third major goal is to allow the individual contribution to be evaluated more fairly in a group project. Conventionally, academic group work has a problem of unequal contribution where a few students end up bearing a heavy load of work and yet others gain the same credit despite not contributing much. TeamTrack will facilitate evidence-based evaluation tracking the data on task ownership, its history of completion, its activity of participation, and time-tracking information. This does not imply that the system is used to override instructor judgment, rather this is a more well-founded supporting evidence that is meant to be used to consider the involvement of the students more precisely and impartially.

The fourth overall goal is to deliver intelligent assistance by AI-supported insights of projects. TeamTrack is not just aimed to store the data on collaboration, but also assist the users to interpret the data acquired in a more meaningful manner. By means of AI-based summaries and analytics, the platform will focus on pointing out such patterns as uneven distribution of workload, inactivity, delays in projects, and the general progress of a team. This would assist users to make more informed decisions, detect problems at an earlier stage and less manual burden is needed to scan through the project activity. By so doing, AI element becomes a support system that enhances monitoring of the projects without ousting human supervision.

The fifth main goal is to make sure that the platform is scalable, maintainable and appropriate to enhance in the future. TeamTrack is also supposed to develop more than the first implementation phase, therefore the structure should support the future development without the need to renew the structure. Due to this reason, the system will be designed based on modularity and distinct separation of the frontend, backend, database, logging, and AI modules.

This goal will make sure that the platform has the ability to be expanded over time with more analytics, increased security, and institutional-level functionality should it be required. In general, the main goals of TeamTrack include facilitating academic cooperation, enhancing efficiency of workflow, ensuring better transparency and accountability, promoting equitable contribution evaluation, and offering AI-assisted project understanding within scalable and sustainable system. These goals define the top-level orientation of the platform and base the non-functional and functional goals that are given further.

3.3.2 Functional Objectives

The functional objectives of TeamTrack are the key system functions that will be provided by the platform to help manage the business of the academic group projects. These objectives are informed by the overall vision of the system, and presuppose the pragmatic elements required to achieve the issues prioritized in the specifics above, such as the lack of efficient collaboration tools, ineffective contribution tracking, and the lack of transparency in academic work completed as a team. As TeamTrack is supposed to be a centralized collaboration platform, the functional objectives are aimed at the key operations which allow the user to control projects, track participation, and enhance team coordination.

The secure user access and role-based interaction is one of the primary functional goals of the system. The system has to enable users to securely log-in and use the system depending on the role assigned to them. The platform should be able to allow different users, including students, team leaders, and instructors, to interact with the platform depending on their permissions. This would be done to ensure the project information is secured, as well as enabling each user to do his or her desired action within the system.

The other significant functional task is to assist project and team management in a centralized workstation. The system must be able to create and maintain project environments that project members can work together in an organized manner. In such workspaces, a team ought to be in a position to arrange project activities and control the participation and store all the pertinent project information in a single location. This contributes to less reliance on numerous disintegrated tools and develops a more structured cooperation process.

Task management is another central system functionality that TeamTrack has to accommodate. The users are expected to be able to create tasks, allocate duties, provide updates, and track the progress of the work in progress in the course of the project. This feature is required in the process of making sure that work is allocated in a clear manner and that all members can know what has already been done, what is yet to be done and who is doing what activity. Through this, task management will become one of the main operating functions of the platform.

The other useful functional purpose is to facilitate the tracking of contributions in the form of time log and activity records. The system must have features of tracking user activities, updating on the tasks, and the amount of time spent on project activities. This role aids the larger purpose of

accountability by generating observable documentations of person involvement. It is also useful in terms of the distribution of the work between the team members during the project.

Communication in the project environment should also be facilitated by the platform. The team members must have a means of sharing updates or discussing the tasks and being notified about the developments of the project without fully depending on the external communication tools. The system can maintain the discussions in reference to the project activities and minimize the discontinuity that frequently happens under the academic teamwork by facilitating communication within the same workspace.

Moreover, TeamTrack must offer dashboard and analytics capabilities that could allow users to process project and contribution data in a better way. The system must be capable of displaying systematic summaries of progress, participation, and workload trend to enable both the students and the instructors to comprehend team performance in a better way. These characteristics are particularly effective to detect the imbalance in the contribution and to enhance the activity of the project in a general sense.

Another functional goal is to facilitate AI-assisted analysis and reporting. The system must then utilize the project data it has at hand to produce useful summaries, insights and indicators to aid the users to track the team progress and detect issues like inactivity or unbalanced distribution of work. This functionality is aimed at helping the users to comprehend the conditions of projects faster and more precisely. Nonetheless, the AI aspect will be used to facilitate decision-making and not to substitute human judgment.

Lastly, the system ought to have a record of the project activities and changes to ensure that users are able to view the project as it was developed. It is significant to maintain this information to ensure transparency, accountability and referral in future. An effective project history also can be used to clear doubt on how tasks will be accomplished, the level of contribution or change that will be introduced in the project lifecycle.

The functional goals of TeamTrack are, in general, focused on the ability to provide access security, organized project management, organization of tasks, monitoring of contributions, internal communication, analytical reporting, support with the assistance of AI, and transparent project history. These functions combined establish the primary processes needed to make TeamTrack a valuable academic collaboration and accountability tool.

3.3.3 Non-Functional Objectives

The non-functional objectives of TeamTrack define the quality attributes and operational standards that the system is expected to maintain in addition to its core functions. Although the functional objectives state how the system is supposed to carry out the functions, the non-functional

objectives state how the system is expected to carry out the functions. The significance of these objectives is that the success of TeamTrack does not solely rely on the availability of features like task management, contribution tracking, and communication but also reliability, security, usability, and maintainability of the entire platform. These attributes are particularly significant in an academic setting since the system should be convenient, secure, and user-friendly by both the students and instructors.

Usability is one of the chief non-functional goals of TeamTrack. The system must offer easy to use, clear and clean interface so that users do not become unnecessarily tangled by the system and do not need training to interact with the platform. Given that the targeted users might not be equally familiar with the technicalities, the platform should show its features in a manner that is easy and user-friendly. The users are expected to access project information, update the tasks, watch the analytics, and communicate with other team members effectively. A friendly interface will be highly crucial in the ensuring that the system makes work easier rather than add more complexity to the system.

Another important non-functional objective is performance. This system has to respond to the operations of the users in a reasonable time slot and should be relied upon to support the normal project operations without causing delays to interfere with the workflow. The introduction of dashboards, the process of updating the tasks status, time logs formation, and the project progress are to be managed in a way that the platform is not considered to be complicated to work with on a day-to-day basis. Performance is especially of great importance in a collaborative environment where users want real-time or nearly real-time access to project information.

Security is the other non-functional objective of TeamTrack. The system contains user accounts, contribution records, communication data and project-related information, therefore it must help in ensuring that such information is not read or used by an unauthorized user. The authentication scheme that is applied, secure sessions, user permission control and secure management of user data must contain secure controls that will have confidence in the platform. In addition to this, horizontal communication between system layers should be strongly secured to reduce the possibility of leakage of data. Given the fact that TeamTrack is comprised of different user roles and may have sensitive academic records, security is one of the biggest quality requirements of the system.

The other non-functional objective is reliability. System should be uniform and should provide the stable availability of its functions in the case of normal utilization. They should be able to position the customers by depending on the platform to aid project work without having to crash down and lose data too often. This becomes even more crucial in the academic settings where project deadlines, history of input and proof of assessment may be based on the presence of the system and its endurance over time.

It must also be a simple system to maintain. TeamTrack will be improved because the project will be developed and that is, the platform must be setup to enable future upgrades, enhancements and debugging. Sustainable system architecture also allows the developers to change some of its

parts, without affecting the entire platform. It is especially important to TeamTrack because it involves a combination of several modules, such as front-end engagement, back-end processing, contribution analytics, and AI-assisted support. The system, and show the practical aspects that are required to achieve the problems highlighted in the detail above, which include inefficient collaboration tools, imprecise contribution tracking, and lack of transparency in academic work done as a team. This is made in a modular and structured design to provide an assurance that the system can still be enhanced as time progresses.

Another non-functional objective is scalability. Though the present scope of the project could be restricted into a smaller academic usage scenario, the system must be in a position to serve the future expansion in terms of the number of users, number of projects and system functionalities. The platform must thus be designed in such a manner that it can be extended without having to undergo significant redesign. This comes into consideration should TeamTrack be adopted in the future among more student groups, courses or departments.

Moreover, TeamTrack ought to be compatible as well as accessible. The site must be compatible with standard devices and contemporary online platforms in order to make it easily accessible to end users as a convenient addition to their regular school routine. As the students and educators might be using various systems and screen size, the interface must be functional and reliable across supported systems. The design should also be made to be accessible so that the system can serve a greater number of people and be practical to a wider audience.

Lastly, transparency and auditability should be quality objectives that are supported by the system. As TeamTrack will be focused on enhancing accountability when working in a group, the platform should maintain the records of activities in the project, history of involvement, and data related to contributions. This should be in a form that can be reviewed and understood by the users in order to use the system as a reliable source of project evidence. The non-functional goal of transparency reinforces the general credibility of the platform and contributes to the more equitable academic cooperation.

All in all, non-functional objectives of TeamTrack are to make sure that the system is usable, efficient, secure, reliable, maintainable, scalable, compatible, and transparent. These features are necessary to enable the platform to be viable and reliable in actual academic settings. The functional objectives when combined with these provide the definition of what TeamTrack should do, as well as, the standard at which it should perform.

3.3.4 Success Criteria and KPIs

The success criteria and the key performance indicators (KPIs) of TeamTrack are the tools used to consider whether the proposed system is achieving its planned targets. Although the sections above explain what the platform is supposed to do and the manner in which it should operate, this

section is concerned with the practical means through which the success of the system could be assessed. The question of whether the TeamTrack platform is technically functioning or not is not the issue of success in the case of TeamTrack, but whether it has a positive impact on the process of collaboration, whether it leads to greater accountability, whether it can reasonably track the contribution, and whether it can provide assistance in the form of a useful insight into the performance of the academic group projects.

One of the most important success criteria is that the collaboration activities should be successfully centralized in a single platform. TeamTrack will be considered effective when the users are able to maintain their project tasks, communication, time records and other associated information about their contributions easily and without the heavy use of different tools which do not integrate with one another. The other KPI that may be implemented in this objective is the number of features utilized in the system which will be the number of task updates, time log entries, in-platform communication, and interaction with the dashboard. When the functionality of the platform is used more by taking advantage of its key features, it would indicate that it is functioning as an academic collaboration space that is built into the systems.

A better visibility of team participation and project progress is another success criterion which is a major success criterion. As the development of the teamwork is a primary purpose of TeamTrack, the contributions records and development of the project should be simplified to be reviewed through the system. Their completeness of task records, the number of logged project activities, the consistency of time-tracking entries, and the availability of contribution summaries per team member are the KPIs that can be used to measure this. In case the platform is capable of giving accessible and concise participation data, then it will be fulfilling a significant role of its proposed solution.

Another criterion of success is the promotion of more just appreciation of individual contribution. TeamTrack is supposed to assist in minimizing uncertainty in determining the individual and the degree who made contributions to the project. This may be evaluated based on KPIs like the availability of personal recent histories, the rate of accomplished job attributed to definite users, and the quality of reporting on contributions created as a result of the system history. The better the system can relate the results of projects to the individual participation the better it will succeed in the support of accountability and fairness.

The success criterion is also system usability. Although TeamTrack may be effective in terms of features required, the platform cannot be successful when the users struggle to navigate and find it over-complicated to use. The corresponding KPIs related to usability are ease of navigation, results of having performed the most frequent actions, the feedback given by the user about the interface clarity, and the decrease in confusion levels when performing the routine project operations. With a positive user experience, it would demonstrate that the system is a viable one that can be used in real academics and not merely one that works on paper.

Another factor that leads to the overall success of the platform is performance and reliability. TeamTrack must react effectively to the user actions and be stable within a normal operation. KPIs

that may be possible in this area are the speed of page response, the successfulness of dashboards and project information loading, the low rate of system errors, and the availability of the necessary functions on a regular basis. When there is a smooth and reliable interaction between the users and the system, the platform will be more acceptable in terms of continued academic cooperation.

The second success criterion is effectiveness of AI-assisted insights. Given that TeamTrack has an AI-based feature of tracking and reporting projects, the system is expected to deliver meaningful and useful summaries or indicators that can be meaningful and useful to users. The quantity of AI-generated summaries produced, the relevance of generated insights when applied to project activity, and user opinion on whether generated insights can be used to make sense of the progress or contribution patterns can be relevant KPIs. The AI capability will be discussed as successful when it will be valued as an extension of the user experience and not an isolated technical feature.

There is also a need of success in security and controlled access. TeamTrack must ensure that the appropriate data is only accessed by the authorized personnel and that the user roles are addressed accordingly. The KPIs in this regard may be the successful authentication rates, role-based permissions implementation, and absence of the unauthorized access to the secured project information. This is essential considering that in regard to the system, academic records and information and data that consider contributions, it is important to make sure that the platform is trusted.

Overall, TeamTrack may be analyzed with the help of a set of pragmatic, technical, and user-specific indicators in relation to its success. These are the platform usage volume, the visibility of the history of contributions, the availability of all project activity data, the interface friendliness, the reliability of the system performance, the usefulness of AI-generated information, and the usefulness of access control. Taken together, these success measures and KPIs can give a systematic way of measuring TeamTrack as a platform of achieving its objectives as an academic cooperation and accountability engine.

4. Software Requirements Specification (UPDATED SRS)

4.1 Functional Requirements

4.1.1 User Registration and Authentication

Description and Priority

This will be the gateway to the Team Track platform that is secure. It is also configured to specifically fit an academic setting where all the contributions, time logs, and communications are

credited with verified university identities. It is a High Priority feature since it is a requirement to all the other system functionalities.

Functional Requirements

FR-1: The system will also provide the ability to open an account through the entry of full name, university email, and the use of a secure password.

FR-2: The system will ensure that the given email address uses the standard university domain format (i.e. @university.edu).

FR-3: The system will send an automated account activation email with a unique (time-sensitive) link (which will have a life span of 24 hours) to the registered email address.

FR-4: The system will solely permit activation of an account and further log-in once the user has managed to click the activation link.

FR-5: The system will have a secure log-in interface where a user will have to enter his/her registered e-mail and an encrypted password to enter.

FR-6: The system will have a functionality of Forgot Password that will enable the user to request a password reset through his registered university email.

FR-7: The system will enforce password complexity policy of at least 8 character with at least one upper case letter, one lower case letter, one number, and one special character.

FR-8: In case the user is denied access because of an incorrect set of credentials or because the account is not activated, an error message will be displayed clearly to the user in the system.

FR-9: The system will automatically end the session of a user after a time of idleness (e.g. 60 minutes) to avoid access to unauthorized information in open-to-the-world laboratory facilities.

FR-10: The system will enable users to effectively log out of the platform with the data of all the sessions being wiped off the browser.

4.1.2 Role-Based Access Control (RBAC)

Description and Priority

This option determines the groups of authorization and permissions by the various user classes within the Team Track platform. The use of RBAC will allow integrity of data and avoid illegal changes to project settings, task assignment, and contribution logs. This is a High Priority characteristic because it will uphold the core of the system, which is fairness and accountability within a university environment.

Functional Requirements

FR-12: The system will have four major user roles: Student Team Member, Team Leader, Instructor (Supervisor), and System Administrator.

FR-13: Team Leaders will be able to control project-level environments, such as workspace names, project description, and milestone deadline in the system.

FR-14: The system will give Team Leaders the unique right to add people to a workspace and allot or re-allot work.

FR-15: The system will permit Student Team Members to access all project data but will only permit them to write updates to their own Time Log entries and Task status updates to which they are assigned.

FR-16: The system will impose read-only permissions on the role of an Instructor, allowing only seeing the project progress, contribution analytics, and logs of activities.

FR-17: The system should only enable the System Administrator to have access to the entire platform in terms of management (e.g. approvals of accounts and technical maintenance) but not in terms of editing project-specific academic content.

FR-18: The system will automatically update a user interface (e.g. dashboard buttons and menu options) according to his/her active role in a particular project.

FR-19: The system will have a role-management interface that will allow the incumbent Team Leader to demote a Member to Leader position or relinquish leadership.

FR-20: The system will reject any data modification request made by a user within a user session that lacks the necessary write permissions on that particular entity.

FR-21: The system will show a standardized error prompt in case a user tries to overcome role restrictions through direct manipulation with URL.

(UPDATED) FR-22: The system will maintain a role-change history log that records all role assignments, modifications, and removals within a project workspace.

4.1.3 Project and Team Management

Description and Priority

This option allows the formation of collaborative workspaces (projects) that are centralized and the control of the team structures within these spaces. It enables users to set project boundaries, membership management and hierarchy that is necessary to set in academic accountability. This is a High Priority feature considering that this is the main container of all the other activities (tasks, logs and communication).

Functional Requirements

FR-23: The system will enable authorized users to create a new workspace by filling out a project title, a short description, and an estimated completion date.

FR-24: a system should also automatically give the role of Team Leader to the user that creates the workspace.

FR-25: The system will create a unique Project Join Code in alphanumeric that will be assigned to each new workspace to make it easy to enroll students.

FR-26: The system will enable Team Leaders to add other registered users of the university into a workspace through their university email address.

FR-27: There will be a Team Membership List of each workspace in the system, with the names, university emails, and designated role of all participants.

FR-28: The system will enable the Team Leaders to change metadata of projects (e.g., make changes to the description or a deadline) any time prior to the archiving of the project.

FR-29: The system will provide the team leaders to demote a student team member to a team leader to facilitate the common leadership models.

FR-30: The system will enable Team Leaders to fire members out of a workspace, which will strip such a user of access to the project data and retain his/her history of activities to ensure transparency.

FR-31: Within the system, each workspace will have a Project Dashboard, which shows a top-level overview of current work and a timeline of the most recent activity on the team.

FR-32: The system will have a Project Archiving feature, which will prevent any further editing to the workspace, but leave all the analytics of contributions and communication history in the assessable state.

4.1.4 Task Management System

Description and Priority

This is the main feature that drives the group work in a workspace. It enables the issue, delegation and status monitoring of individual units of work. It has both Board and List views thus assuring the visibility of the team members and leaders on the progress of the project. It is a High Priority feature because it produces the "Activity Log" and "Engagement Indicators " required by the analytics of the system.

Functional Requirements

FR-33: The system will enable Team Leaders and the Student Team Members to add tasks to an active workspace by specifying a title, description, a priority (Low, Medium, High), and a specific deadline.

FR-34: The system will enable Team Leaders to delegate to a member only one task to improve the level of individual accountability.

FR-35: The system will have a Kanban Board that will show columns of tasks organized according to their status: Backlog, In Progress, Review and Done.

FR-36: The system will include a List view where the user can sort tasks based on due date, priority or assignee.

FR-37: The system would automatically refresh the Activity Log whenever a task is created, edited, re-allocated or modified status.

FR-38: The system will enable users to add comments to certain tasks to enable the context-related communication.

FR-39: The system will include the Task Activity Indicators to identify stalling through highlighting of tasks that are not updated in a designated time period (e.g. in 48 hours).

FR-40: The system will have the ability to designate a task as being blocked, and have an obligatory text box to describe what has caused the blocking.

FR-41: The system will allow search functionality which will search specific tasks depending on the words in the title or description.

FR-42: The system will have a warning displayed on the dashboard regarding any task that has a deadline of less than 24 hours and has not been done so.

(UPDATED) FR-43: The system will allow task dependencies to be defined, enabling tasks to be linked such that one task cannot begin until another is completed.

4.1.5 Time Tracking and Effort Logging (CORE)

Description and Priority

This option enables each user of the team to document actual time used on particular activities in a workspace. The factual foundation of the analytics of the system by recording effort data using both live timers and manual logs will form the basis of the analytics of the system in terms of "Contribution Analytics" (Section 3.7), and "AI-Assisted Insights" (Section 3.8). A feature of this type of High Priority is crucial to address the issue of unequal participation that was determined in the System Overview.

Functional Requirements

FR-44: The system will also have an automated Start/Stop Timer of all tasks allocated to the user that has logged into the system.

FR-45: It will provide the system that will enable the users to manually add Manual Time Records (Date, Start Time, End Time) to record project work done out of the web platform.

FR-46: The system will enforce a mandatory textual description of work (minimum 10 characters) to every time log entry prior to it being saved in the database.

FR-47: The system will not allow a user to have multiple running active timers on various tasks at the same time.

FR-48: The system will be able to automatically estimate the length of every log and associate it with the user id and task id in question.

FR-49: The system will include a Personal Timesheet view where users are able to view a summary of the logged hours with each week and project displayed separately.

FR-50: The system will enable Team Leaders and Instructors to access Aggregated Timesheets that indicate aggregate effort put in by all the project members.

FR-51: The system will enable users to amend or remove their personal time logs in a 24-hour period (Grace Period) after which the time log will become frozen to ensure the integrity of analytics.

FR-52: The system will show a live Effort Indicator (e.g., a progress bar or pie chart) of how much time has been logged against how complicated the task should be.

FR-53: The system will produce an error message in case a manual time log has an overlap with an existing record of the same user.

4.1.6 Team Chat and Communication Module

Description and Priority

The Team Chat and Communication Module allows project members to communicate with each other within a project. This feature gives users functionalities such as sending messages related to tasks, giving updates and progress checks, and overall collaborating with each other, and ensures all the communication stays relevant to the project. Messages are stored and can be reviewed later.

Priority: High

Functional Requirements

FR-54: The system shall allow users to send text messages within a project-specific chat

FR-55: The system shall associate each message with a sender, timestamp, and project

FR-56: The system shall display message history for each project in chronological order

- FR-57: The system shall only allow access to project chat messages to project members
- FR-58: The system shall validate messages to prevent empty or invalid submissions
- FR-59: The system shall allow project members to view messages after logging out and logging back in (persistent messages).
- FR-60: The system shall support basic message formatting safety by sanitising message content before storing or displaying it.
- FR-61: The system shall provide a clear indication of who sent each message (e.g. showing the name or role of sender).

4.1.7 Contribution Analytics Dashboard

Description and Priority

The Contribution Analytics Dashboard provides information on individual and team participation within a project. The dashboard aims to improve accountability as well as showing workload between members by visualizing metrics (charts etc.) like time spent on tasks, task completion, and frequency of activity. This feature assists team leaders and instructors in finding imbalances in contribution between members and monitoring the overall advancement of the project

Priority: High

Functional Requirements

- FR-62: The system shall calculate total time spent by each user on project tasks.
- FR-63: The system shall present contribution data using visual summaries such as charts or tables
- FR-64: The system shall display task completion or status statistics per user per project
- FR-65: The system shall allow project leaders to view contribution analytics for all team members (or can all team members see the contribution)
- FR-66: The system shall derive analytics data from tasks, time log, and activity records.
- FR-67: The system shall update contribution analytics automatically when new task or time log data is recorded.
- FR-68: The system shall provide a comprehensive overall project summary display showing totals for the entire team (e.g., total hours, tasks completed).
- FR-69: The system shall allow the dashboard to display contribution data for the chosen time range (e.g., weekly view for the whole project timeline).

FR-70: The system shall highlight any signs of contribution imbalance within the team (e.g., unusually low logged time compared to the team average) as something to review

FR-71: The system shall allow authorised roles to export or download a simple summary report (e.g. CSV/PDF style output) of contribution analytics.

(UPDATED) FR-72: The system shall provide comparative contribution indicators that display each user's contribution relative to team averages within the project.

4.1.8 AI-Driven Project Assistant

Functional Requirements

FR-73: The system will include a natural language processing (NLP) interface that can be used to understand user queries about the project status, deadlines, and team activity.

FR-74: The system will be using a rule-based algorithm that will continually scan the distribution of completed tasks and recorded hours of group members to produce a score under the term of a Workload Variance.

FR-75: The system will send an automatic message to the Team Leader in case a group member has no documented activities (login, task update, chat) within a defined number of hours (default: 72 hours).

FR-76: The system will also include Smart Recommendations of task assignment which will suggest particular members of a new task, basing the recommendation on the availability and past speed of completion.

FR-77: The system will compute a Deadline Risk Probability of active tasks that will be computed by calculating the remaining time and time average of similar past tasks.

FR-78: The system will produce "Contribution Insight" reports to the Instructors which explicitly point out gaps between self-reported time logs and actual system activity (i.e. long hours logged and few artifacts produced).

FR-79: In order to be transparent, all AI generated suggestions should be explicitly labeled as System Generated Suggestions and that human approval (Team Leader or Instructor) must be received before any data is altered.

(UPDATED) FR-80: The system will allow users to request explanations for AI-generated insights, providing a breakdown of the data and logic used to produce the recommendation.

4.1.9 Notifications and Reminder System

Functional Requirements

FR-81: This system will have a centralized area called the Notification Center, which will be available in the global navigation bar and will be divided into three categories: Tasks, Mentions, and System Alerts.

FR-82: The system will implement a Multi-Channel Alerting mechanism, allowing users to configure whether they receive specific alerts via Email, In-App Popups, or both.

FR-83: The system will also use Deep Linking of all the notification whereby clicking an alert will take the user directly to the task card or chat thread of the project and not the home page.

FR-84: The system will issue the reminders about the Procrastination Prevention at the set schedule (e.g., 7 days, 3 days, 24 hours) before a milestone deadline.

FR-85: The system will automatically combine high volume low priority events (e.g., 5 users updated 10 tasks) into one summary notification so that inbox is not flooded.

FR-86: The system will make sure that urgent tasks with high priority are prioritized above user settings regarding the Do Not Disturb feature to provide urgent deadline alerts.

FR-87: The system will have a Mark as Read feature that will synchronize with the session and in case a user reads a notification on mobile, it must be flagged in read on desktop.

4.1.10 Activity Log and Audit Trail

Functional Requirements

FR-88: The system will capture an unaltered audit history of all tasks, files, and project settings Create, Read, Update, and Delete (CRUD) operations.

FR-89: Every entry in the logs ought to permanently record the following metadata: Actor Identity (user ID), Timestamp (UTCM format) and IP Address, Type of action and the actual Data Field that was altered.

FR-90: The system will also have Version Control logic of text fields (e.g., Task Descriptions) where one can see older versions of the text to retrieve information overwritten by another member.

FR-91: The system will enable Team Leaders and Instructors to use a combination of various criteria to filter the Activity Log (e.g. Show all DELETIONS by USER X in PROJECT Y).

FR-92: To maintain the integrity of data when appealing to the academic authorities, the system will not allow any user role, including Administrators, to manipulate or delete the entries of the Audit Log.

FR-93: The system will offer an "Export Audit Trail" option, which will enable the Instructors to export the complete project history in the form of a time-stamped PDF or CSV file to be used in external grade documentations.

FR-94: The system will automatically store Activity Logs of at least one academic year (or university-established period) of project activities before deleting data.

4.2 Non-Functional Requirements

4.2.1 Security Requirements

This section defines the security requirements for the system. The requirements focus on protecting user accounts, data on projects, and system functionality from unauthorised access or misuse.

Authentication and Access Control

NFR-1: The system shall require authenticated access for all users before permitting access.

This NFR ensures that only registered users are able to access projects, tasks, and collaborate with members in the system.

NFR-2: The system shall ensure that user authentication credentials are securely stored and not retained in plain-text form.

This reduces the risk of exposed credentials and violation of confidentiality, moreover, reduces this risk of unauthorised access to stored data.

NFR-3: The system shall restrict access to project data to authorised project members only.

This prevents users from viewing or interacting with projects they aren't assigned to and keeps things simple.

NFR-4: The system shall enforce role-based access control to limit actions and visibility based on user roles.

Gives access to project leaders and project members according to the features relative to their role (least privilege).

Data Protection and Activity Control

NFR-5: The system shall protect sensitive user and project data during storage and during transmission in deployment environments.

This requirement supports protecting confidentiality and integrity of stored and transmitted data.

NFR-6: The system shall log security-related events such as authentication attempts and role changes.

These logs are for basic monitoring and accountability withing the system.

NFR-7: The system shall ensure that time logs can only be created or modified by authorised project members and project leaders.

This ensured that time tracking data stays reliable while allowing flexibility for contributors and leaders to make necessary corrections.

Session and Input Security

NFR-8: The system shall expire user sessions after a period of inactivity and require reauthentication.

Reduces the risk of unauthorised access from unattended user sessions.

NFR-9: The system shall validate and sanitise all user inputs to reduce common web security risks. This helps prevent any invalid, inconsistent or malicious input from affecting the systems behaviour.

4.2.2 Performance Requirements

This section outlines the performance requirements for the Team Track system. These requirements focus on ensuring that the system remains responsive, reliable, accessible and usable under normal operating conditions.

System Responsiveness and Load Handling

NFR-11: The system shall respond to user actions (loading page, updating tasks, sending messages) within 2 seconds under normal operation.

This ensures that users experience minimal delay when interacting with the system during regular usage.

NFR-12: User authentication actions including login and logout shall complete within two seconds of submission.

Fast authentication improves usability and avoids delays during access to the system.

NFR-13: The system shall be able to support at least 30 active users at the same time without noticeable issues.

This requirement ensures acceptable performance for a typical university group project usage.

(UPDATED) NFR-14: The system shall maintain stable performance without significant degradation when handling simultaneous operations across multiple modules.

Feature-Specific Performance

NFR-15: Messages sent through the Team Chat feature shall appear for the other project members within 1–2 seconds of being sent.

This supports fast and efficient communication between project members.

NFR-16: Changes made to tasks and time logs shall be reflected in the system interface within 1 second of submission or page refresh.

This ensures that updates are not delayed, reducing confusion or any duplicate actions.

NFR-17: The Contribution Analytics Dashboard shall load and display summary information within 3 seconds or page refresh for a project.

This allows users to see contribution insights without significant waiting time.

Scalability and AI Performance

NFR-18: The system shall continue to perform reliably even as the amount of stored data increases.

This requirement supports usability as projects, tasks, and logs accumulate over time.

NFR-19: AI-generated summaries shall be returned within a reasonable response time under normal usage conditions.

This ensures that AI-assisted features remain practical and useful within the system workflow.

4.2.3 Usability Requirements

This section defines usability requirements for Team Track. The requirements aim to make the system easy to learn, user friendly, and visually consistent, allowing users to efficiently complete their tasks without any complexity.

Interface Design and Consistency

NFR-20: The system shall provide a clear and simple user interface that allows users to understand and navigate the main features simply.

This helps reduce the complexity and having to learn the platform for new users.

NFR-21: The system shall have a consistent layout and format including visuals, colours, and terminology across all the pages.

Consistency improves usability and reduces user confusion.

Task Efficiency and Feedback

NFR-22: The system shall allow users to complete tasks such as creating tasks, logging time, and sending messages within a few steps.

This ensures that common actions can be completed quickly and efficiently.

NFR-23: The system shall provide clear feedback messages for user actions, including successful submissions or any errors.

Feedback helps users understand the outcome of their actions and correct mistakes when needed.

(UPDATED) NFR-24: The system shall minimise the number of steps required to perform frequent actions such as task updates, time logging, and communication.

Accessibility and Readability

NFR-25: The system shall function on typical desktop and laptop displays.

This ensures compatibility with commonly used devices in an academic environment.

NFR-26: The system shall display all readable text and appropriately sized interface features.

This supports readability and overall ease of use.

4.2.4 Availability and Reliability Requirements

Availability Requirements

NFR-27: The system will be accessible at least 99 percent of the time without taking into consideration the planned maintenance.

NFR-28: Scheduled maintenance actions will be announced in advance (more than 24 hours).

NFR-29: The system will enable graceful degradation which ensures that core features are made available when the system is partially unavailable.

Reliability Requirements

NFR-30: The system will be able to withstand unforeseen failures in less than 5 minutes without any data loss.

NFR-31: The system should have automated data backups at least twice in 24 hours.

NFR-32: The system will record system failures and critical mistakes to assist in diagnostics and recovery.

(UPDATED) NFR-33: The system shall ensure that no user data is lost during unexpected failures by maintaining transactional consistency.

4.2.5 Data Integrity and Privacy

Data Integrity Requirements

NFR-34: The system must also guarantee that the information held in the system is precise, comprehensive, and congruent throughout the life of the system.

NFR-35: The system will have access controls that will set it to follow that user and project data are not modified by unauthorized individuals.

NFR-36: The system will be capable of verifying all the user inputs to reduce any chances of data corruption or inconsistency.

Data Privacy Requirements

NFR-37: The system will provide usage controls on personal and project-specific data defined by user-identity and permissions.

NFR-38: Authentication details of the user will be stored in an encrypted or hashed form in the system.

NFR-39: The system will process the personal data by complying with the principles of data protection and privacy, such as minimization and limitation of purposes.

4.2.6 Ethical and Responsible Use Requirements

NFR-40: The system should not automatically make decisions that affect users adversely in the absence of human supervision.

NFR-41: The system will provide transparency to automated recommendations or system-generated insights that are available to users.

NFR-42: The system will not have any biased behavior; resources will be made to use the same logic for all users.

NFR-43: Where it is appropriate, the system will ensure that the user has a clear explanation of how the system-generated output is obtained.

(UPDATED) NFR-44: The system shall provide users with the ability to review and override AI-generated suggestions before any action is applied.

4.3 System Constraints

The following constraints define the limitations and conditions under which the Team Track system must operate. These constraints are based on network requirements, platform compatibility, hardware limitations, development scope, and implementation decisions.

4.3.1 Network and Deployment Constraints

The system will need to have an active internet connection to access the system and support real-time features such as communication and notifications.

Team Track will operate within the normal network environment of the university and will be used only by academicians.

In the first release, mass commercial implementation will not be addressed, allowing the system scope and performance expectations to remain aligned with educational goals.

4.3.2 Operating System Constraints

Team Track will be offered as web based application, which will be compatible with a broad spectrum of operating system using the modern web browsers. The system is browser-based and therefore needs no operating system installation as well as platform specific installation.

Supported environments-

- Windows (Windows 7 to Windows 11)
- macOS (modern versions with browser support)
- Linux distributions supporting modern browsers

4.3.3 Hardware Constraints

These requirements provide a responsive and smooth interface to basic academic uses. The system will be configured to run with minimum hardware requirements so as to be accessible in academic settings:

- Processor: Dual-core processor or equivalent
- RAM: Minimum 4 GB
- Storage: Minimal local storage required (data handled server-side)
- Display: Minimum resolution of 1280 × 720
- Input Devices: Keyboard and mouse or trackpad

4.3.4 Academic Time and Resource Constraints

The system is to be finished in a limited period of Capstone project (about four months). The main aspects of core development will include the following options of the task tracking, time tracking, communication, and contribution analytics.

Advanced enterprise-scale features will be left to future development.

The development team will prioritize delivering a stable Minimum Viable Product (MVP) before adding optional features.

4.3.5 Technology Stack Constraints

These limitations allow the system to be implemented using technologies appropriate for student development while maintaining scalability and structure. The system will use a web-based architecture with well-supported technology to maintain the system and make it accessible.

- Backend: Python (Django + Django REST Framework)
- Frontend communication via HTTPS API requests
- Database: Relational database using Django ORM (PostgreSQL/SQLite)

4.3.6 Security and Role-Based Constraints

These are because access control is essential because of the work with sensitive academic contribution information. These constraints will ensure fairness and not allow abuse and misuse and maintain integrity in the academic environment.

- Users can only access projects to which they belong
- Role-Based Access Control (RBAC) must be enforced
- Contribution analytics must remain informational rather than judgmental

4.3.7 AI Feature Constraints

AI functionality is limited by ethical and academic considerations, these following constraints ensure that AI is used responsibly and not relied on too much for automation and decision making:

- AI will only provide recommendations and summaries
- AI will not automate grading or make evaluative decisions
- AI outputs must remain transparent and explainable

4.3.8 Deployment and Scalability Constraints

It is a small-to-medium university-friendly system, as opposed to large-scale business settings.

- Expected concurrent users: 10–20 active users
- Deployment within standard web server environments
- Full real-time features (e.g., WebSockets) may be deferred to future releases

4.4 User Classes and Roles

Team Track will be designed to have several classes of users who have different roles and patterns of interaction.

4.4.1 Student Team Members

The main users of the system will be the student team members. We presume that they would be digital-literate and conversant with web-based platforms. Their primary inter-relationships will involve fulfilling assigned duties, recording the time spent in activities, and being part of the project communication, as well as going through the summary of individual contribution as a reminder of their role and at-team progress [16].

4.4.2 Team Leaders

Student leaders will be the team leaders that will organize group activities. They will also have the ability to control project settings, allocate and give priority to tasks, and view high-level contribution analytics to facilitate balanced participation and effective coordination during the project life cycle besides their normal member capabilities [16].

4.4.3 Instructors or Supervisors (Read-Only Access)

Teachers or managers can be provided with optional read-only permission to see the general progress of the project and summary of project contributions. We will delineate this position to offer supervision and transparency without being able to directly interfere with workforce within teams to promote equitable evaluation in respect of student autonomy [15], [16].

4.4.4 System Administrators

The administration of the platform, user accounts, as well as the availability and security of the system will be the responsibilities of system administrators. We will make sure that this user group does not participate in the project implementation or academic evaluation, and there is no ambiguity between system administration and educational processes [15], [16].

User Class	Description	Key Responsibilities
Student Team Member	Primary system user	Complete tasks, log time, communicate, view contribution summaries
Team Leader	Student coordinator	Assign tasks, manage project settings, review team analytics
Instructor / Supervisor	Optional read-only user	View project progress and contribution summaries
System Administrator	Technical user	Maintain system, manage accounts, ensure security and availability

Table 102: User Classes and Roles

4.5 SRS Updates and Revisions

This document represents the fourth version of the SRS. Earlier requirements work was compared against the more recent design and architecture direction of the project, and the refinements came out of that process. Some expectations became clearer in the later SAS, especially around security, role reinforcement, contribution usability, and responsible AI usage. Those areas needed to be more explicit.

The system base itself has not changed. But for the final design, implementation, and testing stages to hold up, a few requirements required more precision. This way nothing important gets missed while still keeping the overall structure intact.

4.5.1 Functional Requirements Revisions

The functional requirements were updated by adding five new requirements into areas where the architecture and interaction planning showed a clearer system need. These additions mainly enhance authentication, accountability, workflow management, contribution comparison, as well as explainability of AI-aided outputs.

Requirement ID	Added Requirement
FR-22	The system will maintain a role-change history log that records all role assignments, modifications, and removals within a project workspace.
FR-43	The system will allow task dependencies to be defined, enabling tasks to be linked such that one task cannot begin until another is completed.
FR-72	The system shall provide comparative contribution indicators that display each user’s contribution relative to team averages within the project.
FR-80	The system will allow users to request explanations for AI-generated insights, providing a breakdown of the data and logic used to produce the recommendation.

Table 103: New Functional Requirements

Explanations for each addition:

FR-22 was a result of the later architecture and RBAC discussions where it was more clear that we needed better accountability around permissions. We already had roles like Team Leader, Member, Instructor, Administrator and different access levels for each was a given, but it was important to make sure that changes to those roles should not happen invisibly.

So a role-change history log keeps things transparent. If someone later disagrees about who had permission to do what is inside a workspace, there is a record.

For workflow, **FR-43** came in because the initial task management content had a gap. Task creation, assignment, comments, status changes, alerts, all of that was covered. We needed to include explicit representations of dependencies between the task, that was missing. As the design got more detailed, supporting linked tasks became an obvious need. Academic projects are interconnected items, one task often waits on another to finish first. So this requirement makes the planning side feel less like a simple checklist and more like actual project management.

FR-72 came from realizing the contribution analytics may be too surface-level. The system already calculated totals and basic contribution info, but numbers alone do not tell you much when you are trying to spot contribution imbalance. So comparative indicators were added, now you can see whether a member's contribution is balanced, below team average, or above. That small shift gives the analytics some real context instead of just displaying data.

FR-80 was added to go along with the AI component becoming better defined. Supporting students and instructors with insights and recommendations was always the goal, but no one wanted an unexplained black box. So, we approached it by letting users ask for explanations on AI-generated insight. That adds transparency and builds trust and fits the academic setting where recommendations should make sense and be valuable.

4.5.2 Non-Functional Requirements Revisions

Later milestone work brought some quality expectations into sharper focus. That led to revisions in the non-functional requirements. The additions mainly touch on security, performance stability, usability, reliability, and how AI is used ethically.

Requirement ID	Added Requirement
NFR-14	The system shall maintain stable performance without significant degradation when handling simultaneous operations across multiple modules.

NFR-24	The system shall minimise the number of steps required to perform frequent actions such as task updates, time logging, and communication.
NFR-33	The system shall ensure that no user data is lost during unexpected failures by maintaining transactional consistency.
NFR-44	The system shall provide users with the ability to review and override AI-generated suggestions before any action is applied.

Explanations for each addition:

NFR-14 came from the fact that this platform is not just a single isolated module, a lot of components live in this platform, not just one isolated module. Project management, communication, time logging, analytics, notifications, AI-assisted support, all of it is bundled together.

This requirement is basically there to make sure the system doesn't start struggling when many people are using the features at once, stable performance is not optional with many components.

NFR-24 builds on usability. Previous requirements already touch on clarity and feedback, but the workflow side needed to be more explicit. TeamTrack is meant for daily use that's expected, updating tasks, logging time, checking communication, and if those actions take too many clicks, people will stop using the system consistently, so this requirement keeps common actions efficient and straightforward.

NFR-33 was added to improve reliability, especially because TeamTrack relies on logged records as evidence. From the time logs, contribution data, activity history losing any of that directly hits transparency and accountability. The requirement came in to enforce transactional consistency and protect against data loss. Unexpected failures are not minor here.

NFR-44 was introduced to reinforce responsible AI use. The system already positions AI as assistive rather than authoritative, but this new requirement makes sure that users stay in control.

When recommendations come through, people need to be able to review and decide whether to apply them. That is what this requirement ensures. That way, the AI is not acting as an automatic decision-maker.

5. Technology Infrastructure Specification (TIS UPDATE)

5.1 Technology Stack Selection

Here we will select the TeamTrack technology stack and explain why each of the key technologies in the system is selected. This is not supposed to follow the past TIS with all its details, but just to provide a very brief implementation-oriented summary of the technologies that will define development in the future.

TeamTrack is an academic collaboration platform designed to be a web-based application that is intended to be an integrated project management system, task tracking system, intrinsic communications, contribution analytics, and AI recommendations. Because of this, the stack must be based on the responsive user experience, a secure and stable backend processing, a structured storage of the relational data and AI future-proof integration. Technical direction of the project had already been established previously in the milestone work through Nuxt.js, FastAPI, PostgreSQL, Ollama/LLAMA and Firebase-based authentication and these options are again viable to deliver Milestone 4.

5.1.1 Frontend: Vue.js / Nuxt.js

The frontend will be developed in Vue.js application and Nuxt.js as the base application framework in TeamTrack. It is a right choice because TeamTrack requires a friendly and clear interface on dashboards, task boards, project pages, chat panels, analytics displays and user views depending on roles. Such a system as Nuxt.js promotes the development of components, explicit page navigation, re-usability of user interface design, and easier long-term maintenance.

It is also appropriate because frontend can be arranged in a more sensible way with the help of Nuxt.js, than when working with plain HTML, CSS and JavaScript. Since TeamTrack is made up of multiple related pages and user flows, the framework can better support state and is more modular and becomes cleaner as the platform expands. This has been consistent with the earlier TIS decision that has made Nuxt.js the preferred frontend technology, having reviewed options.

The target baseline can be thought of as:

These version options are recommended as a baseline to maintainability and compatibility, and there may be particular patch versions agreed upon in implementation.

The major arguments to apply Vue.js / Nuxt.js.

- supports UI design in components.
- appropriate to academic workflow systems on more than one page.
- improves maintainability and reusability.

- Is effective with dashboard style interfaces.
- promotes additional growth of UI, but not redesign.

Table 104: Frontend Technology Selection Summary

Item	Selected Technology	Role in TeamTrack	Reason for Selection
Frontend Framework	Vue.js / Nuxt.js	Builds the user-facing web application	Supports reusable components, routing, structured UI development, and maintainability
UI Layer	HTML, CSS, JavaScript through Nuxt	Renders screens and user interaction	Provides responsive and modular interface implementation
Runtime Baseline	Node.js LTS	Supports frontend development and package management	Stable environment for modern frontend tooling

5.1.2 Backend: FastAPI

TeamTrack backend will be written in fastapi using python. The system is appropriate with the choice because TeamTrack uses formal APIs to verify the user, manage projects, perform tasks, record time, facilitate messaging, generate analytics, as well as request AI. A backend framework must then be lightweight, modular, secure and capable of managing a range of service endpoints in a readable and maintainable way.

FastAPI is the appropriate option because it has API-first, a good request validation, and a good support of the latest backends. In a project like TeamTrack, especially, it is especially fitting that the frontend, database, and AI module are based on a standardized API communication. Python and FastApi were the two choices of backend preferred by the previous milestone work because of their maintainability, performance, and easy development.

The implementation base could be:

- **Python 3.11 or later**
- **FastAPI**
- **Uvicorn** as the development ASGI server
- **Pydantic** for request and response validation

These choices represent the openness of the backends, the aggregation of the endpoints in a modular manner, and simplified integration with analytics and AI services.

The main motives of FastAPI.

- good compatibility with development of REST API.
- more superior than unstructured validation.
- to modular backend services.
- pythonic AI tooling works well with it.
- semantically maintains cross-module clean services.

Table 105: Backend Technology Selection Summary

Item	Selected Technology	Role in TeamTrack	Reason for Selection
Backend Framework	FastAPI	Provides RESTful API endpoints and business logic	Lightweight, structured, scalable, and suitable for Python-based service development
Language	Python	Core backend programming language	Good readability, rapid development, and strong ecosystem support
API Server	Uvicorn	Runs FastAPI during development/testing	Common and efficient ASGI server for FastAPI-based applications

5.1.3 Database: PostgreSQL

TeamTrack will be using PostgreSQL as the primary relational database. It would be the appropriate choice because the platform has very structured data items such as users, projects, project members, tasks, time logs, messages, notifications or activity logs. These are objects that have explicit relationships and constraints and so a relationship database is more suitable than a document-oriented counterpart.

PostgreSQL is also structured consistent in storage, information integrity, relational modelling, and future reporting as well. Good relational skills of a database are also important because TeamTrack also needs proper contribution records, audit history and analytics preparation. The choice to use PostgreSQL as the database tool that will be used was already made by the previous TIS, and ERD and schema planning also provided reinforcement to the choice with the assistance of SAS.

Appropriate development base can be:

- PostgreSQL 15 or later

- structured schema was consistent with the adopted ERD.
- deployment of schema based on migration.

The key reasons why PostgreSQL is used.

- full support on relational data models.
- is befitting to constraints and integrity rules.
- solid analytics and reporting requests.
- where records of past occupations and information on educational undertakings are required.
- is consistent with the existing ERD and schema planning.

Table 106: Database Technology Selection Summary

Item	Selected Technology	Role in TeamTrack	Reason for Selection
Primary Database	PostgreSQL	Stores structured operational data	Strong relational support, integrity constraints, and analytics-readiness
Data Model Basis	ERD / Schema from SAS	Defines entity relationships	Ensures database design remains aligned with the approved architecture

5.1.4 AI Module: LLaMA via Ollama

TeamTrack AI will be offered by LLaMA with Ollama. This kind of decision is a reflection of the project objective of giving smart and clear advice based on the project activity data such as the identification of imbalance in workload distribution, over-inactivity, and delayable activities. As early as TIS work had been done previously, LLaMA 3.1 was already the direction that was selected to execute the integration of AI with the assistance of Ollama.

This approach is correct because TeamTrack does not require the element of AI to replace the decision of the user. Instead, the AI is an assistant that makes observations or recommendations in regard to project records, which are organized. This is why locally controlled/mediated model integration based on API is a more viable approach, compared to a completely autonomous intelligent system.

A good point to begin implementation can be:

- **LLaMA 3.1**

- **Ollama** for local model orchestration and testing
- backend-mediated prompts and structured activity summaries as input

This setup is useful for experimentation and prototyping, while also allowing the system to maintain human oversight over all AI-generated insights.

Key reasons for using LLaMA via Ollama

- supports **AI-assisted recommendation generation**
- fits the project's **human-in-the-loop** design
- useful for **local experimentation and development**
- integrates with the Python backend in a manageable way
- avoids over-automation in an academic setting

Table 107: AI Technology Selection Summary

Item	Selected Technology	Role in TeamTrack	Reason for Selection
AI Model	LLaMA 3.1	Generates assistant-style recommendations	Supports insight generation without replacing user judgment
AI Runtime / Access Layer	Ollama	Handles local model execution or testing integration	Practical for prototyping and manageable AI orchestration

5.1.5 Authentication: Firebase / OAuth

To perform authentication, Firebase Authentication and/or OAuth-based identity flow will be used as the direction of authentication in TeamTrack. This is as compared to previous milestone planning where authentication and authorization were considered to be key system tasks with role based access control. The system should have a safe way of dealing with sign up, logging in, user identity, as well as guarded access among the students, team leaders and instructor accounts.

Firebase is suitable since it can streamline the identity management, the use of tokens as well as the support of secure sessions. Flow chains that are compatible with OAuth are also handy in situations where the third-party sign-in is required to be secured. Authentication in TeamTrack is

not merely a matter of access to the platform, but also a matter of making sure that the actions of every user, his/her assignments and logs are attributable and secure.

A suitable baseline can be:

- Firebase Authentication
- Sign-in flow of OAuth 2.0 compatibility.
- verify token at the back-end.
- RBAC implementation within backend services.

The important reasons to use Firebase / OAuth.

- makes secure identity and login simple.
- supports authentication by tokens.
- suits well with role-based access enforcement.
- lessens the complexity of custom authentication.
- improves academic project development maintainability.

Table 108: Authentication Technology Selection Summary

Item	Selected Technology	Role in TeamTrack	Reason for Selection
Authentication Provider	Firebase Authentication	Handles user sign-in and identity verification	Simplifies secure authentication management
Identity Flow	OAuth-style token flow	Supports protected access to APIs and sessions	Secure and scalable identity handling
Authorization Model	Backend-enforced RBAC	Controls access by role and ownership	Keeps permission logic centralized and consistent

5.1.6 Overall Stack Summary

The TeamTrack technology stack in general has been chosen to achieve structure development, maintainability, security and extensibility in the future. The choice of technologies is also similar to the ones made in the previous TIS and SAS documentation. The combination of these tools will

provide a viable full-stack base to construct the platform in an academically viable and implementation-ready manner.

Table 109: Overall Technology Stack Overview

Layer	Selected Technology	Main Purpose
Frontend	Vue.js / Nuxt.js	User interface, page routing, dashboards, interaction flows
Backend	Python / FastAPI	API development, business logic, validation, integration
Database	PostgreSQL	Relational storage for users, projects, tasks, logs, and messages
AI Module	LLaMA via Ollama	AI-assisted recommendations and insight generation
Authentication	Firebase / OAuth	Secure login, identity management, token-based access control

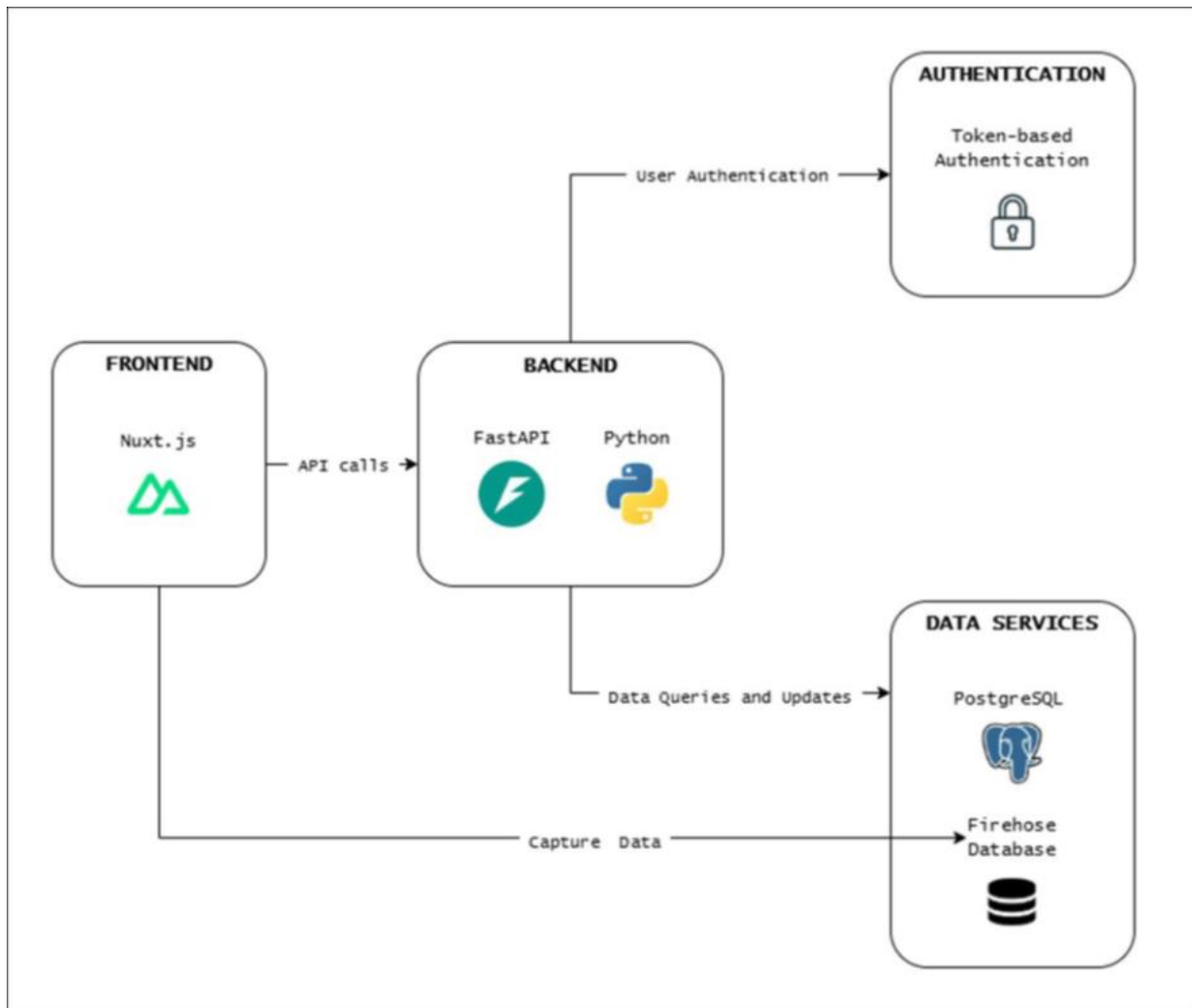
5.2 System Architecture Design

This section is a summary of the design-level artefacts that characterize the structure of TeamTrack. These artefacts of architecture are not new in this milestone of the handover but the artefacts already developed in preceding milestones are being reused as approved design references instead of developing them afresh. The primary intention behind this is to approve the diagrams and design views that will be implemented and handed over.

5.2.1 System Architecture Diagram

The System Architecture Diagram provided in the SAS will be reused in this handover as the major high-level reference of architecture. It presents a stratified interplay of the frontend and backend services, data store, authentication controls, and AI support services. The importance of this diagram is that one can see the structure of the platform at a glance and can see the logic behind separating the major modules.

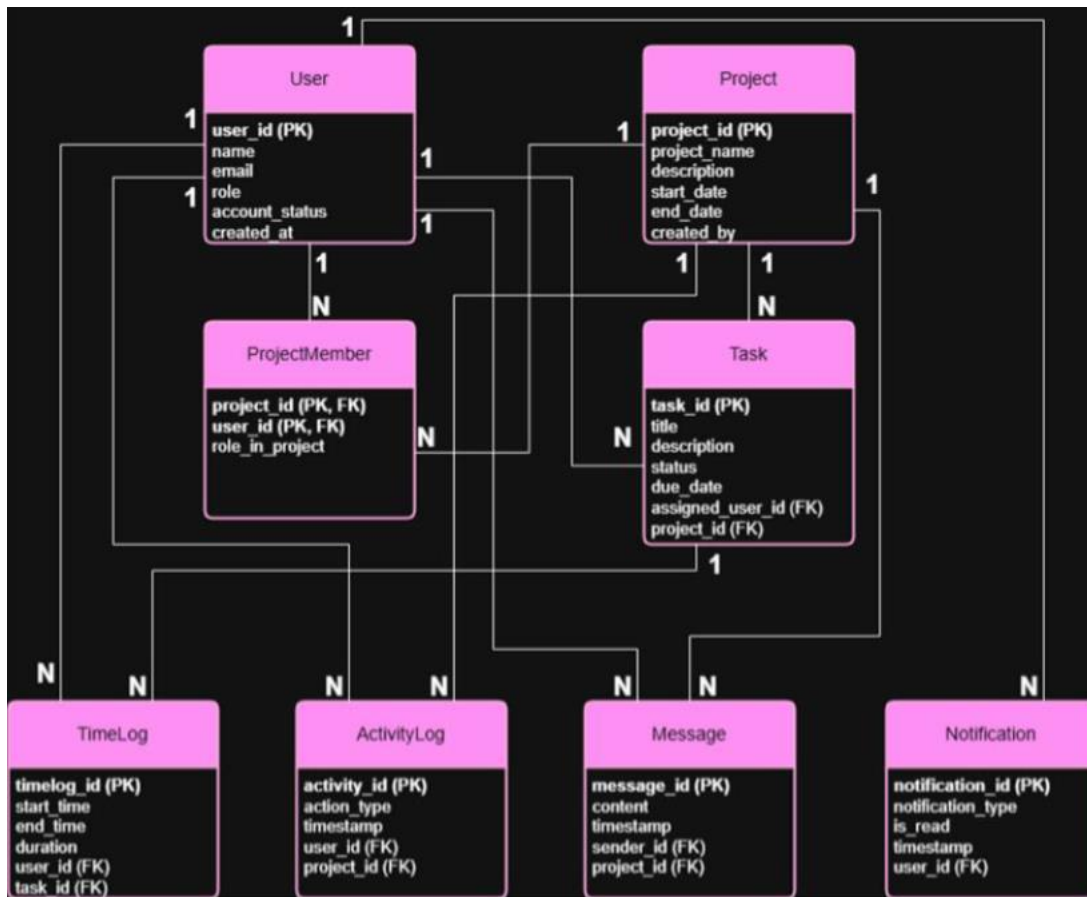
Figure 10: System Architecture



5.2.2 Database Design (ERD / Schema)

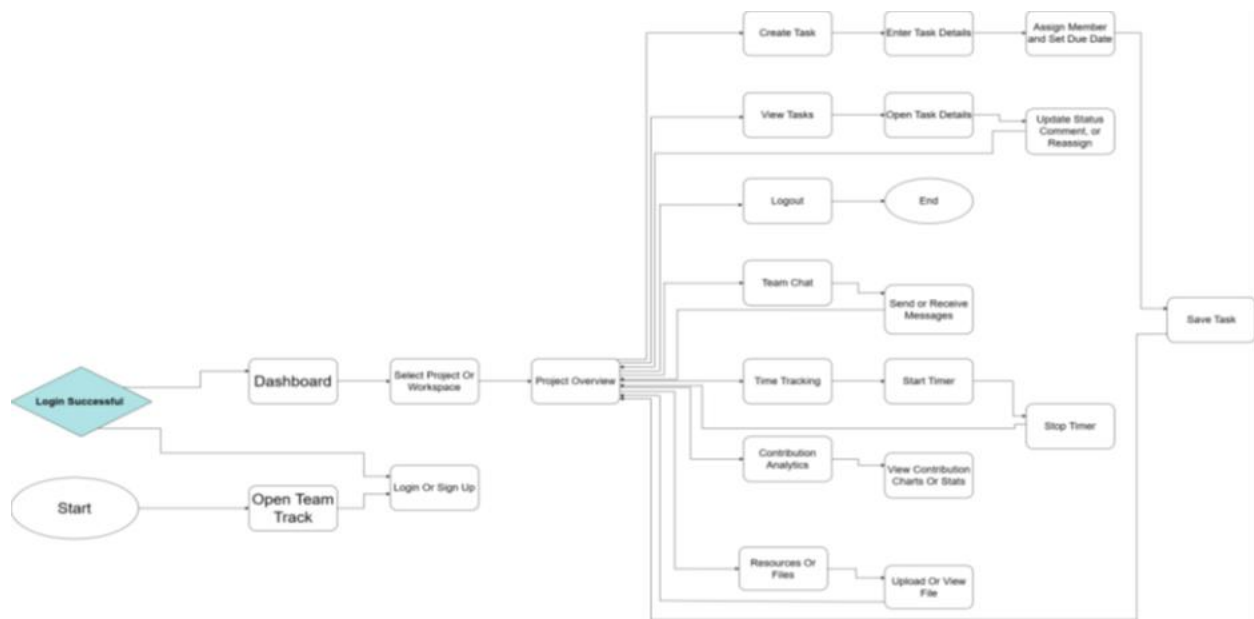
The schema design / ERD that was created in the milestones that preceded this one will also be reused. The schema design / ERD created during the milestones that came before this one will be reused as well. The design has already established the PostgreSQL database of choice and establishes the primary relational objects required by TeamTrack which are users, projects, tasks, messages, time logs, notifications, and activity records. Using the same ERD makes it possible to have consistency, and it does not have a design drift between the milestone documents.

Figure 11: Database Schema



5.2.3 User Flow Diagram

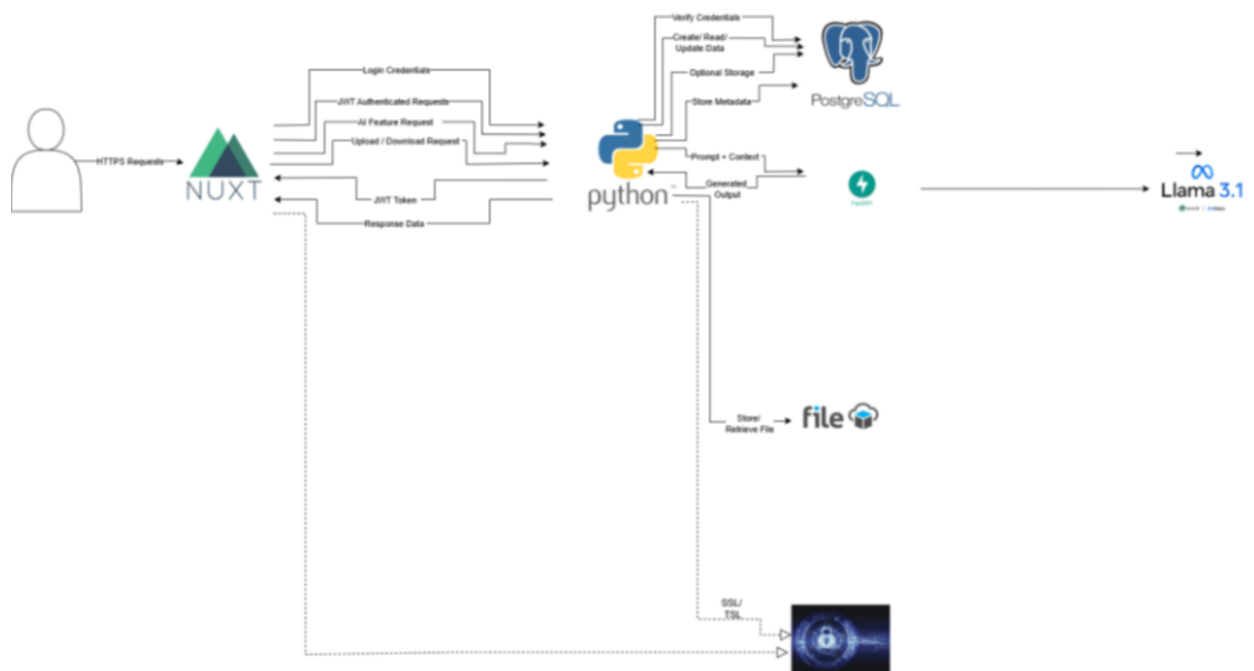
The User Flow Diagram can be also applied to Milestone 4 as TeamTrack is based on the task-based user journeys. The diagram helps to comprehend the flow of the users in authentication, access to the dashboard, access to projects, task handling, chat, analytics, and reviewing pages. It remains as an interface and reference of usability in future development.



5.2.4 System Flow Diagram

The System Flow Diagram will also be reused since it demonstrates the flow of requests among the user interface, the backend services, database, and the AI support functions. It is particularly handy in such a system as TeamTrack where various modules are reliant upon a clearly defined communication among layers.

Figure 12: System Flow Diagram



5.2.5 Architecture Design Summary

The artefacts that have been used in designing are not present in this handover in the form of illustrations. They give a solid architectural foundation to further implementation and assist in ensuring that all the future work can be traceable to the previously accepted design. The project will remain coherent throughout the milestones and, nevertheless, through the assistance of the same diagrams, the written description of this handover might be rewritten and revised.

Table 110: Architecture Artefacts Reused in Milestone 4

Artefact	Source Document	Purpose in Handover
System Architecture Diagram	SAS	Shows high-level structure and component interaction
ERD / Schema	SRS / SAS	Defines core relational data model
User Flow Diagram	TIS	Shows user journey across the platform
System Flow Diagram	TIS	Shows request and data movement between major layers

5.3 Security and Performance Considerations

This section discusses the considerations of the main security and performance factors that underlie the technology decisions of TeamTrack. Since the system operates on the basis of user accounts, project data, record of activities, communications in the team, and contributions tracking, the technologies to be employed must guarantee the presence of secure access control as well as the presence of consistent and effective system behaviour. The following section will prove that the chosen stack is functional, security and operational.

5.3.1 Security Considerations

Security is one of the key demands in TeamTrack since the platform contains the user-related information, project content, logs of contribution, messages, and history of activities. The fact that various users have varying responsibilities as well as levels of visibility implies that the system needs to be able to provide access to the relevant information and functions that should be seen by only authorized users. Authentication, role-based access control, secured communication and secure treatment of project data were already brought out as key requirements in earlier milestone documents.

Security is particularly relevant in TeamTrack as the platform is supposed to facilitate academic accountability. This implies that the information presented to instructors, team leaders and student members should be precise, traceable and not accessed or altered by other parties.

Main security priorities

- authentication of users and validation of identities.
- regulated access according to the role of the user and membership of the project.
- security in data of task, message, and contribution.
- secure communication between front and back end.
- denying unauthorized direct access to a database.

5.3.2 Authentication and Access Control

TeamTrack will use the authentication model that is based on Firebase Authentication and/or OAuth-based access flow and the authorization logic will still be implemented in the backend. This separation is helpful as authentication is considered to identify who the user is whereas authorization is considered to identify what the said user can and cannot do within the platform.

As an example, a team leader can be permitted to delegate and control the workspace, whereas a typical team member could only update and communicate assigned work in the project environment. An instructor can only get access to the selected project summaries in a read-only mode. This is because even when authentication is undertaken by some third-party identity service, it is necessary to have backend-enforced RBAC.

Planned access control direction

- authentication done by secure authentication provider.
- session verification by use of tokens.
- RBAC implemented on backend API.
- checked before secured actions checked ownership and membership of the project.
- read-only access to authorized views only allowed to an instructor.

5.3.3 Data Protection and Secure Handling

The technology stack selected also facilitates the safer management of project structured information. PostgreSQL provides relational integrity and controlled storage, and FastAPI is to make sure that the requests could be validated prior to processing being performed on data. This will reduce chances of inconsistent records, invalid submissions or unsafe interactions between the client and server.

TeamTrack issues data protection, which is not only a technical issue, but also an academic one. As the platform is used to track effort, task history, and contribution summaries, caution should be observed so that the records can be reliable and can only be accessed by the right users.

Main data protection measures

- only backend access to operations on database.
- checking of requests prior to storage.
- concerned update logic on sensitive entities.
- archiving of history and contribution books.
- safe sharing of user session tokens and identifiers.

5.3.4 Performance Considerations

Another significant aspect of the TeamTrack design is performance, which is not an enterprise-scale throughput requirement. Rather, the primary performance objective is to make sure that the application can be responsive, understandable and stable enough to be used in the normal academic environment.

The stack adopted will help this need in realistic means. Nuxt.js can be used to structure the front end effectively, Fast API can handle lightweight back-end endpoint tasks, and PostgreSQL can be used to perform project and task data in structured and consistent queries. Collectively, all of these technologies will help to facilitate a seamless experience when performing typical tasks like glancing at dashboards, updating tasks, reviewing analytics, and recording work.

Main performance priorities

- reactive interface navigation
- effective API communication between the frontend and the backend.
- organized search of project and contribution information.
- a reasonable system load when used by normal multi-user academics.
- assurance of further development of features.

5.3.5 Scalability and Maintainability Considerations

Even though TeamTrack is being constructed with capstone requirements in mind, the selected technologies must also allow further growth and easier maintenance. This is among the reasons why modular structures and distinctly divided layers have been selected. It is simpler to maintain the system in cases where the UI logic, backend services, authentication, and database operations are separated.

It is also useful in case further refinements are required, like the introduction of notifications, more powerful analytics or enhanced instructor dashboards or enhanced AI driven insights. The choice of widely-used and structured technologies will help the team to decrease the possibility of significant redesign in the future.

5.3.6 Security and Performance Summary

In general, our TeamTrack security and performance direction is founded on practical academic system requirements. The technologies chosen assist in ensuring a secure user access, controlled data processing, interactive reaction, and sustainable development. Instead of being optimized to achieve useless complexity, the platform is built to keep it secure, stable and comprehensible in the environment where it is expected to be used.

Table 111: Security and Performance Considerations Summary

Subsection	Focus Area	Main Direction
5.3.1	Security Considerations	Protect user, task, and contribution data through controlled system access
5.3.2	Authentication and Access Control	Use Firebase/OAuth for identity and backend RBAC for permissions
5.3.3	Data Protection and Secure Handling	Use validated backend processing and protected relational storage
5.3.4	Performance Considerations	Support responsive UI and efficient API/data operations
5.3.5	Scalability and Maintainability Considerations	Keep the system modular and easier to extend in later stages
5.3.6	Security and Performance Summary	Summarizes the role of security and performance in the chosen stack

5.4 Technology Investigation and Justification

This section is a description of the rationale of the technologies chosen in TeamTrack. The reason is not to repeat the entire comparison material that was in the previous milestones but in a brief

and implementation focused manner, explain why each of the technologies is still suitable to the project. The chosen tools should be compatible with each other since TeamTrack is a combination of project management, contribution analytics, and AI-assisted support.

Prior milestone work had already explored the possibilities and gave a rationalization of the ultimate path of the system. In the case of Milestone 4, the task is to reformulate the same decisions in a more targeted handover document so that the reviewer can have a clear cut picture of why each technology has been held back.

5.4.1 Frontend Technology Justification

The choice of Vue.js / Nuxt.js can be explained through the nature of the interface of TeamTrack. The system requires a few interactive pages in the form of dashboards, project workspaces, task boards, chat views, analytics screens, and role-based navigation. These features can be built and maintained easily using a component based framework.

Nuxt.js is specifically appropriate as it offers well-organized project structure, routing, reusable components, and better scalability of UI. In a system that is bound to keep on changing in the implementation process, this simplifies the frontend as compared to a simpler approach of a front end made of simple statical pages.

Why the frontend choice is suitable

- supports used and organized UI design.
- appropriate in systems that need heavy dashboard.
- not as difficult as ad hoc frontend code.
- facilitates the further development of user processes and designs.

5.4.2 Backend Technology Justification

The choice of FastAPI depends on the necessity to have a clean backend architecture that can serve numerous service domains, including authentication, project administration, task management, time logs, analytics, and AI implementation. Because TeamTrack relies on the steady API communication, the backend technology should facilitate the development of modular endpoints and high-quality validation.

FastAPI can be used since it is lightweight and modern and supports Python-based service logic. It also has the support of integrating with a data model and request validation, so that a dependable backend structure can be constructed more easily.

Why the backend choice is suitable

- works well for REST-style backend services
- supports modular API grouping
- integrates naturally with Python-based logic
- suitable for validation-heavy systems

5.4.3 Database Technology Justification

PostgreSQL is selected based on the data of TeamTrack. The entities associated with the system are closely related including the users, projects, project members, tasks, logs, messages and notifications. The relationships need to be stored in a structured way, be referentially integrated and have consistency between records.

PostgreSQL has thus been an appropriate decision since it has relational schema design, constraints, and reliable query behaviour. It is also a good basis of reporting and data retrieval concerning analytics.

Why the database choice is suitable

- fits a relational ERD-based design
- supports strong data consistency
- handles structured academic workflow data well
- supports future reporting and analytics needs

5.4.4 AI Technology Justification

The application of LLaMA through Ollama is explained by the AI aim of the project. TeamTrack is not an AI-based application to make final decisions on user behalf. Rather, it applies AI to create observations and suggestions relying on contribution and activity statistics. This renders a recommendation-based AI layer to be suitable as compared to an intelligent system with high autonomy.

The application of LLaMA with the help of Ollama enables a controlled experiment and combination, as well as enables the system to retain human control over all AI-generated products. This is equivalent to the academic context of TeamTrack where transparency and equity are valued.

Why the AI choice is suitable

- aids the dependent and not the independent intelligence.
- practical to be recommended and insight generative.
- consistent with the Python backend direction.

- is within the responsible AI approach of the project.

5.4.5 Authentication Technology Justification

The necessity to provide secure and manageable identity handling is the reason why Firebase Authentication / OAuth-style identity flow is used. A completely custom authentication system would further complicate the project even when project-level permissions and safeguarded user actions are already required by the system.

With an effective authentication service, a team can pay more attention to application logic and still provide safe login and verification through the use of a token. This simplifies the overall system and minimizes risk in development in relation to authentication.

Why the authentication choice is suitable

- simplifies the process in comparison to completely custom login systems.
- supports identity token identity manipulation.
- supports RBAC that is front-ended.
- assists in making authentication maintainable and reliable.

5.4.6 Overall Stack Justification

The chosen technologies, combined with each other, create a harmonious and feasible complete solution of TeamTrack. All of the technologies complement each other, and they are also needed to support a particular aspect of the system. The front-end framework will enable well-organized user interaction, the backend will enable service implementation to be modular, the database will be capable of being relational, the AI layer will include assistive recommendations, and the authentication will also be capable of giving a secure access.

This balance is very notable in a capstone environment because the project must be realistic in terms of technicality; it must be able to be completed within the time constraint and must also be sound enough to demonstrate proper software engineering decisions.

5.4.7 Justification Summary

Overall, technologies used in TeamTrack are selected because of the compatibility with the scale of the project, the possibility to support the required functionality of the system, and the reasonable balance between potentiality and feasibility. The choices are also compliant with the past TIS and SAS documentation which boosts continuity among milestones.

Table 112: *Technology Investigation and Justification Summary*

Subsection	Technology Area	Main Justification
5.4.1	Frontend Technology Justification	Nuxt.js supports structured, reusable, and scalable UI development
5.4.2	Backend Technology Justification	FastAPI supports modular API design and Python-based service logic
5.4.3	Database Technology Justification	PostgreSQL fits the relational nature of TeamTrack data
5.4.4	AI Technology Justification	LLaMA via Ollama supports assistive and transparent AI recommendations
5.4.5	Authentication Technology Justification	Firebase/OAuth simplifies secure identity handling
5.4.6	Overall Stack Justification	The technologies work well together as one coherent full-stack solution
5.4.7	Justification Summary	Summarizes why the chosen technologies remain suitable for TeamTrack

6. Deployment and Hosting Overview

6.1 Deployment Strategy and Environment Setup

The TeamTrack system deployment strategy is meant to make the application scalable, maintainable, secure, and simple to deploy to various environments. The system is based on the containerized deployment strategy where various services of the system are deployed as services. This deployment strategy is consistent with the layered system architecture identified in the system design and architecture documentation in which the frontend, backend, database, and AI services are independent and connected systems.

The main purpose of the deployment strategy would be to maintain consistency in the environment, reliability of the system, modular deployment, and more convenient maintenance. The system can be updated, maintained and scaled without any impact on the entire application by isolating system components into various services and containers.

6.1.1 Deployment Strategy Overview

The deployment of the TeamTrack system will be based on a multi-tier architecture, which will include the following components: the frontend application, the services of the backend API, the database services, the services of activity logging, and the services of the AI module. All the components will be installed separately, but linked by means of safe communication channels.

The following main components are included in the deployment strategy:

- Frontend Web Application
- Backend API Server
- Primary Relational Database
- Activity/Event Logging Database
- AI Processing Module
- Authentication and Security Services
- Containerization Environment
- Secure Communication via HTTPS

All the components are deployed as independent services to promote modularity and isolation of fault. This implies that in the event of failure of one component other components of the system may still be able to run.

6.1.2 Containerized Deployment Environment

The TeamTrack system relies on containerization as a management of deployment and environment setting. Containerization allows the application to run consistently across different environments such as development, testing, and production.

The key system components are containerized using Docker and they include:

- Backend API service
- AI module service
- Database services
- Supporting services

Containerization has a number of benefits:

- Precisely provides a similar environment across all machines.
- Makes deployment and system setup simplified.
- Provides the ability to deploy system modules on their own.
- Improves scalability, maintainability.
- Separate system components to avoid system failures.
- Makes it easy to update and maintain version control.

As an illustration, when the time tracking module or AI module fails, the backend API and the frontend application do not stop operation since they are operating in different containers. This high level of modular deployment increases the maintainability and reliability of the system.

6.1.3 Environment Setup (Development, Testing, Production)

It is designed to work with several deployment environments, such as:

Development Environment

The developers build and test the features of the system in the development environment. Within such an environment, developers execute the frontend, backend, and database on a local basis with containerized services. This enables developers to experiment with features prior to implementation.

Testing Environment

The testing environment is used for system testing, integration testing, and user acceptance testing. This environment is as close to the production environment as it can be so that it becomes stable enough to be deployed.

Production Environment

It is in the production environment where the end system is made available to users. In this environment, there are secure communication, database-backup, logging services, and monitoring tools that help in system reliability and performance.

Each environment uses the same container configuration to ensure consistency across development, testing, and production systems.

6.1.4 Backend and Database Deployment

The backend server will be deployed in the form of a containerized API server that will process user authentication, task management, project management, communication services, and analytics processing. The relational database and the activity logging database are connected to the backend using secure in-house connections.

The system has a two-part data storage:

Primary Relational Database

It stores structured system data such as:

- Users
- Projects
- Tasks

- Time logs
- Messages
- Permissions and roles

Activity/Event Logging Database

This stores system activity logs such as:

- User actions
- Task updates
- Contribution logs
- System events
- Analytics data

Isolating the activity logging database and the main database enhances performance and can be better analyzed and reported without having to interfere with the main system operations.

6.1.5 AI Module Deployment

The AI module is implemented as an independent service, which is connected to the backend API. The AI module uses these data about activities and projects to create summaries, insights and analytics to users.

The AI module is implemented as an independent subsystem to the main backend system so that AI processing does not primarily interfere with the work of the basic system. Such a modular deployment makes the system stable and gives the AI module the ability to be updated or even improved.

The AI service interacts with the backend by specified API endpoints and gets project activity information to analyse and generate reports.

6.1.6 Security and Communication Setup

Of the deployment strategy is security. The system involves the use of secured communication between the system components and the users.

Security measures include:

- SSL/ HTTPS to transmit data securely.
- Authentication and authorization.
- Role-based access control
- Secure API communication
- Access controls to the database.

- Audit trails and logging activities.
- Container isolation for services

Such security control measures will provide security to the user data and project information, and system operations.

6.1.7 Deployment Summary

In short, the deployment strategy of TeamTrack is built upon a containerized, multi-service deployment architecture, in which the frontend, the backend, databases, and the AI module are sourced as individual services. The deployment strategy provides better system scalability, reliability, security, and maintainability.

The environment configuration will facilitate development, testing, and production environment where the system will always behave in a consistent manner in all levels of deployment. Containerization and the modular deployment ensures that the parts of the system could be updated, scaled and maintained without interruptions to the overall system.

The deployment plan under consideration serves the general system architecture and makes sure that the TeamTrack platform can be implemented and supported effectively all through the deployment process.

6.2 Frontend Hosting

TeamTrack system frontend hosting strategy is structured in such a way that it provides the web application to be reachable, scalable, secure and easy to deploy and maintain. Because TeamTrack is a web-based academic collaboration solution, the frontend application needs to be deployed to a stable web hosting solution based on the modern JavaScript framework and the use of secure HTTPS-based communication.

The TeamTrack system frontend is created with Nuxt.js, a framework that is based on Vue.js and provides server-side rendering (SSR), statical site generation, and single-page application deployment. The hosting plan should thus be able to support Node.js applications and be able to efficiently deploy a Nuxt.js application.

The frontend hosting environment is the one that delivers the user interface and the client-side interactions as well as communicates with the backend API via secure HTTP requests. The frontend is not directly connected with the database but is connected with the backend API services that handle the requests and provide the responses.

The key goals of the frontend hosting plan are:

- Provide reliable access to the web application
- Support Nuxt.js deployment and build process
- Allow secure HTTPS-communication.
- Make sure that it can support multiple users.

- Aid constant deployment and changes.
- Offer quick loading services to users.
- Connection with the back-end API services should be allowed.

6.2.1 Nuxt.js Build and Deployment Process

The Nuxt.js frontend application has a defined build and deployment procedure, and only it is hosted on the server. This process ensures that the application is optimized, compiled, and ready for production deployment.

Typical steps involved in the Nuxt.js build and deployment process are:

1. Development Stage

As the development progresses, the front end application is developed and tested locally on the Nuxt development server. Developers use UI components, pages, navigation, and frontend logic and test them to be integrated into backend APIs.

2. Build Process

After the development and testing have been done, the application is now ready to be deployed using the production build process. The compilation process gathers the Nuxt.js application, optimizes the assets, packages the JavaScript files and makes the application ready to be deployed. This will enhance performance, minimize file sizes and minimize loading speed.

3. Production Deployment

Once the construction is complete, the finished frontend is moved to the hosting platform. The nuxt.js application runs on the hosting server and provides the frontend interface to the users via a web browser.

4. API Communication Setup

The front end application is designed to interact with the backend API on secure HTTPS requests. Any activity performed by the user like logging in, creating projects, and updating tasks as well as communicating with others is transmitted to the backend server by means of API endpoints.

5. Updates and Maintenance

When updates are made to the frontend application, the build process is repeated and the updated application is redeployed to the hosting server. This guarantees that the users have access to new features, UIs and bug fixes.

Such systematic building and deployment process makes the frontend application to be maximized, safe and maintainable.

6.2.2 Selection and Justification of Hosting Platform.

A proper choice of a hosting platform is significant to provide adequate system performance, reliability, and scalability. The hosting system should be capable of supporting the Node.js applications, server side rendering, secure communication using HTTPS protocol and connectivity to the backend services.

A number of front end hosting environment options were taken into account and they include:

- Traditional web hosting
- Cloud hosting platforms
- PaaS hosting is the second category of hosting.
- Container-based deployment hosting.

The web hosting services that were chosen as the most appropriate are cloud-based hosting platforms due to their ability to offer scalability, reliability, security, and an easy deployment of web applications in the present day.

The choice of the hosting platform was determined as follows:

- Node.js and Nuxt.js applications support.
- SSL/ HTTPS support of secure communication.
- Scalability Multiple users.
- Easy deployment and updates
- Integration with backend API services
- Reliability and uptime
- Response and speedy transmission of content.
- Academic project deployment cost efficiency.

Cloud hosting services offer automatic scaling, secure hosting facilities and easier deployment in contrast to the traditional hosting setups. They also facilitate continuous deployment processes and this enables the developers to update the frontend application without any problems.

It will have a frontend hosting environment which will be linked to the backend API server via secure API endpoints. Every single user interaction of the frontend interface will be handled by the backend services such that business logic, authentication, and database ones are highly secured and centralized in the backend system.

6.2.3 Frontend Hosting Architecture Summary

The frontend hosting environment serves as the presentation layer of the system in the general structure of the system. Users browse the system using their web browsers which are connected to the hosted frontend application. The frontend then connects to the backend API server which does the requests and communicates with the database and AI services.

The frontend hosting architecture may be represented in the following way:

User Browser → Frontend Hosting Server → Backend API Server → Database / AI Services

This architecture provides isolation of the presentation layer and the services of the back end, enhancing the security, scalability as well as maintainability of the system.

6.2.4 Summary

The hosting strategy of the TeamTrack system is based on the frontend hosting to make sure that the Nuxt.js web application is deployed to a scalable, secure and maintainable hosting system. Nuxt.js build and deployment process ready the application to be used in production and the chosen hosting platform provides the reliability of the accessibility with safe communication and the integration with the backend services.

This hosting plan can be used to accommodate the entire system architecture and make sure the users can access the TeamTrack platform via a web browser and the frontend can communicate safely with the backend services and databases. The front end hosting and backend service separation enhances system security, system scalability, and maintainability.

6.3 Backend Hosting

TeamTrack backend hosting will offer a reliable, well secured and sustainable environment through which the core services of the system will be hosted. As the backend will perform the authentication process, user authorization process, task management process, contribution tracking process, dashboard data processing process, database communication process, and communication with the AI module, it will serve as the primary processing layer of the platform. This is why the backend cannot be taken as an easy supporting component. It should be implemented in a programmed manner that facilitates reliability, clean separation of services and uniformity of development and deployment setting. The official documentation of FastAPI emphasizes the need to pay attention to starting processes, process restarts, replication, memory consumption, and security as an application is to be deployed, which is also consistent with the requirements of TeamTrack as a hosting service [1].

In TeamTrack, the backend is implemented as a specialized service, and not combined with the frontend or database or AI implementation tier. This will be using the layered architecture of the project, with each of the key system components having a particular role. The backend is the application and business logic layer, which accepts the requests made by the frontend, validates the data, and imposes the access rules, performs operations in the application, and sends the communication in response to the other services including PostgreSQL and the locally deployed AI module. This design of the system increases the maintenance and facilitates the future updates since a change in the logic behind the backend can be implemented without a direct impact on the rest of the platform.

The backend hosting plan also serves the technical interests of the project in a wider context. Because TeamTrack is supposed to be carried on to further development phases, the implementation strategy should not simply execute the application in the testing phase. It must be able to facilitate reproducibility within team members, minimize configuration differences and offer a basis that can eventually be expanded to a more production-ready environment. Due to these reasons, the TeamTrack backend will be designed as a containerized FastAPI service that has well-structured runtime settings and is not bound to dependent services.

6.3.1 FastAPI Server Deployment

TeamTrack backend will be served as FastAPI application behind an ASGI server with the process of running the application running on Uvicorn. TeamTrack is suitable in this deployment model since it is an API-first design where nearly every activity on the frontend is transferred to the backend via a specified endpoint. The backend must therefore be in a position to handle structured requests, data validation and effective inter-layer communication. Fast API was built with API construction in mind and its deployment documentation encourages the use of Uvicorn as a standard server process to execute Fast API programs [1].

To achieve consistency and portability, the FastAPI server is going to be containerized in a Docker container along with the Python dependencies and application code it needs. According to the documentation Fast API documentation uses a typical deployment model of creating a Linux container image, usually via Docker, and then deploying the container image on-demand [1]. This is positive to TeamTrack since the identical application setting may be replicated across development platforms as well as later hosting destinations.

The backend may also be run in a more managed and standardized runtime environment as opposed to relying on the local machine setup of individual developers. This minimizes dependency clashes, makes it simpler to onboard the team members and helps them to make easier transitions between implementation and demonstration and eventual handover. The backend container operationally will have the necessary service port and execute the FastAI application with Uvicorn. This containerized service will be dealing with incoming API calls on the

Nuxt.js frontend, authentication and authorization logic, contact with the PostgreSQL database and sending AI-related requests to the dedicated Ollama-based AI module as necessary. The backend thus serves as the regulated central point of the communication throughout the system.

At the present milestone level, TeamTrack does not need a mass production application with a sophisticated traffic management and numerous nodes duplicated. The prototype and academic scope of use is projected to be adequate with a single instance of a backend service. Nevertheless, further scaling is also possible with the deployment model. The deployment instructions about FastAPI describe how applications can utilize the concept of multiple worker processes in situations in which replication is required to serve a larger number of requests and better utilize the resources available to them [1]. This implies that the selected deployment method is light at the current scope of the system but still offers a viable way of expansion to take place in the future.

Another design consideration regarding the deployment of FastAPI server is service isolation. Backend is deliberately not connected with the database engine and AI runtime. This enhances the structure of systems and minimizes system interdependence. When there is a service that is required to be updated, restarted, or even debugged, then such work can be performed in a cleaner manner without disrupting all the components of the system simultaneously. It is particularly essential in TeamTrack because the AI module is a subsystem rather than a dependency that is to assume control of the primary platform. First by making sure that the backend is a dedicated service the project provides the ability to get modularity, easier troubleshooting, and the underneath hosting infrastructure is easier to maintain.

On the whole, the deployment of FastAPI backend into Uvicorn into a Docker container offers TeamTrack with a realist and academically plausible hosting paradigm. It also encourages uniformity, less convoluted design, isolation of the services and is easy to deal with in the context of the current project.

6.3.2 Environment Variables and Configuration Management

The two aspects of the TeamTrack backend deployment strategy, which are of importance, are the configuration management and the environment variables, as they allow the system to be safe, portable and easier to maintain in various environments. TeamTrack will not maintain environment-specific settings, which are hard-coded into the source code but will maintain them as a separate set and load them into the backend on demand. The configuration approach as determined in the Twelve-Factor App methodology is the information that vary across the deploys and recommends that it remain in the environment rather than being hard-coded in the code or the static files [2]. The practice is common to be used in the rollout of the modern application as it offers greater flexibility and reduced the probability of sensitive values leaking.

This strategy will gain particular importance in the case of TeamTrack, where the system includes several settings, which will have to differ in the development, testing, and hosted environment in the future. These include credentials to database connections, JWT secret keys, authorized host settings, debug settings, values of ports to the backing service and values of AI service endpoint. Out of these settings, the codebase of FastAPI can be reused by externalizing the settings where the configuration of the environment varies depending on the environment. This enables portability and novelty required to rewrite or manually modify application logic whenever the deployment context is changed.

This form of configuration management is also supported by FastAPI documentation. It provides guidance in its settings telling that application settings may be managed with Pydantic settings models and by using environment variables and .env files where needed [3]. This would be helpful in the case of TeamTrack since it enables the validation of configuration values in a structured manner within the code of the backend. Practically, that implies that the application can specify anticipated configuration variables, read them at start-up and minimize the errors related to missing or improperly formatted variables.

Security-wise, environment variables configuration management is of particular importance. The repository should not be hard-coded with sensitive values like database credentials, authentication secrets and the private endpoint of service. Having them out of the codebase reduces the likelihood of secrets being accidentally leaked via version control or via collaboration. It is especially applicable in the case of TeamTrack since the platform will handle secured academic information, user-generated records, and contribution information that must be accessed only in a controlled and authorized way [2], [3].

The configuration strategy is also better in maintainability within the team. Once the values of the necessary environment are recorded and organized correctly, every developer will be able to deploy the backend with a reliable configuration environment. That assists in avoiding environment drift, the application works on various systems differently because of different settings. It also helps in future handover since the client or next development team can be provided with a proper list of requirements on configuration even without having to make any amendments to the source code of the application.

Besides this, the fact that configuration and code are decoupled also fits the modular deployment philosophy of TeamTrack. The autonomous services of the backend, database, and AI module will be set up as separate but related, meaning that each service can be set and nevertheless communicate with the rest using constrained variables like ports, host addresses, credentials, and service names. This ensures that the deployment structure will be more structured and can be adjusted to in the event that the hosting model evolves in the future.

In general, the combination of environment variables and structured configuration management makes the TeamTrack backend more robust, as it enhances its security, portability, maintainability, and deployment control. It also makes sure that at runtime options are flexible and safe, and permits the FastAPI backend to be more predictable and workable in various settings.

6.4 Database Hosting

TeamTrack database hosting is built to facilitate dependable, organized and safe storage of the fundamental cooperation information of the platform. The database layer is required to offer high relational consistency and reliable access to the backend since the system deals with the users, projects, roles, tasks, time logs, contribution records and similar activity information. PostgreSQL is the main transactional database in TeamTrack since the system relies on the structured relationships between entities, and the system must enable maintenance of precise record storage among various related modules. According to the official documentation, a database cluster has one or more databases, and each database can have a schema and structured objects, including tables, indexes, and functions, and this structure fits the structured nature of the TeamTrack data model.

The database is not considered as a more or less structured storage layer with loosely arranged information. It is rather designed to serve as the official source of the fundamental academic cooperation documentation of the system. This contains the information like registered users, role assignments, project definitions, tasks ownership, time records, and contribution-related history. These data elements have a strong relationship, and therefore a relational database model would be appropriate compared to a schema-flexible one. This is in line with the previous TeamTrack design choice of having PostgreSQL as the primary structured data storage but high-volume event logging should be done in isolation in order to eliminate the needless load on the transactional database.

PostgreSQL hosting strategy is thus based on stability, access control and compatibility with the rest of the system architecture. The database will be a dedicated service which will be accessed via the FastAPI backend, but not the frontend. This retains the backend as the security and validation boundary of the system and assists in ensuring that data access is controlled and in conformity to the role-based access logic of TeamTrack.

6.4.1 PostgreSQL Deployment and Access

The primary relational database service that TeamTrack will utilize is PostgreSQL where structured transactional data will be hosted. The database will also be introduced as a dedicated service independent of the backend layer and AI layer to ensure that each significant component of the system is in charge of its roles. This helps in the layered architecture of the project and enhances the maintenance by lessening the close linkage between the user logic and the permanent data

storage. Previous design efforts on TeamTrack also focused on the fact that PostgreSQL is suitable since the platform relies on relational integrity, transaction reliability, and the ability to connect different entities such as users, projects, tasks and records of contributions.

Deployment wise, PostgreSQL is able to handle more than one database in a cluster of databases, whereas a given database is accessed by a defined client connection. The use of schemas to group database objects together and to control access more tidily within a database is also supported in PostgreSQL. The documentation indicates that schemas assist in grouping objects into manageable groups and granting user-controlled access to the objects. In the case of TeamTrack, it is helpful to have a well structured hosting model whereby the application database can be structured clearly and managed predictably.

Practically the TeamTrack postgresQL instance will contain some of the central entities needed to make the platform function normally such as user records, project data, task data, role relationships, and time tracking or contribution information. The backend API layer will be the only way by which the database is accessed therefore the front end will never interact with PostgreSQL. This design is significant as it enables the validation, authentication, authorization and the business rules to be centrally located at the back end. It also minimizes the chances of uncontrolled access of critical data by the client.

The access to databases will be organized by providing the backend with environment variables of controlled credentials and connection settings. This makes sure that the connection information does not tie in with the codebase and can be modified easier without having to change the codebase in both development and subsequent deployment situations. Database level access control will also observe the principle of least privilege i.e. the application service must only utilize the permissions it needs to perform its normal operation as opposed to accessing administrative privileges. This assists in fortifying security posture of the general system.

The other strength of PostgreSQL implementation in TeamTrack is that it supports consistency and transactional correctness. As the platform will have operations like updating tasks, assigning project members, history of contributions and logs of time the database should maintain an accurate state even in case of multiple related operations. PostgreSQL is highly achievable with this type of structured application workload, and this is one of the reasons why it was chosen over other options in the context of previous technology evaluation.

In general, the deployment and access of PostgreSQL in TeamTrack is aimed at offering credible structured storage and regulated backend-mediated accessibility and interaction with the modular architecture of the system. This renders the database layer appropriate to both the current academic prototype as well as the extension in the future as the system is extended.

6.4.2 Data Migration Plan

The TeamTrack data migration plan is also aimed at facilitating the safe data base startup, managed schema alterations, and subsequent system data portability. At the present stage of the project, the platform is under development thus migration planning centers around less of a large-scale production conversion and more about how the system can be migrated smoothly between development configuration and stable demonstration and subsequent deployment. It further implies that the migration plan should help to achieve three key objectives namely developing the initial schema in a proper manner, updating the schema in a controlled manner where there is a change in the design and backup and restore data where there is need.

To deploy the first deployment, a database will be created first but dedicated to the TeamTrack project as a PostgreSQL database. The documentation of the PostgreSQL states that a server can host multiple databases and that individual databases are generally utilized by different projects or users. This is very suitable to TeamTrack since the platform ought to have its independent application database and not to share non relevant project data using the same logical space.

Once the database is created, application schema will then be used to establish the necessary tables, relationships and constraints. The schema will be a representation of the primary entities of the system that include the users, projects, tasks, roles, time logs, and records related to contributions. These entities are interdependent and therefore schema changes are to be handled with care as opposed to making changes informally as it develops. A regulated migration process will assist in making sure that the whole team operates on the same database structure as well as ensuring that the compatibility between the backend and the stored data will not be affected by subsequent updates.

TeamTrack migration strategy will thus be versioned where schema changes are made gradually. Instead of re-implementing the database whenever the design is modified, changes need to be added into the database in a monitored order, allowing the schema to keep up with the application. This fact applies especially in a project like TeamTrack because the background logic, contribution analytics, and reporting functions are all based on the definition of tables and relationship reliability. Even in an academic prototype, it is better to have a controlled migration path instead of the ad hoc manual database changes.

Migration plan also entails backup and restore planning. PGSQL also provides the pgdump tool to give a logical dump of a database in a form of SQL and could be later recreated later and that the documentation states that the output could be restarted through either psql or pgrestore on its dump format in accordance to the form of a dump. It also states that the output of pg_dump is usually reloadable into subsequent versions of PostgreSQL and can also be used when changing the machine architecture of a database. In the instance of TeamTrack, this provides a feasible means of data preservation on a test basis, data movement among the environments or the project transfer to a following development phase.

In cases where the amount of data to be loaded is large, e.g. loading ready sample data, or restoring test data, the PostgreSQL documentation recommends the application of efficient bulk-loading methods such as COPY, especially when loading large numbers of rows. This can come in

handy in TeamTrack where one needs to load the database with pre-programmed academic test information in the event of creating or testing.

A practical description of data migration plan of TeamTrack can therefore be summarized in steps as follows. It should first be developed that a special PostgreSQL database is created on the project. Second, the schema is used under controlled and versioned use. Third, seed or test data that is necessary is loaded. Fourth, the backups are generated prior to the significant structural changes or environmental transfers. Lastly, there are restoration techniques applied whenever the system requires to be recreated on a different machine or restored after an unsuccessful change. This method is practical to an academic project since it is liberal enough to be safely developed but not so enormous as to add complexity that is not required by the enterprise.

In general, the data migration plan will maintain the TeamTrack database to be consistent, portable and easy to service with the increase in the project. It assists in a consistent growth, minimizes the danger of schema drift, and offers an obvious route of saving or migrating system data at subsequent phases of deployment.

6.5 AI Module Deployment

The deployment of AI in TeamTrack will be aimed at offering intelligent project support without interdependent with core service offerings of the system. TeamTrack AI component does not aim to substitute the backend, database, or the front-end workflow. Rather, it is a supportive subsystem, which contributes to the creation of summaries, interpretation of contribution-related activity and the provision of AI-based insights that can enhance project monitoring and reporting. The relevance of this design lies in the fact that it enables the platform to enjoy the benefits of AI without having to rely entirely on the uninterrupted operation of the model service to the rest of the system.

That is why the AI layer is installed as another service with the help of LLaMA via Ollama on a local server. This design follows the previous architectural trend of the project where the AI module is regarded as a separate component that does not have a direct relation to the application logic but is related to the back-end. TeamTrack also enhances modularity and eases the task of debugging by maintaining the AI service as a distinct entity and minimizes the risk that AI-related failures will disrupt key project operations, including authentication, task management, or contribution tracking.

The implementation strategy demonstrates viable academic project priorities too. Because TeamTrack is not designed to rely on commercial external AI APIs at the developing stage, the locally hosted model gives a better control over the expenses, privacy, and conduct of the system. In the documentation by Ollama, it is described that after Ollama is started, the API is exposed automatically and accessible locally (via the endpoints of `/api/generate` and `/api/chat`) and

therefore can be used to integrate with a backend service in a controlled local deployment setup [8]. Moreover, the Llama documentation introduces Llama 3.1 as a model family that is supposed to be used widely in usage cases and deployable settings, which makes it a suitable foundation of features of TeamTrack supported by AI [9].

6.5.1 LLaMA via Ollama Local Deployment

TeamTrack AI module deployment will occur locally with Ollama as the model runtime environment and LLaMA as the language model that drives AI-assisted responses. This had to be a deployment approach since Ollama offers a viable means to execute models on-premises and publish them in the form of a straightforward API interface. The official Ollama documentation states that after the service is started, applications are able to communicate with the model by calling local API, which is handy to add AI capabilities to a project without relying on an external hosted service [8].

This local deployment strategy has a number of benefits in the case of TeamTrack. That is, it favors privacy and control. Because project-related prompts and contextual information do not have to be sent to a third-party cloud API, the project has a more efficient control of the manner in which AI-related interactions are processed. This is a valuable thing especially due to the fact that TeamTrack can process summaries of contribution, project descriptions and team activity patterns which are sensitive issues to be addressed in an academic institution. Second, it facilitates cost effectiveness. The project will eliminate the common per-request API expenses in development and testing, as a local AI runtime can be used. Third, it facilitates what is often referred to as independence over external service availability and this is helpful in a student capstone environment where a controlled and reproducible environment is desirable.

These module deployment steps include installation and execution of Ollama on the target machine, downloading the needed LLaMA model, and subsequently enabling the service to make its local inference API available. Model retrieval Ollama documentation indicates that a pull operation can be used to retrieve the models, which in turn are accessed via local API endpoints to do generation and chat tasks [8]. This renders the setup both relatively easy to deploy as well as enables the model to be viewed as a dedicated service but yet it is possible to view the model as an integrated library that is closely bound to the backend.

The second notable point of this local implementation is the fact that the AI model is not tied to the fundamental transactional processes. TeamTrack does not use the AI model to carry out vital database practices or make the platform secure. Rather, the AI module can be applied to supporting functions like summarization, generation of project insight, or interpretation of contribution data that is available. This implies that even in the event that the model service is failed, the primary platform will still be in operation to perform non-AI tasks. The architectural

choice enhances resilience and goes along with the larger system objective of ensuring that AI is useful but not a critical part of the underlying platform functionality.

Also, one should be aware of local AI implementation, which has operational implications. Model inference is based on the accessible machine resources, including memory and processing capacity, and hence the AI module might need additional hardware consideration than typical backend modules. In the case of TeamTrack, this can be accepted due to the narrow scope of the existing system and the fact that the AI service is only designed to help with selective work and not large-volume inferences. Local deployment is therefore still an option that works in the present milestone and prototype scenario.

In general, the local deployment of LLaMA with GPT-neo provides TeamTrack with a managed and configurable AI solution to meet its privacy, cost, and maintainability objectives. It enables AI-assisted capabilities to be added without adding an unwarranted reliance on commercial cloud APIs.

6.5.2 AI Service Integration with Backend

The connection between the AI module and the backend is built in such a way that the FastAPI service will be the sole application layer that will interact with the Ollama runtime. In such an architecture, the frontend does not make calls to the AI service. Rather, any user request that must be processed with the help of AI is forwarded to the backend where the corresponding prompt or context is prepared, the local Ollama API is contacted, and the result is sent back to the frontend, in a controlled format. This maintains the backend as the hub of logic and validation of the system.

This model of integration is significant because of a number of reasons. First, it enhances security and control by making it more difficult to access the AI runtime by the client without some kind of control. Because the backend is involved in all interactions between AI, it can decide what data to communicate with the model, what the prompts ought to look like, and what kind of answers ought to be returned to the user. Second, it promotes uniformity. The backend can normalize the format of AI requests and responses such that the other parts of the platform can deal with AI functionality in a foreseeable manner. Third, it can enhance maintenance since any future modification of the AI model, immediate design, or endpoint service can be managed in the back-end without significant changes in the frontend structure.

The API testing Ollama documentation states that the local applications may communicate with model endpoints (e.g. `/api/chat` and `/api/generate`) and its quickstart instructions demonstrate the endpoints are immediately interacted with by the applications on the same environment [8], [10]. This is very suitable to TeamTrack since with FastAPI back-end, one can make controlled requests to the local AI service when it is needed to generate a summary or insight. The backend can hence serve as the integration point between project data and the AI processing.

Under TeamTrack, the backend-to-AI interface is supposed to assist tasks including the creation of project summaries, the identification of patterns of participation, and help with the interpretation of information regarding contributions that already exists in the platform. The AI service is not applied as the decision-making body on its own, and rather it is an improvement layer created on the basis of the already existing project data. This is noteworthy since academic oversight remains the responsibility of humans and the AI output is to be considered as auxiliary data, but not immediate final decision.

Fault isolation is the other advantage of this integration design. The AI service is not part of the backend process, which is why temporary AI failure will not automatically disrupt the whole platform. Failures can be more gracefully dealt with by the backend since unavailable responses will be identified and fallback messages delivered or the AI-assisted features may not be used and the rest of the application will continue running. It comes in handy specifically in TeamTrack as it does not lose normal project management functions in the absence of the AI component.

The combination of the AI service with the FastAPI backend would result in a better and more maintainable system design. It makes AI operation controlled, modular, and in line with the overall architecture of TeamTrack and enables the intelligent features in a realistic manner.

6.6 CI/CD Workflow

In TeamTrack, we would like our process of development to be systematic, cooperative, and effortless as we approach the eventual delivery. Given the fact that the project has several interrelated components like the frontend, the backend, the authentication functionality as well as the databases interplay, we require a workflow that would assist us to incorporate modifications in a managed manner. That is why we adhere to a simple CI/CD process that allows us to maintain a gradual advancement yet being realistic regarding a capstone project [11], [12].

The current CI/CD workflow is not to be so complicated as a full enterprise Devops operation. Rather, we employ a pragmatic and light process, which fits our scope of project and organizational structure. The rationale behind this workflow is to assist us in co-ordinating updates, decreasing integration problems, having a stable working version of the system, and prepare TeamTrack to be tested, reviewed and handed over in a more professional manner [11], [14].

6.6.1 Purpose of the CI/CD Workflow

Our CI/CD workflow is primarily aimed at making the development more consistent and organized by the team. As various members might be serving on various elements simultaneously, we should have a procedure that enables us to integrate those changes, without making direct uncontrolled

modifications to the venture [12].

With such a workflow, we are able to maintain our development in a more organized manner and decrease the possibilities of confusion in the further stages of the project. It is also useful in our determination of which version of TeamTrack is stable enough to be tested, demonstrated or reviewed [11], [13].

Through this workflow, we aim to:

- ensure that the project development is structured within the team.
- minimize conflict of mergers and integration errors.
- Have a stable version of the system.
- Help with easier build preparation.
- Enhance preparation to milestone review and eventual hand over.

Table 113: *Main Objectives of the CI/CD Workflow*

Objective	Why It Matters for TeamTrack
Organized collaboration	Helps multiple team members work on the project without interfering with each other's changes
Stable integration	Ensures that new updates are added in a controlled and reviewable way
Build consistency	Makes it easier to prepare the same project version for testing and demonstration
Review readiness	Helps us know which system version is suitable for supervisor or client review
Handover preparation	Supports a smoother final delivery of code and related materials

6.6.2 Bitbucket-Based Deployment Pipeline

In the case of TeamTrack, we store Bitbucket as the primary repository and version control. This would enable us to store the project files under a single location, as well as to keep track of the updates, branches and have a visible development history. TeamTrack has multiple modules and

duties, and this is why a common repository framework becomes significant in maintaining the work order [11], [12].

We begin our pipeline when a member of a team is working on a given feature or an improvement on an established development branch. This assists us in isolating the changes in work-in-progress to the now stable system. When the changes are prepared they are submitted to the Bitbucket repository where they could be reviewed and later merged into the main branch in case they were approved [12].

It is through this that we can cease to develop as individuals and begin to integrate as a community in a more disciplined manner. It also assists in securing the stability of the project, as, only approved and accepted changes would be included into the latest main version [11], [12].

The Bitbucket-based pipeline helps in supporting:

- version control of implementation files of all files.
- development of features based on branches.
- tracking of change and update history.
- low-risk frontend and backend integration.
- preparation of a build and validation version of a stable version

Table 114: Bitbucket-Based Pipeline Stages

Stage	Description	Purpose
Feature Development	A team member works on a specific module or task in a controlled branch	Keeps new work separate from the stable version
Code Push	The updated work is pushed to the Bitbucket repository	Makes changes visible and shareable
Merge Review	The pushed code is checked before being integrated	Reduces integration mistakes
Main Branch Update	Approved work is merged into the main branch	Maintains a reliable shared project version
Build Preparation	The stable version is used for dependency setup and validation	Supports testing and handover readiness

6.6.3 Build Process and Automation

When the newest approved changes are present in the stable branch, we pass to the stage of building preparation. This is significant as TeamTrack is not a single isolated file. It contains a frontend application, backend API services, environment settings, and integrations that are required to be properly set up to be tested or demonstrated [11], [14].

Regarding the frontend, we are going to ready the Nuxt.js application by running the dependencies, making sure the configuration is correct, and we are going to get a build-ready version of the interface. To be ready to the backend, we will set up the FastAPI service and check the environment, making sure that the connections in the system include access to the database and the establishment of authentication is configured properly.

Even though our project does not demand a very automated industrial pipeline, we desire that our repeated build steps be as homogeneous as feasible. This minimizes the number of errors made manually and assists us to rebuild the same project condition whenever we require to test, present or submit the system [11], [14].

This typically involves our construction work which involves:

- Getting the last stable version in Bitbucket.
- frontend and backend dependencies Dependencies installed.
- checking configuration files and environment settings
- building a frontend preparation.
- setting up the back-end service environment.
- testing of database and authentication configuration.
- Making sure that an application is fit to be validated or presented.

Table 115: Build and Automation Activities

Activity	Frontend / Backend	Purpose
Dependency Installation	Both	Ensures required libraries and packages are available
Configuration Check	Both	Confirms that environment and project settings are correct
Frontend Build Preparation	Frontend	Makes the UI ready for execution and testing

Backend Setup	Backend	Makes the API and service layer ready to run
Integration Verification	Both	Confirms that connected components can work together properly

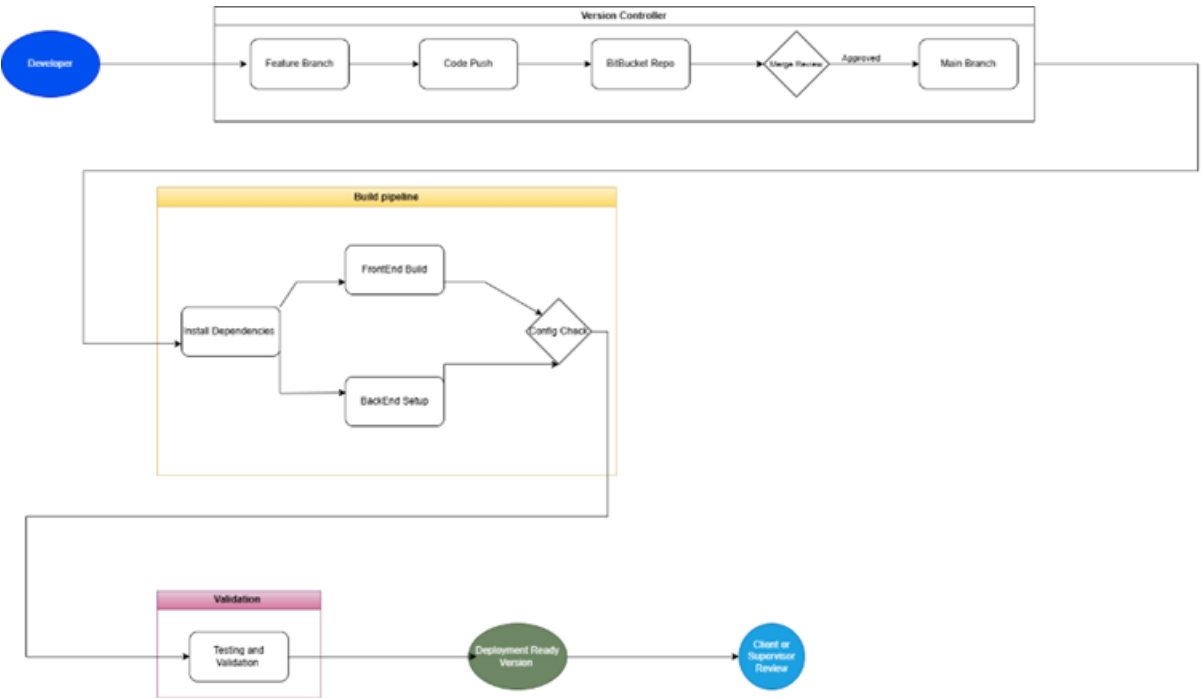


Figure 13: CI/CD Workflow Diagram

Figure description

This figure illustrates how we plan to manage source control, integration, build preparation, validation, and delivery readiness in the TeamTrack project

6.6.4 Validation Before Delivery

We have a validation step in our workflow before we can treat any version of TeamTrack as being ready to be deployed, demonstrated or handed over. This is required since even after the success that the code integration has completed, it does not imply that the project is fully prepared to be reviewed. We need also to ensure that as the integrated parts are run the current version will behave as expected [11], [14].

At this phase, we validate that the frontend loads are correct, backend services are responding correctly and that the completed version of the system is ready to be demonstrated or submitted at a milestone. This makes us feel more certain about the quality of the version that we are offering.

Validation is particularly crucial in TeamTrack since the project consists of the coordination of a number of interconnected modules. However, in case of the wrong set-up of any of the parts, the overall user experience and the quality of demonstration can be influenced [11].

Table 116: Validation Focus Areas

Validation Area	What We Check
Frontend Readiness	Whether the interface loads and works correctly
Backend Readiness	Whether APIs and service logic respond as expected
Configuration Validity	Whether environment settings and connections are correct
Integration Readiness	Whether the connected parts of the system work together
Review Suitability	Whether the current version is fit for milestone review or handover

6.6.5 Benefits of Our Planned Workflow

Our intended CI/CD process provides the TeamTrack project with a number of viable advantages. Firstly, it enhances collaboration since every member has a clear and understandable procedure in processing updates. Second, it assists us to minimize the instability of the project since the changes are introduced more actively than by unrelated edits [12].

It simplifies the milestone preparation as well. As we are aware of the version that has undergone repository integration, build preparation and validation, we may be more assured using the version in our review to supervisors, demonstrations and ultimately handover [11], [14].

Project management wise, the workflow also comes in handy, in that it allows us to have a clear understanding as to what version of TeamTrack we are up to date with, which features are still under development, and which features are complete and are presentation worthy.

The benefits of this workflow are mostly the following:

- enhanced contact among team workers.
- less integration and configuration problems.
- greater transparency of project status.
- more stable preparation of milestones.
- better route to ultimate delivery.

Table 117: Benefits of the Planned CI/CD Workflow

Benefit	Contribution to the Project
Better team coordination	Helps all members follow the same update and integration process
Reduced risk of errors	Limits the chance of unstable or conflicting changes entering the main version
Stable project versioning	Makes it easier to identify a reliable version for review
Easier build preparation	Supports more repeatable setup and testing
Stronger handover readiness	Helps us present a better organized system to the client or supervisor

6.6.6 CI/CD Workflow Summary

In general, our CI/CD process in TeamTrack will serve the purpose of practical cooperation, consistent integration, and more transparent preparation of builds as the project approaches final delivery. Bitbucket is where we take charge of our version control and access to a shared repository and we have a very systematic approach to incorporating updates, creating a build, system verification, and determining which version is fit to be reviewed or handed over [11], [12].

Although we still have a lightweight workflow that can be used in a capstone environment, it is still good practice in software engineering. It aids us in being a more efficient team and aids in the more controlled and professional transfer between the continuous development progress and handing in the milestones and the final handover [11], [14].

Table 118: Overall CI/CD Workflow Summary

Subsection	Focus Area	Description
6.6.1	Purpose of the CI/CD Workflow	Explains why we use a structured workflow to support collaboration and stability
6.6.2	Bitbucket-Based Deployment Pipeline	Describes repository-based integration and controlled merging
6.6.3	Build Process and Automation	Explains how we prepare the frontend and backend for consistent execution
6.6.4	Validation Before Delivery	Describes how we confirm the project is suitable for review and handover
6.6.5	Benefits of Our Planned Workflow	Highlights the practical value of the workflow for the team
6.6.6	CI/CD Workflow Summary	Summarizes the full role of CI/CD in TeamTrack development

6.7 Product Delivery Plan to Client

As TeamTrack enters the last phase of development, we need to determine how the project will be presented to the client or supervisor in a well structured and coherent way. In our case the product delivery does not just imply the delivery of source-code. It also encompasses the supporting documentation which will assist the client to gain insight of the system, survey its design and determine whether the suggested deployment and access method fits the project or not [13], [14].

In Milestone 4, the delivery plan will be designed to demonstrate that we are not only focused on the development but also on project presentation, sharing and reviewing. As TeamTrack is an academic capstone system, our delivery strategy will be to have a practical and manageable delivery approach yet one that also includes the practice of professional software engineering. We would ensure that the client or supervisor may be able to retrieve the pertinent project artefacts in a well-organized manner, and comprehend how the system is planned to be checked and improved prior to the final submission [13], [14].

6.7.1 Purpose of the Product Delivery Plan

Our product delivery plan will primarily aim at giving a clear process of delivering the existing TeamTrack system and supporting materials to the client or supervisor. The handover should not be a hasty and disordered process. Rather, we desire the delivery to be a reflection of the real

structure of the project and be able to give the reviewer a sense of the direction that the implementation will be carried out and also be able to provide the supporting documentation [13].

It is notably significant in TeamTrack since the project is a combination of several layers, such as frontend, backends, authentication, database design, analytics logic, and the artefacts of the user-facing design. When such materials are not delivered in a proper manner, then the reviewer will find it harder to evaluate the general project direction.

In this delivery plan, we will endeavor to:

- deliver the project to the client the most applicable version.
- format the project materials in a systematic and easy to review manner.
- facilitate more convenient access to documentation artefacts and implementation.
- lessen the misunderstanding of what is being delivered and the way it should be reviewed.
- develop a more seamless, end-refined process.

Table 119: Main Objectives of the Product Delivery Plan

Objective	Why It Matters for TeamTrack
Clear handover structure	Helps the client or supervisor understand what is being delivered
Easier project review	Makes it simpler to assess both the system and its supporting documents
Delivery consistency	Ensures implementation materials and documentation are handed over together
Better communication	Reduces confusion about access, setup, and review expectations
Final-stage readiness	Supports a smoother move toward final refinement and submission

6.7.2 Repository Access and Handover Method

In the case of TeamTrack our primary handover approach will be repository access and project sharing which is organized. As the implementation of our project has several implementation parts and artefacts, the repository will serve as the central delivery point, which allows us to keep the system structure intact and retain the relationship between the code and configuration and documentation [12], [13].

The main type of technical delivery will be the stable version in the repository. This enables the client or the supervisor to see the project as the way it is developing. It also aids in the assurance

that the reviewed version of the system is the one that is approved as opposed to an incomplete or an outdated version [12], [13].

Besides the access to the repositories, we will also provide the auxiliary materials that will help understand the project better. These can consist of set up instructions, documentation, diagrams, milestone report, and screenshot among others, that are required to provide adequate explanation of the project. This implies that our handover will not be technical only but also documentation backed [13], [14].

Our handover approach that involves use of repository is expected to comprise:

- access to the most current stable version repository or approved snapshot of the version.
- folders of front and back ends in support of the project have been arranged.
- documentation on the structure of the system.
- configuration or set up instructions where needed.
- re-used architecture and design templates of previous milestones.
- milestone handover documents and other artefacts of review.

The reason why this method makes sense is that it keeps the project delivery organized and handover easier to follow in terms of technical and academic viewpoint.

Table 120: *Repository Access and Handover Components*

Handover Component	Description	Purpose
Stable Repository Version	The latest reviewed project version stored in the repository	Ensures the client sees the most reliable version
Frontend and Backend Files	Main implementation folders for the application	Preserves project structure and technical organization
Supporting Documentation	Setup notes, explanations, milestone documents	Helps the reviewer understand the project more easily
Diagrams and Design Artefacts	Architecture, ERD, user flow, system flow	Supports technical and design-level review
Review Materials	Screenshots, prototypes, and milestone handover content	Helps the client assess the system direction

6.7.3 Delivery Artefacts Provided to the Client

When we offer TeamTrack it is not that we are going to offer a single isolated product. Rather, we would like the handover to be the set of artefacts that are more complete in their representation of

the project. This will assist the client or supervisor to see the technical part of the system as well as the planning, design and implementation direction behind the system [13], [14].

We anticipate that the artefacts we will offer during handover will be:

- the version of the project under consideration now.
- milestone handover report and documentation.
- reused database and architecture diagrams.
- Prototype evidence and artefacts of UI design.
- access of repository-based implementation.
- configure and deploy documentation where necessary.

When we offer these artefacts, we can present a more complete picture of TeamTrack and minimize the possibility of the reviewer guessing how various aspects of the project work together with each other.

Table 121: *Delivery Artefacts Summary*

Artefact	What It Includes	Value to the Client / Supervisor
Stable System Version	Latest approved project version	Shows the current working state of TeamTrack
Repository Access	Project source structure and tracked files	Supports technical review and project transparency
Handover Report	Milestone 4 delivery content	Explains what is being handed over and why
UI and Prototype Artefacts	Screens, flows, and design evidence	Helps evaluate usability and client-facing design
Architecture / ERD Diagrams	System and data structure visuals	Supports system understanding and review
Setup / Deployment Notes	Basic instructions and configuration guidance	Makes the project easier to run or assess

6.7.4 Client Acceptance of the Deployment Approach

Besides the delivery of the project materials, we should also factor in the issue of whether the client or the supervisor feels contented by the approach proposed in the deployment and delivery. This is not necessarily the case in our case as client acceptance may not imply the project is

already being implemented in a fully commercial hosting environment. Rather, it implies that the suggested approach of sharing, reviewing, and preparing the system is deemed to be appropriate to the existing milestone and final project scope [13], [14].

In the case of TeamTrack, the client or supervisor would prefer to feel comfortable with the manner of the presentation and accessibility of the project. This involves ensuring that the repository-based handover is acceptable, that the supporting documentation is adequate, and that the direction of deployment of a capstone system at this point is real [13].

Our approach may be accepted by clients by confirming that:

- the structure of delivery is distinct and comprehensible.
- access based on the repository is appropriate to be reviewed.
- the documentation included is sufficient to have knowledge about the system.
- the implementation direction is equivalent to the level of implementation presently.
- from the delivered version of the project, it can be revised and improved.

This is a crucial component of the handover since we can bring our expectations into line. It further makes sure that the project delivery is technically ready, and it is also acceptable practically to the reviewer [13], [14].

Table 122: Client Acceptance Considerations

Acceptance Area	What the Client / Supervisor May Confirm
Delivery Method	Whether repository-based handover is acceptable
Documentation Support	Whether the provided explanations and reports are sufficient
Deployment Readiness	Whether the project is shared in a realistic and reviewable form
Technical Accessibility	Whether the delivered materials are accessible and understandable
Review Suitability	Whether the handover supports final-stage feedback and refinement

6.7.5 Benefits of the Planned Delivery Approach

TeamTrack can have a number of practical advantages to our proposed delivery strategy. First, it assists us to present the project more clearly and professionally as opposed to giving out disconnected files and unfinished materials. Second, it makes the implementation as well as the

supporting evidence reviewed by the client or supervisor, thus it makes the project more easily evaluated [13], [14].

This is also effective as a team since we can have a better direction to complete final submission. Once the delivery method has been structured, then it is easy on our part to handle updates, prepare documents and also to respond to feedback in the subsequent stage of the project [14].

Our intended delivery strategy has the following key advantages:

- better and more professional project handover.
- greater compliance between documentation and implementation.
- more convenient to the client or supervisor.
- enhanced preparedness of end milestone delivery.
- greater confirmation of systematic project management.

Table 123: *Benefits of the Planned Delivery Approach*

Benefit	Contribution to the Project
More organized handover	Makes the project easier to review and understand
Better documentation support	Ensures that technical work is backed by clear explanations
Easier client review	Helps the reviewer assess the project more efficiently
Improved final-stage preparation	Supports smoother transition into final refinement
Stronger project presentation	Reflects a more professional and structured delivery approach

6.7.6 Product Delivery Plan Summary

Overall, our product delivery plan for TeamTrack is based on a structured handover approach that combines repository access, supporting documentation, design artefacts, and implementation evidence. We want the client or supervisor to receive the project in a form that is clear, accessible, and suitable for meaningful review [13], [14].

By planning the delivery in this way, we are not only focusing on completing the system, but also on presenting it properly. This helps us ensure that the project can be reviewed with less confusion, that feedback can be given more effectively, and that the final milestone can build on a well-prepared handover process [14].

Table 124: *Overall Product Delivery Plan Summary*

Subsection	Focus Area	Description
6.7.1	Purpose of the Product Delivery Plan	Explains why we need a structured and review-friendly handover process
6.7.2	Repository Access and Handover Method	Describes how we will deliver the project through repository-based sharing and supporting materials
6.7.3	Delivery Artefacts Provided to the Client	Summarizes the main project items that will be included in the handover
6.7.4	Client Acceptance of the Deployment Approach	Explains how we view client acceptance of the delivery and deployment method
6.7.5	Benefits of the Planned Delivery Approach	Highlights the practical advantages of the planned handover method
6.7.6	Product Delivery Plan Summary	Summarizes the overall role of delivery planning in Milestone 4

7. Technical Readiness

7.1 Development Environment Setup

7.1.1 Overall Setup Approach

The development setup follows the technology path already laid out in the SRS, TIS, and SAS. Tools are identified based on that, with the aim of keeping all system components operating together in a consistent, structured way starting from the development phase itself.

System Architecture Breakdown

- A web-based layered structure is what the system uses.
- Roles are defined separately.
- Frontend handles user interaction and interface rendering.
- Backend takes care of system logic, validation, and API handling.
- Structured academic and project data sits in the database.

- The AI part is treated as a supporting service rather than a core dependency, so it does not touch system data directly, communicating instead through the backend.

Keeping the separations clear was the idea behind the development environment for this. Less confusion during implementation, and each component can be built and tested individually while still fitting into the overall architecture

7.1.2 Tools and Technologies Setup

The environment of development is created based on a range of key technologies. These are essential to different components of the system and are supposed to collaborate and not to act as an isolated tool.

Regardless of that the main structure still keeps responsibilities clearly separated throughout the system, and also making communication between components straightforward.

- Frontend rendering is handled by Nuxt.js. It manages user interface rendering, routing, and general constructing page.
- For the backend FastAPI is what handles API endpoints, system logic, and incoming requests.
- PostgreSQL is used for structured storage. Users, projects, tasks, activity records, all of that lives there
- LLaMA via Ollama is included as the AI component to support features such as insights and summaries
- The version control is performed with the help of Bitbucket, which enables the team to work on the code together and have a common repository.

These technologies are interconnected using a typical request-response flow. The frontend makes requests to the backend API endpoints, the backend processes the requests and communicates with the database or AI service where necessary and the result is sent to the frontend where it is displayed.

7.1.3 Environment Consistency and Configuration

One major problem to consider in early development is the inability to stay consistent in individual setups, meaning you have to consider each team member's environment differences, tool versions, dependencies, configurations, etc. This should be minimized by having the project be executed using containers (e.g. Docker) where all key services will be executed in a controlled and reproducible environment independent of the machine used by the developer.

Other than this consideration, environment variables will be used to store configuration values like details on database connection, API keys, and service-level settings, avoiding anything like hardcoding sensitive information into source code and supports better transitions between the stages dev → testing → deployment, and is more secure.

Development Aims

- Reduce chance of set up problems among the team members.
- Establish system compatibility.
- Provide stable foundation for development to begin immediately.

7.2 Planned Spike Testing Strategy

7.2.1 Purpose of Spike Testing

The team will also conduct a series of small spike tests to check whether the technologies and system integrations chosen are feasible before getting into the full implementation. These tests are also not intended to provide full features, they are only to ensure that the most critical parts of the system are functional in practice as anticipated.

This allows minimizing any technical uncertainty early on, since spike testing helps identify the potential challenges with integration rather than finding out about them in later stages of the process which would be more hectic to resolve.

7.2.2 Key Areas to be Tested

The spike testing will focus on core interaction points within the system, especially where many components are interacting. These areas will be given priority because they form the core functional support of the system especially in areas of collaboration and tracking and system interactions

- Frontend-backend communication be tested to verify that API requests, responses and data flow is correct in normal conditions.
- Database connectivity will be tested to ensure that the backend is able to execute the create, read, update and delete operations on structured data.
- The authentication and role-based access control will be checked to ensure that users are only allowed to do what they should do in the system depending on the role.
- The contribution tracking feature such as time logging and task activity will be considered to ensure that the system is able to track and record user contributions in the most accurate way possible, as that is one of the main functionalities of the system
- AI integration will be evaluated to ensure that the backend can safely make structured inputs to the AI service and accept meaningful outputs without providing unrestricted data.

7.2.3 Expected Outcome

The result of these spike tests is to assure that the chosen stack and architecture is appropriate to the implementation of the system requirements and hoping that the tests will give the confidence in the areas like the system communication, data handling, access control, and AI integration. When the checks are done, the team will be in a better position to start with a structured

development and there will be less chances of entering into great technical blockers during implementation.

7.3 Codebase Skeleton

7.3.1 Concept and Purpose

Codebase skeleton is the first outline of the project which will be ready prior to the full-scale development. It doesn't include complete features or finished modules, only provides a structured starting point that reflects the architecture and technology stack of the system.

The idea of this skeleton is to make sure that the development is in an organised manner, and it is not the case where team members develop inconsistent or overlapping structures, they will all follow the same structure. This makes it possible to ensure clarity and consistency because the project should have a foundation structure, which will continue to add features.

7.3.2 Expected Project Structure (Assumed)

Based on the architecture and technologies selected, the project will have a modular design with the separation of components of the system. This structure is not finalised but is based on how the system has been designed in previous documents and considering typical implementation of similar architectures. Even though specific directory structure may continue to change during the implementation, it would probably be structured in the following manner.

Table 125: *Proposed Codebase Structure (Initial Setup)*

Area	Initial setup expected
Frontend	Frontend Nuxter.js application with routing, pages and reusable UI components.
Backend	FastAPI project structure with API routes, controllers, and service layers
Database	PostgreSQL connection configuration and initial schema or model definition.
Authentication	Simple user authentication and role-based access control configuration.
AI Module	Integration layer for sending requests to and receiving responses from the AI service
Configuration	Environment variables and system configuration set up.
Version Control	Bitbucket repository and structured project architecture and initial commits.

7.3.3 Readiness for Implementation

Even though the full functionality has not been implemented yet, the initial codebase structure ensures that the project is ready to move into development and enables the team members to start working on particular parts without having to re-define the system structure every time.

This also supports parallel development, where frontend, backend, and other components can be worked on simultaneously, but still being consistent and preserving the system, so in this way the codebase skeleton bridges planning and implementation, the project progress is past theoretical design and prepared for development.

8. Task Backlog and Development Estimates

This part details the entire task backlog during the development phase of TeamTrack, along with the explanation of how the effort and priority were established, a sprint distribution plan, and a dependency and risk management plan that will guide implementation throughout Capstone Computing Project 2. The backlog has been obtained directly based on the functional requirements written in the SRS, as well as based on the module breakdown written in the Software Architecture Specification.

Everything is estimated in terms of hours instead of abstract story points, therefore, allowing the progress to be checked against Toggl time logs on a per-sprint basis. The backlog is the plan of the team, as perceived during the handover, and will be considered as a living document during the development process, where new items will be added and estimates revised as the implementation process starts, and actual velocity statistics will be provided.

8.1 Backlog Snapshot

The backlog table shown below enumerates all the development activities that the team is aware of and has to accomplish before it can deliver the final product to the client. Each component is considered a task, and each item will have a unique identifier, a member of the responsible team, a priority rating, a range of 1 (highest) to 10 (lowest), an estimated effort in hours, and a target sprint. The sprint numbers are those of the three one-week development sprints of the second unit.

The dependency graph and the MVP-first development principle were used to give preference to things. Priority 1 is assigned to core authentication, which is essential to the database schema and project management tasks, since the rest of the system cannot be operated without it. Priority 2 tasks management, time tracking, and contribution analytics because they are the central value of

TeamTrack. The tasks associated with AI modules are Priority 3 since such work is the addition of additional features and not core ones, and a system can be put into a usable condition even when AI integration is not completed.

Task ID	Description	Component	Assigned To	Priority	Est. (hrs)	Sprint
T-01	Configure Firebase / OAuth integration with FastAPI backend	Backend	Hashir	1	8	Sprint 1
T-02	Implement user registration API endpoint (FR-1)	Backend	Hashir	1	4	Sprint 1
T-03	Implement secure login and logout endpoints (FR-2, FR-3)	Backend	Omar	1	5	Sprint 1
T-04	Build JWT session management and token refresh logic (FR-3)	Backend	Siraj	1	5	Sprint 1
T-05	Develop authentication screens in Nuxt.js (Login, Register, Landing)	Frontend	Maria	1	8	Sprint 1
T-06	Implement create and join project API	Backend	Omar	1	6	Sprint 1

	endpoints (FR-5, FR-6)					
T-07	Build role-based access control middleware (FR-8, NFR-4)	Backend	Siraj	2	6	Sprint 1
T-08	Develop Student Dashboard page in Nuxt.js	Frontend	Maria	1	8	Sprint 1
T-09	Implement project detail view and update endpoint (FR-7)	Backend	Omar	2	5	Sprint 2
T-10	Implement task CRUD REST API (FR-9 to FR-12)	Backend	Omar	1	8	Sprint 1
T-11	Build task list and task detail pages in Nuxt.js	Frontend	Maria	2	8	Sprint 2
T-12	Implement task status update workflow and notifications	Backend	Hashir	2	5	Sprint 2
T-13	Build task assignment logic and validation layer	Backend	Hashir	2	4	Sprint 2

T-14	Implement time log API endpoints (FR-13, FR-14)	Backend	Omar	1	6	Sprint 1
T-15	Build time logging UI component in Nuxt.js	Frontend	Maria	2	6	Sprint 2
T-16	Implement time aggregation queries per user and per project	Database	Zernish	2	5	Sprint 2
T-17	Design and implement analytics aggregation queries	Backend	Hashir	2	8	Sprint 2
T-18	Build contribution analytics dashboard UI	Frontend	Siraj	2	10	Sprint 2
T-19	Implement activity log and audit trail endpoints	Backend	Zernish	2	6	Sprint 2
T-20	Set up Ollama with LLaMA 3.1 model and local API service	AI	Siraj & Maria	3	10	Sprint 2
T-21	Implement backend integration endpoint for AI service	Backend	Siraj	3	6	Sprint 2

T-22	Build AI ChatBot UI component and conversation view	Frontend	Maria	3	8	Sprint 3
T-23	Implement AI insight generation logic (recommendations only)	AI	Siraj	3	6	Sprint 3
T-24	Create PostgreSQL schema and run initial migrations	Database	Omar	1	5	Sprint 1
T-25	Validate ERD against functional requirements; refine schema	Database	Omar	1	3	Sprint 1
T-26	Set up database connection pooling and environment configs	Database	Siraj	1	3	Sprint 1
T-27	Implement soft-delete strategy for audit-sensitive tables	Database	Zernish	2	3	Sprint 2
T-28	Confirm all team members have developmen	DevOps	All	1	4	Sprint 1

	t environmen t set up					
T-29	Configure Bitbucket CI/CD pipeline for automated builds	DevOps	Siraj	2	6	Sprint 2
T-30	Deploy Nuxt.js frontend to hosting platform	DevOps	Maria	2	4	Sprint 3
T-31	Deploy FastAPI backend server and configure environmen t vars	DevOps	Omar	2	4	Sprint 3
T-32	Write unit tests for core backend endpoints	Testing	Zernish	2	8	Sprint 2
T-33	Conduct integration testing across frontend and backend	Testing	Zernish	2	8	Sprint 3
T-34	Plan and execute User Acceptance Testing (UAT) with client	Testing	All	3	6	Sprint 3

Table 126: Full Task Backlog with Effort Estimates and Sprint Allocation

The sum of the time spent on all 34 tasks is approximated to be 205. This is allocated at a rate of about 14 hours per person per sprint in the five team members over three sprints of one week, which is very similar to the target of ten effective hours per person per week set out in the unit outline. Toggl will be utilized during the development process to ensure that actual time matches these estimates, and the sprint retrospective will be utilized to adjust estimates accordingly.

8.1.1 Effort Summary by Component

Component	No. of Tasks	Total Hours
Backend	14	82
Frontend	6	48
Database	5	19
AI	2	16
DevOps	4	18
Testing	3	22
Total	34	205

Table 127: Effort Summary by Component

8.2 How Effort and Priority Were Decided

Perhaps one of the most significant planning aspects that a team can make at the beginning of a project is estimating the effort and priority of development tasks. The under-estimation of tasks leads to deadline pressure and quality short-cuts, whereas the over-estimation leads to the unnecessarily conservative planning of sprints, resulting in unused capacity. The strategy that is presented in this section is the collective sense of the team concerning the planning discussions that have been conducted during Milestone 4 and is based on the functional requirements presented in the SRS and the Architectural decisions presented.

8.2.1 Priority Framework

The structured evaluation with three criteria: dependency, criticality, business value, and MVP scope was used to assign priority scores based on a scale of 1-10 (with 1 being the highest priority). The classification into three levels below is a summary of the logic used.

Priority 1 (Must-Have / MVP Core):

Activities with dependencies on other activities and which constitute the minimum viable product. These are database schema setup, authentication, project creation, and the core task management API. Sprint 1 is planned to take up these activities because, at the earliest possible time, the team will have a working skeleton on which to build.

Priority 2 (Should-Have / Core features):

Tasks that constitute the value proposition of TeamTrack but are not blocking the original skeleton. Among them, there are time tracking, contribution analytics, the entire task management UI, CI/CD configuration and automated testing. These activities are allotted on Sprints 1 and 2.

Priority 3 (Nice-to-Have / Improving Features):

Activities related to the AI module, UAT, and final deployment. Necessary as they are, they can be postponed to Sprint 3 without undermining the functional heart of the system.

8.2.2 Effort Estimation Approach

Effort estimates were produced using a combination of relative sizing and direct hour assignment. The team used existing knowledge of the chosen technologies (Vue.js / Nuxt.js, FastAPI, PostgreSQL, Firebase, and LLaMA via Ollama) to form baseline estimates for each task category, and then adjusted individual tasks up or down based on perceived complexity and the degree of novelty.

The following calibration anchors were agreed during planning to keep estimates consistent across team members.

- A straightforward REST API endpoint with known inputs and outputs: 4 hours.
- A database schema change or migration script: 3 hours.
- A full UI page with state management and route integration: 8 hours.
- A new integration between two system components (e.g., backend to AI service): 6 to 10 hours depending on API complexity.
- A test suite for a single module: 6 to 8 hours.

These anchors were not considered as hard rules, but a consistency check. In cases where a task was considered to be too far above or below the closest anchor, the team would discuss the cause of the difference and either accept it or divide the task into smaller tasks. There is no estimate on a task in the backlog that is over 10 hours, and this is in line with the concept of ensuring that the backlog tasks are within a sprint window.

8.2.3 Estimation Confidence and Refinement Plan

The team recognizes that initial estimating is always approximate, especially when the task at hand is one in which the individual team members have little prior experience with the technology. The AI module tasks (T-20 to T-23) and the CI/CD pipeline (T-29) have the greatest level of uncertainty in their estimation and might be re-evaluated following spike testing during Sprint 1. These are the practices that should be adopted to enhance the accuracy of estimation throughout the development period.

- Sprint retrospectives. Sprints will consist of a short comparison of estimated and actual hours on each completed task, with Toggl export data as the source of truth.
- When the actual hours are more than 50 times greater, the activities left within that component will be re-examined and revised prior to the commencement of the next sprint.
- Any task that cannot be done within one sprint will be divided into two sub-tasks whose estimates are independent, and then they will be pulled into the sprint backlog.
- Sprint 3 task estimates would be reviewed and reconfirmed formally at the completion of Sprint 2 and would include velocity data of the first two sprints.

8.3 Sprint Allocation and Delivery Timeline

This section gives a systematic description of all the activities that will be undertaken in all the three development sprints. The allocation has been made to give as much parallel work as possible between components and still obey the ordering of dependencies as found in Section 8.4. Every sprint includes a goal that is a meaningful state of the system at the end of this week that will provide both the team and the client with a clear review of the increment.

8.3.1 Sprint 1: Foundation and Core Skeleton

Sprint 1: This entails developing the skeleton of the system, which is supposed to be running. By the completion of this sprint, a user should be in a position to create an account, login and access a working dashboard. The database schema must be in operation and the endpoints of core project and task management API must be available. This is a list of the top priority tasks of each sprint.

- T-01: Firebase / OAuth integration with FastAPI
- T-02: User registration endpoint
- T-03: Secure login and logout endpoints
- T-04: JWT session management
- T-05: Authentication screens in Nuxt.js
- T-06: Create and join project API
- T-07: Role-based access control middleware
- T-08: Student Dashboard page
- T-10: Task CRUD REST API
- T-14: Time log API endpoints
- T-24: PostgreSQL schema and initial migrations
- T-25: ERD validation against SRS requirements

- T-26: Database connection pooling and environment config
- T-28: Development environment setup verified for all members

Sprint 1 total estimated effort: 79 hours across the team.

8.3.2 Sprint 2: Core Features and Analytics

Sprint 2 is built on Sprint 1 skeleton and the remaining user facing features are added. All core management of tasks and time tracking, contribution analytics features must be installed at the end of this sprint. This sprint also introduces automated testing and CI/CD configuration to ensure that the deployment pipeline is available by the end of the last sprint.

- T-09: Project detail view and update endpoint
- T-11: Task list and detail pages in Nuxt.js
- T-12: Task status update workflow and notifications
- T-13: Task assignment logic and validation layer
- T-15: Time logging UI component
- T-16: Time aggregation queries per user and project
- T-17: Analytics aggregation queries
- T-18: Contribution analytics dashboard UI
- T-19: Activity log and audit trail endpoints
- T-20: Ollama with LLaMA 3.1 local setup
- T-21: Backend integration endpoint for AI service
- T-27: Soft-delete strategy implementation
- T-29: Bitbucket CI/CD pipeline configuration
- T-32: Unit tests for core backend endpoints

Sprint 2 total estimated effort: 90 hours across the team.

8.3.3 Sprint 3: AI, Deployment and Acceptance Testing

Sprint 3 completes the AI module, finalises hosting configuration, and conducts formal integration testing and UAT with the client. By the end of this sprint, the system should be fully deployable and client-accepted.

- T-22: AI ChatBot UI component
- T-23: AI insight generation logic
- T-30: Nuxt.js frontend deployment
- T-31: FastAPI backend deployment and environment variable configuration
- T-33: Integration testing across frontend and backend

- T-34: User Acceptance Testing with the client

Sprint 3 total estimated effort: 36 hours across the team.

8.4 Dependencies, Risk Control and Definition of "Done"

Task-level dependencies are important in a project that has five people, and the components are developed simultaneously. The unresolved dependency between the tasks of two or more team members may block a complete component, slowing integration and demeaning sprint objectives. In this section, the important dependency relationships among the backlog, risk controls that will be implemented by the team and the agreed definition of a task being complete are documented.

8.4.1 Task-Level Dependency Overview

The table below maps the most significant blocking dependencies in the backlog. A dependency means that the predecessor task must reach a stable, testable state before the successor task can begin. Tasks with no predecessors listed can start in parallel from the beginning of Sprint 1.

Task ID	Depends On	Task Description	Reason for Dependency
T-02	T-01	User registration endpoint	Requires Firebase/OAuth connection to be active before authentication flows can be built.
T-03	T-01	Secure login and logout endpoints	OAuth callbacks and token issuance require the Firebase integration to be in place.
T-04	T-03	JWT session management	Tokens can only be issued once login endpoints are functional.
T-05	T-02, T-03	Authentication screens in Nuxt.js	UI flows must connect to working registration and login endpoints.
T-07	T-04	Role-based access control middleware	Middleware reads JWT claims; requires token management to be complete.
T-08	T-05, T-07	Student Dashboard page	Dashboard must enforce role visibility, which depends on RBAC middleware.
T-09	T-06, T-07	Project detail view and update	Update permissions rely on role enforcement middleware.
T-10	T-24	Task CRUD REST API	Tasks are stored in the PostgreSQL schema; schema must exist before API can write data.

T-11	T-10	Task list and detail pages	Frontend pages consume the task endpoints; endpoints must be stable.
T-12	T-10	Task status update workflow	Status updates extend the core task CRUD logic.
T-14	T-24	Time log API endpoints	Time logs are persisted to the database; schema must be live.
T-15	T-14	Time logging UI component	UI sends time data to the time log endpoints; endpoints must be ready.
T-16	T-14	Time aggregation queries	Aggregations run against time log data; data must be ingested first.
T-17	T-16, T-19	Analytics aggregation queries	Analytics combine time log aggregations with activity log data.
T-18	T-17	Contribution analytics dashboard UI	Dashboard reads from analytics endpoints.
T-21	T-20	Backend AI integration endpoint	The FastAPI endpoint wraps the Ollama service; service must be running.
T-22	T-21	AI ChatBot UI component	ChatBot UI calls the backend AI endpoint; endpoint must be live.
T-23	T-21	AI insight generation logic	Insight logic runs inside the AI service integration layer.
T-29	T-26	CI/CD pipeline	Pipeline requires database and backend environment configs to be finalised.
T-30	T-29	Frontend deployment	Deployment pipeline must be configured before automated builds can push to hosting.
T-31	T-29	Backend deployment	Same pipeline dependency as frontend deployment.
T-33	T-30, T-31	Integration testing	Integration tests require both frontend and backend to be deployed to a shared environment.
T-34	T-33	UAT with client	Client acceptance testing takes place after integration testing confirms system stability.

Table 128: Task-Level Dependency Matrix

8.4.2 Risk Controls

Backlog planning identified four risks that are likely to interfere with the sprint schedule. The following controls outline the mitigations that the team would use to ensure that each risk does not materialise into a delay.

Risk: The complexity of the AI modules is underestimated (T-20 to T-23).

Control: On the first and second days of Sprint 1, a special spike activity will be performed to ensure that Ollama is stable in the development environment and that the LLaMA 3.1 model can provide responses in a reasonable time frame. In case the spike shows that there are major problems, AI work will be rescheduled to Sprint 3 and swapped out with fewer risky work in Sprint 2.

Risk: Authentication halting all other parts in case Sprint 1 is late.

Control: The first tasks of Siraj are T-01 to T-04 which are given the highest priority. Day three of Sprint 1 will provide the latest delivery date of a functioning login endpoint so that frontend and other backend work can continue concurrently with the use of stub authentication tokens where necessary.

Risk: Mid-sprint feature additions due to scope creep.

Control: The backlog is considered the sole source of information of what is going to be constructed. Any new idea that is brought up during a sprint will be added to the backlog with a priority score and may only be included in the next sprint. No effort is added to the active sprint backlog with the exception of the explicit approval of all five team members, once the sprint has begun.

Risk: Individual work load bottleneck in case one of the team members is not available.

Control: All of the tasks have a main owner yet any task is a part of a module, known to at least one other member of the team. Daily stand-up meetings will review the Trello board and any task that is blocked will be reassigned within 24 hours of being identified as blocked.

8.4.3 Definition of "Done"

A standard definition of done is a necessity within an Agile team as it helps avoid situations when tasks are declared complete too soon, which leads to the emergence of technical debt that leaves behind integration failures. All these conditions are to be met before one task in the backlog is transferred to the Done column of the Trello board.

- The attribute or component elaborated in the task is operational in the shared development environment.

- The entire code concerning the task has been stored in the Bitbucket repository with a meaningful commit message mentioning the task ID (e.g., [T-10] Implement task CRUD endpoints).
- The implementation has been pulled through at least one team member, not the primary owner, either by a pull request review or walking through with the team at a stand-up meeting.
- Any pertinent unit tests have been developed and are passing.
- The Toggl time tracker of the task is stopped, and the time is checked individually.
- Where a task produces an artefact that is visible to the client (such as a UI page, an API endpoint, or a report), the artefact has been briefly demonstrated to the team and confirmed as matching the wireframes or API contract established in the design documentation.
- If the task affects the database schema, the migration script has been run successfully in the development environment and the change has been verified against the ERD.

Tasks that satisfy all of the above criteria but which have known minor defects or incomplete edge-case handling may be moved to Done with a linked follow-up task recorded in the backlog. This prevents a single minor issue from blocking the sprint report while still ensuring that the issue is tracked and addressed.

9. Client Communication and Acceptance

9.1 UI Feedback and Sign-Off

The UI Feedback and Sign-Off was carried out in order to make sure that the TeamTrack user interface design addressed the functional requirements, the usability requirements, as well as system workflow which had been defined earlier in project paperwork. This process was aimed at confirming that the interface of the system meets the specifications of the user activities needed and that the interface was well-travelled in terms of navigation and layout and that the interface met the specifications outlined in the SRS and TIS documentation and SAS documentation. The validation process and the interface design were done prior to implementation to minimize the usability problems and to be sure that the system design is embraced prior to the development proceeding.

TeamTrack was initially built with low-fidelity wireframes in Balsamiq as its user interface. These wireframes were the primary system displays, navigation, as well as the user processes. The wireframes were grounded on the system modules and user flows presented in the system architecture and requirements documentation, such as authentication, project workspace management, task management, time tracking, communications, analytics dashboard, and AI-

assisted features. The UI design was based on the regular structure of layout, navigation norm, and visual hierarchy to provide usability and readability of all screens. These design objectives and criteria are also stipulated in the architecture documentation where interface consistency, readability and usability have been stipulated as design objectives.

9.1.1 UI Feedback Process

The feedback process on the UI was conducted by design walkthroughs in the form of interface reviews with the Balsamiq wireframes. The walkthroughs were done to model the user interfaces of the system and ensure that the interface undergoes all the necessary user tasks and workflow functions. The main screens that the users explored during the walkthrough sessions included the login page, dashboard, project workspace, task board, time tracking page, communication interface, analytics dashboard, and AI assistant interface. The flow of navigation between the screens was also checked so that the users could navigate the system in a rational and effective way.

The feedback mechanism was aimed at the assessment of whether or not the users were able to accomplish the main tasks of the system, including registering and logging in, creating or joining a project workspace, creating and updating tasks, logging time and activities, viewing contribution analytics, and communicating with team members. These activities are related to system use cases and functional modules identified in the Technology Investigation Summary and Software Architecture documents.

The reviewers used the interface during the UI walkthroughs to compare it against usability, ease of navigation, screen layouts, consistency of workflow and the general user experience. The walkthrough method enabled the team to locate the areas that required enhancement in navigation, supplementary guidance or feedback messages, and simplifying or re-laying out the screen arrangements.

9.1.2 UI Feedback Findings and Improvements

The opinion gathered during the UI walkthrough process led to a number of changes in the interface design. The interface layout and workflow were in general viewed as straightforward and in accordance with the system functionality, however, the navigation clarity, consistency of the layout of the screen and workflow guidance were improved.

The primary enhancements were: the better navigation between the project workspace and task management screens, the better use of the dashboard layout in order to focus the important project information, the better display of the labels to the time tracking and activity logging features, and the better the consistency of the buttons and the navigation menus among the screens. Other feedbacks also proposed making user guidance more user friendly by including more clear page

titles, messages after doing actions (e.g. creating a task, time logging, etc.), and making the analytics dashboard and AI assistant features more visible in the navigation structure.

All these changes are reflected in the new design of the UI in such a way that the interface can support the effective implementation of tasks, the logical flow of navigation, and the active similarity of the user experiences across all modules of the system. The new interface design is more in line with the workflow of the system that was identified in the architecture diagram and system flow diagram.

9.1.3 UI Validation and Acceptance

Upon implementation of the feedback improvements of the revised wireframes, the interface design was re-examined to confirm the system interface addresses all the functional requirements and user workflow. The validation process also made sure that the entire system modules as outlined in the requirement documentation were reflected through the interface and that users were able to accomplish all significant tasks through the interface design.

The validation ensured that the interface is user-authenticated and allows managing an account, project and team management, task management, time tracking and logging activities, team communication, contribution analytics, dashboard, and features that are the work of artificial intelligence. These modules are directly aligned with the system functions which are outlined in the requirements documentation.

It was also confirmed in the validation that the navigation structure is logical, screen layouts are consistent, and the interface is in support of the overall system workflow as was defined in the system architecture and user flow diagrams.

9.1.4 UI Sign-Off

After the UI validation and optimisation, the UI interface design was accepted to be implemented. UI sign-off shows that the system interface design is compliant with functional requirements, usability requirements and workflow requirements stated in project documentation. This sign off indicates that the interface design is reasonable and that the project may go through to the development and implementation stage.

The sign-off procedure and feedback on the UI proposal will assure that the system interface was reviewed, validated and approved prior to its implementation. This minimizes the possibility of usability problems during the later development stages and makes sure that the system design would match with the system architecture and user requirements.

9.2 Product Delivery Plan

Product Delivery Plan will establish the preparation, delivery and acceptance of the TeamTrack system at the end of the project development process. This plan will ensure that the system is presented in an organized and professional way such as documentation, system deployment evidence, testing and user guidance materials. The delivery plan involves the final system satisfying the project objectives and the functional requirements that have been established in the previous project documentation and that the client or the supervisor is able to review, test and accept the system.

TeamTrack system is an artificial intelligence-based team project management system with time tracking that is created to support academic group projects. The culmination of the system will be the system comprising of the full software system, the supporting documentation, evidence of testing, and artefacts of the user interface. The process of deliveries is based on the development and implementation phases outlined in the project process and schedule. The project is based on the development strategy of Agile whereby the development is done in iterative steps, testing, refinement and end system delivery.

9.2.1 Deliverables Included in Product Delivery

The final product delivery would be in the form of a number of components to make sure that the system is reviewable, understandable, and usable. The final system application, project documentation, architecture documentation and the testing and validation evidence are all deliverables.

The products that are to be delivered in the product are:

- The TeamTrack web application system.
- System documentation (SRS, TIS, SAS + CID)
- EDR documentation and database design.
- System architecture drawing and design documentation.
- UI wireframes and prototype designs (Balsamiq)
- Validation and testing documentation.
- User manual and instructions on how to use it.
- Documentation Environment setup and Deployment documentation.
- History of version control and source code.

Such deliverables are consistent with project documentation and deliverables outlined in Technology Investigation Summary and architecture documentation, which establish system modules, architecture and implementation plan.

9.2.2 Delivery Process

The process of product delivery will be conducted in various phases to make sure that the entire system is tested and documented to be handed in finally. Delivery process starts when the implementation and testing phases are accomplished.

The process of delivery has the following stages. The system will then be finished and integrated so that all modules of the system like authentication, project management, task management, time tracking, communication, analytics dashboard, as well as AI-assisted features are all functioning as a unit. Second, the system will be tested and validated to verify that the system workflows and all other required functional requirements are operating properly. Third, documentation will be completed and it will entail system documentation, architecture documentation, testing documentation and user manual. Fourth, the system will be implemented on the final environment based on the deployment plan that was indicated earlier in the report. Lastly, system and documentation will be presented as the ultimate project deliverables.

Such an organized process of delivery means that the end system is comprehensive, fully tested, documented, and is prepared to be reviewed and accepted.

9.2.3 Acceptance Criteria

The system will be said to meet the functional and non- functional requirements outlined in the project documentation and all deliverables to be submitted.

The acceptance criteria entails:

- The system is user authentication and role-based access.
- Users have the ability to create and administer project workspaces.
- Users are able to create, allocate and change tasks.
- The users are able to record time and activities.
- It is done with regard to contributions and history of activity being tracked by the system.
- The contribution data is presented in the analytics dashboard.
- The AI assistant functions are functioning as desired.
- The user interface of the system is usable and consistent.
- Documentation is thorough and professionally done.
- Testing and validation evidence is given.
- The system is able to be deployed and operated within the mentioned environment.

These acceptance criteria are in line with the system functional requirement and system modules in the requirements and architecture documentation.

9.2.4 Final Delivery Summary

Product Delivery Plan makes sure that the TeamTrack system is presented in a well-organized and reasonable way including the system software, documentation, testing evidence, and deployment instructions. The delivery process is done to ensure that the system is delivered in a manner that satisfies the project objectives, functional requirements, and system architecture design and finally submitted and accepted.

The finished product delivery will consist of the full system of TeamTrack, the support documentation, the UI wireframes created with the help of Balsamiq, the testing and validation documentation and the instructions on how to deploy it. Delivery of the system to be followed will be after completing final tests and documentation and making the system ready to review, evaluate, and accept.

9.4 Technology and Development Readiness

9.4.1 System and Environment Readiness

So far TeamTrack's progress is past initial planning and is now ready to transition into implementation with a clear technical direction in mind. The main system components, frontend, backend, database, authentication, AI-assisted features, all of them are already defined. None of these are treated in isolation. They are meant to work together, which means the overall structure is already established.

Nuxt.js, FastAPI, PostgreSQL, and LLaMA via Ollama, that is the technology stack, this is a good foundation for stable development. Bitbucket is the version control platform, we will use it so the team can keep a central codebase and coordinate contributions in a structured way, and it's growth stays coherent, monitored, that is the idea.

As for environments, the setup we planned to be more standard because we want to reduce inconsistencies between team members, since variations in configurations, versions, etc can cause problems where the code would behave differently on different machines.

Issues like that can be avoided by having a common structure and configuration strategy and concentrate more on the actual implementation.

9.4.2 Development Approach and Implementation Readiness

Development ready is where the project stands. The system has already been split into functional areas. Analytics, time monitoring, task management, project and team management, authentication, AI-assisted features, these modules are already identified. Instead of treating everything as one massive build, this setup allows development to happen in a more organized way.

We are going on an iterative approach. Development uses smaller steps depending on the backlog that had been developed previously. The most critical ones will be given priority followed by improvements and additional features. This is the easiest way to monitor the progress and adjustments can also be made more easily in the implementation process.

The project demonstrates readiness for development through:

- Well-defined modules and system structure
- Established stack of technologies
- Collaboration setup and shared version control
- Planned approach to incremental implementation

9.5 Summary of Client Alignment

9.5.1 Alignment with Project Objectives

The state of TeamTrack is still in line with the initial issue defined and goals right now that were defined throughout. Issues including fragmented collaboration tools, limited transparency in group work, and a lack of visibility in individual contributions are still being addressed by the system.

These goals are well reflected in the suggested features, which include task monitoring, time logging, role-based access, and AI-assisted insights.

Essential elements get prioritized first, then enhancements and new features come after. Monitoring progress becomes easier this way, and making adjustments during implementation is also more manageable.

9.5.2 Consistency Across Deliverables

The SRS, TIS, and SAS are quite consistent with the present state and objectives. The documents do not contradict earlier decisions, but each of them is based on the previous one. The SAS develops the system architecture and design, the TIS defines the selected technologies and infrastructure, and the SRS defines the functional and non-functional requirements. Especially considering the occasional updates, every document continues to update to the changes and progress made throughout the planning stage of TeamTrack.

9.5.3 Communication and Alignment Confirmation

The project direction was developed through continuous communication and feedback. Meeting minutes are included in that, they provide some context on how the decision-making process was conducted and how the system changed over time. Important conversations and confirmations, those are captured there.

Based on this ongoing interaction, the recommended solution is in line with the expectation of the client. All of that was discussed in terms of functionality, scope, overall goal, so the course is evident and accepted. All set to proceed to implementation.

9.6 Meeting Minutes

There is a history of deliberations, resolutions, and courses of action that came out of meetings with the team and meetings with the clients that were kept during the Milestone 4 phase. These were meeting minutes that were used as a formal reference to coordinate the project, plan tasks and presentations, as well as documenting handover. This was facilitated by the maintenance of records of the meetings, which guaranteed transparency, accountability, and continuity throughout the milestone, recording important updates and actions agreed upon to be reviewed in the future.

Purpose of Meeting Documentation

The project was supported by meeting minutes by:

- Documenting significant planning and coordination discussions related to the handover document and presentation.
- Capturing decisions reached during planning, client consultation, and review exercises.

Following the assigned tasks and team duties.

- Traceability of client feedback and stakeholder input.
- Facilitating coordination in report writing, presentation preparation, and handover activities.

9.6.1 Meeting Log

Table 33: Meeting logs

Date	Meeting Title	Participants	Key Focus	Outcome
10/03/26	Initial ToC Discussion	Internal team	Discussion of initial Table of Contents, division of parts for the main document and presentation creation	Initial ToC and section allocation finalized
12/03/26	Client Consultation	Client + team representative	Questions on SRS and TIS updates, additional	Client feedback on SRS/TIS

			information requirements, and any changes needed in the final handover document	updates recorded
17/03/26	Team Follow-Up Meeting	Internal team	Progress check on work completed, review of initial presentation design, discussion on handover changes including TIS and SRS updates	Team aligned on progress and revisions
23/03/26	Presentation Rehearsal	Internal team	Rehearsals for the milestone presentation, preparation and review of scripts to pitch the project presentation	Presentation scripts finalized and team rehearsed
27/03/26	Final Review Meeting	Internal team	Finalizing changes to the handover document, discussion on any remaining updates or modifications required	Final document compilation and review completed

Meeting Evidence and Traceability

In order to have accountability and offer verifiable records of project coordination, documentation of all major meetings was kept and used throughout Milestone 4. These meeting reports facilitated section allocation tracing, client feedback, presentation preparation, progress monitoring, and final document review work.

Table 34: Meeting Minutes Summary

Date	Description	Reference
10/03/26	Internal meeting on initial ToC discussion, section distribution, and presentation part allocation	Link
12/03/26	Client consultation on SRS and TIS updates, additional information requirements, and changes for the final handover document	Link
17/03/26	Internal team follow-up on work progress, initial presentation design review, and handover changes including TIS and SRS	Link
23/03/26	Internal team rehearsals for the milestone presentation and preparation of scripts for the project pitch	Link
27/03/26	Internal meeting on finalizing changes and discussing any remaining updates required in the handover document	Link

Key Outcomes from Meetings

The meetings conducted during the Milestone 4 phase resulted in:

- Confirmation of section ownership and report responsibilities for the handover document.
- Incorporation of client feedback on SRS and TIS updates into the final document.
- Alignment of the team on presentation content, structure, and delivery scripts.
- Review and finalization of the handover document with all remaining changes addressed.
- Coordination of team responsibilities ensuring completeness and consistency across all milestone deliverables.

These meeting notes were important in keeping the milestone structured and coordinated. They ensured that client input was captured, team responsibilities were clearly followed up, and the final handover document remained aligned with the agreed technical direction of the project.

10. Appendices

Appendix A: Glossary

This glossary lays out the key technical and system-specific terms used in the TeamTrack handover document. It focuses on important concepts of system functionality, architecture, workflows mentioned in the body report, and other sections.

Term	Definition
Activity Log / Audit Trail	A chronological record of important system and user actions such as task updates, role changes, and project modifications, used for accountability and traceability.
Agile Evidence	Supporting process material such as sprint planning, task ownership, meeting logs, and collaboration tracking that shows the team followed an iterative development approach.
Bitbucket	The version control and repository platform used to manage TeamTrack source code, branches, integration, and collaboration history.

CI/CD (Continuous Integration / Continuous Delivery)	A structured workflow used to integrate, validate, and prepare project updates for review, testing, and handover in a controlled manner.
Contribution Analytics Dashboard	A dashboard that presents user contribution data such as time spent, task progress, and participation patterns to support transparency and accountability.
CRUD (Create, Read, Update, Delete)	The four basic data operations used in the system for managing records such as tasks, files, and project settings.
Docker	A containerization tool used to package and run system components such as the backend, AI module, and services in a consistent environment.
ERD (Entity Relationship Diagram)	A database design diagram that shows the main data entities in the system and the relationships between them.
FastAPI	The Python-based backend framework selected to implement TeamTrack's API services, validation, and business logic.
Firebase Authentication	The authentication service considered for handling secure login, identity verification, and token-based access to the platform.
Kanban Board	A task management view that organizes tasks into workflow stages such as Backlog, In Progress, Review, and Done.
LLaMA 3.1	The language model selected to support AI-assisted recommendations, summaries, and project insights in TeamTrack.
MVP (Minimum Viable Product)	The stable first implementation of the system focused on delivering the essential core features before optional enhancements are added.
Nuxt.js	The frontend framework selected to build TeamTrack's user interface, page routing, dashboards, and reusable UI structure.

OAuth	A token-based authentication and identity flow used to support secure access and session handling between users and the system.
Ollama	The runtime tool used to run and manage the LLaMA model for local AI testing and integration.
PostgreSQL	The relational database selected to store structured system data such as users, projects, tasks, messages, and logs.
Prototype Evidence	Early practical demonstrations of important or uncertain features, used to show how key parts of the system are expected to work.
RBAC (Role-Based Access Control)	The access control model that restricts system actions and visibility based on user role, such as team member, team leader, instructor, or administrator.
SAS (Software Architecture Specification)	The project document that defines TeamTrack's architecture, modules, data flow, interface structure, and design direction.
SRS (Software Requirements Specification)	The project document that defines TeamTrack's functional requirements, non-functional requirements, user roles, interfaces, and constraints.
TIS (Technology Infrastructure Specification / Technology Investigation Summary)	The project document that explains the chosen technology stack, implementation direction, security approach, and technical justification.
Time Tracking / Time Logging	The system feature that records time spent on tasks and activities to support effort tracking and contribution analysis.
Wireframes	Early interface design layouts used to show screen structure, navigation, and user flow before full implementation.

Appendix B: Updated Links

Document	Access Link	Purpose
Project Presentation	Open Document	The capstone presentation delivered by the TeamTrack team, covering platform architecture, design decisions, key features, competitor analysis, and implementation direction. Intended for academic review and demonstration of project scope.
TIS Report	Open Document	Documents the selected technology stack for TeamTrack, including justification for Nuxt.js, FastAPI, PostgreSQL, LLaMA 3.1 via Ollama, and the Curtin University physical hosting environment. Covers security approach, integration strategy, and technical rationale.
SRS Report	Open Document	Defines the complete set of functional and non-functional requirements for TeamTrack, including user roles, system constraints, use cases, and interface expectations. Serves as the primary contract between the team and the client regarding what the system must deliver.
SAS Report	Open Document	Describes the structural and design decisions underpinning TeamTrack, including module decomposition, data flow,

		interface structure, API design principles, and the rationale behind key architectural choices.
--	--	---

Appendix C: References

- [1] FastAPI, “Deployment and Deployment Concepts,” *FastAPI Documentation*. [Online]. Available: <https://fastapi.tiangolo.com/>.
- [2] A. Wiggins, “Store config in the environment,” *The Twelve-Factor App*. [Online]. Available: <https://12factor.net/>.
- [3] FastAPI, “Settings and Environment Variables,” *FastAPI Documentation*. [Online]. Available: <https://fastapi.tiangolo.com/>.
- [4] PostgreSQL Global Development Group, “PostgreSQL 18 Documentation: Creating a Database,” *PostgreSQL Documentation*. [Online]. Available: <https://www.postgresql.org/docs/18/>.
- [5] PostgreSQL Global Development Group, “PostgreSQL 18 Documentation: Schemas,” *PostgreSQL Documentation*. [Online]. Available: <https://www.postgresql.org/docs/18/>.
- [6] PostgreSQL Global Development Group, “PostgreSQL 18 Documentation: SQL Dump,” *PostgreSQL Documentation*. [Online]. Available: <https://www.postgresql.org/docs/18/>.
- [7] PostgreSQL Global Development Group, “PostgreSQL 18 Documentation: Populating a Database,” *PostgreSQL Documentation*. [Online]. Available: <https://www.postgresql.org/docs/18/>.
- [8] Ollama, “API Introduction,” *Ollama Documentation*. [Online]. Available: <https://ollama.com/docs/>.
- [9] Meta, “Llama 3.1 Model Cards and Prompt Formats,” *Llama Documentation*. [Online]. Available: <https://meta.com/docs/>.
- [10] Ollama, “Generate a Chat Message,” *Ollama Documentation*. [Online]. Available: <https://ollama.com/docs/>.
- [11] Atlassian Support, “Get started with Bitbucket Pipelines,” *Atlassian*. [Online]. Available: <https://support.atlassian.com/bitbucket-cloud/docs/get-started-with-bitbucket-pipelines/>.
- [12] Atlassian Support, “Workflow for Git feature branching,” *Atlassian*. [Online]. Available: <https://support.atlassian.com/bitbucket-cloud/docs/workflow-for-git-feature-branching/>.
- [13] GitHub Docs, “Configuring and managing deployments,” *GitHub Docs*. [Online]. Available: <https://docs.github.com/en/actions/how-tos/deploy/configure-and-manage-deployments>.

[14] Microsoft Learn, “Create a Release Pipeline,” *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/training/modules/create-release-pipeline-devops/>.

[15] IEEE, “IEEE Recommended Practice for Software Requirements Specifications,” IEEE Std 830-1998, 1998.

[16] B. Oakley, R. M. Felder, R. Brent, and I. Elhajj, "Turning student groups into effective teams," *Journal of Student-Centered Learning*, vol. 2, no. 1, pp. 9–34, 2004.

Appendix D: Contribution Matrix

TASKS		TEAM MEMBERS				
Main Part	Sub Part	Omar Waheed Khan	Muhammad Hashir	Maria Waheed	Zernish Shahid Nazir	Mohammad Siraj Hussain
Executive Summary	N/A					
Introduction	Purpose of This Document					
	What This Handover Document Contains					
	Out-of-Scope for This Document					
Project Overview	Problem Statement					
	Solution Overview					
	Objectives and Success Criteria					
Software Requirements Specification (SRS)	Functional Requirements					
	Non-Functional Requirements					
	System Constraints					
	User Classes and Roles					
	SRS Updates and Revisions					
	Technology Stack Selection					

Technology Infrastructure Specification (TIS)	System Architecture Design					
	Security and Performance Considerations					
	Technology Investigation and Justification					
Deployment and Hosting Overview	Deployment Strategy and Environment Setup					
	Frontend Hosting					
	Backend Hosting					
	Database Hosting					
	AI Module Deployment					
	CI/CD Workflow					
	Product Delivery Plan to Client					
Technical Readiness	Development Environment Setup					
	Planned Spike Testing Strategy					
	Codebase Skeleton					
Task Backlog and Estimates	Backlog Snapshot					
	How Effort and Priority Were Decided					
	Dependencies, Risk Control and Definition of "Done"					
Client Communication and Acceptance	UI Feedback and Sign-Off					
	Product Delivery Plan					
	Technology and Development Readiness					
	Summary of Client Alignment					
	Meeting Minutes					
Appendices	Appendix A: Glossary					

	Appendix B: Links					
	Appendix C: Presentation					
	Appendix D: References					
	Appendix E: Contribution Matrix					
	Appendix F: Toggl Reports					
Formatting	N/A					

Appendix E: User Manual

1. Purpose of the User Manual

This is a rough guideline of how to navigate and use the TeamTrack system documentation and interface. It will be targeted at ensuring that the reader/evaluator can easily access important sections of the document, to get to know and understand the components of the system, and to find the relevant supporting materials that are included in the document.

The guide will be aimed at document navigation, structural features, and usability amenities, including the table of contents, figure references, and quick navigation provision.

2. Document Navigation Structure

The document has been designed to enable it to navigate easily and find components of the system, design artefacts, and implementation details in a short time.

The following navigation elements are included:

- Table of Contents (TOC)
- List of Tables
- List of Figures
- Section Reference Links
- “Back to Top” navigation button

These elements improve readability, accessibility, and document usability for reviewers and stakeholders.

3. Table of Contents

At the onset of the document, the Table of Contents is given, which contains all the major sections, subsections, and appendices. All the entries are connected to their respective sections so that one can navigate fast.

Any headings in the TOC can be clicked by the users to directly get to the required part of the report.

4. List of Tables

The List of Tables presents all tabular content used throughout the document like:

- Stakeholder tables
- Requirements tables
- Technology comparison tables
- Testing matrices

5. List of Figures

The List of Figures includes all visual artefacts and diagrams used within the document, such as:

- System architecture diagrams
- User flow diagrams
- Database schema diagrams
- UI wireframes and logo artefacts

Each figure is numbered and referenced for easy navigation and traceability.

6. “Back to Top” Navigation Feature

To enhance usability and document accessibility, a “Back to Top” navigation feature is included at the bottom-left of each page.

This feature allows readers to:

- Instantly return to the beginning of the document
- Access the Table of Contents without manual scrolling

- Improve navigation efficiency during review

This is particularly useful for long technical documents where frequent movement between sections is required.

7. How to Use This Document Efficiently

Readers are encouraged to follow these steps when using the document:

5. Begin with the Table of Contents to determine the needed part.
6. Search visual or structured content using the List of Tables or List of Figures.
7. See internal reference explanations on related explanations.
8. The button known as the Back to Top will help you get back to the top of the document easily.

The method can guarantee easy navigation and understanding of the system documentation.

8. Intended Users of the Guide

This quick guide is intended for:

- Academic evaluators
- Project supervisors
- Team members
- Stakeholders reviewing the system documentation

It supports efficient access to project information and enhances overall document usability.

Appendix F: Toggl Reports

Table : Toggl Report Table

Member Name	Toggl Report (Link)
Muhammad Hashir	Link
Omar Waheed Khan	Link
Maria Waheed	Link
Zernish Shahid Nazir	Link
Mohammed Siraj Hussain	Link

Work Distribution Summary for Milestone 4

In Milestone 4, the Handover Report was separated based on section and subsection ownership so that each member of the team could own those sections which were most aligned to the work done in their previous milestones and the knowledge of the subject. The Milestone 4 Table of Contents will have ownership of the Introduction, Project Overview, Software Requirements Specification, Technology Infrastructure Specification, Deployment and Hosting Overview, Technical Readiness, Task Backlog and Estimates, Client Communication and Acceptance, and Appendices. These major sections and subsections are reflected in the final handover document in the completed report.

Section-Wise Contribution Tables

1. Executive Summary

Section / Subsection	Primary Contributor	Contribution Details
Executive Summary	Muhammad Hashir	Drafted the overall summary of the Milestone 4 Handover Report, including the report purpose, project readiness positioning, and transition from design to implementation planning.

2. Introduction

Section / Subsection	Primary Contributor	Contribution Details
2. Introduction	Mohammed Siraj Hussain	Drafted the introductory framing of the handover report.
2.1 Purpose of This Document	Mohammed Siraj Hussain	Wrote the purpose of the handover, including review, continuity, and readiness intent.
2.2 What This Handover Document Contains	Mohammed Siraj Hussain	Documented the artefacts included in the handover and their review value.
2.3 Out-of-Scope for This Document	Mohammed Siraj Hussain	Defined the boundaries of the handover and clarified what was not intended to be covered in full.

3. Project Overview

Section / Subsection	Primary Contributor	Contribution Details
3. Project Overview	Omar Waheed Khan, Maria Waheed	Shared ownership of the overall project overview section.
3.1 Problem Statement	Maria Waheed	Drafted the core problem framing for TeamTrack.
3.1.1 Fragmented Academic Collaboration Tools	Maria Waheed	Wrote the discussion of fragmented tool usage in academic group work.
3.1.2 Lack of Individual Contribution Tracking	Maria Waheed	Wrote the discussion of unequal contribution visibility and fairness issues.
3.1.3 Limited Transparency and Accountability	Maria Waheed	Wrote the discussion of accountability and project visibility limitations.
3.1.4 Absence of AI-Assisted Insights	Maria Waheed	Wrote the discussion of the lack of intelligent support in existing workflows.
3.2 Solution Overview	Maria Waheed	Drafted the proposed TeamTrack solution framing.
3.2.1 TeamTrack Platform Vision	Maria Waheed	Wrote the platform vision statement.
3.2.2 Key Features and Capabilities	Maria Waheed	Wrote the feature and capability overview of the system.
3.2.3 High-Level Architecture	Maria Waheed	Wrote the high-level architectural overview within the solution discussion.
3.3 Objectives and Success Criteria	Omar Waheed Khan	Drafted the goals and evaluation criteria of the proposed system.
3.3.1 Primary Objectives	Omar Waheed Khan	Wrote the high-level objectives of TeamTrack.
3.3.2 Functional Objectives	Omar Waheed Khan	Wrote the core functional objectives.
3.3.3 Non-Functional Objectives	Omar Waheed Khan	Wrote the non-functional quality goals.
3.3.4 Success Criteria and KPIs	Omar Waheed Khan	Wrote the success criteria and key performance indicators.

4. Software Requirements Specification (Updated SRS)

Section / Subsection	Primary Contributor	Contribution Details
4. Software Requirements Specification (SRS)	Zernish Shahid Nazir	Led the updated SRS content for Milestone 4.
4.1 Functional Requirements	Zernish Shahid Nazir	Wrote and consolidated updated functional requirements.
4.2 Non-Functional Requirements	Zernish Shahid Nazir	Wrote the updated non-functional requirements.
4.3 System Constraints	Zernish Shahid Nazir	Documented system, environment, and implementation constraints.
4.4 User Classes and Roles	Zernish Shahid Nazir	Wrote the user role definitions and access-level summary.
4.5 SRS Updates and Revisions	Zernish Shahid Nazir	Documented the revised and added requirements since previous milestones.

5. Technology Infrastructure Specification (TIS Updation)

Section / Subsection	Primary Contributor	Contribution Details
5. Technology Infrastructure Specification (TIS)	Mohammed Siraj Hussain	Led the updated technology infrastructure section.
5.1 Technology Stack Selection	Mohammed Siraj Hussain	Wrote the technology selection rationale and stack overview.
5.1.1 Frontend: Vue.js / Nuxt.js	Mohammed Siraj Hussain	Wrote the frontend technology choice and justification.
5.1.2 Backend: FastAPI	Mohammed Siraj Hussain	Wrote the backend selection and technical basis.

5.1.3 Database: PostgreSQL	Mohammed Siraj Hussain	Wrote the database selection justification.
5.1.4 AI Module: LLaMA via Ollama	Mohammed Siraj Hussain	Wrote the AI module choice and local deployment suitability.
5.1.5 Authentication: Firebase / OAuth	Mohammed Siraj Hussain	Wrote the authentication technology explanation.
5.2 System Architecture Design	Mohammed Siraj Hussain	Summarised architecture artefacts reused in the handover.
5.2.1 System Architecture Diagram	Mohammed Siraj Hussain	Documented the reused high-level system architecture reference.
5.2.2 Database Design (ERD / Schema)	Mohammed Siraj Hussain	Documented the reused data model reference.
5.2.3 User Flow Diagram	Mohammed Siraj Hussain	Documented the user interaction flow reference.
5.2.4 System Flow Diagram	Mohammed Siraj Hussain	Documented the system request/data flow reference.
5.3 Security and Performance Considerations	Mohammed Siraj Hussain	Wrote the security, performance, scalability, and maintainability discussion.
5.4 Technology Investigation and Justification	Mohammed Siraj Hussain	Wrote the final justification for the selected technology stack.

6. Deployment and Hosting Overview

Section / Subsection	Primary Contributor	Contribution Details
6. Deployment and Hosting Overview	Maria Waheed, Omar Waheed Khan, Mohammed Siraj Hussain	Shared responsibility across deployment, hosting, and handover planning.

6.1 Deployment Strategy and Environment Setup	Maria Waheed	Wrote the deployment strategy overview and environment setup discussion.
6.2 Frontend Hosting	Maria Waheed	Wrote the frontend hosting section.
6.2.1 Nuxt.js Build and Deployment Process	Maria Waheed	Wrote the frontend build and deployment process.
6.2.2 Hosting Platform Selection and Justification	Maria Waheed	Wrote the frontend hosting platform discussion and justification.
6.3 Backend Hosting	Omar Waheed Khan	Wrote the backend hosting planning section.
6.3.1 FastAPI Server Deployment	Omar Waheed Khan	Wrote the FastAPI server deployment explanation.
6.3.2 Environment Variables and Configuration Management	Omar Waheed Khan	Wrote the configuration and environment setup planning.
6.4 Database Hosting	Omar Waheed Khan	Wrote the database hosting section.
6.4.1 PostgreSQL Deployment and Access	Omar Waheed Khan	Wrote the PostgreSQL hosting and access explanation.
6.4.2 Data Migration Plan	Omar Waheed Khan	Wrote the migration planning and data portability discussion.
6.5 AI Module Deployment	Omar Waheed Khan	Wrote the AI deployment planning section.
6.5.1 LLaMA via Ollama Local Deployment	Omar Waheed Khan	Wrote the local AI deployment explanation.
6.5.2 AI Service Integration with Backend	Omar Waheed Khan	Wrote the AI-backend integration plan.
6.6 CI/CD Workflow	Mohammed Siraj Hussain	Wrote the CI/CD section.
6.6.1 Bitbucket-Based Deployment Pipeline	Mohammed Siraj Hussain	Wrote the repository and pipeline workflow explanation.

6.6.2 Build Process and Automation	Mohammed Siraj Hussain	Wrote the build and automation planning.
6.7 Product Delivery Plan to Client	Mohammed Siraj Hussain	Wrote the delivery and handover planning section.
6.7.1 Repository Access and Handover Method	Mohammed Siraj Hussain	Wrote the repository access and handover explanation.
6.7.2 Client Acceptance of Deployment Approach	Mohammed Siraj Hussain	Wrote the client-facing acceptance explanation.

7. Technical Readiness

Section / Subsection	Primary Contributor	Contribution Details
7. Technical Readiness	Zernish Shahid Nazir	Led the technical readiness section.
7.1 Development Environment Setup	Zernish Shahid Nazir	Wrote the environment setup discussion.
7.2 Planned Spike Testing Strategy	Zernish Shahid Nazir	Wrote the spike testing strategy and validation readiness plan.
7.3 Codebase Skeleton	Zernish Shahid Nazir	Wrote the initial codebase structure and implementation readiness explanation.

8. Task Backlog and Estimates

Section / Subsection	Primary Contributor	Contribution Details
8. Task Backlog and Estimates	Muhammad Hashir	Led the planning and backlog section.
8.1 Backlog Snapshot	Muhammad Hashir	Wrote the task backlog overview and planning snapshot.
8.2 How Effort and Priority Were Decided	Muhammad Hashir	Wrote the prioritisation and effort estimation rationale.

8.3 Dependencies, Risk Control and Definition of “Done”	Muhammad Hashir	Wrote the dependencies and completion criteria discussion.
---	-----------------	--

9. Client Communication and Acceptance

Section / Subsection	Primary Contributor	Contribution Details
9. Client Communication and Acceptance	Maria Waheed, Zernish Shahid Nazir, Muhammad Hashir	Shared responsibility across client feedback, readiness confirmation, and meeting evidence.
9.2 UI Feedback and Sign-Off	Maria Waheed	Wrote the UI review, validation, and sign-off section.
9.3 Product Delivery Plan	Maria Waheed	Wrote the client-facing product delivery explanation.
9.4 Technology and Development Readiness	Zernish Shahid Nazir	Wrote the readiness explanation for technology and development preparation.
9.5 Summary of Client Alignment	Zernish Shahid Nazir	Wrote the final alignment summary between team work and client expectations.
9.6 Meeting Minutes	Muhammad Hashir	Documented the meeting records and supporting evidence.

10. Appendices

Appendix	Primary Contributor	Contribution Details
Appendix A: Glossary	Zernish Shahid Nazir	Prepared glossary and terminology support material.
Appendix B: Links	Muhammad Hashir	Prepared supporting links and reference access material.
Appendix C: Presentation	Hashir, Omar Waheed Khan	Shared responsibility for presentation-related appendix material.
Appendix D: References	Omar Waheed Khan	Prepared the references appendix.

Appendix E: Contribution Matrix	Omar Waheed Khan	Prepared the contribution matrix appendix.
Appendix F: Toggl Reports	Muhammad Hashir	Prepared the Toggl tracking appendix.

Overall Contribution Percentage Summary

Team Member	Primary Milestone 4 Areas	Approx. Ownership
Muhammad Hashir	Executive Summary, Task Backlog and Estimates, Meeting Minutes, Links, Toggl Reports	18%–22%
Mohammed Siraj Hussain	Introduction, TIS, CI/CD Workflow, Product Delivery Plan to Client	20%–24%
Omar Waheed Khan	Objectives and Success Criteria, Backend Hosting, Database Hosting, AI Module Deployment, References, Contribution Matrix, Presentation support	22%–26%
Maria Waheed	Problem Statement, Solution Overview, Deployment Strategy, Frontend Hosting, UI Feedback and Sign-Off, Product Delivery Plan	20%–24%
Zernish Shahid Nazir	Updated SRS, Technical Readiness, Technology and Development Readiness, Client Alignment Summary, Glossary	18%–22%

Closing Statement

Though certain sections and subsections of Milestone 4 were the primary properties of each of the team members, the Handover Report was a truly collaborative group submission. In addition to the personal drafting tasks, the internal discussion, reviewing of sections, refining of content and finalizing of the report were all shared among all the members. Feedback and change requests identified at the client meetings were also included in the team and this necessitated amendments in various sections of the document in order to make the handover more consistent with client expectations, the direction of the project and the readiness to implement. Consequently, the ultimate Milestone 4 submission represents not only individual section ownership but also collaborative work to review, update and enhance the report before submission. This structure meets the need that the final report must clearly assign the responsibility on each written section and also recognise the shared effort on revision and finalization.